

Building AI Voice Studio

A Complete Python & wxPython Guide
From Zero to Desktop Application

This book teaches you every concept of Python — basic, intermediate, and advanced — through the lens of a real-world desktop application: **AI Voice Studio**, a text-to-speech studio built with wxPython, sherpa-onnx, and NumPy.

By the time you finish, you will be able to build, test, and package a complete Python desktop application on your own.

Second Edition • 2026 • Python 3.13 • wxPython 4.3

Based on AI Voice Studio (AIVS) open-source project • Version 2026.3.1

Includes Part X: the Voice Lab engine subsystem, per-engine Python environments, the CPU/GPU compute choice, the audio I/O fallback and the licensing model

Preface

Who This Book Is For

This book is designed for programmers who want to learn Python from the ground up and apply their knowledge to build a real, functional desktop application. Whether you are a complete beginner, an intermediate developer expanding your skills, or an experienced programmer new to Python, this book provides a structured path from fundamentals to advanced topics.

How This Book Is Organized

The book is divided into ten parts:

1. **Part I — Python Fundamentals** (Chapters 1–10): Variables, control flow, functions, file I/O, error handling, modules, and object-oriented basics.
2. **Part II — Intermediate Python** (Chapters 11–18): Advanced OOP, decorators, context managers, regular expressions, testing, and structured data.
3. **Part III — Advanced Python** (Chapters 19–26): Concurrency, multiprocessing, async I/O, type hints, metaclasses, design patterns, and introspection.
4. **Part IV — wxPython GUI Development** (Chapters 27–36): Building desktop interfaces, event handling, threading in GUI apps, accessibility, and theming.
5. **Part V — Dependencies Deep Dive** (Chapters 37–42): NumPy, Requests, sherpa-onnx, PDF/DOCX parsing, FFmpeg, and virtual environments.
6. **Part VI — Building AI Voice Studio** (Chapters 43–60): The complete project from architecture to every major subsystem.
7. **Part VII — Testing & Deployment** (Chapters 61–66): Unit testing, mocking, PyInstaller, Inno Setup, and CI/CD.
8. **Part VIII — Advanced Topics** (Chapters 67–72): Managed runtime, subprocess architecture, logging, performance, and security.
9. **Part IX — Second Edition Additions** (Chapters 73–75): Playlist generation, DAISY 3 book building, and reusable dialogs and progress windows.
10. **Part X — Voice Lab & the Current System** (Chapters 76–82): The voice-clone engine subsystem, one Python environment per engine, the CPU/GPU compute choice, audio decoding without FFmpeg, out-of-order recording, the first-launch terms dialog, and licensing.

i Info: About this edition. Parts I–IX teach Python, wxPython and every third-party library this application uses. **Part X** is written against the *current* code base and documents the subsystems that grew after the first edition: the Voice Lab clone engines, one virtualenv per engine, the compute band (CPU 80–95 %, full GPU), the soundfile audio fallback, out-of-order recording for long books, the terms dialog, and the licence inventory. Every code excerpt in Part X is taken from the shipped source and annotated line by line.

The Running Example: AI Voice Studio

Rather than teaching concepts in isolation, every chapter applies what it teaches to **AI Voice Studio** (AIVS) — a production-quality desktop application that converts text documents into spoken audio using neural TTS engines. The project demonstrates:

- wxPython GUI with accessibility support (NVDA, JAWS, Narrator)
- Multiple TTS engines (Piper, Kokoro, Matcha, Kitten, OmniVoice, SAPI5/Windows voices, plus the Voice Lab clone engines: Pocket TTS, Bark, F5-TTS)

- Document parsing (PDF, DOCX, HTML, Markdown, EPUB)
- Background threading for non-blocking synthesis
- Virtual environment management and addon architecture
- PyInstaller packaging and Inno Setup installers
- 370+ unit tests with comprehensive coverage

Prerequisites

- A computer running Windows (primary), macOS, or Linux
- Python 3.11 or later installed
- A text editor or IDE (VS Code, PyCharm, or similar)
- No prior Python experience required — we start from scratch

How to Use This Book

Each chapter follows a consistent structure:

Section	Purpose
Learning Objectives	What you will master in this chapter
Core Concepts	Thorough explanation with syntax and semantics
Code Examples	Working examples with output, drawn from the AIVS project
AIVS in Action	How the concept appears in the real project
Exercises	Practice problems to reinforce learning
Key Takeaways	Summary of the most important points

i Info: Interactive Learning: Every code example in this book can be typed into a Python REPL or saved to a `.py` file and executed. The AI Voice Studio project files are referenced by their actual paths in the repository.

Begin your journey: start with [Chapter 1: Introduction & Setup](#)

Introduction & Setup

Learning Objectives

- Understand what Python is and why it dominates desktop and AI development
 - Install Python 3.13 on Windows, macOS, or Linux
 - Set up a command-line terminal and run your first Python program
 - Explore the AI Voice Studio project structure
 - Create and activate a virtual environment
 - Install project dependencies
-

1.1 What Is Python?

Python is a high-level, interpreted, general-purpose programming language created by Guido van Rossum and first released in 1991. It emphasizes code readability with its use of significant indentation and English-like syntax. Today, Python is one of the most popular programming languages in the world, powering everything from web servers to artificial intelligence to desktop applications.

Why Python?

Feature	Benefit
Interpreted	No compilation step — run code immediately
Dynamically typed	No type declarations needed (but available via hints)
Extensive stdlib	Over 200 modules in the standard library
Massive ecosystem	400,000+ packages on PyPI
Cross-platform	Windows, macOS, Linux with one codebase
AI/ML dominance	NumPy, PyTorch, TensorFlow all use Python
Desktop GUIs	wxPython, PyQt, Tkinter, Kivy

Python in AI Voice Studio

AI Voice Studio is a desktop application that converts text documents into spoken audio using neural text-to-speech (TTS) engines. It is built entirely in Python with wxPython for the GUI. This makes it a perfect learning vehicle because it touches every major area of Python development:

- **Core Python:** File I/O, string processing, data structures, error handling
 - **OOP:** Classes, inheritance, polymorphism, encapsulation
 - **Concurrency:** Threading for background TTS synthesis
 - **GUI:** wxPython for native Windows interface
 - **AI/ML:** sherpa-onnx for neural speech synthesis
 - **Networking:** HTTP downloads of voice models
 - **Packaging:** PyInstaller and Inno Setup for distribution
-

1.2 Installing Python

System Requirements

- Operating System: Windows 10/11 (64-bit), macOS 12+, or Ubuntu 20.04+
- Python version: 3.11 or later (we use 3.13)
- RAM: 4 GB minimum, 8 GB recommended
- Disk: 2 GB free space (for TTS voice models)

Windows Installation

1. Visit python.org/downloads
2. Download the Windows installer (64-bit)
3. Run the installer. **Important:** Check the box that says "Add python.exe to PATH" before clicking Install
4. Verify the installation by opening a terminal and typing:

```
python --version  
# Output: Python 3.13.0
```

Explain Line by Line Code Description (Block 1)

1. Line 1: `python --version`

`python --version`

2. Line 2: `# Output: Python 3.13.0`

A comment: *Output: Python 3.13.0.*

⚠ **Warning:** Always add Python to PATH during installation. If you skip this, you will not be able to run `python` or `pip` from the terminal without specifying the full path.

macOS / Linux Installation

```
# macOS (using Homebrew)
brew install python@3.13

# Ubuntu/Debian
sudo apt update
sudo apt install python3.13 python3.13-venv python3.13-dev

# Verify
python3 --version
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# macOS (using Homebrew)`
A comment: *macOS (using Homebrew)*.
2. **Line 2:** `brew install python@3.13`
`brew install python@3.13`
3. **Line 3:** `# Ubuntu/Debian`
A comment: *Ubuntu/Debian*.
4. **Line 4:** `sudo apt update`
`sudo apt update`
5. **Line 5:** `sudo apt install python3.13 python3.13-venv python3.13-dev`
`sudo apt install python3.13 python3.13-venv python3.13-dev`
6. **Line 6:** `# Verify`
A comment: *Verify*.
7. **Line 7:** `python3 --version`
`python3 --version`

1.3 The Python REPL

The **REPL** (Read-Eval-Print Loop) is Python's interactive shell. Open a terminal and type `python` to enter it:

```
$ python
Python 3.13.0 (main, Oct 7 2024)
Type "help", "copyright", "credits" or "license" for more information.
>>> # You are now in the Python REPL
>>> 2 + 2
4
>>> "Hello, " + "World!"
'Hello, World!'
>>> import sys
>>> sys.version
'3.13.0 (main, Oct 7 2024, 05:45:20) ...'
>>> exit() # or press Ctrl+Z (Windows) / Ctrl+D (macOS/Linux)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `$ python`

```
$ python
```

2. **Line 2:** `Python 3.13.0 (main, Oct 7 2024)`

```
Python 3.13.0 (main, Oct 7 2024)
```

3. **Line 3:** `Type "help", "copyright", "credits" or "license" for more information.`

```
Type "help", "copyright", "credits" or "license" for more information.
```

4. **Line 4:** `>>> # You are now in the Python REPL`

```
>>> # You are now in the Python REPL
```

5. **Line 5:** `>>> 2 + 2`

```
>>> 2 + 2
```

6. **Line 6:** `4`

```
4
```

7. **Line 7:** `>>> "Hello, " + "World!"`

```
>>> "Hello, " + "World!"
```

8. **Line 8:** `'Hello, World!'`

```
'Hello, World!'
```

9. **Line 9:** `>>> import sys`

```
>>> import sys
```

10. **Line 10:** `>>> sys.version`

```
>>> sys.version
```

11. **Line 11:** `'3.13.0 (main, Oct 7 2024, 05:45:20) ...'`

```
'3.13.0 (main, Oct 7 2024, 05:45:20) ...'
```

12. **Line 12:** `>>> exit() # or press Ctrl+Z (Windows) / Ctrl+D (macOS/Linux)`

```
>>> exit() # or press Ctrl+Z (Windows) / Ctrl+D (macOS/Linux)
```

The REPL is your sandbox for experimenting. You can type any Python expression and see the result immediately. We will use the REPL extensively throughout this book.

Running a Python Script

While the REPL is great for experiments, real programs live in files with a `.py` extension:

```
# hello.py
print("Hello, AI Voice Studio!")
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# hello.py`
A comment: *hello.py*.
2. **Line 2:** `print("Hello, AI Voice Studio!")`
Outputs text to the console. The primary way to display information during development and debugging.

```
$ python hello.py
Hello, AI Voice Studio!
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `$ python hello.py`
`$ python hello.py`
2. **Line 2:** `Hello, AI Voice Studio!`
`Hello, AI Voice Studio!`

1.4 The AI Voice Studio Project

Project Structure

Let's explore the project we will be building throughout this book. The AI Voice Studio repository has this structure:

```

ai_voice_studio/
├── main.py                # Application entry point
├── requirements.txt       # Core dependencies
├── LICENSE                # GPL-3.0 License
├── README.md             # Project documentation
├── PROJECT_SPEC.md       # Full feature specification
├── DEV_PLAN.md           # Development plan
├── ai_voice_studio/      # Main package
│   ├── __init__.py
│   ├── constants.py      # Application constants
│   ├── paths.py          # File path management
│   ├── settings.py       # User preferences (JSON)
│   ├── project.py        # Project persistence
│   ├── compute.py        # GPU/CPU detection
│   ├── util.py           # Utility functions
│   ├── heavy_logging.py  # Developer logging
│   ├── runtime.py        # Runtime management
│   ├── gui/              # wxPython GUI modules
│   │   ├── main_frame.py # Main window
│   │   ├── settings_dialog.py # Settings window
│   │   ├── model_panels.py # Voice download/management
│   │   ├── new_project_wizard.py # Project creation wizard
│   │   ├── recording_dialog.py # Recording window
│   │   ├── ally.py       # Accessibility helpers
│   │   ├── theme.py      # Light/Dark themes
│   │   └── events.py     # Custom wx events
│   ├── tts/              # TTS engine subsystem
│   │   ├── catalog.py    # Voice catalog (JSON data)
│   │   ├── engine.py     # TTS engine wrapper
│   │   ├── models.py     # Model store & file resolution
│   │   └── downloader.py # Voice model downloads
│   ├── audio/            # Audio output subsystem
│   │   ├── output.py     # WAV/MP3/FLAC output
│   │   └── ffmpeg.py     # FFmpeg management
│   ├── documents/        # Document parsing
│   │   ├── parsers.py    # PDF/DOCX/HTML parsers
│   │   ├── splitter.py   # Text splitting logic
│   │   └── daisy_builder.py # DAISY 2.02 book builder
│   ├── jobs/             # Background processing
│   │   └── synthesizer.py # Background TTS worker
│   ├── voicelab/         # Voice cloning (Pocket TTS, Bark, F5-TTS)
│   │   ├── engines.py    # Engine registry & built-in voices
│   │   └── worker.py     # Cloning subprocess (own env per engine)
│   ├── omnivoice/        # OmniVoice TTS engine
│   │   ├── __init__.py   # OmniVoiceEngine
│   │   └── worker.py     # OmniVoice subprocess
│   ├── addons/           # Addon system
│   │   └── __init__.py   # AddonManager
│   └── python_runtime/   # Managed Python environment
│       └── __init__.py   # PythonRuntime
├── tests/                # Unit test suite
│   ├── test_catalog.py
│   ├── test_compute.py
│   ├── test_engine.py
│   ├── test_parsers.py
│   ├── test_project.py
│   ├── test_splitter.py
│   ├── test_settings.py
│   ├── test_paths.py
│   └── ... (12 test files, 99+ tests)
├── packaging/            # Build & installer scripts
└── build.ps1

```

```
| | ai_voice_studio.spec
| | installer_64.iss
| | installer_common.iss
| tools/                # Developer utilities
| | dead_code_audit.py
| docs/                 # Documentation
| | book/                # This book!
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `ai_voice_studio/`
`ai_voice_studio/`
2. **Line 2:** `|─ main.py` # Application entry point
`|─ main.py` # Application entry point
3. **Line 3:** `|─ requirements.txt` # Core dependencies
`|─ requirements.txt` # Core dependencies
4. **Line 4:** `|─ LICENSE` # GPL-3.0 license
`|─ LICENSE` # GPL-3.0 license
5. **Line 5:** `|─ README.md` # Project documentation
`|─ README.md` # Project documentation
6. **Line 6:** `|─ PROJECT_SPEC.md` # Full feature specification
`|─ PROJECT_SPEC.md` # Full feature specification
7. **Line 7:** `|─ DEV_PLAN.md` # Development plan
`|─ DEV_PLAN.md` # Development plan
8. **Line 8:** `|─ ai_voice_studio/` # Main package
`|─ ai_voice_studio/` # Main package
9. **Line 9:** `| |─ __init__.py`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
10. **Line 10:** `| |─ constants.py` # Application constants
`| |─ constants.py` # Application constants
11. **Line 11:** `| |─ paths.py` # File path management
`| |─ paths.py` # File path management
12. **Line 12:** `| |─ settings.py` # User preferences (JSON)
`| |─ settings.py` # User preferences (JSON)
13. **Line 13:** `| |─ project.py` # Project persistence
`| |─ project.py` # Project persistence


```
14. Line 14: | └─ compute.py          # GPU/CPU detection
           | └─ compute.py          # GPU/CPU detection

15. Line 15: | └─ util.py           # Utility functions
           | └─ util.py           # Utility functions

16. Line 16: | └─ heavy_logging.py   # Developer logging
           | └─ heavy_logging.py   # Developer logging

17. Line 17: | └─ runtime.py         # Runtime management
           | └─ runtime.py         # Runtime management

18. Line 18: | └─ gui/              # wxPython GUI modules
           | └─ gui/              # wxPython GUI modules

19. Line 19: | | └─ main_frame.py    # Main window
           | | └─ main_frame.py    # Main window

20. Line 20: | | └─ settings_dialog.py # Settings window
           | | └─ settings_dialog.py # Settings window

21. Line 21: | | └─ model_panels.py  # Voice download/management
           | | └─ model_panels.py  # Voice download/management

22. Line 22: | | └─ new_project_wizard.py # Project creation wizard
           | | └─ new_project_wizard.py # Project creation wizard

23. Line 23: | | └─ recording_dialog.py # Recording window
           | | └─ recording_dialog.py # Recording window

24. Line 24: | | └─ a11y.py          # Accessibility helpers
           | | └─ a11y.py          # Accessibility helpers

25. Line 25: | | └─ theme.py         # Light/Dark themes
           | | └─ theme.py         # Light/Dark themes

26. Line 26: | | └─ events.py        # Custom wx events
           | | └─ events.py        # Custom wx events

27. Line 27: | └─ tts/              # TTS engine subsystem
```

```
| | └─ tts/                                # TTS engine subsystem

28. Line 28: | | └─ catalog.py                # Voice catalog (JSON data)
| | └─ catalog.py                # Voice catalog (JSON data)

29. Line 29: | | └─ engine.py                # TTS engine wrapper
| | └─ engine.py                # TTS engine wrapper

30. Line 30: | | └─ models.py                # Model store & file resolution
| | └─ models.py                # Model store & file resolution

31. Line 31: | | └─ downloader.py           # Voice model downloads
| | └─ downloader.py           # Voice model downloads

32. Line 32: | └─ audio/                    # Audio output subsystem
| └─ audio/                    # Audio output subsystem

33. Line 33: | | └─ output.py               # WAV/MP3/FLAC output
| | └─ output.py               # WAV/MP3/FLAC output

34. Line 34: | | └─ ffmpeg.py               # FFmpeg management
| | └─ ffmpeg.py               # FFmpeg management

35. Line 35: | └─ documents/                # Document parsing
| └─ documents/                # Document parsing

36. Line 36: | | └─ parsers.py              # PDF/DOCX/HTML parsers
| | └─ parsers.py              # PDF/DOCX/HTML parsers

37. Line 37: | | └─ splitter.py             # Text splitting logic
| | └─ splitter.py             # Text splitting logic

38. Line 38: | | └─ daisy_builder.py        # DAISY 2.02 book builder
| | └─ daisy_builder.py        # DAISY 2.02 book builder

39. Line 39: | └─ jobs/                     # Background processing
| └─ jobs/                     # Background processing

40. Line 40: | | └─ synthesizer.py          # Background TTS worker
| | └─ synthesizer.py          # Background TTS worker
```

41. **Line 41:** | | | voicelab/ # Voice cloning (Pocket TTS, Bark, F5-TTS)
The Voice Lab package: the engines that clone a voice from a short sample. Each one is installed into its own Python environment (see Chapter 77).
42. **Line 42:** | | | engines.py # Engine registry & built-in voices
The engine registry as data: package names, built-in voices, licences and the CUDA wheel index.
43. **Line 43:** | | | worker.py # Cloning subprocess (own env per engine)
A standalone script the application starts with the chosen engine's own interpreter, so PyTorch never loads inside the application.
44. **Line 44:** | | | omnivoice/ # OmniVoice TTS engine
| | | omnivoice/ # OmniVoice TTS engine
45. **Line 45:** | | | __init__.py # OmniVoiceEngine
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
46. **Line 46:** | | | worker.py # OmniVoice subprocess
| | | worker.py # OmniVoice subprocess
47. **Line 47:** | | | addons/ # Addon system
| | | addons/ # Addon system
48. **Line 48:** | | | __init__.py # AddonManager
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
49. **Line 49:** | | | python_runtime/ # Managed Python environment
| | | python_runtime/ # Managed Python environment
50. **Line 50:** | | | __init__.py # PythonRuntime
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
51. **Line 51:** | | | tests/ # Unit test suite
| | | tests/ # Unit test suite
52. **Line 52:** | | | test_catalog.py
| | | test_catalog.py
53. **Line 53:** | | | test_compute.py
| | | test_compute.py
54. **Line 54:** | | | test_engine.py

```
| | └─ test_engine.py

55. Line 55: | | └─ test_parsers.py
| | └─ test_parsers.py

56. Line 56: | | └─ test_project.py
| | └─ test_project.py

57. Line 57: | | └─ test_splitter.py
| | └─ test_splitter.py

58. Line 58: | | └─ test_settings.py
| | └─ test_settings.py

59. Line 59: | | └─ test_paths.py
| | └─ test_paths.py

60. Line 60: | └─ ... (12 test files, 99+ tests)
| └─ ... (12 test files, 99+ tests)

61. Line 61: | └─ packaging/ # Build & installer scripts
| └─ packaging/ # Build & installer scripts

62. Line 62: | | └─ build.ps1
| | └─ build.ps1

63. Line 63: | | └─ ai_voice_studio.spec
| | └─ ai_voice_studio.spec

64. Line 64: | | └─ installer_64.iss
| | └─ installer_64.iss

65. Line 65: | └─ installer_common.iss
| └─ installer_common.iss

66. Line 66: | └─ tools/ # Developer utilities
| └─ tools/ # Developer utilities

67. Line 67: | └─ dead_code_audit.py
| └─ dead_code_audit.py
```

```
68. Line 68: docs/ # Documentation
69. Line 69: book/ # This book!
```

What Each Module Does

Module	Purpose	Key Concepts
main.py	Entry point: creates wx.App, shows MainFrame	Script execution, if __name__
constants.py	All magic numbers and string constants	Module-level constants, enums
paths.py	Cross-platform path management	os.path , pathlib
settings.py	JSON-based user preferences	File I/O, JSON, defaults
project.py	Project persistence (save/load)	JSON, atomic writes
gui/main_frame.py	Main application window	wxPython frames, menus, events
tts/engine.py	Sherpa-onnx TTS wrapper	CTypes, audio processing
jobs/synthesizer.py	Background TTS processing	Threading, events, callbacks
addons/__init__.py	Plugin/addon system	Dynamic imports, JSON manifests
python_runtime/__init__.py	Managed virtual environment	Subprocess, venv creation

1.5 Setting Up the Development Environment

Step 1: Clone the Repository

```
# Clone the repository (replace with your actual URL)
git clone https://github.com/your-org/ai-voice-studio.git
cd ai-voice-studio
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `# Clone the repository (replace with your actual URL)`
A comment: *Clone the repository (replace with your actual URL).*
2. **Line 2:** `git clone https://github.com/your-org/ai-voice-studio.git`
`git clone https://github.com/your-org/ai-voice-studio.git`
3. **Line 3:** `cd ai-voice-studio`
`cd ai-voice-studio`

Step 2: Create a Virtual Environment

A **virtual environment** is an isolated Python environment that prevents dependency conflicts between projects. This is one of the most important practices in Python development:

```
# Create a virtual environment named .venv
python -m venv .venv

# Activate it (Windows PowerShell)
.venv\Scripts\Activate.ps1

# Activate it (Windows Command Prompt)
.venv\Scripts\activate.bat

# Activate it (macOS/Linux)
source .venv/bin/activate
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# Create a virtual environment named .venv`

A comment: *Create a virtual environment named .venv.*

2. **Line 2:** `python -m venv .venv`

`python -m venv .venv`

3. **Line 3:** `# Activate it (Windows PowerShell)`

A comment: *Activate it (Windows PowerShell).*

4. **Line 4:** `.venv\Scripts\Activate.ps1`

`.venv\Scripts\Activate.ps1`

5. **Line 5:** `# Activate it (Windows Command Prompt)`

A comment: *Activate it (Windows Command Prompt).*

6. **Line 6:** `.venv\Scripts\activate.bat`

`.venv\Scripts\activate.bat`

7. **Line 7:** `# Activate it (macOS/Linux)`

A comment: *Activate it (macOS/Linux).*

8. **Line 8:** `source .venv/bin/activate`

`source .venv/bin/activate`

Once activated, your terminal prompt changes to show `(.venv)` , and all Python/pip commands use the isolated environment:

```
(.venv) $ python --version
Python 3.13.0
(.venv) $ pip --version
pip 24.2 from .venv/lib/python3.13/site-packages/pip
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `(.venv) $ python --version`
`(.venv) $ python --version`
2. **Line 2:** `Python 3.13.0`
`Python 3.13.0`
3. **Line 3:** `(.venv) $ pip --version`
`(.venv) $ pip --version`
4. **Line 4:** `pip 24.2 from .venv/lib/python3.13/site-packages/pip`
`pip 24.2 from .venv/lib/python3.13/site-packages/pip`

Step 3: Install Dependencies

AI Voice Studio requires six core dependencies:

```
# Install from requirements.txt
pip install -r requirements.txt
```

Explain Line by Line Code Description (Block 10)

1. **Line 1:** `# Install from requirements.txt`
A comment: *Install from requirements.txt.*
2. **Line 2:** `pip install -r requirements.txt`
`pip install -r requirements.txt`

Package	Version	Purpose
wxPython	≥4.3.1	Cross-platform native GUI toolkit
sherpa-onnx	≥1.13.0	Neural TTS engine (ONNX runtime)
numpy	≥2.0	Numerical array operations for audio
requests	≥2.31	HTTP client for downloading models
pypdf	≥5.0	PDF document parsing
python-docx	≥1.1	Word document (.docx) parsing

 **Tip: Why virtual environments?** Without them, installing wxPython for one project could overwrite packages needed by another. The `.venv` directory contains a complete, isolated Python installation. It is listed in `.gitignore` so it is never committed to version control.

Step 4: Verify the Installation

```
# Test all core imports
python -c "
import wx
import sherpa_onnx
import numpy
import requests
import pypdf
import docx
print('All imports successful!')
print(f'wxPython: {wx.__version__}')
print(f'NumPy: {numpy.__version__}')
print(f'sherpa-onnx: {sherpa_onnx.__version__}')
"
```

Explain Line by Line Code Description (Block 11)

1. **Line 1:** `# Test all core imports`

A comment: *Test all core imports.*

2. **Line 2:** `python -c "`

`python -c "`

3. **Line 3:** `import wx`

Imports the `wx` module.

4. **Line 4:** `import sherpa_onnx`

Imports the `sherpa_onnx` module.

5. **Line 5:** `import numpy`

Imports the `numpy` module.

6. **Line 6:** `import requests`

Imports the `requests` module.

7. **Line 7:** `import pypdf`

Imports the `pypdf` module.

8. **Line 8:** `import docx`

Imports the `docx` module.

9. **Line 9:** `print('All imports successful!')`

Outputs text to the console. The primary way to display information during development and debugging.

10. **Line 10:** `print(f'wxPython: {wx.__version__}')`

Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `print(f'NumPy: {numpy.__version__}')`

Outputs text to the console. The primary way to display information during development and debugging.

12. **Line 12:** `print(f'sherpa-onnx: {sherpa_onnx.__version__}')`

Outputs text to the console. The primary way to display information during development and debugging.

13. **Line 13:** `"`

`"`

Step 5: Run the Tests

```
# Install pytest first
pip install pytest

# Run the full test suite
python -m pytest tests/ -v
```

Explain Line by Line Code Description (Block 12)

1. **Line 1:** `# Install pytest first`
A comment: *Install pytest first.*
2. **Line 2:** `pip install pytest`
`pip install pytest`
3. **Line 3:** `# Run the full test suite`
A comment: *Run the full test suite.*
4. **Line 4:** `python -m pytest tests/ -v`
`python -m pytest tests/ -v`

You should see all 99 tests pass:

```
tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique PASSED
tests/test_catalog.py::CatalogStructureTest::test_required_tts_present PASSED
tests/test_compute.py::CudaDetectionTest::test_cpu_always_available PASSED
...
===== 99 passed in 12.34s =====
```

Explain Line by Line Code Description (Block 13)

1. **Line 1:** `tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique PASSED`
`tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique PASSED`
2. **Line 2:** `tests/test_catalog.py::CatalogStructureTest::test_required_tts_present PASSED`
`tests/test_catalog.py::CatalogStructureTest::test_required_tts_present PASSED`
3. **Line 3:** `tests/test_compute.py::CudaDetectionTest::test_cpu_always_available PASSED`
`tests/test_compute.py::CudaDetectionTest::test_cpu_always_available PASSED`
4. **Line 4:** `...`
Ellipsis: abstract method placeholder.
5. **Line 5:** `===== 99 passed in 12.34s =====`
`===== 99 passed in 12.34s =====`

Step 6: Run the Application

```
python main.py
```

Explain Line by Line Code Description (Block 14)

1. **Line 1:** `python main.py`
`python main.py`

The AI Voice Studio main window should appear. You will see a menu bar with File, Edit, Tools, Settings, and Help menus, and a status bar at the bottom.

1.6 Your First Python Program

Before we dive into the project, let's write a simple Python program to make sure everything works:

```
# first_program.py

# This is a comment - Python ignores it

# Variables don't need type declarations
app_name = "AI Voice Studio"
version = 2.0
is_open_source = True

# Print with f-strings (formatted string literals)
print(f"Welcome to {app_name} v{version}")
print(f"Open source: {is_open_source}")

# List of TTS engines
engines = ["Piper", "Kokoro", "Matcha", "Kitten"]
print(f"Available engines: {' '.join(engines)}")

# Simple function
def greet(name):
    return f"Hello, {name}! Ready to build TTS apps."

print(greet("Developer"))
```

Explain Line by Line Code Description (Block 15)

1. **Line 1:** `# first_program.py`
A comment: *first_program.py*.
2. **Line 2:** `# This is a comment - Python ignores it`
A comment: *This is a comment - Python ignores it*.
3. **Line 3:** `# Variables don't need type declarations`
A comment: *Variables don't need type declarations*.
4. **Line 4:** `app_name = "AI Voice Studio"`
Assigns `"AI Voice Studio"` to `app_name`.
5. **Line 5:** `version = 2.0`
Assigns `2.0` to `version`.
6. **Line 6:** `is_open_source = True`
Assigns `True` to `is_open_source`.
7. **Line 7:** `# Print with f-strings (formatted string literals)`
A comment: *Print with f-strings (formatted string literals)*.
8. **Line 8:** `print(f"Welcome to {app_name} v{version}")`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(f"Open source: {is_open_source}")`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# List of TTS engines`
A comment: *List of TTS engines*.
11. **Line 11:** `engines = ["Piper", "Kokoro", "Matcha", "Kitten"]`
Assigns `["Piper", "Kokoro", "Matcha", "Kitten"]` to `engines`.
12. **Line 12:** `print(f"Available engines: {' '.join(engines)}")`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `# Simple function`
A comment: *Simple function*.
14. **Line 14:** `def greet(name):`

Defines function `greet` accepting (name).

15. **Line 15:** `return f"Hello, {name}! Ready to build TTS apps."`

Returns `f"Hello, {name}! Ready to build TTS apps."` to the caller.

16. **Line 16:** `print(greet("Developer"))`

Outputs text to the console. The primary way to display information during development and debugging.

```
$ python first_program.py
Welcome to AI Voice Studio v2.0
Open source: True
Available engines: Piper, Kokoro, Matcha, Kitten
Hello, Developer! Ready to build TTS apps.
```

Explain Line by Line Code Description (Block 16)

1. **Line 1:** `$ python first_program.py`

`$ python first_program.py`

2. **Line 2:** `Welcome to AI Voice Studio v2.0`

`Welcome to AI Voice Studio v2.0`

3. **Line 3:** `Open source: True`

`Open source: True`

4. **Line 4:** `Available engines: Piper, Kokoro, Matcha, Kitten`

`Available engines: Piper, Kokoro, Matcha, Kitten`

5. **Line 5:** `Hello, Developer! Ready to build TTS apps.`

`Hello, Developer! Ready to build TTS apps.`

Congratulations! You have just written your first Python program.

1.7 A Note on Python Versions

This book targets **Python 3.13**, but all code is compatible with Python 3.11+. Here are the key version features we use:

Version	Feature	Example
3.6+	f-strings	<code>f"Hello {name}"</code>
3.7+	dataclasses	<code>@dataclass class Foo:</code>
3.8+	Walrus operator <code>:=</code>	<code>if (n := len(x)) > 5:</code>
3.9+	Built-in generics	<code>list[str]</code> instead of <code>List[str]</code>
3.10+	match/case	<code>match command: case "quit":</code>
3.11+	Exception groups	<code>except* ValueError:</code>
3.12+	Type parameter syntax	<code>def foo[T](x: T) -> T:</code>

1.8 IDE Setup

While any text editor works, a proper IDE dramatically improves productivity.

VS Code (Recommended)

1. Install [VS Code](#)
2. Install the **Python** extension by Microsoft
3. Install the **Pylance** extension for type checking
4. Open the project folder and select the `.venv` as the interpreter

PyCharm

1. Install [PyCharm](#) (Community is free)
2. Open the project directory
3. Go to Settings → Project → Python Interpreter → Add → Existing Environment → select `.venv/Scripts/python.exe`

1.9 The GitHub Copilot / AI-Assisted Development

Modern Python development often includes AI assistants. Throughout this book, we write code that is *readable and well-documented* — exactly the kind of code that AI assistants work best with. The AI Voice Studio project itself follows these principles:

- Clear variable and function names (no single-letter variables except loop counters)
 - Docstrings on every public class and function
 - Type hints on function signatures
 - Constants instead of magic numbers
-

1.10 Key Takeaways

- **Python is interpreted** — you write code and run it directly, no compilation step needed
 - **Virtual environments** are essential for isolating project dependencies
 - **AI Voice Studio** is a production-quality Python desktop app that demonstrates every major Python concept
 - The project has **53 Python source files** across 13 subpackages, plus 12 test files
 - All core dependencies (wxPython, sherpa-onnx, numpy, requests, pypdf, python-docx) are installed from `requirements.txt`
 - The test suite has **99+ tests** — good testing is not optional in real projects
 - An IDE with Python support makes development much more productive
-

 **Exercise: Exercise 1.1:** Create a virtual environment, install the dependencies from `requirements.txt`, and run `python main.py`. Verify the application window appears.

 **Exercise: Exercise 1.2:** Write a script called `count_lines.py` that counts the number of lines in every `.py` file under the `ai_voice_studio/` directory. Hint: use `os.walk()` and a loop.

 **Exercise: Exercise 1.3:** Open the `main.py` file and identify the imports at the top. For each import, write a one-sentence explanation of what it does. (You can look at the import statements — you will understand them fully by Chapter 9.)

[Next: Chapter 2 — Variables & Data Types →](#)

Variables & Data Types

Learning Objectives

- Understand Python's dynamic typing system
 - Master all primitive data types: int, float, bool, str, None
 - Learn variable naming conventions and best practices
 - Understand type checking and type conversion
 - See how AI Voice Studio uses variables and constants
-

2.1 Python Is Dynamically Typed

Unlike languages such as C, Java, or Go, Python does not require you to declare the type of a variable. The type is determined at runtime based on the value assigned:

```
# Python figures out the type automatically
name = "AI Voice Studio"      # str
version = 2.0                 # float
year = 2026                   # int
is_released = True            # bool
nothing = None                 # NoneType

# You can reassign to a different type (though you usually shouldn't)
x = 42                        # int
x = "hello"                   # now str - Python doesn't care
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Python figures out the type automatically`
A comment: *Python figures out the type automatically.*
2. **Line 2:** `name = "AI Voice Studio" # str`
Assigns "AI Voice Studio" # str to name .
3. **Line 3:** `version = 2.0 # float`
Assigns 2.0 # float to version .
4. **Line 4:** `year = 2026 # int`
5. **Line 5:** `is_released = True # bool`
Assigns True # bool to is_released .
6. **Line 6:** `nothing = None # NoneType`
Assigns None # NoneType to nothing .
7. **Line 7:** `# You can reassign to a different type (though you usually shouldn't)`
A comment: *You can reassign to a different type (though you usually shouldn't).*
8. **Line 8:** `x = 42 # int`
Assigns 42 # int to x .
9. **Line 9:** `x = "hello" # now str – Python doesn't care`
Assigns "hello" # now str – Python doesn't care to x .

This flexibility makes Python fast to write but requires discipline. The project uses **type hints** (Chapter 22) to document expected types without enforcing them at runtime.

The `type()` Function

Use `type()` to check a variable's type at any time:

```
>>> type(42)
<class 'int'>
>>> type(3.14)
<class 'float'>
>>> type("hello")
<class 'str'>
>>> type(True)
<class 'bool'>
>>> type(None)
<class 'NoneType'>
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `>>> type(42)`

Returns the type/class of an object. Used for debugging and type checking.

2. **Line 2:** `<class 'int'>`

`<class 'int'>`

3. **Line 3:** `>>> type(3.14)`

Returns the type/class of an object. Used for debugging and type checking.

4. **Line 4:** `<class 'float'>`

`<class 'float'>`

5. **Line 5:** `>>> type("hello")`

Returns the type/class of an object. Used for debugging and type checking.

6. **Line 6:** `<class 'str'>`

`<class 'str'>`

7. **Line 7:** `>>> type(True)`

Returns the type/class of an object. Used for debugging and type checking.

8. **Line 8:** `<class 'bool'>`

`<class 'bool'>`

9. **Line 9:** `>>> type(None)`

Returns the type/class of an object. Used for debugging and type checking.

10. **Line 10:** `<class 'NoneType'>`

`<class 'NoneType'>`

The `isinstance()` Function

More Pythonic than `type() == SomeType` is using `isinstance()`, which also works with inheritance:

```
x = 42
print(isinstance(x, int))          # True
print(isinstance(x, (int, float))) # True - can check multiple types
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `x = 42`
Assigns `42` to `x`.
2. **Line 2:** `print(isinstance(x, int))` `# True`
Checks if an object is an instance of a class. Preferred over `type()` for type checking.
3. **Line 3:** `print(isinstance(x, (int, float)))` `# True - can check multiple types`
Checks if an object is an instance of a class. Preferred over `type()` for type checking.

2.2 Numeric Types

Integers (`int`)

Python integers have **arbitrary precision** — they can be as large as your memory allows. No overflow:

```
# Normal integers
segments = 42
sample_rate = 24000

# Very large integers (no overflow!)
huge = 10 ** 100 # a googol

# Different bases
binary = 0b1010 # = 10 in decimal
octal = 0o17    # = 15 in decimal
hexadecimal = 0xFF # = 255 in decimal

# Underscores for readability (Python 3.6+)
max_concurrent = 1_000_000 # same as 1000000
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# Normal integers`
A comment: *Normal integers*.
2. **Line 2:** `segments = 42`
Assigns `42` to `segments`.
3. **Line 3:** `sample_rate = 24000`
Assigns `24000` to `sample_rate`.
4. **Line 4:** `# Very large integers (no overflow!)`
A comment: *Very large integers (no overflow!)*.
5. **Line 5:** `huge = 10 ** 100 # a googol`
Assigns `10 ** 100 # a googol` to `huge`.
6. **Line 6:** `# Different bases`
A comment: *Different bases*.
7. **Line 7:** `binary = 0b1010 # = 10 in decimal`
Assigns `0b1010 # = 10 in decimal` to `binary`.
8. **Line 8:** `octal = 0o17 # = 15 in decimal`
Assigns `0o17 # = 15 in decimal` to `octal`.
9. **Line 9:** `hexadecimal = 0xFF # = 255 in decimal`
Assigns `0xFF # = 255 in decimal` to `hexadecimal`.
10. **Line 10:** `# Underscores for readability (Python 3.6+)`
A comment: *Underscores for readability (Python 3.6+)*.
11. **Line 11:** `max_concurrent = 1_000_000 # same as 1000000`
Assigns `1_000_000 # same as 1000000` to `max_concurrent`.

Floats (float)

```
# Floats (64-bit double precision)
pitch = 1.0
volume = 0.75
speed_factor = 1.5

# Scientific notation
sample_size = 1.6e-19 # 1.6 × 10-19

# ⚠ Float precision is limited
print(0.1 + 0.2) # 0.30000000000000004
print(0.1 + 0.2 == 0.3) # False!

# For precise decimal arithmetic, use the decimal module
from decimal import Decimal
print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3")) # True
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# Floats (64-bit double precision)`
A comment: *Floats (64-bit double precision).*
2. **Line 2:** `pitch = 1.0`
Assigns `1.0` to `pitch`.
3. **Line 3:** `volume = 0.75`
Assigns `0.75` to `volume`.
4. **Line 4:** `speed_factor = 1.5`
Assigns `1.5` to `speed_factor`.
5. **Line 5:** `# Scientific notation`
A comment: *Scientific notation.*
6. **Line 6:** `sample_size = 1.6e-19 # 1.6 × 10-19`
Assigns `1.6e-19` `# 1.6 × 10-19` to `sample_size`.
7. **Line 7:** `# ⚠ Float precision is limited`
A comment: *⚠ Float precision is limited.*
8. **Line 8:** `print(0.1 + 0.2) # 0.30000000000000004`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(0.1 + 0.2 == 0.3) # False!`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# For precise decimal arithmetic, use the decimal module`
A comment: *For precise decimal arithmetic, use the decimal module.*
11. **Line 11:** `from decimal import Decimal`
Imports `Decimal` from `decimal`.
12. **Line 12:** `print(Decimal("0.1") + Decimal("0.2") == Decimal("0.3")) # True`
Outputs text to the console. The primary way to display information during development and debugging.

⚠ **Warning:** Float comparison is a classic Python gotcha. Never use `==` to compare floats for equality. Instead, check if they are close enough: `abs(a - b) < 1e-9` or use `math.isclose(a, b)`.

Complex Numbers

```
# Complex numbers (used in signal processing / audio)
z = 3 + 4j
print(z.real)    # 3.0
print(z.imag)    # 4.0
print(abs(z))    # 5.0 (magnitude)
```

Explain Line by Line Code Description (Block 6)

- Line 1:** `# Complex numbers (used in signal processing / audio)`
A comment: *Complex numbers (used in signal processing / audio)*.
- Line 2:** `z = 3 + 4j`
Assigns `3 + 4j` to `z`.
- Line 3:** `print(z.real) # 3.0`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 4:** `print(z.imag) # 4.0`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 5:** `print(abs(z)) # 5.0 (magnitude)`
Outputs text to the console. The primary way to display information during development and debugging.

2.3 Booleans

`bool` is a subclass of `int` — `True` is 1, `False` is 0:

```
is_running = True
has_gpu = False

# Booleans participate in arithmetic
print(True + True + False) # 2
print(True * 10)           # 10

# Truthiness: many values are "truthy" or "falsy"
print(bool(1))              # True
print(bool(0))              # False
print(bool("hello"))        # True (non-empty string)
print(bool(""))             # False (empty string)
print(bool([]))             # False (empty list)
print(bool([1]))            # True (non-empty list)
print(bool(None))           # False
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `is_running = True`
Assigns `True` to `is_running`.
2. **Line 2:** `has_gpu = False`
Assigns `False` to `has_gpu`.
3. **Line 3:** `# Booleans participate in arithmetic`
A comment: *Booleans participate in arithmetic.*
4. **Line 4:** `print(True + True + False) # 2`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `print(True * 10) # 10`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `# Truthiness: many values are "truthy" or "falsy"`
A comment: *Truthiness: many values are "truthy" or "falsy".*
7. **Line 7:** `print(bool(1)) # True`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `print(bool(0)) # False`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(bool("hello")) # True (non-empty string)`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `print(bool("")) # False (empty string)`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `print(bool([])) # False (empty list)`
Outputs text to the console. The primary way to display information during development and debugging.
12. **Line 12:** `print(bool([1])) # True (non-empty list)`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `print(bool(None)) # False`
Outputs text to the console. The primary way to display information during development and debugging.

Boolean Operators

```
# and, or, not
has_gpu = True
runtime_installed = False

can_use_cuda = has_gpu and runtime_installed # False
use_cpu = not has_gpu or runtime_installed # False

# Short-circuit evaluation (important for performance)
# "and" stops at the first False, "or" stops at the first True
result = False and expensive_function() # expensive_function() never called!
```

Explain Line by Line Code Description (Block 8)

- Line 1:** `# and, or, not`
A comment: *and, or, not*.
- Line 2:** `has_gpu = True`
Assigns `True` to `has_gpu`.
- Line 3:** `runtime_installed = False`
Assigns `False` to `runtime_installed`.
- Line 4:** `can_use_cuda = has_gpu and runtime_installed # False`
Assigns `has_gpu and runtime_installed # False` to `can_use_cuda`.
- Line 5:** `use_cpu = not has_gpu or runtime_installed # False`
Assigns `not has_gpu or runtime_installed # False` to `use_cpu`.
- Line 6:** `# Short-circuit evaluation (important for performance)`
A comment: *Short-circuit evaluation (important for performance)*.
- Line 7:** `# "and" stops at the first False, "or" stops at the first True`
A comment: *"and" stops at the first False, "or" stops at the first True*.
- Line 8:** `result = False and expensive_function() # expensive_function() never called!`
Assigns `False and expensive_function() # expensive_function() never called!` to `result`.

2.4 Strings

Strings in Python are **immutable sequences of Unicode characters**. They are one of the most commonly used types:

```
# String creation (three ways)
name = "AI Voice Studio"      # double quotes
name = 'AI Voice Studio'      # single quotes (same thing)
doc = """This is a
multi-line string."""         # triple quotes

# Strings are immutable
s = "hello"
# s[0] = "H" # TypeError! Can't change characters

# Must create a new string
s = "H" + s[1:]               # "Hello"
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `# String creation (three ways)`
A comment: *String creation (three ways).*
2. **Line 2:** `name = "AI Voice Studio" # double quotes`
Assigns `"AI Voice Studio"` # double quotes to `name` .
3. **Line 3:** `name = 'AI Voice Studio' # single quotes (same thing)`
Assigns `'AI Voice Studio'` # single quotes (same thing) to `name` .
4. **Line 4:** `doc = """This is a`
Assigns `"""This is a` to `doc` .
5. **Line 5:** `multi-line string.""" # triple quotes`
`multi-line string."""` # triple quotes
6. **Line 6:** `# Strings are immutable`
A comment: *Strings are immutable.*
7. **Line 7:** `s = "hello"`
Assigns `"hello"` to `s` .
8. **Line 8:** `# s[0] = "H" # TypeError! Can't change characters`
A comment: *s[0] = "H" # TypeError! Can't change characters.*
9. **Line 9:** `# Must create a new string`
A comment: *Must create a new string.*
10. **Line 10:** `s = "H" + s[1:] # "Hello"`
Assigns `"H" + s[1:]` # "Hello" to `s` .

Common String Operations

```
text = "Hello, AI Voice Studio!"

# Length
print(len(text))          # 23

# Indexing (0-based)
print(text[0])            # "H"
print(text[-1])           # "!" (last character)

# Slicing [start:stop:step]
print(text[7:19])          # "AI Voice Stud"
print(text[::-2])          # "HloAVcsad!" (every other char)

# Methods
print(text.upper())        # "HELLO, AI VOICE STUDIO!"
print(text.lower())        # "hello, ai voice studio!"
print(text.replace("AI", "ML")) # "Hello, ML Voice Studio!"
print(text.split(", "))    # ["Hello", "AI Voice Studio!"]
print(text.find("Voice"))   # 7 (index of first match)
print(text.startswith("Hello")) # True
print(text.endswith("!"))   # True

# Stripping whitespace
padded = "  hello  "
print(padded.strip())       # "hello"
print(padded.lstrip())      # "hello "
```

Explain Line by Line Code Description (Block 10)

1. **Line 1:** `text = "Hello, AI Voice Studio!"`
Assigns "Hello, AI Voice Studio!" to text .
2. **Line 2:** `# Length`
A comment: *Length*.
3. **Line 3:** `print(len(text))` `# 23`
Outputs text to the console. The primary way to display information during development and debugging.
4. **Line 4:** `# Indexing (0-based)`
A comment: *Indexing (0-based)*.
5. **Line 5:** `print(text[0])` `# "H"`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `print(text[-1])` `# "!" (last character)`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# Slicing [start:stop:step]`
A comment: *Slicing [start:stop:step]*.
8. **Line 8:** `print(text[7:19])` `# "AI Voice Stud"`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(text[::2])` `# "HloAVcsad!" (every other char)`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# Methods`
A comment: *Methods*.
11. **Line 11:** `print(text.upper())` `# "HELLO, AI VOICE STUDIO!"`
Converts all characters to uppercase. Returns a new string (strings are immutable).
12. **Line 12:** `print(text.lower())` `# "hello, ai voice studio!"`
Converts all characters to lowercase. Returns a new string.
13. **Line 13:** `print(text.replace("AI", "ML"))` `# "Hello, ML Voice Studio!"`
Returns a copy with all occurrences of old replaced by new. Strings are immutable.
14. **Line 14:** `print(text.split(", "))` `# ["Hello", "AI Voice Studio!"]`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.

15. **Line 15:** `print(text.find("Voice"))` # 7 (index of first match)
Outputs text to the console. The primary way to display information during development and debugging.
16. **Line 16:** `print(text.startswith("Hello"))` # True
Returns True if the string starts with the given prefix.
17. **Line 17:** `print(text.endswith("!"))` # True
Returns True if the string ends with the given suffix.
18. **Line 18:** `# Stripping whitespace`
A comment: *Stripping whitespace*.
19. **Line 19:** `padded = " hello "`
Assigns `" hello "` to `padded`.
20. **Line 20:** `print(padded.strip())` # "hello"
Removes leading and trailing whitespace (or specified characters). Returns a new string.
21. **Line 21:** `print(padded.lstrip())` # "hello "
Removes leading whitespace only.

String Formatting (f-strings)

F-strings (Python 3.6+) are the preferred way to embed expressions in strings:

```
name = "Piper"
voices = 153

# Basic interpolation
print(f"TTS engine: {name}")

# Expressions inside braces
print(f"{name.upper()} has {voices * 2} total voice files")

# Format specifiers
pi = 3.14159
print(f"Pi to 2 decimal places: {pi:.2f}") # 3.14
print(f"Right-aligned: '{42:10}'") # '         42'
print(f"Zero-padded: {42:05d}") # 00042
print(f"Percentage: {0.857:.1%}") # 85.7%
print(f"Bytes: {1048576:,.0f}") # 1,048,576

# Debugging: = shows expression AND result
voices = 153
print(f"{voices =}") # voices = 153
```


Explain Line by Line Code Description (Block 11)

1. **Line 1:** `name = "Piper"`
Assigns "Piper" to name .
2. **Line 2:** `voices = 153`
Assigns 153 to voices .
3. **Line 3:** `# Basic interpolation`
A comment: *Basic interpolation*.
4. **Line 4:** `print(f"TTS engine: {name}")`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `# Expressions inside braces`
A comment: *Expressions inside braces*.
6. **Line 6:** `print(f"{name.upper()} has {voices * 2} total voice files")`
Converts all characters to uppercase. Returns a new string (strings are immutable).
7. **Line 7:** `# Format specifiers`
A comment: *Format specifiers*.
8. **Line 8:** `pi = 3.14159`
Assigns 3.14159 to pi .
9. **Line 9:** `print(f"Pi to 2 decimal places: {pi:.2f}") # 3.14`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `print(f"Right-aligned: '{42:10}')" # ' 42'`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `print(f"Zero-padded: {42:05d}") # 00042`
Outputs text to the console. The primary way to display information during development and debugging.
12. **Line 12:** `print(f"Percentage: {0.857:.1%}") # 85.7%`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `print(f"Bytes: {1048576:,.0f}") # 1,048,576`
Outputs text to the console. The primary way to display information during development and debugging.
14. **Line 14:** `# Debugging: = shows expression AND result`

A comment: *Debugging: = shows expression AND result.*

15. Line 15: `voices = 153`

Assigns 153 to voices .

16. Line 16: `print(f"{voices = }")` # voices = 153

Outputs text to the console. The primary way to display information during development and debugging.

Strings in AI Voice Studio

String processing is central to text-to-speech. Here are real examples from the project:

```
# From ai_voice_studio/util.py
import re

def sanitize_filename(name: str, max_length: int = 80) -> str:
    """Remove or replace characters that are unsafe in filenames."""
    # Replace characters illegal on Windows
    name = re.sub(r'[<>:"/\|?*]', '_', name)
    # Collapse whitespace
    name = re.sub(r'\s+', ' ', name).strip()
    # Truncate to max length
    if len(name) > max_length:
        name = name[:max_length].rstrip()
    return name

# Usage:
safe = sanitize_filename("My Book: Chapter 1? Yes!")
print(safe) # "My Book_ Chapter 1_ Yes!"
```

Explain Line by Line Code Description (Block 12)

1. **Line 1:** `# From ai_voice_studio/util.py`

A comment: *From ai_voice_studio/util.py.*

2. **Line 2:** `import re`

Imports the `re` module.

3. **Line 3:** `def sanitize_filename(name: str, max_length: int = 80) -> str:`

Defines function `sanitize_filename` accepting (name, max_length), returning `str`.

4. **Line 4:** `"""Remove or replace characters that are unsafe in filenames."""`

Docstring: *Remove or replace characters that are unsafe in filenames..* Documents the purpose of the following code.

5. **Line 5:** `# Replace characters illegal on Windows`

A comment: *Replace characters illegal on Windows.*

6. **Line 6:** `name = re.sub(r'[<>:"/\|?*]', '_', name)`

Replaces all matches with the replacement string.

7. **Line 7:** `# Collapse whitespace`

A comment: *Collapse whitespace.*

8. **Line 8:** `name = re.sub(r'\s+', ' ', name).strip()`

Removes leading and trailing whitespace (or specified characters). Returns a new string.

9. **Line 9:** `# Truncate to max length`

A comment: *Truncate to max length.*

10. **Line 10:** `if len(name) > max_length:`

Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

11. **Line 11:** `name = name[:max_length].rstrip()`

Removes trailing whitespace only.

12. **Line 12:** `return name`

Returns `name` to the caller.

13. **Line 13:** `# Usage:`

A comment: *Usage:.*

14. **Line 14:** `safe = sanitize_filename("My Book: Chapter 1? Yes!")`

```
Assigns sanitize_filename("My Book: Chapter 1? Yes!") to safe .
```

15. **Line 15:** `print(safe) # "My Book_ Chapter 1_ Yes!"`

Outputs text to the console. The primary way to display information during development and debugging.

2.5 The None Type

`None` is Python's way of representing "nothing" or "no value." It is a singleton — there is exactly one `None` object in the entire Python runtime:

```
result = None

# Always compare with "is", not "=="
if result is None:
    print("No result yet")

# ⚠️ Wrong:
if result == None: # Works but un-Pythonic; can be overridden
    pass

# None has a falsy boolean value
print(bool(None)) # False
```

Explain Line by Line Code Description (Block 13)

1. **Line 1:** `result = None`
Assigns `None` to `result`.
2. **Line 2:** `# Always compare with "is", not "=="`
A comment: *Always compare with "is", not "=="*.
3. **Line 3:** `if result is None:`
Checks condition: `result is None`. The indented block runs only when this is `True`.
4. **Line 4:** `print("No result yet")`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `# ⚠️ Wrong:`
A comment: `⚠️ Wrong:`.
6. **Line 6:** `if result == None: # Works but un-Pythonic; can be overridden`
Checks condition: `result == None`. The indented block runs only when this is `True`.
7. **Line 7:** `pass`
Placeholder: does nothing.
8. **Line 8:** `# None has a falsy boolean value`
A comment: *None has a falsy boolean value*.
9. **Line 9:** `print(bool(None)) # False`
Outputs text to the console. The primary way to display information during development and debugging.

In AI Voice Studio, `None` is used extensively for "not set" values:

```
# From ai_voice_studio/clone/__init__.py
class CloneClient:
    def __init__(self, timeout: float = 600.0):
        self._proc: subprocess.Popen | None = None # No process yet

    def _ensure_proc(self):
        if self._proc is not None and self._proc.poll() is None:
            return # Process already running
        # ... start new process
```

Explain Line by Line Code Description (Block 14)

1. **Line 1:** `# From ai_voice_studio/clone/__init__.py`
A comment: *From ai_voice_studio/clone/__init__.py.*
2. **Line 2:** `class CloneClient:`
Defines class `CloneClient` .
3. **Line 3:** `def __init__(self, timeout: float = 600.0):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `self._proc: subprocess.Popen | None = None # No process yet`
Starts a process. More control than `run()` — can read `stdout/stderr` asynchronously.
5. **Line 5:** `def _ensure_proc(self):`
Defines function `_ensure_proc` accepting `(self)`.
6. **Line 6:** `if self._proc is not None and self._proc.poll() is None:`
Checks condition: `self._proc is not None and self._proc.poll() is None` . The indented block runs only when this is `True` .
7. **Line 7:** `return # Process already running`
Returns `# Process already running` to the caller.
8. **Line 8:** `# ... start new process`
A comment: ... *start new process.*

2.6 Variable Naming Conventions

Python has clear conventions for naming:

```

# Variables and functions: snake_case
sample_rate = 24000
output_dir = "/tmp/audio"

def get_engine(engine_id: str):
    pass

# Classes: PascalCase
class TtsEngine:
    pass

class RecordingDialog:
    pass

# Constants: UPPER_SNAKE_CASE
SAMPLE_RATE = 24000
PITCH_MIN = 0.5
PITCH_MAX = 2.0
DEFAULT_FORMAT = "wav"

# Private attributes: Leading underscore
class Settings:
    def __init__(self):
        self._data = {} # Convention: "don't touch from outside"
        self.__secret = 42 # Name mangling (different from _)

# Dunder/magic methods: double underscores
def __init__(self):
    pass

```

Explain Line by Line Code Description (Block 15)

1. **Line 1:** `# Variables and functions: snake_case`
A comment: *Variables and functions: snake_case*.

2. **Line 2:** `sample_rate = 24000`
Assigns `24000` to `sample_rate` .

3. **Line 3:** `output_dir = "/tmp/audio"`
Assigns `"/tmp/audio"` to `output_dir` .

4. **Line 4:** `def get_engine(engine_id: str):`
Defines function `get_engine` accepting `(engine_id)`.

5. **Line 5:** `pass`
Placeholder: does nothing.

6. **Line 6:** `# Classes: PascalCase`
A comment: *Classes: PascalCase*.

7. **Line 7:** `class TtsEngine:`
Defines class `TtsEngine` .

8. **Line 8:** `pass`
Placeholder: does nothing.

9. **Line 9:** `class RecordingDialog:`
Defines class `RecordingDialog` .

10. **Line 10:** `pass`
Placeholder: does nothing.

11. **Line 11:** `# Constants: UPPER_SNAKE_CASE`
A comment: *Constants: UPPER_SNAKE_CASE*.

12. **Line 12:** `SAMPLE_RATE = 24000`
Assigns `24000` to `SAMPLE_RATE` .

13. **Line 13:** `PITCH_MIN = 0.5`
Assigns `0.5` to `PITCH_MIN` .

14. **Line 14:** `PITCH_MAX = 2.0`
Assigns `2.0` to `PITCH_MAX`.
15. **Line 15:** `DEFAULT_FORMAT = "wav"`
Assigns `"wav"` to `DEFAULT_FORMAT`.
16. **Line 16:** `# Private attributes: leading underscore`
A comment: *Private attributes: leading underscore.*
17. **Line 17:** `class Settings:`
Defines class `Settings`.
18. **Line 18:** `def __init__(self):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
19. **Line 19:** `self._data = {} # Convention: "don't touch from outside"`
Stores `{}` `# Convention: "don't touch from outside"` in instance attribute `_data`.
20. **Line 20:** `self.__secret = 42 # Name mangling (different from _)`
Stores `42` `# Name mangling (different from _)` in instance attribute `__secret`.
21. **Line 21:** `# Dunder/magic methods: double underscores`
A comment: *Dunder/magic methods: double underscores.*
22. **Line 22:** `def __init__(self):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
23. **Line 23:** `pass`
Placeholder: does nothing.

Reserved Words (Keywords)

You cannot use Python keywords as variable names:

```
import keyword
print(keyword.kwlist)
# ['False', 'None', 'True', 'and', 'as', 'assert', 'async',
#  'await', 'break', 'class', 'continue', 'def', 'del', 'elif',
#  'else', 'except', 'finally', 'for', 'from', 'global', 'if',
#  'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',
#  'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']
```

Explain Line by Line Code Description (Block 16)

1. **Line 1:** `import keyword`
Imports the `keyword` module.
2. **Line 2:** `print(keyword.kwlist)`
Outputs text to the console. The primary way to display information during development and debugging.
3. **Line 3:** `# ['False', 'None', 'True', 'and', 'as', 'assert', 'async',`
A comment: `['False', 'None', 'True', 'and', 'as', 'assert', 'async',`.
4. **Line 4:** `# 'await', 'break', 'class', 'continue', 'def', 'del', 'elif',`
A comment: `'await', 'break', 'class', 'continue', 'def', 'del', 'elif',`.
5. **Line 5:** `# 'else', 'except', 'finally', 'for', 'from', 'global', 'if',`
A comment: `'else', 'except', 'finally', 'for', 'from', 'global', 'if',`.
6. **Line 6:** `# 'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',`
A comment: `'import', 'in', 'is', 'lambda', 'nonlocal', 'not', 'or',`.
7. **Line 7:** `# 'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']`
A comment: `'pass', 'raise', 'return', 'try', 'while', 'with', 'yield']`.

💡 **Tip:** Good variable names are **self-documenting**. In the AI Voice Studio codebase, `variant_id` is immediately understandable, while `vid` requires you to look up what it means. Always choose clarity over brevity.

2.7 Type Conversion (Casting)

Python does not automatically convert between types, but you can do it explicitly:

```
# int → float
x = float(42)      # 42.0

# float → int (truncates toward zero)
x = int(3.9)       # 3 (NOT 4!)
x = int(-3.9)      # -3

# int/float → str
s = str(42)        # "42"
s = str(3.14)      # "3.14"

# str → int
n = int("42")      # 42
# int("hello")    # ValueError!

# str → float
f = float("3.14")  # 3.14

# int → bool
print(bool(0))     # False
print(bool(1))     # True
print(bool(42))    # True (any non-zero is True)
```

Explain Line by Line Code Description (Block 17)

1. **Line 1:** `# int → float`

A comment: *int* → *float*.

2. **Line 2:** `x = float(42) # 42.0`

Converts a value to a floating-point number. Used for decimal arithmetic.

3. **Line 3:** `# float → int (truncates toward zero)`

A comment: *float* → *int* (*truncates toward zero*).

4. **Line 4:** `x = int(3.9) # 3 (NOT 4!)`

Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.

5. **Line 5:** `x = int(-3.9) # -3`

Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.

6. **Line 6:** `# int/float → str`

A comment: *int/float* → *str*.

7. **Line 7:** `s = str(42) # "42"`

Converts a value to its string representation. Essential for string concatenation and formatting.

8. **Line 8:** `s = str(3.14) # "3.14"`

Converts a value to its string representation. Essential for string concatenation and formatting.

9. **Line 9:** `# str → int`

A comment: *str* → *int*.

10. **Line 10:** `n = int("42") # 42`

Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.

11. **Line 11:** `# int("hello") # ValueError!`

A comment: *int("hello")* # *ValueError!*.

12. **Line 12:** `# str → float`

A comment: *str* → *float*.

13. **Line 13:** `f = float("3.14") # 3.14`

Converts a value to a floating-point number. Used for decimal arithmetic.

14. **Line 14:** `# int → bool`

A comment: *int* → *bool*.

15. **Line 15:** `print(bool(0))` # False
Outputs text to the console. The primary way to display information during development and debugging.
16. **Line 16:** `print(bool(1))` # True
Outputs text to the console. The primary way to display information during development and debugging.
17. **Line 17:** `print(bool(42))` # True (any non-zero is True)
Outputs text to the console. The primary way to display information during development and debugging.

Type Conversion in AIVS

The project frequently converts slider values (integers) to float parameters:

```
# From recording_dialog.py - slider values are ints, params are floats
def _current_params(self):
    return {
        "rate": self.rate.GetValue() / 100.0,    # int → float
        "pitch": self.pitch.GetValue() / 100.0,
        "volume": self.volume.GetValue() / 100.0,
    }

# The slider holds 100 = 1.00, 150 = 1.50, etc.
# This avoids floating-point slider precision issues.
```

Explain Line by Line Code Description (Block 18)

1. **Line 1:** `# From recording_dialog.py – slider values are ints, params are floats`

A comment: *From recording_dialog.py – slider values are ints, params are floats.*

2. **Line 2:** `def _current_params(self):`

Defines function `_current_params` accepting (self).

3. **Line 3:** `return {`

Returns `{` to the caller.

4. **Line 4:** `"rate": self.rate.GetValue() / 100.0, # int → float`

`"rate": self.rate.GetValue() / 100.0, # int → float`

5. **Line 5:** `"pitch": self.pitch.GetValue() / 100.0,`

`"pitch": self.pitch.GetValue() / 100.0,`

6. **Line 6:** `"volume": self.volume.GetValue() / 100.0,`

`"volume": self.volume.GetValue() / 100.0,`

7. **Line 7:** `}`

`}`

8. **Line 8:** `# The slider holds 100 = 1.00, 150 = 1.50, etc.`

A comment: *The slider holds 100 = 1.00, 150 = 1.50, etc..*

9. **Line 9:** `# This avoids floating-point slider precision issues.`

A comment: *This avoids floating-point slider precision issues..*

2.8 Mutable vs. Immutable

This distinction is crucial in Python. **Immutable** objects cannot be changed after creation; **mutable** objects can:

Immutable	Mutable
int , float , complex	list
str	dict
tuple	set
frozenset	bytearray
bytes	Objects (instances of classes)
NoneType	

```

# Immutable: assignment creates a new object
a = "hello"
b = a          # b points to the SAME string
a = "world"    # Now a points to a NEW string
print(b)       # "hello" – b is unchanged!

# Mutable: assignment creates a reference
a = [1, 2, 3]
b = a          # b points to the SAME list
a.append(4)    # Modifies the shared list
print(b)       # [1, 2, 3, 4] – b is also changed!

# To make an independent copy:
b = a.copy()   # or b = list(a) or b = a[:]

```

Explain Line by Line Code Description (Block 19)

1. **Line 1:** `# Immutable: assignment creates a new object`

A comment: *Immutable: assignment creates a new object.*

2. **Line 2:** `a = "hello"`

Assigns `"hello"` to `a`.

3. **Line 3:** `b = a` `# b points to the SAME string`

Assigns `a` `# b points to the SAME string` to `b`.

4. **Line 4:** `a = "world"` `# Now a points to a NEW string`

Assigns `"world"` `# Now a points to a NEW string` to `a`.

5. **Line 5:** `print(b)` `# "hello" - b is unchanged!`

Outputs text to the console. The primary way to display information during development and debugging.

6. **Line 6:** `# Mutable: assignment creates a reference`

A comment: *Mutable: assignment creates a reference.*

7. **Line 7:** `a = [1, 2, 3]`

Assigns `[1, 2, 3]` to `a`.

8. **Line 8:** `b = a` `# b points to the SAME list`

Assigns `a` `# b points to the SAME list` to `b`.

9. **Line 9:** `a.append(4)` `# Modifies the shared list`

Adds a single item to the end of the list.

10. **Line 10:** `print(b)` `# [1, 2, 3, 4] - b is also changed!`

Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `# To make an independent copy:`

A comment: *To make an independent copy:.*

12. **Line 12:** `b = a.copy()` `# or b = list(a) or b = a[:]`

Returns a shallow copy of the dictionary.

 **Note:** The project stores settings as a mutable `dict`. When saving, it serializes the entire dict to JSON. This "snapshot" approach means the settings are effectively immutable at the file level, even though the in-memory dict is mutable.

2.9 Identity vs. Equality

Python has two ways to compare objects:

```
# == checks VALUE equality
a = [1, 2, 3]
b = [1, 2, 3]
print(a == b)    # True – same contents

# "is" checks IDENTITY (same object in memory)
print(a is b)    # False – different list objects

# For singletons like None, always use "is"
x = None
print(x is None)    # True ✓
print(x == None)    # Also True, but un-Pythonic X

# Small integers are cached (CPython optimization)
a = 256
b = 256
print(a is b)    # True – both refer to cached object

a = 257
b = 257
print(a is b)    # False (usually) – outside the cache range
```

Explain Line by Line Code Description (Block 20)

1. **Line 1:** `# == checks VALUE equality`
A comment: `==` checks *VALUE* equality.

2. **Line 2:** `a = [1, 2, 3]`
Assigns `[1, 2, 3]` to `a`.

3. **Line 3:** `b = [1, 2, 3]`
Assigns `[1, 2, 3]` to `b`.

4. **Line 4:** `print(a == b) # True – same contents`
Outputs text to the console. The primary way to display information during development and debugging.

5. **Line 5:** `# "is" checks IDENTITY (same object in memory)`
A comment: `"is"` checks *IDENTITY* (same object in memory).

6. **Line 6:** `print(a is b) # False – different list objects`
Outputs text to the console. The primary way to display information during development and debugging.

7. **Line 7:** `# For singletons like None, always use "is"`
A comment: *For singletons like None, always use "is"*.

8. **Line 8:** `x = None`
Assigns `None` to `x`.

9. **Line 9:** `print(x is None) # True ✓`
Outputs text to the console. The primary way to display information during development and debugging.

10. **Line 10:** `print(x == None) # Also True, but un-Pythonic ✗`
Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `# Small integers are cached (CPython optimization)`
A comment: *Small integers are cached (CPython optimization)*.

12. **Line 12:** `a = 256`
Assigns `256` to `a`.

13. **Line 13:** `b = 256`
Assigns `256` to `b`.

14. **Line 14:** `print(a is b) # True – both refer to cached object`

Outputs text to the console. The primary way to display information during development and debugging.

15. **Line 15:** `a = 257`

Assigns 257 to a .

16. **Line 16:** `b = 257`

Assigns 257 to b .

17. **Line 17:** `print(a is b) # False (usually) – outside the cache range`

Outputs text to the console. The primary way to display information during development and debugging.

2.10 Augmented Assignment

Shorthand operators that modify and assign in one step:

```
x = 10
x += 5    # x = x + 5 → 15
x -= 3    # x = x - 3 → 12
x *= 2    # x = x * 2 → 24
x /= 4    # x = x / 4 → 6.0 (note: always float after /)
x //= 2   # x = x // 2 → 3.0 (floor division)
x %= 2    # x = x % 2 → 1.0 (modulo)

# For strings, += creates a new string
msg = "Hello"
msg += ", World!" # "Hello, World!"

# For lists, += extends in-place (mutates)
items = [1, 2]
items += [3, 4] # items is now [1, 2, 3, 4]
```

Explain Line by Line Code Description (Block 21)

1. **Line 1:** `x = 10`
Assigns 10 to x.
2. **Line 2:** `x += 5 # x = x + 5 → 15`
Updates x by += 5 # x = x + 5 → 15.
3. **Line 3:** `x -= 3 # x = x - 3 → 12`
Updates x by -= 3 # x = x - 3 → 12.
4. **Line 4:** `x *= 2 # x = x * 2 → 24`
Updates x by *= 2 # x = x * 2 → 24.
5. **Line 5:** `x /= 4 # x = x / 4 → 6.0 (note: always float after /)`
Updates x by /= 4 # x = x / 4 → 6.0 (note: always float after /).
6. **Line 6:** `x //= 2 # x = x // 2 → 3.0 (floor division)`
Updates x by //= 2 # x = x // 2 → 3.0 (floor division).
7. **Line 7:** `x %= 2 # x = x % 2 → 1.0 (modulo)`
Updates x by %= 2 # x = x % 2 → 1.0 (modulo).
8. **Line 8:** `# For strings, += creates a new string`
A comment: *For strings, += creates a new string.*
9. **Line 9:** `msg = "Hello"`
Assigns "Hello" to msg.
10. **Line 10:** `msg += ", World!" # "Hello, World!"`
Updates msg by += ", World!" # "Hello, World!".
11. **Line 11:** `# For lists, += extends in-place (mutates)`
A comment: *For lists, += extends in-place (mutates).*
12. **Line 12:** `items = [1, 2]`
Assigns [1, 2] to items.
13. **Line 13:** `items += [3, 4] # items is now [1, 2, 3, 4]`

```
Updates items by += [3, 4] # items is now [1, 2, 3, 4] .
```

2.11 Unpacking

Python's unpacking syntax makes assignments clean and readable:

```
# Tuple unpacking
x, y, z = 1, 2, 3

# Swap variables (no temp variable needed!)
a, b = 10, 20
a, b = b, a # a=20, b=10

# Star unpacking (Python 3+)
first, *middle, last = [1, 2, 3, 4, 5]
# first=1, middle=[2,3,4], last=5

# Ignoring values with _
name, _, version = "Piper", "en_US", "medium"

# Dict unpacking (Python 3.5+)
defaults = {"rate": 1.0, "pitch": 1.0, "volume": 1.0}
overrides = {"rate": 1.5}
params = {**defaults, **overrides} # {"rate": 1.5, "pitch": 1.0, "volume": 1.0}
```

Explain Line by Line Code Description (Block 22)

1. **Line 1:** `# Tuple unpacking`

A comment: *Tuple unpacking.*

2. **Line 2:** `x, y, z = 1, 2, 3`

`x, y, z = 1, 2, 3`

3. **Line 3:** `# Swap variables (no temp variable needed!)`

A comment: *Swap variables (no temp variable needed!).*

4. **Line 4:** `a, b = 10, 20`

`a, b = 10, 20`

5. **Line 5:** `a, b = b, a # a=20, b=10`

`a, b = b, a # a=20, b=10`

6. **Line 6:** `# Star unpacking (Python 3+)`

A comment: *Star unpacking (Python 3+).*

7. **Line 7:** `first, *middle, last = [1, 2, 3, 4, 5]`

`first, *middle, last = [1, 2, 3, 4, 5]`

8. **Line 8:** `# first=1, middle=[2,3,4], last=5`

A comment: *first=1, middle=[2,3,4], last=5.*

9. **Line 9:** `# Ignoring values with _`

A comment: *Ignoring values with _.*

10. **Line 10:** `name, _, version = "Piper", "en_US", "medium"`

`name, _, version = "Piper", "en_US", "medium"`

11. **Line 11:** `# Dict unpacking (Python 3.5+)`

A comment: *Dict unpacking (Python 3.5+).*

12. **Line 12:** `defaults = {"rate": 1.0, "pitch": 1.0, "volume": 1.0}`

Assigns `{"rate": 1.0, "pitch": 1.0, "volume": 1.0}` to `defaults` .

13. **Line 13:** `overrides = {"rate": 1.5}`

Assigns `{"rate": 1.5}` to `overrides` .

14. Line 14: `params = {**defaults, **overrides} # {"rate": 1.5, "pitch": 1.0, "volume": 1.0}`
Assigns `{**defaults, **overrides} # {"rate": 1.5, "pitch": 1.0, "volume": 1.0}` to `params`.

2.12 Type Hints (Preview)

Python 3.5+ supports optional type annotations. These do not enforce types at runtime but help IDEs, linters, and human readers:

```
# Variable annotations
name: str = "AI Voice Studio"
version: float = 2.0
voices: list[str] = ["af", "am_adam", "bf_emma"]

# Function annotations
def sanitize_filename(name: str, max_length: int = 80) -> str:
    return name[:max_length]

# Optional values
from typing import Optional
engine_id: Optional[str] = None

# Modern Python 3.10+ (use | instead of Optional)
engine_id: str | None = None

# Union types
from typing import Union
value: Union[int, float] = 42
# Python 3.10+: value: int | float = 42
```

Explain Line by Line Code Description (Block 23)

1. **Line 1:** `# Variable annotations`

A comment: *Variable annotations.*

2. **Line 2:** `name: str = "AI Voice Studio"`

`name: str = "AI Voice Studio"`

3. **Line 3:** `version: float = 2.0`

`version: float = 2.0`

4. **Line 4:** `voices: list[str] = ["af", "am_adam", "bf_emma"]`

`voices: list[str] = ["af", "am_adam", "bf_emma"]`

5. **Line 5:** `# Function annotations`

A comment: *Function annotations.*

6. **Line 6:** `def sanitize_filename(name: str, max_length: int = 80) -> str:`

Defines function `sanitize_filename` accepting (name, max_length), returning `str` .

7. **Line 7:** `return name[:max_length]`

Returns `name[:max_length]` to the caller.

8. **Line 8:** `# Optional values`

A comment: *Optional values.*

9. **Line 9:** `from typing import Optional`

Imports `Optional` from `typing` .

10. **Line 10:** `engine_id: Optional[str] = None`

Type hint meaning the value can be T or None. Equivalent to `T | None` in Python 3.10+.

11. **Line 11:** `# Modern Python 3.10+ (use | instead of Optional)`

A comment: *Modern Python 3.10+ (use | instead of Optional).*

12. **Line 12:** `engine_id: str | None = None`

`engine_id: str | None = None`

13. **Line 13:** `# Union types`

A comment: *Union types.*

14. **Line 14:** `from typing import Union`
Imports `Union` from `typing`.
15. **Line 15:** `value: Union[int, float] = 42`
Type hint meaning the value can be one of several types.
16. **Line 16:** `# Python 3.10+: value: int | float = 42`
A comment: *Python 3.10+: value: int | float = 42.*

We will cover type hints in depth in Chapter 22. For now, just know they exist.

2.13 Variables in the AIVS Constants Module

The AI Voice Studio project centralizes all magic numbers and strings in `constants.py`. This is a best practice for larger projects:

```
# From ai_voice_studio/constants.py (simplified)
```

```
# Application metadata
```

```
APP_NAME = "AI Voice Studio"
```

```
APP_VERSION = "2.0.0"
```

```
# Audio parameters
```

```
RATE_MIN = 0.5
```

```
RATE_MAX = 2.0
```

```
PITCH_MIN = 0.5
```

```
PITCH_MAX = 2.0
```

```
VOLUME_MIN = 0.0
```

```
VOLUME_MAX = 2.0
```

```
# Output formats
```

```
FORMAT_WAV = "wav"
```

```
FORMAT_MP3 = "mp3"
```

```
FORMAT_FLAC = "flac"
```

```
OUTPUT_FORMAT_CHOICES = [
```

```
    (FORMAT_WAV, "WAV (always available)"),
```

```
    (FORMAT_MP3, "MP3 (requires FFmpeg)"),
```

```
    (FORMAT_FLAC, "FLAC (requires FFmpeg)"),
```

```
]
```

```
# Themes
```

```
THEME_LIGHT = "light"
```

```
THEME_DARK = "dark"
```

```
THEME_SYSTEM = "system"
```

```
# DAISY book constants
```

```
DAISY_NCC_FILE = "ncc.html"
```

```
DAISY_PACKAGE_FILE = "package.opf"
```

```
DAISY_AUDIO_DIR_NAME = "audio"
```

```
# Punctuation modes
```

```
PUNCTUATION_DEFAULT = "default"
```

```
PUNCTUATION_CHOICES = [
```

```
    ("default", "Default"),
```

```
    ("all", "Read all punctuation"),
```

```
    ("none", "No punctuation"),
```

```
]
```

Explain Line by Line Code Description (Block 24)

1. **Line 1:** `# From ai_voice_studio/constants.py (simplified)`

A comment: *From ai_voice_studio/constants.py (simplified).*

2. **Line 2:** `# Application metadata`

A comment: *Application metadata.*

3. **Line 3:** `APP_NAME = "AI Voice Studio"`

Assigns "AI Voice Studio" to APP_NAME .

4. **Line 4:** `APP_VERSION = "2.0.0"`

Assigns "2.0.0" to APP_VERSION .

5. **Line 5:** `# Audio parameters`

A comment: *Audio parameters.*

6. **Line 6:** `RATE_MIN = 0.5`

Assigns 0.5 to RATE_MIN .

7. **Line 7:** `RATE_MAX = 2.0`

Assigns 2.0 to RATE_MAX .

8. **Line 8:** `PITCH_MIN = 0.5`

Assigns 0.5 to PITCH_MIN .

9. **Line 9:** `PITCH_MAX = 2.0`

Assigns 2.0 to PITCH_MAX .

10. **Line 10:** `VOLUME_MIN = 0.0`

Assigns 0.0 to VOLUME_MIN .

11. **Line 11:** `VOLUME_MAX = 2.0`

Assigns 2.0 to VOLUME_MAX .

12. **Line 12:** `# Output formats`

A comment: *Output formats.*

13. **Line 13:** `FORMAT_WAV = "wav"`

Assigns "wav" to FORMAT_WAV .

14. Line 14: `FORMAT_MP3 = "mp3"`
Assigns `"mp3"` to `FORMAT_MP3` .

15. Line 15: `FORMAT_FLAC = "flac"`
Assigns `"flac"` to `FORMAT_FLAC` .

16. Line 16: `OUTPUT_FORMAT_CHOICES = [`
Assigns `[` to `OUTPUT_FORMAT_CHOICES` .

17. Line 17: `(FORMAT_WAV, "WAV (always available)",`
`(FORMAT_WAV, "WAV (always available)",`

18. Line 18: `(FORMAT_MP3, "MP3 (requires FFmpeg)",`
`(FORMAT_MP3, "MP3 (requires FFmpeg)",`

19. Line 19: `(FORMAT_FLAC, "FLAC (requires FFmpeg)",`
`(FORMAT_FLAC, "FLAC (requires FFmpeg)",`

20. Line 20: `]`
`]`

21. Line 21: `# Themes`
A comment: *Themes*.

22. Line 22: `THEME_LIGHT = "light"`
Assigns `"light"` to `THEME_LIGHT` .

23. Line 23: `THEME_DARK = "dark"`
Assigns `"dark"` to `THEME_DARK` .

24. Line 24: `THEME_SYSTEM = "system"`
Assigns `"system"` to `THEME_SYSTEM` .

25. Line 25: `# DAISY book constants`
A comment: *DAISY book constants*.

26. Line 26: `DAISY_NCC_FILE = "ncc.html"`
Assigns `"ncc.html"` to `DAISY_NCC_FILE` .

27. Line 27: `DAISY_PACKAGE_FILE = "package.opf"`

```

Assigns "package.opf" to DAISY_PACKAGE_FILE .

28. Line 28: DAISY_AUDIO_DIR_NAME = "audio"
Assigns "audio" to DAISY_AUDIO_DIR_NAME .

29. Line 29: # Punctuation modes
A comment: Punctuation modes.

30. Line 30: PUNCTUATION_DEFAULT = "default"
Assigns "default" to PUNCTUATION_DEFAULT .

31. Line 31: PUNCTUATION_CHOICES = [
Assigns [ to PUNCTUATION_CHOICES .

32. Line 32: ("default", "Default"),
("default", "Default"),

33. Line 33: ("all", "Read all punctuation"),
("all", "Read all punctuation"),

34. Line 34: ("none", "No punctuation"),
("none", "No punctuation"),

35. Line 35: ]
]

```

 **Tip: Why constants?** If you use the string `"wav"` in 15 different files and later decide to rename it, you would have to find and replace in all 15 files. With a constant, you change it in one place. Constants also serve as documentation — `FORMAT_WAV` tells you what the value means at a glance.

2.14 Key Takeaways

- **Python is dynamically typed** — variables have no declared type, but every value has a type
- **Five core types:** `int`, `float`, `bool`, `str`, `None`
- **Integers have arbitrary precision** — no overflow
- **Floats have limited precision** — never use `==` for comparison

- **Strings are immutable** — operations create new strings
 - Use `is` for None comparison, `==` for value equality
 - **Constants go in UPPER_SNAKE_CASE** and live in a dedicated module
 - **Type hints** document expected types without enforcing them
 - **F-strings** are the modern way to format strings
-

 **Exercise: Exercise 2.1:** Create a variable for each of the six AIVS TTS engines (Piper, Kokoro, Matcha, Kitten, Bark, OmniVoice) and print them all on one line, separated by commas.

 **Exercise: Exercise 2.2:** Write a function that converts a sample count to a duration string. Example:
`samples_to_duration(48000, 24000) → "2.0 seconds" .`

 **Exercise: Exercise 2.3:** Look at the `constants.py` file in the project. How many constant values are defined? Pick three and explain what they control in the application.

Control Flow

Learning Objectives

- Master `if`, `elif`, `else` branching
 - Use `for` and `while` loops effectively
 - Understand `break`, `continue`, and `else` on loops
 - Apply the Python 3.10+ `match/case` statement
 - See control flow patterns in AI Voice Studio
-

3.1 `if` / `elif` / `else`

The `if` statement executes code based on a boolean condition:

```
# Basic if
has_gpu = True
if has_gpu:
    print("Using CUDA GPU")

# if / else
compute = "cpu"
if compute == "cuda":
    print("GPU mode")
else:
    print("CPU mode")

# if / elif / else (chain multiple conditions)
quality = "medium"
if quality == "low":
    size_mb = 50
elif quality == "medium":
    size_mb = 200
elif quality == "high":
    size_mb = 500
else:
    raise ValueError(f"Unknown quality: {quality}")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Basic if`
A comment: *Basic if*.
2. **Line 2:** `has_gpu = True`
Assigns `True` to `has_gpu`.
3. **Line 3:** `if has_gpu:`
Checks condition: `has_gpu`. The indented block runs only when this is `True`.
4. **Line 4:** `print("Using CUDA GPU")`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `# if / else`
A comment: *if / else*.
6. **Line 6:** `compute = "cpu"`
Assigns `"cpu"` to `compute`.
7. **Line 7:** `if compute == "cuda":`
Checks condition: `compute == "cuda"`. The indented block runs only when this is `True`.
8. **Line 8:** `print("GPU mode")`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `else:`
Fallback: executes when none of the above conditions matched.
10. **Line 10:** `print("CPU mode")`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `# if / elif / else (chain multiple conditions)`
A comment: *if / elif / else (chain multiple conditions)*.
12. **Line 12:** `quality = "medium"`
Assigns `"medium"` to `quality`.
13. **Line 13:** `if quality == "low":`
Checks condition: `quality == "low"`. The indented block runs only when this is `True`.
14. **Line 14:** `size_mb = 50`

Assigns 50 to size_mb .

15. Line 15: elif quality == "medium":

Alternative condition: quality == "medium" . Checked only if all previous conditions were False .

16. Line 16: size_mb = 200

Assigns 200 to size_mb .

17. Line 17: elif quality == "high":

Alternative condition: quality == "high" . Checked only if all previous conditions were False .

18. Line 18: size_mb = 500

Assigns 500 to size_mb .

19. Line 19: else:

Fallback: executes when none of the above conditions matched.

20. Line 20: raise ValueError(f"Unknown quality: {quality}")

Raised when a function receives an argument of the right type but inappropriate value.

Important: No Parentheses or Braces

```
# Python style – NO parentheses around conditions
if x > 5:          # ✓ Correct
    pass

# if (x > 5):      # ✗ Works but un-Pythonic
#     pass

# Indentation defines the block (4 spaces standard)
if True:
    print("line 1")
    print("line 2") # Still inside the if
print("line 3")    # Outside the if
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Python style – NO parentheses around conditions`
A comment: *Python style – NO parentheses around conditions.*
2. **Line 2:** `if x > 5: # ✓ Correct`
Checks condition: `x > 5`. The indented block runs only when this is `True`.
3. **Line 3:** `pass`
Placeholder: does nothing.
4. **Line 4:** `# if (x > 5): # ✗ Works but un-Pythonic`
A comment: *if (x > 5): # ✗ Works but un-Pythonic.*
5. **Line 5:** `# pass`
A comment: *pass.*
6. **Line 6:** `# Indentation defines the block (4 spaces standard)`
A comment: *Indentation defines the block (4 spaces standard).*
7. **Line 7:** `if True:`
Checks condition: `True`. The indented block runs only when this is `True`.
8. **Line 8:** `print("line 1")`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print("line 2") # Still inside the if`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `print("line 3") # Outside the if`
Outputs text to the console. The primary way to display information during development and debugging.

Conditional Expressions (Ternary Operator)

```
# One-line if/else
format_label = "WAV" if fmt == "wav" else fmt.upper()

# Used frequently in AIVS for default values
display_name = voice.get("name") or voice["id"] # fallback

# Nested ternary (avoid this – hard to read)
label = "A" if x < 10 else "B" if x < 100 else "C"
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# One-line if/else`
A comment: *One-line if/else*.
2. **Line 2:** `format_label = "WAV" if fmt == "wav" else fmt.upper()`
Converts all characters to uppercase. Returns a new string (strings are immutable).
3. **Line 3:** `# Used frequently in AIVS for default values`
A comment: *Used frequently in AIVS for default values*.
4. **Line 4:** `display_name = voice.get("name") or voice["id"] # fallback`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
5. **Line 5:** `# Nested ternary (avoid this – hard to read)`
A comment: *Nested ternary (avoid this – hard to read)*.
6. **Line 6:** `label = "A" if x < 10 else "B" if x < 100 else "C"`
Assigns `"A" if x < 10 else "B" if x < 100 else "C"` to `label`.

3.2 for Loops

Python's `for` loop iterates over any **iterable** — lists, strings, tuples, dicts, sets, ranges, files, generators, and more:

```
# Iterate over a list
engines = ["Piper", "Kokoro", "Matcha"]
for engine in engines:
    print(f>Loading {engine}...")

# Iterate over a string
for char in "hello":
    print(char, end=" ") # h e l l o

# range() - generate number sequences
for i in range(5):      # 0, 1, 2, 3, 4
    print(i)

for i in range(2, 10, 3): # start, stop, step: 2, 5, 8
    print(i)

# Iterate with index using enumerate()
for idx, engine in enumerate(engines):
    print(f"{idx}: {engine}")
# 0: Piper
# 1: Kokoro
# 2: Matcha
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# Iterate over a list`
A comment: *Iterate over a list.*
2. **Line 2:** `engines = ["Piper", "Kokoro", "Matcha"]`
Assigns `["Piper", "Kokoro", "Matcha"]` to `engines`.
3. **Line 3:** `for engine in engines:`
Iterates over `engines`, assigning each element to `engine`.
4. **Line 4:** `print(f>Loading {engine}...")`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `# Iterate over a string`
A comment: *Iterate over a string.*
6. **Line 6:** `for char in "hello":`
Iterates over `"hello"`, assigning each element to `char`.
7. **Line 7:** `print(char, end=" ") # h e l l o`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `# range() – generate number sequences`
A comment: *range() – generate number sequences.*
9. **Line 9:** `for i in range(5): # 0, 1, 2, 3, 4`
Generates a sequence of integers. Commonly used in for loops.
10. **Line 10:** `print(i)`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `for i in range(2, 10, 3): # start, stop, step: 2, 5, 8`
Generates a sequence of integers. Commonly used in for loops.
12. **Line 12:** `print(i)`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `# Iterate with index using enumerate()`
A comment: *Iterate with index using enumerate().*
14. **Line 14:** `for idx, engine in enumerate(engines):`

Adds a counter to an iterable, returning (index, value) pairs. Preferred over manual index tracking.

15. **Line 15:** `print(f"{idx}: {engine}")`

Outputs text to the console. The primary way to display information during development and debugging.

16. **Line 16:** `# 0: Piper`

A comment: *0: Piper*.

17. **Line 17:** `# 1: Kokoro`

A comment: *1: Kokoro*.

18. **Line 18:** `# 2: Matcha`

A comment: *2: Matcha*.

Iterating Over Dictionaries

```
voices = {
    "piper": "English",
    "kokoro": "Multilingual",
    "matcha": "English",
}

# Iterate over keys (default)
for tts_id in voices:
    print(tts_id)

# Iterate over key-value pairs (most common)
for tts_id, language in voices.items():
    print(f"{tts_id} → {language}")

# Iterate over values only
for lang in voices.values():
    print(lang)
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `voices = {`
Assigns `{` to `voices`.
2. **Line 2:** `"piper": "English",`
`"piper": "English",`
3. **Line 3:** `"kokoro": "Multilingual",`
`"kokoro": "Multilingual",`
4. **Line 4:** `"matcha": "English",`
`"matcha": "English",`
5. **Line 5:** `}`
`}`
6. **Line 6:** `# Iterate over keys (default)`
A comment: *Iterate over keys (default).*
7. **Line 7:** `for tts_id in voices:`
Iterates over `voices`, assigning each element to `tts_id`.
8. **Line 8:** `print(tts_id)`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `# Iterate over key-value pairs (most common)`
A comment: *Iterate over key-value pairs (most common).*
10. **Line 10:** `for tts_id, language in voices.items():`
Returns a view of the dictionary's (key, value) pairs.
11. **Line 11:** `print(f"{tts_id} → {language}")`
Outputs text to the console. The primary way to display information during development and debugging.
12. **Line 12:** `# Iterate over values only`
A comment: *Iterate over values only.*
13. **Line 13:** `for lang in voices.values():`
Returns a view of the dictionary's values.
14. **Line 14:** `print(lang)`

Outputs text to the console. The primary way to display information during development and debugging.

Loop Patterns in AI Voice Studio

```
# From tts/catalog.py - iterating through nested catalog structure
for tts in catalog.get_tts_list():
    for lang in tts.get("languages", []):
        for variant in lang.get("variants", []):
            for voice in variant.get("voices", []):
                print(f"{tts['id']} / {lang['code']} / {voice['id']}")

# From tts/models.py - iterating installed voices to build a list
voices = []
for tts in catalog.get_tts_list():
    for lang in tts.get("languages", []):
        for variant in lang.get("variants", []):
            for voice in variant.get("voices", []):
                # Build a flat list of voice entries
                voices.append({
                    "tts": tts["id"],
                    "language": lang["code"],
                    "voice": voice["id"],
                    "sid": voice.get("sid", 0),
                })
```


Explain Line by Line Code Description (Block 6)

1. **Line 1:** `# From tts/catalog.py – iterating through nested catalog structure`
A comment: *From tts/catalog.py – iterating through nested catalog structure.*
2. **Line 2:** `for tts in catalog.get_tts_list():`
Iterates over `catalog.get_tts_list()`, assigning each element to `tts`.
3. **Line 3:** `for lang in tts.get("languages", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
4. **Line 4:** `for variant in lang.get("variants", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
5. **Line 5:** `for voice in variant.get("voices", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
6. **Line 6:** `print(f"{tts['id']} / {lang['code']} / {voice['id']}")`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# From tts/models.py – iterating installed voices to build a list`
A comment: *From tts/models.py – iterating installed voices to build a list.*
8. **Line 8:** `voices = []`
Assigns `[]` to `voices`.
9. **Line 9:** `for tts in catalog.get_tts_list():`
Iterates over `catalog.get_tts_list()`, assigning each element to `tts`.
10. **Line 10:** `for lang in tts.get("languages", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
11. **Line 11:** `for variant in lang.get("variants", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
12. **Line 12:** `for voice in variant.get("voices", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
13. **Line 13:** `# Build a flat list of voice entries`
A comment: *Build a flat list of voice entries.*
14. **Line 14:** `voices.append({`

Adds a single item to the end of the list.

15. **Line 15:** `"tts": tts["id"],`
`"tts": tts["id"],`

16. **Line 16:** `"language": lang["code"],`
`"language": lang["code"],`

17. **Line 17:** `"voice": voice["id"],`
`"voice": voice["id"],`

18. **Line 18:** `"sid": voice.get("sid", 0),`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

19. **Line 19:** `})`
`})`

3.3 while Loops

`while` loops repeat as long as a condition is true:

```
# Basic while loop
countdown = 5
while countdown > 0:
    print(f"{countdown}...")
    countdown -= 1
print("Go!")

# Reading input until a sentinel value
while True:
    line = input("Enter command (or 'quit'): ")
    if line == "quit":
        break
    print(f"Executing: {line}")

# Polling pattern (used in AIVS server health checks)
import time
timeout = 30
deadline = time.monotonic() + timeout
while time.monotonic() < deadline:
    if health_check():
        print("Server is ready!")
        break
    time.sleep(1.0)
else:
    print("Server did not start in time")
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `# Basic while loop`

A comment: *Basic while loop.*

2. **Line 2:** `countdown = 5`

Assigns `5` to `countdown`.

3. **Line 3:** `while countdown > 0:`

While loop: repeats as long as `countdown > 0`.

4. **Line 4:** `print(f"{countdown}...")`

Outputs text to the console. The primary way to display information during development and debugging.

5. **Line 5:** `countdown -= 1`

Updates `countdown` by `-= 1`.

6. **Line 6:** `print("Go!")`

Outputs text to the console. The primary way to display information during development and debugging.

7. **Line 7:** `# Reading input until a sentinel value`

A comment: *Reading input until a sentinel value.*

8. **Line 8:** `while True:`

While loop: repeats as long as `True`.

9. **Line 9:** `line = input("Enter command (or 'quit'): ")`

Reads a line of text from the user via the console. Always returns a string.

10. **Line 10:** `if line == "quit":`

Checks condition: `line == "quit"`. The indented block runs only when this is `True`.

11. **Line 11:** `break`

Exits the innermost loop.

12. **Line 12:** `print(f"Executing: {line}")`

Outputs text to the console. The primary way to display information during development and debugging.

13. **Line 13:** `# Polling pattern (used in AIVS server health checks)`

A comment: *Polling pattern (used in AIVS server health checks).*

14. **Line 14:** `import time`

Imports the `time` module.

15. **Line 15:** `timeout = 30`

Assigns `30` to `timeout`.

16. **Line 16:** `deadline = time.monotonic() + timeout`

Assigns `time.monotonic() + timeout` to `deadline`.

17. **Line 17:** `while time.monotonic() < deadline:`

While loop: repeats as long as `time.monotonic() < deadline`.

18. **Line 18:** `if health_check():`

Checks condition: `health_check()`. The indented block runs only when this is `True`.

19. **Line 19:** `print("Server is ready!")`

Outputs text to the console. The primary way to display information during development and debugging.

20. **Line 20:** `break`

Exits the innermost loop.

21. **Line 21:** `time.sleep(1.0)`

`time.sleep(1.0)`

22. **Line 22:** `else:`

Fallback: executes when none of the above conditions matched.

23. **Line 23:** `print("Server did not start in time")`

Outputs text to the console. The primary way to display information during development and debugging.

The while-else Pattern

When a `while` loop finishes normally (condition becomes false), the `else` block runs. If the loop exits via `break`, the `else` is skipped:

```
# From omnivoice_server/__init__.py - health check polling
while time.monotonic() < deadline:
    try:
        with urllib.request.urlopen(url, timeout=5) as resp:
            if resp.status == 200:
                self._ready.set()
                return # Success - else block is skipped
    except (urllib.error.URLError, OSError):
        pass
    time.sleep(1.0)
raise OmniVoiceServerError(
    f"Server did not become ready within {timeout}"
)
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# From omnivoice_server/__init__.py - health check polling`
A comment: *From omnivoice_server/__init__.py - health check polling.*
2. **Line 2:** `while time.monotonic() < deadline:`
While loop: repeats as long as `time.monotonic() < deadline`.
3. **Line 3:** `try:`
Begins exception handling. Code in this block is monitored for errors.
4. **Line 4:** `with urllib.request.urlopen(url, timeout=5) as resp:`
Opens a file and returns a file object. Supports modes: 'r' (read), 'w' (write), 'a' (append), 'b' (binary).
5. **Line 5:** `if resp.status == 200:`
Checks condition: `resp.status == 200`. The indented block runs only when this is `True`.
6. **Line 6:** `self._ready.set()`
`self._ready.set()`
7. **Line 7:** `return # Success - else block is skipped`
Returns `# Success - else block is skipped` to the caller.
8. **Line 8:** `except (urllib.error.URLError, OSError):`
`except (urllib.error.URLError, OSError):`
9. **Line 9:** `pass`
Placeholder: does nothing.
10. **Line 10:** `time.sleep(1.0)`
`time.sleep(1.0)`
11. **Line 11:** `raise OmniVoiceServerError(`
Manually raises an exception. Use to signal error conditions to callers.
12. **Line 12:** `f"Server did not become ready within {timeout}"`
`f"Server did not become ready within {timeout}"`
13. **Line 13:** `)`
`)`

3.4 break, continue, and pass

```
# break – exit the loop immediately
for engine in ["piper", "kokoro", "matcha"]:
    if engine_is_available(engine):
        chosen = engine
        break # Stop searching once we find one
else:
    raise RuntimeError("No TTS engine available!")

# continue – skip to the next iteration
for voice in all_voices:
    if not voice.get("enabled", True):
        continue # Skip disabled voices
    print(voice["name"])

# pass – do nothing (placeholder)
class NotImplementedEngine:
    pass # Will implement later

# pass is used when syntax requires a block
for i in range(10):
    if i % 2 == 0:
        pass # TODO: handle even numbers later
    else:
        print(i)
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `# break – exit the loop immediately`
A comment: *break – exit the loop immediately.*
2. **Line 2:** `for engine in ["piper", "kokoro", "matcha"]:`
Iterates over `["piper", "kokoro", "matcha"]`, assigning each element to `engine`.
3. **Line 3:** `if engine_is_available(engine):`
Checks condition: `engine_is_available(engine)`. The indented block runs only when this is `True`.
4. **Line 4:** `chosen = engine`
Assigns `engine` to `chosen`.
5. **Line 5:** `break # Stop searching once we find one`
`break # Stop searching once we find one`
6. **Line 6:** `else:`
Fallback: executes when none of the above conditions matched.
7. **Line 7:** `raise RuntimeError("No TTS engine available!")`
Raised when an error doesn't fall into any other category.
8. **Line 8:** `# continue – skip to the next iteration`
A comment: *continue – skip to the next iteration.*
9. **Line 9:** `for voice in all_voices:`
Iterates over `all_voices`, assigning each element to `voice`.
10. **Line 10:** `if not voice.get("enabled", True):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
11. **Line 11:** `continue # Skip disabled voices`
`continue # Skip disabled voices`
12. **Line 12:** `print(voice["name"])`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `# pass – do nothing (placeholder)`
A comment: *pass – do nothing (placeholder).*
14. **Line 14:** `class NotImplementedEngine:`

Defines class `NotImplementedEngine` .

15. **Line 15:** `pass # Will implement later`

`pass # Will implement later`

16. **Line 16:** `# pass is used when syntax requires a block`

A comment: *pass is used when syntax requires a block.*

17. **Line 17:** `for i in range(10):`

Generates a sequence of integers. Commonly used in for loops.

18. **Line 18:** `if i % 2 == 0:`

Checks condition: `i % 2 == 0` . The indented block runs only when this is `True` .

19. **Line 19:** `pass # TODO: handle even numbers later`

`pass # TODO: handle even numbers later`

20. **Line 20:** `else:`

Fallback: executes when none of the above conditions matched.

21. **Line 21:** `print(i)`

Outputs text to the console. The primary way to display information during development and debugging.

3.5 List Comprehensions

List comprehensions are a concise way to create lists. They combine a loop and an expression into a single line:

```
# Traditional Loop
squares = []
for x in range(10):
    squares.append(x ** 2)

# List comprehension (equivalent, more Pythonic)
squares = [x ** 2 for x in range(10)]
# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]

# With filtering
even_squares = [x ** 2 for x in range(10) if x % 2 == 0]
# [0, 4, 16, 36, 64]

# Dict comprehension
engine_names = {t["id"]: t["name"] for t in tts_list}
# {"piper": "Piper TTS", "kokoro": "Kokoro TTS", ...}

# Set comprehension
languages = {v["language"] for v in installed_voices}
# {"en", "zh", "ja", "multi", ...}

# Generator expression (lazy - doesn't create list in memory)
total = sum(len(v["text"]) for v in segments)
```

Explain Line by Line Code Description (Block 10)

1. **Line 1:** `# Traditional loop`

A comment: *Traditional loop.*

2. **Line 2:** `squares = []`

Assigns `[]` to `squares`.

3. **Line 3:** `for x in range(10):`

Generates a sequence of integers. Commonly used in for loops.

4. **Line 4:** `squares.append(x ** 2)`

Adds a single item to the end of the list.

5. **Line 5:** `# List comprehension (equivalent, more Pythonic)`

A comment: *List comprehension (equivalent, more Pythonic).*

6. **Line 6:** `squares = [x ** 2 for x in range(10)]`

Generates a sequence of integers. Commonly used in for loops.

7. **Line 7:** `# [0, 1, 4, 9, 16, 25, 36, 49, 64, 81]`

A comment: *[0, 1, 4, 9, 16, 25, 36, 49, 64, 81].*

8. **Line 8:** `# With filtering`

A comment: *With filtering.*

9. **Line 9:** `even_squares = [x ** 2 for x in range(10) if x % 2 == 0]`

Generates a sequence of integers. Commonly used in for loops.

10. **Line 10:** `# [0, 4, 16, 36, 64]`

A comment: *[0, 4, 16, 36, 64].*

11. **Line 11:** `# Dict comprehension`

A comment: *Dict comprehension.*

12. **Line 12:** `engine_names = {t["id"]: t["name"] for t in tts_list}`

Assigns `{t["id"]: t["name"] for t in tts_list}` to `engine_names`.

13. **Line 13:** `# {"piper": "Piper TTS", "kokoro": "Kokoro TTS", ...}`

A comment: *{"piper": "Piper TTS", "kokoro": "Kokoro TTS", ...}.*

14. **Line 14:** `# Set comprehension`

A comment: *Set comprehension.*

15. **Line 15:** `languages = {v["language"] for v in installed_voices}`
Assigns `{v["language"] for v in installed_voices}` to `languages` .
16. **Line 16:** `# {"en", "zh", "ja", "multi", ...}`
A comment: `{"en", "zh", "ja", "multi", ...}`.
17. **Line 17:** `# Generator expression (lazy – doesn't create list in memory)`
A comment: *Generator expression (lazy – doesn't create list in memory)*.
18. **Line 18:** `total = sum(len(v["text"]) for v in segments)`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

Comprehensions in AIVS

```
# From tts/catalog.py
def get_tts_list():
    """Return all TTS engines from the catalog."""
    return list(_CATALOG) # Convert tuple to List

# From gui/recording_dialog.py
tts_ids = sorted({v["tts"] for v in self._voices})
# Set comprehension inside sorted() – collect unique TTS IDs

# From tts/models.py – filtering installed variants
missing = [
    v for v in variants
    if not self.store.is_variant_installed(tts_id, lang_code, v)
]
# List comprehension with negated condition
```

Explain Line by Line Code Description (Block 11)

1. **Line 1:** `# From tts/catalog.py`

A comment: *From tts/catalog.py.*

2. **Line 2:** `def get_tts_list():`

Defines function `get_tts_list` .

3. **Line 3:** `"""Return all TTS engines from the catalog."""`

Docstring: *Return all TTS engines from the catalog..* Documents the purpose of the following code.

4. **Line 4:** `return list(_CATALOG) # Convert tuple to list`

Returns `list(_CATALOG) # Convert tuple to list` to the caller.

5. **Line 5:** `# From gui/recording_dialog.py`

A comment: *From gui/recording_dialog.py.*

6. **Line 6:** `tts_ids = sorted({v["tts"] for v in self._voices})`

Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.

7. **Line 7:** `# Set comprehension inside sorted() – collect unique TTS IDs`

A comment: *Set comprehension inside sorted() – collect unique TTS IDs.*

8. **Line 8:** `# From tts/models.py – filtering installed variants`

A comment: *From tts/models.py – filtering installed variants.*

9. **Line 9:** `missing = [`

Assigns `[` to `missing` .

10. **Line 10:** `v for v in variants`

`v for v in variants`

11. **Line 11:** `if not self.store.is_variant_installed(tts_id, lang_code, v)`

`if not self.store.is_variant_installed(tts_id, lang_code, v)`

12. **Line 12:** `]`

`]`

13. **Line 13:** `# List comprehension with negated condition`

A comment: *List comprehension with negated condition.*

3.6 match/case (Python 3.10+)

Python 3.10 introduced `match/case`, similar to `switch/case` in other languages but much more powerful:

```
# Basic match/case
command = "synthesize"

match command:
    case "ping":
        print("Pong!")
    case "synthesize":
        print("Generating audio...")
    case "quit":
        print("Goodbye!")
    case _:
        print(f"Unknown command: {command}") # _ is wildcard

# Matching with capture variables
match request:
    case {"cmd": "synthesize", "text": text}:
        process_text(text)
    case {"cmd": "ping"}:
        respond_ok()
    case _:
        respond_error("Unknown command")

# Matching with guards (extra conditions)
match quality:
    case q if q < 50:
        print("Low quality")
    case q if q < 200:
        print("Medium quality")
    case _:
        print("High quality")
```

Explain Line by Line Code Description (Block 12)

1. **Line 1:** `# Basic match/case`

A comment: *Basic match/case.*

2. **Line 2:** `command = "synthesize"`

Assigns "synthesize" to `command`.

3. **Line 3:** `match command:`

`match command:`

4. **Line 4:** `case "ping":`

`case "ping":`

5. **Line 5:** `print("Pong!")`

Outputs text to the console. The primary way to display information during development and debugging.

6. **Line 6:** `case "synthesize":`

`case "synthesize":`

7. **Line 7:** `print("Generating audio...")`

Outputs text to the console. The primary way to display information during development and debugging.

8. **Line 8:** `case "quit":`

`case "quit":`

9. **Line 9:** `print("Goodbye!")`

Outputs text to the console. The primary way to display information during development and debugging.

10. **Line 10:** `case _:`

`case _:`

11. **Line 11:** `print(f"Unknown command: {command}") # _ is wildcard`

Outputs text to the console. The primary way to display information during development and debugging.

12. **Line 12:** `# Matching with capture variables`

A comment: *Matching with capture variables.*

13. **Line 13:** `match request:`

`match request:`

14. **Line 14:** `case {"cmd": "synthesize", "text": text}:`
`case {"cmd": "synthesize", "text": text}:`
15. **Line 15:** `process_text(text)`
Calls function `process_text()` .
16. **Line 16:** `case {"cmd": "ping"}:`
`case {"cmd": "ping"}:`
17. **Line 17:** `respond_ok()`
Calls function `respond_ok()` .
18. **Line 18:** `case _:`
`case _:`
19. **Line 19:** `respond_error("Unknown command")`
Calls function `respond_error()` .
20. **Line 20:** `# Matching with guards (extra conditions)`
A comment: *Matching with guards (extra conditions).*
21. **Line 21:** `match quality:`
`match quality:`
22. **Line 22:** `case q if q < 50:`
`case q if q < 50:`
23. **Line 23:** `print("Low quality")`
Outputs text to the console. The primary way to display information during development and debugging.
24. **Line 24:** `case q if q < 200:`
`case q if q < 200:`
25. **Line 25:** `print("Medium quality")`
Outputs text to the console. The primary way to display information during development and debugging.
26. **Line 26:** `case _:`
`case _:`
27. **Line 27:** `print("High quality")`
Outputs text to the console. The primary way to display information during development and debugging.

 **Note: Note:** AI Voice Studio uses traditional `if/elif/else` chains rather than `match/case` because it must support Python 3.9+. When Python 3.10+ is guaranteed, `match/case` with dictionary patterns is ideal for processing JSON commands like the TTS worker protocol.

3.7 Nested Loops and Loop else

```
# Nested Loops – common in catalog traversal
for tts in tts_list:
    for lang in tts.get("languages", []):
        for variant in lang.get("variants", []):
            if variant["id"] == target:
                print(f"Found: {tts['id']} / {lang['code']} / {target}")
                break
        else:
            continue # Only reached if inner loop didn't break
    break # Break outer loop too
else:
    print("Not found")
```

Explain Line by Line Code Description (Block 13)

1. **Line 1:** `# Nested loops – common in catalog traversal`

A comment: *Nested loops – common in catalog traversal*.

2. **Line 2:** `for tts in tts_list:`

Iterates over `tts_list`, assigning each element to `tts`.

3. **Line 3:** `for lang in tts.get("languages", []):`

Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

4. **Line 4:** `for variant in lang.get("variants", []):`

Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

5. **Line 5:** `if variant["id"] == target:`

Checks condition: `variant["id"] == target`. The indented block runs only when this is `True`.

6. **Line 6:** `print(f"Found: {tts['id']} / {lang['code']} / {target}")`

Outputs text to the console. The primary way to display information during development and debugging.

7. **Line 7:** `break`

Exits the innermost loop.

8. **Line 8:** `else:`

Fallback: executes when none of the above conditions matched.

9. **Line 9:** `continue # Only reached if inner loop didn't break`

`continue # Only reached if inner loop didn't break`

10. **Line 10:** `break # Break outer loop too`

`break # Break outer loop too`

11. **Line 11:** `else:`

Fallback: executes when none of the above conditions matched.

12. **Line 12:** `print("Not found")`

Outputs text to the console. The primary way to display information during development and debugging.

For-Else Pattern

```
# The "else" after a for loop runs only if the loop completes without break
for file in candidates:
    if os.path.isfile(file):
        print(f"Using: {file}")
        break
else:
    raise FileNotFoundError("No suitable file found")
```

Explain Line by Line Code Description (Block 14)

- Line 1:** `# The "else" after a for loop runs only if the loop completes without break`
A comment: *The "else" after a for loop runs only if the loop completes without break.*
- Line 2:** `for file in candidates:`
Iterates over `candidates`, assigning each element to `file`.
- Line 3:** `if os.path.isfile(file):`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
- Line 4:** `print(f"Using: {file}")`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 5:** `break`
Exits the innermost loop.
- Line 6:** `else:`
Fallback: executes when none of the above conditions matched.
- Line 7:** `raise FileNotFoundError("No suitable file found")`
Raised when a file or directory is requested but doesn't exist.

3.8 Ternary Expressions in Comprehensions

```
# Map-and-filter in one expression
# Get all voice names, or "unnamed" if name is missing
names = [v.get("name", "unnamed") for v in voices]

# Conditional transformation
labels = [
    f"✅ {v['name']}" if v.get("installed") else f"❌ {v['name']}"
    for v in all_voices
]
```

Explain Line by Line Code Description (Block 15)

- Line 1:** `# Map-and-filter in one expression`
A comment: *Map-and-filter in one expression.*
- Line 2:** `# Get all voice names, or "unnamed" if name is missing`
A comment: *Get all voice names, or "unnamed" if name is missing.*
- Line 3:** `names = [v.get("name", "unnamed") for v in voices]`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
- Line 4:** `# Conditional transformation`
A comment: *Conditional transformation.*
- Line 5:** `labels = [`
Assigns `[` to `labels`.
- Line 6:** `f"✅ {v['name']}" if v.get("installed") else f"❌ {v['name']}"`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
- Line 7:** `for v in all_voices`
`for v in all_voices`
- Line 8:** `]`
`]`

3.9 Key Takeaways

- **Indentation** defines blocks — use 4 spaces consistently
 - `for` iterates over any iterable; use `enumerate()` for index + value
 - `while` repeats while a condition is true
 - `break` exits the innermost loop; `continue` skips to next iteration
 - **Loop** `else` runs when the loop completes without `break`
 - **List comprehensions** are concise and often faster than equivalent loops
 - `match/case` (3.10+) is powerful for pattern matching on dicts and sequences
 - The AIVS catalog uses **nested for loops** to traverse TTS → language → variant → voice hierarchies
-

 **Exercise: Exercise 3.1:** Write a list comprehension that generates all possible TTS engine / language combinations from a nested list structure.

 **Exercise: Exercise 3.2:** Rewrite the health-check polling loop from Section 3.3 as a function `wait_for_server(url, timeout)` that returns `True` if the server started, `False` on timeout.

Functions

Learning Objectives

- Define and call functions with parameters and return values
 - Master default arguments, *args, and **kwargs
 - Understand scope (LEGB rule) and closures
 - Use lambda functions for short throwaway operations
 - See function patterns used throughout AI Voice Studio
-

4.1 Defining Functions

```
# Basic function definition
def greet(name: str) -> str:
    """Return a greeting for the given name.""" # Docstring
    return f"Hello, {name}!"

# Call the function
message = greet("Developer")
print(message) # Hello, Developer!

# Functions without a return statement return None implicitly
def log_event(event: str):
    print(f"[LOG] {event}")
    # No return -> returns None

result = log_event("app started")
print(result) # None
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Basic function definition`
A comment: *Basic function definition.*
2. **Line 2:** `def greet(name: str) -> str:`
Defines function `greet` accepting (name), returning `str`.
3. **Line 3:** `"""Return a greeting for the given name.""" # Docstring`
Docstring: *Return a greeting for the given name.* `""" # Docstring.` Documents the purpose of the following code.
4. **Line 4:** `return f"Hello, {name}!"`
Returns `f"Hello, {name}!"` to the caller.
5. **Line 5:** `# Call the function`
A comment: *Call the function.*
6. **Line 6:** `message = greet("Developer")`
Assigns `greet("Developer")` to `message`.
7. **Line 7:** `print(message) # Hello, Developer!`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `# Functions without a return statement return None implicitly`
A comment: *Functions without a return statement return None implicitly.*
9. **Line 9:** `def log_event(event: str):`
Defines function `log_event` accepting (event).
10. **Line 10:** `print(f"[LOG] {event}")`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `# No return → returns None`
A comment: *No return → returns None.*
12. **Line 12:** `result = log_event("app started")`
Assigns `log_event("app started")` to `result`.
13. **Line 13:** `print(result) # None`
Outputs text to the console. The primary way to display information during development and debugging.

Function Anatomy

Component	Purpose
<code>def</code>	Keyword that starts the definition
Function name	snake_case identifier
Parameters	Inputs in parentheses
Return type hint	<code>-> str</code> documents the return type
Docstring	Triple-quoted string explaining what it does
Body	Indented block of code
<code>return</code>	Sends a value back to the caller

4.2 Parameters and Arguments

Positional Parameters

```
def create_audio_file(text: str, output_path: str, sample_rate: int):  
    """Create an audio file from text."""  
    # ... synthesize and save ...  
    print(f"Saved {len(text)} chars to {output_path}")  
  
# Arguments must match parameter order  
create_audio_file("Hello world", "/tmp/out.wav", 24000)
```


Explain Line by Line Code Description (Block 2)

- Line 1:** `def create_audio_file(text: str, output_path: str, sample_rate: int):`
Defines function `create_audio_file` accepting (text, output_path, sample_rate).
- Line 2:** `"""Create an audio file from text."""`
Docstring: *Create an audio file from text..* Documents the purpose of the following code.
- Line 3:** `# ... synthesize and save ...`
A comment: ... *synthesize and save*
- Line 4:** `print(f"Saved {len(text)} chars to {output_path}")`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 5:** `# Arguments must match parameter order`
A comment: *Arguments must match parameter order.*
- Line 6:** `create_audio_file("Hello world", "/tmp/out.wav", 24000)`
Calls function `create_audio_file()` .

Keyword Arguments

```
# Keyword arguments can be in any order
create_audio_file(
    text="Hello world",
    sample_rate=24000,
    output_path="/tmp/out.wav",
)

# Mixing positional and keyword (positional must come first)
create_audio_file("Hello world", output_path="/tmp/out.wav", sample_rate=24000)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Keyword arguments can be in any order`
A comment: *Keyword arguments can be in any order.*
2. **Line 2:** `create_audio_file(`
Calls function `create_audio_file()` .
3. **Line 3:** `text="Hello world",`
Assigns `"Hello world",` to `text` .
4. **Line 4:** `sample_rate=24000,`
Assigns `24000,` to `sample_rate` .
5. **Line 5:** `output_path="/tmp/out.wav",`
Assigns `"/tmp/out.wav",` to `output_path` .
6. **Line 6:** `)`
`)`
7. **Line 7:** `# Mixing positional and keyword (positional must come first)`
A comment: *Mixing positional and keyword (positional must come first).*
8. **Line 8:** `create_audio_file("Hello world", output_path="/tmp/out.wav", sample_rate=24000)`
Calls function `create_audio_file()` .

Default Parameter Values

```
def synthesize(  
    text: str,  
    voice: str = "default",  
    rate: float = 1.0,  
    pitch: float = 1.0,  
    volume: float = 1.0,  
    ) -> "np.ndarray":  
    """Synthesize text with configurable voice parameters."""  
    # ...  
    pass  
  
# Call with defaults  
audio = synthesize("Hello")  
  
# Override some defaults  
audio = synthesize("Hello", rate=1.5, pitch=0.8)  
  
# Override all  
audio = synthesize("Hello", voice="af_bella", rate=1.2, pitch=1.0, volume=0.8)
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `def synthesize(`
`def synthesize(`
2. **Line 2:** `text: str,`
`text: str,`
3. **Line 3:** `voice: str = "default",`
`voice: str = "default",`
4. **Line 4:** `rate: float = 1.0,`
`rate: float = 1.0,`
5. **Line 5:** `pitch: float = 1.0,`
`pitch: float = 1.0,`
6. **Line 6:** `volume: float = 1.0,`
`volume: float = 1.0,`
7. **Line 7:** `) -> "np.ndarray":`
`) -> "np.ndarray":`
8. **Line 8:** `"""Synthesize text with configurable voice parameters."""`
Docstring: *Synthesize text with configurable voice parameters..* Documents the purpose of the following code.
9. **Line 9:** `# ...`
A comment:
10. **Line 10:** `pass`
Placeholder: does nothing.
11. **Line 11:** `# Call with defaults`
A comment: *Call with defaults.*
12. **Line 12:** `audio = synthesize("Hello")`
Assigns `synthesize("Hello")` to `audio` .
13. **Line 13:** `# Override some defaults`
A comment: *Override some defaults.*

14. **Line 14:** `audio = synthesize("Hello", rate=1.5, pitch=0.8)`

Assigns `synthesize("Hello", rate=1.5, pitch=0.8)` to `audio` .

15. **Line 15:** `# Override all`

A comment: *Override all.*

16. **Line 16:** `audio = synthesize("Hello", voice="af_bella", rate=1.2, pitch=1.0, volume=0.8)`

Assigns `synthesize("Hello", voice="af_bella", rate=1.2, pitch=1.0, volume=0.8)` to `audio` .

⚠ **Warning:** Never use mutable objects (lists, dicts) as default parameters! They are created once at function definition time and shared across all calls:

```
# WRONG - shared mutable default!
def add_voice(voice, voices=[]):
    voices.append(voice)
    return voices

# RIGHT - create a new list each call
def add_voice(voice, voices=None):
    if voices is None:
        voices = []
    voices.append(voice)
    return voices
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# WRONG – shared mutable default!`
A comment: *WRONG – shared mutable default!*.
2. **Line 2:** `def add_voice(voice, voices=[]):`
Defines function `add_voice` accepting (voice, voices).
3. **Line 3:** `voices.append(voice)`
Adds a single item to the end of the list.
4. **Line 4:** `return voices`
Returns `voices` to the caller.
5. **Line 5:** `# RIGHT – create a new list each call`
A comment: *RIGHT – create a new list each call*.
6. **Line 6:** `def add_voice(voice, voices=None):`
Defines function `add_voice` accepting (voice, voices).
7. **Line 7:** `if voices is None:`
Checks condition: `voices is None`. The indented block runs only when this is `True`.
8. **Line 8:** `voices = []`
Assigns `[]` to `voices`.
9. **Line 9:** `voices.append(voice)`
Adds a single item to the end of the list.
10. **Line 10:** `return voices`
Returns `voices` to the caller.

Arbitrary Arguments (*args and **kwargs)

```
# *args collects extra positional arguments into a tuple
def log_all(*messages: str):
    for msg in messages:
        print(f"[LOG] {msg}")

log_all("Starting", "Loading models", "Ready")

# **kwargs collects extra keyword arguments into a dict
def create_engine(engine_id: str, **options):
    print(f"Engine: {engine_id}")
    for key, value in options.items():
        print(f"  {key} = {value}")

create_engine("piper", device="cpu", threads=4, verbose=True)
# Engine: piper
#   device = cpu
#   threads = 4
#   verbose = True

# Combining positional, keyword, *args, **kwargs
def configure(name: str, *engines, debug=False, **settings):
    pass

configure("AIVS", "piper", "kokoro", debug=True, rate=1.5)
```


Explain Line by Line Code Description (Block 6)

1. **Line 1:** `# *args collects extra positional arguments into a tuple`
A comment: **args collects extra positional arguments into a tuple.*
2. **Line 2:** `def log_all(*messages: str):`
Returns True if all elements in the iterable are truthy. Short-circuits on first False.
3. **Line 3:** `for msg in messages:`
Iterates over `messages`, assigning each element to `msg`.
4. **Line 4:** `print(f"[LOG] {msg}")`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `log_all("Starting", "Loading models", "Ready")`
Returns True if all elements in the iterable are truthy. Short-circuits on first False.
6. **Line 6:** `# **kwargs collects extra keyword arguments into a dict`
A comment: ***kwargs collects extra keyword arguments into a dict.*
7. **Line 7:** `def create_engine(engine_id: str, **options):`
Defines function `create_engine` accepting (engine_id, **options).
8. **Line 8:** `print(f"Engine: {engine_id}")`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `for key, value in options.items():`
Returns a view of the dictionary's (key, value) pairs.
10. **Line 10:** `print(f" {key} = {value}")`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `create_engine("piper", device="cpu", threads=4, verbose=True)`
Calls function `create_engine()`.
12. **Line 12:** `# Engine: piper`
A comment: *Engine: piper.*
13. **Line 13:** `# device = cpu`
A comment: *device = cpu.*
14. **Line 14:** `# threads = 4`

A comment: *threads = 4*.

15. **Line 15:** `# verbose = True`

A comment: *verbose = True*.

16. **Line 16:** `# Combining positional, keyword, *args, **kwargs`

A comment: *Combining positional, keyword, *args, **kwargs*.

17. **Line 17:** `def configure(name: str, *engines, debug=False, **settings):`

Defines function `configure` accepting (name, *engines, debug, **settings).

18. **Line 18:** `pass`

Placeholder: does nothing.

19. **Line 19:** `configure("AIVS", "piper", "kokoro", debug=True, rate=1.5)`

Calls function `configure()` .

4.3 Return Values

```
# Single return value
def get_sample_rate(voice: dict) -> int:
    return voice.get("sample_rate", 24000)

# Multiple return values (returns a tuple)
def parse_voice_id(voice_id: str) -> tuple[str, str]:
    """Split 'en_US/medium' into (language, variant)."""
    parts = voice_id.split("/")
    return parts[0], parts[1] # Implicit tuple packing

lang, variant = parse_voice_id("en_US/medium") # Tuple unpacking

# Returning None for "not found"
def find_voice(voices: list, voice_id: str) -> dict | None:
    for v in voices:
        if v["id"] == voice_id:
            return v
    return None
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `# Single return value`
A comment: *Single return value.*
2. **Line 2:** `def get_sample_rate(voice: dict) -> int:`
Defines function `get_sample_rate` accepting (voice), returning `int`.
3. **Line 3:** `return voice.get("sample_rate", 24000)`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
4. **Line 4:** `# Multiple return values (returns a tuple)`
A comment: *Multiple return values (returns a tuple).*
5. **Line 5:** `def parse_voice_id(voice_id: str) -> tuple[str, str]:`
Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.
6. **Line 6:** `"""Split 'en_US/medium' into (language, variant)."""`
Docstring: *Split 'en_US/medium' into (language, variant).* Documents the purpose of the following code.
7. **Line 7:** `parts = voice_id.split("/")`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
8. **Line 8:** `return parts[0], parts[1] # Implicit tuple packing`
Returns `parts[0], parts[1] # Implicit tuple packing` to the caller.
9. **Line 9:** `lang, variant = parse_voice_id("en_US/medium") # Tuple unpacking`
Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.
10. **Line 10:** `# Returning None for "not found"`
A comment: *Returning None for "not found".*
11. **Line 11:** `def find_voice(voices: list, voice_id: str) -> dict | None:`
Defines function `find_voice` accepting (voices, voice_id), returning `dict | None`.
12. **Line 12:** `for v in voices:`
Iterates over `voices`, assigning each element to `v`.
13. **Line 13:** `if v["id"] == voice_id:`
Checks condition: `v["id"] == voice_id`. The indented block runs only when this is `True`.
14. **Line 14:** `return v`

Returns `v` to the caller.

15. **Line 15:** `return None`

Returns `None` (no value).

4.4 Scope — The LEGB Rule

Python looks up variable names in this order: **L**ocal → **E**nclosing → **G**lobal → **B**uilt-in.

```
# Global scope
APP_NAME = "AI Voice Studio"

def outer():
    # Enclosing scope
    version = "2.0"

    def inner():
        # Local scope
        print(APP_NAME)    # Global - found
        print(version)    # Enclosing - found
        print(len("hi"))  # Built-in - found
        name = "inner"    # Local variable

    inner()

outer()
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# Global scope`
A comment: *Global scope*.
2. **Line 2:** `APP_NAME = "AI Voice Studio"`
Assigns "AI Voice Studio" to APP_NAME .
3. **Line 3:** `def outer():`
Defines function outer .
4. **Line 4:** `# Enclosing scope`
A comment: *Enclosing scope*.
5. **Line 5:** `version = "2.0"`
Assigns "2.0" to version .
6. **Line 6:** `def inner():`
Defines function inner .
7. **Line 7:** `# Local scope`
A comment: *Local scope*.
8. **Line 8:** `print(APP_NAME) # Global – found`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(version) # Enclosing – found`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `print(len("hi")) # Built-in – found`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `name = "inner" # Local variable`
Assigns "inner" # Local variable to name .
12. **Line 12:** `inner()`
Calls function inner() .
13. **Line 13:** `outer()`
Calls function outer() .

global and nonlocal

```
# global - modify a global variable from inside a function
counter = 0

def increment():
    global counter
    counter += 1

increment()
print(counter)  # 1

# nonlocal - modify an enclosing scope variable
def make_counter():
    count = 0
    def increment():
        nonlocal count
        count += 1
        return count
    return increment

counter = make_counter()
print(counter())  # 1
print(counter())  # 2
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `# global – modify a global variable from inside a function`

A comment: *global – modify a global variable from inside a function.*

2. **Line 2:** `counter = 0`

Assigns `0` to `counter`.

3. **Line 3:** `def increment():`

Defines function `increment`.

4. **Line 4:** `global counter`

`global counter`

5. **Line 5:** `counter += 1`

Updates `counter` by `+= 1`.

6. **Line 6:** `increment()`

Calls function `increment()`.

7. **Line 7:** `print(counter) # 1`

Outputs text to the console. The primary way to display information during development and debugging.

8. **Line 8:** `# nonlocal – modify an enclosing scope variable`

A comment: *nonlocal – modify an enclosing scope variable.*

9. **Line 9:** `def make_counter():`

Defines function `make_counter`.

10. **Line 10:** `count = 0`

Assigns `0` to `count`.

11. **Line 11:** `def increment():`

Defines function `increment`.

12. **Line 12:** `nonlocal count`

`nonlocal count`

13. **Line 13:** `count += 1`

Updates `count` by `+= 1`.

14. **Line 14:** `return count`
Returns `count` to the caller.
15. **Line 15:** `return increment`
Returns `increment` to the caller.
16. **Line 16:** `counter = make_counter()`
Assigns `make_counter()` to `counter`.
17. **Line 17:** `print(counter()) # 1`
Outputs text to the console. The primary way to display information during development and debugging.
18. **Line 18:** `print(counter()) # 2`
Outputs text to the console. The primary way to display information during development and debugging.

4.5 Lambda Functions

Lambdas are anonymous, single-expression functions:

```
# Regular function
def double(x):
    return x * 2

# Equivalent Lambda
double = lambda x: x * 2

# Common use cases

# Sorting with a key function
voices = [
    {"name": "Zoe", "sid": 5},
    {"name": "Adam", "sid": 1},
    {"name": "Bella", "sid": 3},
]
sorted_voices = sorted(voices, key=lambda v: v["sid"])

# Filtering
big_voices = list(filter(lambda v: v["sid"] > 3, voices))

# Mapping
names = list(map(lambda v: v["name"], voices))

# In AIVS - callback in thread setup
threading.Thread(
    target=lambda: self._download_variant(tts_id, lang_code, variant_id),
    daemon=True,
).start()
```


Explain Line by Line Code Description (Block 10)

1. **Line 1:** `# Regular function`

A comment: *Regular function.*

2. **Line 2:** `def double(x):`

Defines function `double` accepting (x).

3. **Line 3:** `return x * 2`

Returns `x * 2` to the caller.

4. **Line 4:** `# Equivalent lambda`

A comment: *Equivalent lambda.*

5. **Line 5:** `double = lambda x: x * 2`

Creates an anonymous function and assigns it to `double`.

6. **Line 6:** `# Common use cases`

A comment: *Common use cases.*

7. **Line 7:** `# Sorting with a key function`

A comment: *Sorting with a key function.*

8. **Line 8:** `voices = [`

Assigns `[` to `voices`.

9. **Line 9:** `{"name": "Zoe", "sid": 5},`

`{"name": "Zoe", "sid": 5},`

10. **Line 10:** `{"name": "Adam", "sid": 1},`

`{"name": "Adam", "sid": 1},`

11. **Line 11:** `{"name": "Bella", "sid": 3},`

`{"name": "Bella", "sid": 3},`

12. **Line 12:** `]`

`]`

13. **Line 13:** `sorted_voices = sorted(voices, key=lambda v: v["sid"])`

Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.

14. **Line 14:** `# Filtering`
A comment: *Filtering*.
15. **Line 15:** `big_voices = list(filter(lambda v: v["sid"] > 3, voices))`
Returns items from an iterable where the function returns True. Returns an iterator.
16. **Line 16:** `# Mapping`
A comment: *Mapping*.
17. **Line 17:** `names = list(map(lambda v: v["name"], voices))`
Applies a function to every item in an iterable. Returns an iterator. Often replaced by list comprehensions.
18. **Line 18:** `# In AIVS – callback in thread setup`
A comment: *In AIVS – callback in thread setup*.
19. **Line 19:** `threading.Thread(`
Creates a background thread. Use `daemon=True` for threads that should not block app exit.
20. **Line 20:** `target=lambda: self._download_variant(tts_id, lang_code, variant_id),`
Assigns `lambda: self._download_variant(tts_id, lang_code, variant_id),` to `target`.
21. **Line 21:** `daemon=True,`
Assigns `True,` to `daemon`.
22. **Line 22:** `).start()`
`).start()`

4.6 Higher-Order Functions

Functions that take or return other functions:

```
# Function as argument
def apply_to_segments(segments: list, func):
    """Apply a function to each segment."""
    results = []
    for seg in segments:
        results.append(func(seg))
    return results

# Usage
titles = apply_to_segments(segments, lambda s: s["title"])
lengths = apply_to_segments(segments, lambda s: len(s["text"]))

# Built-in higher-order functions
def is_installed(voice):
    return voice.get("installed", False)

# filter: keep items where function returns True
installed = list(filter(is_installed, all_voices))

# map: transform each item
ids = list(map(lambda v: v["id"], all_voices))

# sorted with key
by_name = sorted(all_voices, key=lambda v: v.get("name", "").lower())
```

Explain Line by Line Code Description (Block 11)

1. **Line 1:** `# Function as argument`
A comment: *Function as argument.*
2. **Line 2:** `def apply_to_segments(segments: list, func):`
Defines function `apply_to_segments` accepting (segments, func).
3. **Line 3:** `"""Apply a function to each segment."""`
Docstring: *Apply a function to each segment..* Documents the purpose of the following code.
4. **Line 4:** `results = []`
Assigns `[]` to `results`.
5. **Line 5:** `for seg in segments:`
Iterates over `segments`, assigning each element to `seg`.
6. **Line 6:** `results.append(func(seg))`
Adds a single item to the end of the list.
7. **Line 7:** `return results`
Returns `results` to the caller.
8. **Line 8:** `# Usage`
A comment: *Usage.*
9. **Line 9:** `titles = apply_to_segments(segments, lambda s: s["title"])`
Assigns `apply_to_segments(segments, lambda s: s["title"])` to `titles`.
10. **Line 10:** `lengths = apply_to_segments(segments, lambda s: len(s["text"]))`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
11. **Line 11:** `# Built-in higher-order functions`
A comment: *Built-in higher-order functions.*
12. **Line 12:** `def is_installed(voice):`
Defines function `is_installed` accepting (voice).
13. **Line 13:** `return voice.get("installed", False)`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
14. **Line 14:** `# filter: keep items where function returns True`

A comment: *filter: keep items where function returns True.*

15. **Line 15:** `installed = list(filter(is_installed, all_voices))`

Returns items from an iterable where the function returns True. Returns an iterator.

16. **Line 16:** `# map: transform each item`

A comment: *map: transform each item.*

17. **Line 17:** `ids = list(map(lambda v: v["id"], all_voices))`

Applies a function to every item in an iterable. Returns an iterator. Often replaced by list comprehensions.

18. **Line 18:** `# sorted with key`

A comment: *sorted with key.*

19. **Line 19:** `by_name = sorted(all_voices, key=lambda v: v.get("name", "").lower())`

Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.

4.7 Functions in AI Voice Studio

Here are real function signatures from the project:

```

# From ai_voice_studio/util.py
def sanitize_filename(name: str, max_length: int = 80) -> str:
    """Remove or replace characters that are unsafe in filenames."""

def find_python() -> str:
    """Find a usable Python interpreter on the system."""

# From ai_voice_studio/paths.py
def user_data_dir() -> str:
    """Return the user data directory (%APPDATA%/AIVoiceStudio)."""

def models_dir() -> str:
    """Return the directory where TTS models are stored."""

def project_dir(name: str) -> str:
    """Return the directory for a project with the given name."""

# From ai_voice_studio/tts/engine.py
def get_engine(voice_entry: dict) -> "TtsEngine":
    """Get or create a cached TTS engine for the given voice."""

def _shift_pitch(samples: "np.ndarray", pitch: float) -> "np.ndarray":
    """Shift the pitch of audio samples without changing speed."""

def _apply_volume(samples: "np.ndarray", volume: float) -> "np.ndarray":
    """Apply volume scaling to audio samples."""

# From ai_voice_studio/compute.py
def resolve_compute(mode: str) -> str:
    """Resolve 'auto' to the best available compute back-end."""

def available_compute() -> list[str]:
    """Return list of available compute back-end keys."""

```

Explain Line by Line Code Description (Block 12)

1. **Line 1:** `# From ai_voice_studio/util.py`
A comment: *From ai_voice_studio/util.py.*
2. **Line 2:** `def sanitize_filename(name: str, max_length: int = 80) -> str:`
Defines function `sanitize_filename` accepting (name, max_length), returning `str`.
3. **Line 3:** `"""Remove or replace characters that are unsafe in filenames."""`
Docstring: *Remove or replace characters that are unsafe in filenames..* Documents the purpose of the following code.
4. **Line 4:** `def find_python() -> str:`
Defines function `find_python`, returning `str`.
5. **Line 5:** `"""Find a usable Python interpreter on the system."""`
Docstring: *Find a usable Python interpreter on the system..* Documents the purpose of the following code.
6. **Line 6:** `# From ai_voice_studio/paths.py`
A comment: *From ai_voice_studio/paths.py.*
7. **Line 7:** `def user_data_dir() -> str:`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
8. **Line 8:** `"""Return the user data directory (%APPDATA%/AIVoiceStudio)."""`
Docstring: *Return the user data directory (%APPDATA%/AIVoiceStudio)..* Documents the purpose of the following code.
9. **Line 9:** `def models_dir() -> str:`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
10. **Line 10:** `"""Return the directory where TTS models are stored."""`
Docstring: *Return the directory where TTS models are stored..* Documents the purpose of the following code.
11. **Line 11:** `def project_dir(name: str) -> str:`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
12. **Line 12:** `"""Return the directory for a project with the given name."""`
Docstring: *Return the directory for a project with the given name..* Documents the purpose of the following code.
13. **Line 13:** `# From ai_voice_studio/tts/engine.py`
A comment: *From ai_voice_studio/tts/engine.py.*
14. **Line 14:** `def get_engine(voice_entry: dict) -> "TtsEngine":`

Begins exception handling. Code in this block is monitored for errors.

15. **Line 15:** `"""Get or create a cached TTS engine for the given voice."""`

Docstring: *Get or create a cached TTS engine for the given voice..* Documents the purpose of the following code.

16. **Line 16:** `def _shift_pitch(samples: "np.ndarray", pitch: float) -> "np.ndarray":`

Defines function `_shift_pitch` accepting (samples, pitch), returning `"np.ndarray"` .

17. **Line 17:** `"""Shift the pitch of audio samples without changing speed."""`

Docstring: *Shift the pitch of audio samples without changing speed..* Documents the purpose of the following code.

18. **Line 18:** `def _apply_volume(samples: "np.ndarray", volume: float) -> "np.ndarray":`

Defines function `_apply_volume` accepting (samples, volume), returning `"np.ndarray"` .

19. **Line 19:** `"""Apply volume scaling to audio samples."""`

Docstring: *Apply volume scaling to audio samples..* Documents the purpose of the following code.

20. **Line 20:** `# From ai_voice_studio/compute.py`

A comment: *From ai_voice_studio/compute.py.*

21. **Line 21:** `def resolve_compute(mode: str) -> str:`

Defines function `resolve_compute` accepting (mode), returning `str` .

22. **Line 22:** `"""Resolve 'auto' to the best available compute back-end."""`

Docstring: *Resolve 'auto' to the best available compute back-end..* Documents the purpose of the following code.

23. **Line 23:** `def available_compute() -> list[str]:`

Defines function `available_compute` , returning `list[str]` .

24. **Line 24:** `"""Return list of available compute back-end keys."""`

Docstring: *Return list of available compute back-end keys..* Documents the purpose of the following code.

4.8 Recursive Functions

```
# Factorial (classic example)
def factorial(n: int) -> int:
    if n <= 1:
        return 1
    return n * factorial(n - 1)

# Recursive directory traversal (used in AIVS)
import os

def find_all_files(directory: str) -> list[str]:
    """Recursively find all files in a directory."""
    result = []
    for entry in os.listdir(directory):
        full_path = os.path.join(directory, entry)
        if os.path.isfile(full_path):
            result.append(full_path)
        elif os.path.isdir(full_path):
            result.extend(find_all_files(full_path))
    return result
```

Explain Line by Line Code Description (Block 13)

1. **Line 1:** `# Factorial (classic example)`

A comment: *Factorial (classic example)*.

2. **Line 2:** `def factorial(n: int) -> int:`

Defines function `factorial` accepting (n), returning `int`.

3. **Line 3:** `if n <= 1:`

Checks condition: `n <= 1`. The indented block runs only when this is `True`.

4. **Line 4:** `return 1`

Returns `1` to the caller.

5. **Line 5:** `return n * factorial(n - 1)`

Returns `n * factorial(n - 1)` to the caller.

6. **Line 6:** `# Recursive directory traversal (used in AIVS)`

A comment: *Recursive directory traversal (used in AIVS)*.

7. **Line 7:** `import os`

Imports the `os` module.

8. **Line 8:** `def find_all_files(directory: str) -> list[str]:`

Defines function `find_all_files` accepting (directory), returning `list[str]`.

9. **Line 9:** `"""Recursively find all files in a directory."""`

Docstring: *Recursively find all files in a directory..* Documents the purpose of the following code.

10. **Line 10:** `result = []`

Assigns `[]` to `result`.

11. **Line 11:** `for entry in os.listdir(directory):`

Returns a list of attributes and methods of an object. Useful for exploration and debugging.

12. **Line 12:** `full_path = os.path.join(directory, entry)`

OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.

13. **Line 13:** `if os.path.isfile(full_path):`

OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.

14. **Line 14:** `result.append(full_path)`
Adds a single item to the end of the list.
15. **Line 15:** `elif os.path.isdir(full_path):`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splittext()`.
16. **Line 16:** `result.extend(find_all_files(full_path))`
Extends the list by appending items from an iterable.
17. **Line 17:** `return result`
Returns `result` to the caller.

4.9 Key Takeaways

- Use **type hints** on parameters and return values for clarity
- **Docstrings** should explain what, not how
- **Default parameters** must be immutable (`None` for mutable defaults)
- `*args` and `**kwargs` enable flexible function signatures
- **Lambdas** are for short, throwaway functions — use `def` for anything complex
- Avoid `global` and `nonlocal` when possible — they make code harder to reason about
- Functions should do **one thing** and do it well (Single Responsibility Principle)

 **Exercise: Exercise 4.1:** Write a function `format_duration(seconds: float) -> str` that converts a number of seconds into "HH:MM:SS" format. Test it with 0, 65, 3661, and 86400.

 **Exercise: Exercise 4.2:** Write a higher-order function `apply_filter(items, predicate)` that returns only items where the predicate returns True. Rewrite the AIVS voice filtering to use it.

Strings & Formatting

Learning Objectives

- Master string methods for manipulation, searching, and transformation
 - Learn advanced f-string formatting with format specifiers
 - Understand Unicode, encoding, and raw strings
 - Apply string processing to TTS text preparation in AIVS
-

5.1 String Creation Review

```
# Single and double quotes are equivalent
s1 = 'hello'
s2 = "hello"

# Triple quotes for multi-line strings
text = """This is
a multi-line
string."""

# Escape sequences
s = "Line 1\nLine 2\tTabbed"
s = "She said \"hello\""
s = "Path: C:\\Users\\doc.txt"

# Raw strings (no escape processing)
path = r"C:\Users\doc.txt"      # Backslashes are literal
regex = r"\d+\.d+"             # Regex patterns

# Bytes vs strings
text = "hello"                 # str (Unicode)
data = b"hello"                # bytes (raw bytes)
encoded = text.encode("utf-8")  # str → bytes
decoded = data.decode("utf-8")  # bytes → str
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Single and double quotes are equivalent`
A comment: *Single and double quotes are equivalent.*
2. **Line 2:** `s1 = 'hello'`
Assigns `'hello'` to `s1`.
3. **Line 3:** `s2 = "hello"`
Assigns `"hello"` to `s2`.
4. **Line 4:** `# Triple quotes for multi-line strings`
A comment: *Triple quotes for multi-line strings.*
5. **Line 5:** `text = """This is`
Assigns `"""This is` to `text`.
6. **Line 6:** `a multi-line`
`a multi-line`
7. **Line 7:** `string."""`
`string."""`
8. **Line 8:** `# Escape sequences`
A comment: *Escape sequences.*
9. **Line 9:** `s = "Line 1\nLine 2\tTabbed"`
Assigns `"Line 1\nLine 2\tTabbed"` to `s`.
10. **Line 10:** `s = "She said \"hello\""`
Assigns `"She said \"hello\""` to `s`.
11. **Line 11:** `s = "Path: C:\\Users\\doc.txt"`
Assigns `"Path: C:\\Users\\doc.txt"` to `s`.
12. **Line 12:** `# Raw strings (no escape processing)`
A comment: *Raw strings (no escape processing).*
13. **Line 13:** `path = r"C:\Users\doc.txt" # Backslashes are literal`
Assigns `r"C:\Users\doc.txt"` # Backslashes are literal to `path`.

14. **Line 14:** `regex = r"\d+\.\d+"` # Regex patterns
Assigns `r"\d+\.\d+"` # Regex patterns to `regex` .
15. **Line 15:** `# Bytes vs strings`
A comment: *Bytes vs strings*.
16. **Line 16:** `text = "hello"` # str (Unicode)
Assigns `"hello"` # str (Unicode) to `text` .
17. **Line 17:** `data = b"hello"` # bytes (raw bytes)
Assigns `b"hello"` # bytes (raw bytes) to `data` .
18. **Line 18:** `encoded = text.encode("utf-8")` # str → bytes
Encodes the string to bytes using the specified encoding (default UTF-8).
19. **Line 19:** `decoded = data.decode("utf-8")` # bytes → str
Assigns `data.decode("utf-8")` # bytes → str to `decoded` .

5.2 Essential String Methods

Case Methods

```
s = "Hello, AI Voice Studio!"
print(s.upper())      # "HELLO, AI VOICE STUDIO!"
print(s.lower())      # "hello, ai voice studio!"
print(s.title())      # "Hello, Ai Voice Studio!"
print(s.capitalize()) # "Hello, ai voice studio!"
print(s.swapcase())   # "hELLO, ai voice studio!"
print(s.casefold())    # Like lower(), but more aggressive
```

Explain Line by Line Code Description (Block 2)

- Line 1:** `s = "Hello, AI Voice Studio!"`
Assigns "Hello, AI Voice Studio!" to `s`.
- Line 2:** `print(s.upper())` # "HELLO, AI VOICE STUDIO!"
Converts all characters to uppercase. Returns a new string (strings are immutable).
- Line 3:** `print(s.lower())` # "hello, ai voice studio!"
Converts all characters to lowercase. Returns a new string.
- Line 4:** `print(s.title())` # "Hello, Ai Voice Studio!"
Capitalizes the first letter of each word.
- Line 5:** `print(s.capitalize())` # "Hello, ai voice studio!"
Capitalizes the first letter of the string.
- Line 6:** `print(s.swapcase())` # "hELLO, ai voice studio!"
Swaps case of each character.
- Line 7:** `print(s.casefold())` # Like lower(), but more aggressive
Outputs text to the console. The primary way to display information during development and debugging.

Search and Test Methods

```
s = "The quick brown fox jumps over the lazy dog"

# Finding positions
print(s.find("fox"))           # 16 (index of first occurrence)
print(s.find("fox", 20))       # -1 (not found after index 20)
print(s.rfind("the"))          # 31 (Last occurrence)
print(s.count("the"))          # 2

# Boolean tests
print(s.startswith("The"))     # True
print(s.endswith("dog"))       # True
print(s.isalpha())             # False (has spaces)
print("abc123".isalnum())       # True
print("123".isdigit())          # True
print("\t\n".isspace())        # True
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `s = "The quick brown fox jumps over the lazy dog"`
Assigns "The quick brown fox jumps over the lazy dog" to `s`.
2. **Line 2:** `# Finding positions`
A comment: *Finding positions*.
3. **Line 3:** `print(s.find("fox"))` `# 16 (index of first occurrence)`
Outputs text to the console. The primary way to display information during development and debugging.
4. **Line 4:** `print(s.find("fox", 20))` `# -1 (not found after index 20)`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `print(s.rfind("the"))` `# 31 (last occurrence)`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `print(s.count("the"))` `# 2`
Returns the number of occurrences of a value.
7. **Line 7:** `# Boolean tests`
A comment: *Boolean tests*.
8. **Line 8:** `print(s.startswith("The"))` `# True`
Returns True if the string starts with the given prefix.
9. **Line 9:** `print(s.endswith("dog"))` `# True`
Returns True if the string ends with the given suffix.
10. **Line 10:** `print(s.isalpha())` `# False (has spaces)`
Returns True if all characters are alphabetic.
11. **Line 11:** `print("abc123".isalnum())` `# True`
Returns True if all characters are alphanumeric.
12. **Line 12:** `print("123".isdigit())` `# True`
Returns True if all characters are digits.
13. **Line 13:** `print(" \t\n".isspace())` `# True`
Outputs text to the console. The primary way to display information during development and debugging.

Transformation Methods

```
# Stripping whitespace
s = "  hello  \n"
print(s.strip())    # "hello"
print(s.lstrip())   # "hello  \n"
print(s.rstrip())   # "  hello"

# Strip specific characters
print("###hello###".strip("#")) # "hello"

# Replacing
s = "Piper / en_US / medium"
print(s.replace(" / ", "/")) # "Piper/en_US/medium"

# Splitting and joining
parts = "en_US/medium".split("/") # ["en_US", "medium"]
joined = " | ".join(parts)         # "en_US | medium"

# Alignment and padding
print(f"{'hi':10}") # 'hi          ' (left-aligned)
print(f"{'hi':^10}") # '    hi    ' (centered)
print(f"{'hi':>10}") # '          hi' (right-aligned)
print(f"{'hi':*^10}") # '****hi****' (fill with *)
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# Stripping whitespace`
A comment: *Stripping whitespace*.
2. **Line 2:** `s = " hello \n"`
Assigns `" hello \n"` to `s`.
3. **Line 3:** `print(s.strip()) # "hello"`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
4. **Line 4:** `print(s.lstrip()) # "hello \n"`
Removes leading whitespace only.
5. **Line 5:** `print(s.rstrip()) # " hello"`
Removes trailing whitespace only.
6. **Line 6:** `# Strip specific characters`
A comment: *Strip specific characters*.
7. **Line 7:** `print("###hello###".strip("#")) # "hello"`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
8. **Line 8:** `# Replacing`
A comment: *Replacing*.
9. **Line 9:** `s = "Piper / en_US / medium"`
Assigns `"Piper / en_US / medium"` to `s`.
10. **Line 10:** `print(s.replace(" / ", "/")) # "Piper/en_US/medium"`
Returns a copy with all occurrences of old replaced by new. Strings are immutable.
11. **Line 11:** `# Splitting and joining`
A comment: *Splitting and joining*.
12. **Line 12:** `parts = "en_US/medium".split("/") # ["en_US", "medium"]`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
13. **Line 13:** `joined = " | ".join(parts) # "en_US | medium"`
Joins an iterable of strings with the calling string as separator. `','.join(['a','b'])` returns `'a,b'`.
14. **Line 14:** `# Alignment and padding`
A comment: *Alignment and padding*.

15. **Line 15:** `print(f'{'hi':10}')` # 'hi' (left-aligned)
Outputs text to the console. The primary way to display information during development and debugging.
16. **Line 16:** `print(f'{'hi':^10}')` # ' hi' (centered)
Outputs text to the console. The primary way to display information during development and debugging.
17. **Line 17:** `print(f'{'hi':>10}')` # ' hi' (right-aligned)
Outputs text to the console. The primary way to display information during development and debugging.
18. **Line 18:** `print(f'{'hi':*^10}')` # '****hi****' (fill with *)
Outputs text to the console. The primary way to display information during development and debugging.

5.3 Advanced f-string Formatting

Format Spec	Meaning	Example	Output
<code>.Nf</code>	N decimal places	<code>f"{3.14159:.2f}"</code>	<code>3.14</code>
<code>:d</code>	Decimal integer	<code>f"{42:d}"</code>	<code>42</code>
<code>:05d</code>	Zero-padded int	<code>f"{42:05d}"</code>	<code>00042</code>
<code>:,</code>	Thousands separator	<code>f"{1000000:,}"</code>	<code>1,000,000</code>
<code>:.1%</code>	Percentage	<code>f"{0.857:.1%}"</code>	<code>85.7%</code>
<code>:e</code>	Scientific notation	<code>f"{12345:e}"</code>	<code>1.2345e+04</code>
<code>:b</code>	Binary	<code>f"{42:b}"</code>	<code>101010</code>
<code>:x</code>	Hex lowercase	<code>f"{255:x}"</code>	<code>ff</code>
<code>:+10d</code>	Sign + padding	<code>f"{42:+=10d}"</code>	<code>+ 42</code>

```
# Debugging with = (Python 3.8+)
voices = ["af", "am_adam"]
print(f"{len(voices) = }") # len(voices) = 2
print(f"{voices[0] = }") # voices[0] = 'af'

# Nested quotes work seamlessly
print(f"Engine: {'Piper' if use_piper else 'Kokoro'}")

# Calling methods in f-strings
name = "en_US"
print(f"Language: {name.upper()}") # "Language: EN_US"
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# Debugging with = (Python 3.8+)`
A comment: *Debugging with = (Python 3.8+).*
2. **Line 2:** `voices = ["af", "am_adam"]`
Assigns `["af", "am_adam"]` to `voices` .
3. **Line 3:** `print(f"{len(voices) = }")` `# len(voices) = 2`
Outputs text to the console. The primary way to display information during development and debugging.
4. **Line 4:** `print(f"{voices[0] = }")` `# voices[0] = 'af'`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `# Nested quotes work seamlessly`
A comment: *Nested quotes work seamlessly.*
6. **Line 6:** `print(f"Engine: {'Piper' if use_piper else 'Kokoro'}")`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# Calling methods in f-strings`
A comment: *Calling methods in f-strings.*
8. **Line 8:** `name = "en_US"`
Assigns `"en_US"` to `name` .
9. **Line 9:** `print(f"Language: {name.upper()}")` `# "Language: EN_US"`
Converts all characters to uppercase. Returns a new string (strings are immutable).

5.4 String Slicing Deep Dive

```
s = "AI-Voice-Studio-v2.0"

# s[start:stop:step]
print(s[0:2])      # "AI" (indices 0, 1)
print(s[3:8])      # "Voice"
print(s[::2])       # "AVcsod.0" (every other char)
print(s[::-1])      # Reversed string

# Negative indices
print(s[-3:])       # "2.0" (last 3 chars)
print(s[-5:-3])     # "v2"

# Practical: extract version from filename
filename = "AI-Voice-Studio-v2.0.1"
version_start = filename.rfind("v") + 1
version = filename[version_start:]
print(version)      # "2.0.1"
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `s = "AI-Voice-Studio-v2.0"`
Assigns "AI-Voice-Studio-v2.0" to `s`.
2. **Line 2:** `# s[start:stop:step]`
A comment: `s[start:stop:step]`.
3. **Line 3:** `print(s[0:2])` `# "AI" (indices 0, 1)`
Outputs text to the console. The primary way to display information during development and debugging.
4. **Line 4:** `print(s[3:8])` `# "Voice"`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `print(s[::2])` `# "AVcsod.0" (every other char)`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `print(s[::-1])` `# Reversed string`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# Negative indices`
A comment: *Negative indices*.
8. **Line 8:** `print(s[-3:])` `# "2.0" (last 3 chars)`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(s[-5:-3])` `# "v2"`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# Practical: extract version from filename`
A comment: *Practical: extract version from filename*.
11. **Line 11:** `filename = "AI-Voice-Studio-v2.0.1"`
Assigns "AI-Voice-Studio-v2.0.1" to `filename`.
12. **Line 12:** `version_start = filename.rfind("v") + 1`
Assigns `filename.rfind("v") + 1` to `version_start`.
13. **Line 13:** `version = filename[version_start:]`
Assigns `filename[version_start:]` to `version`.
14. **Line 14:** `print(version)` `# "2.0.1"`

Outputs text to the console. The primary way to display information during development and debugging.

5.5 Multi-line Strings and Text Processing

```
# Triple-quoted strings preserve formatting
template = """
Project: {name}
Segments: {count}
Duration: {duration:.1f}s
"""

print(template.format(name="My Book", count=42, duration=3600.5))

# Text wrapping
import textwrap
long_text = "This is a very long line that needs to be wrapped at a certain width for display purposes."
wrapped = textwrap.fill(long_text, width=40)
print(wrapped)

# Dedent (remove common leading whitespace)
import textwrap
code = """
    if True:
        print("hello")
        print("world")
"""
print(textwrap.dedent(code))
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `# Triple-quoted strings preserve formatting`

A comment: *Triple-quoted strings preserve formatting.*

2. **Line 2:** `template = ""`

Assigns `""` to `template`.

3. **Line 3:** `Project: {name}`

`Project: {name}`

4. **Line 4:** `Segments: {count}`

`Segments: {count}`

5. **Line 5:** `Duration: {duration:.1f}s`

`Duration: {duration:.1f}s`

6. **Line 6:** `"""`

Opens a docstring block for documentation.

7. **Line 7:** `print(template.format(name="My Book", count=42, duration=3600.5))`

Formats the string with positional or keyword arguments. Superseded by f-strings.

8. **Line 8:** `# Text wrapping`

A comment: *Text wrapping.*

9. **Line 9:** `import textwrap`

Imports the `textwrap` module.

10. **Line 10:**

```
long_text = "This is a very long line that needs to be wrapped at a certain width for display purposes."
es."
```

Assigns `"This is a very long line that needs to be wrapped at a certain width for display purposes."` to `long_text`.

11. **Line 11:** `wrapped = textwrap.fill(long_text, width=40)`

Wraps text to a given width, inserting newlines.

12. **Line 12:** `print(wrapped)`

Outputs text to the console. The primary way to display information during development and debugging.

13. **Line 13:** `# Dedent (remove common leading whitespace)`

A comment: *Dedent (remove common leading whitespace).*

14. Line 14: `import textwrap`

Imports the `textwrap` module.

15. Line 15: `code = """`

Assigns `"""` to `code`.

16. Line 16: `if True:`

Checks condition: `True`. The indented block runs only when this is `True`.

17. Line 17: `print("hello")`

Outputs text to the console. The primary way to display information during development and debugging.

18. Line 18: `print("world")`

Outputs text to the console. The primary way to display information during development and debugging.

19. Line 19: `"""`

Opens a docstring block for documentation.

20. Line 20: `print(textwrap.dedent(code))`

Removes common leading whitespace from all lines. Useful for multi-line strings.

5.6 Strings in TTS Processing (AIVS)

The TTS engine requires careful text preparation before synthesis:

```

# From documents/splitter.py - text segmentation for TTS
def _split_sentences(text: str) -> list[str]:
    """Split text into sentences for TTS processing."""
    import re
    # Split on sentence-ending punctuation
    sentences = re.split(r'(?<[.!?])\s+', text)
    return [s.strip() for s in sentences if s.strip()]

# From util.py - filename sanitization
def _sanitize_filename(name: str, max_length: int = 80) -> str:
    """Remove unsafe characters from filenames."""
    import re
    name = re.sub(r'[:>"/\|]?*', '_', name)
    name = re.sub(r'\s+', ' ', name).strip()
    if len(name) > max_length:
        name = name[:max_length].rstrip()
    return name

# From documents/parsers.py - text normalization
def _normalize_text(text: str) -> str:
    """Normalize text for TTS processing."""
    # Remove excessive whitespace
    text = re.sub(r'\n{3,}', '\n\n', text)
    # Normalize Unicode
    text = text.strip()
    return text

# From tts/catalog.py - building voice labels
def _build_voice_label(voice: dict) -> str:
    """Build a display label like 'af_bella (af_bella)'."""
    name = voice.get("name", voice["id"])
    voice_id = voice["id"]
    if name != voice_id:
        return f"{name} ({voice_id})"
    return name

```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# From documents/splitter.py – text segmentation for TTS`
A comment: *From documents/splitter.py – text segmentation for TTS.*
2. **Line 2:** `def _split_sentences(text: str) -> list[str]:`
Defines function `_split_sentences` accepting (text), returning `list[str]`.
3. **Line 3:** `"""Split text into sentences for TTS processing."""`
Docstring: *Split text into sentences for TTS processing..* Documents the purpose of the following code.
4. **Line 4:** `import re`
Imports the `re` module.
5. **Line 5:** `# Split on sentence-ending punctuation`
A comment: *Split on sentence-ending punctuation.*
6. **Line 6:** `sentences = re.split(r'(?<=[!?])\s+', text)`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
7. **Line 7:** `return [s.strip() for s in sentences if s.strip()]`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
8. **Line 8:** `# From util.py – filename sanitization`
A comment: *From util.py – filename sanitization.*
9. **Line 9:** `def sanitize_filename(name: str, max_length: int = 80) -> str:`
Defines function `sanitize_filename` accepting (name, max_length), returning `str`.
10. **Line 10:** `"""Remove unsafe characters from filenames."""`
Docstring: *Remove unsafe characters from filenames..* Documents the purpose of the following code.
11. **Line 11:** `import re`
Imports the `re` module.
12. **Line 12:** `name = re.sub(r'<>:"/\|\\|?*]', '_', name)`
Replaces all matches with the replacement string.
13. **Line 13:** `name = re.sub(r'\s+', ' ', name).strip()`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
14. **Line 14:** `if len(name) > max_length:`

Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

15. **Line 15:** `name = name[:max_length].rstrip()`

Removes trailing whitespace only.

16. **Line 16:** `return name`

Returns `name` to the caller.

17. **Line 17:** `# From documents/parsers.py – text normalization`

A comment: *From documents/parsers.py – text normalization.*

18. **Line 18:** `def _normalize_text(text: str) -> str:`

Defines function `_normalize_text` accepting (text), returning `str`.

19. **Line 19:** `"""Normalize text for TTS processing."""`

Docstring: *Normalize text for TTS processing..* Documents the purpose of the following code.

20. **Line 20:** `# Remove excessive whitespace`

A comment: *Remove excessive whitespace.*

21. **Line 21:** `text = re.sub(r'\n{3,}', '\n\n', text)`

Replaces all matches with the replacement string.

22. **Line 22:** `# Normalize Unicode`

A comment: *Normalize Unicode.*

23. **Line 23:** `text = text.strip()`

Removes leading and trailing whitespace (or specified characters). Returns a new string.

24. **Line 24:** `return text`

Returns `text` to the caller.

25. **Line 25:** `# From tts/catalog.py – building voice labels`

A comment: *From tts/catalog.py – building voice labels.*

26. **Line 26:** `def _build_voice_label(voice: dict) -> str:`

Defines function `_build_voice_label` accepting (voice), returning `str`.

27. **Line 27:** `"""Build a display label like 'af_bella (af_bella)'."""`

Docstring: *Build a display label like 'af_bella (af_bella)'..* Documents the purpose of the following code.

28. **Line 28:** `name = voice.get("name", voice["id"])`

Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

29. **Line 29:** `voice_id = voice["id"]`
Assigns `voice["id"]` to `voice_id`.
30. **Line 30:** `if name != voice_id:`
Checks condition: `name != voice_id`. The indented block runs only when this is `True`.
31. **Line 31:** `return f"{name} ({voice_id})"`
Returns `f"{name} ({voice_id})"` to the caller.
32. **Line 32:** `return name`
Returns `name` to the caller.

5.7 Unicode and Encoding

```
# Python 3 strings are Unicode by default
s = "Hello 你好 مرحبا"
print(len(s)) # 14 characters (not bytes)

# Encode to bytes for storage/transmission
encoded = s.encode("utf-8")
print(len(encoded)) # 18 bytes

# Decode back to string
decoded = encoded.decode("utf-8")

# Unicode categories useful for text processing
import unicodedata
for char in "AĬ":
    print(f"{char}: {unicodedata.name(char)}")
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `# Python 3 strings are Unicode by default`
A comment: *Python 3 strings are Unicode by default.*
2. **Line 2:** `s = "Hello 你好 مرحبا"`
Assigns `"Hello 你好 مرحبا"` to `s`.
3. **Line 3:** `print(len(s)) # 14 characters (not bytes)`
Outputs text to the console. The primary way to display information during development and debugging.
4. **Line 4:** `# Encode to bytes for storage/transmission`
A comment: *Encode to bytes for storage/transmission.*
5. **Line 5:** `encoded = s.encode("utf-8")`
Encodes the string to bytes using the specified encoding (default UTF-8).
6. **Line 6:** `print(len(encoded)) # 18 bytes`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# Decode back to string`
A comment: *Decode back to string.*
8. **Line 8:** `decoded = encoded.decode("utf-8")`
Assigns `encoded.decode("utf-8")` to `decoded`.
9. **Line 9:** `# Unicode categories useful for text processing`
A comment: *Unicode categories useful for text processing.*
10. **Line 10:** `import unicodedata`
Imports the `unicodedata` module.
11. **Line 11:** `for char in "Aí":`
Iterates over `"Aí"`, assigning each element to `char`.
12. **Line 12:** `print(f"{char}: {unicodedata.name(char)}")`
Outputs text to the console. The primary way to display information during development and debugging.

5.8 Regular Expressions (Preview)

Python's `re` module provides powerful pattern matching. We cover it in depth in Chapter 15:

```
import re

# Find all voice IDs
text = "Voices: af, am_adam, bf_emma, bm_george"
ids = re.findall(r'\b[a-z]+(?:_[a-z]+)*\b', text)
print(ids) # ['af', 'am_adam', 'bf_emma', 'bm_george']

# Replace multiple spaces
clean = re.sub(r'\s+', ' ', "too many spaces")
print(clean) # "too many spaces"

# Validate email (simple pattern)
email_re = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'
```

Explain Line by Line Code Description (Block 10)

- Line 1:** `import re`
Imports the `re` module.
- Line 2:** `# Find all voice IDs`
A comment: *Find all voice IDs.*
- Line 3:** `text = "Voices: af, am_adam, bf_emma, bm_george"`
Assigns `"Voices: af, am_adam, bf_emma, bm_george"` to `text`.
- Line 4:** `ids = re.findall(r'\b[a-z]+(?:_[a-z]+)*\b', text)`
Returns all non-overlapping matches as a list of strings.
- Line 5:** `print(ids) # ['af', 'am_adam', 'bf_emma', 'bm_george']`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 6:** `# Replace multiple spaces`
A comment: *Replace multiple spaces.*
- Line 7:** `clean = re.sub(r'\s+', ' ', "too many spaces")`
Replaces all matches with the replacement string.
- Line 8:** `print(clean) # "too many spaces"`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 9:** `# Validate email (simple pattern)`
A comment: *Validate email (simple pattern).*
- Line 10:** `email_re = r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'`
Assigns `r'^[a-zA-Z0-9._%+-]+@[a-zA-Z0-9.-]+\.[a-zA-Z]{2,}$'` to `email_re`.

5.9 Key Takeaways

- **Strings are immutable** — every method returns a new string
- **f-strings** are the preferred formatting method (concise, readable, fast)
- **str methods** cover virtually all text processing needs
- **str.split()** and **str.join()** are your best friends for text manipulation

- **Unicode is built-in** — Python 3 handles international text natively
- The AIVS project uses string processing for **text normalization**, **filename sanitization**, and **voice label formatting**

 **Exercise: Exercise 5.1:** Write a function that takes a filename like `"my great book.txt"` and returns `"my_great_book.txt"` (replacing spaces with underscores, lowercasing).

 **Exercise: Exercise 5.2:** Using f-strings with format specifiers, format a table showing each TTS engine's name, voice count, and license. Right-align numbers and left-align text.

Lists, Tuples, Sets & Dictionaries

Learning Objectives

- Master Python's four core collection types
 - Understand when to use each type
 - Learn collection methods and operations
 - See how AIVS uses collections for voice catalogs and settings
-

6.1 Lists — Ordered, Mutable Sequences

```
# Creating Lists
engines = ["Piper", "Kokoro", "Matcha", "Kitten"]
numbers = [1, 2, 3, 4, 5]
mixed = [42, "hello", True, None] # Any types
empty = []

# Accessing elements (0-based indexing)
print(engines[0])    # "Piper"
print(engines[-1])   # "Kitten" (last element)

# Slicing (returns a new list)
print(engines[1:3])   # ["Kokoro", "Matcha"]
print(engines[::2])   # ["Piper", "Matcha"] (every other)
print(engines[::-1]) # Reversed copy

# Modifying
engines[0] = "Piper v2"
engines.append("Bark")
engines.insert(1, "New Engine")
engines.extend(["OmniVoice", "Server"])
engines.remove("Matcha")
popped = engines.pop() # Remove and return last
popped = engines.pop(0) # Remove and return first
del engines[0]          # Delete by index

# Useful operations
print(len(engines))    # Count
print("Piper" in engines) # Membership test: True
engines.sort()         # Sort in-place (alphabetical)
engines.sort(reverse=True) # Reverse sort
engines.reverse()      # Reverse in-place

# Count and index
engines = ["Piper", "Kokoro", "Piper"]
print(engines.count("Piper")) # 2
print(engines.index("Kokoro")) # 1

# Copying (important!)
a = [1, 2, 3]
b = a.copy()    # Independent copy
b = a[:]        # Also an independent copy
b = list(a)     # Also an independent copy
# b = a         # WRONG: just another reference to same list!
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Creating lists`
A comment: *Creating lists*.
2. **Line 2:** `engines = ["Piper", "Kokoro", "Matcha", "Kitten"]`
Assigns `["Piper", "Kokoro", "Matcha", "Kitten"]` to `engines`.
3. **Line 3:** `numbers = [1, 2, 3, 4, 5]`
Assigns `[1, 2, 3, 4, 5]` to `numbers`.
4. **Line 4:** `mixed = [42, "hello", True, None] # Any types`
Type hint meaning any type is acceptable. Use sparingly.
5. **Line 5:** `empty = []`
Assigns `[]` to `empty`.
6. **Line 6:** `# Accessing elements (0-based indexing)`
A comment: *Accessing elements (0-based indexing)*.
7. **Line 7:** `print(engines[0]) # "Piper"`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `print(engines[-1]) # "Kitten" (last element)`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `# Slicing (returns a new list)`
A comment: *Slicing (returns a new list)*.
10. **Line 10:** `print(engines[1:3]) # ["Kokoro", "Matcha"]`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `print(engines[::2]) # ["Piper", "Matcha"] (every other)`
Outputs text to the console. The primary way to display information during development and debugging.
12. **Line 12:** `print(engines[::-1]) # Reversed copy`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `# Modifying`
A comment: *Modifying*.
14. **Line 14:** `engines[0] = "Piper v2"`

```
engines[0] = "Piper v2"
```

15. **Line 15:** `engines.append("Bark")`

Adds a single item to the end of the list.

16. **Line 16:** `engines.insert(1, "New Engine")`

Inserts an item at the given index. Shifts subsequent items right.

17. **Line 17:** `engines.extend(["OmniVoice", "Server"])`

Extends the list by appending items from an iterable.

18. **Line 18:** `engines.remove("Matcha")`

Removes an element. Raises `KeyError` if not found.

19. **Line 19:** `popped = engines.pop() # Remove and return last`

Removes and returns the value for a key, or a default if not found.

20. **Line 20:** `popped = engines.pop(0) # Remove and return first`

Removes and returns the value for a key, or a default if not found.

21. **Line 21:** `del engines[0] # Delete by index`

```
del engines[0] # Delete by index
```

22. **Line 22:** `# Useful operations`

A comment: *Useful operations*.

23. **Line 23:** `print(len(engines)) # Count`

Outputs text to the console. The primary way to display information during development and debugging.

24. **Line 24:** `print("Piper" in engines) # Membership test: True`

Outputs text to the console. The primary way to display information during development and debugging.

25. **Line 25:** `engines.sort() # Sort in-place (alphabetical)`

Sorts the list in-place. Accepts `key=` and `reverse=` parameters.

26. **Line 26:** `engines.sort(reverse=True) # Reverse sort`

Sorts the list in-place. Accepts `key=` and `reverse=` parameters.

27. **Line 27:** `engines.reverse() # Reverse in-place`

Reverses the list in-place.

28. **Line 28:** `# Count and index`

A comment: *Count and index*.

29. **Line 29:** `engines = ["Piper", "Kokoro", "Piper"]`
Assigns `["Piper", "Kokoro", "Piper"]` to `engines`.
30. **Line 30:** `print(engines.count("Piper")) # 2`
Returns the number of occurrences of a value.
31. **Line 31:** `print(engines.index("Kokoro")) # 1`
Returns the index of the first occurrence of a value.
32. **Line 32:** `# Copying (important!)`
A comment: *Copying (important!)*.
33. **Line 33:** `a = [1, 2, 3]`
Assigns `[1, 2, 3]` to `a`.
34. **Line 34:** `b = a.copy() # Independent copy`
Returns a shallow copy of the dictionary.
35. **Line 35:** `b = a[:] # Also an independent copy`
Assigns `a[:]` `# Also an independent copy` to `b`.
36. **Line 36:** `b = list(a) # Also an independent copy`
Assigns `list(a)` `# Also an independent copy` to `b`.
37. **Line 37:** `# b = a # WRONG: just another reference to same list!`
A comment: *b = a # WRONG: just another reference to same list!.*
-

6.2 Tuples — Ordered, Immutable Sequences

```
# Creating tuples
point = (3, 4)
single = (42,)      # Trailing comma required for single-element tuple
empty = ()
# Parentheses are optional for packing
point = 3, 4        # Also a tuple

# Accessing (same as Lists)
print(point[0])      # 3

# Cannot modify!
# point[0] = 5 # TypeError: 'tuple' does not support item assignment

# Named tuples – readable tuples with field names
from collections import namedtuple

Segment = namedtuple("Segment", ["index", "title", "text"])
seg = Segment(index=1, title="Chapter 1", text="Once upon...")
print(seg.index)     # 1 (named access)
print(seg[0])        # 1 (also indexable)

# Unpacking
index, title, text = seg
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Creating tuples`
A comment: *Creating tuples.*
2. **Line 2:** `point = (3, 4)`
Assigns `(3, 4)` to `point`.
3. **Line 3:** `single = (42,) # Trailing comma required for single-element tuple`
Assigns `(42,)` `# Trailing comma required for single-element tuple` to `single`.
4. **Line 4:** `empty = ()`
Assigns `()` to `empty`.
5. **Line 5:** `# Parentheses are optional for packing`
A comment: *Parentheses are optional for packing.*
6. **Line 6:** `point = 3, 4 # Also a tuple`
Assigns `3, 4` `# Also a tuple` to `point`.
7. **Line 7:** `# Accessing (same as lists)`
A comment: *Accessing (same as lists).*
8. **Line 8:** `print(point[0]) # 3`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `# Cannot modify!`
A comment: *Cannot modify!.*
10. **Line 10:** `# point[0] = 5 # TypeError: 'tuple' does not support item assignment`
A comment: *point[0] = 5 # TypeError: 'tuple' does not support item assignment.*
11. **Line 11:** `# Named tuples – readable tuples with field names`
A comment: *Named tuples – readable tuples with field names.*
12. **Line 12:** `from collections import namedtuple`
Imports `namedtuple` from `collections`.
13. **Line 13:** `Segment = namedtuple("Segment", ["index", "title", "text"])`
Assigns `namedtuple("Segment", ["index", "title", "text"])` to `Segment`.
14. **Line 14:** `seg = Segment(index=1, title="Chapter 1", text="Once upon...")`

Assigns `Segment(index=1, title="Chapter 1", text="Once upon...")` to `seg`.

15. **Line 15:** `print(seg.index)` # 1 (named access)

Outputs text to the console. The primary way to display information during development and debugging.

16. **Line 16:** `print(seg[0])` # 1 (also indexable)

Outputs text to the console. The primary way to display information during development and debugging.

17. **Line 17:** `# Unpacking`

A comment: *Unpacking*.

18. **Line 18:** `index, title, text = seg`

`index, title, text = seg`

Tuples vs Lists

Feature	List	Tuple
Mutable?	Yes	No
Syntax	<code>[1, 2, 3]</code>	<code>(1, 2, 3)</code>
Performance	Slower	Faster (immutable, cached)
Dict key?	No	Yes
Use case	Dynamic collections	Fixed records, dict keys

6.3 Dictionaries — Key-Value Mapping

```
# Creating dicts
voice = {
    "id": "af_bella",
    "name": "Bella",
    "sid": 2,
    "language": "en",
}

# Accessing
print(voice["id"])           # "af_bella"
print(voice.get("speed", 1.0)) # 1.0 (default if key missing)
print(voice.get("speed"))     # None (no default)

# Modifying
voice["sample_rate"] = 24000 # Add/update
del voice["sid"]             # Delete
removed = voice.pop("language") # Remove and return

# Iteration
for key in voice:             # Keys (default)
    print(key)

for key, value in voice.items(): # Key-value pairs
    print(f"{key}: {value}")

for value in voice.values():    # Values only
    print(value)

# Merging
defaults = {"rate": 1.0, "pitch": 1.0}
overrides = {"rate": 1.5, "volume": 0.8}

# Python 3.9+ merge operator
merged = defaults | overrides # {"rate": 1.5, "pitch": 1.0, "volume": 0.8}

# Classic merge
merged = {**defaults, **overrides}

# setdefault – get or set in one step
config = {}
config.setdefault("threads", 4) # Sets and returns 4
config.setdefault("threads", 8) # Returns existing 4
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Creating dicts`

A comment: *Creating dicts.*

2. **Line 2:** `voice = {`

Assigns `{` to `voice`.

3. **Line 3:** `"id": "af_bella",`

`"id": "af_bella",`

4. **Line 4:** `"name": "Bella",`

`"name": "Bella",`

5. **Line 5:** `"sid": 2,`

`"sid": 2,`

6. **Line 6:** `"language": "en",`

`"language": "en",`

7. **Line 7:** `}`

`}`

8. **Line 8:** `# Accessing`

A comment: *Accessing.*

9. **Line 9:** `print(voice["id"]) # "af_bella"`

Outputs text to the console. The primary way to display information during development and debugging.

10. **Line 10:** `print(voice.get("speed", 1.0)) # 1.0 (default if key missing)`

Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `print(voice.get("speed")) # None (no default)`

Outputs text to the console. The primary way to display information during development and debugging.

12. **Line 12:** `# Modifying`

A comment: *Modifying.*

13. **Line 13:** `voice["sample_rate"] = 24000 # Add/update`

`voice["sample_rate"] = 24000 # Add/update`

14. **Line 14:** `del voice["sid"]` `# Delete`
`del voice["sid"]` `# Delete`
15. **Line 15:** `removed = voice.pop("language")` `# Remove and return`
Removes and returns the value for a key, or a default if not found.
16. **Line 16:** `# Iteration`
A comment: *Iteration*.
17. **Line 17:** `for key in voice:` `# Keys (default)`
Iterates over `voice`, assigning each element to `key`.
18. **Line 18:** `print(key)`
Outputs text to the console. The primary way to display information during development and debugging.
19. **Line 19:** `for key, value in voice.items():` `# Key-value pairs`
Returns a view of the dictionary's (key, value) pairs.
20. **Line 20:** `print(f"{key}: {value}")`
Outputs text to the console. The primary way to display information during development and debugging.
21. **Line 21:** `for value in voice.values():` `# Values only`
Returns a view of the dictionary's values.
22. **Line 22:** `print(value)`
Outputs text to the console. The primary way to display information during development and debugging.
23. **Line 23:** `# Merging`
A comment: *Merging*.
24. **Line 24:** `defaults = {"rate": 1.0, "pitch": 1.0}`
Assigns `{"rate": 1.0, "pitch": 1.0}` to `defaults`.
25. **Line 25:** `overrides = {"rate": 1.5, "volume": 0.8}`
Assigns `{"rate": 1.5, "volume": 0.8}` to `overrides`.
26. **Line 26:** `# Python 3.9+ merge operator`
A comment: *Python 3.9+ merge operator*.
27. **Line 27:** `merged = defaults | overrides` `# {"rate": 1.5, "pitch": 1.0, "volume": 0.8}`
Assigns `defaults | overrides` `# {"rate": 1.5, "pitch": 1.0, "volume": 0.8}` to `merged`.

28. Line 28: `# Classic merge`

A comment: *Classic merge*.

29. Line 29: `merged = {**defaults, **overrides}`

Assigns `{**defaults, **overrides}` to `merged` .

30. Line 30: `# setdefault – get or set in one step`

A comment: *setdefault – get or set in one step*.

31. Line 31: `config = {}`

Assigns `{}` to `config` .

32. Line 32: `config.setdefault("threads", 4) # Sets and returns 4`

Returns the value for a key, or inserts it with a default if not found.

33. Line 33: `config.setdefault("threads", 8) # Returns existing 4`

Returns the value for a key, or inserts it with a default if not found.

Dictionaries in AI/VS Settings

```
# From ai_voice_studio/settings.py
class Settings:
    def __init__(self):
        self._data: dict = {
            "theme": "system",
            "audio_mode": "page_with_h1",
            "recording.rate": 1.0,
            "recording.pitch": 1.0,
            "recording.volume": 1.0,
            "recording.output_format": "wav",
            "developer_mode": False,
        }

    def get(self, key: str, default=None):
        """Dot-notation get: settings.get('recording.rate')"""
        keys = key.split(".")
        obj = self._data
        for k in keys:
            if isinstance(obj, dict):
                obj = obj.get(k)
                if obj is None:
                    return default
            else:
                return default
        return obj

    def set(self, key: str, value):
        """Dot-notation set: settings.set('recording.rate', 1.5)"""
        keys = key.split(".")
        obj = self._data
        for k in keys[:-1]:
            obj = obj.setdefault(k, {})
        obj[keys[-1]] = value
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# From ai_voice_studio/settings.py`

A comment: *From ai_voice_studio/settings.py.*

2. **Line 2:** `class Settings:`

Defines class `Settings`.

3. **Line 3:** `def __init__(self):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

4. **Line 4:** `self._data: dict = {`

`self._data: dict = {`

5. **Line 5:** `"theme": "system",`

`"theme": "system",`

6. **Line 6:** `"audio_mode": "page_with_h1",`

`"audio_mode": "page_with_h1",`

7. **Line 7:** `"recording.rate": 1.0,`

`"recording.rate": 1.0,`

8. **Line 8:** `"recording.pitch": 1.0,`

`"recording.pitch": 1.0,`

9. **Line 9:** `"recording.volume": 1.0,`

`"recording.volume": 1.0,`

10. **Line 10:** `"recording.output_format": "wav",`

`"recording.output_format": "wav",`

11. **Line 11:** `"developer_mode": False,`

`"developer_mode": False,`

12. **Line 12:** `}`

`}`

13. **Line 13:** `def get(self, key: str, default=None):`

Defines function `get` accepting (self, key, default).

14. **Line 14:** `"""Dot-notation get: settings.get('recording.rate')"""`
Docstring: *Dot-notation get: settings.get('recording.rate')*. Documents the purpose of the following code.
15. **Line 15:** `keys = key.split(".")`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
16. **Line 16:** `obj = self._data`
Assigns `self._data` to `obj`.
17. **Line 17:** `for k in keys:`
Iterates over `keys`, assigning each element to `k`.
18. **Line 18:** `if isinstance(obj, dict):`
Checks if an object is an instance of a class. Preferred over `type()` for type checking.
19. **Line 19:** `obj = obj.get(k)`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
20. **Line 20:** `if obj is None:`
Checks condition: `obj is None`. The indented block runs only when this is `True`.
21. **Line 21:** `return default`
Returns `default` to the caller.
22. **Line 22:** `else:`
Fallback: executes when none of the above conditions matched.
23. **Line 23:** `return default`
Returns `default` to the caller.
24. **Line 24:** `return obj`
Returns `obj` to the caller.
25. **Line 25:** `def set(self, key: str, value):`
Defines function `set` accepting (self, key, value).
26. **Line 26:** `"""Dot-notation set: settings.set('recording.rate', 1.5)"""`
Docstring: *Dot-notation set: settings.set('recording.rate', 1.5)*. Documents the purpose of the following code.
27. **Line 27:** `keys = key.split(".")`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.

28. **Line 28:** `obj = self._data`
Assigns `self._data` to `obj`.

29. **Line 29:** `for k in keys[:-1]:`
Iterates over `keys[:-1]`, assigning each element to `k`.

30. **Line 30:** `obj = obj.setdefault(k, {})`
Returns the value for a key, or inserts it with a default if not found.

31. **Line 31:** `obj[keys[-1]] = value`
`obj[keys[-1]] = value`

6.4 Sets — Unordered, Unique Elements

```
# Creating sets
languages = {"en", "zh", "ja", "ko"}
empty_set = set() # {} creates an empty dict, not set!

# From a list (removes duplicates)
voices = ["af", "am", "af", "bf", "am"]
unique_voices = set(voices) # {"af", "am", "bf"}

# Set operations
a = {1, 2, 3, 4}
b = {3, 4, 5, 6}

print(a | b)    # Union: {1, 2, 3, 4, 5, 6}
print(a & b)    # Intersection: {3, 4}
print(a - b)    # Difference: {1, 2}
print(a ^ b)    # Symmetric difference: {1, 2, 5, 6}

# Membership (very fast - O(1) Lookup)
print(3 in a)   # True

# Frozenset - immutable set (can be dict key)
frozen = frozenset([1, 2, 3])
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# Creating sets`
A comment: *Creating sets*.
2. **Line 2:** `languages = {"en", "zh", "ja", "ko"}`
Assigns `{"en", "zh", "ja", "ko"}` to `languages`.
3. **Line 3:** `empty_set = set() # {} creates an empty dict, not set!`
Assigns `set() # {} creates an empty dict, not set!` to `empty_set`.
4. **Line 4:** `# From a list (removes duplicates)`
A comment: *From a list (removes duplicates)*.
5. **Line 5:** `voices = ["af", "am", "af", "bf", "am"]`
Assigns `["af", "am", "af", "bf", "am"]` to `voices`.
6. **Line 6:** `unique_voices = set(voices) # {"af", "am", "bf"}`
Assigns `set(voices) # {"af", "am", "bf"}` to `unique_voices`.
7. **Line 7:** `# Set operations`
A comment: *Set operations*.
8. **Line 8:** `a = {1, 2, 3, 4}`
Assigns `{1, 2, 3, 4}` to `a`.
9. **Line 9:** `b = {3, 4, 5, 6}`
Assigns `{3, 4, 5, 6}` to `b`.
10. **Line 10:** `print(a | b) # Union: {1, 2, 3, 4, 5, 6}`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `print(a & b) # Intersection: {3, 4}`
Outputs text to the console. The primary way to display information during development and debugging.
12. **Line 12:** `print(a - b) # Difference: {1, 2}`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `print(a ^ b) # Symmetric difference: {1, 2, 5, 6}`
Outputs text to the console. The primary way to display information during development and debugging.
14. **Line 14:** `# Membership (very fast - O(1) lookup)`

A comment: *Membership (very fast — $O(1)$ lookup).*

15. **Line 15:** `print(3 in a) # True`

Outputs text to the console. The primary way to display information during development and debugging.

16. **Line 16:** `# Frozenset — immutable set (can be dict key)`

A comment: *Frozenset — immutable set (can be dict key).*

17. **Line 17:** `frozen = frozenset([1, 2, 3])`

Assigns `frozenset([1, 2, 3])` to `frozen`.

Sets in AIVS

```
# From gui/recording_dialog.py — collecting unique TTS engine IDs
tts_ids = sorted({v["tts"] for v in self._voices})
# Set comprehension removes duplicates, sorted() makes it a list

# Checking which compute modes are available
available = set(compute.available_compute())
if "cuda" in available:
    # Show GPU option
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `# From gui/recording_dialog.py – collecting unique TTS engine IDs`
A comment: *From gui/recording_dialog.py – collecting unique TTS engine IDs.*
2. **Line 2:** `tts_ids = sorted({v["tts"] for v in self._voices})`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.
3. **Line 3:** `# Set comprehension removes duplicates, sorted() makes it a list`
A comment: *Set comprehension removes duplicates, sorted() makes it a list.*
4. **Line 4:** `# Checking which compute modes are available`
A comment: *Checking which compute modes are available.*
5. **Line 5:** `available = set(compute.available_compute())`
Assigns `set(compute.available_compute())` to `available`.
6. **Line 6:** `if "cuda" in available:`
Checks condition: `"cuda" in available`. The indented block runs only when this is `True`.
7. **Line 7:** `# Show GPU option`
A comment: *Show GPU option.*

6.5 Choosing the Right Collection

Need	Use	Why
Ordered sequence, mutable	<code>list</code>	Index access, append, sort
Ordered sequence, immutable	<code>tuple</code>	Hashable, slightly faster
Key-value mapping	<code>dict</code>	$O(1)$ lookup by key
Unique elements, fast membership	<code>set</code>	$O(1)$ <code>in</code> test
Record with named fields	<code>namedtuple</code>	Readable, immutable
Fixed-structure record	<code>@dataclass</code>	Mutable, type-hinted (Ch 10)

6.6 Nested Collections

```
# List of dicts – very common in real projects
voices = [
    {"id": "af", "name": "AF", "sid": 0},
    {"id": "am_adam", "name": "Adam", "sid": 1},
    {"id": "bf_emma", "name": "Emma", "sid": 2},
]

# Dict of lists
by_language = {
    "en": ["af", "am_adam", "bf_emma"],
    "zh": ["zf_xiaobei", "zm_009"],
}

# Deeply nested (AIVS catalog structure)
catalog_entry = {
    "id": "kokoro",
    "name": "Kokoro TTS",
    "languages": [
        {
            "code": "multi",
            "name": "Multilingual",
            "variants": [
                {
                    "id": "v1_1",
                    "artifact": "kokoro-multi-lang-v1_1.tar.bz2",
                    "voices": [
                        {"id": "af_maple", "name": "Maple", "sid": 0},
                        # ... 103 voices total
                    ],
                },
            ],
        },
    ],
}

# Accessing nested data
first_voice = catalog_entry["languages"][0]["variants"][0]["voices"][0]
print(first_voice["id"]) # "af_maple"
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `# List of dicts – very common in real projects`

A comment: *List of dicts – very common in real projects.*

2. **Line 2:** `voices = [`

Assigns `[` to `voices` .

3. **Line 3:** `{"id": "af", "name": "AF", "sid": 0},`

`{"id": "af", "name": "AF", "sid": 0},`

4. **Line 4:** `{"id": "am_adam", "name": "Adam", "sid": 1},`

`{"id": "am_adam", "name": "Adam", "sid": 1},`

5. **Line 5:** `{"id": "bf_emma", "name": "Emma", "sid": 2},`

`{"id": "bf_emma", "name": "Emma", "sid": 2},`

6. **Line 6:** `]`

`]`

7. **Line 7:** `# Dict of lists`

A comment: *Dict of lists.*

8. **Line 8:** `by_language = {`

Assigns `{` to `by_language` .

9. **Line 9:** `"en": ["af", "am_adam", "bf_emma"],`

`"en": ["af", "am_adam", "bf_emma"],`

10. **Line 10:** `"zh": ["zf_xiaobei", "zm_009"],`

`"zh": ["zf_xiaobei", "zm_009"],`

11. **Line 11:** `}`

`}`

12. **Line 12:** `# Deeply nested (AIVS catalog structure)`

A comment: *Deeply nested (AIVS catalog structure).*

13. **Line 13:** `catalog_entry = {`

Assigns `{` to `catalog_entry` .

```
14. Line 14: "id": "kokoro",
    "id": "kokoro",

15. Line 15: "name": "Kokoro TTS",
    "name": "Kokoro TTS",

16. Line 16: "languages": [
    "languages": [

17. Line 17: {
    {

18. Line 18: "code": "multi",
    "code": "multi",

19. Line 19: "name": "Multilingual",
    "name": "Multilingual",

20. Line 20: "variants": [
    "variants": [

21. Line 21: {
    {

22. Line 22: "id": "v1_1",
    "id": "v1_1",

23. Line 23: "artifact": "kokoro-multi-lang-v1_1.tar.bz2",
    "artifact": "kokoro-multi-lang-v1_1.tar.bz2",

24. Line 24: "voices": [
    "voices": [

25. Line 25: {"id": "af_maple", "name": "Maple", "sid": 0},
    {"id": "af_maple", "name": "Maple", "sid": 0},

26. Line 26: # ... 103 voices total
    A comment: ... 103 voices total.

27. Line 27: ],
```

```
    ],  
  
28. Line 28: },  
    },  
  
29. Line 29: ],  
    ],  
  
30. Line 30: },  
    },  
  
31. Line 31: ],  
    ],  
  
32. Line 32: }  
    }  
  
33. Line 33: # Accessing nested data  
    A comment: Accessing nested data.  
  
34. Line 34: first_voice = catalog_entry["languages"][0]["variants"][0]["voices"][0]  
    Assigns catalog_entry["languages"][0]["variants"][0]["voices"][0] to first_voice .  
  
35. Line 35: print(first_voice["id"]) # "af_maple"  
    Outputs text to the console. The primary way to display information during development and debugging.
```

6.7 Useful Collection Functions

```
# sorted() - returns a new sorted list
voices = ["bf", "af", "am"]
sorted_voices = sorted(voices)           # ["af", "am", "bf"]
sorted_voices = sorted(voices, reverse=True) # ["bf", "am", "af"]
sorted_voices = sorted(voices, key=len)    # ["af", "am", "bf"]

# sorted with dict key
voices_sorted = sorted(voice_list, key=lambda v: v.get("name", ""))

# min, max, sum
sids = [v["sid"] for v in voices]
print(min(sids), max(sids), sum(sids))

# any() and all()
has_gpu = any(detected.values())        # True if any value is True
all_ok = all(status.values())           # True if all values are True

# zip() - iterate multiple sequences in parallel
names = ["Piper", "Kokoro"]
counts = [153, 103]
for name, count in zip(names, counts):
    print(f"{name}: {count} voices")

# reversed() - returns an iterator
for v in reversed([1, 2, 3]):
    print(v) # 3, 2, 1

# enumerate() with custom start
for i, voice in enumerate(voices, start=1):
    print(f"Voice #{i}: {voice}")
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# sorted() – returns a new sorted list`
A comment: *sorted()* – returns a new sorted list.
2. **Line 2:** `voices = ["bf", "af", "am"]`
Assigns `["bf", "af", "am"]` to `voices`.
3. **Line 3:** `sorted_voices = sorted(voices) # ["af", "am", "bf"]`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.
4. **Line 4:** `sorted_voices = sorted(voices, reverse=True) # ["bf", "am", "af"]`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.
5. **Line 5:** `sorted_voices = sorted(voices, key=len) # ["af", "am", "bf"]`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.
6. **Line 6:** `# sorted with dict key`
A comment: *sorted with dict key*.
7. **Line 7:** `voices_sorted = sorted(voice_list, key=lambda v: v.get("name", ""))`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.
8. **Line 8:** `# min, max, sum`
A comment: *min, max, sum*.
9. **Line 9:** `sids = [v["sid"] for v in voices]`
Assigns `[v["sid"] for v in voices]` to `sids`.
10. **Line 10:** `print(min(sids), max(sids), sum(sids))`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `# any() and all()`
A comment: *any() and all()*.
12. **Line 12:** `has_gpu = any(detected.values()) # True if any value is True`
Returns a view of the dictionary's values.
13. **Line 13:** `all_ok = all(status.values()) # True if all values are True`
Returns a view of the dictionary's values.
14. **Line 14:** `# zip() – iterate multiple sequences in parallel`
A comment: *zip()* – iterate multiple sequences in parallel.

15. **Line 15:** `names = ["Piper", "Kokoro"]`
Assigns `["Piper", "Kokoro"]` to `names`.
16. **Line 16:** `counts = [153, 103]`
Assigns `[153, 103]` to `counts`.
17. **Line 17:** `for name, count in zip(names, counts):`
Combines multiple iterables element-wise. Stops at the shortest input. Used for parallel iteration.
18. **Line 18:** `print(f"{name}: {count} voices")`
Outputs text to the console. The primary way to display information during development and debugging.
19. **Line 19:** `# reversed() – returns an iterator`
A comment: *reversed() – returns an iterator.*
20. **Line 20:** `for v in reversed([1, 2, 3]):`
Returns a reverse iterator over a sequence. Does not modify the original.
21. **Line 21:** `print(v) # 3, 2, 1`
Outputs text to the console. The primary way to display information during development and debugging.
22. **Line 22:** `# enumerate() with custom start`
A comment: *enumerate() with custom start.*
23. **Line 23:** `for i, voice in enumerate(voices, start=1):`
Adds a counter to an iterable, returning (index, value) pairs. Preferred over manual index tracking.
24. **Line 24:** `print(f"Voice #{i}: {voice}")`
Outputs text to the console. The primary way to display information during development and debugging.
-

6.8 collections Module

```
from collections import Counter, defaultdict, deque, OrderedDict

# Counter – count occurrences
text = "hello world hello python hello"
word_counts = Counter(text.split())
print(word_counts)           # Counter({'hello': 3, 'world': 1, ...})
print(word_counts.most_common(2)) # [('hello', 3), ('world', 1)]

# defaultdict – dict with default values
voices_by_tts = defaultdict(list)
for v in all_voices:
    voices_by_tts[v["tts"]].append(v) # No KeyError if key missing

# deque – double-ended queue (fast insert/delete at both ends)
history = deque(maxlen=10) # Auto-removes oldest
history.append("action1")
history.appendleft("action0")
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `from collections import Counter, defaultdict, deque, OrderedDict`
Imports `Counter, defaultdict, deque, OrderedDict` from `collections`.
2. **Line 2:** `# Counter – count occurrences`
A comment: *Counter – count occurrences*.
3. **Line 3:** `text = "hello world hello python hello"`
Assigns `"hello world hello python hello"` to `text`.
4. **Line 4:** `word_counts = Counter(text.split())`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
5. **Line 5:** `print(word_counts) # Counter({'hello': 3, 'world': 1, ...})`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `print(word_counts.most_common(2)) # [('hello', 3), ('world', 1)]`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# defaultdict – dict with default values`
A comment: *defaultdict – dict with default values*.
8. **Line 8:** `voices_by_tts = defaultdict(list)`
Assigns `defaultdict(list)` to `voices_by_tts`.
9. **Line 9:** `for v in all_voices:`
Iterates over `all_voices`, assigning each element to `v`.
10. **Line 10:** `voices_by_tts[v["tts"]].append(v) # No KeyError if key missing`
Adds a single item to the end of the list.
11. **Line 11:** `# deque – double-ended queue (fast insert/delete at both ends)`
A comment: *deque – double-ended queue (fast insert/delete at both ends)*.
12. **Line 12:** `history = deque(maxlen=10) # Auto-removes oldest`
Assigns `deque(maxlen=10)` # Auto-removes oldest to `history`.
13. **Line 13:** `history.append("action1")`
Adds a single item to the end of the list.
14. **Line 14:** `history.appendleft("action0")`

```
history.appendleft("action0")
```

6.9 Key Takeaways

- **Lists** for ordered, mutable sequences (most common)
- **Tuples** for fixed records and dict keys
- **Dicts** for key-value data (the backbone of AIVS settings and catalog)
- **Sets** for unique elements and fast membership testing
- **Copy correctly** — `b = a` creates a reference, not a copy
- **Dict.get()** with a default avoids `KeyError`
- The AIVS catalog is a deeply nested dict structure: TTS → language → variant → voice

 **Exercise: Exercise 6.1:** Given a list of voice dictionaries, write a dict comprehension that groups them by language code. Result: `{"en": [...], "zh": [...], ...}`

 **Exercise: Exercise 6.2:** Implement a simple `LRUCache` class using `OrderedDict` with a `maxlen` parameter.

File I/O

Learning Objectives

- Read and write text and binary files
 - Understand file modes, encoding, and context managers
 - Use the `pathlib` module for modern path handling
 - See how AIVS reads/writes JSON settings and project files
-

7.1 Opening and Closing Files

```
# The "classic" way (don't do this in modern Python)
f = open("data.txt", "r")
content = f.read()
f.close() # Must close manually!

# The correct way – context manager (auto-closes)
with open("data.txt", "r") as f:
    content = f.read()

# f is automatically closed here, even if an exception occurs
```

Explain Line by Line Code Description (Block 1)

- Line 1:** `# The "classic" way (don't do this in modern Python)`
A comment: *The "classic" way (don't do this in modern Python).*
- Line 2:** `f = open("data.txt", "r")`
Opens a file and returns a file object. Supports modes: 'r' (read), 'w' (write), 'a' (append), 'b' (binary).
- Line 3:** `content = f.read()`
Reads the entire file contents as a string.
- Line 4:** `f.close() # Must close manually!`
Closes the file and flushes the write buffer. Prefer using 'with' instead.
- Line 5:** `# The correct way – context manager (auto-closes)`
A comment: *The correct way – context manager (auto-closes).*
- Line 6:** `with open("data.txt", "r") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
- Line 7:** `content = f.read()`
Reads the entire file contents as a string.
- Line 8:** `# f is automatically closed here, even if an exception occurs`
A comment: *f is automatically closed here, even if an exception occurs.*

File Modes

Mode	Description
"r"	Read (default). File must exist.
"w"	Write. Creates or truncates file.
"a"	Append. Creates if doesn't exist.
"x"	Exclusive create. Fails if file exists.
"r+"	Read + write. File must exist.
"rb"	Read binary.
"wb"	Write binary.

7.2 Reading Files

```
# Read entire file
with open("config.json", "r", encoding="utf-8") as f:
    content = f.read() # Returns a single string

# Read line by line (memory-efficient for large files)
with open("segments.txt", "r", encoding="utf-8") as f:
    for line in f:
        line = line.rstrip("\n") # Strip trailing newline
        print(line)

# Read all lines into a list
with open("segments.txt", "r", encoding="utf-8") as f:
    lines = f.readlines() # List of strings (with \n)

# Read as bytes (for binary files like audio)
with open("audio.wav", "rb") as f:
    data = f.read() # Returns bytes object
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Read entire file`
A comment: *Read entire file.*
2. **Line 2:** `with open("config.json", "r", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
3. **Line 3:** `content = f.read() # Returns a single string`
Reads the entire file contents as a string.
4. **Line 4:** `# Read line by line (memory-efficient for large files)`
A comment: *Read line by line (memory-efficient for large files).*
5. **Line 5:** `with open("segments.txt", "r", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
6. **Line 6:** `for line in f:`
Iterates over `f`, assigning each element to `line`.
7. **Line 7:** `line = line.rstrip("\n") # Strip trailing newline`
Removes trailing whitespace only.
8. **Line 8:** `print(line)`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `# Read all lines into a list`
A comment: *Read all lines into a list.*
10. **Line 10:** `with open("segments.txt", "r", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
11. **Line 11:** `lines = f.readlines() # List of strings (with \n)`
Reads all lines and returns them as a list of strings.
12. **Line 12:** `# Read as bytes (for binary files like audio)`
A comment: *Read as bytes (for binary files like audio).*
13. **Line 13:** `with open("audio.wav", "rb") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
14. **Line 14:** `data = f.read() # Returns bytes object`

Reads the entire file contents as a string.

7.3 Writing Files

```
# Write text
with open("output.txt", "w", encoding="utf-8") as f:
    f.write("Hello, world!\n")
    f.write("Second line\n")

# writelines (no automatic newlines)
with open("output.txt", "w", encoding="utf-8") as f:
    lines = ["line 1\n", "line 2\n", "line 3\n"]
    f.writelines(lines)

# Append mode
with open("log.txt", "a", encoding="utf-8") as f:
    f.write("New log entry\n")

# Print to file
with open("report.txt", "w") as f:
    print("Status: OK", file=f)
    print("Voices: 42", file=f)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Write text`
A comment: *Write text*.
2. **Line 2:** `with open("output.txt", "w", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
3. **Line 3:** `f.write("Hello, world!\n")`
Writes a string to the file. In text mode, does not add a newline automatically.
4. **Line 4:** `f.write("Second line\n")`
Writes a string to the file. In text mode, does not add a newline automatically.
5. **Line 5:** `# writelines (no automatic newlines)`
A comment: *writelines (no automatic newlines)*.
6. **Line 6:** `with open("output.txt", "w", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
7. **Line 7:** `lines = ["line 1\n", "line 2\n", "line 3\n"]`
Assigns `["line 1\n", "line 2\n", "line 3\n"]` to `lines`.
8. **Line 8:** `f.writelines(lines)`
Writes a list of strings to the file without adding newlines.
9. **Line 9:** `# Append mode`
A comment: *Append mode*.
10. **Line 10:** `with open("log.txt", "a", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
11. **Line 11:** `f.write("New log entry\n")`
Writes a string to the file. In text mode, does not add a newline automatically.
12. **Line 12:** `# Print to file`
A comment: *Print to file*.
13. **Line 13:** `with open("report.txt", "w") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
14. **Line 14:** `print("Status: OK", file=f)`
Outputs text to the console. The primary way to display information during development and debugging.

15. **Line 15:** `print("Voices: 42", file=f)`

Outputs text to the console. The primary way to display information during development and debugging.

7.4 JSON File I/O

JSON is the backbone of AIVS — settings, project files, and catalog data all use it:

```
import json

# Reading JSON
with open("settings.json", "r", encoding="utf-8") as f:
    settings = json.load(f) # Returns a Python dict

# Writing JSON (with nice formatting)
with open("settings.json", "w", encoding="utf-8") as f:
    json.dump(settings, f, indent=2, ensure_ascii=False)

# JSON string ↔ Python object
text = json.dumps({"rate": 1.5, "pitch": 1.0}, indent=2)
obj = json.loads(text)
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `import json`
Imports the `json` module.
2. **Line 2:** `# Reading JSON`
A comment: *Reading JSON*.
3. **Line 3:** `with open("settings.json", "r", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
4. **Line 4:** `settings = json.load(f) # Returns a Python dict`
Deserializes JSON from a file into a Python object.
5. **Line 5:** `# Writing JSON (with nice formatting)`
A comment: *Writing JSON (with nice formatting)*.
6. **Line 6:** `with open("settings.json", "w", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
7. **Line 7:** `json.dump(settings, f, indent=2, ensure_ascii=False)`
Serializes a Python object to JSON and writes to a file.
8. **Line 8:** `# JSON string ↔ Python object`
A comment: *JSON string ↔ Python object*.
9. **Line 9:** `text = json.dumps({"rate": 1.5, "pitch": 1.0}, indent=2)`
Serializes a Python object to a JSON string.
10. **Line 10:** `obj = json.loads(text)`
Deserializes a JSON string into a Python object.

AIVS Settings Pattern

```
# From ai_voice_studio/settings.py
def load(self) -> None:
    """Load settings from disk, merging with defaults."""
    if os.path.isfile(self._path):
        with open(self._path, "r", encoding="utf-8") as f:
            stored = json.load(f)
            # Merge: defaults + stored values
            self._data = {**self._defaults, **stored}

def save(self) -> None:
    """Save current settings to disk."""
    os.makedirs(os.path.dirname(self._path), exist_ok=True)
    with open(self._path, "w", encoding="utf-8") as f:
        json.dump(self._data, f, indent=2, ensure_ascii=False)
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# From ai_voice_studio/settings.py`
A comment: *From ai_voice_studio/settings.py*.
2. **Line 2:** `def load(self) -> None:`
Defines function `load` accepting (self), returning `None`.
3. **Line 3:** `"""Load settings from disk, merging with defaults."""`
Docstring: *Load settings from disk, merging with defaults..* Documents the purpose of the following code.
4. **Line 4:** `if os.path.isfile(self._path):`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
5. **Line 5:** `with open(self._path, "r", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
6. **Line 6:** `stored = json.load(f)`
Deserializes JSON from a file into a Python object.
7. **Line 7:** `# Merge: defaults + stored values`
A comment: *Merge: defaults + stored values*.
8. **Line 8:** `self._data = {**self._defaults, **stored}`
Stores `{**self._defaults, **stored}` in instance attribute `_data`.
9. **Line 9:** `def save(self) -> None:`
Defines function `save` accepting (self), returning `None`.
10. **Line 10:** `"""Save current settings to disk."""`
Docstring: *Save current settings to disk..* Documents the purpose of the following code.
11. **Line 11:** `os.makedirs(os.path.dirname(self._path), exist_ok=True)`
Recursively creates directories. `exist_ok=True` prevents errors if directory exists.
12. **Line 12:** `with open(self._path, "w", encoding="utf-8") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
13. **Line 13:** `json.dump(self._data, f, indent=2, ensure_ascii=False)`
Serializes a Python object to JSON and writes to a file.

7.5 Binary File I/O

```
# Writing binary data (audio samples)
import numpy as np

samples = np.array([0, 1000, -1000, 2000], dtype=np.int16)
with open("output.raw", "wb") as f:
    f.write(samples.tobytes())

# Reading binary data
with open("output.raw", "rb") as f:
    raw = f.read()
samples = np.frombuffer(raw, dtype=np.int16)
```

Explain Line by Line Code Description (Block 6)

- Line 1:** `# Writing binary data (audio samples)`
A comment: *Writing binary data (audio samples).*
- Line 2:** `import numpy as np`
Imports the `numpy as np` module.
- Line 3:** `samples = np.array([0, 1000, -1000, 2000], dtype=np.int16)`
Assigns `np.array([0, 1000, -1000, 2000], dtype=np.int16)` to `samples`.
- Line 4:** `with open("output.raw", "wb") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
- Line 5:** `f.write(samples.tobytes())`
Writes a string to the file. In text mode, does not add a newline automatically.
- Line 6:** `# Reading binary data`
A comment: *Reading binary data.*
- Line 7:** `with open("output.raw", "rb") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
- Line 8:** `raw = f.read()`
Reads the entire file contents as a string.
- Line 9:** `samples = np.frombuffer(raw, dtype=np.int16)`
Assigns `np.frombuffer(raw, dtype=np.int16)` to `samples`.

7.6 pathlib — Modern Path Handling

```
from pathlib import Path

# Create Path objects
p = Path("ai_voice_studio")
p = Path.home() / "AIVoiceStudio"
p = Path(".") / "models" / "piper"

# Path properties
print(p.name)      # "piper" (filename)
print(p.stem)      # "piper" (without extension)
print(p.suffix)     # "" (extension)
print(p.parent)     # PosixPath('models')
print(p.exists())   # True/False
print(p.is_file())  # True/False
print(p.is_dir())   # True/False

# Read/write text files
text = Path("notes.txt").read_text(encoding="utf-8")
Path("output.txt").write_text("Hello!\n", encoding="utf-8")

# Read/write bytes
data = Path("image.png").read_bytes()
Path("copy.png").write_bytes(data)

# Create directories
Path("output/audio").mkdir(parents=True, exist_ok=True)

# Glob pattern matching
py_files = list(Path(".").glob("**/*.py")) # Recursive
test_files = list(Path("tests").glob("test_*.py"))

# Iterating directory
for p in Path("models").iterdir():
    if p.is_dir():
        print(f"Directory: {p.name}")
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `from pathlib import Path`
Imports `Path` from `pathlib`.
2. **Line 2:** `# Create Path objects`
A comment: *Create Path objects.*
3. **Line 3:** `p = Path("ai_voice_studio")`
Assigns `Path("ai_voice_studio")` to `p`.
4. **Line 4:** `p = Path.home() / "AIVoiceStudio"`
Assigns `Path.home() / "AIVoiceStudio"` to `p`.
5. **Line 5:** `p = Path(".") / "models" / "piper"`
Assigns `Path(".") / "models" / "piper"` to `p`.
6. **Line 6:** `# Path properties`
A comment: *Path properties.*
7. **Line 7:** `print(p.name) # "piper" (filename)`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `print(p.stem) # "piper" (without extension)`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(p.suffix) # "" (extension)`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `print(p.parent) # PosixPath('models')`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `print(p.exists()) # True/False`
Outputs text to the console. The primary way to display information during development and debugging.
12. **Line 12:** `print(p.is_file()) # True/False`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `print(p.is_dir()) # True/False`
Outputs text to the console. The primary way to display information during development and debugging.
14. **Line 14:** `# Read/write text files`

A comment: *Read/write text files.*

15. **Line 15:** `text = Path("notes.txt").read_text(encoding="utf-8")`

Assigns `Path("notes.txt").read_text(encoding="utf-8")` to `text` .

16. **Line 16:** `Path("output.txt").write_text("Hello!\n", encoding="utf-8")`

Calls function `Path()` .

17. **Line 17:** `# Read/write bytes`

A comment: *Read/write bytes.*

18. **Line 18:** `data = Path("image.png").read_bytes()`

Assigns `Path("image.png").read_bytes()` to `data` .

19. **Line 19:** `Path("copy.png").write_bytes(data)`

Calls function `Path()` .

20. **Line 20:** `# Create directories`

A comment: *Create directories.*

21. **Line 21:** `Path("output/audio").mkdir(parents=True, exist_ok=True)`

Returns a list of attributes and methods of an object. Useful for exploration and debugging.

22. **Line 22:** `# Glob pattern matching`

A comment: *Glob pattern matching.*

23. **Line 23:** `py_files = list(Path(".").glob("**/*.py")) # Recursive`

Assigns `list(Path(".").glob("**/*.py")) # Recursive` to `py_files` .

24. **Line 24:** `test_files = list(Path("tests").glob("test_*.py"))`

Assigns `list(Path("tests").glob("test_*.py"))` to `test_files` .

25. **Line 25:** `# Iterating directory`

A comment: *Iterating directory.*

26. **Line 26:** `for p in Path("models").iterdir():`

Returns a list of attributes and methods of an object. Useful for exploration and debugging.

27. **Line 27:** `if p.is_dir():`

Returns a list of attributes and methods of an object. Useful for exploration and debugging.

28. **Line 28:** `print(f"Directory: {p.name}")`

Outputs text to the console. The primary way to display information during development and debugging.

7.7 os.path vs pathlib

Operation	os.path	pathlib
Join paths	<code>os.path.join(a, b)</code>	<code>Path(a) / b</code>
Exists?	<code>os.path.exists(p)</code>	<code>Path(p).exists()</code>
Get name	<code>os.path.basename(p)</code>	<code>Path(p).name</code>
Get dir	<code>os.path.dirname(p)</code>	<code>Path(p).parent</code>
Makedirs	<code>os.makedirs(p, exist_ok=True)</code>	<code>Path(p).mkdir(parents=True, exist_ok=True)</code>

The AIVS project uses `os.path` for cross-platform compatibility with older code patterns, but `pathlib` is preferred for new code.

7.8 File I/O in AIVS

```
# From project.py - atomic JSON write
def save_project(project_dir: str, data: dict) -> None:
    """Save project.json atomically (write to temp, then rename)."""
    import tempfile
    path = os.path.join(project_dir, "project.json")
    # Write to a temp file first (atomic on same filesystem)
    fd, tmp = tempfile.mkstemp(dir=project_dir, suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2, ensure_ascii=False)
        # Atomic rename (on same filesystem)
        os.replace(tmp, path)
    except:
        os.unlink(tmp)
        raise

# From tts/downloader.py - streaming download with progress
def _download_file(url: str, dest: str, progress=None) -> None:
    """Download a file with progress reporting."""
    import requests
    with requests.get(url, stream=True) as resp:
        total = int(resp.headers.get("content-length", 0))
        downloaded = 0
        with open(dest, "wb") as f:
            for chunk in resp.iter_content(chunk_size=8192):
                f.write(chunk)
                downloaded += len(chunk)
                if progress:
                    progress(downloaded, total)
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# From project.py - atomic JSON write`
A comment: *From project.py - atomic JSON write.*
2. **Line 2:** `def save_project(project_dir: str, data: dict) -> None:`
Defines function `save_project` accepting `(project_dir, data)`, returning `None`.
3. **Line 3:** `"""Save project.json atomically (write to temp, then rename)."""`
Docstring: *Save project.json atomically (write to temp, then rename).*.. Documents the purpose of the following code.
4. **Line 4:** `import tempfile`
Imports the `tempfile` module.
5. **Line 5:** `path = os.path.join(project_dir, "project.json")`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
6. **Line 6:** `# Write to a temp file first (atomic on same filesystem)`
A comment: *Write to a temp file first (atomic on same filesystem).*
7. **Line 7:** `fd, tmp = tempfile.mkstemp(dir=project_dir, suffix=".tmp")`
`fd, tmp = tempfile.mkstemp(dir=project_dir, suffix=".tmp")`
8. **Line 8:** `try:`
Begins exception handling. Code in this block is monitored for errors.
9. **Line 9:** `with os.fdopen(fd, "w", encoding="utf-8") as f:`
Opens a file and returns a file object. Supports modes: 'r' (read), 'w' (write), 'a' (append), 'b' (binary).
10. **Line 10:** `json.dump(data, f, indent=2, ensure_ascii=False)`
Serializes a Python object to JSON and writes to a file.
11. **Line 11:** `# Atomic rename (on same filesystem)`
A comment: *Atomic rename (on same filesystem).*
12. **Line 12:** `os.replace(tmp, path)`
Returns a copy with all occurrences of old replaced by new. Strings are immutable.
13. **Line 13:** `except:`
Catches exceptions. Use specific exception types when possible.
14. **Line 14:** `os.unlink(tmp)`

```
os.unlink(tmp)
```

15. Line 15: `raise`

```
raise
```

16. Line 16: `# From tts/downloader.py – streaming download with progress`

A comment: *From tts/downloader.py – streaming download with progress.*

17. Line 17: `def _download_file(url: str, dest: str, progress=None) -> None:`

Defines function `_download_file` accepting (url, dest, progress), returning `None`.

18. Line 18: `"""Download a file with progress reporting."""`

Docstring: *Download a file with progress reporting..* Documents the purpose of the following code.

19. Line 19: `import requests`

Imports the `requests` module.

20. Line 20: `with requests.get(url, stream=True) as resp:`

Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

21. Line 21: `total = int(resp.headers.get("content-length", 0))`

Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

22. Line 22: `downloaded = 0`

Assigns `0` to `downloaded`.

23. Line 23: `with open(dest, "wb") as f:`

Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.

24. Line 24: `for chunk in resp.iter_content(chunk_size=8192):`

Iterates over `resp.iter_content(chunk_size=8192)`, assigning each element to `chunk`.

25. Line 25: `f.write(chunk)`

Writes a string to the file. In text mode, does not add a newline automatically.

26. Line 26: `downloaded += len(chunk)`

Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

27. Line 27: `if progress:`

Checks condition: `progress`. The indented block runs only when this is `True`.

28. Line 28: `progress(downloaded, total)`

Calls function `progress()` .

7.9 Temporary Files

```
import tempfile

# Temporary file (auto-deleted when closed)
with tempfile.NamedTemporaryFile(suffix=".wav", delete=True) as tmp:
    tmp.write(audio_data)
    tmp.flush()
    # Process the file...
# File is deleted automatically

# Temporary directory
with tempfile.TemporaryDirectory() as tmpdir:
    # Work with tmpdir...
    pass
# Directory and all contents deleted automatically
```

Explain Line by Line Code Description (Block 9)

1. **Line 1:** `import tempfile`
Imports the `tempfile` module.
2. **Line 2:** `# Temporary file (auto-deleted when closed)`
A comment: *Temporary file (auto-deleted when closed).*
3. **Line 3:** `with tempfile.NamedTemporaryFile(suffix=".wav", delete=True) as tmp:`
Creates a temporary file that is deleted when closed.
4. **Line 4:** `tmp.write(audio_data)`
Writes a string to the file. In text mode, does not add a newline automatically.
5. **Line 5:** `tmp.flush()`
`tmp.flush()`
6. **Line 6:** `# Process the file...`
A comment: *Process the file....*
7. **Line 7:** `# File is deleted automatically`
A comment: *File is deleted automatically.*
8. **Line 8:** `# Temporary directory`
A comment: *Temporary directory.*
9. **Line 9:** `with tempfile.TemporaryDirectory() as tmpdir:`
Context manager: `tempfile.TemporaryDirectory() as tmpdir`. Automatically manages resource cleanup.
10. **Line 10:** `# Work with tmpdir...`
A comment: *Work with tmpdir....*
11. **Line 11:** `pass`
Placeholder: does nothing.
12. **Line 12:** `# Directory and all contents deleted automatically`
A comment: *Directory and all contents deleted automatically.*

7.10 Key Takeaways

- **Always use** `with` **statements** for file I/O (context managers auto-close)
- **Always specify** `encoding="utf-8"` for text files
- **JSON** is the standard format for settings and project data in AIVS
- **Atomic writes** (write to temp file, then rename) prevent corruption
- **pathlib** is the modern way to handle file paths
- For streaming large files, read in **chunks** to avoid loading everything into memory

 **Exercise: Exercise 7.1:** Write a function that reads a text file and returns a `Counter` of word frequencies. Test it on the project's README.md.

 **Exercise: Exercise 7.2:** Implement a `safe_json_write(path, data)` function that writes JSON atomically with proper error handling.

Error Handling

Learning Objectives

- Understand Python's exception hierarchy
 - Use try/except/else/finally for robust error handling
 - Create custom exceptions
 - Apply best practices for exception handling in AIVS
-

8.1 The Exception Hierarchy

```
BaseException
├── KeyboardInterrupt
├── SystemExit
├── GeneratorExit
├── Exception
│   ├── ValueError
│   ├── TypeError
│   ├── KeyError
│   ├── FileNotFoundError
│   ├── OSError
│   │   ├── FileNotFoundError
│   │   ├── PermissionError
│   │   └── TimeoutError
│   ├── RuntimeError
│   ├── IndexError
│   ├── AttributeError
│   └── ... (hundreds more)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `BaseException`
`BaseException`
2. **Line 2:** `KeyboardInterrupt`
`KeyboardInterrupt`
3. **Line 3:** `SystemExit`
`SystemExit`
4. **Line 4:** `GeneratorExit`
`GeneratorExit`
5. **Line 5:** `Exception`
`Exception`
6. **Line 6:** `ValueError`
Raised when a function receives an argument of the right type but inappropriate value.
7. **Line 7:** `TypeError`
Raised when an operation is applied to an object of inappropriate type.
8. **Line 8:** `KeyError`
Raised when a dictionary key is not found.
9. **Line 9:** `FileNotFoundError`
Raised when a file or directory is requested but doesn't exist.
10. **Line 10:** `OSError`
`OSError`
11. **Line 11:** `FileNotFoundError`
Raised when a file or directory is requested but doesn't exist.
12. **Line 12:** `PermissionError`
`PermissionError`
13. **Line 13:** `TimeoutError`
`TimeoutError`

- 14. **Line 14:** `RuntimeError`
Raised when an error doesn't fall into any other category.
- 15. **Line 15:** `IndexError`
Raised when a sequence index is out of range.
- 16. **Line 16:** `AttributeError`
Raised when an attribute reference or assignment fails.
- 17. **Line 17:** `... (hundreds more)`
`... (hundreds more)`

8.2 try / except / else / finally

```
try:
    # Code that might fail
    result = 10 / 0
except ZeroDivisionError:
    # Handle the specific error
    print("Cannot divide by zero")
else:
    # Runs only if NO exception occurred
    print(f"Result: {result}")
finally:
    # ALWAYS runs, whether or not an exception occurred
    print("Cleanup")

# Catching multiple exception types
try:
    load_voice_model(path)
except FileNotFoundError:
    print("Model not found – download it first")
except PermissionError:
    print("Permission denied")
except OSError as exc:
    print(f"OS error: {exc}")

# Catching multiple types in one handler
try:
    download_model(url)
except (ConnectionError, TimeoutError) as exc:
    print(f"Network error: {exc}")

# Catching everything (use sparingly!)
try:
    risky_operation()
except Exception:
    print("Something went wrong")
# ⚠️ Don't catch BaseException – it kills Ctrl+C!
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `try:`
Begins exception handling. Code in this block is monitored for errors.
2. **Line 2:** `# Code that might fail`
A comment: *Code that might fail.*
3. **Line 3:** `result = 10 / 0`
Assigns `10 / 0` to `result`.
4. **Line 4:** `except ZeroDivisionError:`
Catches `ZeroDivisionError` exceptions.
5. **Line 5:** `# Handle the specific error`
A comment: *Handle the specific error.*
6. **Line 6:** `print("Cannot divide by zero")`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `else:`
Fallback: executes when none of the above conditions matched.
8. **Line 8:** `# Runs only if NO exception occurred`
A comment: *Runs only if NO exception occurred.*
9. **Line 9:** `print(f"Result: {result}")`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `finally:`
Always executes for cleanup code, whether or not an exception occurred.
11. **Line 11:** `# ALWAYS runs, whether or not an exception occurred`
A comment: *ALWAYS runs, whether or not an exception occurred.*
12. **Line 12:** `print("Cleanup")`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `# Catching multiple exception types`
A comment: *Catching multiple exception types.*
14. **Line 14:** `try:`
Begins exception handling. Code in this block is monitored for errors.

15. **Line 15:** `load_voice_model(path)`
Calls function `load_voice_model()`.
16. **Line 16:** `except FileNotFoundError:`
Raised when a file or directory is requested but doesn't exist.
17. **Line 17:** `print("Model not found – download it first")`
Outputs text to the console. The primary way to display information during development and debugging.
18. **Line 18:** `except PermissionError:`
Catches `PermissionError` exceptions.
19. **Line 19:** `print("Permission denied")`
Outputs text to the console. The primary way to display information during development and debugging.
20. **Line 20:** `except OSError as exc:`
Catches `OSError` exceptions.
21. **Line 21:** `print(f"OS error: {exc}")`
Outputs text to the console. The primary way to display information during development and debugging.
22. **Line 22:** `# Catching multiple types in one handler`
A comment: *Catching multiple types in one handler.*
23. **Line 23:** `try:`
Begins exception handling. Code in this block is monitored for errors.
24. **Line 24:** `download_model(url)`
Calls function `download_model()`.
25. **Line 25:** `except (ConnectionError, TimeoutError) as exc:`
`except (ConnectionError, TimeoutError) as exc:`
26. **Line 26:** `print(f"Network error: {exc}")`
Outputs text to the console. The primary way to display information during development and debugging.
27. **Line 27:** `# Catching everything (use sparingly!)`
A comment: *Catching everything (use sparingly!).*
28. **Line 28:** `try:`
Begins exception handling. Code in this block is monitored for errors.

29. **Line 29:** `risky_operation()`
Calls function `risky_operation()` .
30. **Line 30:** `except Exception:`
Catches all exceptions that inherit from `Exception`. Use as a last resort.
31. **Line 31:** `print("Something went wrong")`
Outputs text to the console. The primary way to display information during development and debugging.
32. **Line 32:** `# ⚠ Don't catch BaseException — it kills Ctrl+C!`
A comment: ⚠ *Don't catch BaseException — it kills Ctrl+C!*.

8.3 Raising Exceptions

```
# Raise with a message
raise ValueError("Invalid sample rate: 0")

# Re-raise the current exception (preserves traceback)
try:
    download_model(url)
except DownloadError as exc:
    log.error("Download failed: {exc}")
    raise # Re-raise with original traceback

# Exception chaining
try:
    result = json.loads(data)
except json.JSONDecodeError as exc:
    raise SettingsError("Invalid settings file") from exc
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Raise with a message`
A comment: *Raise with a message.*
2. **Line 2:** `raise ValueError("Invalid sample rate: 0")`
Raised when a function receives an argument of the right type but inappropriate value.
3. **Line 3:** `# Re-raise the current exception (preserves traceback)`
A comment: *Re-raise the current exception (preserves traceback).*
4. **Line 4:** `try:`
Begins exception handling. Code in this block is monitored for errors.
5. **Line 5:** `download_model(url)`
Calls function `download_model()`.
6. **Line 6:** `except DownloadError as exc:`
Catches `DownloadError` exceptions.
7. **Line 7:** `log.error("Download failed: {exc}")`
`log.error("Download failed: {exc}")`
8. **Line 8:** `raise # Re-raise with original traceback`
Manually raises an exception. Use to signal error conditions to callers.
9. **Line 9:** `# Exception chaining`
A comment: *Exception chaining.*
10. **Line 10:** `try:`
Begins exception handling. Code in this block is monitored for errors.
11. **Line 11:** `result = json.loads(data)`
Deserializes a JSON string into a Python object.
12. **Line 12:** `except json.JSONDecodeError as exc:`
Catches `json` exceptions.
13. **Line 13:** `raise SettingsError("Invalid settings file") from exc`
Manually raises an exception. Use to signal error conditions to callers.

8.4 Custom Exceptions

```
# From ai_voice_studio/voicelab/__init__.py
class CloneEngineError(Exception):
    """Raised when a Voice Lab engine is not installed or fails."""
    pass

# From ai_voice_studio/omnivoice/__init__.py
class OmniVoiceError(Exception):
    """Raised when the OmniVoice engine is not installed or fails."""
    pass

# From ai_voice_studio/tts/downloader.py
class DownloadError(Exception):
    """Raised when a model download fails."""
    pass

class DownloadCancelled(Exception):
    """Raised when a download is cancelled by the user."""
    pass

# From ai_voice_studio/documents/parsers.py
class ParseError(Exception):
    """Raised when a document cannot be parsed."""
    pass

# Custom exception with context
class VoiceNotFoundError(Exception):
    def __init__(self, voice_id: str, tts_id: str):
        self.voice_id = voice_id
        self.tts_id = tts_id
        super().__init__(
            f"Voice '{voice_id}' not found in TTS engine '{tts_id}'"
        )
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# From ai_voice_studio/clone/__init__.py`
A comment: *From ai_voice_studio/clone/__init__.py.*
2. **Line 2:** `class CloneEngineError(Exception):`
Defines class `CloneEngineError` inheriting from `Exception` .
3. **Line 3:** `"""Raised when a Voice Lab engine is not installed or fails."""`
Docstring: *Raised when a Voice Lab engine is not installed or fails.* (one of Pocket TTS, Bark, F5-TTS). Documents the purpose of the following code.
4. **Line 4:** `pass`
Placeholder: does nothing.
5. **Line 5:** `# From ai_voice_studio/omnivoice/__init__.py`
A comment: *From ai_voice_studio/omnivoice/__init__.py.*
6. **Line 6:** `class OmniVoiceError(Exception):`
Defines class `OmniVoiceError` inheriting from `Exception` .
7. **Line 7:** `"""Raised when the OmniVoice engine is not installed or fails."""`
Docstring: *Raised when the OmniVoice engine is not installed or fails..* Documents the purpose of the following code.
8. **Line 8:** `pass`
Placeholder: does nothing.
9. **Line 9:** `# From ai_voice_studio/tts/downloader.py`
A comment: *From ai_voice_studio/tts/downloader.py.*
10. **Line 10:** `class DownloadError(Exception):`
Defines class `DownloadError` inheriting from `Exception` .
11. **Line 11:** `"""Raised when a model download fails."""`
Docstring: *Raised when a model download fails..* Documents the purpose of the following code.
12. **Line 12:** `pass`
Placeholder: does nothing.
13. **Line 13:** `class DownloadCancelled(Exception):`
Defines class `DownloadCancelled` inheriting from `Exception` .

14. **Line 14:** `"""Raised when a download is cancelled by the user."""`
Docstring: *Raised when a download is cancelled by the user..* Documents the purpose of the following code.

15. **Line 15:** `pass`
Placeholder: does nothing.

16. **Line 16:** `# From ai_voice_studio/documents/parsers.py`
A comment: *From ai_voice_studio/documents/parsers.py.*

17. **Line 17:** `class ParseError(Exception):`
Defines class `ParseError` inheriting from `Exception`.

18. **Line 18:** `"""Raised when a document cannot be parsed."""`
Docstring: *Raised when a document cannot be parsed..* Documents the purpose of the following code.

19. **Line 19:** `pass`
Placeholder: does nothing.

20. **Line 20:** `# Custom exception with context`
A comment: *Custom exception with context.*

21. **Line 21:** `class VoiceNotFoundError(Exception):`
Defines class `VoiceNotFoundError` inheriting from `Exception`.

22. **Line 22:** `def __init__(self, voice_id: str, tts_id: str):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

23. **Line 23:** `self.voice_id = voice_id`
Stores `voice_id` in instance attribute `voice_id`.

24. **Line 24:** `self.tts_id = tts_id`
Stores `tts_id` in instance attribute `tts_id`.

25. **Line 25:** `super().__init__(`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

26. **Line 26:** `f"Voice '{voice_id}' not found in TTS engine '{tts_id}'"`
`f"Voice '{voice_id}' not found in TTS engine '{tts_id}'"`

27. **Line 27:** `)`
`)`

8.5 Exception Handling Patterns in AIVS

```
# Pattern 1: Graceful degradation with logging
try:
    result = load_model(path)
except FileNotFoundError:
    log.warning("Model not found at {path}")
    result = None
except Exception as exc:
    log.error("Failed to load model: {exc}", exc_info=True)
    result = None

# Pattern 2: Retry with fallback
def download_with_fallback(urls):
    """Try multiple URLs until one works."""
    for url in urls:
        try:
            return download(url)
        except DownloadError as exc:
            log.warning("Failed {url}: {exc}")
    raise DownloadError("All download sources failed")

# Pattern 3: Suppressing expected exceptions
import contextlib
with contextlib.suppress(FileNotFoundError):
    os.unlink(temp_file) # Ignore if already deleted

# Pattern 4: Safe operations in GUI callbacks
def _on_download(self, _):
    try:
        self._start_download()
    except Exception as exc:
        # Never let exceptions crash the GUI!
        wx.MessageBox(
            f"Download failed: {exc}",
            "Error",
            style=wx.OK | wx.ICON_ERROR,
        )

# Pattern 5: noqa comments for intentional bare exceptions
try:
    some_risky_call()
except Exception: # noqa: BLE001
    pass # Intentionally ignore all errors
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# Pattern 1: Graceful degradation with logging`
A comment: *Pattern 1: Graceful degradation with logging.*
2. **Line 2:** `try:`
Begins exception handling. Code in this block is monitored for errors.
3. **Line 3:** `result = load_model(path)`
Assigns `load_model(path)` to `result`.
4. **Line 4:** `except FileNotFoundError:`
Raised when a file or directory is requested but doesn't exist.
5. **Line 5:** `log.warning("Model not found at {path}")`
`log.warning("Model not found at {path}")`
6. **Line 6:** `result = None`
Assigns `None` to `result`.
7. **Line 7:** `except Exception as exc:`
Catches `Exception` exceptions.
8. **Line 8:** `log.error("Failed to load model: {exc}", exc_info=True)`
`log.error("Failed to load model: {exc}", exc_info=True)`
9. **Line 9:** `result = None`
Assigns `None` to `result`.
10. **Line 10:** `# Pattern 2: Retry with fallback`
A comment: *Pattern 2: Retry with fallback.*
11. **Line 11:** `def download_with_fallback(urls):`
Defines function `download_with_fallback` accepting (`urls`).
12. **Line 12:** `"""Try multiple URLs until one works."""`
Docstring: *Try multiple URLs until one works..* Documents the purpose of the following code.
13. **Line 13:** `for url in urls:`
Iterates over `urls`, assigning each element to `url`.

14. **Line 14:** `try:`
Begins exception handling. Code in this block is monitored for errors.
15. **Line 15:** `return download(url)`
Returns `download(url)` to the caller.
16. **Line 16:** `except DownloadError as exc:`
Catches `DownloadError` exceptions.
17. **Line 17:** `log.warning("Failed {url}: {exc}")`
`log.warning("Failed {url}: {exc}")`
18. **Line 18:** `raise DownloadError("All download sources failed")`
Manually raises an exception. Use to signal error conditions to callers.
19. **Line 19:** `# Pattern 3: Suppressing expected exceptions`
A comment: *Pattern 3: Suppressing expected exceptions.*
20. **Line 20:** `import contextlib`
Imports the `contextlib` module.
21. **Line 21:** `with contextlib.suppress(FileNotFoundError):`
Context manager that suppresses specified exceptions.
22. **Line 22:** `os.unlink(temp_file) # Ignore if already deleted`
`os.unlink(temp_file) # Ignore if already deleted`
23. **Line 23:** `# Pattern 4: Safe operations in GUI callbacks`
A comment: *Pattern 4: Safe operations in GUI callbacks.*
24. **Line 24:** `def _on_download(self, _):`
Defines function `_on_download` accepting `(self, _)`.
25. **Line 25:** `try:`
Begins exception handling. Code in this block is monitored for errors.
26. **Line 26:** `self._start_download()`
Calls `self._start_download()`.
27. **Line 27:** `except Exception as exc:`
Catches `Exception` exceptions.

28. **Line 28:** `# Never let exceptions crash the GUI!`
A comment: *Never let exceptions crash the GUI!*.

29. **Line 29:** `wx.MessageBox(`
Creates a wxPython `wx.MessageBox` widget.

30. **Line 30:** `f"Download failed: {exc}",`
`f"Download failed: {exc}",`

31. **Line 31:** `"Error",`
`"Error",`

32. **Line 32:** `style=wx.OK | wx.ICON_ERROR,`
Assigns `wx.OK | wx.ICON_ERROR,` to `style`.

33. **Line 33:** `)`
`)`

34. **Line 34:** `# Pattern 5: noqa comments for intentional bare exceptions`
A comment: *Pattern 5: noqa comments for intentional bare exceptions.*

35. **Line 35:** `try:`
Begins exception handling. Code in this block is monitored for errors.

36. **Line 36:** `some_risky_call()`
Returns True if all elements in the iterable are truthy. Short-circuits on first False.

37. **Line 37:** `except Exception: # noqa: BLE001`
Catches all exceptions that inherit from Exception. Use as a last resort.

38. **Line 38:** `pass # Intentionally ignore all errors`
`pass # Intentionally ignore all errors`

8.6 The assert Statement

```
# Assert is for debugging – disabled with python -O
def get_voice(voices, index):
    assert 0 <= index < len(voices), f"Invalid index {index}"
    return voices[index]

# In tests
def test_catalog_has_required_engines():
    ids = {t["id"] for t in catalog.get_tts_list()}
    assert "piper" in ids, "Piper TTS must be in catalog"
    assert "kokoro" in ids, "Kokoro TTS must be in catalog"
```

Explain Line by Line Code Description (Block 6)

- Line 1:** `# Assert is for debugging – disabled with python -O`
A comment: *Assert is for debugging – disabled with python -O.*
- Line 2:** `def get_voice(voices, index):`
Defines function `get_voice` accepting (voices, index).
- Line 3:** `assert 0 <= index < len(voices), f"Invalid index {index}"`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
- Line 4:** `return voices[index]`
Returns `voices[index]` to the caller.
- Line 5:** `# In tests`
A comment: *In tests.*
- Line 6:** `def test_catalog_has_required_engines():`
Defines function `test_catalog_has_required_engines`.
- Line 7:** `ids = {t["id"] for t in catalog.get_tts_list()}`
Assigns `{t["id"] for t in catalog.get_tts_list()}` to `ids`.
- Line 8:** `assert "piper" in ids, "Piper TTS must be in catalog"`
`assert "piper" in ids, "Piper TTS must be in catalog"`
- Line 9:** `assert "kokoro" in ids, "Kokoro TTS must be in catalog"`
`assert "kokoro" in ids, "Kokoro TTS must be in catalog"`

8.7 Exception Best Practices

Tip: Rules for Exception Handling:

1. **Be specific** — catch `FileNotFoundError`, not `Exception`
2. **Don't swallow errors silently** — at least log them
3. **Never use bare `except:`** — always specify `except Exception:`
4. **Use custom exceptions** to distinguish application errors from library errors
5. **In GUI code**, always catch exceptions to prevent crashes
6. **In worker threads**, catch and report exceptions via events

8.8 Key Takeaways

- `try/except/else/finally` is the full error handling pattern
- **Catch specific exceptions** — never use bare `except:`
- **Custom exceptions** make application errors distinguishable from system errors
- **Always log exceptions** with `exc_info=True` for debugging
- In AIVS, exceptions are caught at **GUI boundaries** and **worker threads**

 **Exercise: Exercise 8.1:** Write a `safe_load_json(path)` function that returns the parsed JSON dict on success, or a default dict on any error. Include proper logging.

Modules & Packages

Learning Objectives

- Understand how Python's import system works
 - Create your own modules and packages
 - Master relative and absolute imports
 - Explore the AI Voice Studio package structure
-

9.1 Import Basics

```
# Import the entire module
import os
import json
print(os.path.join("a", "b"))

# Import with alias
import numpy as np
arr = np.array([1, 2, 3])

# Import specific names
from os.path import join, exists
print(join("a", "b"))

# Import everything (don't do this!)
from os import * # Pollutes namespace

# Conditional import
if sys.platform == "win32":
    import winsound
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Import the entire module`

A comment: *Import the entire module.*

2. **Line 2:** `import os`

Imports the `os` module.

3. **Line 3:** `import json`

Imports the `json` module.

4. **Line 4:** `print(os.path.join("a", "b"))`

OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.

5. **Line 5:** `# Import with alias`

A comment: *Import with alias.*

6. **Line 6:** `import numpy as np`

Imports the `numpy as np` module.

7. **Line 7:** `arr = np.array([1, 2, 3])`

Assigns `np.array([1, 2, 3])` to `arr`.

8. **Line 8:** `# Import specific names`

A comment: *Import specific names.*

9. **Line 9:** `from os.path import join, exists`

OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.

10. **Line 10:** `print(join("a", "b"))`

Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `# Import everything (don't do this!)`

A comment: *Import everything (don't do this!).*

12. **Line 12:** `from os import * # Pollutes namespace`

Imports `*` # Pollutes namespace from `os`.

13. **Line 13:** `# Conditional import`

A comment: *Conditional import.*

14. **Line 14:** `if sys.platform == "win32":`

Checks condition: `sys.platform == "win32"` . The indented block runs only when this is `True` .

15. Line 15: `import winsound`

Imports the `winsound` module.

9.2 Creating Modules

Any `.py` file is a module. If you create `mymodule.py` :

```
# mymodule.py
"""My custom module for audio utilities."""

SAMPLE_RATE = 24000

def format_duration(seconds: float) -> str:
    """Format seconds as HH:MM:SS."""
    h = int(seconds // 3600)
    m = int((seconds % 3600) // 60)
    s = int(seconds % 60)
    return f"{h:02d}:{m:02d}:{s:02d}"
```

Explain Line by Line Code Description (Block 2)

- Line 1:** `# mymodule.py`
A comment: *mymodule.py*.
- Line 2:** `"""My custom module for audio utilities."""`
Docstring: *My custom module for audio utilities..* Documents the purpose of the following code.
- Line 3:** `SAMPLE_RATE = 24000`
Assigns `24000` to `SAMPLE_RATE`.
- Line 4:** `def format_duration(seconds: float) -> str:`
Defines function `format_duration` accepting (seconds), returning `str`.
- Line 5:** `"""Format seconds as HH:MM:SS."""`
Docstring: *Format seconds as HH:MM:SS..* Documents the purpose of the following code.
- Line 6:** `h = int(seconds // 3600)`
Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.
- Line 7:** `m = int((seconds % 3600) // 60)`
Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.
- Line 8:** `s = int(seconds % 60)`
Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.
- Line 9:** `return f"{h:02d}:{m:02d}:{s:02d}"`
Returns `f"{h:02d}:{m:02d}:{s:02d}"` to the caller.

Then in another file:

```
import mymodule
print(mymodule.format_duration(3661)) # "01:01:01"
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `import mymodule`
Imports the `mymodule` module.
2. **Line 2:** `print(mymodule.format_duration(3661)) # "01:01:01"`
Outputs text to the console. The primary way to display information during development and debugging.

9.3 Creating Packages

A **package** is a directory of modules with an `__init__.py` :

```
ai_voice_studio/      # Package directory
├── __init__.py        # Makes it a package (can be empty)
├── paths.py           # Module: ai_voice_studio.paths
├── settings.py        # Module: ai_voice_studio.settings
├── tts/               # Sub-package
│   ├── __init__.py
│   ├── catalog.py     # Module: ai_voice_studio.tts.catalog
│   ├── engine.py
│   └── models.py
├── gui/               # Sub-package
│   ├── __init__.py
│   └── main_frame.py
```


Explain Line by Line Code Description (Block 4)

1. **Line 1:** `ai_voice_studio/` # Package directory
`ai_voice_studio/` # Package directory
2. **Line 2:** `└─ __init__.py` # Makes it a package (can be empty)
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
3. **Line 3:** `└─ paths.py` # Module: ai_voice_studio.paths
`└─ paths.py` # Module: ai_voice_studio.paths
4. **Line 4:** `└─ settings.py` # Module: ai_voice_studio.settings
`└─ settings.py` # Module: ai_voice_studio.settings
5. **Line 5:** `└─ tts/` # Sub-package
`└─ tts/` # Sub-package
6. **Line 6:** `| └─ __init__.py`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
7. **Line 7:** `| └─ catalog.py` # Module: ai_voice_studio.tts.catalog
`| └─ catalog.py` # Module: ai_voice_studio.tts.catalog
8. **Line 8:** `| └─ engine.py`
`| └─ engine.py`
9. **Line 9:** `| └─ models.py`
`| └─ models.py`
10. **Line 10:** `└─ gui/` # Sub-package
`└─ gui/` # Sub-package
11. **Line 11:** `| └─ __init__.py`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
12. **Line 12:** `| └─ main_frame.py`
`| └─ main_frame.py`

```
# __init__.py can be empty, or it can import key names:
# ai_voice_studio/tts/__init__.py
"""TTS engine subsystem."""
# Optionally re-export key names:
# from .catalog import get_tts_list, find_tts
# from .engine import get_engine
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# __init__.py can be empty, or it can import key names:`
A comment: *__init__.py can be empty, or it can import key names:.*
2. **Line 2:** `# ai_voice_studio/tts/__init__.py`
A comment: *ai_voice_studio/tts/__init__.py.*
3. **Line 3:** `"""TTS engine subsystem."""`
Docstring: *TTS engine subsystem..* Documents the purpose of the following code.
4. **Line 4:** `# Optionally re-export key names:`
A comment: *Optionally re-export key names:.*
5. **Line 5:** `# from .catalog import get_tts_list, find_tts`
A comment: *from .catalog import get_tts_list, find_tts.*
6. **Line 6:** `# from .engine import get_engine`
A comment: *from .engine import get_engine.*

9.4 Absolute vs. Relative Imports

```
# Absolute imports (preferred – clearer)
from ai_voice_studio.paths import user_data_dir
from ai_voice_studio.tts.catalog import get_tts_list
from ai_voice_studio.settings import Settings

# Relative imports (within the same package)
from . import paths          # Same package
from .. import settings      # Parent package
from .tts import catalog     # Sibling package
from ..paths import user_data_dir # Parent's module
from .models import ModelStore # Same package's module

# In AIVS, relative imports are used within the package:
# From tts/engine.py:
from .. import paths
from ..settings import Settings
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `# Absolute imports (preferred – clearer)`
A comment: *Absolute imports (preferred – clearer).*
2. **Line 2:** `from ai_voice_studio.paths import user_data_dir`
Imports `user_data_dir` from `ai_voice_studio.paths`.
3. **Line 3:** `from ai_voice_studio.tts.catalog import get_tts_list`
Imports `get_tts_list` from `ai_voice_studio.tts.catalog`.
4. **Line 4:** `from ai_voice_studio.settings import Settings`
Imports `Settings` from `ai_voice_studio.settings`.
5. **Line 5:** `# Relative imports (within the same package)`
A comment: *Relative imports (within the same package).*
6. **Line 6:** `from . import paths # Same package`
Imports `paths` # Same package from `.`.
7. **Line 7:** `from .. import settings # Parent package`
Imports `settings` # Parent package from `..`.
8. **Line 8:** `from .tts import catalog # Sibling package`
Imports `catalog` # Sibling package from `.tts`.
9. **Line 9:** `from ..paths import user_data_dir # Parent's module`
Imports `user_data_dir` # Parent's module from `..paths`.
10. **Line 10:** `from .models import ModelStore # Same package's module`
Imports `ModelStore` # Same package's module from `.models`.
11. **Line 11:** `# In AIVS, relative imports are used within the package:`
A comment: *In AIVS, relative imports are used within the package:.*
12. **Line 12:** `# From tts/engine.py:`
A comment: *From tts/engine.py:.*
13. **Line 13:** `from .. import paths`
Imports `paths` from `..`.

14. **Line 14:** `from ..settings import Settings`
Imports `Settings` from `..settings` .

9.5 The `__name__` and `__main__` Pattern

```
# Every module has a __name__ variable
# When imported: __name__ == "mymodule"
# When run directly: __name__ == "__main__"

def main():
    print("Running my module!")

if __name__ == "__main__":
    main() # Only runs when executed directly
```

Explain Line by Line Code Description (Block 7)

- Line 1:** `# Every module has a __name__ variable`
A comment: *Every module has a `__name__` variable.*
- Line 2:** `# When imported: __name__ == "mymodule"`
A comment: *When imported: `__name__` == "mymodule".*
- Line 3:** `# When run directly: __name__ == "__main__"`
A comment: *When run directly: `__name__` == "__main__".*
- Line 4:** `def main():`
Defines function `main` .
- Line 5:** `print("Running my module!")`
Outputs text to the console. The primary way to display information during development and debugging.
- Line 6:** `if __name__ == "__main__":`
Checks condition: `__name__ == "__main__"` . The indented block runs only when this is `True` .
- Line 7:** `main() # Only runs when executed directly`
Calls function `main()` .

This pattern is used throughout AIVS:

```
# main.py - the application entry point
def main() -> int:
    app = wx.App(False)
    frame = MainFrame(None)
    frame.Show()
    return app.MainLoop()

if __name__ == "__main__":
    sys.exit(main())

# ai_voice_studio/clone/worker.py - subprocess entry point
def main() -> int:
    # Worker Loop reading JSON commands from stdin
    for line in sys.stdin:
        ...
    return 0

if __name__ == "__main__":
    sys.exit(main())
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# main.py – the application entry point`
A comment: *main.py – the application entry point.*
2. **Line 2:** `def main() -> int:`
Defines function `main`, returning `int`.
3. **Line 3:** `app = wx.App(False)`
Creates the wxPython application object. Must be created before any GUI elements. False means don't redirect stdout.
4. **Line 4:** `frame = MainFrame(None)`
Assigns `MainFrame(None)` to `frame`.
5. **Line 5:** `frame.Show()`
Makes the frame visible on screen. The frame is hidden by default.
6. **Line 6:** `return app.MainLoop()`
Starts the wxPython event loop. This blocks until the last window is closed. All GUI events are processed here.
7. **Line 7:** `if __name__ == "__main__":`
Checks condition: `__name__ == "__main__"`. The indented block runs only when this is `True`.
8. **Line 8:** `sys.exit(main())`
`sys.exit(main())`
9. **Line 9:** `# ai_voice_studio/clone/worker.py – subprocess entry point`
A comment: *ai_voice_studio/clone/worker.py – subprocess entry point.*
10. **Line 10:** `def main() -> int:`
Defines function `main`, returning `int`.
11. **Line 11:** `# Worker loop reading JSON commands from stdin`
A comment: *Worker loop reading JSON commands from stdin.*
12. **Line 12:** `for line in sys.stdin:`
Iterates over `sys.stdin`, assigning each element to `line`.
13. **Line 13:** `...`
Ellipsis: abstract method placeholder.
14. **Line 14:** `return 0`

Returns `0` to the caller.

15. **Line 15:** `if __name__ == "__main__":`

Checks condition: `__name__ == "__main__"`. The indented block runs only when this is `True`.

16. **Line 16:** `sys.exit(main())`

`sys.exit(main())`

9.6 Import Hooks and Dynamic Imports

```
# Importlib - dynamic imports at runtime
import importlib
module = importlib.import_module("ai_voice_studio.tts.catalog")

# Used in the addon system
import importlib.util
spec = importlib.util.spec_from_file_location(
    "_addon_my_tts",
    "/path/to/addon/__init__.py",
)
module = importlib.util.module_from_spec(spec)
sys.modules["_addon_my_tts"] = module
spec.loader.exec_module(module)

# Lazy imports (import only when needed)
def heavy_function():
    import numpy # Only imported when function is called
    return np.array([1, 2, 3])
```


Explain Line by Line Code Description (Block 9)

1. **Line 1:** `# Importlib – dynamic imports at runtime`
A comment: *Importlib – dynamic imports at runtime.*
2. **Line 2:** `import importlib`
Imports the `importlib` module.
3. **Line 3:** `module = importlib.import_module("ai_voice_studio.tts.catalog")`
Assigns `importlib.import_module("ai_voice_studio.tts.catalog")` to `module`.
4. **Line 4:** `# Used in the addon system`
A comment: *Used in the addon system.*
5. **Line 5:** `import importlib.util`
Imports the `importlib.util` module.
6. **Line 6:** `spec = importlib.util.spec_from_file_location(`
Assigns `importlib.util.spec_from_file_location(` to `spec`.
7. **Line 7:** `"_addon_my_tts",`
`"_addon_my_tts",`
8. **Line 8:** `"/path/to/addon/__init__.py",`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
9. **Line 9:** `)`
`)`
10. **Line 10:** `module = importlib.util.module_from_spec(spec)`
Assigns `importlib.util.module_from_spec(spec)` to `module`.
11. **Line 11:** `sys.modules["_addon_my_tts"] = module`
`sys.modules["_addon_my_tts"] = module`
12. **Line 12:** `spec.loader.exec_module(module)`
`spec.loader.exec_module(module)`
13. **Line 13:** `# Lazy imports (import only when needed)`
A comment: *Lazy imports (import only when needed).*

14. **Line 14:** `def heavy_function():`
Defines function `heavy_function`.
15. **Line 15:** `import numpy # Only imported when function is called`
Imports the `numpy` # Only imported when function is called module.
16. **Line 16:** `return np.array([1, 2, 3])`
Returns `np.array([1, 2, 3])` to the caller.

9.7 Standard Library Highlights

Module	Purpose	AIVS Usage
<code>os</code>	OS interface, paths, environment	File operations, paths
<code>json</code>	JSON parsing and writing	Settings, project files
<code>logging</code>	Logging framework	Debug and info logging
<code>threading</code>	Thread-based parallelism	Background TTS synthesis
<code>subprocess</code>	Process management	FFmpeg, pip, worker processes
<code>re</code>	Regular expressions	Text processing, parsing
<code>pathlib</code>	Object-oriented paths	Modern path handling
<code>tempfile</code>	Temporary files and dirs	Atomic writes, pip installs
<code>zipfile</code>	ZIP archive handling	DAISY book export
<code>uuid</code>	Unique identifiers	DAISY book UUIDs

9.8 Key Takeaways

- Any `.py` file is a module; a directory with `__init__.py` is a package
- Use absolute imports** for clarity (`from ai_voice_studio.paths import ...`)
- Relative imports** are for within-package references only
- The `if __name__ == "__main__"` pattern allows files to be both imported and run

- **Dynamic imports** via `importlib` enable the AIVS addon system
- Lazy imports (inside functions) improve startup time for heavy dependencies

 **Exercise: Exercise 9.1:** Create a package `audio_utils/` with two modules: `format.py` (duration formatting) and `levels.py` (volume calculations). Import and use both from a script.

Object-Oriented Programming Basics

Learning Objectives

- Understand classes, objects, and the `__init__` method
 - Master instance attributes vs. class attributes
 - Implement methods (instance, class, static)
 - Understand `self` and the class-object relationship
 - See OOP patterns throughout AI Voice Studio
-

10.1 Classes and Objects

```
class TtsVoice:
    """Represents a single TTS voice."""

    # Class attribute (shared by all instances)
    sample_rate = 24000

    def __init__(self, voice_id: str, name: str, sid: int = 0):
        """Initialize a new TtsVoice instance."""
        # Instance attributes (unique to each object)
        self.voice_id = voice_id
        self.name = name
        self.sid = sid
        self.is_loaded = False

    def __repr__(self) -> str:
        """Developer-friendly string representation."""
        return f"TtsVoice({self.voice_id!r}, {self.name!r}, sid={self.sid})"

    def load(self):
        """Load the voice model."""
        print(f>Loading {self.name} (sid={self.sid})...")
        self.is_loaded = True

    def synthesize(self, text: str) -> "np.ndarray":
        """Synthesize text using this voice."""
        if not self.is_loaded:
            self.load()
        print(f>Synthesizing '{text[:30]}' with {self.name}")
        # ... actual synthesis logic

# Creating objects (instances)
bella = TtsVoice("af_bella", "Bella", sid=2)
adam = TtsVoice("am_adam", "Adam", sid=1)

# Using objects
bella.load()
bella.synthesize("Hello world")

# Class attribute shared by all instances
print(TtsVoice.sample_rate)    # 24000
print(bella.sample_rate)       # 24000 (same)

# Check type
print(isinstance(bella, TtsVoice)) # True
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `class TtsVoice:`
Defines class `TtsVoice`.
2. **Line 2:** `"""Represents a single TTS voice."""`
Docstring: *Represents a single TTS voice..* Documents the purpose of the following code.
3. **Line 3:** `# Class attribute (shared by all instances)`
A comment: *Class attribute (shared by all instances).*
4. **Line 4:** `sample_rate = 24000`
Assigns `24000` to `sample_rate`.
5. **Line 5:** `def __init__(self, voice_id: str, name: str, sid: int = 0):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
6. **Line 6:** `"""Initialize a new TtsVoice instance."""`
Docstring: *Initialize a new TtsVoice instance..* Documents the purpose of the following code.
7. **Line 7:** `# Instance attributes (unique to each object)`
A comment: *Instance attributes (unique to each object).*
8. **Line 8:** `self.voice_id = voice_id`
Stores `voice_id` in instance attribute `voice_id`.
9. **Line 9:** `self.name = name`
Stores `name` in instance attribute `name`.
10. **Line 10:** `self.sid = sid`
Stores `sid` in instance attribute `sid`.
11. **Line 11:** `self.is_loaded = False`
Stores `False` in instance attribute `is_loaded`.
12. **Line 12:** `def __repr__(self) -> str:`
Returns a developer-friendly string representation. Called by `repr()` and in the interactive console.
13. **Line 13:** `"""Developer-friendly string representation."""`
Docstring: *Developer-friendly string representation..* Documents the purpose of the following code.
14. **Line 14:** `return f"TtsVoice({self.voice_id!r}, {self.name!r}, sid={self.sid})"`

Returns `f"TtsVoice({self.voice_id!r}, {self.name!r}, sid={self.sid})"` to the caller.

15. Line 15: `def load(self):`

Defines function `load` accepting (self).

16. Line 16: `"""Load the voice model."""`

Docstring: *Load the voice model.*. Documents the purpose of the following code.

17. Line 17: `print(f"Loading {self.name} (sid={self.sid})...")`

Outputs text to the console. The primary way to display information during development and debugging.

18. Line 18: `self.is_loaded = True`

Stores `True` in instance attribute `is_loaded`.

19. Line 19: `def synthesize(self, text: str) -> "np.ndarray":`

Defines function `synthesize` accepting (self, text), returning `"np.ndarray"`.

20. Line 20: `"""Synthesize text using this voice."""`

Docstring: *Synthesize text using this voice.*. Documents the purpose of the following code.

21. Line 21: `if not self.is_loaded:`

Checks condition: `not self.is_loaded`. The indented block runs only when this is `True`.

22. Line 22: `self.load()`

Calls `self.load()`.

23. Line 23: `print(f"Synthesizing '{text[:30]}' with {self.name}")`

Outputs text to the console. The primary way to display information during development and debugging.

24. Line 24: `# ... actual synthesis logic`

A comment: *... actual synthesis logic.*

25. Line 25: `# Creating objects (instances)`

A comment: *Creating objects (instances).*

26. Line 26: `bella = TtsVoice("af_bella", "Bella", sid=2)`

Assigns `TtsVoice("af_bella", "Bella", sid=2)` to `bella`.

27. Line 27: `adam = TtsVoice("am_adam", "Adam", sid=1)`

Assigns `TtsVoice("am_adam", "Adam", sid=1)` to `adam`.

28. Line 28: `# Using objects`

A comment: *Using objects.*

29. Line 29: `bella.load()`

`bella.load()`

30. Line 30: `bella.synthesize("Hello world")`

`bella.synthesize("Hello world")`

31. Line 31: `# Class attribute shared by all instances`

A comment: *Class attribute shared by all instances.*

32. Line 32: `print(TtsVoice.sample_rate) # 24000`

Outputs text to the console. The primary way to display information during development and debugging.

33. Line 33: `print(bella.sample_rate) # 24000 (same)`

Outputs text to the console. The primary way to display information during development and debugging.

34. Line 34: `# Check type`

A comment: *Check type.*

35. Line 35: `print(isinstance(bella, TtsVoice)) # True`

Checks if an object is an instance of a class. Preferred over `type()` for type checking.

10.2 The `__init__` Method

`__init__` is the constructor — it runs when you create a new object:

```
class Settings:
    def __init__(self, path: str | None = None):
        """Initialize settings with optional file path."""
        self._path = path
        self._data: dict = {}
        self._load()

    def _load(self):
        """Load settings from disk."""
        if self._path and os.path.isfile(self._path):
            with open(self._path, "r") as f:
                self._data = json.load(f)

# __init__ parameters have default values
s1 = Settings() # No path
s2 = Settings("/path/to/settings.json") # With path
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `class Settings:`
Defines class `Settings`.
2. **Line 2:** `def __init__(self, path: str | None = None):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
3. **Line 3:** `"""Initialize settings with optional file path."""`
Docstring: *Initialize settings with optional file path..* Documents the purpose of the following code.
4. **Line 4:** `self._path = path`
Stores `path` in instance attribute `_path`.
5. **Line 5:** `self._data: dict = {}`
`self._data: dict = {}`
6. **Line 6:** `self._load()`
Calls `self._load()`.
7. **Line 7:** `def _load(self):`
Defines function `_load` accepting (self).
8. **Line 8:** `"""Load settings from disk."""`
Docstring: *Load settings from disk..* Documents the purpose of the following code.
9. **Line 9:** `if self._path and os.path.isfile(self._path):`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
10. **Line 10:** `with open(self._path, "r") as f:`
Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.
11. **Line 11:** `self._data = json.load(f)`
Deserializes JSON from a file into a Python object.
12. **Line 12:** `# __init__ parameters have default values`
A comment: *__init__ parameters have default values.*
13. **Line 13:** `s1 = Settings() # No path`
Assigns `Settings()` # No path to `s1`.
14. **Line 14:** `s2 = Settings("/path/to/settings.json") # With path`

```
Assigns Settings("/path/to/settings.json") # With path to s2 .
```

10.3 Instance vs. Class Attributes

```
class Engine:
    # Class attributes – shared by ALL instances
    version = "1.0"
    _cache = {} # Shared cache

    def __init__(self, engine_id: str):
        # Instance attributes – unique per instance
        self.engine_id = engine_id
        self.is_ready = False

e1 = Engine("piper")
e2 = Engine("kokoro")

# Instance attributes differ
print(e1.engine_id) # "piper"
print(e2.engine_id) # "kokoro"

# Class attributes are shared
print(e1.version) # "1.0"
print(e2.version) # "1.0"

# Modifying a class attribute on an instance creates an instance attribute
e1.version = "2.0" # Only e1 is affected
print(e1.version) # "2.0"
print(e2.version) # "1.0" (still uses class attribute)
print(Engine.version) # "1.0" (class attribute unchanged)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `class Engine:`
Defines class `Engine` .
2. **Line 2:** `# Class attributes – shared by ALL instances`
A comment: *Class attributes – shared by ALL instances.*
3. **Line 3:** `version = "1.0"`
Assigns `"1.0"` to `version` .
4. **Line 4:** `_cache = {} # Shared cache`
Assigns `{}` `# Shared cache` to `_cache` .
5. **Line 5:** `def __init__(self, engine_id: str):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
6. **Line 6:** `# Instance attributes – unique per instance`
A comment: *Instance attributes – unique per instance.*
7. **Line 7:** `self.engine_id = engine_id`
Stores `engine_id` in instance attribute `engine_id` .
8. **Line 8:** `self.is_ready = False`
Stores `False` in instance attribute `is_ready` .
9. **Line 9:** `e1 = Engine("piper")`
Assigns `Engine("piper")` to `e1` .
10. **Line 10:** `e2 = Engine("kokoro")`
Assigns `Engine("kokoro")` to `e2` .
11. **Line 11:** `# Instance attributes differ`
A comment: *Instance attributes differ.*
12. **Line 12:** `print(e1.engine_id) # "piper"`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `print(e2.engine_id) # "kokoro"`
Outputs text to the console. The primary way to display information during development and debugging.

14. **Line 14:** `# Class attributes are shared`
A comment: *Class attributes are shared.*
 15. **Line 15:** `print(e1.version) # "1.0"`
Outputs text to the console. The primary way to display information during development and debugging.
 16. **Line 16:** `print(e2.version) # "1.0"`
Outputs text to the console. The primary way to display information during development and debugging.
 17. **Line 17:** `# Modifying a class attribute on an instance creates an instance attribute`
A comment: *Modifying a class attribute on an instance creates an instance attribute.*
 18. **Line 18:** `e1.version = "2.0" # Only e1 is affected`
Assigns `"2.0"` # Only e1 is affected to `e1.version` .
 19. **Line 19:** `print(e1.version) # "2.0"`
Outputs text to the console. The primary way to display information during development and debugging.
 20. **Line 20:** `print(e2.version) # "1.0" (still uses class attribute)`
Outputs text to the console. The primary way to display information during development and debugging.
 21. **Line 21:** `print(Engine.version) # "1.0" (class attribute unchanged)`
Outputs text to the console. The primary way to display information during development and debugging.
-

10.4 Types of Methods

```
class AudioFile:
    # Class attribute
    MAX_SIZE_MB = 100

    def __init__(self, path: str, data: bytes):
        self.path = path
        self.data = data

    # Instance method – accesses 'self'
    def size_mb(self) -> float:
        return len(self.data) / (1024 * 1024)

    # Class method – accesses 'cls' (the class itself)
    @classmethod
    def from_file(cls, path: str) -> "AudioFile":
        """Alternative constructor from a file path."""
        with open(path, "rb") as f:
            data = f.read()
        return cls(path, data) # cls() = AudioFile()

    # Static method – no access to self or cls
    @staticmethod
    def is_valid_format(path: str) -> bool:
        """Check if file extension is supported."""
        return path.lower().endswith((".wav", ".mp3", ".flac"))

# Using all three types
f1 = AudioFile("output.wav", audio_bytes) # Regular constructor
f2 = AudioFile.from_file("output.wav")    # Class method constructor
print(f1.size_mb())                       # Instance method
print(AudioFile.is_valid_format("test.wav")) # Static method
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `class AudioFile:`
Defines class `AudioFile` .
2. **Line 2:** `# Class attribute`
A comment: *Class attribute*.
3. **Line 3:** `MAX_SIZE_MB = 100`
Assigns `100` to `MAX_SIZE_MB` .
4. **Line 4:** `def __init__(self, path: str, data: bytes):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `self.path = path`
Stores `path` in instance attribute `path` .
6. **Line 6:** `self.data = data`
Stores `data` in instance attribute `data` .
7. **Line 7:** `# Instance method — accesses 'self'`
A comment: *Instance method — accesses 'self'*.
8. **Line 8:** `def size_mb(self) -> float:`
Defines function `size_mb` accepting (self), returning `float` .
9. **Line 9:** `return len(self.data) / (1024 * 1024)`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
10. **Line 10:** `# Class method — accesses 'cls' (the class itself)`
A comment: *Class method — accesses 'cls' (the class itself)*.
11. **Line 11:** `@classmethod`
Decorator that receives the class (cls) as the first argument instead of the instance.
12. **Line 12:** `def from_file(cls, path: str) -> "AudioFile":`
Defines function `from_file` accepting (cls, path), returning `"AudioFile"` .
13. **Line 13:** `"""Alternative constructor from a file path."""`
Docstring: *Alternative constructor from a file path..* Documents the purpose of the following code.
14. **Line 14:** `with open(path, "rb") as f:`

Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.

15. **Line 15:** `data = f.read()`

Reads the entire file contents as a string.

16. **Line 16:** `return cls(path, data) # cls() = AudioFile()`

Returns `cls(path, data) # cls() = AudioFile()` to the caller.

17. **Line 17:** `# Static method – no access to self or cls`

A comment: *Static method – no access to self or cls.*

18. **Line 18:** `@staticmethod`

Decorator that creates a method without 'self' or 'cls' parameter. Called on the class, not an instance.

19. **Line 19:** `def is_valid_format(path: str) -> bool:`

Formats a value according to a format specification. Used in f-strings and `.format()` method.

20. **Line 20:** `"""Check if file extension is supported."""`

Docstring: *Check if file extension is supported..* Documents the purpose of the following code.

21. **Line 21:** `return path.lower().endswith((".wav", ".mp3", ".flac"))`

Returns True if the string ends with the given suffix.

22. **Line 22:** `# Using all three types`

A comment: *Using all three types.*

23. **Line 23:** `f1 = AudioFile("output.wav", audio_bytes) # Regular constructor`

Assigns `AudioFile("output.wav", audio_bytes) # Regular constructor` to `f1`.

24. **Line 24:** `f2 = AudioFile.from_file("output.wav") # Class method constructor`

Assigns `AudioFile.from_file("output.wav") # Class method constructor` to `f2`.

25. **Line 25:** `print(f1.size_mb()) # Instance method`

Outputs text to the console. The primary way to display information during development and debugging.

26. **Line 26:** `print(AudioFile.is_valid_format("test.wav")) # Static method`

Formats a value according to a format specification. Used in f-strings and `.format()` method.

10.5 Special (Dunder) Methods

```
class Segment:
    def __init__(self, index: int, title: str, text: str):
        self.index = index
        self.title = title
        self.text = text

    def __repr__(self) -> str:
        """Developer string: Segment(1, 'Chapter 1')"""
        return f"Segment({self.index}, {self.title!r})"

    def __str__(self) -> str:
        """User-facing string: "Chapter 1" """
        return self.title

    def __eq__(self, other) -> bool:
        """Equality comparison."""
        if not isinstance(other, Segment):
            return NotImplemented
        return self.index == other.index

    def __lt__(self, other) -> bool:
        """Less-than comparison (enables sorting)."""
        return self.index < other.index

    def __len__(self) -> int:
        """len(segment) returns text length."""
        return len(self.text)

    def __bool__(self) -> bool:
        """bool(segment) is True if text is non-empty."""
        return bool(self.text.strip())

# Usage
seg = Segment(1, "Chapter 1", "Once upon a time...")
print(repr(seg)) # Segment(1, 'Chapter 1')
print(str(seg)) # Chapter 1
print(len(seg)) # 22

segments = [seg2, seg1]
sorted_segs = sorted(segments) # Works because of __lt__
```


Explain Line by Line Code Description (Block 5)

1. **Line 1:** `class Segment:`
Defines class `Segment`.
2. **Line 2:** `def __init__(self, index: int, title: str, text: str):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
3. **Line 3:** `self.index = index`
Stores `index` in instance attribute `index`.
4. **Line 4:** `self.title = title`
Stores `title` in instance attribute `title`.
5. **Line 5:** `self.text = text`
Stores `text` in instance attribute `text`.
6. **Line 6:** `def __repr__(self) -> str:`
Returns a developer-friendly string representation. Called by `repr()` and in the interactive console.
7. **Line 7:** `"""Developer string: Segment(1, 'Chapter 1')"""`
Docstring: *Developer string: Segment(1, 'Chapter 1')*. Documents the purpose of the following code.
8. **Line 8:** `return f"Segment({self.index}, {self.title!r})"`
Returns `f"Segment({self.index}, {self.title!r})"` to the caller.
9. **Line 9:** `def __str__(self) -> str:`
Returns a human-readable string representation. Called by `print()` and `str()`.
10. **Line 10:** `"""User-facing string: "Chapter 1" """`
Docstring: *User-facing string: "Chapter 1"*. Documents the purpose of the following code.
11. **Line 11:** `return self.title`
Returns `self.title` to the caller.
12. **Line 12:** `def __eq__(self, other) -> bool:`
Defines equality comparison (`==`). Called when comparing two objects.
13. **Line 13:** `"""Equality comparison."""`
Docstring: *Equality comparison..*. Documents the purpose of the following code.
14. **Line 14:** `if not isinstance(other, Segment):`

Checks if an object is an instance of a class. Preferred over `type()` for type checking.

15. Line 15: `return NotImplemented`

Returns `NotImplemented` to the caller.

16. Line 16: `return self.index == other.index`

Returns `self.index == other.index` to the caller.

17. Line 17: `def __lt__(self, other) -> bool:`

Defines less-than comparison (<). Used for sorting.

18. Line 18: `"""Less-than comparison (enables sorting)."""`

Docstring: *Less-than comparison (enables sorting)*.. Documents the purpose of the following code.

19. Line 19: `return self.index < other.index`

Returns `self.index < other.index` to the caller.

20. Line 20: `def __len__(self) -> int:`

Returns the length of the object. Called by `len()`.

21. Line 21: `"""len(segment) returns text length."""`

Docstring: *len(segment) returns text length*.. Documents the purpose of the following code.

22. Line 22: `return len(self.text)`

Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

23. Line 23: `def __bool__(self) -> bool:`

Defines truth value for use in if statements and boolean operations.

24. Line 24: `"""bool(segment) is True if text is non-empty."""`

Docstring: *bool(segment) is True if text is non-empty*.. Documents the purpose of the following code.

25. Line 25: `return bool(self.text.strip())`

Removes leading and trailing whitespace (or specified characters). Returns a new string.

26. Line 26: `# Usage`

A comment: *Usage*.

27. Line 27: `seg = Segment(1, "Chapter 1", "Once upon a time...")`

Assigns `Segment(1, "Chapter 1", "Once upon a time...")` to `seg`.

28. Line 28: `print(repr(seg)) # Segment(1, 'Chapter 1')`

Outputs text to the console. The primary way to display information during development and debugging.

29. **Line 29:** `print(str(seg)) # Chapter 1`
Outputs text to the console. The primary way to display information during development and debugging.
30. **Line 30:** `print(len(seg)) # 22`
Outputs text to the console. The primary way to display information during development and debugging.
31. **Line 31:** `segments = [seg2, seg1]`
Assigns `[seg2, seg1]` to `segments`.
32. **Line 32:** `sorted_segs = sorted(segments) # Works because of __lt__`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.

10.6 Properties

```
class VoiceEntry:
    def __init__(self, voice_id: str, sid: int):
        self._voice_id = voice_id
        self._sid = sid

    @property
    def voice_id(self) -> str:
        """Read-only property."""
        return self._voice_id

    @property
    def display_name(self) -> str:
        """Computed property – not stored, calculated on access."""
        return f"{self._voice_id} (sid={self._sid})"

    @property
    def sid(self) -> int:
        return self._sid

    @sid.setter
    def sid(self, value: int):
        if value < 0:
            raise ValueError("SID must be non-negative")
        self._sid = value

v = VoiceEntry("af_bella", 2)
print(v.display_name) # "af_bella (sid=2)" – computed, not stored
v.sid = 5             # Uses setter with validation
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `class VoiceEntry:`
Begins exception handling. Code in this block is monitored for errors.
2. **Line 2:** `def __init__(self, voice_id: str, sid: int):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
3. **Line 3:** `self._voice_id = voice_id`
Stores `voice_id` in instance attribute `_voice_id`.
4. **Line 4:** `self._sid = sid`
Stores `sid` in instance attribute `_sid`.
5. **Line 5:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
6. **Line 6:** `def voice_id(self) -> str:`
Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.
7. **Line 7:** `"""Read-only property."""`
Docstring: *Read-only property*.. Documents the purpose of the following code.
8. **Line 8:** `return self._voice_id`
Returns `self._voice_id` to the caller.
9. **Line 9:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
10. **Line 10:** `def display_name(self) -> str:`
Defines function `display_name` accepting (self), returning `str`.
11. **Line 11:** `"""Computed property – not stored, calculated on access."""`
Docstring: *Computed property – not stored, calculated on access*.. Documents the purpose of the following code.
12. **Line 12:** `return f"{self._voice_id} (sid={self._sid})"`
Returns `f"{self._voice_id} (sid={self._sid})"` to the caller.
13. **Line 13:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
14. **Line 14:** `def sid(self) -> int:`

Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.

15. Line 15: `return self._sid`

Returns `self._sid` to the caller.

16. Line 16: `@sid.setter`

Decorator: `@sid.setter`. Wraps the following function/class.

17. Line 17: `def sid(self, value: int):`

Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.

18. Line 18: `if value < 0:`

Checks condition: `value < 0`. The indented block runs only when this is `True`.

19. Line 19: `raise ValueError("SID must be non-negative")`

Raised when a function receives an argument of the right type but inappropriate value.

20. Line 20: `self._sid = value`

Stores `value` in instance attribute `_sid`.

21. Line 21: `v = VoiceEntry("af_bella", 2)`

Assigns `VoiceEntry("af_bella", 2)` to `v`.

22. Line 22: `print(v.display_name) # "af_bella (sid=2)" - computed, not stored`

Outputs text to the console. The primary way to display information during development and debugging.

23. Line 23: `v.sid = 5 # Uses setter with validation`

Assigns `5` `# Uses setter with validation` to `v.sid`.

10.7 OOP Patterns in AIVS

```
# From tts/models.py - ModelStore manages voice models on disk
class ModelStore:
    """Manages downloaded TTS voice models."""

    def __init__(self, state_file: str | None = None):
        self._state_file = state_file or self._default_state_file()
        self._state: dict = self._load_state()

    def variant_dir(self, tts_id: str, lang: str, variant: str) -> str:
        """Return the disk path for a voice variant."""
        return os.path.join(paths.models_dir(), tts_id, lang, variant)

    def is_variant_installed(self, tts_id, lang, variant) -> bool:
        """Check if a variant is installed on disk."""
        return os.path.isdir(self.variant_dir(tts_id, lang, variant))

    def installed_voices(self) -> list[dict]:
        """Return list of all installed voice entries."""
        ...

# From clone/_init_.py - CloneClient manages a subprocess
class CloneClient:
    def __init__(self, timeout: float = 600.0):
        self._proc: subprocess.Popen | None = None
        self._lock = threading.RLock()
        self._timeout = timeout
        self._next_id = 0

    def _ensure_proc(self):
        """Start the worker subprocess if not running."""
        if self._proc is not None and self._proc.poll() is None:
            return # Already running
        # Start new subprocess...

    def synthesize(self, text, sample_wav, language="en", speed=1.0):
        """Synthesize text using the worker subprocess."""
        self._ensure_proc()
        resp = self._request({"cmd": "synthesize", "text": text, ...})
        return np.frombuffer(base64.b64decode(resp["wav"]), dtype=np.int16)

    def close(self):
        """Shut down the worker subprocess."""
        with self._lock:
            if self._proc:
                self._proc.stdin.write(json.dumps({"cmd": "quit"}) + "\n")
                self._proc.terminate()
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `# From tts/models.py - ModelStore manages voice models on disk`
A comment: *From tts/models.py - ModelStore manages voice models on disk.*
2. **Line 2:** `class ModelStore:`
Defines class `ModelStore`.
3. **Line 3:** `"""Manages downloaded TTS voice models."""`
Docstring: *Manages downloaded TTS voice models..* Documents the purpose of the following code.
4. **Line 4:** `def __init__(self, state_file: str | None = None):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `self._state_file = state_file or self._default_state_file()`
Stores `state_file` or `self._default_state_file()` in instance attribute `_state_file`.
6. **Line 6:** `self._state: dict = self._load_state()`
`self._state: dict = self._load_state()`
7. **Line 7:** `def variant_dir(self, tts_id: str, lang: str, variant: str) -> str:`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
8. **Line 8:** `"""Return the disk path for a voice variant."""`
Docstring: *Return the disk path for a voice variant..* Documents the purpose of the following code.
9. **Line 9:** `return os.path.join(paths.models_dir(), tts_id, lang, variant)`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
10. **Line 10:** `def is_variant_installed(self, tts_id, lang, variant) -> bool:`
Defines function `is_variant_installed` accepting `(self, tts_id, lang, variant)`, returning `bool`.
11. **Line 11:** `"""Check if a variant is installed on disk."""`
Docstring: *Check if a variant is installed on disk..* Documents the purpose of the following code.
12. **Line 12:** `return os.path.isdir(self.variant_dir(tts_id, lang, variant))`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
13. **Line 13:** `def installed_voices(self) -> list[dict]:`
Defines function `installed_voices` accepting `(self)`, returning `list[dict]`.
14. **Line 14:** `"""Return list of all installed voice entries."""`

Docstring: *Return list of all installed voice entries..* Documents the purpose of the following code.

15. Line 15: `...`

Ellipsis: abstract method placeholder.

16. Line 16: `# From clone/__init__.py - CloneClient manages a subprocess`

A comment: *From clone/__init__.py - CloneClient manages a subprocess.*

17. Line 17: `class CloneClient:`

Defines class `CloneClient`.

18. Line 18: `def __init__(self, timeout: float = 600.0):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

19. Line 19: `self._proc: subprocess.Popen | None = None`

Starts a process. More control than `run()` — can read `stdout/stderr` asynchronously.

20. Line 20: `self._lock = threading.RLock()`

A reentrant lock that allows the same thread to acquire it multiple times.

21. Line 21: `self._timeout = timeout`

Stores `timeout` in instance attribute `_timeout`.

22. Line 22: `self._next_id = 0`

Stores `0` in instance attribute `_next_id`.

23. Line 23: `def _ensure_proc(self):`

Defines function `_ensure_proc` accepting `(self)`.

24. Line 24: `"""Start the worker subprocess if not running."""`

Docstring: *Start the worker subprocess if not running..* Documents the purpose of the following code.

25. Line 25: `if self._proc is not None and self._proc.poll() is None:`

Checks condition: `self._proc is not None and self._proc.poll() is None`. The indented block runs only when this is `True`.

26. Line 26: `return # Already running`

Returns `# Already running` to the caller.

27. Line 27: `# Start new subprocess...`

A comment: *Start new subprocess....*

28. Line 28: `def synthesize(self, text, sample_wav, language="en", speed=1.0):`

Defines function `synthesize` accepting (self, text, sample_wav, language, ...).

29. **Line 29:** `"""Synthesize text using the worker subprocess."""`

Docstring: *Synthesize text using the worker subprocess.* Documents the purpose of the following code.

30. **Line 30:** `self._ensure_proc()`

Calls `self._ensure_proc()` .

31. **Line 31:** `resp = self._request({"cmd": "synthesize", "text": text, ...})`

Assigns `self._request({"cmd": "synthesize", "text": text, ...})` to `resp` .

32. **Line 32:** `return np.frombuffer(base64.b64decode(resp["wav"]), dtype=np.int16)`

Returns `np.frombuffer(base64.b64decode(resp["wav"]), dtype=np.int16)` to the caller.

33. **Line 33:** `def close(self):`

Defines function `close` accepting (self).

34. **Line 34:** `"""Shut down the worker subprocess."""`

Docstring: *Shut down the worker subprocess.* Documents the purpose of the following code.

35. **Line 35:** `with self._lock:`

Context manager: `self._lock` . Automatically manages resource cleanup.

36. **Line 36:** `if self._proc:`

Checks condition: `self._proc` . The indented block runs only when this is `True` .

37. **Line 37:** `self._proc.stdin.write(json.dumps({"cmd": "quit"}) + "\n")`

Serializes a Python object to a JSON string.

38. **Line 38:** `self._proc.terminate()`

`self._proc.terminate()`

10.8 Composition vs. Inheritance

```
# Composition (preferred): objects contain other objects
class RecordingDialog:
    def __init__(self, parent, project_dir, settings, store):
        self.settings = settings    # Has-a Settings
        self.store = store          # Has-a ModelStore
        self.worker = None          # Will have-a SynthesisWorker

# Inheritance (less common): is-a relationship
class DownloadPanel(wx.Panel):     # DownloadPanel IS-A wx.Panel
    def __init__(self, parent, store, downloader):
        super().__init__(parent)   # Call parent's __init__
        self.store = store
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `# Composition (preferred): objects contain other objects`
A comment: *Composition (preferred): objects contain other objects.*
2. **Line 2:** `class RecordingDialog:`
Defines class `RecordingDialog`.
3. **Line 3:** `def __init__(self, parent, project_dir, settings, store):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `self.settings = settings # Has-a Settings`
Stores `settings` # Has-a Settings in instance attribute `settings`.
5. **Line 5:** `self.store = store # Has-a ModelStore`
Stores `store` # Has-a ModelStore in instance attribute `store`.
6. **Line 6:** `self._worker = None # Will have-a SynthesisWorker`
Stores `None` # Will have-a SynthesisWorker in instance attribute `_worker`.
7. **Line 7:** `# Inheritance (less common): is-a relationship`
A comment: *Inheritance (less common): is-a relationship.*
8. **Line 8:** `class DownloadPanel(wx.Panel): # DownloadPanel IS-A wx.Panel`
A container that sits inside a Frame. Groups controls and provides a background for drawing.
9. **Line 9:** `def __init__(self, parent, store, downloader):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
10. **Line 10:** `super().__init__(parent) # Call parent's __init__`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
11. **Line 11:** `self.store = store`
Stores `store` in instance attribute `store`.

 **Tip: Prefer composition over inheritance.** AIVS uses inheritance mainly for wxPython widgets (because wx requires it), and composition for everything else. Composition is more flexible and easier to test.

10.9 Key Takeaways

- **Classes** define blueprints; **objects** are instances of classes
- `__init__` initializes new objects — it is NOT the constructor (the constructor is `__new__`)
- `self` refers to the current instance — always the first parameter
- **Instance attributes** are per-object; **class attributes** are shared
- `@property` provides computed/validated attribute access
- **Dunder methods** (`__repr__`, `__eq__`, `__len__`) make objects integrate with Python's built-in functions
- **Prefer composition** — objects contain other objects rather than inheriting from them
- The AIVS project has classes for every major subsystem: Settings, ModelStore, CloneClient, RecordingDialog, etc.

 **Exercise: Exercise 10.1:** Create a `VoiceCatalog` class that wraps the TTS catalog data. It should have methods: `get_engines()`, `get_voices(engine_id)`, and `find_voice(voice_id)`. Include `__repr__` and `__len__`.

Intermediate Python

CHAPTER 11

Advanced Object-Oriented Programming

Learning Objectives

- Master inheritance, multiple inheritance, and MRO
 - Use `super()` correctly
 - Create abstract base classes
 - Understand dataclasses
 - Apply OOP design principles in AIVS
-

11.1 Inheritance Deep Dive

```
class BaseEngine:
    """Base class for all TTS engines."""
    sample_rate = 24000

    def __init__(self, voice_entry: dict):
        self.voice_entry = voice_entry

    def synthesize(self, text: str, speed: float = 1.0) -> "np.ndarray":
        raise NotImplementedError("Subclasses must implement synthesize")

    def close(self):
        pass # Default: no cleanup needed

class SherpaEngine(BaseEngine):
    """sherpa-onnx based TTS engine."""
    def __init__(self, voice_entry: dict, model_files: dict):
        super().__init__(voice_entry) # Call parent's __init__
        self.model_files = model_files
        self._model = None

    def synthesize(self, text, speed=1.0):
        if self._model is None:
            self._model = self._load_model()
        return self._model.generate(text, speed=speed)

class ClonedVoiceEngine(BaseEngine):
    """Voice Lab voice cloning engine (Pocket TTS, Bark, F5-TTS)."""
    def __init__(self, voice_entry: dict, client: "CloneClient"):
        super().__init__(voice_entry)
        self._client = client
        self.sample_wav = voice_entry.get("sample")

    def synthesize(self, text, speed=1.0):
        return self._client.synthesize(text, self.sample_wav, speed=speed)

    def close(self):
        self._client.close() # Override: shut down worker process
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `class BaseEngine:`
Defines class `BaseEngine` .
2. **Line 2:** `"""Base class for all TTS engines."""`
Docstring: *Base class for all TTS engines..* Documents the purpose of the following code.
3. **Line 3:** `sample_rate = 24000`
Assigns `24000` to `sample_rate` .
4. **Line 4:** `def __init__(self, voice_entry: dict):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `self.voice_entry = voice_entry`
Stores `voice_entry` in instance attribute `voice_entry` .
6. **Line 6:** `def synthesize(self, text: str, speed: float = 1.0) -> "np.ndarray":`
Defines function `synthesize` accepting (self, text, speed), returning `"np.ndarray"` .
7. **Line 7:** `raise NotImplementedError("Subclasses must implement synthesize")`
Manually raises an exception. Use to signal error conditions to callers.
8. **Line 8:** `def close(self):`
Defines function `close` accepting (self).
9. **Line 9:** `pass # Default: no cleanup needed`
`pass # Default: no cleanup needed`
10. **Line 10:** `class SherpaEngine(BaseEngine):`
Defines class `SherpaEngine` inheriting from `BaseEngine` .
11. **Line 11:** `"""sherpa-onnx based TTS engine."""`
Docstring: *sherpa-onnx based TTS engine..* Documents the purpose of the following code.
12. **Line 12:** `def __init__(self, voice_entry: dict, model_files: dict):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
13. **Line 13:** `super().__init__(voice_entry) # Call parent's __init__`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

14. **Line 14:** `self.model_files = model_files`
Stores `model_files` in instance attribute `model_files` .
15. **Line 15:** `self._model = None`
Stores `None` in instance attribute `_model` .
16. **Line 16:** `def synthesize(self, text, speed=1.0):`
Defines function `synthesize` accepting (self, text, speed).
17. **Line 17:** `if self._model is None:`
Checks condition: `self._model is None` . The indented block runs only when this is `True` .
18. **Line 18:** `self._model = self._load_model()`
Stores `self._load_model()` in instance attribute `_model` .
19. **Line 19:** `return self._model.generate(text, speed=speed)`
Returns `self._model.generate(text, speed=speed)` to the caller.
20. **Line 20:** `class ClonedVoiceEngine(BaseEngine):`
Defines class `ClonedVoiceEngine` inheriting from `BaseEngine` .
21. **Line 21:** `"""Voice Lab voice cloning engine (Pocket TTS, Bark, F5-TTS)."""`
Docstring: *Voice Lab voice cloning engine*. Documents the purpose of the following code.
22. **Line 22:** `def __init__(self, voice_entry: dict, client: "CloneClient"):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
23. **Line 23:** `super().__init__(voice_entry)`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
24. **Line 24:** `self._client = client`
Stores `client` in instance attribute `_client` .
25. **Line 25:** `self.sample_wav = voice_entry.get("sample")`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
26. **Line 26:** `def synthesize(self, text, speed=1.0):`
Defines function `synthesize` accepting (self, text, speed).
27. **Line 27:** `return self._client.synthesize(text, self.sample_wav, speed=speed)`
Returns `self._client.synthesize(text, self.sample_wav, speed=speed)` to the caller.

28. Line 28: `def close(self):`

Defines function `close` accepting (self).

29. Line 29: `self._client.close() # Override: shut down worker process`

Closes the file and flushes the write buffer. Prefer using 'with' instead.

11.2 Abstract Base Classes

```
from abc import ABC, abstractmethod

class TtsEngine(ABC):
    """Abstract base class – cannot be instantiated directly."""

    sample_rate: int = 24000

    @abstractmethod
    def synthesize(self, text: str, sid: int = 0,
                  speed: float = 1.0, pitch: float = 1.0,
                  volume: float = 1.0) -> "np.ndarray":
        """Synthesize text to audio samples."""
        ...

    def close(self):
        """Clean up resources. Override if needed."""
        pass

# This will fail:
# engine = TtsEngine() # TypeError: Can't instantiate abstract class

# Must implement all abstract methods
class MyEngine(TtsEngine):
    def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):
        # ... actual synthesis
        return np.zeros(100, dtype=np.int16)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `from abc import ABC, abstractmethod`

Imports `ABC, abstractmethod` from `abc`.

2. **Line 2:** `class TtsEngine(ABC):`

Defines class `TtsEngine` inheriting from `ABC`.

3. **Line 3:** `"""Abstract base class – cannot be instantiated directly."""`

Docstring: *Abstract base class – cannot be instantiated directly..* Documents the purpose of the following code.

4. **Line 4:** `sample_rate: int = 24000`

`sample_rate: int = 24000`

5. **Line 5:** `@abstractmethod`

Decorator marking a method as abstract. Subclasses must override it.

6. **Line 6:** `def synthesize(self, text: str, sid: int = 0,`

`def synthesize(self, text: str, sid: int = 0,`

7. **Line 7:** `speed: float = 1.0, pitch: float = 1.0,`

`speed: float = 1.0, pitch: float = 1.0,`

8. **Line 8:** `volume: float = 1.0) -> "np.ndarray":`

`volume: float = 1.0) -> "np.ndarray":`

9. **Line 9:** `"""Synthesize text to audio samples."""`

Docstring: *Synthesize text to audio samples..* Documents the purpose of the following code.

10. **Line 10:** `...`

Ellipsis: abstract method placeholder.

11. **Line 11:** `def close(self):`

Defines function `close` accepting (self).

12. **Line 12:** `"""Clean up resources. Override if needed."""`

Docstring: *Clean up resources. Override if needed..* Documents the purpose of the following code.

13. **Line 13:** `pass`

Placeholder: does nothing.

14. **Line 14:** `# This will fail:`
A comment: *This will fail.*
15. **Line 15:** `# engine = TtsEngine() # TypeError: Can't instantiate abstract class`
A comment: *engine = TtsEngine() # TypeError: Can't instantiate abstract class.*
16. **Line 16:** `# Must implement all abstract methods`
A comment: *Must implement all abstract methods.*
17. **Line 17:** `class MyEngine(TtsEngine):`
Defines class `MyEngine` inheriting from `TtsEngine`.
18. **Line 18:** `def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):`
Defines function `synthesize` accepting (self, text, sid, speed, ...).
19. **Line 19:** `# ... actual synthesis`
A comment: *... actual synthesis.*
20. **Line 20:** `return np.zeros(100, dtype=np.int16)`
Returns `np.zeros(100, dtype=np.int16)` to the caller.

11.3 Dataclasses

Dataclasses auto-generate `__init__`, `__repr__`, `__eq__`, and more:

```

from dataclasses import dataclass, field

@dataclass
class VoiceParams:
    """TTS voice parameters for a recording session."""
    rate: float = 1.0
    pitch: float = 1.0
    volume: float = 1.0
    compute: str = "auto"
    output_format: str = "wav"
    punctuation: str = "default"

# Auto-generated __init__
params = VoiceParams(rate=1.5, pitch=0.8)
print(params) # VoiceParams(rate=1.5, pitch=0.8, volume=1.0, ...)

# Auto-generated __eq__
params2 = VoiceParams(rate=1.5, pitch=0.8)
print(params == params2) # True

@dataclass(frozen=True) # Immutable
class AudioConfig:
    sample_rate: int = 24000
    channels: int = 1
    sample_width: int = 2 # 16-bit = 2 bytes

# frozen=True makes it hashable (can be dict key or set member)
configs = {AudioConfig(): "default"}

@dataclass
class Segment:
    index: int
    title: str
    text: str
    status: str = "pending"
    saved: str | None = None
    duration: float = 0.0

    def is_done(self) -> bool:
        return self.status == "done" and self.saved is not None

```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `from dataclasses import dataclass, field`
Imports `dataclass, field` from `dataclasses`.
2. **Line 2:** `@dataclass`
Decorator: `@dataclass`. Wraps the following function/class.
3. **Line 3:** `class VoiceParams:`
Defines class `VoiceParams`.
4. **Line 4:** `"""TTS voice parameters for a recording session."""`
Docstring: *TTS voice parameters for a recording session..* Documents the purpose of the following code.
5. **Line 5:** `rate: float = 1.0`
`rate: float = 1.0`
6. **Line 6:** `pitch: float = 1.0`
`pitch: float = 1.0`
7. **Line 7:** `volume: float = 1.0`
`volume: float = 1.0`
8. **Line 8:** `compute: str = "auto"`
`compute: str = "auto"`
9. **Line 9:** `output_format: str = "wav"`
`output_format: str = "wav"`
10. **Line 10:** `punctuation: str = "default"`
`punctuation: str = "default"`
11. **Line 11:** `# Auto-generated __init__`
A comment: *Auto-generated __init__*.
12. **Line 12:** `params = VoiceParams(rate=1.5, pitch=0.8)`
Assigns `VoiceParams(rate=1.5, pitch=0.8)` to `params`.
13. **Line 13:** `print(params) # VoiceParams(rate=1.5, pitch=0.8, volume=1.0, ...)`
Outputs text to the console. The primary way to display information during development and debugging.

14. **Line 14:** `# Auto-generated __eq__`
A comment: *Auto-generated __eq__*.
15. **Line 15:** `params2 = VoiceParams(rate=1.5, pitch=0.8)`
Assigns `VoiceParams(rate=1.5, pitch=0.8)` to `params2`.
16. **Line 16:** `print(params == params2) # True`
Outputs text to the console. The primary way to display information during development and debugging.
17. **Line 17:** `@dataclass(frozen=True) # Immutable`
Decorator: `@dataclass(frozen=True) # Immutable`. Wraps the following function/class.
18. **Line 18:** `class AudioConfig:`
Defines class `AudioConfig`.
19. **Line 19:** `sample_rate: int = 24000`
`sample_rate: int = 24000`
20. **Line 20:** `channels: int = 1`
`channels: int = 1`
21. **Line 21:** `sample_width: int = 2 # 16-bit = 2 bytes`
`sample_width: int = 2 # 16-bit = 2 bytes`
22. **Line 22:** `# frozen=True makes it hashable (can be dict key or set member)`
A comment: *frozen=True makes it hashable (can be dict key or set member)*.
23. **Line 23:** `configs = {AudioConfig(): "default"}`
Assigns `{AudioConfig(): "default"}` to `configs`.
24. **Line 24:** `@dataclass`
Decorator: `@dataclass`. Wraps the following function/class.
25. **Line 25:** `class Segment:`
Defines class `Segment`.
26. **Line 26:** `index: int`
`index: int`
27. **Line 27:** `title: str`

title: str

28. Line 28: text: str

text: str

29. Line 29: status: str = "pending"

status: str = "pending"

30. Line 30: saved: str | None = None

saved: str | None = None

31. Line 31: duration: float = 0.0

duration: float = 0.0

32. Line 32: def is_done(self) -> bool:

Defines function `is_done` accepting (self), returning `bool` .

33. Line 33: return self.status == "done" and self.saved is not None

Returns `None` (no value).

11.4 Mixins

```
# Mixins provide reusable behavior
class LoggingMixin:
    """Adds a logger to any class."""
    @property
    def log(self):
        import logging
        return logging.getLogger(type(self).__name__)

class SingletonMixin:
    """Makes a class a singleton."""
    _instance = None
    _lock = threading.Lock()

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

# Usage
class AddonManager(SingletonMixin, LoggingMixin):
    def discover(self):
        self.log.info("Discovering addons...")
```


Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# Mixins provide reusable behavior`
A comment: *Mixins provide reusable behavior.*
2. **Line 2:** `class LoggingMixin:`
Defines class `LoggingMixin`.
3. **Line 3:** `"""Adds a logger to any class."""`
Docstring: *Adds a logger to any class..* Documents the purpose of the following code.
4. **Line 4:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
5. **Line 5:** `def log(self):`
Defines function `log` accepting (self).
6. **Line 6:** `import logging`
Imports the `logging` module.
7. **Line 7:** `return logging.getLogger(type(self).__name__)`
Creates or retrieves a logger by name. Hierarchical naming enables per-module log levels.
8. **Line 8:** `class SingletonMixin:`
Defines class `SingletonMixin`.
9. **Line 9:** `"""Makes a class a singleton."""`
Docstring: *Makes a class a singleton..* Documents the purpose of the following code.
10. **Line 10:** `_instance = None`
Assigns `None` to `_instance`.
11. **Line 11:** `_lock = threading.Lock()`
A mutual exclusion lock. Only one thread can hold it at a time. Prevents race conditions.
12. **Line 12:** `def __new__(cls, *args, **kwargs):`
Defines function `__new__` accepting (cls, *args, **kwargs).
13. **Line 13:** `if cls._instance is None:`
Checks condition: `cls._instance is None`. The indented block runs only when this is `True`.

14. **Line 14:** `with cls._lock:`
Context manager: `cls._lock` . Automatically manages resource cleanup.
15. **Line 15:** `if cls._instance is None:`
Checks condition: `cls._instance is None` . The indented block runs only when this is `True` .
16. **Line 16:** `cls._instance = super().__new__(cls)`
Returns a proxy object that delegates method calls to a parent class. Used in `__init__` to call the parent constructor.
17. **Line 17:** `return cls._instance`
Returns `cls._instance` to the caller.
18. **Line 18:** `# Usage`
A comment: *Usage*.
19. **Line 19:** `class AddonManager(SingletonMixin, LoggingMixin):`
Defines class `AddonManager` inheriting from `SingletonMixin, LoggingMixin` .
20. **Line 20:** `def discover(self):`
Defines function `discover` accepting (self).
21. **Line 21:** `self.log.info("Discovering addons...")`
`self.log.info("Discovering addons...")`
-

11.5 Enums

```
from enum import Enum, auto

class ComputeMode(Enum):
    CPU = "cpu"
    CUDA = "cuda"
    AUTO = "auto"

class ProjectType(Enum):
    AUDIO_PLAYLIST = auto()
    DAISY_AUDIO = auto()
    DAISY_AUDIO_TEXT = auto()
    CLIPBOARD = auto()

# Usage
mode = ComputeMode.CUDA
print(mode.value) # "cuda"
print(mode.name)  # "CUDA"

if mode == ComputeMode.CUDA:
    print("GPU mode!")
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `from enum import Enum, auto`

Imports `Enum, auto` from `enum` .

2. **Line 2:** `class ComputeMode(Enum):`

Defines class `ComputeMode` inheriting from `Enum` .

3. **Line 3:** `CPU = "cpu"`

Assigns `"cpu"` to `CPU` .

4. **Line 4:** `CUDA = "cuda"`

Assigns `"cuda"` to `CUDA` .

5. **Line 5:** `AUTO = "auto"`

Assigns `"auto"` to `AUTO` .

6. **Line 6:** `class ProjectType(Enum):`

Defines class `ProjectType` inheriting from `Enum` .

7. **Line 7:** `AUDIO_PLAYLIST = auto()`

Assigns `auto()` to `AUDIO_PLAYLIST` .

8. **Line 8:** `DAISY_AUDIO = auto()`

Assigns `auto()` to `DAISY_AUDIO` .

9. **Line 9:** `DAISY_AUDIO_TEXT = auto()`

Assigns `auto()` to `DAISY_AUDIO_TEXT` .

10. **Line 10:** `CLIPBOARD = auto()`

Assigns `auto()` to `CLIPBOARD` .

11. **Line 11:** `# Usage`

A comment: *Usage*.

12. **Line 12:** `mode = ComputeMode.CUDA`

Assigns `ComputeMode.CUDA` to `mode` .

13. **Line 13:** `print(mode.value) # "cuda"`

Outputs text to the console. The primary way to display information during development and debugging.

14. **Line 14:** `print(mode.name) # "CUDA"`

Outputs text to the console. The primary way to display information during development and debugging.

15. **Line 15:** `if mode == ComputeMode.CUDA:`

Checks condition: `mode == ComputeMode.CUDA`. The indented block runs only when this is `True`.

16. **Line 16:** `print("GPU mode!")`

Outputs text to the console. The primary way to display information during development and debugging.

11.6 `__slots__` for Memory Efficiency

```
class LightweightVoice:
    """Uses __slots__ to reduce memory per instance."""
    __slots__ = ("voice_id", "name", "sid")

    def __init__(self, voice_id, name, sid):
        self.voice_id = voice_id
        self.name = name
        self.sid = sid

# Cannot add arbitrary attributes
v = LightweightVoice("af", "AF", 0)
# v.extra = "data" # AttributeError!

# Uses ~40% Less memory than a regular class
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `class LightweightVoice:`
Defines class `LightweightVoice`.
2. **Line 2:** `"""Uses __slots__ to reduce memory per instance."""`
Docstring: *Uses __slots__ to reduce memory per instance..* Documents the purpose of the following code.
3. **Line 3:** `__slots__ = ("voice_id", "name", "sid")`
Assigns `("voice_id", "name", "sid")` to `__slots__`.
4. **Line 4:** `def __init__(self, voice_id, name, sid):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `self.voice_id = voice_id`
Stores `voice_id` in instance attribute `voice_id`.
6. **Line 6:** `self.name = name`
Stores `name` in instance attribute `name`.
7. **Line 7:** `self.sid = sid`
Stores `sid` in instance attribute `sid`.
8. **Line 8:** `# Cannot add arbitrary attributes`
A comment: *Cannot add arbitrary attributes.*
9. **Line 9:** `v = LightweightVoice("af", "AF", 0)`
Assigns `LightweightVoice("af", "AF", 0)` to `v`.
10. **Line 10:** `# v.extra = "data" # AttributeError!`
A comment: *v.extra = "data" # AttributeError!.*
11. **Line 11:** `# Uses ~40% less memory than a regular class`
A comment: *Uses ~40% less memory than a regular class.*

11.7 Key Takeaways

- **Abstract base classes** define interfaces that subclasses must implement

- **Dataclasses** auto-generate boilerplate methods — use them for data containers
- **Mixins** provide reusable behavior without deep inheritance hierarchies
- `super()` should always be used — never hardcode parent class names
- **Enums** represent fixed sets of values (compute modes, project types)
- **Composition over inheritance** remains the golden rule

 **Exercise: Exercise 11.1:** Create a `@dataclass` for a `VoiceModel` with fields: `name`, `tts_id`, `language`, `variant`, `path`, `installed` (bool). Add a method `can_use_compute(mode)` that checks compatibility.

Iterators & Generators

Learning Objectives

- Understand the iterator protocol (`__iter__` and `__next__`)
 - Create generator functions with `yield`
 - Use generator expressions for memory-efficient processing
 - Apply `itertools` for advanced iteration patterns
-

12.1 The Iterator Protocol

```
# Every iterable has __iter__ that returns an iterator
# Every iterator has __next__ that returns the next value

class VoiceIterator:
    """Iterates over voice entries in the catalog."""
    def __init__(self, catalog):
        self._catalog = catalog
        self._tts_idx = 0
        self._lang_idx = 0
        self._var_idx = 0
        self._voice_idx = 0

    def __iter__(self):
        return self # Iterators return themselves

    def __next__(self):
        # Navigate through nested structure
        while self._tts_idx < len(self._catalog):
            tts = self._catalog[self._tts_idx]
            langs = tts.get("languages", [])
            while self._lang_idx < len(langs):
                lang = langs[self._lang_idx]
                variants = lang.get("variants", [])
                while self._var_idx < len(variants):
                    variant = variants[self._var_idx]
                    voices = variant.get("voices", [])
                    while self._voice_idx < len(voices):
                        voice = voices[self._voice_idx]
                        self._voice_idx += 1
                        return {"tts": tts["id"], "voice": voice}
                    self._var_idx += 1
                    self._voice_idx = 0
                self._lang_idx += 1
                self._var_idx = 0
            self._tts_idx += 1
            self._lang_idx = 0
        raise StopIteration

# Usage
for voice in VoiceIterator(catalog):
    print(voice)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Every iterable has __iter__ that returns an iterator`

A comment: *Every iterable has __iter__ that returns an iterator.*

2. **Line 2:** `# Every iterator has __next__ that returns the next value`

A comment: *Every iterator has __next__ that returns the next value.*

3. **Line 3:** `class VoiceIterator:`

Defines class `VoiceIterator`.

4. **Line 4:** `"""Iterates over voice entries in the catalog."""`

Docstring: *Iterates over voice entries in the catalog.*. Documents the purpose of the following code.

5. **Line 5:** `def __init__(self, catalog):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

6. **Line 6:** `self._catalog = catalog`

Stores `catalog` in instance attribute `_catalog`.

7. **Line 7:** `self._tts_idx = 0`

Stores `0` in instance attribute `_tts_idx`.

8. **Line 8:** `self._lang_idx = 0`

Stores `0` in instance attribute `_lang_idx`.

9. **Line 9:** `self._var_idx = 0`

Stores `0` in instance attribute `_var_idx`.

10. **Line 10:** `self._voice_idx = 0`

Stores `0` in instance attribute `_voice_idx`.

11. **Line 11:** `def __iter__(self):`

Returns an iterator. Called by for loops.

12. **Line 12:** `return self # Iterators return themselves`

Returns `self # Iterators return themselves` to the caller.

13. **Line 13:** `def __next__(self):`

Returns the next item from an iterator. Raises StopIteration when done.

14. **Line 14:** `# Navigate through nested structure`
A comment: *Navigate through nested structure.*
15. **Line 15:** `while self._tts_idx < len(self._catalog):`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
16. **Line 16:** `tts = self._catalog[self._tts_idx]`
Assigns `self._catalog[self._tts_idx]` to `tts`.
17. **Line 17:** `langs = tts.get("languages", [])`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
18. **Line 18:** `while self._lang_idx < len(langs):`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
19. **Line 19:** `lang = langs[self._lang_idx]`
Assigns `langs[self._lang_idx]` to `lang`.
20. **Line 20:** `variants = lang.get("variants", [])`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
21. **Line 21:** `while self._var_idx < len(variants):`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
22. **Line 22:** `variant = variants[self._var_idx]`
Assigns `variants[self._var_idx]` to `variant`.
23. **Line 23:** `voices = variant.get("voices", [])`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
24. **Line 24:** `while self._voice_idx < len(voices):`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
25. **Line 25:** `voice = voices[self._voice_idx]`
Assigns `voices[self._voice_idx]` to `voice`.
26. **Line 26:** `self._voice_idx += 1`
`self._voice_idx += 1`
27. **Line 27:** `return {"tts": tts["id"], "voice": voice}`
Returns `{"tts": tts["id"], "voice": voice}` to the caller.

```
28. Line 28: self._var_idx += 1
    self._var_idx += 1

29. Line 29: self._voice_idx = 0
    Stores 0 in instance attribute _voice_idx .

30. Line 30: self._lang_idx += 1
    self._lang_idx += 1

31. Line 31: self._var_idx = 0
    Stores 0 in instance attribute _var_idx .

32. Line 32: self._tts_idx += 1
    self._tts_idx += 1

33. Line 33: self._lang_idx = 0
    Stores 0 in instance attribute _lang_idx .

34. Line 34: raise StopIteration
    Raised by iterators when there are no more values to yield.

35. Line 35: # Usage
    A comment: Usage.

36. Line 36: for voice in VoiceIterator(catalog):
    Iterates over VoiceIterator(catalog) , assigning each element to voice .

37. Line 37: print(voice)
    Outputs text to the console. The primary way to display information during development and debugging.
```

12.2 Generator Functions

Generators use `yield` to produce values lazily, one at a time:

```
def voice_entries(catalog):
    """Yield all voice entries from the catalog (lazy)."""
    for tts in catalog:
        for lang in tts.get("languages", []):
            for variant in lang.get("variants", []):
                for voice in variant.get("voices", []):
                    yield {
                        "tts": tts["id"],
                        "language": lang["code"],
                        "variant": variant["id"],
                        "voice": voice,
                    }

# Usage - memory efficient (no list in memory)
for entry in voice_entries(catalog):
    print(entry["voice"]["id"])

# Can convert to list if needed
all_entries = list(voice_entries(catalog))
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def voice_entries(catalog):`
Defines function `voice_entries` accepting (catalog).
2. **Line 2:** `"""Yield all voice entries from the catalog (lazy)."""`
Docstring: *Yield all voice entries from the catalog (lazy)*.. Documents the purpose of the following code.
3. **Line 3:** `for tts in catalog:`
Iterates over `catalog`, assigning each element to `tts`.
4. **Line 4:** `for lang in tts.get("languages", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
5. **Line 5:** `for variant in lang.get("variants", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
6. **Line 6:** `for voice in variant.get("voices", []):`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
7. **Line 7:** `yield {`
Pauses the function and sends a value to the caller. The function can be resumed later. Makes the function a generator.
8. **Line 8:** `"tts": tts["id"],`
`"tts": tts["id"],`
9. **Line 9:** `"language": lang["code"],`
`"language": lang["code"],`
10. **Line 10:** `"variant": variant["id"],`
`"variant": variant["id"],`
11. **Line 11:** `"voice": voice,`
`"voice": voice,`
12. **Line 12:** `}`
`}`
13. **Line 13:** `# Usage – memory efficient (no list in memory)`
A comment: *Usage – memory efficient (no list in memory)*.

14. **Line 14:** `for entry in voice_entries(catalog):`
Iterates over `voice_entries(catalog)` , assigning each element to `entry` .
15. **Line 15:** `print(entry["voice"]["id"])`
Outputs text to the console. The primary way to display information during development and debugging.
16. **Line 16:** `# Can convert to list if needed`
A comment: *Can convert to list if needed.*
17. **Line 17:** `all_entries = list(voice_entries(catalog))`
Assigns `list(voice_entries(catalog))` to `all_entries` .

yield from — Delegating to Sub-generators

```
def all_paths(directory):  
    """Recursively yield all file paths."""  
    for entry in os.scandir(directory):  
        if entry.is_file():  
            yield entry.path  
        elif entry.is_dir():  
            yield from all_paths(entry.path) # Delegate to sub-generator
```

Explain Line by Line Code Description (Block 3)

- Line 1:** `def all_paths(directory):`
Defines function `all_paths` accepting `(directory)`.
- Line 2:** `"""Recursively yield all file paths."""`
Docstring: *Recursively yield all file paths..* Documents the purpose of the following code.
- Line 3:** `for entry in os.scandir(directory):`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
- Line 4:** `if entry.is_file():`
Checks condition: `entry.is_file()`. The indented block runs only when this is `True`.
- Line 5:** `yield entry.path`
Pauses the function and sends a value to the caller. The function can be resumed later. Makes the function a generator.
- Line 6:** `elif entry.is_dir():`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
- Line 7:** `yield from all_paths(entry.path) # Delegate to sub-generator`
Delegates to a sub-generator. Shorthand for iterating and yielding each item.

12.3 Generator Expressions

```
# List comprehension creates list in memory
big_list = [x ** 2 for x in range(1_000_000)] # ~8MB in memory

# Generator expression - lazy evaluation (almost no memory)
big_gen = (x ** 2 for x in range(1_000_000)) # One number at a time

# Common pattern: sum/count/min/max on generators
total_chars = sum(len(v["text"]) for v in segments)
max_length = max(len(s["text"]) for s in segments)
has_empty = any(not s["text"].strip() for s in segments)
```


Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# List comprehension creates list in memory`
A comment: *List comprehension creates list in memory.*
2. **Line 2:** `big_list = [x ** 2 for x in range(1_000_000)] # ~8MB in memory`
Generates a sequence of integers. Commonly used in for loops.
3. **Line 3:** `# Generator expression – lazy evaluation (almost no memory)`
A comment: *Generator expression – lazy evaluation (almost no memory).*
4. **Line 4:** `big_gen = (x ** 2 for x in range(1_000_000)) # One number at a time`
Generates a sequence of integers. Commonly used in for loops.
5. **Line 5:** `# Common pattern: sum/count/min/max on generators`
A comment: *Common pattern: sum/count/min/max on generators.*
6. **Line 6:** `total_chars = sum(len(v["text"]) for v in segments)`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
7. **Line 7:** `max_length = max(len(s["text"]) for s in segments)`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
8. **Line 8:** `has_empty = any(not s["text"].strip() for s in segments)`
Removes leading and trailing whitespace (or specified characters). Returns a new string.

12.4 itertools — Advanced Iteration

```
import itertools

# chain – concatenate iterables
all_voices = list(itertools.chain(piper_voices, kokoro_voices))

# islice – slice an iterator
first_10 = list(itertools.islice(voice_iterator, 10))

# groupby – group consecutive elements
from itertools import groupby
voices_sorted = sorted(voices, key=lambda v: v["tts"])
for tts_id, group in groupby(voices_sorted, key=lambda v: v["tts"]):
    print(f"{tts_id}: {len(list(group))} voices")

# product – Cartesian product
engines = ["piper", "kokoro"]
languages = ["en", "zh"]
for engine, lang in itertools.product(engines, languages):
    print(f"{engine}/{lang}")

# cycle – infinite cycling
spinner = itertools.cycle(["|", "/", "-", "\\"])
for _ in range(8):
    print(f"\r{next(spinner)} Loading...", end="")
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `import itertools`
Imports the `itertools` module.
2. **Line 2:** `# chain – concatenate iterables`
A comment: *chain – concatenate iterables*.
3. **Line 3:** `all_voices = list(itertools.chain(piper_voices, kokoro_voices))`
Combines multiple iterables into a single sequence.
4. **Line 4:** `# islice – slice an iterator`
A comment: *islice – slice an iterator*.
5. **Line 5:** `first_10 = list(itertools.islice(voice_iterator, 10))`
Creates a slice object for sequence slicing. Useful for dynamic slice construction.
6. **Line 6:** `# groupby – group consecutive elements`
A comment: *groupby – group consecutive elements*.
7. **Line 7:** `from itertools import groupby`
Imports `groupby` from `itertools`.
8. **Line 8:** `voices_sorted = sorted(voices, key=lambda v: v["tts"])`
Returns a new sorted list from any iterable. Does not modify the original. Accepts a `key=` parameter for custom sorting.
9. **Line 9:** `for tts_id, group in groupby(voices_sorted, key=lambda v: v["tts"]):`
`for tts_id, group in groupby(voices_sorted, key=lambda v: v["tts"]):`
10. **Line 10:** `print(f"{tts_id}: {len(list(group))} voices")`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `# product – Cartesian product`
A comment: *product – Cartesian product*.
12. **Line 12:** `engines = ["piper", "kokoro"]`
Assigns `["piper", "kokoro"]` to `engines`.
13. **Line 13:** `languages = ["en", "zh"]`
Assigns `["en", "zh"]` to `languages`.
14. **Line 14:** `for engine, lang in itertools.product(engines, languages):`

Cartesian product of input iterables.

15. **Line 15:** `print(f"{engine}/{lang}")`

Outputs text to the console. The primary way to display information during development and debugging.

16. **Line 16:** `# cycle – infinite cycling`

A comment: *cycle – infinite cycling*.

17. **Line 17:** `spinner = itertools.cycle(["|", "/", "-", "\\"])`

Assigns `itertools.cycle(["|", "/", "-", "\\"])` to `spinner`.

18. **Line 18:** `for _ in range(8):`

Generates a sequence of integers. Commonly used in for loops.

19. **Line 19:** `print(f"\r{next(spinner)} Loading...", end="")`

Outputs text to the console. The primary way to display information during development and debugging.

12.5 Real Example: Segmented File Reading

```
# From AIVS: reading text files in chunks for large documents
def read_in_chunks(file_path: str, chunk_size: int = 4096):
    """Read a file in chunks – memory efficient."""
    with open(file_path, "r", encoding="utf-8") as f:
        while True:
            chunk = f.read(chunk_size)
            if not chunk:
                break
            yield chunk

# Count words without loading entire file
def count_words(file_path: str) -> int:
    return sum(chunk.count(" ") + 1 for chunk in read_in_chunks(file_path))
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `# From AIVS: reading text files in chunks for large documents`

A comment: *From AIVS: reading text files in chunks for large documents.*

2. **Line 2:** `def read_in_chunks(file_path: str, chunk_size: int = 4096):`

Defines function `read_in_chunks` accepting `(file_path, chunk_size)`.

3. **Line 3:** `"""Read a file in chunks – memory efficient."""`

Docstring: *Read a file in chunks – memory efficient..* Documents the purpose of the following code.

4. **Line 4:** `with open(file_path, "r", encoding="utf-8") as f:`

Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.

5. **Line 5:** `while True:`

While loop: repeats as long as `True` .

6. **Line 6:** `chunk = f.read(chunk_size)`

Assigns `f.read(chunk_size)` to `chunk` .

7. **Line 7:** `if not chunk:`

Checks condition: `not chunk` . The indented block runs only when this is `True` .

8. **Line 8:** `break`

Exits the innermost loop.

9. **Line 9:** `yield chunk`

Pauses the function and sends a value to the caller. The function can be resumed later. Makes the function a generator.

10. **Line 10:** `# Count words without loading entire file`

A comment: *Count words without loading entire file.*

11. **Line 11:** `def count_words(file_path: str) -> int:`

Defines function `count_words` accepting `(file_path)`, returning `int` .

12. **Line 12:** `return sum(chunk.count(" ") + 1 for chunk in read_in_chunks(file_path))`

Returns the number of occurrences of a value.

12.6 Key Takeaways

- **Generators** are the Pythonic way to create iterators — much simpler than implementing `__iter__` / `__next__`
 - **yield** pauses the function and produces a value; **yield from** delegates to sub-generators
 - Generators are **lazy** — they produce values on demand, saving memory
 - **Generator expressions** are like list comprehensions but with parentheses: `(x for x in ...)`
 - **itertools** provides powerful iteration building blocks (chain, groupby, product, etc.)
-

Decorators

Learning Objectives

- Understand how decorators work (functions wrapping functions)
 - Create custom decorators with and without parameters
 - Use `functools.wraps` to preserve metadata
 - See decorator patterns in AI/VS (property, classmethod, etc.)
-

13.1 Function Basics — Functions Are Objects

```
# Functions are first-class objects in Python
def greet(name):
    return f"Hello, {name}!"

# Assign to a variable
say_hello = greet
print(say_hello("World")) # "Hello, World!"

# Pass as argument
def apply(func, value):
    return func(value)

print(apply(greet, "Python")) # "Hello, Python!"

# Return from another function
def make_greeting(greeting):
    def inner(name):
        return f"{greeting}, {name}!"
    return inner

hello = make_greeting("Hello")
hola = make_greeting("Hola")
print(hello("Alice")) # "Hello, Alice!"
print(hola("Bob")) # "Hola, Bob!"
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Functions are first-class objects in Python`

A comment: *Functions are first-class objects in Python.*

2. **Line 2:** `def greet(name):`

Defines function `greet` accepting (name).

3. **Line 3:** `return f"Hello, {name}!"`

Returns `f"Hello, {name}!"` to the caller.

4. **Line 4:** `# Assign to a variable`

A comment: *Assign to a variable.*

5. **Line 5:** `say_hello = greet`

Assigns `greet` to `say_hello`.

6. **Line 6:** `print(say_hello("World")) # "Hello, World!"`

Outputs text to the console. The primary way to display information during development and debugging.

7. **Line 7:** `# Pass as argument`

A comment: *Pass as argument.*

8. **Line 8:** `def apply(func, value):`

Defines function `apply` accepting (func, value).

9. **Line 9:** `return func(value)`

Returns `func(value)` to the caller.

10. **Line 10:** `print(apply(greet, "Python")) # "Hello, Python!"`

Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `# Return from another function`

A comment: *Return from another function.*

12. **Line 12:** `def make_greeting(greeting):`

Defines function `make_greeting` accepting (greeting).

13. **Line 13:** `def inner(name):`

Defines function `inner` accepting (name).

14. **Line 14:** `return f"{greeting}, {name}!"`
Returns `f"{greeting}, {name}!"` to the caller.
 15. **Line 15:** `return inner`
Returns `inner` to the caller.
 16. **Line 16:** `hello = make_greeting("Hello")`
Assigns `make_greeting("Hello")` to `hello` .
 17. **Line 17:** `hola = make_greeting("Hola")`
Assigns `make_greeting("Hola")` to `hola` .
 18. **Line 18:** `print(hello("Alice")) # "Hello, Alice!"`
Outputs text to the console. The primary way to display information during development and debugging.
 19. **Line 19:** `print(hola("Bob")) # "Hola, Bob!"`
Outputs text to the console. The primary way to display information during development and debugging.
-

13.2 Writing Your First Decorator

```
# A decorator is a function that takes a function and returns a new function
import functools

def timer(func):
    """Decorator that prints how long a function takes."""
    @functools.wraps(func) # Preserves the original function's name/docstring
    def wrapper(*args, **kwargs):
        import time
        start = time.perf_counter()
        result = func(*args, **kwargs)
        elapsed = time.perf_counter() - start
        print(f"{func.__name__} took {elapsed:.3f}s")
        return result
    return wrapper

# Apply with @ syntax
@timer
def load_model(path):
    import time; time.sleep(0.1) # Simulate Loading
    return "model"

result = load_model("model.onnx")
# Output: load_model took 0.102s

# Equivalent without @ syntax
def load_model2(path):
    return "model"
load_model2 = timer(load_model2)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# A decorator is a function that takes a function and returns a new function`

A comment: *A decorator is a function that takes a function and returns a new function.*

2. **Line 2:** `import functools`

Imports the `functools` module.

3. **Line 3:** `def timer(func):`

Defines function `timer` accepting `(func)`.

4. **Line 4:** `"""Decorator that prints how long a function takes."""`

Docstring: *Decorator that prints how long a function takes..* Documents the purpose of the following code.

5. **Line 5:** `@functools.wraps(func) # Preserves the original function's name/docstring`

Decorator that preserves the wrapped function's name, docstring, and other metadata.

6. **Line 6:** `def wrapper(*args, **kwargs):`

Defines function `wrapper` accepting `(*args, **kwargs)`.

7. **Line 7:** `import time`

Imports the `time` module.

8. **Line 8:** `start = time.perf_counter()`

Assigns `time.perf_counter()` to `start`.

9. **Line 9:** `result = func(*args, **kwargs)`

Assigns `func(*args, **kwargs)` to `result`.

10. **Line 10:** `elapsed = time.perf_counter() - start`

Assigns `time.perf_counter() - start` to `elapsed`.

11. **Line 11:** `print(f"{func.__name__} took {elapsed:.3f}s")`

Outputs text to the console. The primary way to display information during development and debugging.

12. **Line 12:** `return result`

Returns `result` to the caller.

13. **Line 13:** `return wrapper`

Returns `wrapper` to the caller.

14. **Line 14:** `# Apply with @ syntax`
A comment: *Apply with @ syntax.*
15. **Line 15:** `@timer`
Decorator: `@timer` . Wraps the following function/class.
16. **Line 16:** `def load_model(path):`
Defines function `load_model` accepting (path).
17. **Line 17:** `import time; time.sleep(0.1) # Simulate loading`
Imports the `time; time.sleep(0.1) # Simulate loading` module.
18. **Line 18:** `return "model"`
Returns `"model"` to the caller.
19. **Line 19:** `result = load_model("model.onnx")`
Assigns `load_model("model.onnx")` to `result` .
20. **Line 20:** `# Output: load_model took 0.102s`
A comment: *Output: load_model took 0.102s.*
21. **Line 21:** `# Equivalent without @ syntax`
A comment: *Equivalent without @ syntax.*
22. **Line 22:** `def load_model2(path):`
Defines function `load_model2` accepting (path).
23. **Line 23:** `return "model"`
Returns `"model"` to the caller.
24. **Line 24:** `load_model2 = timer(load_model2)`
Assigns `timer(load_model2)` to `load_model2` .
-

13.3 Decorators with Arguments

```
def retry(max_attempts=3, delay=1.0):
    """Decorator that retries a function on failure."""
    def decorator(func):
        @functools.wraps(func)
        def wrapper(*args, **kwargs):
            for attempt in range(max_attempts):
                try:
                    return func(*args, **kwargs)
                except Exception as exc:
                    if attempt == max_attempts - 1:
                        raise
                    print(f"Attempt {attempt+1} failed, retrying...")
                    time.sleep(delay)
            return wrapper
        return decorator

@retry(max_attempts=5, delay=2.0)
def download_model(url):
    import requests
    return requests.get(url, timeout=30)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `def retry(max_attempts=3, delay=1.0):`
Defines function `retry` accepting `(max_attempts, delay)`.
2. **Line 2:** `"""Decorator that retries a function on failure."""`
Docstring: *Decorator that retries a function on failure..* Documents the purpose of the following code.
3. **Line 3:** `def decorator(func):`
Defines function `decorator` accepting `(func)`.
4. **Line 4:** `@functools.wraps(func)`
Decorator that preserves the wrapped function's name, docstring, and other metadata.
5. **Line 5:** `def wrapper(*args, **kwargs):`
Defines function `wrapper` accepting `(*args, **kwargs)`.
6. **Line 6:** `for attempt in range(max_attempts):`
Generates a sequence of integers. Commonly used in for loops.
7. **Line 7:** `try:`
Begins exception handling. Code in this block is monitored for errors.
8. **Line 8:** `return func(*args, **kwargs)`
Returns `func(*args, **kwargs)` to the caller.
9. **Line 9:** `except Exception as exc:`
Catches `Exception` exceptions.
10. **Line 10:** `if attempt == max_attempts - 1:`
Checks condition: `attempt == max_attempts - 1`. The indented block runs only when this is `True`.
11. **Line 11:** `raise`
`raise`
12. **Line 12:** `print(f"Attempt {attempt+1} failed, retrying...")`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `time.sleep(delay)`
`time.sleep(delay)`

14. **Line 14:** `return wrapper`
Returns `wrapper` to the caller.
 15. **Line 15:** `return decorator`
Returns `decorator` to the caller.
 16. **Line 16:** `@retry(max_attempts=5, delay=2.0)`
Decorator: `@retry(max_attempts=5, delay=2.0)` . Wraps the following function/class.
 17. **Line 17:** `def download_model(url):`
Defines function `download_model` accepting (url).
 18. **Line 18:** `import requests`
Imports the `requests` module.
 19. **Line 19:** `return requests.get(url, timeout=30)`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
-

13.4 Common Built-in Decorators

```
class MyClass:
    # @property - computed attribute
    @property
    def display_name(self):
        return f"{self._name} v{self._version}"

    # @classmethod - receives class instead of instance
    @classmethod
    def from_dict(cls, data: dict):
        return cls(data["name"], data["version"])

    # @staticmethod - no access to self or cls
    @staticmethod
    def is_valid(name: str) -> bool:
        return bool(name.strip())

# functools.lru_cache - memoization
@functools.lru_cache(maxsize=128)
def expensive_lookup(voice_id: str) -> dict:
    """Cache expensive database queries."""
    return db.query(voice_id) # Only runs once per voice_id

# functools.cache (Python 3.9+) - unlimited cache
@functools.cache
def parse_version(version_str: str) -> tuple:
    return tuple(int(x) for x in version_str.split("."))
```


Explain Line by Line Code Description (Block 4)

1. **Line 1:** `class MyClass:`
Defines class `MyClass`.
2. **Line 2:** `# @property – computed attribute`
A comment: *@property – computed attribute.*
3. **Line 3:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
4. **Line 4:** `def display_name(self):`
Defines function `display_name` accepting (self).
5. **Line 5:** `return f"{self._name} v{self._version}"`
Returns `f"{self._name} v{self._version}"` to the caller.
6. **Line 6:** `# @classmethod – receives class instead of instance`
A comment: *@classmethod – receives class instead of instance.*
7. **Line 7:** `@classmethod`
Decorator that receives the class (cls) as the first argument instead of the instance.
8. **Line 8:** `def from_dict(cls, data: dict):`
Defines function `from_dict` accepting (cls, data).
9. **Line 9:** `return cls(data["name"], data["version"])`
Returns `cls(data["name"], data["version"])` to the caller.
10. **Line 10:** `# @staticmethod – no access to self or cls`
A comment: *@staticmethod – no access to self or cls.*
11. **Line 11:** `@staticmethod`
Decorator that creates a method without 'self' or 'cls' parameter. Called on the class, not an instance.
12. **Line 12:** `def is_valid(name: str) -> bool:`
Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.
13. **Line 13:** `return bool(name.strip())`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
14. **Line 14:** `# functools.lru_cache – memoization`

A comment: *functools.lru_cache* — memoization.

15. **Line 15:** `@functools.lru_cache(maxsize=128)`

Decorator that caches function results based on arguments.

16. **Line 16:** `def expensive_lookup(voice_id: str) -> dict:`

Defines function `expensive_lookup` accepting `(voice_id)`, returning `dict`.

17. **Line 17:** `"""Cache expensive database queries."""`

Docstring: *Cache expensive database queries..* Documents the purpose of the following code.

18. **Line 18:** `return db.query(voice_id) # Only runs once per voice_id`

Returns `db.query(voice_id)` # Only runs once per `voice_id` to the caller.

19. **Line 19:** `# functools.cache (Python 3.9+) — unlimited cache`

A comment: *functools.cache (Python 3.9+) — unlimited cache.*

20. **Line 20:** `@functools.cache`

Decorator: `@functools.cache`. Wraps the following function/class.

21. **Line 21:** `def parse_version(version_str: str) -> tuple:`

Defines function `parse_version` accepting `(version_str)`, returning `tuple`.

22. **Line 22:** `return tuple(int(x) for x in version_str.split("."))`

Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.

13.5 Decorator Patterns in AIVS

The @property decorator pattern used throughout AIVS:

```
class Settings:
    @property
    def theme(self) -> str:
        return self._data.get("theme", "system")

    @property
    def compute(self) -> str:
        return self._data.get("compute", "auto")
```

Context manager decorator pattern:

```
from contextlib import contextmanager
```

```
@contextmanager
```

```
def busy_cursor():
    """Show busy cursor during an operation."""
    wx.BeginBusyCursor()
    try:
        yield
    finally:
        wx.EndBusyCursor()
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# The @property decorator pattern used throughout AIVS:`
A comment: *The @property decorator pattern used throughout AIVS:*.
2. **Line 2:** `class Settings:`
Defines class `Settings` .
3. **Line 3:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
4. **Line 4:** `def theme(self) -> str:`
Defines function `theme` accepting (self), returning `str` .
5. **Line 5:** `return self._data.get("theme", "system")`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
6. **Line 6:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
7. **Line 7:** `def compute(self) -> str:`
Defines function `compute` accepting (self), returning `str` .
8. **Line 8:** `return self._data.get("compute", "auto")`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
9. **Line 9:** `# Context manager decorator pattern:`
A comment: *Context manager decorator pattern:*.
10. **Line 10:** `from contextlib import contextmanager`
Imports `contextmanager` from `contextlib` .
11. **Line 11:** `@contextmanager`
Decorator: `@contextmanager` . Wraps the following function/class.
12. **Line 12:** `def busy_cursor():`
Defines function `busy_cursor` .
13. **Line 13:** `"""Show busy cursor during an operation."""`
Docstring: *Show busy cursor during an operation.*. Documents the purpose of the following code.
14. **Line 14:** `wx.BeginBusyCursor()`

Changes the cursor to a busy indicator (hourglass).

15. **Line 15:** `try:`

Begins exception handling. Code in this block is monitored for errors.

16. **Line 16:** `yield`

`yield`

17. **Line 17:** `finally:`

Always executes for cleanup code, whether or not an exception occurred.

18. **Line 18:** `wx.EndBusyCursor()`

Restores the cursor to normal after `BeginBusyCursor()`.

13.6 Key Takeaways

- A decorator is a **function that wraps another function**, modifying its behavior
- Always use `@functools.wraps(func)` to preserve the original function's metadata
- Decorators with arguments need **three levels** of nesting
- Built-in decorators: `@property`, `@classmethod`, `@staticmethod`, `@lru_cache`
- Decorators are used in AIVS for caching, timing, and GUI cursor management

 **Exercise: Exercise 13.1:** Write a `@log_calls` decorator that prints the function name and all arguments before each call.

Context Managers

Learning Objectives

- Understand the `with` statement and context manager protocol
 - Create context managers using classes and `@contextmanager`
 - Apply context managers for resource management in AIVS
-

14.1 The Context Manager Protocol

```
# Context managers implement __enter__ and __exit__
class ManagedEngine:
    """TTS engine that cleans up when done."""

    def __init__(self, engine_id: str):
        self.engine_id = engine_id
        self._model = None

    def __enter__(self):
        """Called when entering the 'with' block."""
        print(f>Loading {self.engine_id}...")
        self._model = "loaded"
        return self # This is assigned to 'as' variable

    def __exit__(self, exc_type, exc_val, exc_tb):
        """Called when exiting the 'with' block (always)."""
        print(f>Unloading {self.engine_id}...")
        self._model = None
        return False # Don't suppress exceptions

# Usage
with ManagedEngine("piper") as engine:
    print(f>Using {engine.engine_id}")
    # ... use the engine ...
# "Unloading piper..." is printed here, even if an exception occurs
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Context managers implement __enter__ and __exit__`

A comment: *Context managers implement __enter__ and __exit__.*

2. **Line 2:** `class ManagedEngine:`

Defines class `ManagedEngine`.

3. **Line 3:** `"""TTS engine that cleans up when done."""`

Docstring: *TTS engine that cleans up when done..* Documents the purpose of the following code.

4. **Line 4:** `def __init__(self, engine_id: str):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

5. **Line 5:** `self.engine_id = engine_id`

Stores `engine_id` in instance attribute `engine_id`.

6. **Line 6:** `self._model = None`

Stores `None` in instance attribute `_model`.

7. **Line 7:** `def __enter__(self):`

Called when entering a 'with' block. Returns the context manager's value.

8. **Line 8:** `"""Called when entering the 'with' block."""`

Docstring: *Called when entering the 'with' block..* Documents the purpose of the following code.

9. **Line 9:** `print(f"Loading {self.engine_id}...")`

Outputs text to the console. The primary way to display information during development and debugging.

10. **Line 10:** `self._model = "loaded"`

Stores `"loaded"` in instance attribute `_model`.

11. **Line 11:** `return self # This is assigned to 'as' variable`

Returns `self # This is assigned to 'as' variable` to the caller.

12. **Line 12:** `def __exit__(self, exc_type, exc_val, exc_tb):`

Called when leaving a 'with' block. Handles cleanup and exception suppression.

13. **Line 13:** `"""Called when exiting the 'with' block (always)."""`

Docstring: *Called when exiting the 'with' block (always)..* Documents the purpose of the following code.

14. **Line 14:** `print(f"Unloading {self.engine_id}...")`

Outputs text to the console. The primary way to display information during development and debugging.

15. **Line 15:** `self._model = None`

Stores `None` in instance attribute `_model`.

16. **Line 16:** `return False # Don't suppress exceptions`

Returns `False # Don't suppress exceptions` to the caller.

17. **Line 17:** `# Usage`

A comment: *Usage*.

18. **Line 18:** `with ManagedEngine("piper") as engine:`

Context manager: `ManagedEngine("piper") as engine`. Automatically manages resource cleanup.

19. **Line 19:** `print(f"Using {engine.engine_id}")`

Outputs text to the console. The primary way to display information during development and debugging.

20. **Line 20:** `# ... use the engine ...`

A comment: *... use the engine ...*

21. **Line 21:** `# "Unloading piper..." is printed here, even if an exception occurs`

A comment: *"Unloading piper..." is printed here, even if an exception occurs.*

The `__exit__` Parameters

Parameter	Type	Description
<code>exc_type</code>	<code>type</code> <code>None</code>	The exception class (or <code>None</code> if no exception)
<code>exc_val</code>	<code>Exception</code> <code>None</code>	The exception instance
<code>exc_tb</code>	<code>traceback</code> <code>None</code>	The traceback object

```
def __exit__(self, exc_type, exc_val, exc_tb):
    if exc_type is not None:
        print(f"Exception occurred: {exc_val}")
    return False # False = don't suppress the exception
                # return True # True = suppress the exception
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def __exit__(self, exc_type, exc_val, exc_tb):`
Called when leaving a 'with' block. Handles cleanup and exception suppression.
2. **Line 2:** `if exc_type is not None:`
Checks condition: `exc_type is not None`. The indented block runs only when this is `True`.
3. **Line 3:** `print(f"Exception occurred: {exc_val}")`
Outputs text to the console. The primary way to display information during development and debugging.
4. **Line 4:** `return False # False = don't suppress the exception`
Returns `False # False = don't suppress the exception` to the caller.
5. **Line 5:** `# return True # True = suppress the exception`
A comment: `return True # True = suppress the exception`.

14.2 @contextmanager Decorator

```
from contextlib import contextmanager

@contextmanager
def managed_engine(engine_id: str):
    """Simpler context manager using @contextmanager."""
    print(f>Loading {engine_id}...")
    engine = "loaded"
    try:
        yield engine # This is the 'as' value
    finally:
        print(f>Unloading {engine_id}...")

# Usage (identical to the class version)
with managed_engine("piper") as engine:
    print(f>Using {engine}")
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `from contextlib import contextmanager`
Imports `contextmanager` from `contextlib`.
2. **Line 2:** `@contextmanager`
Decorator: `@contextmanager`. Wraps the following function/class.
3. **Line 3:** `def managed_engine(engine_id: str):`
Defines function `managed_engine` accepting `(engine_id)`.
4. **Line 4:** `"""Simpler context manager using @contextmanager."""`
Docstring: *Simpler context manager using @contextmanager..* Documents the purpose of the following code.
5. **Line 5:** `print(f"Loading {engine_id}...")`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `engine = "loaded"`
Assigns `"loaded"` to `engine`.
7. **Line 7:** `try:`
Begins exception handling. Code in this block is monitored for errors.
8. **Line 8:** `yield engine # This is the 'as' value`
Pauses the function and sends a value to the caller. The function can be resumed later. Makes the function a generator.
9. **Line 9:** `finally:`
Always executes for cleanup code, whether or not an exception occurred.
10. **Line 10:** `print(f"Unloading {engine_id}...")`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `# Usage (identical to the class version)`
A comment: *Usage (identical to the class version).*
12. **Line 12:** `with managed_engine("piper") as engine:`
Context manager: `managed_engine("piper") as engine`. Automatically manages resource cleanup.
13. **Line 13:** `print(f"Using {engine}")`
Outputs text to the console. The primary way to display information during development and debugging.

14.3 Context Managers in AIVS

```
# GUI busy cursor management
from contextlib import contextmanager

@contextmanager
def busy_cursor():
    """Show busy cursor during document parsing."""
    wx.BeginBusyCursor()
    try:
        yield
    finally:
        wx.EndBusyCursor()

# Usage in NewProjectWizard
with busy_cursor():
    doc = parse_document(self.source_path)
    segments = split_document(doc, mode)

# Thread-safe Lock management
class CloneClient:
    def _request(self, req: dict) -> dict:
        with self._lock: # Thread-safe access
            # ... send request and read response

# Temporary directory for downloads
import tempfile
with tempfile.TemporaryDirectory() as tmpdir:
    # Download to tmpdir, process, then it's auto-cleaned
    download_to(tmpdir)

# contextlib.suppress – ignore specific exceptions
from contextlib import suppress
with suppress(FileNotFoundError):
    os.unlink(temp_file) # Ignore if file doesn't exist
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# GUI busy cursor management`

A comment: *GUI busy cursor management*.

2. **Line 2:** `from contextlib import contextmanager`

Imports `contextmanager` from `contextlib`.

3. **Line 3:** `@contextmanager`

Decorator: `@contextmanager`. Wraps the following function/class.

4. **Line 4:** `def busy_cursor():`

Defines function `busy_cursor`.

5. **Line 5:** `"""Show busy cursor during document parsing."""`

Docstring: *Show busy cursor during document parsing..* Documents the purpose of the following code.

6. **Line 6:** `wx.BeginBusyCursor()`

Changes the cursor to a busy indicator (hourglass).

7. **Line 7:** `try:`

Begins exception handling. Code in this block is monitored for errors.

8. **Line 8:** `yield`

`yield`

9. **Line 9:** `finally:`

Always executes for cleanup code, whether or not an exception occurred.

10. **Line 10:** `wx.EndBusyCursor()`

Restores the cursor to normal after `BeginBusyCursor()`.

11. **Line 11:** `# Usage in NewProjectWizard`

A comment: *Usage in NewProjectWizard*.

12. **Line 12:** `with busy_cursor():`

Context manager: `busy_cursor()`. Automatically manages resource cleanup.

13. **Line 13:** `doc = parse_document(self.source_path)`

Assigns `parse_document(self.source_path)` to `doc`.

14. **Line 14:** `segments = split_document(doc, mode)`

Assigns `split_document(doc, mode)` to `segments` .

15. **Line 15:** `# Thread-safe lock management`

A comment: *Thread-safe lock management.*

16. **Line 16:** `class CloneClient:`

Defines class `CloneClient` .

17. **Line 17:** `def _request(self, req: dict) -> dict:`

Defines function `_request` accepting (self, req), returning `dict` .

18. **Line 18:** `with self._lock: # Thread-safe access`

Context manager: `self._lock` . Automatically manages resource cleanup.

19. **Line 19:** `# ... send request and read response`

A comment: *... send request and read response.*

20. **Line 20:** `# Temporary directory for downloads`

A comment: *Temporary directory for downloads.*

21. **Line 21:** `import tempfile`

Imports the `tempfile` module.

22. **Line 22:** `with tempfile.TemporaryDirectory() as tmpdir:`

Context manager: `tempfile.TemporaryDirectory() as tmpdir` . Automatically manages resource cleanup.

23. **Line 23:** `# Download to tmpdir, process, then it's auto-cleaned`

A comment: *Download to tmpdir, process, then it's auto-cleaned.*

24. **Line 24:** `download_to(tmpdir)`

Calls function `download_to()` .

25. **Line 25:** `# contextlib.suppress – ignore specific exceptions`

A comment: *contextlib.suppress – ignore specific exceptions.*

26. **Line 26:** `from contextlib import contextlib`

Imports `contextlib` from `contextlib` .

27. **Line 27:** `with contextlib.suppress(FileNotFoundError):`

Context manager that suppresses specified exceptions.

28. **Line 28:** `os.unlink(temp_file) # Ignore if file doesn't exist`

```
os.unlink(temp_file) # Ignore if file doesn't exist
```

14.4 Key Takeaways

- Context managers ensure **cleanup always happens**, even on exceptions
 - `@contextmanager` is simpler than implementing `__enter__` / `__exit__`
 - Use context managers for **files, locks, subprocesses, GUI state**
 - The AIVS project uses context managers for **thread locks, temp directories**, and **GUI busy cursor**
-

Regular Expressions

Learning Objectives

- Understand regex syntax and patterns
 - Use Python's `re` module for search, match, and substitution
 - Apply regex for text processing in AIVS
-

15.1 Regex Basics

```
import re

# Simple patterns
text = "Voice af_bella has sample rate 24000"

# re.search - find first match anywhere in string
match = re.search(r'\d+', text)
print(match.group()) # "24000"

# re.findall - find ALL matches as strings
numbers = re.findall(r'\d+', text)
print(numbers) # ["24000"]

# re.match - match only at the START of string
match = re.match(r'Voice (\w+)', text)
print(match.group(1)) # "af_bella" (captured group)

# re.sub - substitute matches
clean = re.sub(r'\s+', ' ', "too many spaces")
print(clean) # "too many spaces"
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import re`
Imports the `re` module.
2. **Line 2:** `# Simple patterns`
A comment: *Simple patterns*.
3. **Line 3:** `text = "Voice af_bella has sample rate 24000"`
Assigns `"Voice af_bella has sample rate 24000"` to `text`.
4. **Line 4:** `# re.search – find first match anywhere in string`
A comment: *re.search – find first match anywhere in string*.
5. **Line 5:** `match = re.search(r'\d+', text)`
Searches anywhere in the string for a match.
6. **Line 6:** `print(match.group()) # "24000"`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# re.findall – find ALL matches as strings`
A comment: *re.findall – find ALL matches as strings*.
8. **Line 8:** `numbers = re.findall(r'\d+', text)`
Returns all non-overlapping matches as a list of strings.
9. **Line 9:** `print(numbers) # ["24000"]`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# re.match – match only at the START of string`
A comment: *re.match – match only at the START of string*.
11. **Line 11:** `match = re.match(r'Voice (\w+)', text)`
Checks for a match only at the beginning of the string.
12. **Line 12:** `print(match.group(1)) # "af_bella" (captured group)`
Outputs text to the console. The primary way to display information during development and debugging.
13. **Line 13:** `# re.sub – substitute matches`
A comment: *re.sub – substitute matches*.
14. **Line 14:** `clean = re.sub(r'\s+', ' ', "too many spaces")`
Replaces all matches with the replacement string.

15. **Line 15:** `print(clean) # "too many spaces"`

Outputs text to the console. The primary way to display information during development and debugging.

15.2 Pattern Syntax

Pattern	Meaning	Example
<code>.</code>	Any character	<code>r'h.t'</code> → "hat", "hot"
<code>\d</code>	Digit [0-9]	<code>r'\d\d'</code> → "42"
<code>\w</code>	Word char [a-zA-Z0-9_]	<code>r'\w+'</code> → "hello"
<code>\s</code>	Whitespace	<code>r'\s+'</code> → " "
<code>*</code>	Zero or more	<code>r'ab*c'</code> → "ac", "abc"
<code>+</code>	One or more	<code>r'ab+c'</code> → "abc"
<code>?</code>	Zero or one	<code>r'colou?r'</code> → "color"
<code>{n,m}</code>	n to m repetitions	<code>r'\d{2,4}'</code> → "12", "1234"
<code>[abc]</code>	Character set	<code>r'[aeiou]'</code> → vowels
<code>[^abc]</code>	Negated set	<code>r'[^0-9]'</code> → non-digits
<code> </code>	Alternation (or)	<code>r'cat dog'</code>
<code>()</code>	Capture group	<code>r'(\w+)@(\w+)'</code>
<code>(?:)</code>	Non-capturing group	<code>r'(? :http https)'</code>
<code>\b</code>	Word boundary	<code>r'\bcat\b'</code>
<code>^</code>	Start of string	<code>r'^Hello'</code>
<code>\$</code>	End of string	<code>r'\.txt\$'</code>

15.3 Common Patterns

```
# Validate file extension
def is_valid_docx(path: str) -> bool:
    return bool(re.match(r'\.docx$', path, re.IGNORECASE))

# Extract version numbers
text = "Version 2.0.1 (build 20260822)"
version = re.search(r'Version (\d+\.\d+\.\d+)', text)
print(version.group(1)) # "2.0.1"

# Split on multiple delimiters
parts = re.split(r'[;,\s]+', "Piper, Kokoro ; Matcha")
print(parts) # ['Piper', 'Kokoro', 'Matcha']

# Clean up whitespace in text
text = "Too\n\n\nmany\n\n\nnewlines"
clean = re.sub(r'\n{3,}', '\n\n', text)

# Named groups
pattern = r'(?P<tts>\w+)/(?P<lang>\w+)/(?P<variant>\w+)'
match = re.match(pattern, "piper/en_US/medium")
if match:
    print(match.group("tts"))      # "piper"
    print(match.group("lang"))     # "en_US"
    print(match.group("variant"))  # "medium"
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Validate file extension`
A comment: *Validate file extension.*
2. **Line 2:** `def is_valid_docx(path: str) -> bool:`
Defines function `is_valid_docx` accepting (path), returning `bool`.
3. **Line 3:** `return bool(re.match(r'\.docx$', path, re.IGNORECASE))`
Checks for a match only at the beginning of the string.
4. **Line 4:** `# Extract version numbers`
A comment: *Extract version numbers.*
5. **Line 5:** `text = "Version 2.0.1 (build 20260822)"`
Assigns `"Version 2.0.1 (build 20260822)"` to `text`.
6. **Line 6:** `version = re.search(r'Version (\d+\.\d+\.\d+)', text)`
Searches anywhere in the string for a match.
7. **Line 7:** `print(version.group(1)) # "2.0.1"`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `# Split on multiple delimiters`
A comment: *Split on multiple delimiters.*
9. **Line 9:** `parts = re.split(r'[;,\s]+', "Piper, Kokoro ; Matcha")`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
10. **Line 10:** `print(parts) # ['Piper', 'Kokoro', 'Matcha']`
Outputs text to the console. The primary way to display information during development and debugging.
11. **Line 11:** `# Clean up whitespace in text`
A comment: *Clean up whitespace in text.*
12. **Line 12:** `text = "Too\n\nmany\n\nnewlines"`
Assigns `"Too\n\nmany\n\nnewlines"` to `text`.
13. **Line 13:** `clean = re.sub(r'\n{3,}', '\n\n', text)`
Replaces all matches with the replacement string.
14. **Line 14:** `# Named groups`

A comment: *Named groups*.

15. **Line 15:** `pattern = r'(?P<tts>\w+)/(?P<lang>\w+)/(?P<variant>\w+)'`

Assigns `r'(?P<tts>\w+)/(?P<lang>\w+)/(?P<variant>\w+)'` to `pattern`.

16. **Line 16:** `match = re.match(pattern, "piper/en_US/medium")`

Checks for a match only at the beginning of the string.

17. **Line 17:** `if match:`

Checks condition: `match`. The indented block runs only when this is `True`.

18. **Line 18:** `print(match.group("tts")) # "piper"`

Outputs text to the console. The primary way to display information during development and debugging.

19. **Line 19:** `print(match.group("lang")) # "en_US"`

Outputs text to the console. The primary way to display information during development and debugging.

20. **Line 20:** `print(match.group("variant")) # "medium"`

Outputs text to the console. The primary way to display information during development and debugging.

15.4 Regex in AIVS

```
# From util.py - filename sanitization
import re
def sanitize_filename(name: str, max_length: int = 80) -> str:
    name = re.sub(r'<>:"/\||?*]', '_', name)
    name = re.sub(r'\s+', ' ', name).strip()
    if len(name) > max_length:
        name = name[:max_length].rstrip()
    return name

# From documents/daisy_builder.py - DAISY ID sanitization
def _sanitize_id(text: str) -> str:
    s = re.sub(r"^[a-zA-Z0-9_-]", "_", text.lower())
    s = re.sub(r"_+", "_", s).strip("_")
    return s[:60] or "item"

# From documents/splitter.py - sentence splitting
def _split_sentences(text: str) -> list[str]:
    sentences = re.split(r'(?<=[!?])\s+', text)
    return [s.strip() for s in sentences if s.strip()]

# Compiled patterns (faster for repeated use)
_FILE_EXT = re.compile(r'(\.w+)$')
def get_extension(path: str) -> str:
    m = _FILE_EXT.search(path)
    return m.group(1) if m else ""
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From util.py - filename sanitization`
A comment: *From util.py - filename sanitization.*
2. **Line 2:** `import re`
Imports the `re` module.
3. **Line 3:** `def sanitize_filename(name: str, max_length: int = 80) -> str:`
Defines function `sanitize_filename` accepting (name, max_length), returning `str`.
4. **Line 4:** `name = re.sub(r'[\<>:"/\|]?*', '_', name)`
Replaces all matches with the replacement string.
5. **Line 5:** `name = re.sub(r'\s+', ' ', name).strip()`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
6. **Line 6:** `if len(name) > max_length:`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
7. **Line 7:** `name = name[:max_length].rstrip()`
Removes trailing whitespace only.
8. **Line 8:** `return name`
Returns `name` to the caller.
9. **Line 9:** `# From documents/daisy_builder.py - DAISY ID sanitization`
A comment: *From documents/daisy_builder.py - DAISY ID sanitization.*
10. **Line 10:** `def _sanitize_id(text: str) -> str:`
Returns the unique identity of an object (memory address in CPython). Used with 'is' for identity checks.
11. **Line 11:** `s = re.sub(r"^[a-zA-Z0-9_-]", "_", text.lower())`
Converts all characters to lowercase. Returns a new string.
12. **Line 12:** `s = re.sub(r"_+", "_", s).strip("_")`
Removes leading and trailing whitespace (or specified characters). Returns a new string.
13. **Line 13:** `return s[:60] or "item"`
Returns `s[:60]` or `"item"` to the caller.
14. **Line 14:** `# From documents/splitter.py - sentence splitting`

A comment: *From documents/splitter.py — sentence splitting.*

15. **Line 15:** `def _split_sentences(text: str) -> list[str]:`

Defines function `_split_sentences` accepting (text), returning `list[str]` .

16. **Line 16:** `sentences = re.split(r'(?<=[.!?])\s+', text)`

Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.

17. **Line 17:** `return [s.strip() for s in sentences if s.strip()]`

Removes leading and trailing whitespace (or specified characters). Returns a new string.

18. **Line 18:** `# Compiled patterns (faster for repeated use)`

A comment: *Compiled patterns (faster for repeated use).*

19. **Line 19:** `_FILE_EXT = re.compile(r'(\.w+)$')`

Assigns `re.compile(r'(\.w+)$')` to `_FILE_EXT` .

20. **Line 20:** `def get_extension(path: str) -> str:`

Defines function `get_extension` accepting (path), returning `str` .

21. **Line 21:** `m = _FILE_EXT.search(path)`

Assigns `_FILE_EXT.search(path)` to `m` .

22. **Line 22:** `return m.group(1) if m else ""`

Returns `m.group(1) if m else ""` to the caller.

15.5 Key Takeaways

- `re.search()` finds anywhere; `re.match()` finds at start; `re.findall()` finds all
 - **Raw strings** (`r'...'`) are essential for regex patterns
 - **Compiled patterns** (`re.compile()`) are faster for repeated use
 - AIVS uses regex for **filename sanitization**, **text splitting**, and **pattern matching**
-

Date & Time

Learning Objectives

- Use the `datetime` module for dates, times, and timezones
 - Format and parse dates with `strftime` and `strptime`
 - Use `time.monotonic()` for performance timing
 - See time handling in AIVS (DAISY timestamps, logging)
-

16.1 datetime Module

```
from datetime import datetime, date, time, timedelta, timezone

# Current date and time
now = datetime.now()
print(now)           # 2026-08-29 14:30:00.123456
print(now.date())    # 2026-08-29
print(now.time())    # 14:30:00.123456

# Create specific dates
birthday = date(2026, 8, 29)
release = datetime(2026, 12, 1, 10, 30, 0)

# UTC time
utc_now = datetime.now(timezone.utc)

# Time deltas
tomorrow = now + timedelta(days=1)
next_week = now + timedelta(weeks=1)
one_hour_later = now + timedelta(hours=1)
diff = release - now # timedelta object
print(diff.days)    # Days until release
```


Explain Line by Line Code Description (Block 1)

1. **Line 1:** `from datetime import datetime, date, time, timedelta, timezone`
Imports `datetime, date, time, timedelta, timezone` from `datetime`.
2. **Line 2:** `# Current date and time`
A comment: *Current date and time.*
3. **Line 3:** `now = datetime.now()`
Assigns `datetime.now()` to `now`.
4. **Line 4:** `print(now) # 2026-08-29 14:30:00.123456`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `print(now.date()) # 2026-08-29`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `print(now.time()) # 14:30:00.123456`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# Create specific dates`
A comment: *Create specific dates.*
8. **Line 8:** `birthday = date(2026, 8, 29)`
Assigns `date(2026, 8, 29)` to `birthday`.
9. **Line 9:** `release = datetime(2026, 12, 1, 10, 30, 0)`
Assigns `datetime(2026, 12, 1, 10, 30, 0)` to `release`.
10. **Line 10:** `# UTC time`
A comment: *UTC time.*
11. **Line 11:** `utc_now = datetime.now(timezone.utc)`
Assigns `datetime.now(timezone.utc)` to `utc_now`.
12. **Line 12:** `# Time deltas`
A comment: *Time deltas.*
13. **Line 13:** `tomorrow = now + timedelta(days=1)`
Assigns `now + timedelta(days=1)` to `tomorrow`.
14. **Line 14:** `next_week = now + timedelta(weeks=1)`

Assigns `now + timedelta(weeks=1)` to `next_week` .

15. **Line 15:** `one_hour_later = now + timedelta(hours=1)`

Assigns `now + timedelta(hours=1)` to `one_hour_later` .

16. **Line 16:** `diff = release - now # timedelta object`

Assigns `release - now # timedelta object` to `diff` .

17. **Line 17:** `print(diff.days) # Days until release`

Outputs text to the console. The primary way to display information during development and debugging.

16.2 Formatting and Parsing

```
# strftime - format datetime as string
now = datetime(2026, 8, 29, 14, 30)
print(now.strftime("%Y-%m-%d %H:%M:%S")) # "2026-08-29 14:30:00"
print(now.strftime("%B %d, %Y"))        # "August 29, 2026"
print(now.strftime("%c"))                # Locale-specific

# strptime - parse string into datetime
dt = datetime.strptime("2026-08-29", "%Y-%m-%d")

# ISO 8601 format (used in AIVS DAISY builder)
iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
print(iso) # "2026-08-29T19:30:00Z"
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# strftime — format datetime as string`

A comment: *strftime* — *format datetime as string*.

2. **Line 2:** `now = datetime(2026, 8, 29, 14, 30)`

Assigns `datetime(2026, 8, 29, 14, 30)` to `now` .

3. **Line 3:** `print(now.strftime("%Y-%m-%d %H:%M:%S")) # "2026-08-29 14:30:00"`

Outputs text to the console. The primary way to display information during development and debugging.

4. **Line 4:** `print(now.strftime("%B %d, %Y")) # "August 29, 2026"`

Outputs text to the console. The primary way to display information during development and debugging.

5. **Line 5:** `print(now.strftime("%c")) # Locale-specific`

Outputs text to the console. The primary way to display information during development and debugging.

6. **Line 6:** `# strptime — parse string into datetime`

A comment: *strptime* — *parse string into datetime*.

7. **Line 7:** `dt = datetime.strptime("2026-08-29", "%Y-%m-%d")`

Assigns `datetime.strptime("2026-08-29", "%Y-%m-%d")` to `dt` .

8. **Line 8:** `# ISO 8601 format (used in AIVS DAISY builder)`

A comment: *ISO 8601 format (used in AIVS DAISY builder)*.

9. **Line 9:** `iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")`

Assigns `datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")` to `iso` .

10. **Line 10:** `print(iso) # "2026-08-29T19:30:00Z"`

Outputs text to the console. The primary way to display information during development and debugging.

Format Codes Reference

Code	Meaning	Example
%Y	4-digit year	2026
%m	Month (01-12)	08
%d	Day (01-31)	29
%H	Hour 24h (00-23)	14
%M	Minute (00-59)	30
%S	Second (00-59)	00
%B	Full month name	August
%A	Full weekday name	Saturday
%Z	Timezone name	UTC

16.3 Time Performance Measurement

```
import time

# time.monotonic() - best for measuring elapsed time
start = time.monotonic()
# ... do work ...
elapsed = time.monotonic() - start
print(f"Elapsed: {elapsed:.3f}s")

# Used in AIVS server health check polling
def _wait_ready(self, timeout: float):
    """Poll health endpoint until server responds."""
    deadline = time.monotonic() + timeout
    while time.monotonic() < deadline:
        if self.health_check():
            return
        time.sleep(1.0)
    raise TimeoutError(f"Server not ready in {timeout}s")

# time.perf_counter() - highest resolution timer
start = time.perf_counter()
result = expensive_computation()
duration = time.perf_counter() - start
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `import time`
Imports the `time` module.
2. **Line 2:** `# time.monotonic() – best for measuring elapsed time`
A comment: *time.monotonic() – best for measuring elapsed time.*
3. **Line 3:** `start = time.monotonic()`
Assigns `time.monotonic()` to `start`.
4. **Line 4:** `# ... do work ...`
A comment: *... do work*
5. **Line 5:** `elapsed = time.monotonic() - start`
Assigns `time.monotonic() - start` to `elapsed`.
6. **Line 6:** `print(f"Elapsed: {elapsed:.3f}s")`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# Used in AIVS server health check polling`
A comment: *Used in AIVS server health check polling.*
8. **Line 8:** `def _wait_ready(self, timeout: float):`
Defines function `_wait_ready` accepting (self, timeout).
9. **Line 9:** `"""Poll health endpoint until server responds."""`
Docstring: *Poll health endpoint until server responds..* Documents the purpose of the following code.
10. **Line 10:** `deadline = time.monotonic() + timeout`
Assigns `time.monotonic() + timeout` to `deadline`.
11. **Line 11:** `while time.monotonic() < deadline:`
While loop: repeats as long as `time.monotonic() < deadline`.
12. **Line 12:** `if self.health_check():`
Checks condition: `self.health_check()`. The indented block runs only when this is `True`.
13. **Line 13:** `return`
Returns `None` implicitly (exits the function).

14. **Line 14:** `time.sleep(1.0)`
`time.sleep(1.0)`
15. **Line 15:** `raise TimeoutError(f"Server not ready in {timeout}s")`
Manually raises an exception. Use to signal error conditions to callers.
16. **Line 16:** `# time.perf_counter() – highest resolution timer`
A comment: *time.perf_counter() – highest resolution timer.*
17. **Line 17:** `start = time.perf_counter()`
Assigns `time.perf_counter()` to `start`.
18. **Line 18:** `result = expensive_computation()`
Assigns `expensive_computation()` to `result`.
19. **Line 19:** `duration = time.perf_counter() - start`
Assigns `time.perf_counter() - start` to `duration`.

16.4 Dates in AIVS

```
# From documents/daisy_builder.py – DAISY book metadata
from datetime import datetime, timezone

now_iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
date_only = datetime.now(timezone.utc).strftime("%Y-%m-%d")

# Used in NCC.html and package.opf metadata:
# <meta name="dc:date" content="2026-08-29" />
# <dc:date>2026-08-29T19:30:00Z</dc:date>
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# From documents/daisy_builder.py – DAISY book metadata`
A comment: *From documents/daisy_builder.py – DAISY book metadata.*
2. **Line 2:** `from datetime import datetime, timezone`
Imports `datetime, timezone` from `datetime` .
3. **Line 3:** `now_iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")`
Assigns `datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")` to `now_iso` .
4. **Line 4:** `date_only = datetime.now(timezone.utc).strftime("%Y-%m-%d")`
Assigns `datetime.now(timezone.utc).strftime("%Y-%m-%d")` to `date_only` .
5. **Line 5:** `# Used in NCC.html and package.opf metadata:`
A comment: *Used in NCC.html and package.opf metadata:.*
6. **Line 6:** `# <meta name="dc:date" content="2026-08-29" />`
A comment: *<meta name="dc:date" content="2026-08-29" />.*
7. **Line 7:** `# <dc:date>2026-08-29T19:30:00Z</dc:date>`
A comment: *<dc:date>2026-08-29T19:30:00Z</dc:date>.*

16.5 Key Takeaways

- `datetime.now()` for current time; `timezone.utc` for UTC
 - `strftime()` formats; `strptime()` parses
 - `time.monotonic()` for measuring elapsed time (not affected by clock changes)
 - AIVS uses ISO 8601 timestamps for DAISY book metadata
-

Testing with unittest & pytest

Learning Objectives

- Write unit tests with `unittest` and `pytest`
 - Use assertions effectively
 - Understand test organization and naming conventions
 - See the AIVS test suite structure
-

17.1 Why Test?

AI Voice Studio has **99+ tests** across 12 test files. Testing ensures:

- **Correctness** — code does what it's supposed to
 - **Regression prevention** — changes don't break existing functionality
 - **Documentation** — tests show how code is intended to be used
 - **Confidence** — refactoring safely
-

17.2 unittest — The Standard Library

```
import unittest
from ai_voice_studio.util import sanitize_filename

class SanitizeFilenameTest(unittest.TestCase):
    """Tests for the sanitize_filename utility function."""

    def test_removes_illegal_characters(self):
        """Characters illegal on Windows should be replaced."""
        result = sanitize_filename('File: "Name" <bad>?')
        assert ":" not in result
        assert '"' not in result
        assert "<" not in result

    def test_truncates_long_names(self):
        """Names exceeding max_length should be truncated."""
        long_name = "A" * 200
        result = sanitize_filename(long_name, max_length=80)
        assert len(result) <= 80

    def test_strips_whitespace(self):
        """Leading/trailing whitespace should be removed."""
        result = sanitize_filename(" hello ")
        assert result == "hello"

    def test_empty_name_returns_fallback(self):
        """Empty names should return a valid filename."""
        result = sanitize_filename("")
        assert result # Non-empty

# Run with: python -m unittest tests/test_util.py
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import unittest`
Imports the `unittest` module.
2. **Line 2:** `from ai_voice_studio.util import sanitize_filename`
Imports `sanitize_filename` from `ai_voice_studio.util`.
3. **Line 3:** `class SanitizeFilenameTest(unittest.TestCase):`
Defines class `SanitizeFilenameTest` inheriting from `unittest.TestCase`.
4. **Line 4:** `"""Tests for the sanitize_filename utility function."""`
Docstring: *Tests for the sanitize_filename utility function..* Documents the purpose of the following code.
5. **Line 5:** `def test_removes_illegal_characters(self):`
Defines function `test_removes_illegal_characters` accepting (self).
6. **Line 6:** `"""Characters illegal on Windows should be replaced."""`
Docstring: *Characters illegal on Windows should be replaced..* Documents the purpose of the following code.
7. **Line 7:** `result = sanitize_filename('File: "Name" <bad>?')`
Assigns `sanitize_filename('File: "Name" <bad>?')` to `result`.
8. **Line 8:** `assert ":" not in result`
`assert ":" not in result`
9. **Line 9:** `assert "'" not in result`
`assert "'" not in result`
10. **Line 10:** `assert "<" not in result`
`assert "<" not in result`
11. **Line 11:** `def test_truncates_long_names(self):`
Defines function `test_truncates_long_names` accepting (self).
12. **Line 12:** `"""Names exceeding max_length should be truncated."""`
Docstring: *Names exceeding max_length should be truncated..* Documents the purpose of the following code.
13. **Line 13:** `long_name = "A" * 200`
Assigns `"A" * 200` to `long_name`.

14. **Line 14:** `result = sanitize_filename(long_name, max_length=80)`
Assigns `sanitize_filename(long_name, max_length=80)` to `result`.
15. **Line 15:** `assert len(result) <= 80`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
16. **Line 16:** `def test_strips_whitespace(self):`
Defines function `test_strips_whitespace` accepting (self).
17. **Line 17:** `"""Leading/trailing whitespace should be removed."""`
Docstring: *Leading/trailing whitespace should be removed..* Documents the purpose of the following code.
18. **Line 18:** `result = sanitize_filename(" hello ")`
Assigns `sanitize_filename(" hello ")` to `result`.
19. **Line 19:** `assert result == "hello"`
`assert result == "hello"`
20. **Line 20:** `def test_empty_name_returns_fallback(self):`
Defines function `test_empty_name_returns_fallback` accepting (self).
21. **Line 21:** `"""Empty names should return a valid filename."""`
Docstring: *Empty names should return a valid filename..* Documents the purpose of the following code.
22. **Line 22:** `result = sanitize_filename("")`
Assigns `sanitize_filename("")` to `result`.
23. **Line 23:** `assert result # Non-empty`
`assert result # Non-empty`
24. **Line 24:** `# Run with: python -m unittest tests/test_util.py`
A comment: *Run with: python -m unittest tests/test_util.py.*
-

17.3 pytest — The Preferred Framework

```
# pytest tests are simpler — just use assert statements
from ai_voice_studio.tts import catalog

def test_tts_ids_are_unique():
    """Every TTS engine must have a unique ID."""
    ids = [t["id"] for t in catalog.get_tts_list()]
    assert len(ids) == len(set(ids))

def test_required_engines_present():
    """Piper, Kokoro, and Matcha must be in the catalog."""
    ids = {t["id"] for t in catalog.get_tts_list()}
    assert "piper" in ids
    assert "kokoro" in ids
    assert "matcha" in ids

def test_piper_voices_have_artifacts():
    """Every Piper voice must have a download artifact."""
    piper = catalog.find_tts("piper")
    assert piper is not None
    for lang in piper["languages"]:
        for variant in lang["variants"]:
            assert variant.get("artifact") or variant.get("voices")

# Run with: python -m pytest tests/ -v
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# pytest tests are simpler – just use assert statements`
A comment: *pytest tests are simpler – just use assert statements.*
2. **Line 2:** `from ai_voice_studio.tts import catalog`
Imports `catalog` from `ai_voice_studio.tts`.
3. **Line 3:** `def test_tts_ids_are_unique():`
Defines function `test_tts_ids_are_unique`.
4. **Line 4:** `"""Every TTS engine must have a unique ID."""`
Docstring: *Every TTS engine must have a unique ID.*. Documents the purpose of the following code.
5. **Line 5:** `ids = [t["id"] for t in catalog.get_tts_list()]`
Assigns `[t["id"] for t in catalog.get_tts_list()]` to `ids`.
6. **Line 6:** `assert len(ids) == len(set(ids))`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.
7. **Line 7:** `def test_required_engines_present():`
Defines function `test_required_engines_present`.
8. **Line 8:** `"""Piper, Kokoro, and Matcha must be in the catalog."""`
Docstring: *Piper, Kokoro, and Matcha must be in the catalog.*. Documents the purpose of the following code.
9. **Line 9:** `ids = {t["id"] for t in catalog.get_tts_list()}`
Assigns `{t["id"] for t in catalog.get_tts_list()}` to `ids`.
10. **Line 10:** `assert "piper" in ids`
`assert "piper" in ids`
11. **Line 11:** `assert "kokoro" in ids`
`assert "kokoro" in ids`
12. **Line 12:** `assert "matcha" in ids`
`assert "matcha" in ids`
13. **Line 13:** `def test_piper_voices_have_artifacts():`
Defines function `test_piper_voices_have_artifacts`.

14. **Line 14:** `"""Every Piper voice must have a download artifact."""`
Docstring: *Every Piper voice must have a download artifact..* Documents the purpose of the following code.
15. **Line 15:** `piper = catalog.find_tts("piper")`
Assigns `catalog.find_tts("piper")` to `piper` .
16. **Line 16:** `assert piper is not None`
`assert piper is not None`
17. **Line 17:** `for lang in piper["languages"]:`
Iterates over `piper["languages"]` , assigning each element to `lang` .
18. **Line 18:** `for variant in lang["variants"]:`
Iterates over `lang["variants"]` , assigning each element to `variant` .
19. **Line 19:** `assert variant.get("artifact") or variant.get("voices")`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
20. **Line 20:** `# Run with: python -m pytest tests/ -v`
A comment: *Run with: python -m pytest tests/ -v.*
-

17.4 Test Fixtures and Setup

```
import unittest
import tempfile
from unittest import mock

class ResolveVoiceFilesTest(unittest.TestCase):
    def setUp(self):
        """Called before each test method."""
        self.tmp = tempfile.TemporaryDirectory()
        self.models_root = os.path.join(self.tmp.name, "models")
        # Mock the paths module to use our temp directory
        self.patcher = mock.patch(
            "ai_voice_studio.paths.user_data_dir",
            return_value=self.tmp.name,
        )
        self.patcher.start()

    def tearDown(self):
        """Called after each test method."""
        self.patcher.stop()
        self.tmp.cleanup()

    def _voice_dir(self, name: str) -> str:
        """Create a voice directory in the temp area."""
        d = os.path.join(self.models_root, name)
        os.makedirs(d, exist_ok=True)
        return d

    def test_matcha_finds_acoustic_model(self):
        vdir = self._voice_dir("matcha_en/default")
        open(os.path.join(vdir, "model-steps-3.onnx"), "w").close()
        open(os.path.join(vdir, "tokens.txt"), "w").close()
        # ... test resolve_voice_files finds the right files
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `import unittest`
Imports the `unittest` module.
2. **Line 2:** `import tempfile`
Imports the `tempfile` module.
3. **Line 3:** `from unittest import mock`
Imports `mock` from `unittest`.
4. **Line 4:** `class ResolveVoiceFilesTest(unittest.TestCase):`
Defines class `ResolveVoiceFilesTest` inheriting from `unittest.TestCase`.
5. **Line 5:** `def setUp(self):`
Defines function `setUp` accepting (self).
6. **Line 6:** `"""Called before each test method."""`
Docstring: *Called before each test method..* Documents the purpose of the following code.
7. **Line 7:** `self.tmp = tempfile.TemporaryDirectory()`
Stores `tempfile.TemporaryDirectory()` in instance attribute `tmp`.
8. **Line 8:** `self.models_root = os.path.join(self.tmp.name, "models")`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
9. **Line 9:** `# Mock the paths module to use our temp directory`
A comment: *Mock the paths module to use our temp directory.*
10. **Line 10:** `self.patcher = mock.patch(`
Stores `mock.patch(` in instance attribute `patcher`.
11. **Line 11:** `"ai_voice_studio.paths.user_data_dir",`
`"ai_voice_studio.paths.user_data_dir",`
12. **Line 12:** `return_value=self.tmp.name,`
Assigns `self.tmp.name,` to `return_value`.
13. **Line 13:** `)`
`)`

14. **Line 14:** `self.patcher.start()`
`self.patcher.start()`
15. **Line 15:** `def tearDown(self):`
Defines function `tearDown` accepting (self).
16. **Line 16:** `"""Called after each test method."""`
Docstring: *Called after each test method.*. Documents the purpose of the following code.
17. **Line 17:** `self.patcher.stop()`
`self.patcher.stop()`
18. **Line 18:** `self.tmp.cleanup()`
`self.tmp.cleanup()`
19. **Line 19:** `def _voice_dir(self, name: str) -> str:`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
20. **Line 20:** `"""Create a voice directory in the temp area."""`
Docstring: *Create a voice directory in the temp area.*. Documents the purpose of the following code.
21. **Line 21:** `d = os.path.join(self.models_root, name)`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
22. **Line 22:** `os.makedirs(d, exist_ok=True)`
Recursively creates directories. `exist_ok=True` prevents errors if directory exists.
23. **Line 23:** `return d`
Returns `d` to the caller.
24. **Line 24:** `def test_matcha_finds_acoustic_model(self):`
Defines function `test_matcha_finds_acoustic_model` accepting (self).
25. **Line 25:** `vdir = self._voice_dir("matcha_en/default")`
Returns a list of attributes and methods of an object. Useful for exploration and debugging.
26. **Line 26:** `open(os.path.join(vdir, "model-steps-3.onnx"), "w").close()`
Closes the file and flushes the write buffer. Prefer using 'with' instead.
27. **Line 27:** `open(os.path.join(vdir, "tokens.txt"), "w").close()`
Closes the file and flushes the write buffer. Prefer using 'with' instead.

28. **Line 28:** `# ... test resolve_voice_files finds the right files`

A comment: ... *test resolve_voice_files finds the right files.*

17.5 AIVS Test Suite Structure

Test File	Tests	What It Tests
test_catalog.py	14	TTS catalog structure, voice entries, artifacts
test_compute.py	6	GPU/CPU detection logic
test_engine.py	8	TTS engine wrapper, pitch/volume
test_parsers.py	12	PDF, DOCX, HTML, Markdown parsing
test_project.py	10	Project save/load, segment management
test_splitter.py	15	Text splitting algorithms
test_settings.py	8	Settings persistence, dot-notation
test_paths.py	7	Path resolution
test_gui_smoke.py	6	GUI creation smoke tests
test_runtime.py	5	Python runtime management
test_clone.py	4	Voice cloning engine
test_daisy.py	4	DAISY book generation

```
# Running the test suite
# Install pytest first
pip install pytest

# Run all tests with verbose output
python -m pytest tests/ -v

# Run a specific test file
python -m pytest tests/test_catalog.py -v

# Run a specific test
python -m pytest tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique -v

# Run tests matching a pattern
python -m pytest tests/ -k "piper" -v
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# Running the test suite`
A comment: *Running the test suite.*
2. **Line 2:** `# Install pytest first`
A comment: *Install pytest first.*
3. **Line 3:** `pip install pytest`
`pip install pytest`
4. **Line 4:** `# Run all tests with verbose output`
A comment: *Run all tests with verbose output.*
5. **Line 5:** `python -m pytest tests/ -v`
`python -m pytest tests/ -v`
6. **Line 6:** `# Run a specific test file`
A comment: *Run a specific test file.*
7. **Line 7:** `python -m pytest tests/test_catalog.py -v`
`python -m pytest tests/test_catalog.py -v`
8. **Line 8:** `# Run a specific test`
A comment: *Run a specific test.*
9. **Line 9:** `python -m pytest tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique -v`
`python -m pytest tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique -v`
10. **Line 10:** `# Run tests matching a pattern`
A comment: *Run tests matching a pattern.*
11. **Line 11:** `python -m pytest tests/ -k "piper" -v`
`python -m pytest tests/ -k "piper" -v`

17.6 Key Takeaways

- Test names should be **descriptive**: `test_piper_voices_have_artifacts`

- **setUp/tearDown** create and clean up test fixtures
- Use **tempfile.TemporaryDirectory** for file system tests
- The AIVS test suite has **99+ tests** across 12 files
- **pytest** is simpler than unittest but unittest is in the stdlib

 **Exercise: Exercise 17.1:** Write 5 tests for the `sanitize_filename` function covering: special characters, long names, empty input, whitespace, and unicode characters.

JSON, XML & Structured Data

Learning Objectives

- Master JSON serialization and deserialization
 - Understand XML basics for DAISY book generation
 - See structured data patterns throughout AIVS
-

18.1 JSON in Depth

```
import json

# Python ↔ JSON type mapping
# dict    → object
# list    → array
# str     → string
# int/float → number
# True    → true
# False   → false
# None    → null

# Complex serialization
data = {
    "name": "My Project",
    "segments": [
        {"index": 1, "title": "Chapter 1", "text": "..."},
        {"index": 2, "title": "Chapter 2", "text": "..."},
    ],
    "tts": {
        "voice": "af_bella",
        "rate": 1.0,
        "pitch": 1.0,
    },
}

# Serialize with nice formatting
json_str = json.dumps(data, indent=2, ensure_ascii=False)
print(json_str)

# Deserialize
loaded = json.loads(json_str)

# Write to file
with open("project.json", "w", encoding="utf-8") as f:
    json.dump(data, f, indent=2, ensure_ascii=False)

# Read from file
with open("project.json", "r", encoding="utf-8") as f:
    loaded = json.load(f)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import json`
Imports the `json` module.
2. **Line 2:** `# Python ↔ JSON type mapping`
A comment: *Python ↔ JSON type mapping.*
3. **Line 3:** `# dict → object`
A comment: *dict → object.*
4. **Line 4:** `# list → array`
A comment: *list → array.*
5. **Line 5:** `# str → string`
A comment: *str → string.*
6. **Line 6:** `# int/float → number`
A comment: *int/float → number.*
7. **Line 7:** `# True → true`
A comment: *True → true.*
8. **Line 8:** `# False → false`
A comment: *False → false.*
9. **Line 9:** `# None → null`
A comment: *None → null.*
10. **Line 10:** `# Complex serialization`
A comment: *Complex serialization.*
11. **Line 11:** `data = {`
Assigns `{` to `data`.
12. **Line 12:** `"name": "My Project",`
`"name": "My Project",`
13. **Line 13:** `"segments": [`
`"segments": [`
14. **Line 14:** `{"index": 1, "title": "Chapter 1", "text": "..."},`

```
{ "index": 1, "title": "Chapter 1", "text": "..."},

15. Line 15: { "index": 2, "title": "Chapter 2", "text": "..."},
    { "index": 2, "title": "Chapter 2", "text": "..."},

16. Line 16: ],
    ],

17. Line 17: "tts": {
    "tts": {

18. Line 18: "voice": "af_bella",
    "voice": "af_bella",

19. Line 19: "rate": 1.0,
    "rate": 1.0,

20. Line 20: "pitch": 1.0,
    "pitch": 1.0,

21. Line 21: },
    },

22. Line 22: }
    }

23. Line 23: # Serialize with nice formatting
    A comment: Serialize with nice formatting.

24. Line 24: json_str = json.dumps(data, indent=2, ensure_ascii=False)
    Serializes a Python object to a JSON string.

25. Line 25: print(json_str)
    Outputs text to the console. The primary way to display information during development and debugging.

26. Line 26: # Deserialize
    A comment: Deserialize.

27. Line 27: loaded = json.loads(json_str)
    Deserializes a JSON string into a Python object.

28. Line 28: # Write to file
```


A comment: *Write to file.*

29. **Line 29:** `with open("project.json", "w", encoding="utf-8") as f:`

Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.

30. **Line 30:** `json.dump(data, f, indent=2, ensure_ascii=False)`

Serializes a Python object to JSON and writes to a file.

31. **Line 31:** `# Read from file`

A comment: *Read from file.*

32. **Line 32:** `with open("project.json", "r", encoding="utf-8") as f:`

Opens a file using a context manager. The file is automatically closed when the block exits, even if an error occurs.

33. **Line 33:** `loaded = json.load(f)`

Deserializes JSON from a file into a Python object.

18.2 JSON in AIVS

JSON is the backbone of AIVS. Three critical files use it:

1. Settings File

```
# settings.json – user preferences
{
  "theme": "dark",
  "audio_mode": "page_with_h1",
  "recording.rate": 1.0,
  "recording.pitch": 1.0,
  "recording.volume": 1.0,
  "recording.output_format": "wav",
  "recording.punctuation": "default",
  "compute": "auto",
  "developer_mode": false,
  "last_model": {
    "tts": "piper",
    "language": "en_US",
    "voice": "af_bella"
  },
  "recent_projects": [
    ["My Book", "C:\\Users\\...\\My Book"]
  ]
}
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# settings.json – user preferences`

A comment: *settings.json – user preferences*.

2. **Line 2:** `{`

`{`

3. **Line 3:** `"theme": "dark",`

`"theme": "dark",`

4. **Line 4:** `"audio_mode": "page_with_h1",`

`"audio_mode": "page_with_h1",`

5. **Line 5:** `"recording.rate": 1.0,`

`"recording.rate": 1.0,`

6. **Line 6:** `"recording.pitch": 1.0,`

`"recording.pitch": 1.0,`

7. **Line 7:** `"recording.volume": 1.0,`

`"recording.volume": 1.0,`

8. **Line 8:** `"recording.output_format": "wav",`

`"recording.output_format": "wav",`

9. **Line 9:** `"recording.punctuation": "default",`

`"recording.punctuation": "default",`

10. **Line 10:** `"compute": "auto",`

`"compute": "auto",`

11. **Line 11:** `"developer_mode": false,`

`"developer_mode": false,`

12. **Line 12:** `"last_model": {`

`"last_model": {`

13. **Line 13:** `"tts": "piper",`

`"tts": "piper",`

14. Line 14: `"language": "en_US",`
`"language": "en_US",`

15. Line 15: `"voice": "af_bella"`
`"voice": "af_bella"`

16. Line 16: `},`
`},`

17. Line 17: `"recent_projects": [`
`"recent_projects": [`

18. Line 18: `["My Book", "C:\\Users\\...\\My Book"]`
`["My Book", "C:\\Users\\...\\My Book"]`

19. Line 19: `]`
`]`

20. Line 20: `}`
`}`

2. Project File

```
# project.json - per-project state
{
  "name": "My Audiobook",
  "source": "C:\\Documents\\book.pdf",
  "mode": "page_with_h1",
  "project_type": "audio_playlist",
  "segments": [
    {
      "index": 1,
      "title": "Chapter 1: Introduction",
      "text": "Once upon a time...",
      "status": "done",
      "saved": "Chapter_1_Introduction.wav",
      "duration": 120.5
    },
    {
      "index": 2,
      "title": "Chapter 2: The Beginning",
      "text": "It was a dark...",
      "status": "pending"
    }
  ],
  "tts": {
    "tts": "piper",
    "language": "en_US",
    "variant": "medium",
    "voice": "af_bella",
    "rate": 1.0,
    "pitch": 1.0,
    "volume": 1.0,
    "punctuation": "default",
    "output_format": "wav",
    "compute": "cpu"
  }
}
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# project.json – per-project state`

A comment: *project.json – per-project state*.

2. **Line 2:** `{`

`{`

3. **Line 3:** `"name": "My Audiobook",`

`"name": "My Audiobook",`

4. **Line 4:** `"source": "C:\\Documents\\book.pdf",`

`"source": "C:\\Documents\\book.pdf",`

5. **Line 5:** `"mode": "page_with_h1",`

`"mode": "page_with_h1",`

6. **Line 6:** `"project_type": "audio_playlist",`

`"project_type": "audio_playlist",`

7. **Line 7:** `"segments": [`

`"segments": [`

8. **Line 8:** `{`

`{`

9. **Line 9:** `"index": 1,`

`"index": 1,`

10. **Line 10:** `"title": "Chapter 1: Introduction",`

`"title": "Chapter 1: Introduction",`

11. **Line 11:** `"text": "Once upon a time...",`

`"text": "Once upon a time...",`

12. **Line 12:** `"status": "done",`

`"status": "done",`

13. **Line 13:** `"saved": "Chapter_1_Introduction.wav",`

`"saved": "Chapter_1_Introduction.wav",`

```
14. Line 14: "duration": 120.5
    "duration": 120.5

15. Line 15: },
    },

16. Line 16: {
    {

17. Line 17: "index": 2,
    "index": 2,

18. Line 18: "title": "Chapter 2: The Beginning",
    "title": "Chapter 2: The Beginning",

19. Line 19: "text": "It was a dark...",
    "text": "It was a dark...",

20. Line 20: "status": "pending"
    "status": "pending"

21. Line 21: }
    }

22. Line 22: ],
    ],

23. Line 23: "tts": {
    "tts": {

24. Line 24: "tts": "piper",
    "tts": "piper",

25. Line 25: "language": "en_US",
    "language": "en_US",

26. Line 26: "variant": "medium",
    "variant": "medium",

27. Line 27: "voice": "af_bella",
```

```

    "voice": "af_bella",

28. Line 28: "rate": 1.0,
    "rate": 1.0,

29. Line 29: "pitch": 1.0,
    "pitch": 1.0,

30. Line 30: "volume": 1.0,
    "volume": 1.0,

31. Line 31: "punctuation": "default",
    "punctuation": "default",

32. Line 32: "output_format": "wav",
    "output_format": "wav",

33. Line 33: "compute": "cpu"
    "compute": "cpu"

34. Line 34: }

35. Line 35: }

```

3. TTS Worker Protocol

```

# JSON Lines protocol between app and worker subprocesses
# Request (app → worker):
{"cmd": "synthesize", "text": "Hello world", "speed": 1.0}
# Response (worker → app):
{"ok": true, "wav": "base64-encoded-audio..."}

# Error response:
{"ok": false, "error": "Engine not installed"}

```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# JSON lines protocol between app and worker subprocesses`

A comment: *JSON lines protocol between app and worker subprocesses.*

2. **Line 2:** `# Request (app → worker):`

A comment: *Request (app → worker):.*

3. **Line 3:** `{"cmd": "synthesize", "text": "Hello world", "speed": 1.0}`

`{"cmd": "synthesize", "text": "Hello world", "speed": 1.0}`

4. **Line 4:** `# Response (worker → app):`

A comment: *Response (worker → app):.*

5. **Line 5:** `{"ok": true, "wav": "base64-encoded-audio..."}`

`{"ok": true, "wav": "base64-encoded-audio..."}`

6. **Line 6:** `# Error response:`

A comment: *Error response:.*

7. **Line 7:** `{"ok": false, "error": "Engine not installed"}`

`{"ok": false, "error": "Engine not installed"}`

18.3 XML Basics (for DAISY)

```
# AIVS generates XML for DAISY 2.02 books
# NCC.html (Navigation Center Combined)
ncc = f"""<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"
    "http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">
<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">
<head>
    <title>{title}</title>
    <meta name="dc:title" content="{title}" />
    <meta name="dc:identifier" content="{book_uid}" />
</head>
<body>
    <h1>{title}</h1>
    <h2><a href="Chapter_1.smil">Chapter 1</a></h2>
</body>
</html>"""

# package.opf (Open Packaging Format)
opf = """<?xml version="1.0" encoding="UTF-8"?>
<package xmlns="http://www.idpf.org/2007/opf" version="2.0">
    <metadata xmlns:dc="http://purl.org/dc/elements/1.1/">
        <dc:title>{title}</dc:title>
        <dc:identifier>{uid}</dc:identifier>
        <dc:language>{lang}</dc:language>
    </metadata>
</package>"""

# SMIL files (audio-text synchronization)
smil = """<?xml version="1.0" encoding="UTF-8"?>
<smil xmlns="http://www.w3.org/2001/SMIL20/Language">
    <body>
        <seq>
            <par>
                <audio src="audio/Chapter_1.wav" />
            </par>
        </seq>
    </body>
</smil>"""
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# AIVS generates XML for DAISY 2.02 books`
A comment: *AIVS generates XML for DAISY 2.02 books.*
2. **Line 2:** `# NCC.html (Navigation Center Combined)`
A comment: *NCC.html (Navigation Center Combined).*
3. **Line 3:** `ncc = f"""<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"`
Assigns `f"""<!DOCTYPE html PUBLIC "-//W3C//DTD XHTML 1.0 Transitional//EN"` to `ncc`.
4. **Line 4:** `"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">`
`"http://www.w3.org/TR/xhtml1/DTD/xhtml1-transitional.dtd">`
5. **Line 5:** `<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">`
`<html xmlns="http://www.w3.org/1999/xhtml" xml:lang="en" lang="en">`
6. **Line 6:** `<head>`
`<head>`
7. **Line 7:** `<title>{title}</title>`
`<title>{title}</title>`
8. **Line 8:** `<meta name="dc:title" content="{title}" />`
`<meta name="dc:title" content="{title}" />`
9. **Line 9:** `<meta name="dc:identifier" content="{book_uid}" />`
`<meta name="dc:identifier" content="{book_uid}" />`
10. **Line 10:** `</head>`
`</head>`
11. **Line 11:** `<body>`
`<body>`
12. **Line 12:** `<h1>{title}</h1>`
`<h1>{title}</h1>`
13. **Line 13:** `<h2><a href="Chapter_1.smil">Chapter 1</a></h2>`
`<h2><a href="Chapter_1.smil">Chapter 1</a></h2>`

14. **Line 14:** `</body>`
`</body>`
15. **Line 15:** `</html>""`
`</html>""`
16. **Line 16:** `# package.opf (Open Packaging Format)`
A comment: *package.opf (Open Packaging Format)*.
17. **Line 17:** `opf = ""<?xml version="1.0" encoding="UTF-8"?>`
Assigns `""<?xml version="1.0" encoding="UTF-8"?>` to `opf`.
18. **Line 18:** `<package xmlns="http://www.idpf.org/2007/opf" version="2.0">`
`<package xmlns="http://www.idpf.org/2007/opf" version="2.0">`
19. **Line 19:** `<metadata xmlns:dc="http://purl.org/dc/elements/1.1/">`
`<metadata xmlns:dc="http://purl.org/dc/elements/1.1/">`
20. **Line 20:** `<dc:title>{title}</dc:title>`
`<dc:title>{title}</dc:title>`
21. **Line 21:** `<dc:identifier>{uid}</dc:identifier>`
`<dc:identifier>{uid}</dc:identifier>`
22. **Line 22:** `<dc:language>{lang}</dc:language>`
`<dc:language>{lang}</dc:language>`
23. **Line 23:** `</metadata>`
`</metadata>`
24. **Line 24:** `</package>""`
`</package>""`
25. **Line 25:** `# SMIL files (audio-text synchronization)`
A comment: *SMIL files (audio-text synchronization)*.
26. **Line 26:** `smil = ""<?xml version="1.0" encoding="UTF-8"?>`
Assigns `""<?xml version="1.0" encoding="UTF-8"?>` to `smil`.
27. **Line 27:** `<smil xmlns="http://www.w3.org/2001/SMIL20/Language">`

```
<smil xmlns="http://www.w3.org/2001/SMIL20/Language">
```

28. Line 28: `<body>`

```
<body>
```

29. Line 29: `<seq>`

```
<seq>
```

30. Line 30: `<par>`

```
<par>
```

31. Line 31: `<audio src="audio/Chapter_1.wav" />`

```
<audio src="audio/Chapter_1.wav" />
```

32. Line 32: `</par>`

```
</par>
```

33. Line 33: `</seq>`

```
</seq>
```

34. Line 34: `</body>`

```
</body>
```

35. Line 35: `</smil>""`

```
</smil>""
```

18.4 Structured Data Patterns

```
# Pattern: Default dict with deep merge
def deep_merge(base: dict, override: dict) -> dict:
    """Merge two dicts, with override taking precedence."""
    result = dict(base)
    for key, value in override.items():
        if key in result and isinstance(result[key], dict) and isinstance(value, dict):
            result[key] = deep_merge(result[key], value)
        else:
            result[key] = value
    return result

# Pattern: Typed dict for structured data
from typing import TypedDict

class SegmentDict(TypedDict, total=False):
    index: int
    title: str
    text: str
    status: str
    saved: str
    duration: float
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `# Pattern: Default dict with deep merge`
A comment: *Pattern: Default dict with deep merge.*
2. **Line 2:** `def deep_merge(base: dict, override: dict) -> dict:`
Defines function `deep_merge` accepting (base, override), returning `dict`.
3. **Line 3:** `"""Merge two dicts, with override taking precedence."""`
Docstring: *Merge two dicts, with override taking precedence..* Documents the purpose of the following code.
4. **Line 4:** `result = dict(base)`
Assigns `dict(base)` to `result`.
5. **Line 5:** `for key, value in override.items():`
Returns a view of the dictionary's (key, value) pairs.
6. **Line 6:** `if key in result and isinstance(result[key], dict) and isinstance(value, dict):`
Checks if an object is an instance of a class. Preferred over `type()` for type checking.
7. **Line 7:** `result[key] = deep_merge(result[key], value)`
`result[key] = deep_merge(result[key], value)`
8. **Line 8:** `else:`
Fallback: executes when none of the above conditions matched.
9. **Line 9:** `result[key] = value`
`result[key] = value`
10. **Line 10:** `return result`
Returns `result` to the caller.
11. **Line 11:** `# Pattern: Typed dict for structured data`
A comment: *Pattern: Typed dict for structured data.*
12. **Line 12:** `from typing import TypedDict`
Imports `TypedDict` from `typing`.
13. **Line 13:** `class SegmentDict(TypedDict, total=False):`
Defines class `SegmentDict` inheriting from `TypedDict, total=False`.

```
14. Line 14: index: int
    index: int

15. Line 15: title: str
    title: str

16. Line 16: text: str
    text: str

17. Line 17: status: str
    status: str

18. Line 18: saved: str
    saved: str

19. Line 19: duration: float
    duration: float
```

18.5 Key Takeaways

- **JSON** is the primary data format in AIVS — settings, projects, and worker protocols all use it
 - `json.dumps/loads` for strings; `json.dump/load` for files
 - Always use `ensure_ascii=False` for Unicode text
 - XML is used for **DAISY 2.02** book structure (NCC, OPF, SMIL files)
 - The worker protocol uses **JSON lines** (one JSON object per line)
-

Advanced Python

CHAPTER 19

Concurrency & Threading

Learning Objectives

- Understand the GIL (Global Interpreter Lock) and its implications
- Use `threading` for I/O-bound concurrency
- Synchronize access with locks, events, and conditions
- See threading patterns in AIVS (background TTS, downloads)

19.1 The GIL and When to Use Threads

Python's **GIL** (Global Interpreter Lock) means only one thread executes Python bytecode at a time. This makes threads unsuitable for CPU-bound work but excellent for **I/O-bound** tasks:

Task Type	Solution	AIVS Example
I/O-bound (network, disk)	<code>threading</code>	Downloading models, file I/O
CPU-bound (computation)	<code>multiprocessing</code>	N/A (use subprocesses instead)
GUI + background work	<code>threading</code>	TTS synthesis + UI updates

19.2 Basic Threading

```
import threading

# Simple thread
def download_model(url, dest):
    import time
    time.sleep(2) # Simulate download
    print(f"Downloaded to {dest}")

t = threading.Thread(target=download_model, args=("http://...", "/tmp/model"))
t.start() # Starts in background
t.join() # Wait for completion

# Daemon thread (dies when main program exits)
t = threading.Thread(target=background_task, daemon=True)
t.start()

# No need to join - it dies with the main thread
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import threading`
Imports the `threading` module.
2. **Line 2:** `# Simple thread`
A comment: *Simple thread*.
3. **Line 3:** `def download_model(url, dest):`
Defines function `download_model` accepting (url, dest).
4. **Line 4:** `import time`
Imports the `time` module.
5. **Line 5:** `time.sleep(2) # Simulate download`
`time.sleep(2) # Simulate download`
6. **Line 6:** `print(f"Downloaded to {dest}")`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `t = threading.Thread(target=download_model, args=("http://...", "/tmp/model"))`
Creates a background thread. Use `daemon=True` for threads that should not block app exit.
8. **Line 8:** `t.start() # Starts in background`
`t.start() # Starts in background`
9. **Line 9:** `t.join() # Wait for completion`
Joins an iterable of strings with the calling string as separator. `','.join(['a','b'])` returns 'a, b'.
10. **Line 10:** `# Daemon thread (dies when main program exits)`
A comment: *Daemon thread (dies when main program exits)*.
11. **Line 11:** `t = threading.Thread(target=background_task, daemon=True)`
Creates a background thread. Use `daemon=True` for threads that should not block app exit.
12. **Line 12:** `t.start()`
`t.start()`
13. **Line 13:** `# No need to join – it dies with the main thread`
A comment: *No need to join – it dies with the main thread*.

19.3 Thread Synchronization

```
import threading

# Lock - mutual exclusion
_lock = threading.Lock()

def thread_safe_operation():
    with _lock:
        # Only one thread can execute this block at a time
        critical_section()

# RLock - reentrant lock (same thread can acquire multiple times)
_rlock = threading.RLock()

# Event - signaling between threads
cancel_event = threading.Event()

def worker():
    while not cancel_event.is_set():
        # Do work...
    if cancel_event.is_set():
        break # Check between each segment

# Usage in AIVS - cancel a download
cancel_event = threading.Event()
t = threading.Thread(target=download, args=(url, dest, cancel_event))
t.start()

# User clicks Cancel
cancel_event.set() # Signal the thread to stop
t.join()
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `import threading`
Imports the `threading` module.
2. **Line 2:** `# Lock – mutual exclusion`
A comment: *Lock – mutual exclusion.*
3. **Line 3:** `_lock = threading.Lock()`
A mutual exclusion lock. Only one thread can hold it at a time. Prevents race conditions.
4. **Line 4:** `def thread_safe_operation():`
Defines function `thread_safe_operation`.
5. **Line 5:** `with _lock:`
Context manager: `_lock`. Automatically manages resource cleanup.
6. **Line 6:** `# Only one thread can execute this block at a time`
A comment: *Only one thread can execute this block at a time.*
7. **Line 7:** `critical_section()`
Calls function `critical_section()`.
8. **Line 8:** `# RLock – reentrant lock (same thread can acquire multiple times)`
A comment: *RLock – reentrant lock (same thread can acquire multiple times).*
9. **Line 9:** `_rlock = threading.RLock()`
A reentrant lock that allows the same thread to acquire it multiple times.
10. **Line 10:** `# Event – signaling between threads`
A comment: *Event – signaling between threads.*
11. **Line 11:** `cancel_event = threading.Event()`
A flag that threads can wait on. Used for signaling between threads.
12. **Line 12:** `def worker():`
Defines function `worker`.
13. **Line 13:** `while not cancel_event.is_set():`
While loop: repeats as long as `not cancel_event.is_set()`.
14. **Line 14:** `# Do work...`

A comment: *Do work...*

15. **Line 15:** `if cancel_event.is_set():`

Checks condition: `cancel_event.is_set()` . The indented block runs only when this is `True` .

16. **Line 16:** `break # Check between each segment`

`break # Check between each segment`

17. **Line 17:** `# Usage in AIVS – cancel a download`

A comment: *Usage in AIVS – cancel a download.*

18. **Line 18:** `cancel_event = threading.Event()`

A flag that threads can wait on. Used for signaling between threads.

19. **Line 19:** `t = threading.Thread(target=download, args=(url, dest, cancel_event))`

Creates a background thread. Use `daemon=True` for threads that should not block app exit.

20. **Line 20:** `t.start()`

`t.start()`

21. **Line 21:** `# User clicks Cancel`

A comment: *User clicks Cancel.*

22. **Line 22:** `cancel_event.set() # Signal the thread to stop`

`cancel_event.set() # Signal the thread to stop`

23. **Line 23:** `t.join()`

Joins an iterable of strings with the calling string as separator. `','.join(['a','b'])` returns 'a, b'.

19.4 Threading in AIVS

The Synthesis Worker Pattern

```
# From jobs/synthesizer.py – background TTS processing
class SynthesisWorker(threading.Thread):
    """Background thread that synthesizes audio segments."""

    def __init__(self, segments, voice_entry, params,
                 output_dir, on_segment_done, on_all_done,
                 on_error, cancel_event, pause_event, on_status):
        super().__init__(daemon=True)
        self._segments = segments
        self._voice_entry = voice_entry
        self._cancel = cancel_event
        self._pause = pause_event
        self._on_done = on_segment_done
        self._on_all_done = on_all_done
        self._on_error = on_error
        self._on_status = on_status
        self._output_dir = output_dir

    def run(self):
        """Main thread loop – runs in background thread."""
        try:
            engine = get_engine(self._voice_entry)
            for i, seg in enumerate(self._segments):
                # Check for cancellation
                if self._cancel.is_set():
                    break

                # Check for pause
                while self._pause.is_set():
                    if self._cancel.is_set():
                        break
                    time.sleep(0.1)

                # Synthesize
                self._on_status(f"Recording segment {i+1}...")
                audio = engine.synthesize(
                    seg.text,
                    speed=self._params["rate"],
                    pitch=self._params["pitch"],
                    volume=self._params["volume"],
                )

                # Save to disk immediately
                path = self._save_audio(seg, audio)

                # Notify GUI (thread-safe via wx.PostEvent)
                self._on_done(i, seg.title, path)

            self._on_all_done()
        except Exception as exc:
            self._on_error(str(exc))

    def cancel(self):
        self._cancel.set()
```

```
def pause(self):  
    self._pause.set()  
  
def resume(self):  
    self._pause.clear()
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From jobs/synthesizer.py – background TTS processing`
A comment: *From jobs/synthesizer.py – background TTS processing.*
2. **Line 2:** `class SynthesisWorker(threading.Thread):`
Creates a background thread. Use `daemon=True` for threads that should not block app exit.
3. **Line 3:** `"""Background thread that synthesizes audio segments."""`
Docstring: *Background thread that synthesizes audio segments..* Documents the purpose of the following code.
4. **Line 4:** `def __init__(self, segments, voice_entry, params,`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `output_dir, on_segment_done, on_all_done,`
`output_dir, on_segment_done, on_all_done,`
6. **Line 6:** `on_error, cancel_event, pause_event, on_status):`
`on_error, cancel_event, pause_event, on_status):`
7. **Line 7:** `super().__init__(daemon=True)`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
8. **Line 8:** `self._segments = segments`
Stores `segments` in instance attribute `_segments`.
9. **Line 9:** `self._voice_entry = voice_entry`
Stores `voice_entry` in instance attribute `_voice_entry`.
10. **Line 10:** `self._cancel = cancel_event`
Stores `cancel_event` in instance attribute `_cancel`.
11. **Line 11:** `self._pause = pause_event`
Stores `pause_event` in instance attribute `_pause`.
12. **Line 12:** `self._on_done = on_segment_done`
Stores `on_segment_done` in instance attribute `_on_done`.
13. **Line 13:** `self._on_all_done = on_all_done`
Stores `on_all_done` in instance attribute `_on_all_done`.

14. **Line 14:** `self._on_error = on_error`
Stores `on_error` in instance attribute `_on_error`.
15. **Line 15:** `self._on_status = on_status`
Stores `on_status` in instance attribute `_on_status`.
16. **Line 16:** `self._output_dir = output_dir`
Stores `output_dir` in instance attribute `_output_dir`.
17. **Line 17:** `def run(self):`
Defines function `run` accepting (self).
18. **Line 18:** `"""Main thread loop – runs in background thread."""`
Docstring: *Main thread loop – runs in background thread.* Documents the purpose of the following code.
19. **Line 19:** `try:`
Begins exception handling. Code in this block is monitored for errors.
20. **Line 20:** `engine = get_engine(self._voice_entry)`
Assigns `get_engine(self._voice_entry)` to `engine`.
21. **Line 21:** `for i, seg in enumerate(self._segments):`
Adds a counter to an iterable, returning (index, value) pairs. Preferred over manual index tracking.
22. **Line 22:** `# Check for cancellation`
A comment: *Check for cancellation.*
23. **Line 23:** `if self._cancel.is_set():`
Checks condition: `self._cancel.is_set()`. The indented block runs only when this is `True`.
24. **Line 24:** `break`
Exits the innermost loop.
25. **Line 25:** `# Check for pause`
A comment: *Check for pause.*
26. **Line 26:** `while self._pause.is_set():`
While loop: repeats as long as `self._pause.is_set()`.
27. **Line 27:** `if self._cancel.is_set():`
Checks condition: `self._cancel.is_set()`. The indented block runs only when this is `True`.

28. **Line 28:** `break`
Exits the innermost loop.

29. **Line 29:** `time.sleep(0.1)`
`time.sleep(0.1)`

30. **Line 30:** `# Synthesize`
A comment: *Synthesize*.

31. **Line 31:** `self._on_status(f"Recording segment {i+1}...")`
Calls `self._on_status()`.

32. **Line 32:** `audio = engine.synthesize(`
Assigns `engine.synthesize(` to `audio`.

33. **Line 33:** `seg.text,`
`seg.text,`

34. **Line 34:** `speed=self._params["rate"],`
Assigns `self._params["rate"],` to `speed`.

35. **Line 35:** `pitch=self._params["pitch"],`
Assigns `self._params["pitch"],` to `pitch`.

36. **Line 36:** `volume=self._params["volume"],`
Assigns `self._params["volume"],` to `volume`.

37. **Line 37:** `)`
`)`

38. **Line 38:** `# Save to disk immediately`
A comment: *Save to disk immediately*.

39. **Line 39:** `path = self._save_audio(seg, audio)`
Assigns `self._save_audio(seg, audio)` to `path`.

40. **Line 40:** `# Notify GUI (thread-safe via wx.PostEvent)`
A comment: *Notify GUI (thread-safe via wx.PostEvent)*.

41. **Line 41:** `self._on_done(i, seg.title, path)`
Calls `self._on_done()`.

42. **Line 42:** `self._on_all_done()`
Calls `self._on_all_done()` .
43. **Line 43:** `except Exception as exc:`
Catches `Exception` exceptions.
44. **Line 44:** `self._on_error(str(exc))`
Converts a value to its string representation. Essential for string concatenation and formatting.
45. **Line 45:** `def cancel(self):`
Defines function `cancel` accepting (self).
46. **Line 46:** `self._cancel.set()`
`self._cancel.set()`
47. **Line 47:** `def pause(self):`
Defines function `pause` accepting (self).
48. **Line 48:** `self._pause.set()`
`self._pause.set()`
49. **Line 49:** `def resume(self):`
Defines function `resume` accepting (self).
50. **Line 50:** `self._pause.clear()`
Removes all items from the dictionary.

Thread-Safe GUI Communication

```
# wxPython requires UI updates on the main thread
# Workers post events, GUI handles them on the main thread

# From gui/events.py
_SYNTH_STATUS = wx.NewEventType()
EVT_SYNTH_STATUS = wx.PyEventBinder(_SYNTH_STATUS, 1)

class SynthStatusEvent(wx.CommandEvent):
    def __init__(self, message: str):
        super().__init__(_SYNTH_STATUS)
        self.message = message

# In the worker thread:
def _on_status(self, msg):
    wx.PostEvent(self, SynthStatusEvent(msg))

# In the GUI (main thread):
self.Bind(EVT_SYNTH_STATUS, self._handle_status)

def _handle_status(self, evt: SynthStatusEvent):
    self.status_label.SetLabel(evt.message) # Safe - on main thread
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# wxPython requires UI updates on the main thread`
A comment: *wxPython requires UI updates on the main thread.*
2. **Line 2:** `# Workers post events, GUI handles them on the main thread`
A comment: *Workers post events, GUI handles them on the main thread.*
3. **Line 3:** `# From gui/events.py`
A comment: *From gui/events.py.*
4. **Line 4:** `_SYNTH_STATUS = wx.NewEventType()`
Assigns `wx.NewEventType()` to `_SYNTH_STATUS`.
5. **Line 5:** `EVT_SYNTH_STATUS = wx.PyEventBinder(_SYNTH_STATUS, 1)`
Assigns `wx.PyEventBinder(_SYNTH_STATUS, 1)` to `EVT_SYNTH_STATUS`.
6. **Line 6:** `class SynthStatusEvent(wx.PyCommandEvent):`
Defines class `SynthStatusEvent` inheriting from `wx.PyCommandEvent`.
7. **Line 7:** `def __init__(self, message: str):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
8. **Line 8:** `super().__init__(_SYNTH_STATUS)`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
9. **Line 9:** `self.message = message`
Stores `message` in instance attribute `message`.
10. **Line 10:** `# In the worker thread:`
A comment: *In the worker thread:.*
11. **Line 11:** `def _on_status(self, msg):`
Defines function `_on_status` accepting (self, msg).
12. **Line 12:** `wx.PostEvent(self, SynthStatusEvent(msg))`
Creates a wxPython `wx.PostEvent` widget.
13. **Line 13:** `# In the GUI (main thread):`
A comment: *In the GUI (main thread):.*
14. **Line 14:** `self.Bind(EVT_SYNTH_STATUS, self._handle_status)`

Registers an event handler on the frame/window itself. Used for frame-level events.

15. **Line 15:** `def _handle_status(self, evt: SynthStatusEvent):`
Defines function `_handle_status` accepting (self, evt).
16. **Line 16:** `self.status_label.SetLabel(evt.message) # Safe – on main thread`
`self.status_label.SetLabel(evt.message) # Safe – on main thread`

19.5 Download Threading Pattern

```
# From gui/model_panels.py – background download
def _on_download(self, _):
    self._cancel_event = threading.Event()
    self._thread = threading.Thread(
        target=self._download_job,
        args=(tts_id, lang_code, variants),
        daemon=True,
    )
    self._thread.start()

def _download_job(self, tts_id, lang_code, variants):
    def progress(name, done, total):
        wx.PostEvent(self, DownloadProgressEvent(name, done, total))

    try:
        for variant_id in variants:
            self.downloader.download_variant(
                tts_id, lang_code, variant_id,
                progress=progress,
                cancel_event=self._cancel_event,
            )
        wx.PostEvent(self, DownloadFinishedEvent(True, "Done!"))
    except DownloadCancelled:
        wx.PostEvent(self, DownloadFinishedEvent(False, "Cancelled"))
    except Exception as exc:
        wx.PostEvent(self, DownloadFinishedEvent(False, str(exc)))
```

Explain Line by Line Code Description (Block 5)

1. **Line 1:** `# From gui/model_panels.py – background download`
A comment: *From gui/model_panels.py – background download.*
2. **Line 2:** `def _on_download(self, _):`
Defines function `_on_download` accepting (self, _).
3. **Line 3:** `self._cancel_event = threading.Event()`
A flag that threads can wait on. Used for signaling between threads.
4. **Line 4:** `self._thread = threading.Thread(`
Creates a background thread. Use `daemon=True` for threads that should not block app exit.
5. **Line 5:** `target=self._download_job,`
Assigns `self._download_job`, to `target`.
6. **Line 6:** `args=(tts_id, lang_code, variants),`
Assigns `(tts_id, lang_code, variants)`, to `args`.
7. **Line 7:** `daemon=True,`
Assigns `True`, to `daemon`.
8. **Line 8:** `)`
`)`
9. **Line 9:** `self._thread.start()`
`self._thread.start()`
10. **Line 10:** `def _download_job(self, tts_id, lang_code, variants):`
Defines function `_download_job` accepting (self, tts_id, lang_code, variants).
11. **Line 11:** `def progress(name, done, total):`
Defines function `progress` accepting (name, done, total).
12. **Line 12:** `wx.PostEvent(self, DownloadProgressEvent(name, done, total))`
Creates a wxPython `wx.PostEvent` widget.
13. **Line 13:** `try:`
Begins exception handling. Code in this block is monitored for errors.

14. **Line 14:** `for variant_id in variants:`
Iterates over `variants`, assigning each element to `variant_id`.

15. **Line 15:** `self.downloader.download_variant(`
`self.downloader.download_variant(`

16. **Line 16:** `tts_id, lang_code, variant_id,`
`tts_id, lang_code, variant_id,`

17. **Line 17:** `progress=progress,`
Assigns `progress`, to `progress`.

18. **Line 18:** `cancel_event=self._cancel_event,`
Assigns `self._cancel_event`, to `cancel_event`.

19. **Line 19:** `)`
`)`

20. **Line 20:** `wx.PostEvent(self, DownloadFinishedEvent(True, "Done!"))`
Creates a wxPython `wx.PostEvent` widget.

21. **Line 21:** `except DownloadCancelled:`
Catches `DownloadCancelled` exceptions.

22. **Line 22:** `wx.PostEvent(self, DownloadFinishedEvent(False, "Cancelled"))`
Creates a wxPython `wx.PostEvent` widget.

23. **Line 23:** `except Exception as exc:`
Catches `Exception` exceptions.

24. **Line 24:** `wx.PostEvent(self, DownloadFinishedEvent(False, str(exc)))`
Converts a value to its string representation. Essential for string concatenation and formatting.

19.6 Key Takeaways

- **The GIL** limits true parallelism in threads — use threads for I/O, not CPU work
- **threading.Thread** creates background threads; always use `daemon=True` for non-critical threads

- **threading.Event** is the primary mechanism for cancellation in AIVS
- **wx.PostEvent()** is the only safe way to update GUI from worker threads
- The AIVS **SynthesisWorker** pattern: thread + cancel event + pause event + GUI events

 **Exercise: Exercise 19.1:** Create a simple download simulator using threads. It should download 5 files concurrently, report progress via events, and support cancellation.

Multiprocessing

Learning Objectives

- Understand when to use processes vs. threads
 - Use the `subprocess` module for process management
 - See the subprocess architecture in AIVS
-

20.1 Processes vs. Threads

Feature	Threads	Processes
GIL	Shared (limited parallelism)	Each has its own (true parallelism)
Memory	Shared	Separate (need IPC)
Creation	Fast	Slower
Use case	I/O-bound	CPU-bound
AIVS use	TTS synthesis, downloads	Voice Lab clone worker, OmniVoice worker

20.2 The subprocess Module

```
import subprocess

# Run a command and wait
result = subprocess.run(
    ["python", "-c", "print('hello')"],
    capture_output=True,
    text=True,
)
print(result.stdout)      # "hello\n"
print(result.returncode)  # 0

# Start a Long-running process
proc = subprocess.Popen(
    ["python", "-m", "worker"],
    stdin=subprocess.PIPE,
    stdout=subprocess.PIPE,
    text=True,
)

# Communicate via stdin/stdout
proc.stdin.write('{"cmd": "ping"}\n')
proc.stdin.flush()
response = proc.stdout.readline() # Read one Line

# Terminate
proc.terminate()
proc.wait(timeout=5)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import subprocess`
Imports the `subprocess` module.
2. **Line 2:** `# Run a command and wait`
A comment: *Run a command and wait.*
3. **Line 3:** `result = subprocess.run(`
Runs a command and waits for it to complete. Returns a `CompletedProcess`.
4. **Line 4:** `["python", "-c", "print('hello')"],`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `capture_output=True,`
Assigns `True`, to `capture_output`.
6. **Line 6:** `text=True,`
Assigns `True`, to `text`.
7. **Line 7:** `)`
`)`
8. **Line 8:** `print(result.stdout) # "hello\n"`
Outputs text to the console. The primary way to display information during development and debugging.
9. **Line 9:** `print(result.returncode) # 0`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# Start a long-running process`
A comment: *Start a long-running process.*
11. **Line 11:** `proc = subprocess.Popen(`
Starts a process. More control than `run()` — can read `stdout/stderr` asynchronously.
12. **Line 12:** `["python", "-m", "worker"],`
`["python", "-m", "worker"],`
13. **Line 13:** `stdin=subprocess.PIPE,`
Assigns `subprocess.PIPE`, to `stdin`.
14. **Line 14:** `stdout=subprocess.PIPE,`

```
Assigns subprocess.PIPE, to stdout .

15. Line 15: text=True,
Assigns True, to text .

16. Line 16: )
)

17. Line 17: # Communicate via stdin/stdout
A comment: Communicate via stdin/stdout.

18. Line 18: proc.stdin.write('{ "cmd": "ping"}\n')
Writes a string to the file. In text mode, does not add a newline automatically.

19. Line 19: proc.stdin.flush()
proc.stdin.flush()

20. Line 20: response = proc.stdout.readline() # Read one line
Reads a single line from the file, including the newline character.

21. Line 21: # Terminate
A comment: Terminate.

22. Line 22: proc.terminate()
proc.terminate()

23. Line 23: proc.wait(timeout=5)
proc.wait(timeout=5)
```

20.3 AIVS Subprocess Architecture

AI Voice Studio uses subprocesses to isolate heavy runtimes (PyTorch, ONNX) from the main GUI process. This is a critical architectural decision:

```

# From clone/__init__.py - CloneClient manages a subprocess
class CloneClient:
    def _ensure_proc(self):
        if self._proc is not None and self._proc.poll() is None:
            return
        cmd = [sys.executable, "-m", "ai_voice_studio.clone.worker"]
        self._proc = subprocess.Popen(
            cmd,
            stdin=subprocess.PIPE,
            stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL,
            text=True,
            bufsize=1,
        )
        # Verify the worker started
        resp = self._request({"cmd": "ping"})
        if not resp.get("ok"):
            raise CloneEngineError("Worker failed to start")

    def _request(self, req: dict) -> dict:
        """Send JSON request, read JSON response."""
        with self._lock:
            req["id"] = self._next_id
            self._next_id += 1
            self._proc.stdin.write(json.dumps(req) + "\n")
            self._proc.stdin.flush()
            line = self._proc.stdout.readline()
            return json.loads(line)

# Protocol: JSON Lines over stdin/stdout
# Request: {"cmd": "synthesize", "text": "Hello", "speed": 1.0}
# Response: {"ok": true, "wav": "base64..."}

```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From clone/__init__.py - CloneClient manages a subprocess`

A comment: *From clone/__init__.py - CloneClient manages a subprocess.*

2. **Line 2:** `class CloneClient:`

Defines class `CloneClient` .

3. **Line 3:** `def _ensure_proc(self):`

Defines function `_ensure_proc` accepting (self).

4. **Line 4:** `if self._proc is not None and self._proc.poll() is None:`

Checks condition: `self._proc is not None and self._proc.poll() is None` . The indented block runs only when this is `True` .

5. **Line 5:** `return`

Returns `None` implicitly (exits the function).

6. **Line 6:** `cmd = [sys.executable, "-m", "ai_voice_studio.clone.worker"]`

Assigns `[sys.executable, "-m", "ai_voice_studio.clone.worker"]` to `cmd` .

7. **Line 7:** `self._proc = subprocess.Popen(`

Starts a process. More control than `run()` — can read `stdout/stderr` asynchronously.

8. **Line 8:** `cmd,`

`cmd,`

9. **Line 9:** `stdin=subprocess.PIPE,`

Assigns `subprocess.PIPE,` to `stdin` .

10. **Line 10:** `stdout=subprocess.PIPE,`

Assigns `subprocess.PIPE,` to `stdout` .

11. **Line 11:** `stderr=subprocess.DEVNULL,`

Assigns `subprocess.DEVNULL,` to `stderr` .

12. **Line 12:** `text=True,`

Assigns `True,` to `text` .

13. **Line 13:** `bufsize=1,`

Assigns 1, to bufsize .

14. Line 14:)

)

15. Line 15: # Verify the worker started

A comment: *Verify the worker started.*

16. Line 16: resp = self._request({"cmd": "ping"})

Assigns self._request({"cmd": "ping"}) to resp .

17. Line 17: if not resp.get("ok"):

Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.

18. Line 18: raise CloneEngineError("Worker failed to start")

Manually raises an exception. Use to signal error conditions to callers.

19. Line 19: def _request(self, req: dict) -> dict:

Defines function _request accepting (self, req), returning dict .

20. Line 20: """Send JSON request, read JSON response."""

Docstring: *Send JSON request, read JSON response..* Documents the purpose of the following code.

21. Line 21: with self._lock:

Context manager: self._lock . Automatically manages resource cleanup.

22. Line 22: req["id"] = self._next_id

req["id"] = self._next_id

23. Line 23: self._next_id += 1

self._next_id += 1

24. Line 24: self._proc.stdin.write(json.dumps(req) + "\n")

Serializes a Python object to a JSON string.

25. Line 25: self._proc.stdin.flush()

self._proc.stdin.flush()

26. Line 26: line = self._proc.stdout.readline()

Reads a single line from the file, including the newline character.

27. Line 27: return json.loads(line)

Deserializes a JSON string into a Python object.

28. **Line 28:** `# Protocol: JSON lines over stdin/stdout`

A comment: *Protocol: JSON lines over stdin/stdout.*

29. **Line 29:** `# Request: {"cmd": "synthesize", "text": "Hello", "speed": 1.0}`

A comment: *Request: {"cmd": "synthesize", "text": "Hello", "speed": 1.0}.*

30. **Line 30:** `# Response: {"ok": true, "wav": "base64..."}`

A comment: *Response: {"ok": true, "wav": "base64..."}*.

20.4 Windows-Specific: CREATE_NO_WINDOW

```
# On Windows, hide console windows for subprocesses
creationflags = 0
if sys.platform == "win32":
    creationflags = getattr(subprocess, "CREATE_NO_WINDOW", 0)

proc = subprocess.Popen(
    cmd,
    creationflags=creationflags,
    ...
)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# On Windows, hide console windows for subprocesses`
A comment: *On Windows, hide console windows for subprocesses.*
2. **Line 2:** `creationflags = 0`
Assigns `0` to `creationflags`.
3. **Line 3:** `if sys.platform == "win32":`
Checks condition: `sys.platform == "win32"`. The indented block runs only when this is `True`.
4. **Line 4:** `creationflags = getattr(subprocess, "CREATE_NO_WINDOW", 0)`
Gets an attribute value by name. Supports a default value if the attribute doesn't exist.
5. **Line 5:** `proc = subprocess.Popen(`
Starts a process. More control than `run()` — can read `stdout/stderr` asynchronously.
6. **Line 6:** `cmd,`
`cmd,`
7. **Line 7:** `creationflags=creationflags,`
Assigns `creationflags`, to `creationflags`.
8. **Line 8:** `...`
Ellipsis: abstract method placeholder.
9. **Line 9:** `)`
`)`

20.5 Key Takeaways

- Use **subprocess** when you need true isolation (different runtime, memory space)
- AIVS uses **subprocesses** for TTS engines to prevent runtime conflicts
- **JSON lines protocol** over `stdin/stdout` is the communication pattern
- Always use **threading.RLock** to synchronize access to the subprocess

Async I/O

Learning Objectives

- Understand asyncio's event loop model
 - Write async functions with `async/await`
 - Understand when async is appropriate vs. threads
 - See where AIVS could benefit from async patterns
-

21.1 What Is Async I/O?

`asyncio` is Python's built-in framework for writing concurrent code using **coroutines**. Unlike threads, async code runs in a **single thread** and cooperatively yields control at `await` points.

Feature	Threads	Async
Parallelism	Preemptive (GIL)	Cooperative (single thread)
Overhead	Per-thread stack (~8MB)	Per-coroutine (~KB)
Best for	Mixed I/O + some CPU	High-concurrency I/O
GUI compatible?	Yes (PostEvent)	Needs event loop integration

21.2 Async Basics

```
import asyncio

# Define a coroutine
async def fetch_model(url: str) -> bytes:
    import aiohttp
    async with aiohttp.ClientSession() as session:
        async with session.get(url) as response:
            return await response.read()

# Run a coroutine
data = asyncio.run(fetch_model("https://example.com/model.onnx"))

# Multiple concurrent coroutines
async def download_all(urls: list[str]):
    tasks = [fetch_model(url) for url in urls]
    results = await asyncio.gather(*tasks)
    return results
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import asyncio`
Imports the `asyncio` module.
2. **Line 2:** `# Define a coroutine`
A comment: *Define a coroutine.*
3. **Line 3:** `async def fetch_model(url: str) -> bytes:`
Defines an asynchronous function (coroutine). Can use 'await' inside.
4. **Line 4:** `import aiohttp`
Imports the `aiohttp` module.
5. **Line 5:** `async with aiohttp.ClientSession() as session:`
`async with aiohttp.ClientSession() as session:`
6. **Line 6:** `async with session.get(url) as response:`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
7. **Line 7:** `return await response.read()`
Reads the entire file contents as a string.
8. **Line 8:** `# Run a coroutine`
A comment: *Run a coroutine.*
9. **Line 9:** `data = asyncio.run(fetch_model("https://example.com/model.onnx"))`
Runs an async function in the event loop. Entry point for async programs.
10. **Line 10:** `# Multiple concurrent coroutines`
A comment: *Multiple concurrent coroutines.*
11. **Line 11:** `async def download_all(urls: list[str]):`
Defines an asynchronous function (coroutine). Can use 'await' inside.
12. **Line 12:** `tasks = [fetch_model(url) for url in urls]`
Assigns `[fetch_model(url) for url in urls]` to `tasks`.
13. **Line 13:** `results = await asyncio.gather(*tasks)`
Runs multiple coroutines concurrently and returns their results.
14. **Line 14:** `return results`

Returns `results` to the caller.

21.3 When Async Isn't the Right Choice

AI Voice Studio uses **threads**, not `async`, because:

- **wxPython is not async-compatible** — its event loop conflicts with `asyncio`
- **Blocking libraries** — `sherpa-onnx` and `NumPy` are synchronous
- **Subprocess isolation** — heavy work runs in separate processes anyway
- **Thread simplicity** — for 5-10 concurrent tasks, threads are simpler

i Info: When to use `async`: Web servers handling thousands of connections, web scrapers, `WebSocket` clients. For a desktop app with a handful of background tasks, threads are the better choice.

21.4 Key Takeaways

- `async/await` is for I/O-bound concurrency in a single thread
- `asyncio.gather()` runs multiple coroutines concurrently
- AIVS uses **threads** + **subprocesses** instead, because `wxPython` and ML libraries are synchronous
- `Async` shines for **high-concurrency network I/O** (hundreds of connections)

Type Hints & Static Typing

Learning Objectives

- Understand type hints and their benefits
 - Use modern type hint syntax (Python 3.9-3.12)
 - See type hints throughout the AIVS codebase
-

22.1 Why Type Hints?

Type hints document expected types without enforcing them at runtime. Benefits:

- **IDE support** — autocompletion, error detection
 - **Documentation** — function signatures are self-documenting
 - **Catch bugs early** — static type checkers (mypy, Pylance) catch type errors before runtime
 - **Refactoring safety** — rename a method and see all affected call sites
-

22.2 Basic Type Hints

```
# Variable annotations
name: str = "AI Voice Studio"
version: float = 2.0
voices: list[str] = ["af", "am_adam"]

# Function annotations
def greet(name: str) -> str:
    return f"Hello, {name}!"

# No return value (returns None)
def log(message: str) -> None:
    print(message)

# Optional values
def find(items: list, index: int) -> str | None:
    if 0 <= index < len(items):
        return items[index]
    return None
```


Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Variable annotations`
A comment: *Variable annotations.*

2. **Line 2:** `name: str = "AI Voice Studio"`
`name: str = "AI Voice Studio"`

3. **Line 3:** `version: float = 2.0`
`version: float = 2.0`

4. **Line 4:** `voices: list[str] = ["af", "am_adam"]`
`voices: list[str] = ["af", "am_adam"]`

5. **Line 5:** `# Function annotations`
A comment: *Function annotations.*

6. **Line 6:** `def greet(name: str) -> str:`
Defines function `greet` accepting (name), returning `str`.

7. **Line 7:** `return f"Hello, {name}!"`
Returns `f"Hello, {name}!"` to the caller.

8. **Line 8:** `# No return value (returns None)`
A comment: *No return value (returns None).*

9. **Line 9:** `def log(message: str) -> None:`
Defines function `log` accepting (message), returning `None`.

10. **Line 10:** `print(message)`
Outputs text to the console. The primary way to display information during development and debugging.

11. **Line 11:** `# Optional values`
A comment: *Optional values.*

12. **Line 12:** `def find(items: list, index: int) -> str | None:`
Defines function `find` accepting (items, index), returning `str | None`.

13. **Line 13:** `if 0 <= index < len(items):`
Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

14. **Line 14:** `return items[index]`
Returns `items[index]` to the caller.
15. **Line 15:** `return None`
Returns `None` (no value).

22.3 Complex Types

```
from typing import TypeAlias

# Dict types
config: dict[str, str | int | bool] = {"rate": 1.0}
voice_entry: dict[str, any] = {"id": "af"}

# TypedDict for structured dicts
from typing import TypedDict

class VoiceParams(TypedDict):
    rate: float
    pitch: float
    volume: float
    output_format: str

# Callable types
from typing import Callable
callback: Callable[[str, int, int], None] # func(name, done, total) -> None

# Union types
value: str | int | float # Python 3.10+

# Type aliases
Path: TypeAlias = str
AudioSamples: TypeAlias = "np.ndarray"

# Generic types
from typing import TypeVar, Generic
T = TypeVar("T")

class Cache(Generic[T]):
    def get(self, key: str) -> T | None: ...
    def set(self, key: str, value: T) -> None: ...
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `from typing import TypeAlias`
Imports `TypeAlias` from `typing`.
2. **Line 2:** `# Dict types`
A comment: *Dict types*.
3. **Line 3:** `config: dict[str, str | int | bool] = {"rate": 1.0}`
`config: dict[str, str | int | bool] = {"rate": 1.0}`
4. **Line 4:** `voice_entry: dict[str, any] = {"id": "af"}`
Begins exception handling. Code in this block is monitored for errors.
5. **Line 5:** `# TypedDict for structured dicts`
A comment: *TypedDict for structured dicts*.
6. **Line 6:** `from typing import TypedDict`
Imports `TypedDict` from `typing`.
7. **Line 7:** `class VoiceParams(TypedDict):`
Defines class `VoiceParams` inheriting from `TypedDict`.
8. **Line 8:** `rate: float`
`rate: float`
9. **Line 9:** `pitch: float`
`pitch: float`
10. **Line 10:** `volume: float`
`volume: float`
11. **Line 11:** `output_format: str`
`output_format: str`
12. **Line 12:** `# Callable types`
A comment: *Callable types*.
13. **Line 13:** `from typing import Callable`
Imports `Callable` from `typing`.

14. **Line 14:** `callback: Callable[[str, int, int], None] # func(name, done, total) -> None`
Type hint for a callable (function) with specific parameter and return types.
15. **Line 15:** `# Union types`
A comment: *Union types*.
16. **Line 16:** `value: str | int | float # Python 3.10+`
`value: str | int | float # Python 3.10+`
17. **Line 17:** `# Type aliases`
A comment: *Type aliases*.
18. **Line 18:** `Path: TypeAlias = str`
`Path: TypeAlias = str`
19. **Line 19:** `AudioSamples: TypeAlias = "np.ndarray"`
`AudioSamples: TypeAlias = "np.ndarray"`
20. **Line 20:** `# Generic types`
A comment: *Generic types*.
21. **Line 21:** `from typing import TypeVar, Generic`
Imports `TypeVar, Generic` from `typing`.
22. **Line 22:** `T = TypeVar("T")`
Assigns `TypeVar("T")` to `T`.
23. **Line 23:** `class Cache(Generic[T]):`
Defines class `Cache` inheriting from `Generic[T]`.
24. **Line 24:** `def get(self, key: str) -> T | None: ...`
Defines function `get` accepting (self, key), returning `T | None`.
25. **Line 25:** `def set(self, key: str, value: T) -> None: ...`
Defines function `set` accepting (self, key, value), returning `None`.

22.4 Type Hints in AIVS

```
# From tts/engine.py - complete function signatures
def get_engine(voice_entry: dict) -> "TtsEngine":
    """Get or create a cached TTS engine."""

def _shift_pitch(samples: "np.ndarray", pitch: float) -> "np.ndarray":
    """Shift the pitch of audio samples."""

# From settings.py - properties with type hints
class Settings:
    @property
    def theme(self) -> str:
        return self._data.get("theme", "system")

    @property
    def recent_projects(self) -> list[list[str]]:
        return self._data.get("recent_projects", [])

# From clone/__init__.py - complex type annotations
class CloneClient:
    def synthesize(
        self,
        text: str,
        sample_wav: str,
        language: str = "en",
        speed: float = 1.0,
    ) -> np.ndarray:
        """Synthesize text; returns int16 samples at 24 kHz."""

# From python_runtime/__init__.py
class PythonRuntime:
    def pip_install(
        self,
        packages: list[str] | str,
        progress: Callable[[str, int, int], None] | None = None,
        cancel_event: threading.Event | None = None,
    ) -> dict[str, str]:
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From tts/engine.py – complete function signatures`
A comment: *From tts/engine.py – complete function signatures.*
2. **Line 2:** `def get_engine(voice_entry: dict) -> "TtsEngine":`
Begins exception handling. Code in this block is monitored for errors.
3. **Line 3:** `"""Get or create a cached TTS engine."""`
Docstring: *Get or create a cached TTS engine..* Documents the purpose of the following code.
4. **Line 4:** `def _shift_pitch(samples: "np.ndarray", pitch: float) -> "np.ndarray":`
Defines function `_shift_pitch` accepting (samples, pitch), returning `"np.ndarray"`.
5. **Line 5:** `"""Shift the pitch of audio samples."""`
Docstring: *Shift the pitch of audio samples..* Documents the purpose of the following code.
6. **Line 6:** `# From settings.py – properties with type hints`
A comment: *From settings.py – properties with type hints.*
7. **Line 7:** `class Settings:`
Defines class `Settings`.
8. **Line 8:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
9. **Line 9:** `def theme(self) -> str:`
Defines function `theme` accepting (self), returning `str`.
10. **Line 10:** `return self._data.get("theme", "system")`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
11. **Line 11:** `@property`
Decorator that converts a method into a managed attribute. Enables getter/setter syntax.
12. **Line 12:** `def recent_projects(self) -> list[list[str]]:`
Defines function `recent_projects` accepting (self), returning `list[list[str]]`.
13. **Line 13:** `return self._data.get("recent_projects", [])`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
14. **Line 14:** `# From clone/__init__.py – complex type annotations`

A comment: *From clone/___init__.py — complex type annotations.*

15. Line 15: `class CloneClient:`

Defines class `CloneClient` .

16. Line 16: `def synthesize(`

`def synthesize(`

17. Line 17: `self,`

`self,`

18. Line 18: `text: str,`

`text: str,`

19. Line 19: `sample_wav: str,`

`sample_wav: str,`

20. Line 20: `language: str = "en",`

`language: str = "en",`

21. Line 21: `speed: float = 1.0,`

`speed: float = 1.0,`

22. Line 22: `) -> np.ndarray:`

`) -> np.ndarray:`

23. Line 23: `"""Synthesize text; returns int16 samples at 24 kHz."""`

Docstring: *Synthesize text; returns int16 samples at 24 kHz..* Documents the purpose of the following code.

24. Line 24: `# From python_runtime/___init__.py`

A comment: *From python_runtime/___init__.py.*

25. Line 25: `class PythonRuntime:`

Defines class `PythonRuntime` .

26. Line 26: `def pip_install(`

Returns True if all elements in the iterable are truthy. Short-circuits on first False.

27. Line 27: `self,`

`self,`

28. **Line 28:** `packages: list[str] | str,`
`packages: list[str] | str,`
29. **Line 29:** `progress: Callable[[str, int, int], None] | None = None,`
Type hint for a callable (function) with specific parameter and return types.
30. **Line 30:** `cancel_event: threading.Event | None = None,`
A flag that threads can wait on. Used for signaling between threads.
31. **Line 31:** `) -> dict[str, str]:`
`) -> dict[str, str]:`

22.5 Forward References

```
# When a class references itself or a not-yet-defined class
class Node:
    def __init__(self, children: list["Node"] | None = None):
        self.children = children or []

# Or use from __future__ import annotations (Python 3.7+)
from __future__ import annotations # ALL annotations are strings by default

class Node:
    def __init__(self, children: list[Node] | None = None):
        self.children = children or # No quotes needed!
```


Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# When a class references itself or a not-yet-defined class`
A comment: *When a class references itself or a not-yet-defined class.*
2. **Line 2:** `class Node:`
Defines class `Node` .
3. **Line 3:** `def __init__(self, children: list["Node"] | None = None):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `self.children = children or []`
Stores `children or []` in instance attribute `children` .
5. **Line 5:** `# Or use from __future__ import annotations (Python 3.7+)`
A comment: *Or use from __future__ import annotations (Python 3.7+).*
6. **Line 6:** `from __future__ import annotations # All annotations are strings by default`
Imports `annotations # All annotations are strings by default` from `__future__` .
7. **Line 7:** `class Node:`
Defines class `Node` .
8. **Line 8:** `def __init__(self, children: list[Node] | None = None):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
9. **Line 9:** `self.children = children or # No quotes needed!`
Stores `children or # No quotes needed!` in instance attribute `children` .

22.6 Key Takeaways

- Type hints are **optional documentation** that help IDEs and catch bugs
- Use **Python 3.10+ syntax**: `str | None` instead of `Optional[str]`
- **TypedDict** for structured dicts, **Callable** for function types
- AIVS uses type hints on **all public function signatures**
- `from __future__ import annotations` enables forward references

Metaclasses & Descriptors

Learning Objectives

- Understand how Python creates classes
 - Use metaclasses for class-level customization
 - Understand descriptors (the mechanism behind `@property`)
 - Know when to use these advanced features (and when not to)
-

23.1 How Python Creates Classes

```
# Every class has a metaclass – by default it's 'type'
class MyClass:
    pass

print(type(MyClass))    # <class 'type'>
print(type(42))         # <class 'int'>
print(type(MyClass()))  # <class 'MyClass'>

# type(name, bases, namespace) creates a class
MyClass = type("MyClass", (), {"x": 42})
print(MyClass().x)      # 42
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Every class has a metaclass – by default it's 'type'`
A comment: *Every class has a metaclass – by default it's 'type'.*
2. **Line 2:** `class MyClass:`
Defines class `MyClass`.
3. **Line 3:** `pass`
Placeholder: does nothing.
4. **Line 4:** `print(type(MyClass)) # <class 'type'>`
Outputs text to the console. The primary way to display information during development and debugging.
5. **Line 5:** `print(type(42)) # <class 'int'>`
Outputs text to the console. The primary way to display information during development and debugging.
6. **Line 6:** `print(type(MyClass())) # <class 'MyClass'>`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# type(name, bases, namespace) creates a class`
A comment: *type(name, bases, namespace) creates a class.*
8. **Line 8:** `MyClass = type("MyClass", (), {"x": 42})`
Returns the type/class of an object. Used for debugging and type checking.
9. **Line 9:** `print(MyClass().x) # 42`
Outputs text to the console. The primary way to display information during development and debugging.

23.2 Custom Metaclasses

```
class AutoRegisterMeta(type):
    """Automatically register subclasses."""
    _registry: dict[str, type] = {}

    def __new__(mcs, name, bases, namespace):
        cls = super().__new__(mcs, name, bases, namespace)
        if bases: # Don't register the base class itself
            mcs._registry[name] = cls
        return cls

class Engine(metaclass=AutoRegisterMeta):
    pass

class PiperEngine(Engine):
    pass # Auto-registered!

class KokoroEngine(Engine):
    pass # Auto-registered!

print(AutoRegisterMeta._registry)
# {'PiperEngine': <class 'PiperEngine'>, 'KokoroEngine': <class 'KokoroEngine'>}
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `class AutoRegisterMeta(type):`

Defines class `AutoRegisterMeta` inheriting from `type` .

2. **Line 2:** `"""Automatically register subclasses."""`

Docstring: *Automatically register subclasses.*. Documents the purpose of the following code.

3. **Line 3:** `_registry: dict[str, type] = {}`

Begins exception handling. Code in this block is monitored for errors.

4. **Line 4:** `def __new__(mcs, name, bases, namespace):`

Defines function `__new__` accepting (mcs, name, bases, namespace).

5. **Line 5:** `cls = super().__new__(mcs, name, bases, namespace)`

Returns a proxy object that delegates method calls to a parent class. Used in `__init__` to call the parent constructor.

6. **Line 6:** `if bases: # Don't register the base class itself`

Checks condition: `bases` . The indented block runs only when this is `True` .

7. **Line 7:** `mcs._registry[name] = cls`

`mcs._registry[name] = cls`

8. **Line 8:** `return cls`

Returns `cls` to the caller.

9. **Line 9:** `class Engine(metaclass=AutoRegisterMeta):`

Defines class `Engine` inheriting from `metaclass=AutoRegisterMeta` .

10. **Line 10:** `pass`

Placeholder: does nothing.

11. **Line 11:** `class PiperEngine(Engine):`

Defines class `PiperEngine` inheriting from `Engine` .

12. **Line 12:** `pass # Auto-registered!`

`pass # Auto-registered!`

13. **Line 13:** `class KokoroEngine(Engine):`

Defines class `KokoroEngine` inheriting from `Engine` .

14. **Line 14:** `pass # Auto-registered!`
`pass # Auto-registered!`
15. **Line 15:** `print(AutoRegisterMeta._registry)`
Outputs text to the console. The primary way to display information during development and debugging.
16. **Line 16:** `# {'PiperEngine': <class 'PiperEngine'>, 'KokoroEngine': <class 'KokoroEngine'>}`
A comment: `{'PiperEngine': <class 'PiperEngine'>, 'KokoroEngine': <class 'KokoroEngine'>}`.

23.3 Descriptors

Descriptors are objects that define `__get__`, `__set__`, or `__delete__`. They power `@property`:

```
class Validated:
    """Descriptor that validates values on assignment."""

    def __init__(self, min_val: float, max_val: float):
        self.min_val = min_val
        self.max_val = max_val

    def __set_name__(self, owner, name):
        self.name = name

    def __get__(self, obj, objtype=None):
        if obj is None:
            return self
        return obj.__dict__.get(self.name, 1.0)

    def __set__(self, obj, value):
        if not (self.min_val <= value <= self.max_val):
            raise ValueError(
                f"{self.name} must be between {self.min_val} and {self.max_val}"
            )
        obj.__dict__[self.name] = value

class VoiceParams:
    rate = Validated(0.5, 2.0)
    pitch = Validated(0.5, 2.0)
    volume = Validated(0.0, 2.0)

params = VoiceParams()
params.rate = 1.5 # OK
# params.rate = 5.0 # ValueError!
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `class Validated:`
Defines class `Validated` .
2. **Line 2:** `"""Descriptor that validates values on assignment."""`
Docstring: *Descriptor that validates values on assignment.*. Documents the purpose of the following code.
3. **Line 3:** `def __init__(self, min_val: float, max_val: float):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `self.min_val = min_val`
Stores `min_val` in instance attribute `min_val` .
5. **Line 5:** `self.max_val = max_val`
Stores `max_val` in instance attribute `max_val` .
6. **Line 6:** `def __set_name__(self, owner, name):`
Defines function `__set_name__` accepting (self, owner, name).
7. **Line 7:** `self.name = name`
Stores `name` in instance attribute `name` .
8. **Line 8:** `def __get__(self, obj, objtype=None):`
Defines function `__get__` accepting (self, obj, objtype).
9. **Line 9:** `if obj is None:`
Checks condition: `obj is None` . The indented block runs only when this is `True` .
10. **Line 10:** `return self`
Returns `self` to the caller.
11. **Line 11:** `return obj.__dict__.get(self.name, 1.0)`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
12. **Line 12:** `def __set__(self, obj, value):`
Defines function `__set__` accepting (self, obj, value).
13. **Line 13:** `if not (self.min_val <= value <= self.max_val):`
Checks condition: `not (self.min_val <= value <= self.max_val)` . The indented block runs only when this is `True` .

14. **Line 14:** `raise ValueError(`
Raised when a function receives an argument of the right type but inappropriate value.
15. **Line 15:** `f"{self.name} must be between {self.min_val} and {self.max_val}"`
`f"{self.name} must be between {self.min_val} and {self.max_val}"`
16. **Line 16:** `)`
`)`
17. **Line 17:** `obj.__dict__[self.name] = value`
`obj.__dict__[self.name] = value`
18. **Line 18:** `class VoiceParams:`
Defines class `VoiceParams`.
19. **Line 19:** `rate = Validated(0.5, 2.0)`
Assigns `Validated(0.5, 2.0)` to `rate`.
20. **Line 20:** `pitch = Validated(0.5, 2.0)`
Assigns `Validated(0.5, 2.0)` to `pitch`.
21. **Line 21:** `volume = Validated(0.0, 2.0)`
Assigns `Validated(0.0, 2.0)` to `volume`.
22. **Line 22:** `params = VoiceParams()`
Assigns `VoiceParams()` to `params`.
23. **Line 23:** `params.rate = 1.5 # OK`
Assigns `1.5 # OK` to `params.rate`.
24. **Line 24:** `# params.rate = 5.0 # ValueError!`
A comment: `params.rate = 5.0 # ValueError!`.

23.4 Key Takeaways

- **Metaclasses** customize class creation — powerful but rarely needed
- **Descriptors** power `@property` and can add validation, caching, etc.

- In practice, prefer **@property** and **@dataclass** over metaclasses
 - AIVS doesn't use custom metaclasses — it uses **composition** and **decorators** instead
-

Design Patterns in Python

Learning Objectives

- Implement common design patterns in Pythonic ways
 - See Singleton, Observer, Strategy, and Factory patterns in AIVS
 - Understand when patterns are useful vs. over-engineering
-

24.1 Singleton Pattern

```
# Ensures only one instance exists
class Settings:
    _instance = None
    _lock = threading.Lock()

    def __new__(cls, *args, **kwargs):
        if cls._instance is None:
            with cls._lock:
                if cls._instance is None:
                    cls._instance = super().__new__(cls)
        return cls._instance

# From AIVS - the addon manager is a singleton
_default_manager: AddonManager | None = None
_manager_lock = threading.Lock()

def get_addon_manager() -> AddonManager:
    global _default_manager
    if _default_manager is None:
        with _manager_lock:
            if _default_manager is None:
                _default_manager = AddonManager()
    return _default_manager
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Ensures only one instance exists`
A comment: *Ensures only one instance exists.*
2. **Line 2:** `class Settings:`
Defines class `Settings`.
3. **Line 3:** `_instance = None`
Assigns `None` to `_instance`.
4. **Line 4:** `_lock = threading.Lock()`
A mutual exclusion lock. Only one thread can hold it at a time. Prevents race conditions.
5. **Line 5:** `def __new__(cls, *args, **kwargs):`
Defines function `__new__` accepting `(cls, *args, **kwargs)`.
6. **Line 6:** `if cls._instance is None:`
Checks condition: `cls._instance is None`. The indented block runs only when this is `True`.
7. **Line 7:** `with cls._lock:`
Context manager: `cls._lock`. Automatically manages resource cleanup.
8. **Line 8:** `if cls._instance is None:`
Checks condition: `cls._instance is None`. The indented block runs only when this is `True`.
9. **Line 9:** `cls._instance = super().__new__(cls)`
Returns a proxy object that delegates method calls to a parent class. Used in `__init__` to call the parent constructor.
10. **Line 10:** `return cls._instance`
Returns `cls._instance` to the caller.
11. **Line 11:** `# From AIVS – the addon manager is a singleton`
A comment: *From AIVS – the addon manager is a singleton.*
12. **Line 12:** `_default_manager: AddonManager | None = None`
`_default_manager: AddonManager | None = None`
13. **Line 13:** `_manager_lock = threading.Lock()`
A mutual exclusion lock. Only one thread can hold it at a time. Prevents race conditions.

14. **Line 14:** `def get_addon_manager() -> AddonManager:`
Defines function `get_addon_manager`, returning `AddonManager`.
15. **Line 15:** `global _default_manager`
`global _default_manager`
16. **Line 16:** `if _default_manager is None:`
Checks condition: `_default_manager is None`. The indented block runs only when this is `True`.
17. **Line 17:** `with _manager_lock:`
Context manager: `_manager_lock`. Automatically manages resource cleanup.
18. **Line 18:** `if _default_manager is None:`
Checks condition: `_default_manager is None`. The indented block runs only when this is `True`.
19. **Line 19:** `_default_manager = AddonManager()`
Assigns `AddonManager()` to `_default_manager`.
20. **Line 20:** `return _default_manager`
Returns `_default_manager` to the caller.
-

24.2 Observer Pattern

```
# wxPython events ARE the observer pattern
# Custom events notify the GUI of background changes

# From gui/events.py
_DOWNLOAD_PROGRESS = wx.NewEventType()
EVT_DOWNLOAD_PROGRESS = wx.EventBinder(_DOWNLOAD_PROGRESS, 1)

class DownloadProgressEvent(wx.CommandEvent):
    def __init__(self, name: str, done: int, total: int):
        super().__init__(_DOWNLOAD_PROGRESS)
        self.name = name
        self.done = done
        self.total = total

# Observer (worker thread) posts events:
wx.PostEvent(self, DownloadProgressEvent(name, 50, 100))

# Subscriber (GUI) handles events:
self.Bind(EVT_DOWNLOAD_PROGRESS, self._on_progress)

def _on_progress(self, evt):
    self.gauge.SetValue(evt.done * 100 // evt.total)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# wxPython events ARE the observer pattern`
A comment: *wxPython events ARE the observer pattern.*
2. **Line 2:** `# Custom events notify the GUI of background changes`
A comment: *Custom events notify the GUI of background changes.*
3. **Line 3:** `# From gui/events.py`
A comment: *From gui/events.py.*
4. **Line 4:** `_DOWNLOAD_PROGRESS = wx.NewEventType()`
Assigns `wx.NewEventType()` to `_DOWNLOAD_PROGRESS`.
5. **Line 5:** `EVT_DOWNLOAD_PROGRESS = wx.PyEventBinder(_DOWNLOAD_PROGRESS, 1)`
Assigns `wx.PyEventBinder(_DOWNLOAD_PROGRESS, 1)` to `EVT_DOWNLOAD_PROGRESS`.
6. **Line 6:** `class DownloadProgressEvent(wx.CommandEvent):`
Defines class `DownloadProgressEvent` inheriting from `wx.CommandEvent`.
7. **Line 7:** `def __init__(self, name: str, done: int, total: int):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
8. **Line 8:** `super().__init__(_DOWNLOAD_PROGRESS)`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
9. **Line 9:** `self.name = name`
Stores `name` in instance attribute `name`.
10. **Line 10:** `self.done = done`
Stores `done` in instance attribute `done`.
11. **Line 11:** `self.total = total`
Stores `total` in instance attribute `total`.
12. **Line 12:** `# Observer (worker thread) posts events:`
A comment: *Observer (worker thread) posts events:.*
13. **Line 13:** `wx.PostEvent(self, DownloadProgressEvent(name, 50, 100))`
Creates a wxPython `wx.PostEvent` widget.

14. **Line 14:** `# Subscriber (GUI) handles events:`
A comment: *Subscriber (GUI) handles events:*.
15. **Line 15:** `self.Bind(EVT_DOWNLOAD_PROGRESS, self._on_progress)`
Registers an event handler on the frame/window itself. Used for frame-level events.
16. **Line 16:** `def _on_progress(self, evt):`
Defines function `_on_progress` accepting (self, evt).
17. **Line 17:** `self.gauge.SetValue(evt.done * 100 // evt.total)`
`self.gauge.SetValue(evt.done * 100 // evt.total)`

24.3 Strategy Pattern

```
# Different document parsers for different file types
class Parser(Protocol):
    def parse(self, path: str) -> Document: ...

class PdfParser:
    def parse(self, path): ...

class DocxParser:
    def parse(self, path): ...

# Strategy selection at runtime
parsers = {
    ".pdf": PdfParser(),
    ".docx": DocxParser(),
}

def parse_document(path: str) -> Document:
    ext = os.path.splitext(path)[1].lower()
    parser = parsers.get(ext)
    if parser is None:
        raise ParseError(f"Unsupported: {ext}")
    return parser.parse(path)
```


Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Different document parsers for different file types`

A comment: *Different document parsers for different file types.*

2. **Line 2:** `class Parser(Protocol):`

Defines class `Parser` inheriting from `Protocol`.

3. **Line 3:** `def parse(self, path: str) -> Document: ...`

Defines function `parse` accepting (self, path), returning `Document`.

4. **Line 4:** `class PdfParser:`

Defines class `PdfParser`.

5. **Line 5:** `def parse(self, path): ...`

Defines function `parse` accepting (self, path).

6. **Line 6:** `class DocxParser:`

Defines class `DocxParser`.

7. **Line 7:** `def parse(self, path): ...`

Defines function `parse` accepting (self, path).

8. **Line 8:** `# Strategy selection at runtime`

A comment: *Strategy selection at runtime.*

9. **Line 9:** `parsers = {`

Assigns `{` to `parsers`.

10. **Line 10:** `" .pdf": PdfParser(),`

`" .pdf": PdfParser(),`

11. **Line 11:** `" .docx": DocxParser(),`

`" .docx": DocxParser(),`

12. **Line 12:** `}`

`}`

13. **Line 13:** `def parse_document(path: str) -> Document:`

Defines function `parse_document` accepting (path), returning `Document`.

14. **Line 14:** `ext = os.path.splitext(path)[1].lower()`
Converts all characters to lowercase. Returns a new string.
15. **Line 15:** `parser = parsers.get(ext)`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
16. **Line 16:** `if parser is None:`
Checks condition: `parser is None`. The indented block runs only when this is `True`.
17. **Line 17:** `raise ParseError(f"Unsupported: {ext}")`
Manually raises an exception. Use to signal error conditions to callers.
18. **Line 18:** `return parser.parse(path)`
Returns `parser.parse(path)` to the caller.

24.4 Factory Pattern

```
# From tts/engine.py - factory that creates the right engine
def get_engine(voice_entry: dict) -> TtsEngine:
    """Factory: create the appropriate engine for a voice."""
    engine_type = voice_entry.get("engine", "vits")

    if engine_type == "xtts":
        client = CloneClient()
        return XtTsCloneEngine(voice_entry, client)
    elif engine_type == "omnivoice":
        return OmniVoiceEngine(voice_entry)
    elif engine_type == "omnivoice_server":
        return OmniVoiceServerEngine(voice_entry)
    else:
        return SherpaEngine(voice_entry) # Default: sherpa-onnx
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# From tts/engine.py – factory that creates the right engine`
A comment: *From tts/engine.py – factory that creates the right engine.*
2. **Line 2:** `def get_engine(voice_entry: dict) -> TtsEngine:`
Begins exception handling. Code in this block is monitored for errors.
3. **Line 3:** `"""Factory: create the appropriate engine for a voice."""`
Docstring: *Factory: create the appropriate engine for a voice..* Documents the purpose of the following code.
4. **Line 4:** `engine_type = voice_entry.get("engine", "vits")`
Returns the value for a key, or a default value if the key doesn't exist. Safer than direct access.
5. **Line 5:** `if engine_type == "xtts":`
Checks condition: `engine_type == "xtts"`. The indented block runs only when this is `True`.
6. **Line 6:** `client = CloneClient()`
Assigns `CloneClient()` to `client`.
7. **Line 7:** `return XtTsCloneEngine(voice_entry, client)`
Returns `XtTsCloneEngine(voice_entry, client)` to the caller.
8. **Line 8:** `elif engine_type == "omnivoice":`
Alternative condition: `engine_type == "omnivoice"`. Checked only if all previous conditions were `False`.
9. **Line 9:** `return OmniVoiceEngine(voice_entry)`
Returns `OmniVoiceEngine(voice_entry)` to the caller.
10. **Line 10:** `elif engine_type == "omnivoice_server":`
Alternative condition: `engine_type == "omnivoice_server"`. Checked only if all previous conditions were `False`.
11. **Line 11:** `return OmniVoiceServerEngine(voice_entry)`
Returns `OmniVoiceServerEngine(voice_entry)` to the caller.
12. **Line 12:** `else:`
Fallback: executes when none of the above conditions matched.
13. **Line 13:** `return SherpaEngine(voice_entry) # Default: sherpa-onnx`
Returns `SherpaEngine(voice_entry) # Default: sherpa-onnx` to the caller.

24.5 Key Takeaways

- **Singleton** for shared state (Settings, AddonManager)
 - **Observer** for thread-GUI communication (wx events)
 - **Strategy** for swappable algorithms (document parsers)
 - **Factory** for object creation based on type (TTS engines)
 - Python makes patterns simpler — often a dict or function replaces a full class hierarchy
-

Introspection & Reflection

Learning Objectives

- Examine objects at runtime with `dir()`, `type()`, `getattr()`
 - Use `inspect` module for deep code analysis
 - See introspection patterns in AIVS addon system
-

25.1 Inspecting Objects

```
# dir() - List all attributes
class Settings:
    def __init__(self):
        self.theme = "dark"
        self._data = {}

s = Settings()
print(dir(s)) # Lists all attributes and methods

# type() - get the type
print(type(s)) # <class 'Settings'>

# getattr/setattr - dynamic attribute access
print(getattr(s, "theme")) # "dark"
print(getattr(s, "missing", "default")) # "default"
setattr(s, "volume", 0.8)
print(hasattr(s, "volume")) # True

# inspect module - deep inspection
import inspect

# Get source code
print(inspect.getsource(Settings.__init__))

# Get function signature
sig = inspect.signature(sanitize_filename)
print(sig) # (name: str, max_length: int = 80) -> str

# Get all members of a module
for name, obj in inspect.getmembers(catalog):
    if inspect.isfunction(obj):
        print(f"Function: {name}")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# dir() - list all attributes`
A comment: *dir()* — list all attributes.
2. **Line 2:** `class Settings:`
Defines class `Settings`.
3. **Line 3:** `def __init__(self):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `self.theme = "dark"`
Stores `"dark"` in instance attribute `theme`.
5. **Line 5:** `self._data = {}`
Stores `{}` in instance attribute `_data`.
6. **Line 6:** `s = Settings()`
Assigns `Settings()` to `s`.
7. **Line 7:** `print(dir(s)) # Lists all attributes and methods`
Outputs text to the console. The primary way to display information during development and debugging.
8. **Line 8:** `# type() - get the type`
A comment: *type()* — get the type.
9. **Line 9:** `print(type(s)) # <class 'Settings'>`
Outputs text to the console. The primary way to display information during development and debugging.
10. **Line 10:** `# getattr/setattr - dynamic attribute access`
A comment: *getattr/setattr* — dynamic attribute access.
11. **Line 11:** `print(getattr(s, "theme")) # "dark"`
Gets an attribute value by name. Supports a default value if the attribute doesn't exist.
12. **Line 12:** `print(getattr(s, "missing", "default")) # "default"`
Gets an attribute value by name. Supports a default value if the attribute doesn't exist.
13. **Line 13:** `setattr(s, "volume", 0.8)`
Sets an attribute value by name. Dynamically adds or modifies object attributes.
14. **Line 14:** `print(hasattr(s, "volume")) # True`

Returns True if an object has the named attribute. Useful before calling getattr.

15. **Line 15:** `# inspect module – deep inspection`

A comment: *inspect module – deep inspection.*

16. **Line 16:** `import inspect`

Imports the `inspect` module.

17. **Line 17:** `# Get source code`

A comment: *Get source code.*

18. **Line 18:** `print(inspect.getsource(Settings.__init__))`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

19. **Line 19:** `# Get function signature`

A comment: *Get function signature.*

20. **Line 20:** `sig = inspect.signature(sanitize_filename)`

Assigns `inspect.signature(sanitize_filename)` to `sig`.

21. **Line 21:** `print(sig) # (name: str, max_length: int = 80) -> str`

Outputs text to the console. The primary way to display information during development and debugging.

22. **Line 22:** `# Get all members of a module`

A comment: *Get all members of a module.*

23. **Line 23:** `for name, obj in inspect.getmembers(catalog):`

`for name, obj in inspect.getmembers(catalog):`

24. **Line 24:** `if inspect.isfunction(obj):`

Checks condition: `inspect.isfunction(obj)`. The indented block runs only when this is `True`.

25. **Line 25:** `print(f"Function: {name}")`

Outputs text to the console. The primary way to display information during development and debugging.

25.2 Introspection in the Addon System

```
# From addons/__init__.py - dynamic module loading
import importlib.util

def _register_engine(self, manifest, addon_dir):
    """Import and cache the addon's engine class."""
    module_name, class_name = manifest.engine_entry_point.split(":")
    full_path = os.path.join(addon_dir, module_name + ".py")

    # Dynamic import from file
    spec = importlib.util.spec_from_file_location(
        f"_aivs_engine_{manifest.engine_id}",
        full_path,
    )
    if spec and spec.loader:
        module = importlib.util.module_from_spec(spec)
        sys.modules[f"_aivs_engine_{manifest.engine_id}"] = module
        spec.loader.exec_module(module)

    # Get class by name (introspection)
    engine_class = getattr(module, class_name, None)
    if engine_class:
        self._engines[manifest.engine_id] = {
            "class": engine_class,
            "manifest": manifest,
        }
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From addons/__init__.py – dynamic module loading`
A comment: *From addons/__init__.py – dynamic module loading.*
2. **Line 2:** `import importlib.util`
Imports the `importlib.util` module.
3. **Line 3:** `def _register_engine(self, manifest, addon_dir):`
Defines function `_register_engine` accepting (self, manifest, addon_dir).
4. **Line 4:** `"""Import and cache the addon's engine class."""`
Docstring: *Import and cache the addon's engine class..* Documents the purpose of the following code.
5. **Line 5:** `module_name, class_name = manifest.engine_entry_point.split(":")`
Splits a string into a list by the given delimiter. Without arguments, splits on whitespace.
6. **Line 6:** `full_path = os.path.join(addon_dir, module_name + ".py")`
OS-agnostic path manipulation. Use `join()`, `exists()`, `dirname()`, `basename()`, `splitext()`.
7. **Line 7:** `# Dynamic import from file`
A comment: *Dynamic import from file.*
8. **Line 8:** `spec = importlib.util.spec_from_file_location(`
Assigns `importlib.util.spec_from_file_location(` to `spec`.
9. **Line 9:** `f"_aivs_engine_{manifest.engine_id}",`
`f"_aivs_engine_{manifest.engine_id}",`
10. **Line 10:** `full_path,`
`full_path,`
11. **Line 11:** `)`
`)`
12. **Line 12:** `if spec and spec.loader:`
Checks condition: `spec and spec.loader`. The indented block runs only when this is `True`.
13. **Line 13:** `module = importlib.util.module_from_spec(spec)`
Assigns `importlib.util.module_from_spec(spec)` to `module`.

```
14. Line 14: sys.modules[f"_aivs_engine_{manifest.engine_id}"] = module
    sys.modules[f"_aivs_engine_{manifest.engine_id}"] = module

15. Line 15: spec.loader.exec_module(module)
    spec.loader.exec_module(module)

16. Line 16: # Get class by name (introspection)
    A comment: Get class by name (introspection).

17. Line 17: engine_class = getattr(module, class_name, None)
    Gets an attribute value by name. Supports a default value if the attribute doesn't exist.

18. Line 18: if engine_class:
    Checks condition: engine_class . The indented block runs only when this is True .

19. Line 19: self._engines[manifest.engine_id] = {
    self._engines[manifest.engine_id] = {

20. Line 20: "class": engine_class,
    "class": engine_class,

21. Line 21: "manifest": manifest,
    "manifest": manifest,

22. Line 22: }
    }
```

25.3 Key Takeaways

- `dir()` lists attributes; `getattr()` accesses them dynamically
 - `inspect` module provides deep reflection (source code, signatures, members)
 - The AIVS addon system uses **importlib** for dynamic module loading
 - Introspection is powerful but should be used judiciously
-

C Extensions & ctypes

Learning Objectives

- Understand how Python interfaces with C libraries
 - Use `ctypes` to call shared libraries
 - See how sherpa-onnx is accessed in AIVS
-

26.1 ctypes Basics

```
import ctypes

# Load a shared library
libc = ctypes.CDLL("libc.so.6") # Linux
# libc = ctypes.CDLL("msvcrt.dll") # Windows

# Call C functions
libc.printf(b"Hello from C!\n")

# Type mapping
ctypes.c_int(42)           # C int
ctypes.c_double(3.14)      # C double
ctypes.c_char_p(b"hi")    # C char*
ctypes.c_bool(True)        # C bool
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import ctypes`
Imports the `ctypes` module.
2. **Line 2:** `# Load a shared library`
A comment: *Load a shared library.*
3. **Line 3:** `libc = ctypes.CDLL("libc.so.6") # Linux`
Assigns `ctypes.CDLL("libc.so.6") # Linux` to `libc`.
4. **Line 4:** `# libc = ctypes.CDLL("msvcrt.dll") # Windows`
A comment: *libc = ctypes.CDLL("msvcrt.dll") # Windows.*
5. **Line 5:** `# Call C functions`
A comment: *Call C functions.*
6. **Line 6:** `libc.printf(b"Hello from C!\n")`
`libc.printf(b"Hello from C!\n")`
7. **Line 7:** `# Type mapping`
A comment: *Type mapping.*
8. **Line 8:** `ctypes.c_int(42) # C int`
Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.
9. **Line 9:** `ctypes.c_double(3.14) # C double`
`ctypes.c_double(3.14) # C double`
10. **Line 10:** `ctypes.c_char_p(b"hi") # C char*`
`ctypes.c_char_p(b"hi") # C char*`
11. **Line 11:** `ctypes.c_bool(True) # C bool`
Converts a value to its boolean equivalent. Zero, empty strings, `None`, and empty collections are `False`; everything else is `True`.

26.2 How sherpa-onnx Works

```
# sherpa-onnx is a Python package wrapping C++ ONNX inference
# AIVS accesses it like any Python library:
import sherpa_onnx

# Create TTS config
config = sherpa_onnx.OfflineTtsConfig(
    model=sherpa_onnx.OfflineTtsModelConfig(
        vits=sherpa_onnx.OfflineTtsVitsModelConfig(
            model="model.onnx",
            tokens="tokens.txt",
            data_dir="espeak-ng-data",
        ),
    ),
)

# Create engine and synthesize
tts = sherpa_onnx.OfflineTts(config)
audio = tts.generate(text="Hello world", sid=0, speed=1.0)

# audio.samples is a numpy array
import numpy as np
samples = np.array(audio.samples, dtype=np.int16)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# sherpa-onnx is a Python package wrapping C++ ONNX inference`

A comment: *sherpa-onnx is a Python package wrapping C++ ONNX inference.*

2. **Line 2:** `# AIVS accesses it like any Python library:`

A comment: *AIVS accesses it like any Python library:.*

3. **Line 3:** `import sherpa_onnx`

Imports the `sherpa_onnx` module.

4. **Line 4:** `# Create TTS config`

A comment: *Create TTS config.*

5. **Line 5:** `config = sherpa_onnx.OfflineTtsConfig(`

Assigns `sherpa_onnx.OfflineTtsConfig(` to `config` .

6. **Line 6:** `model=sherpa_onnx.OfflineTtsModelConfig(`

Assigns `sherpa_onnx.OfflineTtsModelConfig(` to `model` .

7. **Line 7:** `vits=sherpa_onnx.OfflineTtsVitsModelConfig(`

Assigns `sherpa_onnx.OfflineTtsVitsModelConfig(` to `vits` .

8. **Line 8:** `model="model.onnx",`

Assigns `"model.onnx",` to `model` .

9. **Line 9:** `tokens="tokens.txt",`

Assigns `"tokens.txt",` to `tokens` .

10. **Line 10:** `data_dir="espeak-ng-data",`

Assigns `"espeak-ng-data",` to `data_dir` .

11. **Line 11:** `),`

`)`,

12. **Line 12:** `),`

`)`,

13. **Line 13:** `)`

`)`

14. **Line 14:** `# Create engine and synthesize`
A comment: *Create engine and synthesize.*
15. **Line 15:** `tts = sherpa_onnx.OfflineTts(config)`
Assigns `sherpa_onnx.OfflineTts(config)` to `tts` .
16. **Line 16:** `audio = tts.generate(text="Hello world", sid=0, speed=1.0)`
Assigns `tts.generate(text="Hello world", sid=0, speed=1.0)` to `audio` .
17. **Line 17:** `# audio.samples is a numpy array`
A comment: *audio.samples is a numpy array.*
18. **Line 18:** `import numpy as np`
Imports the `numpy as np` module.
19. **Line 19:** `samples = np.array(audio.samples, dtype=np.int16)`
Assigns `np.array(audio.samples, dtype=np.int16)` to `samples` .

26.3 NumPy and C

```
# NumPy arrays are stored as C arrays in memory
import numpy as np

arr = np.array([1, 2, 3, 4], dtype=np.int16)

# Access the C pointer
ptr = arr.ctypes.data # Memory address
print(f"Array at {ptr}, {arr.shape} elements of {arr.dtype}")

# This is how audio data flows through the system:
# sherpa-onnx → C++ inference → numpy array → wxPython playback
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# NumPy arrays are stored as C arrays in memory`
A comment: *NumPy arrays are stored as C arrays in memory.*
2. **Line 2:** `import numpy as np`
Imports the `numpy as np` module.
3. **Line 3:** `arr = np.array([1, 2, 3, 4], dtype=np.int16)`
Assigns `np.array([1, 2, 3, 4], dtype=np.int16)` to `arr`.
4. **Line 4:** `# Access the C pointer`
A comment: *Access the C pointer.*
5. **Line 5:** `ptr = arr.ctypes.data # Memory address`
Assigns `arr.ctypes.data # Memory address` to `ptr`.
6. **Line 6:** `print(f"Array at {ptr}, {arr.shape} elements of {arr.dtype}")`
Outputs text to the console. The primary way to display information during development and debugging.
7. **Line 7:** `# This is how audio data flows through the system:`
A comment: *This is how audio data flows through the system:.*
8. **Line 8:** `# sherpa-onnx → C++ inference → numpy array → wxPython playback`
A comment: *sherpa-onnx → C++ inference → numpy array → wxPython playback.*

26.4 Key Takeaways

- **ctypes** enables calling C shared libraries from Python
- **NumPy** provides the bridge between Python and C-level array operations
- AIVS uses **sherpa-onnx** (C++ ONNX inference) and **NumPy** for audio processing
- The audio pipeline: **Text** → **sherpa-onnx (C++)** → **numpy array** → **WAV file**

wxPython GUI Development

CHAPTER 27

wxPython Introduction

Learning Objectives

- Understand wxPython's architecture and widget model
 - Create your first wxPython application
 - Explore the AIVS MainFrame
-

27.1 What Is wxPython?

wxPython is a Python wrapper around the wxWidgets C++ GUI toolkit. It provides **native-looking widgets** on each platform — Windows controls look like Windows controls, macOS controls look like macOS controls, etc.

Feature	wxPython	Tkinter	PyQt
Native look?	Yes (wraps native widgets)	No (custom rendering)	Mostly
License	wxWindows (permissive)	PSF (Python)	GPL/Commercial
Accessibility	Good (MSAA/UIA on Windows)	Basic	Good
Learning curve	Moderate	Easy	Steep

27.2 Hello wxPython

```
import wx

class MyFrame(wx.Frame):
    def __init__(self):
        super().__init__(
            None,
            title="Hello wxPython",
            size=(400, 300),
        )
        panel = wx.Panel(self)
        label = wx.StaticText(panel, label="Hello, World!")
        sizer = wx.BoxSizer(wx.VERTICAL)
        sizer.Add(label, 0, wx.ALL, 20)
        panel.SetSizer(sizer)

# Application entry point
app = wx.App(False)
frame = MyFrame()
frame.Show()
app.MainLoop()
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import wx`
Imports the `wx` module.
2. **Line 2:** `class MyFrame(wx.Frame):`
The top-level window class. `MainFrame` inherits from this. Provides the window frame, title bar, and system menu.
3. **Line 3:** `def __init__(self):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `super().__init__(`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `None,`
`None,`
6. **Line 6:** `title="Hello wxPython",`
Assigns `"Hello wxPython",` to `title`.
7. **Line 7:** `size=(400, 300),`
Assigns `(400, 300),` to `size`.
8. **Line 8:** `)`
`)`
9. **Line 9:** `panel = wx.Panel(self)`
Creates a container window inside a Frame. Groups child controls and manages tab traversal order.
10. **Line 10:** `label = wx.StaticText(panel, label="Hello, World!")`
Creates a non-editable text label.
11. **Line 11:** `sizer = wx.BoxSizer(wx.VERTICAL)`
Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).
12. **Line 12:** `sizer.Add(label, 0, wx.ALL, 20)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
13. **Line 13:** `panel.SetSizer(sizer)`
Assigns a sizer to the panel. The sizer takes control of laying out the panel's children.
14. **Line 14:** `# Application entry point`

A comment: *Application entry point.*

15. **Line 15:** `app = wx.App(False)`

Creates the wxPython application object. Must be created before any GUI elements. False means don't redirect stdout.

16. **Line 16:** `frame = MyFrame()`

Assigns `MyFrame()` to `frame`.

17. **Line 17:** `frame.Show()`

Makes the frame visible on screen. The frame is hidden by default.

18. **Line 18:** `app.MainLoop()`

Starts the wxPython event loop. This blocks until the last window is closed. All GUI events are processed here.

27.3 wx.App and wx.Frame

```
# Every wxPython app needs exactly ONE wx.App
app = wx.App(False) # False = don't redirect stdout

# wx.Frame is the main window
frame = wx.Frame(
    None,          # Parent (None = top-level)
    title="My App",
    size=(800, 600),
    style=wx.DEFAULT_FRAME_STYLE,
)

# Frame properties
frame.SetTitle("New Title")
frame.SetSize((1024, 768))
frame.Centre() # Center on screen

# Menu bar
menubar = wx.MenuBar()
file_menu = wx.Menu()
file_menu.Append(wx.ID_NEW, "&New\tCtrl+N")
file_menu.Append(wx.ID_OPEN, "&Open\tCtrl+O")
file_menu.Append(wx.ID_EXIT, "E&xit")
menubar.Append(file_menu, "&File")
frame.SetMenuBar(menubar)

# Status bar
frame.CreateStatusBar()
frame.SetStatusText("Ready")

frame.Show()
app.MainLoop()
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Every wxPython app needs exactly ONE wx.App`

A comment: *Every wxPython app needs exactly ONE wx.App.*

2. **Line 2:** `app = wx.App(False) # False = don't redirect stdout`

Creates the wxPython application object. Must be created before any GUI elements. False means don't redirect stdout.

3. **Line 3:** `# wx.Frame is the main window`

A comment: *wx.Frame is the main window.*

4. **Line 4:** `frame = wx.Frame(`

Creates a top-level window with title bar, menus, and status bar. The main container for desktop apps.

5. **Line 5:** `None, # Parent (None = top-level)`

`None, # Parent (None = top-level)`

6. **Line 6:** `title="My App",`

Assigns "My App", to title .

7. **Line 7:** `size=(800, 600),`

Assigns (800, 600), to size .

8. **Line 8:** `style=wx.DEFAULT_FRAME_STYLE,`

Assigns wx.DEFAULT_FRAME_STYLE, to style .

9. **Line 9:** `)`

)

10. **Line 10:** `# Frame properties`

A comment: *Frame properties.*

11. **Line 11:** `frame.SetTitle("New Title")`

Changes the frame's title bar text.

12. **Line 12:** `frame.SetSize((1024, 768))`

Sets the frame's size in pixels (width, height).

13. **Line 13:** `frame.Centre() # Center on screen`

Centers the frame on the screen.

14. **Line 14:** `# Menu bar`

A comment: *Menu bar.*

15. **Line 15:** `menubar = wx.MenuBar()`

Creates a horizontal menu bar at the top of a Frame.

16. **Line 16:** `file_menu = wx.Menu()`

Creates a dropdown menu. Contains menu items, separators, and sub-menus.

17. **Line 17:** `file_menu.Append(wx.ID_NEW, "&New\tCtrl+N")`

Adds a menu item with an ID, label, and optional help text. Use \t for accelerators.

18. **Line 18:** `file_menu.Append(wx.ID_OPEN, "&Open\tCtrl+O")`

Adds a menu item with an ID, label, and optional help text. Use \t for accelerators.

19. **Line 19:** `file_menu.Append(wx.ID_EXIT, "E&xit")`

Adds a menu item with an ID, label, and optional help text. Use \t for accelerators.

20. **Line 20:** `menubar.Append(file_menu, "&File")`

Adds a menu to the menu bar with a label.

21. **Line 21:** `frame.SetMenuBar(menubar)`

Attaches a menu bar to the frame.

22. **Line 22:** `# Status bar`

A comment: *Status bar.*

23. **Line 23:** `frame.CreateStatusBar()`

Creates and attaches a status bar to the frame.

24. **Line 24:** `frame.SetStatusText("Ready")`

Sets the text displayed in the status bar.

25. **Line 25:** `frame.Show()`

Makes the frame visible on screen. The frame is hidden by default.

26. **Line 26:** `app.MainLoop()`

Starts the wxPython event loop. This blocks until the last window is closed. All GUI events are processed here.

27.4 The AIVS MainFrame

```
# From gui/main_frame.py
class MainFrame(wx.Frame):
    def __init__(self, parent, settings: Settings, store: ModelStore):
        super().__init__(
            parent,
            title=f"{APP_NAME} v{APP_VERSION}",
            size=(800, 600),
        )
        self.settings = settings
        self.store = store

        self._build_menu()
        self._build_ui()
        self._bind_events()

        self.Centre()
        self.SetStatusText("Ready")

    def _build_menu(self):
        """Build the menu bar with File, Edit, Tools, Settings, Help."""
        menubar = wx.MenuBar()

        # File menu
        file_menu = wx.Menu()
        file_menu.Append(ID_NEW_PROJECT, "&New Project\tCtrl+Shift+N")
        file_menu.Append(ID_OPEN_PROJECT, "&Open Project\tCtrl+O")
        file_menu.AppendSeparator()
        file_menu.Append(wx.ID_EXIT, "E&xit\tAlt+F4")
        menubar.Append(file_menu, "&File")

        # Tools menu
        tools_menu = wx.Menu()
        tools_menu.Append(ID_RECORD, "&Record\tCtrl+Shift+R")
        menubar.Append(tools_menu, "&Tools")

        # Settings menu
        settings_menu = wx.Menu()
        settings_menu.Append(ID_SETTINGS, "&Settings\tCtrl+,")
        menubar.Append(settings_menu, "&Settings")

        self.SetMenuBar(menubar)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From gui/main_frame.py`
A comment: *From gui/main_frame.py*.
2. **Line 2:** `class MainFrame(wx.Frame):`
The top-level window class. MainFrame inherits from this. Provides the window frame, title bar, and system menu.
3. **Line 3:** `def __init__(self, parent, settings: Settings, store: ModelStore):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `super().__init__(`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `parent,`
`parent,`
6. **Line 6:** `title=f"{APP_NAME} v{APP_VERSION}",`
Assigns `f"{APP_NAME} v{APP_VERSION}"` to `title` .
7. **Line 7:** `size=(800, 600),`
Assigns `(800, 600)` to `size` .
8. **Line 8:** `)`
`)`
9. **Line 9:** `self.settings = settings`
Stores `settings` in instance attribute `settings` .
10. **Line 10:** `self.store = store`
Stores `store` in instance attribute `store` .
11. **Line 11:** `self._build_menu()`
Calls `self._build_menu()` .
12. **Line 12:** `self._build_ui()`
Calls `self._build_ui()` .
13. **Line 13:** `self._bind_events()`
Calls `self._bind_events()` .

14. **Line 14:** `self.Centre()`
Calls `self.Centre()`.
15. **Line 15:** `self.SetStatusText("Ready")`
Sets the text in the specified status bar pane.
16. **Line 16:** `def _build_menu(self):`
Defines function `_build_menu` accepting (self).
17. **Line 17:** `"""Build the menu bar with File, Edit, Tools, Settings, Help."""`
Docstring: *Build the menu bar with File, Edit, Tools, Settings, Help..* Documents the purpose of the following code.
18. **Line 18:** `menubar = wx.MenuBar()`
Creates a horizontal menu bar at the top of a Frame.
19. **Line 19:** `# File menu`
A comment: *File menu*.
20. **Line 20:** `file_menu = wx.Menu()`
Creates a dropdown menu. Contains menu items, separators, and sub-menus.
21. **Line 21:** `file_menu.Append(ID_NEW_PROJECT, "&New Project\tCtrl+Shift+N")`
Adds a menu item with an ID, label, and optional help text. Use `\t` for accelerators.
22. **Line 22:** `file_menu.Append(ID_OPEN_PROJECT, "&Open Project\tCtrl+O")`
Adds a menu item with an ID, label, and optional help text. Use `\t` for accelerators.
23. **Line 23:** `file_menu.AppendSeparator()`
Adds a visual separator line between menu items.
24. **Line 24:** `file_menu.Append(wx.ID_EXIT, "E&xit\tAlt+F4")`
Adds a menu item with an ID, label, and optional help text. Use `\t` for accelerators.
25. **Line 25:** `menubar.Append(file_menu, "&File")`
Adds a menu to the menu bar with a label.
26. **Line 26:** `# Tools menu`
A comment: *Tools menu*.
27. **Line 27:** `tools_menu = wx.Menu()`
Creates a dropdown menu. Contains menu items, separators, and sub-menus.
28. **Line 28:** `tools_menu.Append(ID_RECORD, "&Record\tCtrl+Shift+R")`

Adds a menu item with an ID, label, and optional help text. Use \t for accelerators.

29. **Line 29:** `menubar.Append(tools_menu, "&Tools")`

Adds a menu to the menu bar with a label.

30. **Line 30:** `# Settings menu`

A comment: *Settings menu*.

31. **Line 31:** `settings_menu = wx.Menu()`

Creates a dropdown menu. Contains menu items, separators, and sub-menus.

32. **Line 32:** `settings_menu.Append(ID_SETTINGS, "&Settings\tCtrl+,")`

Adds a menu item with an ID, label, and optional help text. Use \t for accelerators.

33. **Line 33:** `menubar.Append(settings_menu, "&Settings")`

Adds a menu to the menu bar with a label.

34. **Line 34:** `self.SetMenuBar(menubar)`

Calls `self.SetMenuBar()` .

27.5 The Widget Hierarchy

```
wx.App                # Application object
└─ wx.Frame           # Window
    └─ wx.MenuBar      # Menu bar
        └─ wx.Menu      # Drop-down menu
            └─ wx.MenuItem # Menu item
    └─ wx.Panel         # Container panel
        └─ wx.StaticText # Label
        └─ wx.TextCtrl   # Text input
        └─ wx.Button     # Button
        └─ wx.ComboBox   # Drop-down combo
        └─ wx.CheckBox   # Checkbox
        └─ wx.RadioButton # Radio button
        └─ wx.Slider     # Slider
        └─ wx.Gauge      # Progress bar
        └─ wx.ListBox    # List box
    └─ wx.StatusBar     # Status bar
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `wx.App` # Application object

`wx.App` # Application object

2. **Line 2:** `wx.Frame` # Window

3. **Line 3:** `wx.MenuBar` # Menu bar

The horizontal menu bar at the top of a Frame. Contains File, Edit, Tools, Help menus.

4. **Line 4:** `wx.Menu` # Drop-down menu

A dropdown menu that appears when clicking a menu bar item or right-clicking.

5. **Line 5:** `wx.MenuItem` # Menu item

A dropdown menu that appears when clicking a menu bar item or right-clicking.

6. **Line 6:** `wx.Panel` # Container panel

A container that sits inside a Frame. Groups controls and provides a background for drawing.

7. **Line 7:** `wx.StaticText` # Label

A non-editable text label widget. The primary way to display text information in wxPython.

8. **Line 8:** `wx.TextCtrl` # Text input

A text input control. Can be single-line, multi-line, read-only, or password-masked.

9. **Line 9:** `wx.Button` # Button

A clickable push button. Clicking it generates an EVT_BUTTON event.

10. **Line 10:** `wx.ComboBox` # Drop-down combo

`wx.ComboBox` # Drop-down combo

11. **Line 11:** `wx.CheckBox` # Checkbox

A toggle control with a label. Returns True when checked, False when unchecked.

12. **Line 12:** `wx.RadioButton` # Radio button

`wx.RadioButton` # Radio button

13. **Line 13:** `wx.Slider` # Slider

A slider control for numeric values. The range is set in the constructor; GetValue() returns the current position.

14. **Line 14:** `wx.Gauge` # Progress bar

A progress bar. SetRange() sets the max value; SetValue() updates the current progress.

15. **Line 15:** `└─ wx.ListBox` `# List box`
A list control for displaying and selecting items. Used for voice selection and recent projects.
16. **Line 16:** `└─ wx.StatusBar` `# Status bar`
A bar at the bottom of the frame showing status messages, progress, and context info.

27.6 Key Takeaways

- **wx.App** manages the event loop; create exactly one
- **wx.Frame** is the main window with menus, status bar
- Widgets are organized in a **parent-child hierarchy**
- AIVS MainFrame has **5 menus** (File, Edit, Tools, Settings, Help)
- wxPython provides **native widgets** with platform-specific appearance

 **Exercise: Exercise 27.1:** Create a wxPython application with a Frame, a menu with "File → Exit", and a status bar showing "Ready".

Sizers & Layout

Learning Objectives

- Understand wxPython's layout system (sizers)
 - Use BoxSizer, FlexGridSizer, and GridSizer
 - See layout patterns in AIVS dialogs
-

28.1 Why Sizers?

wxPython does not use absolute positioning. Instead, **sizers** automatically arrange widgets, handling resizing, padding, and alignment.

28.2 BoxSizer — The Most Common

```
# Vertical layout (widgets stack top to bottom)
sizer = wx.BoxSizer(wx.VERTICAL)
sizer.Add(label, 0, wx.ALL, 10)      # 0 = don't expand
sizer.Add(text_ctrl, 1, wx.EXPAND | wx.ALL, 5) # 1 = expand proportionally
sizer.Add(button, 0, wx.ALIGN_RIGHT | wx.ALL, 5)

# Horizontal layout (widgets go left to right)
hbox = wx.BoxSizer(wx.HORIZONTAL)
hbox.Add(start_btn, 0, wx.ALL, 4)
hbox.Add(pause_btn, 0, wx.ALL, 4)
hbox.Add(stop_btn, 0, wx.ALL, 4)

# Add with proportion, flags, and border
sizer.Add(
    widget,
    proportion=1,      # Relative size (0=fixed, 1=shares space)
    flag=wx.EXPAND | wx.ALL, # Behavior
    border=10,         # Border width in pixels
)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Vertical layout (widgets stack top to bottom)`
A comment: *Vertical layout (widgets stack top to bottom).*
2. **Line 2:** `sizer = wx.BoxSizer(wx.VERTICAL)`
Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).
3. **Line 3:** `sizer.Add(label, 0, wx.ALL, 10) # 0 = don't expand`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
4. **Line 4:** `sizer.Add(text_ctrl, 1, wx.EXPAND | wx.ALL, 5) # 1 = expand proportionally`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
5. **Line 5:** `sizer.Add(button, 0, wx.ALIGN_RIGHT | wx.ALL, 5)`
Flag: aligns the widget to the right of the sizer cell.
6. **Line 6:** `# Horizontal layout (widgets go left to right)`
A comment: *Horizontal layout (widgets go left to right).*
7. **Line 7:** `hbox = wx.BoxSizer(wx.HORIZONTAL)`
Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).
8. **Line 8:** `hbox.Add(start_btn, 0, wx.ALL, 4)`
Border flag: adds a border on all four sides of the widget.
9. **Line 9:** `hbox.Add(pause_btn, 0, wx.ALL, 4)`
Border flag: adds a border on all four sides of the widget.
10. **Line 10:** `hbox.Add(stop_btn, 0, wx.ALL, 4)`
Border flag: adds a border on all four sides of the widget.
11. **Line 11:** `# Add with proportion, flags, and border`
A comment: *Add with proportion, flags, and border.*
12. **Line 12:** `sizer.Add(`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
13. **Line 13:** `widget,`
`widget,`
14. **Line 14:** `proportion=1, # Relative size (0=fixed, 1=shares space)`
Assigns `1,` `# Relative size (0=fixed, 1=shares space)` to `proportion`.

15. **Line 15:** `flag=wx.EXPAND | wx.ALL, # Behavior`
Flag: the widget stretches to fill the sizer's available space.
16. **Line 16:** `border=10, # Border width in pixels`
Assigns `10,` # Border width in pixels to `border` .
17. **Line 17:** `)`
`)`

Sizer Flags

Flag	Meaning
<code>wx.EXPAND</code>	Widget fills available space
<code>wx.ALL</code>	Border on all sides
<code>wx.LEFT/wx.RIGHT/wx.TOP/wx.BOTTOM</code>	Border on specific sides
<code>wx.ALIGN_CENTER</code>	Center in available space
<code>wx.ALIGN_RIGHT</code>	Align to right

28.3 FlexGridSizer — For Form Layouts

```
# From AIVS settings - Label-control grid layout
grid = wx.FlexGridSizer(cols=2, vgap=6, hgap=8)
grid.AddGrowableCol(1) # Column 1 (control) stretches

# Label + control pairs
grid.Add(
    wx.StaticText(self, label="TTS engine:"),
    0, wx.ALIGN_CENTER_VERTICAL | wx.LEFT | wx.RIGHT, 2,
)
grid.Add(
    wx.ComboBox(self, style=wx.CB_READONLY),
    1, wx.EXPAND | wx.LEFT | wx.RIGHT, 2,
)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From AIVS settings – label-control grid layout`

A comment: *From AIVS settings – label-control grid layout.*

2. **Line 2:** `grid = wx.FlexGridSizer(cols=2, vgap=6, hgap=8)`

Like GridSizer but rows/columns can have different sizes.

3. **Line 3:** `grid.AddGrowableCol(1) # Column 1 (control) stretches`

`grid.AddGrowableCol(1) # Column 1 (control) stretches`

4. **Line 4:** `# Label + control pairs`

A comment: *Label + control pairs.*

5. **Line 5:** `grid.Add(`

`grid.Add(`

6. **Line 6:** `wx.StaticText(self, label="TTS engine:"),`

Creates a non-editable text label.

7. **Line 7:** `0, wx.ALIGN_CENTER_VERTICAL | wx.LEFT | wx.RIGHT, 2,`

Flag: centers the widget vertically.

8. **Line 8:** `)`

`)`

9. **Line 9:** `grid.Add(`

`grid.Add(`

10. **Line 10:** `wx.ComboBox(self, style=wx.CB_READONLY),`

ComboBox style: prevents typing custom values.

11. **Line 11:** `1, wx.EXPAND | wx.LEFT | wx.RIGHT, 2,`

Flag: the widget stretches to fill the sizer's available space.

12. **Line 12:** `)`

`)`

28.4 Layout in AIVS Recording Dialog

```
# From recording_dialog.py - nested sizer layout
def _build_ui(self):
    sizer = wx.BoxSizer(wx.VERTICAL)

    # Title
    title = wx.StaticText(self, label=f"Project: {title_text}")
    sizer.Add(title, 0, wx.ALL, 6)

    # Text preview
    self.text_preview = wx.TextCtrl(
        self, style=wx.TE_MULTILINE | wx.TE_READONLY, size=(-1, 130),
    )
    sizer.Add(self.text_preview, 0, wx.EXPAND | wx.LEFT | wx.RIGHT, 6)

    # Model selectors (grid layout)
    grid = wx.FlexGridSizer(cols=2, vgap=6, hgap=8)
    grid.AddGrowbleCol(1)
    add_labeled(self, grid, "TTS engine", self.tts_combo)
    add_labeled(self, grid, "Language", self.lang_combo)
    add_labeled(self, grid, "Voice", self.voice_combo)
    sizer.Add(grid, 0, wx.EXPAND | wx.ALL, 6)

    # Buttons (horizontal)
    btns = wx.BoxSizer(wx.HORIZONTAL)
    btns.Add(self.start_btn, 0, wx.ALL, 4)
    btns.Add(self.pause_btn, 0, wx.ALL, 4)
    btns.Add(self.stop_btn, 0, wx.ALL, 4)
    sizer.Add(btns, 0, wx.LEFT, 2)

    # Progress bar (expands horizontally)
    self.gauge = wx.Gauge(self, range=100, size=(-1, 22))
    sizer.Add(self.gauge, 0, wx.EXPAND | wx.ALL, 6)

    self.SetSizer(sizer)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From recording_dialog.py – nested sizer layout`
A comment: *From recording_dialog.py – nested sizer layout.*
2. **Line 2:** `def _build_ui(self):`
Defines function `_build_ui` accepting (self).
3. **Line 3:** `sizer = wx.BoxSizer(wx.VERTICAL)`
Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).
4. **Line 4:** `# Title`
A comment: *Title.*
5. **Line 5:** `title = wx.StaticText(self, label=f"Project: {title_text}")`
Creates a non-editable text label.
6. **Line 6:** `sizer.Add(title, 0, wx.ALL, 6)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
7. **Line 7:** `# Text preview`
A comment: *Text preview.*
8. **Line 8:** `self.text_preview = wx.TextCtrl(`
Creates a text input control. Can be single-line (for paths) or multi-line (for text display).
9. **Line 9:** `self, style=wx.TE_MULTILINE | wx.TE_READONLY, size=(-1, 130),`
TextCtrl style: enables multi-line text input.
10. **Line 10:** `)`
`)`
11. **Line 11:** `sizer.Add(self.text_preview, 0, wx.EXPAND | wx.LEFT | wx.RIGHT, 6)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
12. **Line 12:** `# Model selectors (grid layout)`
A comment: *Model selectors (grid layout).*
13. **Line 13:** `grid = wx.FlexGridSizer(cols=2, vgap=6, hgap=8)`
Like GridSizer but rows/columns can have different sizes.
14. **Line 14:** `grid.AddGrowbleCol(1)`

```
grid.AddGrowableCol(1)
```

15. **Line 15:** `add_labeled(self, grid, "TTS engine", self.tts_combo)`

Calls function `add_labeled()` .

16. **Line 16:** `add_labeled(self, grid, "Language", self.lang_combo)`

Calls function `add_labeled()` .

17. **Line 17:** `add_labeled(self, grid, "Voice", self.voice_combo)`

Calls function `add_labeled()` .

18. **Line 18:** `sizer.Add(grid, 0, wx.EXPAND | wx.ALL, 6)`

Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.

19. **Line 19:** `# Buttons (horizontal)`

A comment: *Buttons (horizontal)*.

20. **Line 20:** `btns = wx.BoxSizer(wx.HORIZONTAL)`

Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).

21. **Line 21:** `btns.Add(self.start_btn, 0, wx.ALL, 4)`

Border flag: adds a border on all four sides of the widget.

22. **Line 22:** `btns.Add(self.pause_btn, 0, wx.ALL, 4)`

Border flag: adds a border on all four sides of the widget.

23. **Line 23:** `btns.Add(self.stop_btn, 0, wx.ALL, 4)`

Border flag: adds a border on all four sides of the widget.

24. **Line 24:** `sizer.Add(btns, 0, wx.LEFT, 2)`

Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.

25. **Line 25:** `# Progress bar (expands horizontally)`

A comment: *Progress bar (expands horizontally)*.

26. **Line 26:** `self.gauge = wx.Gauge(self, range=100, size=(-1, 22))`

Creates a progress bar control. Shows download and synthesis progress.

27. **Line 27:** `sizer.Add(self.gauge, 0, wx.EXPAND | wx.ALL, 6)`

Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.

28. **Line 28:** `self.SetSizer(sizer)`

Calls `self.SetSizer()` .

28.5 Key Takeaways

- **BoxSizer** for linear layouts (vertical/horizontal)
 - **FlexGridSizer** for form-style layouts (label + control pairs)
 - **Proportion** controls how space is shared (0 = fixed, 1+ = proportional)
 - **Flags** control alignment, expansion, and borders
 - Nest sizers for complex layouts
-

Controls & Widgets

Learning Objectives

- Use common wxPython controls: TextCtrl, ComboBox, Slider, Gauge
 - Handle control events
 - See control patterns in AIVS
-

29.1 Essential Controls

```
# StaticText – read-only Label
label = wx.StaticText(parent, label="TTS Engine:")

# TextCtrl – text input
text = wx.TextCtrl(parent, value="Default text")
text = wx.TextCtrl(parent, style=wx.TE_MULTILINE | wx.TE_READONLY)

# Button
btn = wx.Button(parent, label="Start Recording")
btn.Disable() # Gray out
btn.Enable()  # Re-enable

# ComboBox – dropdown selection
combo = wx.ComboBox(parent, style=wx.CB_READONLY)
combo.Append("Option 1", "value1") # Label, client data
combo.Append("Option 2", "value2")
combo.SetSelection(0)
value = combo.GetClientData(combo.GetSelection()) # "value1"

# CheckBox
cb = wx.CheckBox(parent, label="Developer mode")
cb.SetValue(True)

# RadioButton
rb1 = wx.RadioButton(parent, label="Light", style=wx.RB_GROUP)
rb2 = wx.RadioButton(parent, label="Dark")

# Slider
slider = wx.Slider(parent, minValue=50, maxValue=200, value=100)
value = slider.GetValue() / 100.0 # 1.00

# Gauge (progress bar)
gauge = wx.Gauge(parent, range=100)
gauge.SetValue(50) # 50%
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# StaticText – read-only label`
A comment: *StaticText* – read-only label.
2. **Line 2:** `label = wx.StaticText(parent, label="TTS Engine:")`
Creates a non-editable text label.
3. **Line 3:** `# TextCtrl – text input`
A comment: *TextCtrl* – text input.
4. **Line 4:** `text = wx.TextCtrl(parent, value="Default text")`
Creates a text input control. Can be single-line (for paths) or multi-line (for text display).
5. **Line 5:** `text = wx.TextCtrl(parent, style=wx.TE_MULTILINE | wx.TE_READONLY)`
TextCtrl style: enables multi-line text input.
6. **Line 6:** `# Button`
A comment: *Button*.
7. **Line 7:** `btn = wx.Button(parent, label="Start Recording")`
Creates a clickable push button. The most common way to trigger user actions.
8. **Line 8:** `btn.Disable() # Gray out`
`btn.Disable() # Gray out`
9. **Line 9:** `btn.Enable() # Re-enable`
`btn.Enable() # Re-enable`
10. **Line 10:** `# ComboBox – dropdown selection`
A comment: *ComboBox* – dropdown selection.
11. **Line 11:** `combo = wx.ComboBox(parent, style=wx.CB_READONLY)`
ComboBox style: prevents typing custom values.
12. **Line 12:** `combo.Append("Option 1", "value1") # label, client data`
`combo.Append("Option 1", "value1") # label, client data`
13. **Line 13:** `combo.Append("Option 2", "value2")`
`combo.Append("Option 2", "value2")`
14. **Line 14:** `combo.SetSelection(0)`

```
combo.SetSelection(0)
```

15. **Line 15:** `value = combo.GetClientData(combo.GetSelection()) # "value1"`
Assigns `combo.GetClientData(combo.GetSelection()) # "value1"` to `value`.

16. **Line 16:** `# CheckBox`
A comment: *CheckBox*.

17. **Line 17:** `cb = wx.CheckBox(parent, label="Developer mode")`
Creates a check box toggle. Returns True when checked, False when unchecked.

18. **Line 18:** `cb.SetValue(True)`
`cb.SetValue(True)`

19. **Line 19:** `# RadioButton`
A comment: *RadioButton*.

20. **Line 20:** `rb1 = wx.RadioButton(parent, label="Light", style=wx.RB_GROUP)`
Assigns `wx.RadioButton(parent, label="Light", style=wx.RB_GROUP)` to `rb1`.

21. **Line 21:** `rb2 = wx.RadioButton(parent, label="Dark")`
Assigns `wx.RadioButton(parent, label="Dark")` to `rb2`.

22. **Line 22:** `# Slider`
A comment: *Slider*.

23. **Line 23:** `slider = wx.Slider(parent, minValue=50, maxValue=200, value=100)`
Creates a slider control for selecting a value within a range. Used for rate, pitch, volume.

24. **Line 24:** `value = slider.GetValue() / 100.0 # 1.00`
Assigns `slider.GetValue() / 100.0 # 1.00` to `value`.

25. **Line 25:** `# Gauge (progress bar)`
A comment: *Gauge (progress bar)*.

26. **Line 26:** `gauge = wx.Gauge(parent, range=100)`
Creates a progress bar control. Shows download and synthesis progress.

27. **Line 27:** `gauge.SetValue(50) # 50%`
`gauge.SetValue(50) # 50%`

29.2 The add_labeled Helper (AIVS)

```
# From gui/ally.py – reusable helper for label+control pairs
def add_labeled(parent, sizer, label_text, control, **kwargs):
    """Add a label-control pair to a sizer."""
    label = wx.StaticText(parent, label=label_text + ":")
    control.SetName(label_text) # Accessibility name
    sizer.Add(label, 0, wx.ALIGN_CENTER_VERTICAL | wx.ALL, 4)
    sizer.Add(control, 1, wx.EXPAND | wx.ALL, 4)
    return label

# Usage
add_labeled(self, grid, "TTS engine", self.tts_combo)
add_labeled(self, grid, "Language", self.lang_combo)
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From gui/a11y.py – reusable helper for label+control pairs`
A comment: *From gui/a11y.py – reusable helper for label+control pairs.*
2. **Line 2:** `def add_labeled(parent, sizer, label_text, control, **kwargs):`
Defines function `add_labeled` accepting (parent, sizer, label_text, control, ...).
3. **Line 3:** `"""Add a label-control pair to a sizer."""`
Docstring: *Add a label-control pair to a sizer..* Documents the purpose of the following code.
4. **Line 4:** `label = wx.StaticText(parent, label=label_text + ":")`
Creates a non-editable text label.
5. **Line 5:** `control.SetName(label_text) # Accessibility name`
`control.SetName(label_text) # Accessibility name`
6. **Line 6:** `sizer.Add(label, 0, wx.ALIGN_CENTER_VERTICAL | wx.ALL, 4)`
Flag: centers the widget vertically.
7. **Line 7:** `sizer.Add(control, 1, wx.EXPAND | wx.ALL, 4)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
8. **Line 8:** `return label`
Returns `label` to the caller.
9. **Line 9:** `# Usage`
A comment: *Usage.*
10. **Line 10:** `add_labeled(self, grid, "TTS engine", self.tts_combo)`
Calls function `add_labeled()` .
11. **Line 11:** `add_labeled(self, grid, "Language", self.lang_combo)`
Calls function `add_labeled()` .

29.3 Slider with Value Display

```
# From recording_dialog.py - slider with live value label
def _slider(self, grid, name: str, lo: float, hi: float) -> wx.Slider:
    label = wx.StaticText(self, label=name + ":")
    slider = wx.Slider(
        self,
        minValue=int(lo * 100),
        maxValue=int(hi * 100),
        value=int(1.0 * 100),
    )
    value_label = wx.StaticText(self, label="1.00")
    slider.value_label = value_label # Attach label to slider

    # Update label when slider moves
    slider.Bind(
        wx.EVT_SLIDER,
        lambda evt, s=slider, vl=value_label, n=name: (
            vl.SetLabel(f"{s.GetValue() / 100.0:.2f}"),
            s.SetName(f"{n}: {s.GetValue() / 100.0:.2f}"),
        ),
    )

    box = wx.BoxSizer(wx.HORIZONTAL)
    box.Add(slider, 1, wx.EXPAND)
    box.Add(value_label, 0, wx.LEFT | wx.ALIGN_CENTER_VERTICAL, 6)
    grid.Add(label, 0, wx.ALIGN_CENTER_VERTICAL)
    grid.Add(box, 1, wx.EXPAND)
    return slider
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From recording_dialog.py – slider with live value label`
A comment: *From recording_dialog.py – slider with live value label.*
2. **Line 2:** `def _slider(self, grid, name: str, lo: float, hi: float) -> wx.Slider:`
A slider control for numeric values. The range is set in the constructor; GetValue() returns the current position.
3. **Line 3:** `label = wx.StaticText(self, label=name + ":")`
Creates a non-editable text label.
4. **Line 4:** `slider = wx.Slider(`
Creates a slider control for selecting a value within a range. Used for rate, pitch, volume.
5. **Line 5:** `self,`
`self,`
6. **Line 6:** `minValue=int(lo * 100),`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
7. **Line 7:** `maxValue=int(hi * 100),`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
8. **Line 8:** `value=int(1.0 * 100),`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
9. **Line 9:** `)`
`)`
10. **Line 10:** `value_label = wx.StaticText(self, label="1.00")`
Creates a non-editable text label.
11. **Line 11:** `slider.value_label = value_label # Attach label to slider`
Assigns `value_label # Attach label to slider` to `slider.value_label`.
12. **Line 12:** `# Update label when slider moves`
A comment: *Update label when slider moves.*
13. **Line 13:** `slider.Bind(`
`slider.Bind(`
14. **Line 14:** `wx.EVT_SLIDER,`

Event triggered when a Slider value changes.

15. **Line 15:** `lambda evt, s=slider, vl=value_label, n=name: (`
`lambda evt, s=slider, vl=value_label, n=name: (`

16. **Line 16:** `vl.SetLabel(f"{s.GetValue() / 100.0:.2f}"),`
`vl.SetLabel(f"{s.GetValue() / 100.0:.2f}"),`

17. **Line 17:** `s.SetName(f"{n}: {s.GetValue() / 100.0:.2f}"),`
`s.SetName(f"{n}: {s.GetValue() / 100.0:.2f}"),`

18. **Line 18:** `),`
`),`

19. **Line 19:** `)`
`)`

20. **Line 20:** `box = wx.BoxSizer(wx.HORIZONTAL)`

Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).

21. **Line 21:** `box.Add(slider, 1, wx.EXPAND)`

Flag: the widget stretches to fill the sizer's available space.

22. **Line 22:** `box.Add(value_label, 0, wx.LEFT | wx.ALIGN_CENTER_VERTICAL, 6)`

Flag: centers the widget vertically.

23. **Line 23:** `grid.Add(label, 0, wx.ALIGN_CENTER_VERTICAL)`

Flag: centers the widget vertically.

24. **Line 24:** `grid.Add(box, 1, wx.EXPAND)`

Flag: the widget stretches to fill the sizer's available space.

25. **Line 25:** `return slider`

Returns `slider` to the caller.

29.4 Key Takeaways

- wxPython provides **native controls** for every common UI pattern
- **ComboBox** with CB_READONLY is the preferred dropdown (stores client data)

- **Slider** values are integers — divide by 100 for float parameters
 - AIVS uses the **add_labeled** helper for consistent label+control pairs
-

[← Ch 28](#) | [Ch 30 — Events](#) →

Event Handling

Learning Objectives

- Understand wxPython's event system (Bind/PostEvent)
 - Create custom event types for worker-GUI communication
 - See event patterns in AIVS
-

30.1 Basic Event Binding

```
# Bind a button click
self.start_btn.Bind(wx.EVT_BUTTON, self._on_start)

# Bind with Lambda
self.close_btn.Bind(wx.EVT_BUTTON, lambda _: self.Close())

# Bind combo box selection
self.tts_combo.Bind(wx.EVT_COMBOBOX, self._on_tts_changed)

# Bind window close
self.Bind(wx.EVT_CLOSE, self._on_close)

# Bind slider change
self.slider.Bind(wx.EVT_SLIDER, self._on_slider_change)

# Common event types
wx.EVT_BUTTON      # Button clicked
wx.EVT_COMBOBOX    # ComboBox selection
wx.EVT_TEXT        # TextCtrl changed
wx.EVT_CHECKBOX    # CheckBox toggled
wx.EVT_CLOSE       # Window close requested
wx.EVT_SLIDER      # Slider moved
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Bind a button click`
A comment: *Bind a button click.*
2. **Line 2:** `self.start_btn.Bind(wx.EVT_BUTTON, self._on_start)`
Event triggered when a button is clicked. Bind with: `widget.Bind(wx.EVT_BUTTON, handler)`.
3. **Line 3:** `# Bind with lambda`
A comment: *Bind with lambda.*
4. **Line 4:** `self.close_btn.Bind(wx.EVT_BUTTON, lambda _: self.Close())`
Event triggered when a button is clicked. Bind with: `widget.Bind(wx.EVT_BUTTON, handler)`.
5. **Line 5:** `# Bind combo box selection`
A comment: *Bind combo box selection.*
6. **Line 6:** `self.tts_combo.Bind(wx.EVT_COMBOBOX, self._on_tts_changed)`
Event triggered when a ComboBox selection changes.
7. **Line 7:** `# Bind window close`
A comment: *Bind window close.*
8. **Line 8:** `self.Bind(wx.EVT_CLOSE, self._on_close)`
Event triggered when the window close button (X) is clicked.
9. **Line 9:** `# Bind slider change`
A comment: *Bind slider change.*
10. **Line 10:** `self.slider.Bind(wx.EVT_SLIDER, self._on_slider_change)`
Event triggered when a Slider value changes.
11. **Line 11:** `# Common event types`
A comment: *Common event types.*
12. **Line 12:** `wx.EVT_BUTTON` `# Button clicked`
Event triggered when a button is clicked. Bind with: `widget.Bind(wx.EVT_BUTTON, handler)`.
13. **Line 13:** `wx.EVT_COMBOBOX` `# ComboBox selection`
Event triggered when a ComboBox selection changes.
14. **Line 14:** `wx.EVT_TEXT` `# TextCtrl changed`
Event triggered when text in a TextCtrl changes.

15. **Line 15:** `wx.EVT_CHECKBOX` # CheckBox toggled
Event triggered when a CheckBox is toggled.
16. **Line 16:** `wx.EVT_CLOSE` # Window close requested
Event triggered when the window close button (X) is clicked.
17. **Line 17:** `wx.EVT_SLIDER` # Slider moved
Event triggered when a Slider value changes.

30.2 Custom Events (Worker → GUI)

```
# From gui/events.py
# Step 1: Create a unique event type
_SYNTH_STATUS = wx.NewEventType()
EVT_SYNTH_STATUS = wx.PyEventBinder(_SYNTH_STATUS, 1)

# Step 2: Create an event class
class SynthStatusEvent(wx.CommandEvent):
    def __init__(self, message: str):
        super().__init__(_SYNTH_STATUS)
        self.message = message

# Step 3: Post from worker thread
def _on_status(self, msg):
    wx.PostEvent(self, SynthStatusEvent(msg))

# Step 4: Handle in GUI (main thread)
self.Bind(EVT_SYNTH_STATUS, self._handle_status)

def _handle_status(self, evt: SynthStatusEvent):
    self.status_label.SetLabel(evt.message)
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From gui/events.py`
A comment: *From gui/events.py.*
2. **Line 2:** `# Step 1: Create a unique event type`
A comment: *Step 1: Create a unique event type.*
3. **Line 3:** `_SYNTH_STATUS = wx.NewEventType()`
Assigns `wx.NewEventType()` to `_SYNTH_STATUS`.
4. **Line 4:** `EVT_SYNTH_STATUS = wx.PyEventBinder(_SYNTH_STATUS, 1)`
Assigns `wx.PyEventBinder(_SYNTH_STATUS, 1)` to `EVT_SYNTH_STATUS`.
5. **Line 5:** `# Step 2: Create an event class`
A comment: *Step 2: Create an event class.*
6. **Line 6:** `class SynthStatusEvent(wx.PyCommandEvent):`
Defines class `SynthStatusEvent` inheriting from `wx.PyCommandEvent`.
7. **Line 7:** `def __init__(self, message: str):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
8. **Line 8:** `super().__init__(_SYNTH_STATUS)`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
9. **Line 9:** `self.message = message`
Stores `message` in instance attribute `message`.
10. **Line 10:** `# Step 3: Post from worker thread`
A comment: *Step 3: Post from worker thread.*
11. **Line 11:** `def _on_status(self, msg):`
Defines function `_on_status` accepting (self, msg).
12. **Line 12:** `wx.PostEvent(self, SynthStatusEvent(msg))`
Creates a wxPython `wx.PostEvent` widget.
13. **Line 13:** `# Step 4: Handle in GUI (main thread)`
A comment: *Step 4: Handle in GUI (main thread).*
14. **Line 14:** `self.Bind(EVT_SYNTH_STATUS, self._handle_status)`

Registers an event handler on the frame/window itself. Used for frame-level events.

15. **Line 15:** `def _handle_status(self, evt: SynthStatusEvent):`

Defines function `_handle_status` accepting (self, evt).

16. **Line 16:** `self.status_label.SetLabel(evt.message)`

`self.status_label.SetLabel(evt.message)`

Complete Event System in AIVS

Event	Purpose	Posted From
<code>SynthStatusEvent</code>	Status update message	SynthesisWorker
<code>SynthSegmentDoneEvent</code>	Segment recording complete	SynthesisWorker
<code>SynthFinishedEvent</code>	All recording done	SynthesisWorker
<code>SynthErrorEvent</code>	Recording error	SynthesisWorker
<code>DownloadProgressEvent</code>	Download progress update	Download thread
<code>DownloadFinishedEvent</code>	Download complete/failed	Download thread

30.3 Event Propagation

```
# Events propagate up the widget hierarchy
# Use evt.Skip() to let parent handle the event too

def _on_close(self, evt):
    if self._worker and self._worker.is_alive():
        answer = wx.MessageBox(
            "Recording is running. Stop and close?",
            "Close",
            style=wx.YES_NO | wx.ICON_QUESTION,
        )
        if answer != wx.YES:
            return # Don't close - consume the event
        self._worker.cancel()
    self._save_tts_to_project()
    self.EndModal(wx.ID_CLOSE)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Events propagate up the widget hierarchy`
A comment: *Events propagate up the widget hierarchy.*
2. **Line 2:** `# Use evt.Skip() to let parent handle the event too`
A comment: *Use evt.Skip() to let parent handle the event too.*
3. **Line 3:** `def _on_close(self, evt):`
Defines function `_on_close` accepting (self, evt).
4. **Line 4:** `if self._worker and self._worker.is_alive():`
Checks condition: `self._worker and self._worker.is_alive()`. The indented block runs only when this is `True`.
5. **Line 5:** `answer = wx.MessageBox(`
Assigns `wx.MessageBox(` to `answer`.
6. **Line 6:** `"Recording is running. Stop and close?",`
`"Recording is running. Stop and close?",`
7. **Line 7:** `"Close",`
`"Close",`
8. **Line 8:** `style=wx.YES_NO | wx.ICON_QUESTION,`
Assigns `wx.YES_NO | wx.ICON_QUESTION,` to `style`.
9. **Line 9:** `)`
`)`
10. **Line 10:** `if answer != wx.YES:`
Checks condition: `answer != wx.YES`. The indented block runs only when this is `True`.
11. **Line 11:** `return # Don't close – consume the event`
Returns `# Don't close – consume the event` to the caller.
12. **Line 12:** `self._worker.cancel()`
`self._worker.cancel()`
13. **Line 13:** `self._save_tts_to_project()`
Calls `self._save_tts_to_project()`.

14. **Line 14:** `self.EndModal(wx.ID_CLOSE)`
Standard ID for Close button.

30.4 Key Takeaways

- **Bind()** connects events to handler methods
 - **wx.PostEvent()** is the only safe way to send events from worker threads
 - Custom events use **NewEventType()** + **PyEventBinder()**
 - AIVS has **6 custom event types** for synthesis and download communication
 - **evt.Skip()** allows event propagation to parent widgets
-

Dialogs & Wizards

Learning Objectives

- Create modal and modeless dialogs
 - Build multi-step wizards
 - See the AIVS NewProjectWizard and SettingsDialog
-

31.1 Modal Dialogs

```
# wx.Dialog - blocks the parent until closed
class MyDialog(wx.Dialog):
    def __init__(self, parent):
        super().__init__(parent, title="Settings", size=(600, 400))
        panel = wx.Panel(self)
        sizer = wx.BoxSizer(wx.VERTICAL)
        sizer.Add(wx.StaticText(panel, label="Configure settings here"))
        ok_btn = wx.Button(panel, wx.ID_OK, label="Save")
        cancel_btn = wx.Button(panel, wx.ID_CANCEL, label="Cancel")
        btn_sizer = wx.BoxSizer(wx.HORIZONTAL)
        btn_sizer.Add(ok_btn, 0, wx.ALL, 5)
        btn_sizer.Add(cancel_btn, 0, wx.ALL, 5)
        sizer.Add(btn_sizer)
        panel.SetSizer(sizer)

# Show modal dialog
dlg = MyDialog(self)
result = dlg.ShowModal() # Blocks until closed
if result == wx.ID_OK:
    # User clicked Save
    pass
dlg.Destroy() # Always destroy after use
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# wx.Dialog – blocks the parent until closed`
A comment: *wx.Dialog – blocks the parent until closed.*
2. **Line 2:** `class MyDialog(wx.Dialog):`
A window that typically requires user interaction before continuing. Can be modal (blocks parent) or modeless.
3. **Line 3:** `def __init__(self, parent):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `super().__init__(parent, title="Settings", size=(600, 400))`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `panel = wx.Panel(self)`
Creates a container window inside a Frame. Groups child controls and manages tab traversal order.
6. **Line 6:** `sizer = wx.BoxSizer(wx.VERTICAL)`
Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).
7. **Line 7:** `sizer.Add(wx.StaticText(panel, label="Configure settings here"))`
Creates a non-editable text label.
8. **Line 8:** `ok_btn = wx.Button(panel, wx.ID_OK, label="Save")`
Creates a clickable push button. The most common way to trigger user actions.
9. **Line 9:** `cancel_btn = wx.Button(panel, wx.ID_CANCEL, label="Cancel")`
Standard ID for Cancel button in dialogs.
10. **Line 10:** `btn_sizer = wx.BoxSizer(wx.HORIZONTAL)`
Creates a layout manager that arranges children horizontally (HORIZONTAL) or vertically (VERTICAL).
11. **Line 11:** `btn_sizer.Add(ok_btn, 0, wx.ALL, 5)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
12. **Line 12:** `btn_sizer.Add(cancel_btn, 0, wx.ALL, 5)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
13. **Line 13:** `sizer.Add(btn_sizer)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
14. **Line 14:** `panel.SetSizer(sizer)`
Assigns a sizer to the panel. The sizer takes control of laying out the panel's children.

15. **Line 15:** `# Show modal dialog`
A comment: *Show modal dialog.*
16. **Line 16:** `dlg = MyDialog(self)`
Assigns `MyDialog(self)` to `dlg`.
17. **Line 17:** `result = dlg.ShowModal() # Blocks until closed`
Displays a dialog modally (blocks input to the parent window until closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.
18. **Line 18:** `if result == wx.ID_OK:`
Standard ID for OK button in dialogs.
19. **Line 19:** `# User clicked Save`
A comment: *User clicked Save.*
20. **Line 20:** `pass`
Placeholder: does nothing.
21. **Line 21:** `dlg.Destroy() # Always destroy after use`
Destroys the dialog window and frees its resources. Must be called after `ShowModal()` returns.
-

31.2 Standard Dialogs

```
# File open dialog
with wx.FileDialog(
    self,
    "Open document",
    wildcard="PDF (*.pdf)|*.pdf|All files|*.*",
    style=wx.FD_OPEN | wx.FD_FILE_MUST_EXIST,
) as dlg:
    if dlg.ShowModal() == wx.ID_OK:
        path = dlg.GetPath()

# File save dialog
with wx.FileDialog(
    self,
    "Save DAISY book",
    defaultFile="book.zip",
    wildcard="ZIP (*.zip)|*.zip",
    style=wx.FD_SAVE | wx.FD_OVERWRITE_PROMPT,
) as dlg:
    if dlg.ShowModal() == wx.ID_OK:
        zip_path = dlg.GetPath()

# Message box
answer = wx.MessageBox(
    "Recording complete!",
    "Done",
    style=wx.OK | wx.ICON_INFORMATION,
)
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# File open dialog`
A comment: *File open dialog*.
2. **Line 2:** `with wx.FileDialog(`
Opens a native file open/save dialog. Lets the user browse and select files.
3. **Line 3:** `self,`
`self,`
4. **Line 4:** `"Open document",`
`"Open document",`
5. **Line 5:** `wildcard="PDF (*.pdf)|*.pdf|All files|*.*",`
Assigns `"PDF (*.pdf)|*.pdf|All files|*.*",` to `wildcard`.
6. **Line 6:** `style=wx.FD_OPEN | wx.FD_FILE_MUST_EXIST,`
Assigns `wx.FD_OPEN | wx.FD_FILE_MUST_EXIST,` to `style`.
7. **Line 7:** `) as dlg:`
`) as dlg:`
8. **Line 8:** `if dlg.ShowModal() == wx.ID_OK:`
Displays a dialog modally (blocks input to the parent window until closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.
9. **Line 9:** `path = dlg.GetPath()`
Assigns `dlg.GetPath()` to `path`.
10. **Line 10:** `# File save dialog`
A comment: *File save dialog*.
11. **Line 11:** `with wx.FileDialog(`
Opens a native file open/save dialog. Lets the user browse and select files.
12. **Line 12:** `self,`
`self,`
13. **Line 13:** `"Save DAISY book",`
`"Save DAISY book",`

14. **Line 14:** `defaultFile="book.zip",`
Assigns `"book.zip",` to `defaultFile` .
15. **Line 15:** `wildcard="ZIP (*.zip)|*.zip",`
Assigns `"ZIP (*.zip)|*.zip",` to `wildcard` .
16. **Line 16:** `style=wx.FD_SAVE | wx.FD_OVERWRITE_PROMPT,`
Assigns `wx.FD_SAVE | wx.FD_OVERWRITE_PROMPT,` to `style` .
17. **Line 17:** `) as dlg:`
`) as dlg:`
18. **Line 18:** `if dlg.ShowModal() == wx.ID_OK:`
Displays a dialog modally (blocks input to the parent window until closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.
19. **Line 19:** `zip_path = dlg.GetPath()`
Assigns `dlg.GetPath()` to `zip_path` .
20. **Line 20:** `# Message box`
A comment: *Message box*.
21. **Line 21:** `answer = wx.MessageBox(`
Assigns `wx.MessageBox(` to `answer` .
22. **Line 22:** `"Recording complete!",`
`"Recording complete!",`
23. **Line 23:** `"Done",`
`"Done",`
24. **Line 24:** `style=wx.OK | wx.ICON_INFORMATION,`
Assigns `wx.OK | wx.ICON_INFORMATION,` to `style` .
25. **Line 25:** `)`
`)`
-

31.3 The New Project Wizard

```
# From gui/new_project_wizard.py
from wx.adv import Wizard, WizardPage

class NewProjectWizard(Wizard):
    def __init__(self, parent, settings, store):
        super().__init__(parent, title="New Project - AI Voice Studio")

        # Page 1: project details
        self.page_details = _DetailsPage(self)
        self.page_details.name_ctrl = wx.TextCtrl(...)
        self.page_details.open_btn = wx.Button(...)

        # Page 2: audio mode selection
        self.page_mode = _ModePage(self, settings)
        self.page_mode.radios = [...]
        self.page_mode.description = wx.StaticText(...)

        self.GetPageAreaSizer().Add(self.page_details)

        self.Bind(EVT_WIZARD_FINISHED, self._on_finish)

    def _on_finish(self, _):
        """Called when user clicks Finish."""
        name = self.page_details.name_ctrl.GetValue().strip()
        mode = self.page_mode.selected()

        # Parse document and create project
        doc = parse_document(self.source_path)
        segments = split_document(doc, mode)
        project.create_project(pdir, name, self.source_path, mode, segments)
        self.EndModal(wx.ID_OK)

        # Open the recording window
        dlg = RecordingDialog(self.GetParent(), pdir, self.settings, self.store)
        dlg.ShowModal()
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From gui/new_project_wizard.py`
A comment: *From gui/new_project_wizard.py.*
2. **Line 2:** `from wx.adv import Wizard, WizardPage`
Imports `Wizard, WizardPage` from `wx.adv`.
3. **Line 3:** `class NewProjectWizard(Wizard):`
Defines class `NewProjectWizard` inheriting from `Wizard`.
4. **Line 4:** `def __init__(self, parent, settings, store):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `super().__init__(parent, title="New Project - AI Voice Studio")`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
6. **Line 6:** `# Page 1: project details`
A comment: *Page 1: project details.*
7. **Line 7:** `self.page_details = _DetailsPage(self)`
Stores `_DetailsPage(self)` in instance attribute `page_details`.
8. **Line 8:** `self.page_details.name_ctrl = wx.TextCtrl(...)`
Creates a text input control. Can be single-line (for paths) or multi-line (for text display).
9. **Line 9:** `self.page_details.open_btn = wx.Button(...)`
Creates a clickable push button. The most common way to trigger user actions.
10. **Line 10:** `# Page 2: audio mode selection`
A comment: *Page 2: audio mode selection.*
11. **Line 11:** `self.page_mode = _ModePage(self, settings)`
Stores `_ModePage(self, settings)` in instance attribute `page_mode`.
12. **Line 12:** `self.page_mode.radios = [...]`
Assigns `[...]` to `self.page_mode.radios`.
13. **Line 13:** `self.page_mode.description = wx.StaticText(...)`
Creates a non-editable text label.
14. **Line 14:** `self.GetPageAreaSizer().Add(self.page_details)`

Calls `self.GetPageAreaSizer()` .

15. **Line 15:** `self.Bind(EVT_WIZARD_FINISHED, self._on_finish)`

Registers an event handler on the frame/window itself. Used for frame-level events.

16. **Line 16:** `def _on_finish(self, _):`

Defines function `_on_finish` accepting (self, _).

17. **Line 17:** `"""Called when user clicks Finish."""`

Docstring: *Called when user clicks Finish..* Documents the purpose of the following code.

18. **Line 18:** `name = self.page_details.name_ctrl.GetValue().strip()`

Removes leading and trailing whitespace (or specified characters). Returns a new string.

19. **Line 19:** `mode = self.page_mode.selected()`

Assigns `self.page_mode.selected()` to `mode` .

20. **Line 20:** `# Parse document and create project`

A comment: *Parse document and create project.*

21. **Line 21:** `doc = parse_document(self.source_path)`

Assigns `parse_document(self.source_path)` to `doc` .

22. **Line 22:** `segments = split_document(doc, mode)`

Assigns `split_document(doc, mode)` to `segments` .

23. **Line 23:** `project.create_project(pdir, name, self.source_path, mode, segments)`

`project.create_project(pdir, name, self.source_path, mode, segments)`

24. **Line 24:** `self.EndModal(wx.ID_OK)`

Ends a modal dialog with the given return code (e.g., `wx.ID_OK`).

25. **Line 25:** `# Open the recording window`

A comment: *Open the recording window.*

26. **Line 26:** `dlg = RecordingDialog(self.GetParent(), pdir, self.settings, self.store)`

Assigns `RecordingDialog(self.GetParent(), pdir, self.settings, self.store)` to `dlg` .

27. **Line 27:** `dlg.ShowModal()`

Displays a dialog modally (blocks input to the parent window until closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.

31.4 Key Takeaways

- **Modal dialogs** block the parent — use for settings, wizards, confirmations
 - **ShowModal()** returns `wx.ID_OK` or `wx.ID_CANCEL`
 - **Always call Destroy()** on dialogs (or use `with` statement)
 - **wx.adv.Wizard** provides multi-step wizard pages with Next/Back/Finish
 - The AIVS wizard has **2 pages**: Details (name, document, punctuation) and Mode (splitting method)
-

Custom Drawing & Device Context

Learning Objectives

- Understand wxPython's Device Context (DC) drawing model
 - Create custom-painted widgets
 - See how AIVS themes and custom controls work
-

32.1 Device Context (DC) Basics

```
import wx

class WaveformPanel(wx.Panel):
    """Custom panel that draws an audio waveform."""

    def __init__(self, parent):
        super().__init__(parent)
        self._samples = []
        self.Bind(wx.EVT_PAINT, self._on_paint)

    def set_samples(self, samples):
        self._samples = samples
        self.Refresh() # Trigger repaint

    def _on_paint(self, event):
        dc = wx.PaintDC(self)
        width, height = self.GetSize()

        # Clear background
        dc.SetBackground(wx.Brush(wx.WHITE))
        dc.Clear()

        if not self._samples:
            return

        # Draw waveform
        dc.SetPen(wx.Pen(wx.BLUE, 1))
        mid = height // 2
        step = max(1, len(self._samples) // width)

        for x in range(width):
            idx = int(x * len(self._samples) / width)
            sample = self._samples[idx] / 32768.0
            y = int(mid - sample * mid * 0.9)
            dc.DrawPoint(x, y)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import wx`
Imports the `wx` module.
2. **Line 2:** `class WaveformPanel(wx.Panel):`
A container that sits inside a Frame. Groups controls and provides a background for drawing.
3. **Line 3:** `"""Custom panel that draws an audio waveform."""`
Docstring: *Custom panel that draws an audio waveform..* Documents the purpose of the following code.
4. **Line 4:** `def __init__(self, parent):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `super().__init__(parent)`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
6. **Line 6:** `self._samples = []`
Stores `[]` in instance attribute `_samples`.
7. **Line 7:** `self.Bind(wx.EVT_PAINT, self._on_paint)`
Event triggered when the window needs repainting.
8. **Line 8:** `def set_samples(self, samples):`
Defines function `set_samples` accepting (self, samples).
9. **Line 9:** `self._samples = samples`
Stores `samples` in instance attribute `_samples`.
10. **Line 10:** `self.Refresh() # Trigger repaint`
Calls `self.Refresh()`.
11. **Line 11:** `def _on_paint(self, event):`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
12. **Line 12:** `dc = wx.PaintDC(self)`
A DC for handling EVT_PAINT events. Auto-finishes painting when the context is destroyed.
13. **Line 13:** `width, height = self.GetSize()`
`width, height = self.GetSize()`
14. **Line 14:** `# Clear background`

A comment: *Clear background.*

15. **Line 15:** `dc.SetBackground(wx.Brush(wx.WHITE))`

Rounds a number to the given number of decimal places. Uses banker's rounding (round half to even).

16. **Line 16:** `dc.Clear()`

`dc.Clear()`

17. **Line 17:** `if not self._samples:`

Checks condition: `not self._samples`. The indented block runs only when this is `True`.

18. **Line 18:** `return`

Returns `None` implicitly (exits the function).

19. **Line 19:** `# Draw waveform`

A comment: *Draw waveform.*

20. **Line 20:** `dc.SetPen(wx.Pen(wx.BLUE, 1))`

`dc.SetPen(wx.Pen(wx.BLUE, 1))`

21. **Line 21:** `mid = height // 2`

Assigns `height // 2` to `mid`.

22. **Line 22:** `step = max(1, len(self._samples) // width)`

Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

23. **Line 23:** `for x in range(width):`

Generates a sequence of integers. Commonly used in for loops.

24. **Line 24:** `idx = int(x * len(self._samples) / width)`

Returns the number of items in a sequence (string, list, dict, tuple). Essential for loop bounds and validation.

25. **Line 25:** `sample = self._samples[idx] / 32768.0`

Assigns `self._samples[idx] / 32768.0` to `sample`.

26. **Line 26:** `y = int(mid - sample * mid * 0.9)`

Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.

27. **Line 27:** `dc.DrawPoint(x, y)`

Converts a value to an integer. Truncates floats toward zero. Raises `ValueError` for invalid strings.

32.2 Common DC Operations

```
# Drawing primitives
dc.DrawLine(x1, y1, x2, y2)           # Line
dc.DrawRectangle(x, y, w, h)          # Rectangle
dc.DrawCircle(cx, cy, radius)         # Circle
dc.DrawEllipse(x, y, w, h)            # Ellipse
dc.DrawText("Hello", x, y)            # Text

# Pen (outlines) and Brush (fills)
dc.SetPen(wx.Pen(wx.RED, 2))          # 2px red outline
dc.SetBrush(wx.Brush(wx.BLUE))        # Blue fill

# Font
dc.SetFont(wx.Font(12, wx.FONTFAMILY_SWISS, wx.FONTSTYLE_NORMAL, wx.FONTWEIGHT_BOLD))
dc.SetTextForeground(wx.BLACK)

# Buffered DC (flicker-free drawing)
def _on_paint(self, event):
    dc = wx.PaintDC(self)
    buf_dc = wx.BufferedDC(dc) # Double-buffered
    # Draw on buf_dc...
    # Automatically blits to screen when done
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Drawing primitives`
A comment: *Drawing primitives.*
2. **Line 2:** `dc.DrawLine(x1, y1, x2, y2) # Line`
`dc.DrawLine(x1, y1, x2, y2) # Line`
3. **Line 3:** `dc.DrawRectangle(x, y, w, h) # Rectangle`
`dc.DrawRectangle(x, y, w, h) # Rectangle`
4. **Line 4:** `dc.DrawCircle(cx, cy, radius) # Circle`
`dc.DrawCircle(cx, cy, radius) # Circle`
5. **Line 5:** `dc.DrawEllipse(x, y, w, h) # Ellipse`
`dc.DrawEllipse(x, y, w, h) # Ellipse`
6. **Line 6:** `dc.DrawText("Hello", x, y) # Text`
`dc.DrawText("Hello", x, y) # Text`
7. **Line 7:** `# Pen (outlines) and Brush (fills)`
A comment: *Pen (outlines) and Brush (fills).*
8. **Line 8:** `dc.SetPen(wx.Pen(wx.RED, 2)) # 2px red outline`
`dc.SetPen(wx.Pen(wx.RED, 2)) # 2px red outline`
9. **Line 9:** `dc.SetBrush(wx.Brush(wx.BLUE)) # Blue fill`
`dc.SetBrush(wx.Brush(wx.BLUE)) # Blue fill`
10. **Line 10:** `# Font`
A comment: *Font.*
11. **Line 11:** `dc.SetFont(wx.Font(12, wx.FONTFAMILY_SWISS, wx.FONTSTYLE_NORMAL, wx.FONTWEIGHT_BOLD))`
Represents a font face, size, and style. Used for consistent typography.
12. **Line 12:** `dc.SetTextForeground(wx.BLACK)`
Rounds a number to the given number of decimal places. Uses banker's rounding (round half to even).
13. **Line 13:** `# Buffered DC (flicker-free drawing)`
A comment: *Buffered DC (flicker-free drawing).*

14. **Line 14:** `def _on_paint(self, event):`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
15. **Line 15:** `dc = wx.PaintDC(self)`
A DC for handling EVT_PAINT events. Auto-finishes painting when the context is destroyed.
16. **Line 16:** `buf_dc = wx.BufferedDC(dc) # Double-buffered`
A DC that buffers drawing operations to prevent flicker.
17. **Line 17:** `# Draw on buf_dc...`
A comment: *Draw on buf_dc....*
18. **Line 18:** `# Automatically blits to screen when done`
A comment: *Automatically blits to screen when done.*

32.3 Themed Drawing in AIVS

```
# From gui/theme.py - applying colors to controls
def _paint(window: wx.Window, colors: dict) -> None:
    """Apply theme colors to a control."""
    try:
        if isinstance(window, (wx.Panel, wx.Dialog, wx.Frame)):
            window.SetBackgroundColour(colors["bg"])
        elif isinstance(window, (wx.StaticText, wx.CheckBox)):
            window.SetForegroundColour(colors["fg"])
        elif isinstance(window, (wx.TextCtrl, wx.ComboBox)):
            window.SetBackgroundColour(colors["control_bg"])
            window.SetForegroundColour(colors["control_fg"])
    except Exception:
        pass # Some controls don't support color changes
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From gui/theme.py – applying colors to controls`
A comment: *From gui/theme.py – applying colors to controls.*
2. **Line 2:** `def _paint(window: wx.Window, colors: dict) -> None:`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
3. **Line 3:** `"""Apply theme colors to a control."""`
Docstring: *Apply theme colors to a control..* Documents the purpose of the following code.
4. **Line 4:** `try:`
Begins exception handling. Code in this block is monitored for errors.
5. **Line 5:** `if isinstance(window, (wx.Panel, wx.Dialog, wx.Frame)):`
Checks if an object is an instance of a class. Preferred over type() for type checking.
6. **Line 6:** `window.SetBackgroundColour(colors["bg"])`
`window.SetBackgroundColour(colors["bg"])`
7. **Line 7:** `elif isinstance(window, (wx.StaticText, wx.CheckBox)):`
A non-editable text label widget. The primary way to display text information in wxPython.
8. **Line 8:** `window.SetForegroundColour(colors["fg"])`
`window.SetForegroundColour(colors["fg"])`
9. **Line 9:** `elif isinstance(window, (wx.TextCtrl, wx.ComboBox)):`
Checks if an object is an instance of a class. Preferred over type() for type checking.
10. **Line 10:** `window.SetBackgroundColour(colors["control_bg"])`
`window.SetBackgroundColour(colors["control_bg"])`
11. **Line 11:** `window.SetForegroundColour(colors["control_fg"])`
`window.SetForegroundColour(colors["control_fg"])`
12. **Line 12:** `except Exception:`
Catches all exceptions that inherit from Exception. Use as a last resort.
13. **Line 13:** `pass # Some controls don't support color changes`
`pass # Some controls don't support color changes`

32.4 Key Takeaways

- **DC (Device Context)** is the drawing API — paint, draw, fill
 - **BufferedDC** prevents flicker by double-buffering
 - **Bind(EVT_PAINT)** for custom drawing; call `Refresh()` to trigger repaint
 - AIVS uses DC drawing for **theme application** across all widgets
-

Threading in GUI Applications

Learning Objectives

- Understand why threading + GUI requires special care
 - Master the `wx.PostEvent` pattern for thread-safe GUI updates
 - See the complete `SynthesisWorker` pattern in AIVS
-

33.1 The Rule: Never Touch GUI from Worker Threads

⚠ **Warning: Rule #1:** wxPython widgets are NOT thread-safe. You must NEVER call widget methods from a worker thread. Always use `wx.PostEvent()` to send messages to the main thread, where the GUI handler updates the widgets.

```
# ❌ WRONG – crashes or corrupts GUI
def worker(self):
    self.status_label.SetLabel("Working...") # DANGER!

# ✓ CORRECT – post event to main thread
def worker(self):
    wx.PostEvent(self, SynthStatusEvent("Working..."))

# ✓ CORRECT – use wx.CallAfter for simple cases
def worker(self):
    wx.CallAfter(self.status_label.SetLabel, "Working...")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# ✗ WRONG – crashes or corrupts GUI`
A comment: ✗ *WRONG – crashes or corrupts GUI.*
2. **Line 2:** `def worker(self):`
Defines function `worker` accepting (self).
3. **Line 3:** `self.status_label.SetLabel("Working...") # DANGER!`
`self.status_label.SetLabel("Working...") # DANGER!`
4. **Line 4:** `# ✓ CORRECT – post event to main thread`
A comment: ✓ *CORRECT – post event to main thread.*
5. **Line 5:** `def worker(self):`
Defines function `worker` accepting (self).
6. **Line 6:** `wx.PostEvent(self, SynthStatusEvent("Working..."))`
Creates a wxPython `wx.PostEvent` widget.
7. **Line 7:** `# ✓ CORRECT – use wx.CallAfter for simple cases`
A comment: ✓ *CORRECT – use wx.CallAfter for simple cases.*
8. **Line 8:** `def worker(self):`
Defines function `worker` accepting (self).
9. **Line 9:** `wx.CallAfter(self.status_label.SetLabel, "Working...")`
Schedules a function to run on the main thread after the current event handler. Essential for thread-safe GUI updates.

33.2 The Complete Pattern

The AIVS Recording Dialog demonstrates the full pattern:


```

# 1. Define custom events
_SYNTH_STATUS = wx.NewEventType()
EVT_SYNTH_STATUS = wx.PyEventBinder(_SYNTH_STATUS, 1)

class SynthStatusEvent(wx.PyCommandEvent):
    def __init__(self, message: str):
        super().__init__(_SYNTH_STATUS)
        self.message = message

# 2. Create the worker thread
class SynthesisWorker(threading.Thread):
    def __init__(self, ..., on_status, on_segment_done, on_all_done, on_error):
        super().__init__(daemon=True)
        self._on_status = on_status
        self._on_done = on_segment_done
        self._on_all_done = on_all_done
        self._on_error = on_error

    def run(self):
        try:
            for i, seg in enumerate(self._segments):
                self._on_status(f"Recording {seg.title}...")
                # ... synthesize ...
                self._on_done(i, seg.title, path)
            self._on_all_done()
        except Exception as exc:
            self._on_error(str(exc))

# 3. Launch from GUI (main thread)
def _on_start(self, _):
    self._worker = SynthesisWorker(
        segments=self._segments,
        on_status=lambda msg: wx.PostEvent(self, SynthStatusEvent(msg)),
        on_segment_done=lambda i, t, p: wx.PostEvent(self, SynthSegmentDoneEvent(i, t, p)),
        on_all_done=lambda: wx.PostEvent(self, SynthFinishedEvent()),
        on_error=lambda msg: wx.PostEvent(self, SynthErrorEvent(msg)),
    )
    self._worker.start()

# 4. Handle events (main thread)
self.Bind(EVT_SYNTH_STATUS, self._on_synth_status)
self.Bind(EVT_SYNTH_SEGMENT_DONE, self._on_segment_done)
self.Bind(EVT_SYNTH_FINISHED, self._on_finished)
self.Bind(EVT_SYNTH_ERROR, self._on_error)

def _on_synth_status(self, evt):
    self.status_label.SetLabel(evt.message) # Safe - main thread

```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# 1. Define custom events`

A comment: 1. Define custom events.

2. **Line 2:** `_SYNTH_STATUS = wx.NewEventType()`

Assigns `wx.NewEventType()` to `_SYNTH_STATUS`.

3. **Line 3:** `EVT_SYNTH_STATUS = wx.PyEventBinder(_SYNTH_STATUS, 1)`

Assigns `wx.PyEventBinder(_SYNTH_STATUS, 1)` to `EVT_SYNTH_STATUS`.

4. **Line 4:** `class SynthStatusEvent(wx.PyCommandEvent):`

Defines class `SynthStatusEvent` inheriting from `wx.PyCommandEvent`.

5. **Line 5:** `def __init__(self, message: str):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

6. **Line 6:** `super().__init__(_SYNTH_STATUS)`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

7. **Line 7:** `self.message = message`

Stores `message` in instance attribute `message`.

8. **Line 8:** `# 2. Create the worker thread`

A comment: 2. Create the worker thread.

9. **Line 9:** `class SynthesisWorker(threading.Thread):`

Creates a background thread. Use `daemon=True` for threads that should not block app exit.

10. **Line 10:** `def __init__(self, ..., on_status, on_segment_done, on_all_done, on_error):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

11. **Line 11:** `super().__init__(daemon=True)`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

12. **Line 12:** `self._on_status = on_status`

Stores `on_status` in instance attribute `_on_status`.

13. **Line 13:** `self._on_done = on_segment_done`

Stores `on_segment_done` in instance attribute `_on_done`.

14. **Line 14:** `self._on_all_done = on_all_done`

Stores `on_all_done` in instance attribute `_on_all_done`.

15. **Line 15:** `self._on_error = on_error`
Stores `on_error` in instance attribute `_on_error`.

16. **Line 16:** `def run(self):`
Defines function `run` accepting (self).

17. **Line 17:** `try:`
Begins exception handling. Code in this block is monitored for errors.

18. **Line 18:** `for i, seg in enumerate(self._segments):`
Adds a counter to an iterable, returning (index, value) pairs. Preferred over manual index tracking.

19. **Line 19:** `self._on_status(f"Recording {seg.title}...")`
Calls `self._on_status()`.

20. **Line 20:** `# ... synthesize ...`
A comment: ... *synthesize*

21. **Line 21:** `self._on_done(i, seg.title, path)`
Calls `self._on_done()`.

22. **Line 22:** `self._on_all_done()`
Calls `self._on_all_done()`.

23. **Line 23:** `except Exception as exc:`
Catches `Exception` exceptions.

24. **Line 24:** `self._on_error(str(exc))`
Converts a value to its string representation. Essential for string concatenation and formatting.

25. **Line 25:** `# 3. Launch from GUI (main thread)`
A comment: 3. *Launch from GUI (main thread)*.

26. **Line 26:** `def _on_start(self, _):`
Defines function `_on_start` accepting (self, _).

27. **Line 27:** `self._worker = SynthesisWorker(`
Stores `SynthesisWorker(` in instance attribute `_worker`.

28. **Line 28:** `segments=self._segments,`

Assigns `self._segments`, to `segments` .

29. **Line 29:** `on_status=lambda msg: wx.PostEvent(self, SynthStatusEvent(msg)),`

Creates an anonymous function and assigns it to `on_status` .

30. **Line 30:** `on_segment_done=lambda i, t, p: wx.PostEvent(self, SynthSegmentDoneEvent(i, t, p)),`

Creates an anonymous function and assigns it to `on_segment_done` .

31. **Line 31:** `on_all_done=lambda: wx.PostEvent(self, SynthFinishedEvent()),`

Assigns `lambda: wx.PostEvent(self, SynthFinishedEvent()),` to `on_all_done` .

32. **Line 32:** `on_error=lambda msg: wx.PostEvent(self, SynthErrorEvent(msg)),`

Creates an anonymous function and assigns it to `on_error` .

33. **Line 33:** `)`

`)`

34. **Line 34:** `self._worker.start()`

`self._worker.start()`

35. **Line 35:** `# 4. Handle events (main thread)`

A comment: *4. Handle events (main thread).*

36. **Line 36:** `self.Bind(EVT_SYNTH_STATUS, self._on_synth_status)`

Registers an event handler on the frame/window itself. Used for frame-level events.

37. **Line 37:** `self.Bind(EVT_SYNTH_SEGMENT_DONE, self._on_segment_done)`

Registers an event handler on the frame/window itself. Used for frame-level events.

38. **Line 38:** `self.Bind(EVT_SYNTH_FINISHED, self._on_finished)`

Registers an event handler on the frame/window itself. Used for frame-level events.

39. **Line 39:** `self.Bind(EVT_SYNTH_ERROR, self._on_error)`

Registers an event handler on the frame/window itself. Used for frame-level events.

40. **Line 40:** `def _on_synth_status(self, evt):`

Defines function `_on_synth_status` accepting (self, evt).

41. **Line 41:** `self.status_label.SetLabel(evt.message) # Safe – main thread`

`self.status_label.SetLabel(evt.message) # Safe – main thread`

33.3 Cancel and Pause Patterns

```
# Using threading.Event for cancellation
self._cancel_event = threading.Event()

# Worker checks periodically
def run(self):
    for seg in self._segments:
        if self._cancel_event.is_set():
            break # Exit gracefully
        # ... process segment ...

# GUI signals cancellation
def _on_stop(self, _):
    self._cancel_event.set()
    self.status_label.SetLabel("Stopping...")

# Pause uses a separate event
self._pause_event = threading.Event()

def run(self):
    for seg in self._segments:
        while self._pause_event.is_set():
            if self._cancel_event.is_set():
                break
            time.sleep(0.1) # Wait while paused
        # ... process segment ...
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Using threading.Event for cancellation`
A comment: *Using threading.Event for cancellation.*
2. **Line 2:** `self._cancel_event = threading.Event()`
A flag that threads can wait on. Used for signaling between threads.
3. **Line 3:** `# Worker checks periodically`
A comment: *Worker checks periodically.*
4. **Line 4:** `def run(self):`
Defines function `run` accepting (self).
5. **Line 5:** `for seg in self._segments:`
Iterates over `self._segments`, assigning each element to `seg`.
6. **Line 6:** `if self._cancel_event.is_set():`
Checks condition: `self._cancel_event.is_set()`. The indented block runs only when this is `True`.
7. **Line 7:** `break # Exit gracefully`
`break # Exit gracefully`
8. **Line 8:** `# ... process segment ...`
A comment: *... process segment*
9. **Line 9:** `# GUI signals cancellation`
A comment: *GUI signals cancellation.*
10. **Line 10:** `def _on_stop(self, _):`
Defines function `_on_stop` accepting (self, _).
11. **Line 11:** `self._cancel_event.set()`
`self._cancel_event.set()`
12. **Line 12:** `self.status_label.SetLabel("Stopping...")`
`self.status_label.SetLabel("Stopping...")`
13. **Line 13:** `# Pause uses a separate event`
A comment: *Pause uses a separate event.*

14. **Line 14:** `self._pause_event = threading.Event()`
A flag that threads can wait on. Used for signaling between threads.

15. **Line 15:** `def run(self):`
Defines function `run` accepting `(self)`.

16. **Line 16:** `for seg in self._segments:`
Iterates over `self._segments`, assigning each element to `seg`.

17. **Line 17:** `while self._pause_event.is_set():`
While loop: repeats as long as `self._pause_event.is_set()`.

18. **Line 18:** `if self._cancel_event.is_set():`
Checks condition: `self._cancel_event.is_set()`. The indented block runs only when this is `True`.

19. **Line 19:** `break`
Exits the innermost loop.

20. **Line 20:** `time.sleep(0.1) # Wait while paused`
`time.sleep(0.1) # Wait while paused`

21. **Line 21:** `# ... process segment ...`
A comment: ... *process segment*

33.4 Key Takeaways

- **Never touch GUI widgets from worker threads**
 - **wx.PostEvent()** sends events from worker → GUI safely
 - **wx.CallAfter()** is a simpler alternative for single calls
 - Use **threading.Event** for cancel/pause signaling
 - The AIVS pattern: **Worker thread** → **custom wx events** → **GUI handlers**
-

Accessibility (a11y)

Learning Objectives

- Understand why accessibility matters in desktop apps
 - Make wxPython apps accessible with `SetName()` and proper labels
 - See the AIVS a11y helper system
-

34.1 What Is Accessibility?

Accessibility (a11y) ensures that people with disabilities can use your software. For a **text-to-speech application** like AIVS, this is especially important because many users rely on screen readers (NVDA, JAWS, Narrator).

34.2 wxPython Accessibility Basics

```
# Every control needs a readable name for screen readers
button = wx.Button(parent, label="Start recording")
button.SetName("Start recording") # MSAA/UIA accessible name

# Text controls should be read-only and have clear names
text = wx.TextCtrl(
    parent,
    style=wx.TE_MULTILINE | wx.TE_READONLY,
    size=(-1, 130),
)
text.SetName("Text to record")
text.SetToolTip("Read-only preview of the text that will be synthesized")

# ComboBoxes need accessible names
combo = wx.ComboBox(parent, style=wx.CB_READONLY)
combo.SetName("Select TTS engine")

# Announce state changes
self.status = wx.StaticText(self, label="Ready.")
self.status.SetName("Recording status")
self.status.SetToolTip("Current status of the recording process")
```


Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Every control needs a readable name for screen readers`
A comment: *Every control needs a readable name for screen readers.*
2. **Line 2:** `button = wx.Button(parent, label="Start recording")`
Creates a clickable push button. The most common way to trigger user actions.
3. **Line 3:** `button.SetName("Start recording") # MSAA/UIA accessible name`
`button.SetName("Start recording") # MSAA/UIA accessible name`
4. **Line 4:** `# Text controls should be read-only and have clear names`
A comment: *Text controls should be read-only and have clear names.*
5. **Line 5:** `text = wx.TextCtrl(`
Creates a text input control. Can be single-line (for paths) or multi-line (for text display).
6. **Line 6:** `parent,`
`parent,`
7. **Line 7:** `style=wx.TE_MULTILINE | wx.TE_READONLY,`
TextCtrl style: enables multi-line text input.
8. **Line 8:** `size=(-1, 130),`
Assigns `(-1, 130),` to `size`.
9. **Line 9:** `)`
`)`
10. **Line 10:** `text.SetName("Text to record")`
`text.SetName("Text to record")`
11. **Line 11:** `text.SetToolTip("Read-only preview of the text that will be synthesized")`
`text.SetToolTip("Read-only preview of the text that will be synthesized")`
12. **Line 12:** `# ComboBoxes need accessible names`
A comment: *ComboBoxes need accessible names.*
13. **Line 13:** `combo = wx.ComboBox(parent, style=wx.CB_READONLY)`
ComboBox style: prevents typing custom values.
14. **Line 14:** `combo.SetName("Select TTS engine")`

```
combo.SetName("Select TTS engine")
```

15. **Line 15:** `# Announce state changes`

A comment: *Announce state changes.*

16. **Line 16:** `self.status = wx.StaticText(self, label="Ready.")`

Creates a non-editable text label.

17. **Line 17:** `self.status.SetName("Recording status")`

```
self.status.SetName("Recording status")
```

18. **Line 18:** `self.status.SetToolTip("Current status of the recording process")`

```
self.status.SetToolTip("Current status of the recording process")
```

34.3 AIVS a11y Helper

```
# From gui/a11y.py
def add_labeled(parent, sizer, label_text, control, **kwargs):
    """Add a label-control pair with screen reader support."""
    label = wx.StaticText(parent, label=label_text + ":")
    _tint(label) # Apply theme colors
    sizer.Add(label, 0, wx.ALIGN_CENTER_VERTICAL | wx.ALL, 4)
    control.SetName(label_text) # Set accessible name
    sizer.Add(control, 1, wx.EXPAND | wx.ALL, 4)
    return label

def add_check(parent, sizer, label_text, checked=False):
    """CheckBox with accessible name."""
    box = wx.CheckBox(parent, label=label_text)
    box.SetValue(checked)
    box.SetName(label_text)
    sizer.Add(box, 0, wx.ALL, 4)
    return box

def set_accessible_name(control, name):
    control.SetName(name)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From gui/a11y.py`
A comment: *From gui/a11y.py.*
2. **Line 2:** `def add_labeled(parent, sizer, label_text, control, **kwargs):`
Defines function `add_labeled` accepting (parent, sizer, label_text, control, ...).
3. **Line 3:** `"""Add a label-control pair with screen reader support."""`
Docstring: *Add a label-control pair with screen reader support..* Documents the purpose of the following code.
4. **Line 4:** `label = wx.StaticText(parent, label=label_text + ":")`
Creates a non-editable text label.
5. **Line 5:** `_tint(label) # Apply theme colors`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
6. **Line 6:** `sizer.Add(label, 0, wx.ALIGN_CENTER_VERTICAL | wx.ALL, 4)`
Flag: centers the widget vertically.
7. **Line 7:** `control.SetName(label_text) # Set accessible name`
`control.SetName(label_text) # Set accessible name`
8. **Line 8:** `sizer.Add(control, 1, wx.EXPAND | wx.ALL, 4)`
Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.
9. **Line 9:** `return label`
Returns `label` to the caller.
10. **Line 10:** `def add_check(parent, sizer, label_text, checked=False):`
Defines function `add_check` accepting (parent, sizer, label_text, checked).
11. **Line 11:** `"""CheckBox with accessible name."""`
Docstring: *CheckBox with accessible name..* Documents the purpose of the following code.
12. **Line 12:** `box = wx.CheckBox(parent, label=label_text)`
Creates a check box toggle. Returns True when checked, False when unchecked.
13. **Line 13:** `box.SetValue(checked)`
`box.SetValue(checked)`
14. **Line 14:** `box.SetName(label_text)`

```
box.SetName(label_text)
```

15. **Line 15:** `sizer.Add(box, 0, wx.ALL, 4)`

Adds a child window to the sizer. Parameters: window, proportion, flags, border. Flags control alignment and padding.

16. **Line 16:** `return box`

Returns `box` to the caller.

17. **Line 17:** `def set_accessible_name(control, name):`

Defines function `set_accessible_name` accepting (control, name).

18. **Line 18:** `control.SetName(name)`

```
control.SetName(name)
```

34.4 Tab Order and Focus

```
# Tab order follows creation order in the sizer
# AIVS puts the text preview first so screen readers
# announce the content before the controls

# Announce page changes in wizards
def _on_page_changed(self, evt):
    page = evt.GetPage()
    if evt.GetDirection(): # Forward
        if page is self.page_details:
            self.page_details.name_ctrl.SetFocus()
    evt.Skip()

# Set initial focus when dialogs open
self.SetName(f"Recording: {project_name}")
wx.CallAfter(self.text_preview.SetFocus)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Tab order follows creation order in the sizer`
A comment: *Tab order follows creation order in the sizer.*
2. **Line 2:** `# AIVS puts the text preview first so screen readers`
A comment: *AIVS puts the text preview first so screen readers.*
3. **Line 3:** `# announce the content before the controls`
A comment: *announce the content before the controls.*
4. **Line 4:** `# Announce page changes in wizards`
A comment: *Announce page changes in wizards.*
5. **Line 5:** `def _on_page_changed(self, evt):`
Defines function `_on_page_changed` accepting (self, evt).
6. **Line 6:** `page = evt.GetPage()`
Assigns `evt.GetPage()` to `page`.
7. **Line 7:** `if evt.GetDirection(): # Forward`
Checks condition: `evt.GetDirection()`. The indented block runs only when this is `True`.
8. **Line 8:** `if page is self.page_details:`
Checks condition: `page is self.page_details`. The indented block runs only when this is `True`.
9. **Line 9:** `self.page_details.name_ctrl.SetFocus()`
`self.page_details.name_ctrl.SetFocus()`
10. **Line 10:** `evt.Skip()`
`evt.Skip()`
11. **Line 11:** `# Set initial focus when dialogs open`
A comment: *Set initial focus when dialogs open.*
12. **Line 12:** `self.SetName(f"Recording: {project_name}")`
Calls `self.SetName()`.
13. **Line 13:** `wx.CallAfter(self.text_preview.SetFocus)`
Schedules a function to run on the main thread after the current event handler. Essential for thread-safe GUI updates.

34.5 What Changed Since the First Edition: Real MSAA Names

The first edition relied on `wx.Control.SetName()` and labels. Testing with real screen readers showed that `SetName` alone does not produce an MSAA `accName` for every control class — some controls (gauges, read-only text, static labels in sizers) still announced as "pane" or their class name. The helper module now attaches a real `wx.Accessible` implementation:

```
class _NameAccessible(wx.Accessible):
    def __init__(self, win: wx.Window):
        super().__init__(win)
        self._win = win

    def GetName(self, childId):
        name = self._win.GetName()
        if childId == 0 and name:
            return (wx.ACC_OK, name, 0)
        return (wx.ACC_FAIL, "", 0)

    def GetChildCount(self):
        return 0

def set_accessible_name(control: wx.Window, name: str) -> None:
    control.SetName(name)
    if not control.GetAccessible():
        control.SetAccessible(_NameAccessible(control))

def update_accessible_name(control: wx.Window, name: str) -> None:
    control.SetName(name)
    acc = control.GetAccessible()
    if acc is not None:
        acc.NotifyEvent(wx.OBJ_CHANGE, 0)

def finalize_accessibility(container: wx.Window) -> None:
    def walk(win: wx.Window) -> None:
        if win.GetName():
            set_accessible_name(win, win.GetName())
        for child in win.GetChildren():
            walk(child)
    walk(container)
```

Explain Line by Line Code Description (Block 5)

1. **Lines 1-4:** `class _NameAccessible(wx.Accessible):`

A minimal MSAA provider. Subclassing `wx.Accessible` and attaching it to a window is the official wxPython way to answer the screen reader's `IAccessible::get_accName` call with a real string — no more reliance on heuristics that differ per control class.

2. **Lines 6-9:** `def GetName(self, childId):`

The one method screen readers call constantly. It returns a tuple: (`wx.ACC_OK`, the window's name, child id o). When the reader hovers or focuses the control, this is the name it announces. Returning `wx.ACC_FAIL` for the object-without-name case lets wx fall back to its own behaviour instead of announcing an empty string.

3. **Lines 11-12:** `def GetChildCount(self): return 0`

Declares the control atomic — no MSAA children to enumerate. This keeps NVDA from descending into leaf controls looking for child elements that do not exist.

4. **Lines 15-18:** `def set_accessible_name(control, name):`

The single entry point every widget builder calls. It does both halves: `SetName(name)` (used by wx itself and by tests) and attaches the MSAA provider exactly once — `GetAccessible()` is checked so repeated calls never stack duplicate providers.

5. **Lines 21-25:** `def update_accessible_name(control, name):`

For controls whose label changes at runtime (the Rate/Pitch/Volume sliders announce "Rate: 1.35"). After updating the name it fires `acc.NotifyEvent(wx.OBJ_CHANGE, 0)`, which pushes an MSAA event so a focused screen reader re-announces the new value as the slider moves — without the event, NVDA would keep reading the stale label.

6. **Lines 28-33:** `def finalize_accessibility(container):`

A depth-first walk over the widget tree at dialog-construction time: every window that carries a name gets the MSAA provider attached. One call in each dialog's `__init__` guarantees full coverage, so individual builders cannot forget a control.

i Info: Audit tooling: `tools/probe_msaa.py` and `tools/a11y_audit.py` launch the real dialogs and dump the MSAA tree, so every `accName` is verified against what NVDA actually sees — not what the source code hopes it sees.

34.6 Key Takeaways

- **SetName()** alone is not enough for every control class — a real `wx.Accessible` guarantees the MSAA `accName`
- Runtime label changes need `NotifyEvent(wx.OBJ_CHANGE)` to be re-announced
- `finalize_accessibility()` walks the whole tree so no control is missed

- Verify with tools (`probe_msaa.py`), not assumptions
 - The AIVS project was designed with accessibility as a **core requirement**
-

[← Ch 33](#) | [Ch 35 — Theming](#) [→](#)

Theming & Styling

Learning Objectives

- Implement Light/Dark/System theme support
 - Apply colors to wxPython widgets
 - See the AIVS theme manager
-

35.1 Theme Manager in AIVS

```
# From gui/theme.py
_LIGHT = {"bg": "#ffffff", "fg": "#1a1a1a",
          "control_bg": "#ffffff", "control_fg": "#1a1a1a"}
_DARK = {"bg": "#1e1e1e", "fg": "#e8e8e8",
         "control_bg": "#2d2d2d", "control_fg": "#e8e8e8"}

_state = {"theme": THEME_SYSTEM, "colors": _LIGHT}
_lock = threading.Lock()

def apply_theme(top: wx.Window, theme: str) -> None:
    """Apply theme to window and all descendants."""
    resolved = _resolve(theme)
    colors = _DARK if resolved == THEME_DARK else _LIGHT
    with _lock:
        _state["theme"] = theme
        _state["colors"] = colors
    _paint(top, colors)
    for child in _descendants(top):
        _paint(child, colors)
    top.Refresh()

def _resolve(theme: str) -> str:
    """Resolve 'system' to light or dark based on OS."""
    if theme == THEME_SYSTEM:
        try:
            appearance = wx.SystemSettings.GetAppearance()
            return THEME_DARK if appearance.IsDark() else THEME_LIGHT
        except Exception:
            return THEME_LIGHT
    return theme
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From gui/theme.py`
A comment: *From gui/theme.py*.
2. **Line 2:** `_LIGHT = {"bg": "#ffffff", "fg": "#1a1a1a",`
Assigns `{"bg": "#ffffff", "fg": "#1a1a1a",` to `_LIGHT`.
3. **Line 3:** `"control_bg": "#ffffff", "control_fg": "#1a1a1a"}`
`"control_bg": "#ffffff", "control_fg": "#1a1a1a"}`
4. **Line 4:** `_DARK = {"bg": "#1e1e1e", "fg": "#e8e8e8",`
Assigns `{"bg": "#1e1e1e", "fg": "#e8e8e8",` to `_DARK`.
5. **Line 5:** `"control_bg": "#2d2d2d", "control_fg": "#e8e8e8"}`
`"control_bg": "#2d2d2d", "control_fg": "#e8e8e8"}`
6. **Line 6:** `_state = {"theme": THEME_SYSTEM, "colors": _LIGHT}`
Assigns `{"theme": THEME_SYSTEM, "colors": _LIGHT}` to `_state`.
7. **Line 7:** `_lock = threading.Lock()`
A mutual exclusion lock. Only one thread can hold it at a time. Prevents race conditions.
8. **Line 8:** `def apply_theme(top: wx.Window, theme: str) -> None:`
Defines function `apply_theme` accepting (top, theme), returning `None`.
9. **Line 9:** `"""Apply theme to window and all descendants."""`
Docstring: *Apply theme to window and all descendants..* Documents the purpose of the following code.
10. **Line 10:** `resolved = _resolve(theme)`
Assigns `_resolve(theme)` to `resolved`.
11. **Line 11:** `colors = _DARK if resolved == THEME_DARK else _LIGHT`
Assigns `_DARK if resolved == THEME_DARK else _LIGHT` to `colors`.
12. **Line 12:** `with _lock:`
Context manager: `_lock`. Automatically manages resource cleanup.
13. **Line 13:** `_state["theme"] = theme`
`_state["theme"] = theme`

14. **Line 14:** `_state["colors"] = colors`
`_state["colors"] = colors`
15. **Line 15:** `_paint(top, colors)`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
16. **Line 16:** `for child in _descendants(top):`
Iterates over `_descendants(top)`, assigning each element to `child`.
17. **Line 17:** `_paint(child, colors)`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
18. **Line 18:** `top.Refresh()`
`top.Refresh()`
19. **Line 19:** `def _resolve(theme: str) -> str:`
Defines function `_resolve` accepting (theme), returning `str`.
20. **Line 20:** `"""Resolve 'system' to light or dark based on OS."""`
Docstring: *Resolve 'system' to light or dark based on OS.*. Documents the purpose of the following code.
21. **Line 21:** `if theme == THEME_SYSTEM:`
Checks condition: `theme == THEME_SYSTEM`. The indented block runs only when this is `True`.
22. **Line 22:** `try:`
Begins exception handling. Code in this block is monitored for errors.
23. **Line 23:** `appearance = wx.SystemSettings.GetAppearance()`
Assigns `wx.SystemSettings.GetAppearance()` to `appearance`.
24. **Line 24:** `return THEME_DARK if appearance.IsDark() else THEME_LIGHT`
Returns `THEME_DARK if appearance.IsDark() else THEME_LIGHT` to the caller.
25. **Line 25:** `except Exception:`
Catches all exceptions that inherit from Exception. Use as a last resort.
26. **Line 26:** `return THEME_LIGHT`
Returns `THEME_LIGHT` to the caller.
27. **Line 27:** `return theme`
Returns `theme` to the caller.

35.2 Applying Colors to Widgets

```
def _paint(window: wx.Window, colors: dict) -> None:
    try:
        # Containers get background color
        if isinstance(window, (wx.Panel, wx.Dialog, wx.Frame, wx.ScrolledWindow)):
            window.SetBackgroundColour(colors["bg"])

        # Text labels get foreground color
        elif isinstance(window, (wx.StaticText, wx.CheckBox, wx.RadioButton)):
            window.SetForegroundColour(colors["fg"])

        # Input controls get both
        elif isinstance(window, (wx.TextCtrl, wx.ComboBox, wx.ListBox)):
            window.SetBackgroundColour(colors["control_bg"])
            window.SetForegroundColour(colors["control_fg"])
    except Exception:
        pass # Some widgets don't support color changes
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def _paint(window: wx.Window, colors: dict) -> None:`
Converts a value to an integer. Truncates floats toward zero. Raises ValueError for invalid strings.
2. **Line 2:** `try:`
Begins exception handling. Code in this block is monitored for errors.
3. **Line 3:** `# Containers get background color`
A comment: *Containers get background color.*
4. **Line 4:** `if isinstance(window, (wx.Panel, wx.Dialog, wx.Frame, wx.ScrolledWindow)):`
Checks if an object is an instance of a class. Preferred over type() for type checking.
5. **Line 5:** `window.SetBackgroundColour(colors["bg"])`
`window.SetBackgroundColour(colors["bg"])`
6. **Line 6:** `# Text labels get foreground color`
A comment: *Text labels get foreground color.*
7. **Line 7:** `elif isinstance(window, (wx.StaticText, wx.CheckBox, wx.RadioButton)):`
A non-editable text label widget. The primary way to display text information in wxPython.
8. **Line 8:** `window.SetForegroundColour(colors["fg"])`
`window.SetForegroundColour(colors["fg"])`
9. **Line 9:** `# Input controls get both`
A comment: *Input controls get both.*
10. **Line 10:** `elif isinstance(window, (wx.TextCtrl, wx.ComboBox, wx.ListBox)):`
Checks if an object is an instance of a class. Preferred over type() for type checking.
11. **Line 11:** `window.SetBackgroundColour(colors["control_bg"])`
`window.SetBackgroundColour(colors["control_bg"])`
12. **Line 12:** `window.SetForegroundColour(colors["control_fg"])`
`window.SetForegroundColour(colors["control_fg"])`
13. **Line 13:** `except Exception:`
Catches all exceptions that inherit from Exception. Use as a last resort.
14. **Line 14:** `pass # Some widgets don't support color changes`

```
pass # Some widgets don't support color changes
```

35.3 Key Takeaways

- **Three themes:** Light, Dark, System (auto-detect OS preference)
 - **wx.SystemSettings.GetAppearance()** detects dark mode on Windows 10/11
 - Colors are applied by **walking the widget tree** and setting background/foreground
 - **Buttons keep native look** — only panels, text, and inputs are themed
 - Theme state is stored in a **thread-safe dict** with locking
-

wx.adv & Advanced Widgets

Learning Objectives

- Use wx.adv for wizards, taskbar icons, and advanced controls
 - See the AIVS Wizard implementation
-

36.1 wx.adv.Wizard

```
from wx.adv import Wizard, WizardPage, EVT_WIZARD_FINISHED, EVT_WIZARD_PAGE_CHANGED

class NewProjectWizard(Wizard):
    def __init__(self, parent, settings, store):
        super().__init__(parent, title="New Project")

        # Create pages
        self.page_details = _DetailsPage(self)
        self.page_mode = _ModePage(self, settings)

        # Wire up the page area
        self.GetPageAreaSizer().Add(self.page_details)

        # Events
        self.Bind(EVT_WIZARD_FINISHED, self._on_finish)
        self.Bind(EVT_WIZARD_PAGE_CHANGED, self._on_page_changed)

class _DetailsPage(WizardPage):
    def __init__(self, wizard):
        super().__init__(wizard)
        self.wizard = wizard
        self._build_ui()

    def GetNext(self):
        return self.wizard.page_mode # Next page

    def GetPrev(self):
        return None # This is the first page
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `from wx.adv import Wizard, WizardPage, EVT_WIZARD_FINISHED, EVT_WIZARD_PAGE_CHANGED`
Imports `Wizard, WizardPage, EVT_WIZARD_FINISHED, EVT_WIZARD_PAGE_CHANGED` from `wx.adv`.
2. **Line 2:** `class NewProjectWizard(Wizard):`
Defines class `NewProjectWizard` inheriting from `Wizard`.
3. **Line 3:** `def __init__(self, parent, settings, store):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `super().__init__(parent, title="New Project")`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `# Create pages`
A comment: *Create pages.*
6. **Line 6:** `self.page_details = _DetailsPage(self)`
Stores `_DetailsPage(self)` in instance attribute `page_details`.
7. **Line 7:** `self.page_mode = _ModePage(self, settings)`
Stores `_ModePage(self, settings)` in instance attribute `page_mode`.
8. **Line 8:** `# Wire up the page area`
A comment: *Wire up the page area.*
9. **Line 9:** `self.GetPageAreaSizer().Add(self.page_details)`
Calls `self.GetPageAreaSizer()`.
10. **Line 10:** `# Events`
A comment: *Events.*
11. **Line 11:** `self.Bind(EVT_WIZARD_FINISHED, self._on_finish)`
Registers an event handler on the frame/window itself. Used for frame-level events.
12. **Line 12:** `self.Bind(EVT_WIZARD_PAGE_CHANGED, self._on_page_changed)`
Registers an event handler on the frame/window itself. Used for frame-level events.
13. **Line 13:** `class _DetailsPage(WizardPage):`
Defines class `_DetailsPage` inheriting from `WizardPage`.
14. **Line 14:** `def __init__(self, wizard):`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

15. **Line 15:** `super().__init__(wizard)`

Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.

16. **Line 16:** `self.wizard = wizard`

Stores `wizard` in instance attribute `wizard`.

17. **Line 17:** `self._build_ui()`

Calls `self._build_ui()`.

18. **Line 18:** `def GetNext(self):`

Defines function `GetNext` accepting (self).

19. **Line 19:** `return self.wizard.page_mode # Next page`

Returns `self.wizard.page_mode # Next page` to the caller.

20. **Line 20:** `def GetPrev(self):`

Defines function `GetPrev` accepting (self).

21. **Line 21:** `return None # This is the first page`

Returns `None` (no value).

36.2 TaskbarIcon

```
from wx.adv import TaskBarIcon

class AppTaskBarIcon(TaskBarIcon):
    def __init__(self, frame):
        super().__init__()
        self.frame = frame
        # Set icon (would need an .ico file)
        # icon = wx.Icon("app.ico", wx.BITMAP_TYPE_ICO)
        # self.SetIcon(icon, "AI Voice Studio")

    def CreatePopupMenu(self):
        menu = wx.Menu()
        menu.Append(wx.ID_OPEN, "&Open")
        menu.AppendSeparator()
        menu.Append(wx.ID_EXIT, "E&xit")
        return menu
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `from wx.adv import TaskBarIcon`
Imports `TaskBarIcon` from `wx.adv`.
2. **Line 2:** `class AppTaskBarIcon(TaskBarIcon):`
Defines class `AppTaskBarIcon` inheriting from `TaskBarIcon`.
3. **Line 3:** `def __init__(self, frame):`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
4. **Line 4:** `super().__init__()`
Constructor method. Called when creating a new instance of a class. Initializes the object's attributes.
5. **Line 5:** `self.frame = frame`
Stores `frame` in instance attribute `frame`.
6. **Line 6:** `# Set icon (would need an .ico file)`
A comment: *Set icon (would need an .ico file).*
7. **Line 7:** `# icon = wx.Icon("app.ico", wx.BITMAP_TYPE_ICO)`
A comment: *icon = wx.Icon("app.ico", wx.BITMAP_TYPE_ICO).*
8. **Line 8:** `# self.SetIcon(icon, "AI Voice Studio")`
A comment: *self.SetIcon(icon, "AI Voice Studio").*
9. **Line 9:** `def CreatePopupMenu(self):`
Defines function `CreatePopupMenu` accepting (self).
10. **Line 10:** `menu = wx.Menu()`
Creates a dropdown menu. Contains menu items, separators, and sub-menus.
11. **Line 11:** `menu.Append(wx.ID_OPEN, "&Open")`
Adds a menu item with an ID, label, and optional help text. Use `\t` for accelerators.
12. **Line 12:** `menu.AppendSeparator()`
Adds a visual separator line between menu items.
13. **Line 13:** `menu.Append(wx.ID_EXIT, "E&xit")`
Adds a menu item with an ID, label, and optional help text. Use `\t` for accelerators.
14. **Line 14:** `return menu`

Returns `menu` to the caller.

36.3 wx.adv.HyperlinkCtrl

```
from wx.adv import HyperlinkCtrl, HL_ALIGN_LEFT

link = wx.adv.HyperlinkCtrl(
    self, wx.ID_ANY,
    label="Visit documentation",
    url="https://example.com/docs",
)
```

Explain Line by Line Code Description (Block 3)

- Line 1:** `from wx.adv import HyperlinkCtrl, HL_ALIGN_LEFT`
Imports `HyperlinkCtrl, HL_ALIGN_LEFT` from `wx.adv`.
- Line 2:** `link = wx.adv.HyperlinkCtrl(`
Assigns `wx.adv.HyperlinkCtrl(` to `link`.
- Line 3:** `self, wx.ID_ANY,`
`self, wx.ID_ANY,`
- Line 4:** `label="Visit documentation",`
Assigns `"Visit documentation",` to `label`.
- Line 5:** `url="https://example.com/docs",`
Assigns `"https://example.com/docs",` to `url`.
- Line 6:** `)`
`)`

36.4 Key Takeaways

- **wx.adv.Wizard** provides multi-page wizard UI with Next/Back/Finish

- **wx.adv.TaskBarIcon** for system tray integration
 - AIVS uses the Wizard for **new project creation** (2 pages)
 - The Wizard pattern: create pages, link them with `GetNext()` / `GetPrev()` , handle `EVT_WIZARD_FINISHED`
-

[← Ch 35](#) | **End of Part IV** | [Part V — Ch 37: NumPy](#) →

Dependencies Deep Dive

CHAPTER 37

NumPy for Audio Processing

Learning Objectives

- Understand NumPy arrays and their role in audio
- Perform audio operations: pitch shifting, volume, format conversion
- See NumPy usage throughout AIVS

37.1 NumPy Arrays — The Audio Building Block

```
import numpy as np

# Create arrays from Lists
samples = np.array([0, 1000, -1000, 2000, -2000], dtype=np.int16)

# Audio properties
sample_rate = 24000 # Hz (24 kHz for TTS output)
duration = len(samples) / sample_rate
print(f"Duration: {duration:.3f}s")

# Create synthetic audio (sine wave)
t = np.linspace(0, 1.0, sample_rate) # 1 second of samples
freq = 440 # A4 note
wave = np.sin(2 * np.pi * freq * t)
samples_16bit = (wave * 32767).astype(np.int16)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import numpy as np`
Imports a module.
2. **Line 2:** `# Create arrays from lists`
A comment: *Create arrays from lists.*
3. **Line 3:** `samples = np.array([0, 1000, -1000, 2000, -2000], dtype=np.int16)`
Creates a NumPy array from a list or tuple. The dtype parameter specifies the data type (e.g., np.int16 for 16-bit audio samples).
4. **Line 4:** `# Audio properties`
A comment: *Audio properties.*
5. **Line 5:** `sample_rate = 24000 # Hz (24 kHz for TTS output)`
For loop iteration.
6. **Line 6:** `duration = len(samples) / sample_rate`
Returns the number of items in a sequence (string, list, dict, tuple).
7. **Line 7:** `print(f"Duration: {duration:.3f}s")`
Outputs text to the console. The primary way to display information during development.
8. **Line 8:** `# Create synthetic audio (sine wave)`
A comment: *Create synthetic audio (sine wave).*
9. **Line 9:** `t = np.linspace(0, 1.0, sample_rate) # 1 second of samples`
Creates an array with evenly spaced values between start and stop. num specifies the count.
10. **Line 10:** `freq = 440 # A4 note`
Assigns `440 # A4 note` to `freq`.
11. **Line 11:** `wave = np.sin(2 * np.pi * freq * t)`
Element-wise sine function. Used for generating audio waveforms.
12. **Line 12:** `samples_16bit = (wave * 32767).astype(np.int16)`
16-bit integer type. The standard format for audio samples in TTS output.

37.2 Audio Operations in AIVS

```
# From tts/engine.py - pitch shifting
def _shift_pitch(samples: np.ndarray, pitch: float) -> np.ndarray:
    """Shift pitch without changing duration (resample)."""
    if pitch == 1.0:
        return samples
    # Resample to change pitch
    new_length = int(len(samples) / pitch)
    indices = np.linspace(0, len(samples) - 1, new_length)
    shifted = np.interp(indices, np.arange(len(samples)), samples.astype(np.float64))
    return shifted.astype(np.int16)

# From tts/engine.py - volume adjustment
def _apply_volume(samples: np.ndarray, volume: float) -> np.ndarray:
    """Scale audio samples by volume factor."""
    if volume == 1.0:
        return samples
    scaled = samples.astype(np.float64) * volume
    return np.clip(scaled, -32768, 32767).astype(np.int16)

# From clone/worker.py - base64 encode/decode for IPC
# Worker -> App: encode audio as base64
samples_int16 = np.clip(arr, -1.0, 1.0) * 32767.0
wav_b64 = base64.b64encode(samples_int16.astype(np.int16).tobytes()).decode("ascii")

# App -> decode from base64
raw = base64.b64decode(resp["wav"])
samples = np.frombuffer(raw, dtype=np.int16).copy()
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From tts/engine.py – pitch shifting`
A comment: *From tts/engine.py – pitch shifting.*
2. **Line 2:** `def _shift_pitch(samples: np.ndarray, pitch: float) -> np.ndarray:`
Defines a function.
3. **Line 3:** `"""Shift pitch without changing duration (resample)."""`
Docstring: *Shift pitch without changing duration (resample)..* Documents the following code.
4. **Line 4:** `if pitch == 1.0:`
Conditional check.
5. **Line 5:** `return samples`
Returns a value from a function.
6. **Line 6:** `# Resample to change pitch`
A comment: *Resample to change pitch.*
7. **Line 7:** `new_length = int(len(samples) / pitch)`
Returns the number of items in a sequence (string, list, dict, tuple).
8. **Line 8:** `indices = np.linspace(0, len(samples) - 1, new_length)`
Creates an array with evenly spaced values between start and stop. num specifies the count.
9. **Line 9:** `shifted = np.interp(indices, np.arange(len(samples)), samples.astype(np.float64))`
Creates an array with values from start to stop (exclusive) with a given step.
10. **Line 10:** `return shifted.astype(np.int16)`
16-bit integer type. The standard format for audio samples in TTS output.
11. **Line 11:** `# From tts/engine.py – volume adjustment`
A comment: *From tts/engine.py – volume adjustment.*
12. **Line 12:** `def _apply_volume(samples: np.ndarray, volume: float) -> np.ndarray:`
Defines a function.
13. **Line 13:** `"""Scale audio samples by volume factor."""`
Docstring: *Scale audio samples by volume factor..* Documents the following code.
14. **Line 14:** `if volume == 1.0:`
Conditional check.

15. **Line 15:** `return samples`
Returns a value from a function.
16. **Line 16:** `scaled = samples.astype(np.float64) * volume`
64-bit floating point (double precision). Default float type in NumPy.
17. **Line 17:** `return np.clip(scaled, -32768, 32767).astype(np.int16)`
16-bit integer type. The standard format for audio samples in TTS output.
18. **Line 18:** `# From clone/worker.py - base64 encode/decode for IPC`
A comment: *From clone/worker.py - base64 encode/decode for IPC.*
19. **Line 19:** `# Worker → App: encode audio as base64`
A comment: *Worker → App: encode audio as base64.*
20. **Line 20:** `samples_int16 = np.clip(arr, -1.0, 1.0) * 32767.0`
Clips array values to a range [min, max]. Essential for preventing audio clipping.
21. **Line 21:** `wav_b64 = base64.b64encode(samples_int16.astype(np.int16).tobytes()).decode("ascii")`
16-bit integer type. The standard format for audio samples in TTS output.
22. **Line 22:** `# App → decode from base64`
A comment: *App → decode from base64.*
23. **Line 23:** `raw = base64.b64decode(resp["wav"])`
Assigns `base64.b64decode(resp["wav"])` to `raw`.
24. **Line 24:** `samples = np.frombuffer(raw, dtype=np.int16).copy()`
Creates an array from a binary buffer. Used for decoding raw audio data.

37.3 WAV File Writing

```
import wave

def write_wav(path: str, samples: np.ndarray, sample_rate: int = 24000):
    """Write int16 samples to a WAV file."""
    with wave.open(path, "wb") as wf:
        wf.setnchannels(1)          # Mono
        wf.setsampwidth(2)         # 16-bit = 2 bytes
        wf.setframerate(sample_rate)
        wf.writeframes(samples.tobytes())
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `import wave`
Imports a module.
2. **Line 2:** `def write_wav(path: str, samples: np.ndarray, sample_rate: int = 24000):`
Defines a function.
3. **Line 3:** `"""Write int16 samples to a WAV file."""`
Docstring: *Write int16 samples to a WAV file..* Documents the following code.
4. **Line 4:** `with wave.open(path, "wb") as wf:`
Opens a file and returns a file object.
5. **Line 5:** `wf.setnchannels(1) # Mono`
`wf.setnchannels(1) # Mono`
6. **Line 6:** `wf.setsampwidth(2) # 16-bit = 2 bytes`
`wf.setsampwidth(2) # 16-bit = 2 bytes`
7. **Line 7:** `wf.setframerate(sample_rate)`
`wf.setframerate(sample_rate)`
8. **Line 8:** `wf.writeframes(samples.tobytes())`
`wf.writeframes(samples.tobytes())`

37.4 Key Takeaways

- **NumPy int16 arrays** are the universal audio format in AIVS
 - **24 kHz sample rate** is the standard for TTS output
 - **Base64 encoding** is used to pass audio between subprocesses
 - **np.clip()** prevents audio clipping (values outside -32768..32767)
 - **np.interp()** is used for pitch shifting (resampling)
-

Requests & HTTP

Learning Objectives

- Use the `requests` library for HTTP operations
 - Download files with progress reporting
 - See the AIVS model downloader
-

38.1 HTTP Basics with requests

```
import requests

# GET request
resp = requests.get("https://api.example.com/voices")
print(resp.status_code) # 200
print(resp.json())      # Parsed JSON

# POST request
resp = requests.post(url, json={"text": "Hello"})

# Download with streaming
with requests.get(url, stream=True) as resp:
    total = int(resp.headers.get("content-length", 0))
    downloaded = 0
    with open(dest, "wb") as f:
        for chunk in resp.iter_content(chunk_size=8192):
            f.write(chunk)
            downloaded += len(chunk)

# Error handling
try:
    resp = requests.get(url, timeout=30)
    resp.raise_for_status() # Raises HTTPError for 4xx/5xx
except requests.ConnectionError:
    print("Connection failed")
except requests.Timeout:
    print("Request timed out")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `import requests`
Imports a module.
2. **Line 2:** `# GET request`
A comment: *GET request*.
3. **Line 3:** `resp = requests.get("https://api.example.com/voices")`
Sends an HTTP GET request. Returns a Response object. Used for downloading voice models and checking API endpoints.
4. **Line 4:** `print(resp.status_code) # 200`
The HTTP status code of the response (200=OK, 404=Not Found, 500=Server Error).
5. **Line 5:** `print(resp.json()) # Parsed JSON`
Parses the response body as JSON and returns a Python dict/list.
6. **Line 6:** `# POST request`
A comment: *POST request*.
7. **Line 7:** `resp = requests.post(url, json={"text": "Hello"})`
Sends an HTTP POST request. Used for submitting data to APIs.
8. **Line 8:** `# Download with streaming`
A comment: *Download with streaming*.
9. **Line 9:** `with requests.get(url, stream=True) as resp:`
Sends an HTTP GET request. Returns a Response object. Used for downloading voice models and checking API endpoints.
10. **Line 10:** `total = int(resp.headers.get("content-length", 0))`
Dictionary of response headers.
11. **Line 11:** `downloaded = 0`
Assigns `0` to `downloaded`.
12. **Line 12:** `with open(dest, "wb") as f:`
Opens a file using a context manager. Auto-closes when the block exits.
13. **Line 13:** `for chunk in resp.iter_content(chunk_size=8192):`
The number of bytes to read at a time when streaming downloads.
14. **Line 14:** `f.write(chunk)`

```
f.write(chunk)
```

15. **Line 15:** `downloaded += len(chunk)`

Returns the number of items in a sequence (string, list, dict, tuple).

16. **Line 16:** `# Error handling`

A comment: *Error handling*.

17. **Line 17:** `try:`

Begins exception handling.

18. **Line 18:** `resp = requests.get(url, timeout=30)`

Sends an HTTP GET request. Returns a Response object. Used for downloading voice models and checking API endpoints.

19. **Line 19:** `resp.raise_for_status() # Raises HTTPError for 4xx/5xx`

Raises an HTTPError for bad status codes (4xx, 5xx).

20. **Line 20:** `except requests.ConnectionError:`

Catches `requests` exceptions.

21. **Line 21:** `print("Connection failed")`

Outputs text to the console. The primary way to display information during development.

22. **Line 22:** `except requests.Timeout:`

Returns the transposed array (swaps rows and columns for 2D arrays).

23. **Line 23:** `print("Request timed out")`

Outputs text to the console. The primary way to display information during development.

38.2 AIVS Model Downloader

```
# From tts/downloader.py
def _download_file(url, dest, progress=None, cancel_event=None):
    """Download a file with progress and cancellation."""
    with requests.get(url, stream=True, timeout=60) as resp:
        resp.raise_for_status()
        total = int(resp.headers.get("content-length", 0))
        downloaded = 0
        with open(dest, "wb") as f:
            for chunk in resp.iter_content(chunk_size=65536):
                if cancel_event and cancel_event.is_set():
                    raise DownloadCancelled()
                f.write(chunk)
                downloaded += len(chunk)
                if progress:
                    progress("downloading", downloaded, total)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From tts/downloader.py`
A comment: *From tts/downloader.py*.
2. **Line 2:** `def _download_file(url, dest, progress=None, cancel_event=None):`
Defines a function.
3. **Line 3:** `"""Download a file with progress and cancellation."""`
Docstring: *Download a file with progress and cancellation.. Documents the following code.*
4. **Line 4:** `with requests.get(url, stream=True, timeout=60) as resp:`
Sends an HTTP GET request. Returns a Response object. Used for downloading voice models and checking API endpoints.
5. **Line 5:** `resp.raise_for_status()`
Raises an HTTPError for bad status codes (4xx, 5xx).
6. **Line 6:** `total = int(resp.headers.get("content-length", 0))`
Dictionary of response headers.
7. **Line 7:** `downloaded = 0`
Assigns `0` to `downloaded`.
8. **Line 8:** `with open(dest, "wb") as f:`
Opens a file using a context manager. Auto-closes when the block exits.
9. **Line 9:** `for chunk in resp.iter_content(chunk_size=65536):`
The number of bytes to read at a time when streaming downloads.
10. **Line 10:** `if cancel_event and cancel_event.is_set():`
Conditional check.
11. **Line 11:** `raise DownloadCancelled()`
Manually raises an exception.
12. **Line 12:** `f.write(chunk)`
`f.write(chunk)`
13. **Line 13:** `downloaded += len(chunk)`
Returns the number of items in a sequence (string, list, dict, tuple).
14. **Line 14:** `if progress:`

Conditional check.

15. **Line 15:** `progress("downloading", downloaded, total)`

Calls function `progress()` .

38.3 Key Takeaways

- **requests.get(url, stream=True)** for large file downloads
 - **iter_content(chunk_size)** for memory-efficient streaming
 - **raise_for_status()** converts HTTP errors to exceptions
 - AIVS downloads TTS voice models (50-500 MB) with progress callbacks
 - **Cancellation** is checked between each chunk via `threading.Event`
-

sherpa-onnx TTS Engine

Learning Objectives

- Understand sherpa-onnx's role in AIVS
 - Configure and use the TTS engine
 - See how multiple TTS engines are supported
-

39.1 What Is sherpa-onnx?

sherpa-onnx is a C++ speech processing library with Python bindings, using the ONNX runtime for inference. It supports multiple TTS model formats:

- **VITS** — used by Piper voices
 - **Kokoro** — multi-speaker multi-language
 - **Matcha** — flow-matching TTS with vocoder
 - **Kitten** — small efficient models
-

39.2 TTS Engine Wrapper in AI/VS

```
# From tts/engine.py - simplified
import sherpa_onnx
import numpy as np

class TtsEngine:
    """Wrapper around sherpa-onnx TTS."""
    sample_rate = 24000

    def __init__(self, voice_entry: dict):
        files = resolve_voice_files(voice_entry)
        config = sherpa_onnx.OfflineTtsConfig(
            model=sherpa_onnx.OfflineTtsModelConfig(
                vits=sherpa_onnx.OfflineTtsVitsModelConfig(
                    model=files["model"],
                    tokens=files["tokens"],
                    data_dir=files.get("data_dir", ""),
                    lexicon=files.get("lexicon", ""),
                    dict_dir=files.get("dict_dir", ""),
                ),
                num_threads=1,
            ),
            max_num_sentences=1,
        )
        self._tts = sherpa_onnx.OfflineTts(config)

    def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):
        if not text.strip():
            raise ValueError("Nothing to synthesize")

        # Core synthesis
        audio = self._tts.generate(text=text, sid=sid, speed=speed)
        samples = np.array(audio.samples, dtype=np.int16)

        # Apply post-processing
        if pitch != 1.0:
            samples = _shift_pitch(samples, pitch)
        if volume != 1.0:
            samples = _apply_volume(samples, volume)

        return samples

# Engine caching (one engine per voice)
_engine_cache: dict[str, TtsEngine] = {}

def get_engine(voice_entry: dict) -> TtsEngine:
    """Get or create a cached engine."""
    key = voice_entry.get("voice", "")
    if key not in _engine_cache:
        _engine_cache[key] = TtsEngine(voice_entry)
    return _engine_cache[key]
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From tts/engine.py - simplified`
A comment: *From tts/engine.py - simplified.*
2. **Line 2:** `import sherpa_onnx`
Imports a module.
3. **Line 3:** `import numpy as np`
Imports a module.
4. **Line 4:** `class TtsEngine:`
Defines a new class.
5. **Line 5:** `"""Wrapper around sherpa-onnx TTS."""`
Docstring: *Wrapper around sherpa-onnx TTS.*. Documents the following code.
6. **Line 6:** `sample_rate = 24000`
Assigns `24000` to `sample_rate`.
7. **Line 7:** `def __init__(self, voice_entry: dict):`
Constructor method. Called when creating a new instance of a class.
8. **Line 8:** `files = resolve_voice_files(voice_entry)`
Assigns `resolve_voice_files(voice_entry)` to `files`.
9. **Line 9:** `config = sherpa_onnx.OfflineTtsConfig(`
Configuration container for the offline TTS engine. Specifies model, rule_fsts, and max_num_sentences.
10. **Line 10:** `model=sherpa_onnx.OfflineTtsModelConfig(`
Configuration for the TTS model. Specifies which model type (VITS, Kokoro, Matcha) to use.
11. **Line 11:** `vits=sherpa_onnx.OfflineTtsVitsModelConfig(`
Configuration for VITS-style models. Requires model, tokens, and optionally lexicon, data_dir.
12. **Line 12:** `model=files["model"],`
Path to the ONNX model file. The main neural network weights.
13. **Line 13:** `tokens=files["tokens"],`
Path to the tokens file. Defines the vocabulary the model uses.
14. **Line 14:** `data_dir=files.get("data_dir", ""),`
Path to espeak-ng data directory (for Kokoro models).

15. **Line 15:** `lexicon=files.get("lexicon", ""),`
Path to a pronunciation lexicon file. Maps words to phoneme sequences.
16. **Line 16:** `dict_dir=files.get("dict_dir", ""),`
Assigns `files.get("dict_dir", "")` to `dict_dir`.
17. **Line 17:** `),`
`),`
18. **Line 18:** `num_threads=1,`
Number of CPU threads for inference. Higher values use more CPU but may be faster.
19. **Line 19:** `),`
`),`
20. **Line 20:** `max_num_sentences=1,`
Maximum number of sentences to synthesize in one call. Higher values use more memory.
21. **Line 21:** `)`
`)`
22. **Line 22:** `self._tts = sherpa_onnx.OfflineTts(config)`
Creates an offline TTS engine from a configuration. The main entry point for text-to-speech synthesis.
23. **Line 23:** `def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):`
Defines a function.
24. **Line 24:** `if not text.strip():`
Conditional check.
25. **Line 25:** `raise ValueError("Nothing to synthesize")`
Manually raises an exception.
26. **Line 26:** `# Core synthesis`
A comment: *Core synthesis*.
27. **Line 27:** `audio = self._tts.generate(text=text, sid=sid, speed=speed)`
Generates audio from text. Returns a GeneratedAudio object with `.samples` and `.sample_rate`.
28. **Line 28:** `samples = np.array(audio.samples, dtype=np.int16)`
Creates a NumPy array from a list or tuple. The dtype parameter specifies the data type (e.g., `np.int16` for 16-bit audio samples).

29. **Line 29:** `# Apply post-processing`
A comment: *Apply post-processing.*

30. **Line 30:** `if pitch != 1.0:`
Conditional check.

31. **Line 31:** `samples = _shift_pitch(samples, pitch)`
Assigns `_shift_pitch(samples, pitch)` to `samples`.

32. **Line 32:** `if volume != 1.0:`
Conditional check.

33. **Line 33:** `samples = _apply_volume(samples, volume)`
Assigns `_apply_volume(samples, volume)` to `samples`.

34. **Line 34:** `return samples`
Returns a value from a function.

35. **Line 35:** `# Engine caching (one engine per voice)`
A comment: *Engine caching (one engine per voice).*

36. **Line 36:** `_engine_cache: dict[str, TtsEngine] = {}`
`_engine_cache: dict[str, TtsEngine] = {}`

37. **Line 37:** `def get_engine(voice_entry: dict) -> TtsEngine:`
Begins exception handling.

38. **Line 38:** `"""Get or create a cached engine."""`
Docstring: *Get or create a cached engine..* Documents the following code.

39. **Line 39:** `key = voice_entry.get("voice", "")`
Assigns `voice_entry.get("voice", "")` to `key`.

40. **Line 40:** `if key not in _engine_cache:`
Conditional check.

41. **Line 41:** `_engine_cache[key] = TtsEngine(voice_entry)`
`_engine_cache[key] = TtsEngine(voice_entry)`

42. **Line 42:** `return _engine_cache[key]`
Returns a value from a function.

39.3 Voice Catalog Structure

```
# The catalog maps TTS engines to languages, variants, and voices
# From tts/catalog.py (simplified)
_CATALOG = (
    {
        "id": "piper",
        "name": "Piper TTS",
        "engine": "vits",
        "voice_cloning": True,
        "languages": [
            {
                "code": "en_US",
                "name": "English (US)",
                "variants": [
                    {
                        "id": "medium",
                        "name": "Medium quality",
                        "voices": [
                            {"id": "af", "name": "AF", "sid": 0},
                            {"id": "af_bella", "name": "Bella", "sid": 2},
                            # ... more voices
                        ],
                    },
                ],
            },
        ],
    },
    # ... Kokoro, Matcha, Kitten entries
)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# The catalog maps TTS engines to languages, variants, and voices`

A comment: *The catalog maps TTS engines to languages, variants, and voices.*

2. **Line 2:** `# From tts/catalog.py (simplified)`

A comment: *From tts/catalog.py (simplified).*

3. **Line 3:** `_CATALOG = (`

Assigns `(` to `_CATALOG` .

4. **Line 4:** `{`

`{`

5. **Line 5:** `"id": "piper",`

`"id": "piper",`

6. **Line 6:** `"name": "Piper TTS",`

`"name": "Piper TTS",`

7. **Line 7:** `"engine": "vits",`

`"engine": "vits",`

8. **Line 8:** `"voice_cloning": True,`

`"voice_cloning": True,`

9. **Line 9:** `"languages": [`

`"languages": [`

10. **Line 10:** `{`

`{`

11. **Line 11:** `"code": "en_US",`

`"code": "en_US",`

12. **Line 12:** `"name": "English (US)",`

`"name": "English (US)",`

13. **Line 13:** `"variants": [`

`"variants": [`

```
14. Line 14: {
    {

15. Line 15:     "id": "medium",
    "id": "medium",

16. Line 16:     "name": "Medium quality",
    "name": "Medium quality",

17. Line 17:     "voices": [
    "voices": [

18. Line 18:         {"id": "af", "name": "AF", "sid": 0},
        {"id": "af", "name": "AF", "sid": 0},

19. Line 19:         {"id": "af_bella", "name": "Bella", "sid": 2},
        {"id": "af_bella", "name": "Bella", "sid": 2},

20. Line 20:     # ... more voices
    A comment: ... more voices.

21. Line 21: ],
    ],

22. Line 22: },
    },

23. Line 23: ],
    ],

24. Line 24: },
    },

25. Line 25: ],
    ],

26. Line 26: },
    },

27. Line 27: # ... Kokoro, Matcha, Kitten entries
```


A comment: ... *Kokoro, Matcha, Kitten* entries.

28. Line 28: `)`

39.4 Key Takeaways

- **sherpa-onnx** is the primary TTS engine — supports VITS, Kokoro, Matcha, Kitten
 - **Engine caching** prevents reloading models for each synthesis call
 - **resolve_voice_files()** maps voice entries to actual file paths
 - The **catalog** is a nested dict: TTS → language → variant → voice
 - Post-processing (pitch, volume) is applied **after** synthesis
-

PDF & DOCX Parsing

Learning Objectives

- Extract text from PDF files with pypdf
 - Extract text from DOCX files with python-docx
 - See the AIVS document parser system
-

40.1 PDF Parsing with pypdf

```
from pypdf import PdfReader

reader = PdfReader("document.pdf")
print(f"Pages: {len(reader.pages)}")

# Extract text from each page
for i, page in enumerate(reader.pages):
    text = page.extract_text()
    print(f"Page {i+1}: {len(text)} chars")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `from pypdf import PdfReader`
Imports a module.
2. **Line 2:** `reader = PdfReader("document.pdf")`
Creates a PDF reader object. Use `len(reader)` for page count; iterate for pages.
3. **Line 3:** `print(f"Pages: {len(reader.pages)}")`
List of Page objects in a PDF reader. Each page has `extract_text()`.
4. **Line 4:** `# Extract text from each page`
A comment: *Extract text from each page.*
5. **Line 5:** `for i, page in enumerate(reader.pages):`
Adds a counter to an iterable, returning (index, value) pairs.
6. **Line 6:** `text = page.extract_text()`
Extracts text content from a PDF page. Returns a string.
7. **Line 7:** `print(f"Page {i+1}: {len(text)} chars")`
Outputs text to the console. The primary way to display information during development.

40.2 DOCX Parsing with python-docx

```
from docx import Document

doc = Document("document.docx")

# Extract paragraphs
for para in doc.paragraphs:
    if para.text.strip():
        style = para.style.name # "Heading 1", "Normal", etc.
        print(f"[{style}] {para.text[:80]}")

# Extract tables
for table in doc.tables:
    for row in table.rows:
        cells = [cell.text for cell in row.cells]
        print(" | ".join(cells))
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `from docx import Document`
Imports a module.
2. **Line 2:** `doc = Document("document.docx")`
Opens a Word document (DOCX). Access paragraphs via doc.paragraphs.
3. **Line 3:** `# Extract paragraphs`
A comment: *Extract paragraphs*.
4. **Line 4:** `for para in doc.paragraphs:`
List of Paragraph objects in a Word document. Each paragraph has .text and .style.
5. **Line 5:** `if para.text.strip():`
The text content of a paragraph or page.
6. **Line 6:** `style = para.style.name # "Heading 1", "Normal", etc.`
The style of a paragraph (e.g., 'Heading 1', 'Normal').
7. **Line 7:** `print(f"[{style}] {para.text[:80]}")`
Outputs text to the console. The primary way to display information during development.
8. **Line 8:** `# Extract tables`
A comment: *Extract tables*.
9. **Line 9:** `for table in doc.tables:`
For loop iteration.
10. **Line 10:** `for row in table.rows:`
For loop iteration.
11. **Line 11:** `cells = [cell.text for cell in row.cells]`
The text content of a paragraph or page.
12. **Line 12:** `print(" | ".join(cells))`
Outputs text to the console. The primary way to display information during development.

40.3 AIVS Document Parser

```
# From documents/parsers.py
class ParseError(Exception):
    """Raised when a document cannot be parsed."""

def parse_document(path: str) -> Document:
    """Parse a document and return structured content."""
    ext = os.path.splitext(path)[1].lower()
    if ext == ".pdf":
        return _parse_pdf(path)
    elif ext == ".docx":
        return _parse_docx(path)
    elif ext in (".txt", ".md"):
        return _parse_text(path)
    elif ext in (".html", ".htm"):
        return _parse_html(path)
    else:
        raise ParseError(f"Unsupported file type: {ext}")

class Document:
    """Parsed document with blocks of text."""
    def __init__(self):
        self.blocks: list[TextBlock] = []
        self.metadata: dict = {}

class TextBlock:
    """A section of text with a heading and content."""
    def __init__(self, heading: str, text: str, level: int = 0):
        self.heading = heading
        self.text = text
        self.level = level
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From documents/parsers.py`
A comment: *From documents/parsers.py*.
2. **Line 2:** `class ParseError(Exception):`
Defines a new class.
3. **Line 3:** `"""Raised when a document cannot be parsed."""`
Docstring: *Raised when a document cannot be parsed..* Documents the following code.
4. **Line 4:** `def parse_document(path: str) -> Document:`
Defines a function.
5. **Line 5:** `"""Parse a document and return structured content."""`
Docstring: *Parse a document and return structured content..* Documents the following code.
6. **Line 6:** `ext = os.path.splitext(path)[1].lower()`
Assigns `os.path.splitext(path)[1].lower()` to `ext`.
7. **Line 7:** `if ext == ".pdf":`
Conditional check.
8. **Line 8:** `return _parse_pdf(path)`
Returns a value from a function.
9. **Line 9:** `elif ext == ".docx":`
Alternative condition.
10. **Line 10:** `return _parse_docx(path)`
Returns a value from a function.
11. **Line 11:** `elif ext in (".txt", ".md"):`
Alternative condition.
12. **Line 12:** `return _parse_text(path)`
Returns a value from a function.
13. **Line 13:** `elif ext in (".html", ".htm"):`
Alternative condition.
14. **Line 14:** `return _parse_html(path)`
Returns a value from a function.

15. **Line 15:** `else:`
Fallback branch.
16. **Line 16:** `raise ValueError(f"Unsupported file type: {ext}")`
Manually raises an exception.
17. **Line 17:** `class Document:`
Defines a new class.
18. **Line 18:** `"""Parsed document with blocks of text."""`
Docstring: *Parsed document with blocks of text..* Documents the following code.
19. **Line 19:** `def __init__(self):`
Constructor method. Called when creating a new instance of a class.
20. **Line 20:** `self.blocks: list[TextBlock] = []`
`self.blocks: list[TextBlock] = []`
21. **Line 21:** `self.metadata: dict = {}`
`self.metadata: dict = {}`
22. **Line 22:** `class TextBlock:`
Defines a new class.
23. **Line 23:** `"""A section of text with a heading and content."""`
Docstring: *A section of text with a heading and content..* Documents the following code.
24. **Line 24:** `def __init__(self, heading: str, text: str, level: int = 0):`
Constructor method. Called when creating a new instance of a class.
25. **Line 25:** `self.heading = heading`
Stores `heading` in instance attribute `heading`.
26. **Line 26:** `self.text = text`
The text content of a paragraph or page.
27. **Line 27:** `self.level = level`
Stores `level` in instance attribute `level`.

40.4 Key Takeaways

- **pypdf** extracts text from PDF files page by page
 - **python-docx** extracts paragraphs and tables from DOCX files
 - AIVS supports 7 **file formats**: PDF, DOCX, TXT, MD, HTML, HTM, EPUB
 - The parser returns a **Document** with structured **TextBlocks**
 - **ParseError** is raised for unsupported or corrupt files
-

FFmpeg & Audio Formats

Learning Objectives

- Understand audio format differences (WAV, MP3, FLAC)
 - Use FFmpeg for audio conversion
 - See the AIVS FFmpeg management system
-

41.1 Audio Formats

Format	Type	Size	Quality	FFmpeg Required?
WAV	Uncompressed	Large	Lossless	No (Python wave module)
MP3	Compressed	Small	Lossy	Yes
FLAC	Compressed	Medium	Lossless	Yes

41.2 AIVS FFmpeg Management

```
# From audio/ffmpeg.py
def ensure_ffmpeg() -> str | None:
    """Find or return None if FFmpeg is not installed."""
    ffmpeg_path = os.path.join(paths.user_data_dir(), "ffmpeg", "ffmpeg.exe")
    if os.path.isfile(ffmpeg_path):
        return ffmpeg_path
    return None

def convert_wav(input_path, output_path, format, ffmpeg_exe):
    """Convert WAV to MP3 or FLAC using FFmpeg."""
    import subprocess
    cmd = [ffmpeg_exe, "-i", input_path, "-y", output_path]
    subprocess.run(cmd, capture_output=True, check=True)

# Usage in RecordingDialog
if fmt != "wav":
    ffmpeg_exe = ffmpeg_mod.ensure_ffmpeg()
    if not ffmpeg_exe:
        # Offer to download FFmpeg
        answer = wx.MessageBox(
            "FFmpeg is required for MP3/FLAC. Download now?",
            "FFmpeg required",
            style=wx.YES_NO | wx.ICON_QUESTION,
        )
        if answer == wx.YES:
            ffmpeg_mod.download_ffmpeg()
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From audio/ffmpeg.py`
A comment: *From audio/ffmpeg.py*.
2. **Line 2:** `def ensure_ffmpeg() -> str | None:`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.
3. **Line 3:** `"""Find or return None if FFmpeg is not installed."""`
Docstring: *Find or return None if FFmpeg is not installed..* Documents the following code.
4. **Line 4:** `ffmpeg_path = os.path.join(paths.user_data_dir(), "ffmpeg", "ffmpeg.exe")`
The FFmpeg binary bundled with AIVS. Located in %APPDATA%/AIVoiceStudio/ffmpeg/.
5. **Line 5:** `if os.path.isfile(ffmpeg_path):`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.
6. **Line 6:** `return ffmpeg_path`
Returns a value from a function.
7. **Line 7:** `return None`
Returns a value from a function.
8. **Line 8:** `def convert_wav(input_path, output_path, format, ffmpeg_exe):`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.
9. **Line 9:** `"""Convert WAV to MP3 or FLAC using FFmpeg."""`
Docstring: *Convert WAV to MP3 or FLAC using FFmpeg..* Documents the following code.
10. **Line 10:** `import subprocess`
Imports a module.
11. **Line 11:** `cmd = [ffmpeg_exe, "-i", input_path, "-y", output_path]`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.
12. **Line 12:** `subprocess.run(cmd, capture_output=True, check=True)`
Captures stdout and stderr. Available in subprocess.run().
13. **Line 13:** `# Usage in RecordingDialog`
A comment: *Usage in RecordingDialog*.
14. **Line 14:** `if fmt != "wav":`
Conditional check.

15. **Line 15:** `ffmpeg_exe = ffmpeg_mod.ensure_ffmpeg()`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.

16. **Line 16:** `if not ffmpeg_exe:`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.

17. **Line 17:** `# Offer to download FFmpeg`
A comment: *Offer to download FFmpeg.*

18. **Line 18:** `answer = wx.MessageBox(`
Assigns `wx.MessageBox(` to `answer` .

19. **Line 19:** `"FFmpeg is required for MP3/FLAC. Download now?",`
For loop iteration.

20. **Line 20:** `"FFmpeg required",`
"FFmpeg required",

21. **Line 21:** `style=wx.YES_NO | wx.ICON_QUESTION,`
Assigns `wx.YES_NO | wx.ICON_QUESTION,` to `style` .

22. **Line 22:** `)`
)

23. **Line 23:** `if answer == wx.YES:`
Conditional check.

24. **Line 24:** `ffmpeg_mod.download_ffmpeg()`
A command-line tool for audio/video processing. Used by AIVS for WAV-to-MP3/FLAC conversion.

41.3 Key Takeaways

- **WAV** is always available (Python stdlib); **MP3/FLAC** need FFmpeg
 - FFmpeg is downloaded on-demand to the user's AppData folder
 - AIVS uses **subprocess.run()** to invoke FFmpeg for conversion
 - The **format selection** happens in the Recording Dialog
-

Virtual Environments & Packaging

i Info: Current implementation (2026.3.1). The application no longer has a single managed environment. Every TTS engine that is installed with pip gets its *own* virtualenv under `%APPDATA%\AIVoiceStudio\tts_envs\<engine>` (`tts_envs\pocket_tts` , `tts_envs\bark` , `tts_envs\f5tts`), the two OmniVoice engines deliberately share `tts_envs\omnivoice` , and addons keep the original shared `addon_env` . Engines are installed with the CUDA build of PyTorch from the one index constant `voicelab.engines.PYTORCH_CUDA_INDEX` (`cu128` — the lowest index that still builds for the Python 3.13 the application runs on), and each install ends with an import check (`voicelab.verify_engine_import`). See [Chapter 77](#) for the full design.

Learning Objectives

- Understand virtual environments and why they matter
- Create and manage venvs
- See the AIVS PythonRuntime managed environment

42.1 Why Virtual Environments?

Without venvs, installing `wxPython` for one project might overwrite a package needed by another. Virtual environments create isolated Python installations:

```
# Create
python -m venv .venv

# Activate (Windows)
.venv\Scripts\activate

# Activate (macOS/Linux)
source .venv/bin/activate

# Install packages (only in this venv)
pip install wxPython numpy sherpa-onnx

# Deactivate
deactivate
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Create`

A comment: *Create*.

2. **Line 2:** `python -m venv .venv`

`python -m venv .venv`

3. **Line 3:** `# Activate (Windows)`

A comment: *Activate (Windows)*.

4. **Line 4:** `.venv\Scripts\activate`

`.venv\Scripts\activate`

5. **Line 5:** `# Activate (macOS/Linux)`

A comment: *Activate (macOS/Linux)*.

6. **Line 6:** `source .venv/bin/activate`

`source .venv/bin/activate`

7. **Line 7:** `# Install packages (only in this venv)`

A comment: *Install packages (only in this venv)*.

8. **Line 8:** `pip install wxPython numpy sherpa-onnx`

`pip install wxPython numpy sherpa-onnx`

9. **Line 9:** `# Deactivate`

A comment: *Deactivate*.

10. **Line 10:** `deactivate`

`deactivate`

42.2 AIVS PythonRuntime

```
# From python_runtime/__init__.py
class PythonRuntime:
    """Manages a separate Python venv for addons and optional engines."""

    def __init__(self, env_dir=None):
        self._env_dir = env_dir or os.path.join(
            paths.user_data_dir(), "addon_env"
        )

    def ensure_env(self) -> str:
        """Create the venv if it doesn't exist."""
        if self.is_created:
            return self._env_dir
        subprocess.run(
            [self.system_python, "-m", "venv", self._env_dir, "--clear"],
            check=True,
        )
        return self._env_dir

    def pip_install(self, packages, progress=None, cancel_event=None):
        """Install packages into the managed venv."""
        pip = self.ensure_pip()
        cmd = [pip, "install", "--no-warn-script-location"] + packages
        proc = subprocess.Popen(cmd, stdout=subprocess.PIPE, text=True)
        for line in proc.stdout:
            if progress:
                progress(line, 0, 0)
            if cancel_event and cancel_event.is_set():
                proc.terminate()
                return {"ok": False, "error": "Cancelled"}
        proc.wait()
        return {"ok": proc.returncode == 0}

    def run_in_env(self, script: str):
        """Run a Python script inside the managed venv."""
        return subprocess.run(
            [self.python_exe, "-c", script],
            capture_output=True, text=True,
        )
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From python_runtime/__init__.py`
A comment: *From python_runtime/__init__.py.*
2. **Line 2:** `class PythonRuntime:`
Manages the embedded Python interpreter for addons. Creates isolated venvs.
3. **Line 3:** `"""Manages a separate Python venv for addons and optional engines."""`
Docstring: *Manages a separate Python venv for addons and optional engines..* Documents the following code.
4. **Line 4:** `def __init__(self, env_dir=None):`
Constructor method. Called when creating a new instance of a class.
5. **Line 5:** `self._env_dir = env_dir or os.path.join(`
Stores `env_dir or os.path.join(` in instance attribute `_env_dir`.
6. **Line 6:** `paths.user_data_dir(), "addon_env"`
`paths.user_data_dir(), "addon_env"`
7. **Line 7:** `)`
`)`
8. **Line 8:** `def ensure_env(self) -> str:`
Defines a function.
9. **Line 9:** `"""Create the venv if it doesn't exist."""`
Docstring: *Create the venv if it doesn't exist..* Documents the following code.
10. **Line 10:** `if self.is_created:`
Conditional check.
11. **Line 11:** `return self._env_dir`
Returns a value from a function.
12. **Line 12:** `subprocess.run(`
Runs a command, waits for completion. Returns CompletedProcess. Use `capture_output=True` to capture stdout/stderr.
13. **Line 13:** `[self.system_python, "-m", "venv", self._env_dir, "--clear"],`
`[self.system_python, "-m", "venv", self._env_dir, "--clear"],`
14. **Line 14:** `check=True,`

Assigns `True`, to `check` .

15. Line 15: `)`
`)`

16. Line 16: `return self._env_dir`
Returns a value from a function.

17. Line 17: `def pip_install(self, packages, progress=None, cancel_event=None):`
Returns `True` if all elements in the iterable are `truthy`.

18. Line 18: `"""Install packages into the managed venv."""`
Docstring: *Install packages into the managed venv..* Documents the following code.

19. Line 19: `pip = self.ensure_pip()`
Assigns `self.ensure_pip()` to `pip` .

20. Line 20: `cmd = [pip, "install", "--no-warn-script-location"] + packages`
Assigns `[pip, "install", "--no-warn-script-location"] + packages` to `cmd` .

21. Line 21: `proc = subprocess.Popen(cmd, stdout=subprocess.PIPE, text=True)`
Starts a process. More control than `run()` — can read `stdout/stderr` asynchronously.

22. Line 22: `for line in proc.stdout:`
For loop iteration.

23. Line 23: `if progress:`
Conditional check.

24. Line 24: `progress(line, 0, 0)`
Calls function `progress()` .

25. Line 25: `if cancel_event and cancel_event.is_set():`
Conditional check.

26. Line 26: `proc.terminate()`
`proc.terminate()`

27. Line 27: `return {"ok": False, "error": "Cancelled"}`
Returns a value from a function.

28. Line 28: `proc.wait()`

```
proc.wait()

29. Line 29: return {"ok": proc.returncode == 0}
    Returns a value from a function.

30. Line 30: def run_in_env(self, script: str):
    Defines a function.

31. Line 31: """Run a Python script inside the managed venv."""
    Docstring: Run a Python script inside the managed venv. Documents the following code.

32. Line 32: return subprocess.run(
    Runs a command, waits for completion. Returns CompletedProcess. Use capture_output=True to capture stdout/stderr.

33. Line 33: [self.python_exe, "-c", script],
    [self.python_exe, "-c", script],

34. Line 34: capture_output=True, text=True,
    Captures stdout and stderr. Available in subprocess.run().

35. Line 35: )
    )
```

42.3 Key Takeaways

- **Virtual environments** isolate project dependencies
 - AIVS has **two venvs**: the development venv (requirements.txt) and the managed addon venv
 - The **PythonRuntime** manages the addon venv for pip-installed engines (OmniVoice)
 - **run_in_env()** executes scripts inside the managed venv without affecting the main process
-

Building AI Voice Studio

CHAPTER 43

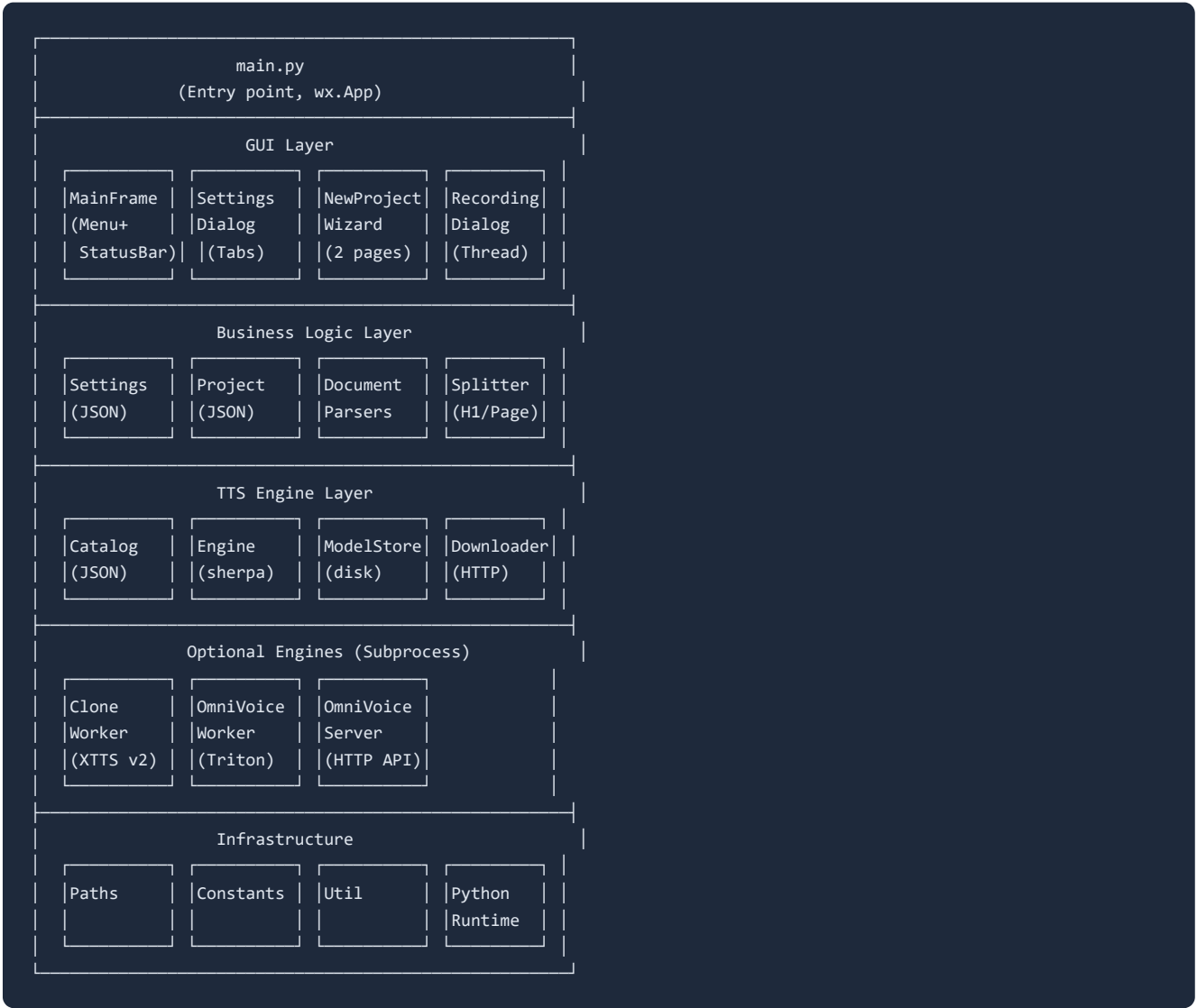
Project Architecture

i Info: Current implementation (2026.3.1). The package has grown these modules since this chapter was written: `project.py` (project.json read/write), `runtime.py` (sherpa-onnx loading and providers), `heavy_logging.py`, `venv_packages.py` (per-environment package probes), `python_runtime/` (one managed virtualenv per TTS engine), `voicelab/` (Pocket TTS, Bark, F5-TTS + their worker), `omnivoice/`, `omnivoice_server/`, `addons/`, `tts/windows_tts.py` (SAPI5 / Windows Core voices), `documents/playlist_builder.py`, `documents/daisy_builder.py` and `documents/daisy3_builder.py`, and in the GUI `gui/compute_choice.py` (the per-category Compute combo), `gui/accept_dialog.py` (first-launch terms), `gui/dialogs.py`, `gui/progress.py`, `gui/events.py` and `gui/clone_engines_panel.py` / `gui/voicelab_options_dialog.py` / `gui/omnivoice_options_dialog.py`. The repository now also carries `docs/book` (the bundled book), `tools/` (accessibility audits and end-to-end probes) and 22 test files. `main.py` starts with the terms-of-use gate before any subsystem is built. See [Chapters 76-82](#) for the parts the rest of this book predates.

Learning Objectives

- Understand the complete architecture of AIVS
 - Map every module to its responsibility
 - Understand the data flow from document → audio
-

43.1 High-Level Architecture



Architecture Diagram (Block 1)

This is an ASCII art diagram showing the AI Voice Studio architecture. The boxes represent modules and the connections show data flow between layers. Refer to Chapter 43 for a detailed explanation of each component.

43.2 Data Flow

```
graph TD
    A[User opens document (PDF/DOCX/TXT)] --> B[Document Parser (parsers.py)]
    B --> C[Document Splitter (splitter.py)]
    C --> D[Project Creator (project.py)]
    D --> E[Recording Dialog (recording_dialog.py)]
    E --> F[Synthesis Worker (synthesizer.py) - Background Thread]
    F --> G[DAISY Builder (daisy_builder.py) - If DAISY project]
    G --> H[Audio files in project directory]
```

User opens document (PDF/DOCX/TXT)

↓

Document Parser (parsers.py)

 | Extract text blocks with headings

↓

Document Splitter (splitter.py)

 | Split into segments by page/heading

↓

Project Creator (project.py)

 | Save segments to project.json

↓

Recording Dialog (recording_dialog.py)

 | User selects voice, rate, pitch

↓

Synthesis Worker (synthesizer.py) – Background Thread

 | For each segment:

 | 1. Get TTS engine (engine.py)

 | 2. Synthesize text → numpy array

 | 3. Apply pitch/volume adjustments

 | 4. Save to WAV (or convert to MP3/FLAC)

 | 5. Mark segment done in project.json

↓

DAISY Builder (daisy_builder.py) – If DAISY project

 | Generate ncc.html, package.opf, SMIL files

↓

Audio files in project directory

Explain Line by Line Code Description (Block 2)

1. **Line 1:** User opens document (PDF/DOCX/TXT)
User opens document (PDF/DOCX/TXT)
2. **Line 2:** |
3. **Line 3:** ▼
▼
4. **Line 4:** Document Parser (parsers.py)
Document Parser (parsers.py)
5. **Line 5:** | Extract text blocks with headings
| Extract text blocks with headings
6. **Line 6:** ▼
▼
7. **Line 7:** Document Splitter (splitter.py)
Document Splitter (splitter.py)
8. **Line 8:** | Split into segments by page/heading
| Split into segments by page/heading
9. **Line 9:** ▼
▼
10. **Line 10:** Project Creator (project.py)
Project Creator (project.py)
11. **Line 11:** | Save segments to project.json
| Save segments to project.json
12. **Line 12:** ▼
▼
13. **Line 13:** Recording Dialog (recording_dialog.py)
Recording Dialog (recording_dialog.py)

14. Line 14: | User selects voice, rate, pitch
| User selects voice, rate, pitch
15. Line 15: ▼
▼
16. Line 16: Synthesis Worker (synthesizer.py) – Background Thread
Synthesis Worker (synthesizer.py) – Background Thread
17. Line 17: | For each segment:
| For each segment:
18. Line 18: | 1. Get TTS engine (engine.py)
| 1. Get TTS engine (engine.py)
19. Line 19: | 2. Synthesize text → numpy array
| 2. Synthesize text → numpy array
20. Line 20: | 3. Apply pitch/volume adjustments
| 3. Apply pitch/volume adjustments
21. Line 21: | 4. Save to WAV (or convert to MP3/FLAC)
| 4. Save to WAV (or convert to MP3/FLAC)
22. Line 22: | 5. Mark segment done in project.json
| 5. Mark segment done in project.json
23. Line 23: ▼
▼
24. Line 24: DAISY Builder (daisy_builder.py) – If DAISY project
DAISY Builder (daisy_builder.py) – If DAISY project
25. Line 25: | Generate ncc.html, package.opf, SMIL files
| Generate ncc.html, package.opf, SMIL files
26. Line 26: ▼
▼
27. Line 27: Audio files in project directory

Audio files in project directory

43.3 Module Responsibilities

Module	Lines	Responsibility
<code>main.py</code>	~80	Entry point, logging setup, app creation
<code>constants.py</code>	~80	All magic numbers, strings, choices
<code>paths.py</code>	~50	Cross-platform path resolution
<code>settings.py</code>	~120	JSON settings with dot-notation access
<code>project.py</code>	~100	Project save/load, segment management
<code>compute.py</code>	~80	GPU/CPU detection, provider mapping
<code>util.py</code>	~60	Filename sanitization, Python finder
<code>tts/catalog.py</code>	~400	TTS engine catalog (JSON data)
<code>tts/engine.py</code>	~200	Sherpa-onnx engine wrapper
<code>tts/models.py</code>	~250	Model store, file resolution
<code>tts/downloader.py</code>	~200	HTTP model downloads
<code>gui/main_frame.py</code>	~300	Main window, menus, keyboard shortcuts
<code>gui/settings_dialog.py</code>	~250	Multi-tab settings dialog
<code>gui/recording_dialog.py</code>	~400	Recording window with voice selection
<code>gui/new_project_wizard.py</code>	~250	2-page project creation wizard
<code>gui/events.py</code>	~80	Custom wx event types
<code>gui/a11y.py</code>	~50	Accessibility helpers
<code>gui/theme.py</code>	~70	Light/Dark/System theme
<code>jobs/synthesizer.py</code>	~150	Background TTS worker thread
<code>documents/parsers.py</code>	~200	PDF/DOCX/HTML/TXT parsing
<code>documents/splitter.py</code>	~150	Text splitting algorithms
<code>documents/daisy_builder.py</code>	~200	DAISY 2.02 book generation
<code>clone/__init__.py</code>	~250	Voice cloning (XTTS v2)
<code>omnivoice/__init__.py</code>	~200	OmniVoice TTS engine

Module	Lines	Responsibility
<code>addons/__init__.py</code>	~250	Plugin/addon system
<code>python_runtime/__init__.py</code>	~200	Managed Python environment

43.4 Key Takeaways

- AIVS has **53 Python source files** across 13 subpackages
 - The architecture follows **layered design**: GUI → Business Logic → TTS Engine → Infrastructure
 - Heavy TTS engines run in **separate subprocesses** (Clone, OmniVoice)
 - The data flow is: **Document** → **Parser** → **Splitter** → **Project** → **Recording** → **Audio**
 - **JSON** is the universal data format (settings, projects, catalog, worker protocol)
-

Paths, Settings & Constants

i Info: Current implementation (2026.3.1). `paths.py` now also lays out `tts_envs\<engine>` (one Python environment per pip-installed TTS engine) alongside `addon_env`, `runtime`, `ffmpeg` and the model and project folders. `settings.py` gained: `preview_compute` (the Compute choice of each Preview button, keyed by category), `clone_engines` (`engine`, `device`, `options.<engine>`), `omnivoice_server` (host, port, device, steps, concurrency, API key, CORS), `daisy`, `recording.per_tts`, and `terms_version` — the last one written only when the user accepts the first-launch terms, and compared with `constants.TERMS_VERSION` on every start. `constants.py` carries `APP_VERSION` and `TERMS_VERSION`, which the installer version must match.

Learning Objectives

- Understand the AIVS path resolution system
 - Master the Settings class with dot-notation access
 - See how constants organize application configuration
-

44.1 Path Resolution

```
# From paths.py
def user_data_dir() -> str:
    """Return %APPDATA%/AIVoiceStudio on Windows."""
    if sys.platform == "win32":
        base = os.environ.get("APPDATA", os.path.expanduser("~"))
    else:
        base = os.path.join(os.path.expanduser("~"), ".config")
    d = os.path.join(base, "AIVoiceStudio")
    os.makedirs(d, exist_ok=True)
    return d

def models_dir() -> str:
    """Return the directory where TTS models are stored."""
    d = os.path.join(user_data_dir(), "models")
    os.makedirs(d, exist_ok=True)
    return d

def project_dir(name: str) -> str:
    """Return the directory for a project."""
    d = os.path.join(user_data_dir(), "projects", sanitize_filename(name))
    os.makedirs(d, exist_ok=True)
    return d

def settings_file() -> str:
    return os.path.join(user_data_dir(), "settings.json")

def log_file() -> str:
    return os.path.join(user_data_dir(), "app.log")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From paths.py`
A comment indicating the source file: *From paths.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
2. **Line 2:** `def user_data_dir() -> str:`
Returns the per-user application data directory. On Windows this is %APPDATA%/AIVoiceStudio; on Linux/macOS it's ~/.aivoicestudio. The directory is created automatically if it doesn't exist.
3. **Line 3:** `"""Return %APPDATA%/AIVoiceStudio on Windows."""`
Docstring: *Return %APPDATA%/AIVoiceStudio on Windows..* Documents the purpose of the following code block.
4. **Line 4:** `if sys.platform == "win32":`
String identifying the OS: 'win32', 'darwin', 'linux'. Used for platform-specific code paths.
5. **Line 5:** `base = os.environ.get("APPDATA", os.path.expanduser("~"))`
Assigns `os.environ.get("APPDATA", os.path.expanduser("~"))` to `base`.
6. **Line 6:** `else:`
Fallback: executes when none of the above conditions matched.
7. **Line 7:** `base = os.path.join(os.path.expanduser("~"), ".config")`
Assigns `os.path.join(os.path.expanduser("~"), ".config")` to `base`.
8. **Line 8:** `d = os.path.join(base, "AIVoiceStudio")`
Assigns `os.path.join(base, "AIVoiceStudio")` to `d`.
9. **Line 9:** `os.makedirs(d, exist_ok=True)`
`os.makedirs(d, exist_ok=True)`
10. **Line 10:** `return d`
Returns `d` to the caller.
11. **Line 11:** `def models_dir() -> str:`
Returns the directory where downloaded TTS voice models are stored. Respects any custom path set in Settings > Paths. Creates the directory if needed.
12. **Line 12:** `"""Return the directory where TTS models are stored."""`
Docstring: *Return the directory where TTS models are stored..* Documents the purpose of the following code block.
13. **Line 13:** `d = os.path.join(user_data_dir(), "models")`

Returns the per-user application data directory. On Windows this is %APPDATA%/AIVoiceStudio; on Linux/macOS it's ~/.aivoicestudio. The directory is created automatically if it doesn't exist.

14. **Line 14:** `os.makedirs(d, exist_ok=True)`

```
os.makedirs(d, exist_ok=True)
```

15. **Line 15:** `return d`

Returns `d` to the caller.

16. **Line 16:** `def project_dir(name: str) -> str:`

Defines function `project_dir` accepting parameters (name), returning `str`.

17. **Line 17:** `"""Return the directory for a project."""`

Docstring: *Return the directory for a project..* Documents the purpose of the following code block.

18. **Line 18:** `d = os.path.join(user_data_dir(), "projects", sanitize_filename(name))`

Returns the per-user application data directory. On Windows this is %APPDATA%/AIVoiceStudio; on Linux/macOS it's ~/.aivoicestudio. The directory is created automatically if it doesn't exist.

19. **Line 19:** `os.makedirs(d, exist_ok=True)`

```
os.makedirs(d, exist_ok=True)
```

20. **Line 20:** `return d`

Returns `d` to the caller.

21. **Line 21:** `def settings_file() -> str:`

Returns the full path to settings.json, the persistent configuration file.

22. **Line 22:** `return os.path.join(user_data_dir(), "settings.json")`

Returns the per-user application data directory. On Windows this is %APPDATA%/AIVoiceStudio; on Linux/macOS it's ~/.aivoicestudio. The directory is created automatically if it doesn't exist.

23. **Line 23:** `def log_file() -> str:`

Returns the full path to app.log, the application log file.

24. **Line 24:** `return os.path.join(user_data_dir(), "app.log")`

Returns the per-user application data directory. On Windows this is %APPDATA%/AIVoiceStudio; on Linux/macOS it's ~/.aivoicestudio. The directory is created automatically if it doesn't exist.

44.2 Settings with Dot-Notation

```
# From settings.py
class Settings:
    _defaults = {
        "theme": "system",
        "audio_mode": "page_with_h1",
        "recording.rate": 1.0,
        "recording.pitch": 1.0,
        "recording.volume": 1.0,
        "recording.output_format": "wav",
        "recording.punctuation": "default",
        "compute": "auto",
        "developer_mode": False,
        "recent_projects": [],
    }

    def __init__(self, path=None):
        self._path = path or settings_file()
        self._data = dict(self._defaults)
        self._load()

    def get(self, key, default=None):
        # Simple: just use the key as-is
        return self._data.get(key, default)

    def set(self, key, value):
        self._data[key] = value
        self.save()

    def save(self):
        os.makedirs(os.path.dirname(self._path), exist_ok=True)
        with open(self._path, "w", encoding="utf-8") as f:
            json.dump(self._data, f, indent=2, ensure_ascii=False)

    @property
    def theme(self) -> str:
        return self._data.get("theme", "system")

    @property
    def compute(self) -> str:
        return self._data.get("compute", "auto")
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From settings.py`

A comment indicating the source file: *From settings.py*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `class Settings:`

Defines class `Settings`.

3. **Line 3:** `_defaults = {`

Assigns `{` to `_defaults`.

4. **Line 4:** `"theme": "system",`

`"theme": "system",`

5. **Line 5:** `"audio_mode": "page_with_h1",`

`"audio_mode": "page_with_h1",`

6. **Line 6:** `"recording.rate": 1.0,`

`"recording.rate": 1.0,`

7. **Line 7:** `"recording.pitch": 1.0,`

`"recording.pitch": 1.0,`

8. **Line 8:** `"recording.volume": 1.0,`

`"recording.volume": 1.0,`

9. **Line 9:** `"recording.output_format": "wav",`

`"recording.output_format": "wav",`

10. **Line 10:** `"recording.punctuation": "default",`

`"recording.punctuation": "default",`

11. **Line 11:** `"compute": "auto",`

`"compute": "auto",`

12. **Line 12:** `"developer_mode": False,`

`"developer_mode": False,`

13. **Line 13:** `"recent_projects": [],`

```
"recent_projects": [],
```

```
14. Line 14: }  
}
```

```
15. Line 15: def __init__(self, path=None):  
    Defines function __init__ accepting parameters (self, path).
```

```
16. Line 16: self._path = path or settings_file()  
    Returns the full path to settings.json, the persistent configuration file.
```

```
17. Line 17: self._data = dict(self._defaults)  
    Stores dict(self._defaults) in the instance attribute _data.
```

```
18. Line 18: self._load()  
    Calls self._load() to perform an operation.
```

```
19. Line 19: def get(self, key, default=None):  
    Defines function get: Retrieves a setting by dotted key path (e.g. 'recording.rate'). Returns default if the key doesn't exist.
```

```
20. Line 20: # Simple: just use the key as-is  
    A comment: Simple: just use the key as-is. Comments are ignored by the Python interpreter and serve as documentation.
```

```
21. Line 21: return self._data.get(key, default)  
    Returns self._data.get(key, default) to the caller.
```

```
22. Line 22: def set(self, key, value):  
    Defines function set: Returns the full path to settings.json, the persistent configuration file.
```

```
23. Line 23: self._data[key] = value  
    self._data[key] = value
```

```
24. Line 24: self.save()  
    Calls self.save() to perform an operation.
```

```
25. Line 25: def save(self):  
    Defines function save accepting parameters (self).
```

```
26. Line 26: os.makedirs(os.path.dirname(self._path), exist_ok=True)
```

```
os.makedirs(os.path.dirname(self._path), exist_ok=True)
```

27. **Line 27:** `with open(self._path, "w", encoding="utf-8") as f:`

Context manager: `open(self._path, "w", encoding="utf-8") as f` . Automatically manages resource cleanup.

28. **Line 28:** `json.dump(self._data, f, indent=2, ensure_ascii=False)`

```
json.dump(self._data, f, indent=2, ensure_ascii=False)
```

29. **Line 29:** `@property`

Decorator: `@property` . Wraps the following function/class to add behavior.

30. **Line 30:** `def theme(self) -> str:`

Defines function `theme` : Applies a visual theme (light/dark/system) to the entire wxPython frame and its children.

31. **Line 31:** `return self._data.get("theme", "system")`

Returns `self._data.get("theme", "system")` to the caller.

32. **Line 32:** `@property`

Decorator: `@property` . Wraps the following function/class to add behavior.

33. **Line 33:** `def compute(self) -> str:`

Defines function `compute` : Probes the system for available compute backends. Returns a dict like `{'cpu': True, 'cuda': False, 'dml': False}`.

34. **Line 34:** `return self._data.get("compute", "auto")`

Returns `self._data.get("compute", "auto")` to the caller.

44.3 Constants Module

```
# From constants.py – organized by category

# Application
APP_NAME = "AI Voice Studio"
APP_VERSION = "2.0.0"

# Audio parameters
RATE_MIN, RATE_MAX = 0.5, 2.0
PITCH_MIN, PITCH_MAX = 0.5, 2.0
VOLUME_MIN, VOLUME_MAX = 0.0, 2.0

# Output formats
FORMAT_WAV = "wav"
OUTPUT_FORMAT_CHOICES = [
    ("wav", "WAV (always available)"),
    ("mp3", "MP3 (requires FFmpeg)"),
    ("flac", "FLAC (requires FFmpeg)"),
]

# Themes
THEME_LIGHT, THEME_DARK, THEME_SYSTEM = "light", "dark", "system"

# Project types
PROJECT_TYPE_AUDIO_PLAYLIST = "audio_playlist"
PROJECT_TYPE_CLIPBOARD = "clipboard"
PROJECT_TYPE_DAISY_AUDIO = "daisy_audio"
# Legacy alias kept for reading older projects (no longer in the wizard)
PROJECT_TYPE_DAISY_AUDIO_TEXT = "daisy_audio_text"
PROJECT_TYPE_DAISY3_AUDIO_TEXT = "daisy3_audio_text"

# DAISY 3 constants
DAISY3_OUTPUT_DIR_NAME = "DAISY3"
DAISY3_DTBOOK_FILE = "book.xml"
DAISY3_NCX_FILE = "ncx.xml"

# DAISY chapter splitting
DAISY_SPLIT_H1 = "h1"
DAISY_SPLIT_ALL_HEADINGS = "all"

# Audio file creation modes
MODE_PAGE_WITH_H1 = "page_with_h1"
MODE_PAGE_ONLY = "page_only"
MODE_H1_ONLY = "h1_only"
MODE_ALL_HEADINGS = "all_headings"
MODE_ONE_FILE = "one_file"
PAGES_PER_FILE_MIN, PAGES_PER_FILE_MAX = 1, 50

# DAISY constants
DAISY_NCC_FILE = "ncc.html"
DAISY_PACKAGE_FILE = "package.opf"
DAISY_AUDIO_DIR_NAME = "audio"
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From constants.py – organized by category`

A comment indicating the source file: *From constants.py – organized by category*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `# Application`

A comment: *Application*. Comments are ignored by the Python interpreter and serve as documentation.

3. **Line 3:** `APP_NAME = "AI Voice Studio"`

Assigns `"AI Voice Studio"` to `APP_NAME`.

4. **Line 4:** `APP_VERSION = "2.0.0"`

Assigns `"2.0.0"` to `APP_VERSION`.

5. **Line 5:** `# Audio parameters`

A comment: *Audio parameters*. Comments are ignored by the Python interpreter and serve as documentation.

6. **Line 6:** `RATE_MIN, RATE_MAX = 0.5, 2.0`

`RATE_MIN, RATE_MAX = 0.5, 2.0`

7. **Line 7:** `PITCH_MIN, PITCH_MAX = 0.5, 2.0`

`PITCH_MIN, PITCH_MAX = 0.5, 2.0`

8. **Line 8:** `VOLUME_MIN, VOLUME_MAX = 0.0, 2.0`

`VOLUME_MIN, VOLUME_MAX = 0.0, 2.0`

9. **Line 9:** `# Output formats`

A comment: *Output formats*. Comments are ignored by the Python interpreter and serve as documentation.

10. **Line 10:** `FORMAT_WAV = "wav"`

Assigns `"wav"` to `FORMAT_WAV`.

11. **Line 11:** `OUTPUT_FORMAT_CHOICES = [`

Assigns `[` to `OUTPUT_FORMAT_CHOICES`.

12. **Line 12:** `("wav", "WAV (always available)",`

`("wav", "WAV (always available)",`

13. **Line 13:** `("mp3", "MP3 (requires FFmpeg)",`

`("mp3", "MP3 (requires FFmpeg)",`

14. **Line 14:** `("flac", "FLAC (requires FFmpeg)",`
`("flac", "FLAC (requires FFmpeg)",`
15. **Line 15:** `]`
`]`
16. **Line 16:** `# Themes`
A comment: *Themes*. Comments are ignored by the Python interpreter and serve as documentation.
17. **Line 17:** `THEME_LIGHT, THEME_DARK, THEME_SYSTEM = "light", "dark", "system"`
`THEME_LIGHT, THEME_DARK, THEME_SYSTEM = "light", "dark", "system"`
18. **Line 18:** `# Project types`
A comment: *Project types*. Comments are ignored by the Python interpreter and serve as documentation.
19. **Line 19:** `PROJECT_TYPE_AUDIO_PLAYLIST = "audio_playlist"`
Assigns `"audio_playlist"` to `PROJECT_TYPE_AUDIO_PLAYLIST`. This is the default project type: per-segment audio files plus generated M3U8/PLS/WPL playlists (Chapter 73).
20. **Line 20:** `PROJECT_TYPE_CLIPBOARD = "clipboard"`
Assigns `"clipboard"` to `PROJECT_TYPE_CLIPBOARD`. Clipboard projects record pasted text directly, with no source document.
21. **Line 21:** `PROJECT_TYPE_DAISY_AUDIO = "daisy_audio"`
Assigns `"daisy_audio"` to `PROJECT_TYPE_DAISY_AUDIO`. DAISY 2.02 audio-only books.
22. **Line 22:** `# Legacy alias kept for reading older projects (no longer in the wizard)`
A comment documenting that the next constant is a compatibility alias: older projects with this type keep loading and building books, but new projects cannot select it.
23. **Line 23:** `PROJECT_TYPE_DAISY_AUDIO_TEXT = "daisy_audio_text"`
Assigns `"daisy_audio_text"` to `PROJECT_TYPE_DAISY_AUDIO_TEXT`. The legacy DAISY 2.02 audio+text type — still honoured on load.
24. **Line 24:** `PROJECT_TYPE_DAISY3_AUDIO_TEXT = "daisy3_audio_text"`
Assigns `"daisy3_audio_text"` to `PROJECT_TYPE_DAISY3_AUDIO_TEXT`. The DAISY 3 (Z39.86-2005) audio+text-with-images type, the only project type that embeds images (Chapter 74).
25. **Line 25:** `# DAISY 3 constants`
A comment introducing the DAISY 3 output names: the builder writes a flat fileset (book.xml + ncx.xml + package.opf + SMILs + audio) inside the `DAISY3` folder.

26. **Line 24:** `DAISY_NCC_FILE = "ncc.html"`
Assigns `"ncc.html"` to `DAISY_NCC_FILE`.
27. **Line 25:** `DAISY_PACKAGE_FILE = "package.opf"`
Assigns `"package.opf"` to `DAISY_PACKAGE_FILE`.
28. **Line 26:** `DAISY_AUDIO_DIR_NAME = "audio"`
Assigns `"audio"` to `DAISY_AUDIO_DIR_NAME`.
29. **Line 27:** `DAISY3_OUTPUT_DIR_NAME = "DAISY3"`
Assigns `"DAISY3"` to `DAISY3_OUTPUT_DIR_NAME`. Every DAISY 3 book is built inside this subfolder of the project.
30. **Line 28:** `DAISY3_DTBOOK_FILE = "book.xml"`
Assigns `"book.xml"` to `DAISY3_DTBOOK_FILE` — the DTBook text document of a DAISY 3 book.
31. **Line 29:** `DAISY3_NCX_FILE = "ncx.xml"`
Assigns `"ncx.xml"` to `DAISY3_NCX_FILE` — the NCX navigation file players use to move between chapters.
32. **Line 30:** `# DAISY chapter splitting`
A comment introducing the two chapter-splitting modes offered on the wizard's DAISY page: heading style 1 only, or break on every heading (styles 1–6).
33. **Line 31:** `DAISY_SPLIT_H1 = "h1"`
Assigns `"h1"` to `DAISY_SPLIT_H1`. Each heading style 1 starts a new DAISY chapter.
34. **Line 32:** `DAISY_SPLIT_ALL_HEADINGS = "all"`
Assigns `"all"` to `DAISY_SPLIT_ALL_HEADINGS`. Any heading starts a new chapter.
35. **Line 33:** `# Audio file creation modes`
A comment introducing the five modes that decide how a document is divided into audio files. The wizard and Settings both read these.
36. **Line 34:** `MODE_PAGE_WITH_H1 = "page_with_h1"`
Assigns `"page_with_h1"` to `MODE_PAGE_WITH_H1`. Page-by-page recording with heading style 1 starting its own file.
37. **Line 35:** `MODE_PAGE_ONLY = "page_only"`
Assigns `"page_only"` to `MODE_PAGE_ONLY`. One file per page — optionally grouping 1–50 pages into one file via `PAGES_PER_FILE_MIN/MAX`.
38. **Line 36:** `MODE_H1_ONLY = "h1_only"`
Assigns `"h1_only"` to `MODE_H1_ONLY`. One file per heading style 1 and its following text.

39. **Line 37:** `MODE_ALL_HEADINGS = "all_headings"`
Assigns `"all_headings"` to `MODE_ALL_HEADINGS`. Same pattern, but breaking on every heading level.
40. **Line 38:** `MODE_ONE_FILE = "one_file"`
Assigns `"one_file"` to `MODE_ONE_FILE`. The whole document becomes one audio file; a recording stopped midway restarts from the beginning, so the Recording window warns about it.
41. **Line 39:** `PAGES_PER_FILE_MIN, PAGES_PER_FILE_MAX = 1, 50`
Assigns the grouping bounds `1` and `50`. The wizard's spin control is built from these, so the GUI and validation share one source of truth.

44.4 Key Takeaways

- **paths.py** centralizes all file locations (user data, models, projects, logs)
 - **settings.py** provides JSON persistence with property-based access
 - **constants.py** eliminates magic numbers and strings across the codebase
 - Settings are loaded at startup and saved immediately on change
-

Project Model & Persistence

i Info: Current implementation (2026.3.1). A project's `tts` block now also holds `omni` (the OmniVoice mode, design instructions, reference clip and generation parameters) and `engine_options` (`{"engine": id, "values": {...}}` , the per-project Voice Lab overrides written by *Engine tuning...*, in addition to the Settings defaults). Each segment still carries `index` , `title` , `text` , `saved` and `status` , and those two fields are what the *Start Selected Recording* picker reads: a segment with `saved` is offered as a recorded file, every title is offered as a file break. Re-recording a file clears `saved` and returns the segment to `pending` . The recording *range* (start position and whether to stop after one file) is a runtime argument, deliberately not stored in the project.

Learning Objectives

- Understand the AIVS project data model
- Implement atomic JSON writes for crash safety
- See the segment lifecycle (pending → done)

45.1 The Project Data Model

A project is a JSON file with this structure:

```
{
  "name": "My Audiobook",
  "source": "C:\\Documents\\book.pdf",
  "mode": "page_with_h1",
  "project_type": "audio_playlist",
  "segments": [
    {
      "index": 1,
      "title": "Chapter 1",
      "text": "Once upon a time...",
      "status": "done",
      "saved": "Chapter_1.wav",
      "duration": 120.5
    },
    {
      "index": 2,
      "title": "Chapter 2",
      "text": "It was a dark..."
    }
  ],
  "tts": { /* voice settings */ }
}
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# A project is a JSON file with this structure:`

A comment: *A project is a JSON file with this structure:*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `{`

`{`

3. **Line 3:** `"name": "My Audiobook",`

`"name": "My Audiobook",`

4. **Line 4:** `"source": "C:\\Documents\\book.pdf",`

`"source": "C:\\Documents\\book.pdf",`

5. **Line 5:** `"mode": "page_with_h1",`

`"mode": "page_with_h1",`

6. **Line 6:** `"project_type": "audio_playlist",`

`"project_type": "audio_playlist",`

7. **Line 7:** `"segments": [`

`"segments": [`

8. **Line 8:** `{`

`{`

9. **Line 9:** `"index": 1,`

`"index": 1,`

10. **Line 10:** `"title": "Chapter 1",`

`"title": "Chapter 1",`

11. **Line 11:** `"text": "Once upon a time...",`

`"text": "Once upon a time...",`

12. **Line 12:** `"status": "done",`

`"status": "done",`

13. **Line 13:** `"saved": "Chapter_1.wav",`

```
"saved": "Chapter_1.wav",

14. Line 14: "duration": 120.5
    "duration": 120.5

15. Line 15: },
    },

16. Line 16: {
    {

17. Line 17: "index": 2,
    "index": 2,

18. Line 18: "title": "Chapter 2",
    "title": "Chapter 2",

19. Line 19: "text": "It was a dark..."
    "text": "It was a dark..."

20. Line 20: }
    }

21. Line 21: ],
    ],

22. Line 22: "tts": { /* voice settings */ }
    "tts": { /* voice settings */ }

23. Line 23: }
    }
```

45.2 Atomic Writes

```
# From project.py - crash-safe save
def save_project(project_dir: str, data: dict) -> None:
    """Save project.json atomically (write to temp, then rename)."""
    import tempfile
    path = os.path.join(project_dir, "project.json")
    fd, tmp = tempfile.mkstemp(dir=project_dir, suffix=".tmp")
    try:
        with os.fdopen(fd, "w", encoding="utf-8") as f:
            json.dump(data, f, indent=2, ensure_ascii=False)
            os.replace(tmp, path) # Atomic on same filesystem
    except:
        os.unlink(tmp)
        raise

def mark_segment_done(project_dir, segment_index, audio_path):
    """Mark a segment as done and save immediately."""
    data = load_project(project_dir)
    for seg in data["segments"]:
        if seg["index"] == segment_index:
            seg["status"] = "done"
            seg["saved"] = os.path.basename(audio_path)
            break
    save_project(project_dir, data)

def first_pending_index(project_dir) -> int:
    """Find the first segment that hasn't been recorded yet."""
    data = load_project(project_dir)
    for seg in data.get("segments", []):
        if seg.get("status") != "done":
            return seg["index"] - 1
    return len(data.get("segments", []))
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From project.py - crash-safe save`

A comment indicating the source file: *From project.py - crash-safe save*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `def save_project(project_dir: str, data: dict) -> None:`

Defines function `save_project` accepting parameters (`project_dir`, `data`), returning `None`.

3. **Line 3:** `"""Save project.json atomically (write to temp, then rename)."""`

Docstring: *Save project.json atomically (write to temp, then rename)*.. Documents the purpose of the following code block.

4. **Line 4:** `import tempfile`

Imports the `tempfile` module.

5. **Line 5:** `path = os.path.join(project_dir, "project.json")`

Assigns `os.path.join(project_dir, "project.json")` to `path`.

6. **Line 6:** `fd, tmp = tempfile.mkstemp(dir=project_dir, suffix=".tmp")`

`fd, tmp = tempfile.mkstemp(dir=project_dir, suffix=".tmp")`

7. **Line 7:** `try:`

Begins exception handling. Errors in this block will be caught below.

8. **Line 8:** `with os.fdopen(fd, "w", encoding="utf-8") as f:`

Context manager: `os.fdopen(fd, "w", encoding="utf-8") as f`. Automatically manages resource cleanup.

9. **Line 9:** `json.dump(data, f, indent=2, ensure_ascii=False)`

`json.dump(data, f, indent=2, ensure_ascii=False)`

10. **Line 10:** `os.replace(tmp, path) # Atomic on same filesystem`

`os.replace(tmp, path) # Atomic on same filesystem`

11. **Line 11:** `except:`

`except:`

12. **Line 12:** `os.unlink(tmp)`

`os.unlink(tmp)`

13. **Line 13:** `raise`

raise

14. **Line 14:** `def mark_segment_done(project_dir, segment_index, audio_path):`

Defines function `mark_segment_done` accepting parameters (`project_dir`, `segment_index`, `audio_path`).

15. **Line 15:** `"""Mark a segment as done and save immediately."""`

Docstring: *Mark a segment as done and save immediately..* Documents the purpose of the following code block.

16. **Line 16:** `data = load_project(project_dir)`

Assigns `load_project(project_dir)` to `data`.

17. **Line 17:** `for seg in data["segments"]:`

Iterates over `data["segments"]`, assigning each element to `seg`.

18. **Line 18:** `if seg["index"] == segment_index:`

Checks condition: `seg["index"] == segment_index`. The indented block below runs only when this is `True`.

19. **Line 19:** `seg["status"] = "done"`

`seg["status"] = "done"`

20. **Line 20:** `seg["saved"] = os.path.basename(audio_path)`

`seg["saved"] = os.path.basename(audio_path)`

21. **Line 21:** `break`

Exits the innermost loop immediately.

22. **Line 22:** `save_project(project_dir, data)`

Calls function `save_project()`.

23. **Line 23:** `def first_pending_index(project_dir) -> int:`

Defines function `first_pending_index` accepting parameters (`project_dir`), returning `int`.

24. **Line 24:** `"""Find the first segment that hasn't been recorded yet."""`

Docstring: *Find the first segment that hasn't been recorded yet..* Documents the purpose of the following code block.

25. **Line 25:** `data = load_project(project_dir)`

Assigns `load_project(project_dir)` to `data`.

26. **Line 26:** `for seg in data.get("segments", []):`

Iterates over `data.get("segments", [])`, assigning each element to `seg`.

27. Line 27: `if seg.get("status") != "done":`

Checks condition: `seg.get("status") != "done"`. The indented block below runs only when this is `True`.

28. Line 28: `return seg["index"] - 1`

Returns `seg["index"] - 1` to the caller.

29. Line 29: `return len(data.get("segments", []))`

Returns `len(data.get("segments", []))` to the caller.

45.3 Key Takeaways

- A project is a **JSON file** with name, source, segments, and TTS settings
 - **Atomic writes** (temp file + rename) prevent corruption on crash
 - Segments are saved **immediately** after recording — reopening the project resumes where you left off
 - **first_pending_index()** enables seamless resume after app restart
-

Document Parsing & Splitting

i Info: Current implementation (2026.3.1). Parsing and splitting are unchanged, but the split result is now part of the user interface in a new way: the file breaks produced here are what *Edit → Start Selected Recording...* lists, so one chapter of a long document can be recorded before the ones before it. The splitter's titles are also what the DAISY builders and the playlist builder name their navigation points after. See [Chapter 80](#).

Learning Objectives

- Understand how AIVS parses different document formats
- Learn the text splitting strategies (page, heading, fixed size)
- See the complete pipeline from document to segments

46.1 Supported Formats

Format	Library	Extraction Method
PDF	pypdf	Per-page text extraction
DOCX	python-docx	Paragraph-based with style info
TXT	stdlib	Direct text reading
Markdown	stdlib + regex	Heading detection via # patterns
HTML/HTM	BeautifulSoup (optional)	Tag-based extraction
EPUB	Custom parser	Chapter-based extraction

46.2 Splitting Modes

```
# From constants.py
MODE_PAGE_WITH_H1 = "page_with_h1"      # Split at H1 headings, fallback to pages
MODE_PAGE_ONLY = "page_only"            # One segment per page
MODE_CHAPTER_ONLY = "chapter_only"      # Split at any heading level
MODE_FIXED_SIZE = "fixed_size"          # Fixed character count

AUDIO_MODE_CHOICES = [
    (MODE_PAGE_WITH_H1, "Pages with H1 headings (Recommended)"),
    (MODE_PAGE_ONLY, "Pages only"),
    (MODE_CHAPTER_ONLY, "Chapters/headings only"),
    (MODE_FIXED_SIZE, "Fixed size chunks"),
]

# From splitter.py
class Segment:
    def __init__(self, index: int, title: str, text: str):
        self.index = index
        self.title = title
        self.text = text

def split_document(doc: Document, mode: str) -> list[Segment]:
    """Split a parsed document into TTS segments."""
    if mode == MODE_PAGE_WITH_H1:
        return _split_by_h1_then_pages(doc)
    elif mode == MODE_PAGE_ONLY:
        return _split_by_pages(doc)
    elif mode == MODE_CHAPTER_ONLY:
        return _split_by_headings(doc)
    elif mode == MODE_FIXED_SIZE:
        return _split_by_size(doc, chunk_size=2000)
    return []
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From constants.py`

A comment indicating the source file: *From constants.py*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `MODE_PAGE_WITH_H1 = "page_with_h1" # Split at H1 headings, fallback to pages`

Assigns `"page_with_h1"` # Split at H1 headings, fallback to pages to `MODE_PAGE_WITH_H1` .

3. **Line 3:** `MODE_PAGE_ONLY = "page_only" # One segment per page`

Assigns `"page_only"` # One segment per page to `MODE_PAGE_ONLY` .

4. **Line 4:** `MODE_CHAPTER_ONLY = "chapter_only" # Split at any heading level`

Assigns `"chapter_only"` # Split at any heading level to `MODE_CHAPTER_ONLY` .

5. **Line 5:** `MODE_FIXED_SIZE = "fixed_size" # Fixed character count`

Assigns `"fixed_size"` # Fixed character count to `MODE_FIXED_SIZE` .

6. **Line 6:** `AUDIO_MODE_CHOICES = [`

Assigns `[` to `AUDIO_MODE_CHOICES` .

7. **Line 7:** `(MODE_PAGE_WITH_H1, "Pages with H1 headings (Recommended)" ),`

`(MODE_PAGE_WITH_H1, "Pages with H1 headings (Recommended)" ),`

8. **Line 8:** `(MODE_PAGE_ONLY, "Pages only" ),`

`(MODE_PAGE_ONLY, "Pages only" ),`

9. **Line 9:** `(MODE_CHAPTER_ONLY, "Chapters/headings only" ),`

`(MODE_CHAPTER_ONLY, "Chapters/headings only" ),`

10. **Line 10:** `(MODE_FIXED_SIZE, "Fixed size chunks" ),`

`(MODE_FIXED_SIZE, "Fixed size chunks" ),`

11. **Line 11:** `]`

`]`

12. **Line 12:** `# From splitter.py`

A comment indicating the source file: *From splitter.py*. This helps the reader locate the code in the AIVoiceStudio codebase.

13. **Line 13:** `class Segment:`

Defines class `Segment` .

14. **Line 14:** `def __init__(self, index: int, title: str, text: str):`

Defines function `__init__` accepting parameters (self, index, title, text).

15. **Line 15:** `self.index = index`

Stores `index` in the instance attribute `index` .

16. **Line 16:** `self.title = title`

Stores `title` in the instance attribute `title` .

17. **Line 17:** `self.text = text`

Stores `text` in the instance attribute `text` .

18. **Line 18:** `def split_document(doc: Document, mode: str) -> list[Segment]:`

Defines function `split_document` accepting parameters (doc, mode), returning `list[Segment]` .

19. **Line 19:** `"""Split a parsed document into TTS segments."""`

Docstring: *Split a parsed document into TTS segments.*. Documents the purpose of the following code block.

20. **Line 20:** `if mode == MODE_PAGE_WITH_H1:`

Checks condition: `mode == MODE_PAGE_WITH_H1` . The indented block below runs only when this is `True` .

21. **Line 21:** `return _split_by_h1_then_pages(doc)`

Returns `_split_by_h1_then_pages(doc)` to the caller.

22. **Line 22:** `elif mode == MODE_PAGE_ONLY:`

Alternative condition: `mode == MODE_PAGE_ONLY` . Checked only if all previous conditions were `False` .

23. **Line 23:** `return _split_by_pages(doc)`

Returns `_split_by_pages(doc)` to the caller.

24. **Line 24:** `elif mode == MODE_CHAPTER_ONLY:`

Alternative condition: `mode == MODE_CHAPTER_ONLY` . Checked only if all previous conditions were `False` .

25. **Line 25:** `return _split_by_headings(doc)`

Returns `_split_by_headings(doc)` to the caller.

26. **Line 26:** `elif mode == MODE_FIXED_SIZE:`

Alternative condition: `mode == MODE_FIXED_SIZE` . Checked only if all previous conditions were `False` .

27. **Line 27:** `return _split_by_size(doc, chunk_size=2000)`
Returns `_split_by_size(doc, chunk_size=2000)` to the caller.

28. **Line 28:** `return []`
Returns `[]` to the caller.

46.3 The Splitting Algorithms

```
def _split_by_h1_then_pages(doc: Document) -> list[Segment]:
    """Split at H1 headings, then at page breaks."""
    segments = []
    current_text = []
    current_title = "Introduction"

    for block in doc.blocks:
        if block.level == 1 and current_text:
            segments.append(Segment(
                index=len(segments) + 1,
                title=current_title,
                text="\n\n".join(current_text),
            ))
            current_text = []
        if block.level == 1:
            current_title = block.heading
            current_text.append(block.text)

    if current_text:
        segments.append(Segment(
            index=len(segments) + 1,
            title=current_title,
            text="\n\n".join(current_text),
        ))

    return segments

def _split_by_size(doc: Document, chunk_size: int = 2000) -> list[Segment]:
    """Split text into fixed-size chunks at sentence boundaries."""
    import re
    full_text = "\n\n".join(b.text for b in doc.blocks)
    sentences = re.split(r'(?<=[!?])\s+', full_text)

    segments = []
    current_chunk = []
    current_len = 0

    for sent in sentences:
        if current_len + len(sent) > chunk_size and current_chunk:
            segments.append(Segment(
                index=len(segments) + 1,
                title=f"Chunk {len(segments) + 1}",
                text=" ".join(current_chunk),
            ))
            current_chunk = []
            current_len = 0
        current_chunk.append(sent)
        current_len += len(sent)

    if current_chunk:
        segments.append(Segment(
            index=len(segments) + 1,
            title=f"Chunk {len(segments) + 1}",
            text=" ".join(current_chunk),
        ))

    return segments
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def _split_by_h1_then_pages(doc: Document) -> list[Segment]:`
Defines function `_split_by_h1_then_pages` accepting parameters (doc), returning `list[Segment]`.
2. **Line 2:** `"""Split at H1 headings, then at page breaks."""`
Docstring: *Split at H1 headings, then at page breaks..* Documents the purpose of the following code block.
3. **Line 3:** `segments = []`
Assigns `[]` to `segments`.
4. **Line 4:** `current_text = []`
Assigns `[]` to `current_text`.
5. **Line 5:** `current_title = "Introduction"`
Assigns `"Introduction"` to `current_title`.
6. **Line 6:** `for block in doc.blocks:`
Iterates over `doc.blocks`, assigning each element to `block`.
7. **Line 7:** `if block.level == 1 and current_text:`
Checks condition: `block.level == 1 and current_text`. The indented block below runs only when this is `True`.
8. **Line 8:** `segments.append(Segment(`
`segments.append(Segment(`
9. **Line 9:** `index=len(segments) + 1,`
Assigns `len(segments) + 1,` to `index`.
10. **Line 10:** `title=current_title,`
Assigns `current_title,` to `title`.
11. **Line 11:** `text="\n\n".join(current_text),`
Assigns `"\n\n".join(current_text),` to `text`.
12. **Line 12:** `)`
`)`
13. **Line 13:** `current_text = []`
Assigns `[]` to `current_text`.

14. **Line 14:** `if block.level == 1:`
Checks condition: `block.level == 1` . The indented block below runs only when this is `True` .
15. **Line 15:** `current_title = block.heading`
Assigns `block.heading` to `current_title` .
16. **Line 16:** `current_text.append(block.text)`
`current_text.append(block.text)`
17. **Line 17:** `if current_text:`
Checks condition: `current_text` . The indented block below runs only when this is `True` .
18. **Line 18:** `segments.append(Segment(`
`segments.append(Segment(`
19. **Line 19:** `index=len(segments) + 1,`
Assigns `len(segments) + 1`, to `index` .
20. **Line 20:** `title=current_title,`
Assigns `current_title`, to `title` .
21. **Line 21:** `text="\n\n".join(current_text),`
Assigns `"\n\n".join(current_text)`, to `text` .
22. **Line 22:** `)`
`)`
23. **Line 23:** `return segments`
Returns `segments` to the caller.
24. **Line 24:** `def _split_by_size(doc: Document, chunk_size: int = 2000) -> list[Segment]:`
Defines function `_split_by_size` accepting parameters (doc, chunk_size), returning `list[Segment]` .
25. **Line 25:** `"""Split text into fixed-size chunks at sentence boundaries."""`
Docstring: *Split text into fixed-size chunks at sentence boundaries.*. Documents the purpose of the following code block.
26. **Line 26:** `import re`
Imports the `re` module.
27. **Line 27:** `full_text = "\n\n".join(b.text for b in doc.blocks)`

Assigns `"\n\n".join(b.text for b in doc.blocks)` to `full_text` .

28. **Line 28:** `sentences = re.split(r'(?<=[.!?])\s+', full_text)`

Assigns `re.split(r'(?<=[.!?])\s+', full_text)` to `sentences` .

29. **Line 29:** `segments = []`

Assigns `[]` to `segments` .

30. **Line 30:** `current_chunk = []`

Assigns `[]` to `current_chunk` .

31. **Line 31:** `current_len = 0`

Assigns `0` to `current_len` .

32. **Line 32:** `for sent in sentences:`

Iterates over `sentences` , assigning each element to `sent` .

33. **Line 33:** `if current_len + len(sent) > chunk_size and current_chunk:`

Checks condition: `current_len + len(sent) > chunk_size and current_chunk` . The indented block below runs only when this is `True` .

34. **Line 34:** `segments.append(Segment(`

`segments.append(Segment(`

35. **Line 35:** `index=len(segments) + 1,`

Assigns `len(segments) + 1` to `index` .

36. **Line 36:** `title=f"Chunk {len(segments) + 1}",`

Assigns `f"Chunk {len(segments) + 1}"` to `title` .

37. **Line 37:** `text=" ".join(current_chunk),`

Assigns `" ".join(current_chunk)` to `text` .

38. **Line 38:** `)`

`)`

39. **Line 39:** `current_chunk = []`

Assigns `[]` to `current_chunk` .

40. **Line 40:** `current_len = 0`

Assigns `0` to `current_len` .

41. **Line 41:** `current_chunk.append(sent)`

`current_chunk.append(sent)`

42. **Line 42:** `current_len += len(sent)`

Updates `current_len` by `+= len(sent)` .

43. **Line 43:** `if current_chunk:`

Checks condition: `current_chunk` . The indented block below runs only when this is `True` .

44. **Line 44:** `segments.append(Segment(`

`segments.append(Segment(`

45. **Line 45:** `index=len(segments) + 1,`

Assigns `len(segments) + 1`, to `index` .

46. **Line 46:** `title=f"Chunk {len(segments) + 1}",`

Assigns `f"Chunk {len(segments) + 1}"`, to `title` .

47. **Line 47:** `text=" ".join(current_chunk),`

Assigns `" ".join(current_chunk)`, to `text` .

48. **Line 48:** `)`

`)`

49. **Line 49:** `return segments`

Returns `segments` to the caller.

46.4 DAISY Chapter Splitting

```
def split_daisy_chapters(blocks: list) -> list[Segment]:
    """Split document into chapters for DAISY book output."""
    segments = []
    for block in blocks:
        if block.level <= 2: # H1 and H2 become chapters
            segments.append(Segment(
                index=len(segments) + 1,
                title=block.heading,
                text=block.text,
            ))
    return segments
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `def split_daisy_chapters(blocks: list) -> list[Segment]:`
Defines function `split_daisy_chapters` accepting parameters (blocks), returning `list[Segment]`.
2. **Line 2:** `"""Split document into chapters for DAISY book output."""`
Docstring: *Split document into chapters for DAISY book output..* Documents the purpose of the following code block.
3. **Line 3:** `segments = []`
Assigns `[]` to `segments`.
4. **Line 4:** `for block in blocks:`
Iterates over `blocks`, assigning each element to `block`.
5. **Line 5:** `if block.level <= 2: # H1 and H2 become chapters`
Checks condition: `block.level <= 2`. The indented block below runs only when this is `True`.
6. **Line 6:** `segments.append(Segment(`
`segments.append(Segment(`
7. **Line 7:** `index=len(segments) + 1,`
Assigns `len(segments) + 1`, to `index`.
8. **Line 8:** `title=block.heading,`
Assigns `block.heading`, to `title`.
9. **Line 9:** `text=block.text,`
Assigns `block.text`, to `text`.
10. **Line 10:** `)`
`)`
11. **Line 11:** `return segments`
Returns `segments` to the caller.

46.5 Key Takeaways

- **4 splitting modes:** page+heading, page-only, heading-only, fixed-size
 - Each mode produces a list of **Segment** objects (index, title, text)
 - DAISY books use **chapter-level splitting** (H1/H2 headings)
 - Fixed-size splitting **respects sentence boundaries**
 - The splitter is **format-agnostic** — it works on parsed Document objects
-

TTS Catalog & Model Store

i Info: Current implementation (2026.3.1). The JSON catalog now describes nine TTS families — Piper, Kokoro-82M, Kitten TTS, VITS zh-AISHELL3, Matcha-TTS, OmniVoice, OmniVoice Server, SAPI5 and the Windows Core voices. The Voice Lab clone engines (Pocket TTS, Bark, F5-TTS) are deliberately *not* in this file: their voices come from the installed Python package, so they live in `voice1ab/engines.py` and are emitted in the same catalog shape — see [Chapter 76](#). Engines that are pip packages also carry a `requires_package` field, which the panels turn into an installed / not-installed state (see [Chapter 77](#)).

Learning Objectives

- Understand the TTS catalog data structure
 - Master the ModelStore for tracking installed voices
 - See how voice file resolution works
-

47.1 The TTS Catalog

```
# From tts/catalog.py – a large tuple of TTS engine definitions
# Each entry has: id, name, engine, license, languages → variants → voices

def get_tts_list() -> list[dict]:
    """Return all TTS engines from the catalog."""
    return list(_CATALOG)

def find_tts(tts_id: str) -> dict | None:
    """Find a TTS engine by ID."""
    for tts in _CATALOG:
        if tts["id"] == tts_id:
            return tts
    return None

def find_language(tts: dict, lang_code: str) -> dict | None:
    for lang in tts.get("languages", []):
        if lang["code"] == lang_code:
            return lang
    return None

def find_variant(tts, lang_code, variant_id) -> dict | None:
    lang = find_language(tts, lang_code)
    if lang:
        for v in lang.get("variants", []):
            if v["id"] == variant_id:
                return v
    return None

def find_voice(tts, lang_code, variant_id, voice_id) -> dict | None:
    variant = find_variant(tts, lang_code, variant_id)
    if variant:
        for v in variant.get("voices", []):
            if v["id"] == voice_id:
                return v
    return None
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From tts/catalog.py - a large tuple of TTS engine definitions`

A comment indicating the source file: *From tts/catalog.py - a large tuple of TTS engine definitions*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `# Each entry has: id, name, engine, license, languages → variants → voices`

A comment: *Each entry has: id, name, engine, license, languages → variants → voices*. Comments are ignored by the Python interpreter and serve as documentation.

3. **Line 3:** `def get_tts_list() -> list[dict]:`

Defines function `get_tts_list`, returning `list[dict]`.

4. **Line 4:** `"""Return all TTS engines from the catalog."""`

Docstring: *Return all TTS engines from the catalog..* Documents the purpose of the following code block.

5. **Line 5:** `return list(_CATALOG)`

Returns `list(_CATALOG)` to the caller.

6. **Line 6:** `def find_tts(tts_id: str) -> dict | None:`

Defines function `find_tts` accepting parameters (`tts_id`), returning `dict | None`.

7. **Line 7:** `"""Find a TTS engine by ID."""`

Docstring: *Find a TTS engine by ID..* Documents the purpose of the following code block.

8. **Line 8:** `for tts in _CATALOG:`

Iterates over `_CATALOG`, assigning each element to `tts`.

9. **Line 9:** `if tts["id"] == tts_id:`

Checks condition: `tts["id"] == tts_id`. The indented block below runs only when this is `True`.

10. **Line 10:** `return tts`

Returns `tts` to the caller.

11. **Line 11:** `return None`

Returns `None` (no value).

12. **Line 12:** `def find_language(tts: dict, lang_code: str) -> dict | None:`

Defines function `find_language` accepting parameters (`tts`, `lang_code`), returning `dict | None`.

13. **Line 13:** `for lang in tts.get("languages", []):`

Iterates over `tts.get("languages", [])`, assigning each element to `lang`.

14. **Line 14:** `if lang["code"] == lang_code:`

Checks condition: `lang["code"] == lang_code`. The indented block below runs only when this is `True`.

15. **Line 15:** `return lang`

Returns `lang` to the caller.

16. **Line 16:** `return None`

Returns `None` (no value).

17. **Line 17:** `def find_variant(tts, lang_code, variant_id) -> dict | None:`

Defines function `find_variant` accepting parameters (tts, lang_code, variant_id), returning `dict | None`.

18. **Line 18:** `lang = find_language(tts, lang_code)`

Assigns `find_language(tts, lang_code)` to `lang`.

19. **Line 19:** `if lang:`

Checks condition: `lang`. The indented block below runs only when this is `True`.

20. **Line 20:** `for v in lang.get("variants", []):`

Iterates over `lang.get("variants", [])`, assigning each element to `v`.

21. **Line 21:** `if v["id"] == variant_id:`

Checks condition: `v["id"] == variant_id`. The indented block below runs only when this is `True`.

22. **Line 22:** `return v`

Returns `v` to the caller.

23. **Line 23:** `return None`

Returns `None` (no value).

24. **Line 24:** `def find_voice(tts, lang_code, variant_id, voice_id) -> dict | None:`

Defines function `find_voice` accepting parameters (tts, lang_code, variant_id, voice_id), returning `dict | None`.

25. **Line 25:** `variant = find_variant(tts, lang_code, variant_id)`

Assigns `find_variant(tts, lang_code, variant_id)` to `variant`.

26. **Line 26:** `if variant:`

Checks condition: `variant`. The indented block below runs only when this is `True`.

27. **Line 27:** `for v in variant.get("voices", []):`

Iterates over `variant.get("voices", [])`, assigning each element to `v`.

28. **Line 28:** `if v["id"] == voice_id:`

Checks condition: `v["id"] == voice_id`. The indented block below runs only when this is `True`.

29. **Line 29:** `return v`

Returns `v` to the caller.

30. **Line 30:** `return None`

Returns `None` (no value).

47.2 The ModelStore

```
# From tts/models.py
class ModelStore:
    """Manages which voice models are installed on disk."""

    def __init__(self, state_file=None):
        self._state_file = state_file or self._default_state_file()
        self._state = self._load_state()

    def variant_dir(self, tts_id, lang, variant) -> str:
        """models/piper/en_US/medium"""
        return os.path.join(paths.models_dir(), tts_id, lang, variant)

    def is_variant_installed(self, tts_id, lang, variant) -> bool:
        return os.path.isdir(self.variant_dir(tts_id, lang, variant))

    def mark_installed(self, tts_id, lang, variant, path):
        self._state.setdefault(tts_id, {})
        self._state[tts_id].setdefault(lang, {})
        self._state[tts_id][lang][variant] = {"path": path}
        self._save_state()

    def installed_voices(self) -> list[dict]:
        """Return flat list of all installed voice entries."""
        voices = []
        for tts_id, langs in self._state.items():
            tts = catalog.find_tts(tts_id)
            for lang_code, variants in langs.items():
                for var_id in variants:
                    variant = catalog.find_variant(tts, lang_code, var_id)
                    if variant:
                        for voice in variant.get("voices", []):
                            entry = dict(voice)
                            entry["tts"] = tts_id
                            entry["language"] = lang_code
                            entry["variant"] = var_id
                            entry["dir"] = self.variant_dir(tts_id, lang_code, var_id)
                            voices.append(entry)

        return voices
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From tts/models.py`
A comment indicating the source file: *From tts/models.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
2. **Line 2:** `class ModelStore:`
Manages downloaded voice models on disk. Tracks which voices are installed, their file paths, and total disk usage.
3. **Line 3:** `"""Manages which voice models are installed on disk."""`
Docstring: *Manages which voice models are installed on disk..* Documents the purpose of the following code block.
4. **Line 4:** `def __init__(self, state_file=None):`
Defines function `__init__` accepting parameters (self, state_file).
5. **Line 5:** `self._state_file = state_file or self._default_state_file()`
Stores `state_file` or `self._default_state_file()` in the instance attribute `_state_file`.
6. **Line 6:** `self._state = self._load_state()`
Stores `self._load_state()` in the instance attribute `_state`.
7. **Line 7:** `def variant_dir(self, tts_id, lang, variant) -> str:`
Defines function `variant_dir` accepting parameters (self, tts_id, lang, variant), returning `str`.
8. **Line 8:** `"""models/piper/en_US/medium"""`
Docstring: *models/piper/en_US/medium*. Documents the purpose of the following code block.
9. **Line 9:** `return os.path.join(paths.models_dir(), tts_id, lang, variant)`
Returns the directory where downloaded TTS voice models are stored. Respects any custom path set in Settings > Paths. Creates the directory if needed.
10. **Line 10:** `def is_variant_installed(self, tts_id, lang, variant) -> bool:`
Defines function `is_variant_installed` accepting parameters (self, tts_id, lang, variant), returning `bool`.
11. **Line 11:** `return os.path.isdir(self.variant_dir(tts_id, lang, variant))`
Returns `os.path.isdir(self.variant_dir(tts_id, lang, variant))` to the caller.
12. **Line 12:** `def mark_installed(self, tts_id, lang, variant, path):`
Defines function `mark_installed` accepting parameters (self, tts_id, lang, variant, ...).
13. **Line 13:** `self._state.setdefault(tts_id, {})`

```
self._state.setdefault(tts_id, {})
```

14. **Line 14:** `self._state[tts_id].setdefault(lang, {})`

```
self._state[tts_id].setdefault(lang, {})
```

15. **Line 15:** `self._state[tts_id][lang][variant] = {"path": path}`

```
self._state[tts_id][lang][variant] = {"path": path}
```

16. **Line 16:** `self._save_state()`

Calls `self._save_state()` to perform an operation.

17. **Line 17:** `def installed_voices(self) -> list[dict]:`

Defines function `installed_voices` accepting parameters (self), returning `list[dict]` .

18. **Line 18:** `"""Return flat list of all installed voice entries."""`

Docstring: *Return flat list of all installed voice entries..* Documents the purpose of the following code block.

19. **Line 19:** `voices = []`

Assigns `[]` to `voices` .

20. **Line 20:** `for tts_id, langs in self._state.items():`

```
for tts_id, langs in self._state.items():
```

21. **Line 21:** `tts = catalog.find_tts(tts_id)`

Assigns `catalog.find_tts(tts_id)` to `tts` .

22. **Line 22:** `for lang_code, variants in langs.items():`

```
for lang_code, variants in langs.items():
```

23. **Line 23:** `for var_id in variants:`

Iterates over `variants` , assigning each element to `var_id` .

24. **Line 24:** `variant = catalog.find_variant(tts, lang_code, var_id)`

Assigns `catalog.find_variant(tts, lang_code, var_id)` to `variant` .

25. **Line 25:** `if variant:`

Checks condition: `variant` . The indented block below runs only when this is `True` .

26. **Line 26:** `for voice in variant.get("voices", []):`

Iterates over `variant.get("voices", [])` , assigning each element to `voice` .

27. **Line 27:** `entry = dict(voice)`
Assigns `dict(voice)` to `entry`.

28. **Line 28:** `entry["tts"] = tts_id`
`entry["tts"] = tts_id`

29. **Line 29:** `entry["language"] = lang_code`
`entry["language"] = lang_code`

30. **Line 30:** `entry["variant"] = var_id`
`entry["variant"] = var_id`

31. **Line 31:** `entry["dir"] = self.variant_dir(tts_id, lang_code, var_id)`
`entry["dir"] = self.variant_dir(tts_id, lang_code, var_id)`

32. **Line 32:** `voices.append(entry)`
`voices.append(entry)`

33. **Line 33:** `return voices`
Returns `voices` to the caller.

47.3 Voice File Resolution

```
# From tts/models.py - resolve voice entry → actual file paths
def resolve_voice_files(entry: dict) -> dict[str, str]:
    """Map a voice entry dict to actual file paths on disk."""
    vdir = entry.get("dir", "")
    engine = entry.get("engine", "vits")

    result = {
        "model": "", "tokens": "", "data_dir": "",
        "lexicon": "", "vocoder": "", "voices_file": "",
        "engine": engine,
    }

    if not vdir or not os.path.isdir(vdir):
        return result

    # Find model file
    for f in os.listdir(vdir):
        if f.endswith(".onnx"):
            result["model"] = os.path.join(vdir, f)
            break

    # Find tokens file
    tokens = os.path.join(vdir, "tokens.txt")
    if os.path.isfile(tokens):
        result["tokens"] = tokens

    # Find voices file (for multi-speaker models)
    voices = os.path.join(vdir, "voices.bin")
    if os.path.isfile(voices):
        result["voices_file"] = voices

    # Shared espeak-ng-data
    shared = os.path.join(paths.models_dir(), "shared", "espeak-ng-data")
    if os.path.isdir(shared) and entry.get("frontend") != "char":
        result["data_dir"] = shared

    return result
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From tts/models.py – resolve voice entry → actual file paths`

A comment indicating the source file: *From tts/models.py – resolve voice entry → actual file paths*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `def resolve_voice_files(entry: dict) -> dict[str, str]:`

Defines function `resolve_voice_files` accepting parameters (`entry`), returning `dict[str, str]`.

3. **Line 3:** `"""Map a voice entry dict to actual file paths on disk."""`

Docstring: *Map a voice entry dict to actual file paths on disk..* Documents the purpose of the following code block.

4. **Line 4:** `vdir = entry.get("dir", "")`

Assigns `entry.get("dir", "")` to `vdir`.

5. **Line 5:** `engine = entry.get("engine", "vits")`

Assigns `entry.get("engine", "vits")` to `engine`.

6. **Line 6:** `result = {`

Assigns `{` to `result`.

7. **Line 7:** `"model": "", "tokens": "", "data_dir": "",`

`"model": "", "tokens": "", "data_dir": "",`

8. **Line 8:** `"lexicon": "", "vocoder": "", "voices_file": "",`

`"lexicon": "", "vocoder": "", "voices_file": "",`

9. **Line 9:** `"engine": engine,`

`"engine": engine,`

10. **Line 10:** `}`

`}`

11. **Line 11:** `if not vdir or not os.path.isdir(vdir):`

Checks condition: `not vdir or not os.path.isdir(vdir)`. The indented block below runs only when this is `True`.

12. **Line 12:** `return result`

Returns `result` to the caller.

13. **Line 13:** `# Find model file`

A comment: *Find model file*. Comments are ignored by the Python interpreter and serve as documentation.

14. **Line 14:** `for f in os.listdir(vdir):`
Iterates over `os.listdir(vdir)`, assigning each element to `f`.
15. **Line 15:** `if f.endswith(".onnx"):`
Checks condition: `f.endswith(".onnx")`. The indented block below runs only when this is `True`.
16. **Line 16:** `result["model"] = os.path.join(vdir, f)`
`result["model"] = os.path.join(vdir, f)`
17. **Line 17:** `break`
Exits the innermost loop immediately.
18. **Line 18:** `# Find tokens file`
A comment: *Find tokens file*. Comments are ignored by the Python interpreter and serve as documentation.
19. **Line 19:** `tokens = os.path.join(vdir, "tokens.txt")`
Assigns `os.path.join(vdir, "tokens.txt")` to `tokens`.
20. **Line 20:** `if os.path.isfile(tokens):`
Checks condition: `os.path.isfile(tokens)`. The indented block below runs only when this is `True`.
21. **Line 21:** `result["tokens"] = tokens`
`result["tokens"] = tokens`
22. **Line 22:** `# Find voices file (for multi-speaker models)`
A comment: *Find voices file (for multi-speaker models)*. Comments are ignored by the Python interpreter and serve as documentation.
23. **Line 23:** `voices = os.path.join(vdir, "voices.bin")`
Assigns `os.path.join(vdir, "voices.bin")` to `voices`.
24. **Line 24:** `if os.path.isfile(voices):`
Checks condition: `os.path.isfile(voices)`. The indented block below runs only when this is `True`.
25. **Line 25:** `result["voices_file"] = voices`
`result["voices_file"] = voices`
26. **Line 26:** `# Shared espeak-ng-data`
A comment: *Shared espeak-ng-data*. Comments are ignored by the Python interpreter and serve as documentation.
27. **Line 27:** `shared = os.path.join(paths.models_dir(), "shared", "espeak-ng-data")`

Returns the directory where downloaded TTS voice models are stored. Respects any custom path set in Settings > Paths. Creates the directory if needed.

28. **Line 28:** `if os.path.isdir(shared) and entry.get("frontend") != "char":`

Checks condition: `os.path.isdir(shared) and entry.get("frontend") != "char"`. The indented block below runs only when this is `True`.

29. **Line 29:** `result["data_dir"] = shared`

`result["data_dir"] = shared`

30. **Line 30:** `return result`

Returns `result` to the caller.

47.4 Key Takeaways

- The **catalog** is a nested data structure: TTS → language → variant → voice
 - **ModelStore** tracks which variants are installed on disk
 - **resolve_voice_files()** maps voice entries to actual file paths
 - **Shared resources** (espeak-ng-data) are stored in a common directory
 - The catalog has **14 TTS engines** with hundreds of voices across dozens of languages
-

TTS Engine Integration

i Info: Current implementation (2026.3.1). The sherpa-onnx wrapper described here is still the engine for the ONNX and built-in voices, and it now sits behind two more layers:

- `tts/engine.py` is reached through `tts/models.py`, which resolves a voice entry — including the pip-installed Voice Lab engines' built-in and cloned voices — and hands sherpa-onnx the right model files.
- `voicelab.VoicelabEngine` implements the same `synthesize()` contract by talking to a worker subprocess running on that engine's own interpreter, so PyTorch and CUDA never load in the application process (see [Chapter 76](#)).
- Reference audio for cloning is decoded without Ffmpeg when TorchCodec cannot load ([Chapter 79](#)), and the engine cache key includes the provider, so a compute change rebuilds the engine.

Learning Objectives

- Understand how multiple TTS engines are unified under one interface
- See the engine factory pattern in action
- Understand pitch, volume, and speed post-processing

48.1 Engine Interface

```
# All engines implement the same contract:
class TtsEngine:
    sample_rate: int = 24000

    def synthesize(
        self, text: str, sid: int = 0,
        speed: float = 1.0, pitch: float = 1.0,
        volume: float = 1.0,
    ) -> np.ndarray:
        """Return int16 audio samples at sample_rate."""
        ...

    def close(self):
        """Clean up resources."""
        pass
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# All engines implement the same contract:`

A comment: *All engines implement the same contract:*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `class TtsEngine:`

Abstract base class that all TTS engines must implement. Defines the `synthesize()` contract: takes text + parameters, returns `int16` audio samples.

3. **Line 3:** `sample_rate: int = 24000`

`sample_rate: int = 24000`

4. **Line 4:** `def synthesize(`

`def synthesize(`

5. **Line 5:** `self, text: str, sid: int = 0,`

`self, text: str, sid: int = 0,`

6. **Line 6:** `speed: float = 1.0, pitch: float = 1.0,`

`speed: float = 1.0, pitch: float = 1.0,`

7. **Line 7:** `volume: float = 1.0,`

`volume: float = 1.0,`

8. **Line 8:** `) -> np.ndarray:`

`) -> np.ndarray:`

9. **Line 9:** `"""Return int16 audio samples at sample_rate."""`

Docstring: *Return int16 audio samples at sample_rate.*. Documents the purpose of the following code block.

10. **Line 10:** `...`

Ellipsis placeholder: indicates an abstract method that subclasses must implement.

11. **Line 11:** `def close(self):`

Defines function `close`: Releases engine resources (loaded models, subprocess handles). Called when switching voices or shutting down.

12. **Line 12:** `"""Clean up resources."""`

Docstring: *Clean up resources.*. Documents the purpose of the following code block.

13. **Line 13:** `pass`

Placeholder: does nothing. Used where Python requires a statement syntactically.

Four Engine Implementations

Engine	Library	Runs In	Voice Cloning?
SherpaEngine	sherpa-onnx	Main process	No
XtTsCloneEngine	coqui-tts	Subprocess	Yes (XTTS v2)
OmniVoiceEngine	omnivoice-triton	Subprocess	Yes (zero-shot)
OmniVoiceServerEngine	omnivoice-server	HTTP server	Yes (server-side)

48.2 Engine Factory

```
def get_engine(voice_entry: dict) -> TtsEngine:
    """Factory: create the right engine for a voice."""
    engine_type = voice_entry.get("engine", "vits")

    if engine_type == "xtts":
        from ..clone import CloneClient, XtTsCloneEngine
        client = CloneClient()
        return XtTsCloneEngine(voice_entry, client)

    elif engine_type == "omnivoice":
        from ..omnivoice import OmniVoiceEngine
        return OmniVoiceEngine(voice_entry)

    elif engine_type == "omnivoice_server":
        from ..omnivoice_server import OmniVoiceServerEngine
        return OmniVoiceServerEngine(voice_entry)

    else:
        return SherpaEngine(voice_entry) # Default: sherpa-onnx
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def get_engine(voice_entry: dict) -> TtsEngine:`
Abstract base class that all TTS engines must implement. Defines the `synthesize()` contract: takes text + parameters, returns int16 audio samples.
2. **Line 2:** `"""Factory: create the right engine for a voice."""`
Docstring: *Factory: create the right engine for a voice..* Documents the purpose of the following code block.
3. **Line 3:** `engine_type = voice_entry.get("engine", "vits")`
Assigns `voice_entry.get("engine", "vits")` to `engine_type`.
4. **Line 4:** `if engine_type == "xtts":`
Checks condition: `engine_type == "xtts"`. The indented block below runs only when this is `True`.
5. **Line 5:** `from ..clone import CloneClient, XtTsCloneEngine`
Voice cloning engine using Coqui TTS (XTTS v2). Runs in a separate subprocess for isolation. Requires a reference audio sample.
6. **Line 6:** `client = CloneClient()`
Manages the XTTS v2 subprocess. Starts the clone server process, sends synthesis requests, and receives audio back.
7. **Line 7:** `return XtTsCloneEngine(voice_entry, client)`
Voice cloning engine using Coqui TTS (XTTS v2). Runs in a separate subprocess for isolation. Requires a reference audio sample.
8. **Line 8:** `elif engine_type == "omnivoice":`
Alternative condition: `engine_type == "omnivoice"`. Checked only if all previous conditions were `False`.
9. **Line 9:** `from ..omnivoice import OmniVoiceEngine`
Zero-shot voice cloning engine using OmniVoice-Triton. Runs in a subprocess. Can clone any voice from a short audio clip.
10. **Line 10:** `return OmniVoiceEngine(voice_entry)`
Zero-shot voice cloning engine using OmniVoice-Triton. Runs in a subprocess. Can clone any voice from a short audio clip.
11. **Line 11:** `elif engine_type == "omnivoice_server":`
Alternative condition: `engine_type == "omnivoice_server"`. Checked only if all previous conditions were `False`.
12. **Line 12:** `from ..omnivoice_server import OmniVoiceServerEngine`
Voice cloning via a remote OmniVoice HTTP server. Communicates over HTTP API instead of subprocess.

13. **Line 13:** `return OmniVoiceServerEngine(voice_entry)`
Voice cloning via a remote OmniVoice HTTP server. Communicates over HTTP API instead of subprocess.
14. **Line 14:** `else:`
Fallback: executes when none of the above conditions matched.
15. **Line 15:** `return SherpaEngine(voice_entry) # Default: sherpa-onnx`
The primary TTS engine using sherpa-onnx. Runs in-process (no subprocess). Supports VITS, Kokoro, and Matcha voice models. No voice cloning.

48.3 Post-Processing Pipeline

```
# After synthesis, audio goes through post-processing:
def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):
    # 1. Core synthesis (sherpa-onnx / subprocess)
    audio = self._tts.generate(text=text, sid=sid, speed=speed)
    samples = np.array(audio.samples, dtype=np.int16)

    # 2. Pitch adjustment
    if pitch != 1.0:
        samples = _shift_pitch(samples, pitch)

    # 3. Volume adjustment
    if volume != 1.0:
        samples = _apply_volume(samples, volume)

    return samples

# Pitch shifting via resampling
def _shift_pitch(samples, pitch):
    new_length = int(len(samples) / pitch)
    indices = np.linspace(0, len(samples) - 1, new_length)
    shifted = np.interp(indices, np.arange(len(samples)),
                        samples.astype(np.float64))
    return shifted.astype(np.int16)

# Volume scaling with clipping
def _apply_volume(samples, volume):
    scaled = samples.astype(np.float64) * volume
    return np.clip(scaled, -32768, 32767).astype(np.int16)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# After synthesis, audio goes through post-processing:`

A comment: *After synthesis, audio goes through post-processing:*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):`

Defines function `synthesize` : Core synthesis method. Takes text string, voice ID (sid), speed/pitch/volume floats. Returns a NumPy array of int16 audio samples at the engine's `sample_rate`.

3. **Line 3:** `# 1. Core synthesis (sherpa-onnx / subprocess)`

A comment: *1. Core synthesis (sherpa-onnx / subprocess)*. Comments are ignored by the Python interpreter and serve as documentation.

4. **Line 4:** `audio = self._tts.generate(text=text, sid=sid, speed=speed)`

Assigns `self._tts.generate(text=text, sid=sid, speed=speed)` to `audio` .

5. **Line 5:** `samples = np.array(audio.samples, dtype=np.int16)`

Assigns `np.array(audio.samples, dtype=np.int16)` to `samples` .

6. **Line 6:** `# 2. Pitch adjustment`

A comment: *2. Pitch adjustment*. Comments are ignored by the Python interpreter and serve as documentation.

7. **Line 7:** `if pitch != 1.0:`

Checks condition: `pitch != 1.0` . The indented block below runs only when this is `True` .

8. **Line 8:** `samples = _shift_pitch(samples, pitch)`

Assigns `_shift_pitch(samples, pitch)` to `samples` .

9. **Line 9:** `# 3. Volume adjustment`

A comment: *3. Volume adjustment*. Comments are ignored by the Python interpreter and serve as documentation.

10. **Line 10:** `if volume != 1.0:`

Checks condition: `volume != 1.0` . The indented block below runs only when this is `True` .

11. **Line 11:** `samples = _apply_volume(samples, volume)`

Assigns `_apply_volume(samples, volume)` to `samples` .

12. **Line 12:** `return samples`

Returns `samples` to the caller.

13. **Line 13:** `# Pitch shifting via resampling`

A comment: *Pitch shifting via resampling*. Comments are ignored by the Python interpreter and serve as documentation.

14. **Line 14:** `def _shift_pitch(samples, pitch):`

Defines function `_shift_pitch` accepting parameters (samples, pitch).

15. **Line 15:** `new_length = int(len(samples) / pitch)`

Assigns `int(len(samples) / pitch)` to `new_length`.

16. **Line 16:** `indices = np.linspace(0, len(samples) - 1, new_length)`

Assigns `np.linspace(0, len(samples) - 1, new_length)` to `indices`.

17. **Line 17:** `shifted = np.interp(indices, np.arange(len(samples)),`

Assigns `np.interp(indices, np.arange(len(samples)),` to `shifted`.

18. **Line 18:** `samples.astype(np.float64)`

`samples.astype(np.float64))`

19. **Line 19:** `return shifted.astype(np.int16)`

Returns `shifted.astype(np.int16)` to the caller.

20. **Line 20:** `# Volume scaling with clipping`

A comment: *Volume scaling with clipping*. Comments are ignored by the Python interpreter and serve as documentation.

21. **Line 21:** `def _apply_volume(samples, volume):`

Defines function `_apply_volume` accepting parameters (samples, volume).

22. **Line 22:** `scaled = samples.astype(np.float64) * volume`

Assigns `samples.astype(np.float64) * volume` to `scaled`.

23. **Line 23:** `return np.clip(scaled, -32768, 32767).astype(np.int16)`

Returns `np.clip(scaled, -32768, 32767).astype(np.int16)` to the caller.

48.4 Key Takeaways

- All engines implement the **same interface** (synthesize → int16 numpy array)
- The **factory pattern** selects the engine based on `voice_entry["engine"]`

- Heavy engines (XTTS, OmniVoice) run in **subprocesses**
- **Post-processing** (pitch, volume) happens after synthesis, before saving
- Engine instances are **cached** to avoid reloading models

Model Downloader & Artifacts

i Info: Current implementation (2026.3.1). The downloader itself is unchanged, but it is no longer the only way an engine arrives. Engines whose catalog entry carries `requires_package` (the Voice Lab engines and both OmniVoice engines) are *excluded* from the Download/Remove list and are installed with pip into their own Python environment instead ([Chapter 77](#)). Those engines' models are fetched from Hugging Face by the engine itself on first synthesis. The shared artifacts (`vocos-22khz` for Matcha, the `espeak-ng` data for MMS) still arrive through this downloader.

Learning Objectives

- Understand the download pipeline for voice models
 - Handle tar.bz2 extraction and file organization
 - See progress reporting and cancellation
-

49.1 Download Pipeline

```
class ModelDownloader:
    """Downloads and manages TTS voice model artifacts."""

    def download_variant(self, tts_id, lang, variant_id,
                        progress=None, cancel_event=None):
        """Download a voice variant (all its files)."""
        tts = catalog.find_tts(tts_id)
        variant = catalog.find_variant(tts, lang, variant_id)

        # Determine download URL from artifact name
        artifact = variant.get("artifact", "")
        url = self._build_url(tts_id, lang, artifact)

        # Download to temp location
        dest_dir = self.store.variant_dir(tts_id, lang, variant_id)
        os.makedirs(dest_dir, exist_ok=True)

        if artifact.endswith(".tar.bz2"):
            self._download_and_extract(url, dest_dir, progress, cancel_event)
        else:
            self._download_file(url, dest_dir, progress, cancel_event)

        # Mark as installed
        self.store.mark_installed(tts_id, lang, variant_id, dest_dir)

    def _download_and_extract(self, url, dest, progress, cancel):
        """Download tar.bz2 and extract to destination."""
        import tarfile, tempfile
        with tempfile.NamedTemporaryFile(suffix=".tar.bz2") as tmp:
            _download_file(url, tmp.name, progress, cancel)
            with tarfile.open(tmp.name, "r:bz2") as tar:
                tar.extractall(dest)

    def remove_variant(self, tts_id, lang, variant_id):
        """Remove a downloaded variant from disk."""
        dest = self.store.variant_dir(tts_id, lang, variant_id)
        shutil.rmtree(dest, ignore_errors=True)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `class ModelDownloader:`
HTTP download manager. Handles chunked downloads, progress callbacks, and resume support for large model files.
2. **Line 2:** `"""Downloads and manages TTS voice model artifacts."""`
Docstring: *Downloads and manages TTS voice model artifacts.*. Documents the purpose of the following code block.
3. **Line 3:** `def download_variant(self, tts_id, lang, variant_id,`
Downloads a voice model variant (e.g. 'en_US/medium') from the catalog URL to the local models directory. Shows progress in the GUI.
4. **Line 4:** `progress=None, cancel_event=None):`
Assigns `None, cancel_event=None):` to `progress` .
5. **Line 5:** `"""Download a voice variant (all its files)."""`
Docstring: *Download a voice variant (all its files).*. Documents the purpose of the following code block.
6. **Line 6:** `tts = catalog.find_tts(tts_id)`
Assigns `catalog.find_tts(tts_id)` to `tts` .
7. **Line 7:** `variant = catalog.find_variant(tts, lang, variant_id)`
Assigns `catalog.find_variant(tts, lang, variant_id)` to `variant` .
8. **Line 8:** `# Determine download URL from artifact name`
A comment: *Determine download URL from artifact name.* Comments are ignored by the Python interpreter and serve as documentation.
9. **Line 9:** `artifact = variant.get("artifact", "")`
Assigns `variant.get("artifact", "")` to `artifact` .
10. **Line 10:** `url = self._build_url(tts_id, lang, artifact)`
Assigns `self._build_url(tts_id, lang, artifact)` to `url` .
11. **Line 11:** `# Download to temp location`
A comment: *Download to temp location.* Comments are ignored by the Python interpreter and serve as documentation.
12. **Line 12:** `dest_dir = self.store.variant_dir(tts_id, lang, variant_id)`
Assigns `self.store.variant_dir(tts_id, lang, variant_id)` to `dest_dir` .
13. **Line 13:** `os.makedirs(dest_dir, exist_ok=True)`
`os.makedirs(dest_dir, exist_ok=True)`

14. **Line 14:** `if artifact.endswith(".tar.bz2"):`
Checks condition: `artifact.endswith(".tar.bz2")` . The indented block below runs only when this is `True` .
15. **Line 15:** `self._download_and_extract(url, dest_dir, progress, cancel_event)`
Calls `self._download_and_extract()` to perform an operation.
16. **Line 16:** `else:`
Fallback: executes when none of the above conditions matched.
17. **Line 17:** `self._download_file(url, dest_dir, progress, cancel_event)`
Calls `self._download_file()` to perform an operation.
18. **Line 18:** `# Mark as installed`
A comment: *Mark as installed*. Comments are ignored by the Python interpreter and serve as documentation.
19. **Line 19:** `self.store.mark_installed(tts_id, lang, variant_id, dest_dir)`
`self.store.mark_installed(tts_id, lang, variant_id, dest_dir)`
20. **Line 20:** `def _download_and_extract(self, url, dest, progress, cancel):`
Defines function `_download_and_extract` accepting parameters (self, url, dest, progress, ...).
21. **Line 21:** `"""Download tar.bz2 and extract to destination."""`
Docstring: *Download tar.bz2 and extract to destination..* Documents the purpose of the following code block.
22. **Line 22:** `import tarfile, tempfile`
Imports the `tarfile, tempfile` module.
23. **Line 23:** `with tempfile.NamedTemporaryFile(suffix=".tar.bz2") as tmp:`
Context manager: `tempfile.NamedTemporaryFile(suffix=".tar.bz2") as tmp` . Automatically manages resource cleanup.
24. **Line 24:** `_download_file(url, tmp.name, progress, cancel)`
Calls function `_download_file()` .
25. **Line 25:** `with tarfile.open(tmp.name, "r:bz2") as tar:`
Context manager: `tarfile.open(tmp.name, "r:bz2") as tar` . Automatically manages resource cleanup.
26. **Line 26:** `tar.extractall(dest)`
`tar.extractall(dest)`
27. **Line 27:** `def remove_variant(self, tts_id, lang, variant_id):`

Defines function `remove_variant` accepting parameters (`self`, `tts_id`, `lang`, `variant_id`).

28. **Line 28:** `"""Remove a downloaded variant from disk."""`

Docstring: *Remove a downloaded variant from disk.* Documents the purpose of the following code block.

29. **Line 29:** `dest = self.store.variant_dir(tts_id, lang, variant_id)`

Assigns `self.store.variant_dir(tts_id, lang, variant_id)` to `dest`.

30. **Line 30:** `shutil.rmtree(dest, ignore_errors=True)`

`shutil.rmtree(dest, ignore_errors=True)`

49.2 Artifact Sources

```
# Voice models come from two sources:

# 1. GitHub releases (Piper, Matcha, Kitten)
url = f"https://github.com/rhasspy/piper/releases/download/..."

# 2. HuggingFace repos (Kokoro)
# Catalog entries specify hf_repo and hf_files:
{
    "artifact": "kokoro-multi-lang-v1_1.tar.bz2",
    "hf_repo": "hexgrad/Kokoro-82M",
    "hf_files": ["kokoro-v1_1.onnx", ...],
}

# 3. Shared artifacts (espeak-ng-data, vocoders)
{
    "shared_artifacts": [
        {"artifact": "vocos-22khz-univ.onnx", "kind": "file",
         "dest": "shared/vocoders"},
    ],
}
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Voice models come from two sources:`

A comment: *Voice models come from two sources:*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `# 1. GitHub releases (Piper, Matcha, Kitten)`

A comment: *1. GitHub releases (Piper, Matcha, Kitten)*. Comments are ignored by the Python interpreter and serve as documentation.

3. **Line 3:** `url = f"https://github.com/rhasspy/piper/releases/download/..."`

Assigns `f"https://github.com/rhasspy/piper/releases/download/..."` to `url`.

4. **Line 4:** `# 2. HuggingFace repos (Kokoro)`

A comment: *2. HuggingFace repos (Kokoro)*. Comments are ignored by the Python interpreter and serve as documentation.

5. **Line 5:** `# Catalog entries specify hf_repo and hf_files:`

A comment: *Catalog entries specify hf_repo and hf_files:*. Comments are ignored by the Python interpreter and serve as documentation.

6. **Line 6:** `{`

`{`

7. **Line 7:** `"artifact": "kokoro-multi-lang-v1_1.tar.bz2",`

`"artifact": "kokoro-multi-lang-v1_1.tar.bz2",`

8. **Line 8:** `"hf_repo": "hexgrad/Kokoro-82M",`

`"hf_repo": "hexgrad/Kokoro-82M",`

9. **Line 9:** `"hf_files": ["kokoro-v1_1.onnx", ...],`

`"hf_files": ["kokoro-v1_1.onnx", ...],`

10. **Line 10:** `}`

`}`

11. **Line 11:** `# 3. Shared artifacts (espeak-ng-data, vocoders)`

A comment: *3. Shared artifacts (espeak-ng-data, vocoders)*. Comments are ignored by the Python interpreter and serve as documentation.

12. **Line 12:** `{`

`{`

```
13. Line 13: "shared_artifacts": [  
    "shared_artifacts": [  
  
14. Line 14: {"artifact": "vocos-22khz-univ.onnx", "kind": "file",  
    {"artifact": "vocos-22khz-univ.onnx", "kind": "file",  
  
15. Line 15: "dest": "shared/vocoders"},  
    "dest": "shared/vocoders"},  
  
16. Line 16: ],  
    ],  
  
17. Line 17: }  
    }
```

49.3 Key Takeaways

- Models are downloaded as **tar.bz2 archives** and extracted to the models directory
 - The **ModelStore** tracks what's installed via a JSON state file
 - **Progress callbacks** update the GUI gauge during download
 - **Cancellation** is checked between chunks via `threading.Event`
 - **Shared artifacts** (vocoders, espeak data) are stored once and referenced by multiple voices
-

Compute Back-end Detection

i Info: Current implementation (2026.3.1). `compute.py` now answers three separate questions, and they must not be confused:

- *Is there an NVIDIA GPU?* — `has_nvidia_gpu()`, a cached `nvidia-smi` driver probe (no PyTorch import, no VRAM or model check, so any CUDA-capable card qualifies).
- *May an ONNX engine use it?* — that additionally needs the optional GPU runtime downloaded from Settings, because sherpa-onnx ships its own runtime and a second `onnxruntime` in one process corrupts the graph loader. The pip-installed Voice Lab engines need only the driver: they bring their own PyTorch.
- *How much of the machine may a run use?* — `cpu_threads()` returns the top of the **80–95 %** band of logical CPUs, and `cuda_device_index()` is always `0` (whatever card the driver exposes).

The choice is offered per category next to every Preview button. Full detail, code and line annotations: [Chapter 78](#).

Learning Objectives

- Understand how AIVS detects available compute back-ends
 - See the GPU detection logic and its implications
 - Understand the compute mode resolution
-

50.1 Compute Modes

```
# From compute.py
COMPUTE_AUTO = "auto"
COMPUTE_CPU = "cpu"
COMPUTE_CUDA = "cuda"
COMPUTE_DML = "dml" # DirectML (deprecated)

def detect(force=False) -> dict:
    """Detect available compute back-ends."""
    result = {"cpu": True, "cuda": False, "dml": False}

    # CUDA requires both NVIDIA GPU AND downloaded runtime
    if _has_nvidia_gpu() and _runtime_installed():
        result["cuda"] = True

    return result

def _has_nvidia_gpu() -> bool:
    """Check for NVIDIA GPU via nvidia-smi."""
    try:
        result = subprocess.run(
            ["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
            capture_output=True, text=True, timeout=10,
        )
        return result.returncode == 0 and bool(result.stdout.strip())
    except Exception:
        return False

def resolve_compute(mode: str) -> str:
    """Resolve 'auto' to the best available mode."""
    if mode != COMPUTE_AUTO:
        return mode
    det = detect()
    if det["cuda"]:
        return COMPUTE_CUDA
    return COMPUTE_CPU

def available_compute() -> list[str]:
    """Return list of available compute modes."""
    det = detect()
    modes = [COMPUTE_CPU]
    if det["cuda"]:
        modes.append(COMPUTE_CUDA)
    return modes

def provider_for(mode: str) -> str:
    """Map compute mode to sherpa-onnx provider name."""
    return {COMPUTE_CUDA: "cuda", COMPUTE_CPU: "cpu"}.get(mode, "cpu")
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From compute.py`

A comment indicating the source file: *From compute.py*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `COMPUTE_AUTO = "auto"`

Assigns `"auto"` to `COMPUTE_AUTO`.

3. **Line 3:** `COMPUTE_CPU = "cpu"`

Assigns `"cpu"` to `COMPUTE_CPU`.

4. **Line 4:** `COMPUTE_CUDA = "cuda"`

Assigns `"cuda"` to `COMPUTE_CUDA`.

5. **Line 5:** `COMPUTE_DML = "dml" # DirectML (deprecated)`

Assigns `"dml" # DirectML (deprecated)` to `COMPUTE_DML`.

6. **Line 6:** `def detect(force=False) -> dict:`

Defines function `detect`: Probes the system for available compute backends. Returns a dict like `{'cpu': True, 'cuda': False, 'dml': False}`.

7. **Line 7:** `"""Detect available compute back-ends."""`

Docstring: *Detect available compute back-ends..* Documents the purpose of the following code block.

8. **Line 8:** `result = {"cpu": True, "cuda": False, "dml": False}`

Assigns `{"cpu": True, "cuda": False, "dml": False}` to `result`.

9. **Line 9:** `# CUDA requires both NVIDIA GPU AND downloaded runtime`

A comment: *CUDA requires both NVIDIA GPU AND downloaded runtime*. Comments are ignored by the Python interpreter and serve as documentation.

10. **Line 10:** `if _has_nvidia_gpu() and _runtime_installed():`

Checks condition: `_has_nvidia_gpu() and _runtime_installed()`. The indented block below runs only when this is `True`.

11. **Line 11:** `result["cuda"] = True`

`result["cuda"] = True`

12. **Line 12:** `return result`

Returns `result` to the caller.

13. **Line 13:** `def _has_nvidia_gpu() -> bool:`
Defines function `_has_nvidia_gpu`, returning `bool`.
14. **Line 14:** `"""Check for NVIDIA GPU via nvidia-smi."""`
Docstring: *Check for NVIDIA GPU via nvidia-smi.* Documents the purpose of the following code block.
15. **Line 15:** `try:`
Begins exception handling. Errors in this block will be caught below.
16. **Line 16:** `result = subprocess.run(`
Assigns `subprocess.run(` to `result`.
17. **Line 17:** `["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],`
`["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],`
18. **Line 18:** `capture_output=True, text=True, timeout=10,`
Assigns `True, text=True, timeout=10,` to `capture_output`.
19. **Line 19:** `)`
`)`
20. **Line 20:** `return result.returncode == 0 and bool(result.stdout.strip())`
Returns `result.returncode == 0 and bool(result.stdout.strip())` to the caller.
21. **Line 21:** `except Exception:`
Catches `Exception` exceptions.
22. **Line 22:** `return False`
Returns `False` to the caller.
23. **Line 23:** `def resolve_compute(mode: str) -> str:`
Defines function `resolve_compute` accepting parameters (mode), returning `str`.
24. **Line 24:** `"""Resolve 'auto' to the best available mode."""`
Docstring: *Resolve 'auto' to the best available mode.* Documents the purpose of the following code block.
25. **Line 25:** `if mode != COMPUTE_AUTO:`
Checks condition: `mode != COMPUTE_AUTO`. The indented block below runs only when this is `True`.
26. **Line 26:** `return mode`

Returns `mode` to the caller.

27. **Line 27:** `det = detect()`

Assigns `detect()` to `det`.

28. **Line 28:** `if det["cuda"]:`

Checks condition: `det["cuda"]`. The indented block below runs only when this is `True`.

29. **Line 29:** `return COMPUTE_CUDA`

Returns `COMPUTE_CUDA` to the caller.

30. **Line 30:** `return COMPUTE_CPU`

Returns `COMPUTE_CPU` to the caller.

31. **Line 31:** `def available_compute() -> list[str]:`

Defines function `available_compute`, returning `list[str]`.

32. **Line 32:** `"""Return list of available compute modes."""`

Docstring: *Return list of available compute modes..* Documents the purpose of the following code block.

33. **Line 33:** `det = detect()`

Assigns `detect()` to `det`.

34. **Line 34:** `modes = [COMPUTE_CPU]`

Assigns `[COMPUTE_CPU]` to `modes`.

35. **Line 35:** `if det["cuda"]:`

Checks condition: `det["cuda"]`. The indented block below runs only when this is `True`.

36. **Line 36:** `modes.append(COMPUTE_CUDA)`

`modes.append(COMPUTE_CUDA)`

37. **Line 37:** `return modes`

Returns `modes` to the caller.

38. **Line 38:** `def provider_for(mode: str) -> str:`

Defines function `provider_for` accepting parameters (mode), returning `str`.

39. **Line 39:** `"""Map compute mode to sherpa-onnx provider name."""`

Docstring: *Map compute mode to sherpa-onnx provider name..* Documents the purpose of the following code block.

40. **Line 40:** `return {COMPUTE_CUDA: "cuda", COMPUTE_CPU: "cpu"}.get(mode, "cpu")`
Returns `{COMPUTE_CUDA: "cuda", COMPUTE_CPU: "cpu"}.get(mode, "cpu")` to the caller.

50.2 Two-Step GPU Requirement

i Info: Key design decision: Even with an NVIDIA GPU, CUDA compute is NOT offered until the user explicitly downloads the GPU runtime from Settings → Compute. This prevents the app from failing on machines with NVIDIA GPUs but without the required CUDA libraries.

```
# Test from test_compute.py
def test_cuda_not_offered_without_downloaded_runtime():
    # Even with NVIDIA GPU, no GPU option until runtime is downloaded
    self._patch(runtime_installed=False, nvidia=True)
    det = compute.detect(force=True)
    self.assertFalse(det["cuda"])
    self.assertNotIn("cuda", compute.available_compute())
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Test from test_compute.py`
A comment indicating the source file: *Test from test_compute.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
2. **Line 2:** `def test_cuda_not_offered_without_downloaded_runtime():`
Defines function `test_cuda_not_offered_without_downloaded_runtime`.
3. **Line 3:** `# Even with NVIDIA GPU, no GPU option until runtime is downloaded`
A comment: *Even with NVIDIA GPU, no GPU option until runtime is downloaded*. Comments are ignored by the Python interpreter and serve as documentation.
4. **Line 4:** `self._patch(runtime_installed=False, nvidia=True)`
Calls `self._patch()` to perform an operation.
5. **Line 5:** `det = compute.detect(force=True)`
Assigns `compute.detect(force=True)` to `det`.
6. **Line 6:** `self.assertFalse(det["cuda"])`
Calls `self.assertFalse()` to perform an operation.
7. **Line 7:** `self.assertNotIn("cuda", compute.available_compute())`
Calls `self.assertNotIn()` to perform an operation.

50.3 Key Takeaways

- **GPU detection** uses `nvidia-smi` subprocess call
 - **CUDA requires two conditions:** NVIDIA GPU present AND runtime downloaded
 - **`resolve_compute("auto")`** picks the best available mode
 - The Recording Dialog shows a **CUDA GPU (OmniVoice)** option when NVIDIA is detected
 - **DML (DirectML)** is no longer offered
-

Main Frame & Menu System

i Info: Current implementation (2026.3.1). The menus now read: *File* (New Project, Open Project), *Edit* (Resume Recording, Restart Project..., Restart All Recording, **Start Selected Recording...**, Remove Project...), *Tools* (Record, Settings) and *Help* (Read Me, User Guide, Addon Development Guide, Accessibility Guidelines, Python & wxPython Book **F1**, **Third-Party Licences**, About). The picker behind *Start Selected Recording...* is implemented in this module (`_picker_entries` + `_StartRecordingDialog`) and is described in [Chapter 80](#). Every document opened from Help goes through one `_open_doc` helper, so source and frozen builds behave the same.

Learning Objectives

- Understand the MainFrame class and its responsibilities
 - See the menu system, keyboard shortcuts, and recent projects
 - Understand the application lifecycle
-

51.1 MainFrame Structure

```
# From gui/main_frame.py
class MainFrame(wx.Frame):
    def __init__(self, parent, settings: Settings, store: ModelStore):
        super().__init__(parent, title=f"{APP_NAME} v{APP_VERSION}", size=(800, 600))
        self.settings = settings
        self.store = store
        self._build_menu()
        self._build_ui()
        self._bind_events()
        self.Centre()
        self.SetStatusText("Ready")

    def _build_menu(self):
        # File menu with New, Open, Recent, Exit
        # Edit menu (future)
        # Tools menu with Record
        # Settings menu with Settings dialog
        # Help menu with About

    def _build_ui(self):
        # Welcome panel with project info
        panel = wx.Panel(self)
        sizer = wx.BoxSizer(wx.VERTICAL)
        welcome = wx.StaticText(panel, label="Welcome to AI Voice Studio")
        sizer.Add(welcome, 0, wx.ALL | wx.ALIGN_CENTER, 20)
        panel.SetSizer(sizer)

    def _bind_events(self):
        # Menu event handlers
        self.Bind(wx.EVT_MENU, self._on_new_project, id=ID_NEW_PROJECT)
        self.Bind(wx.EVT_MENU, self._on_open_project, id=ID_OPEN_PROJECT)
        self.Bind(wx.EVT_MENU, self._on_settings, id=ID_SETTINGS)
        self.Bind(wx.EVT_MENU, self._on_record, id=ID_RECORD)
        self.Bind(wx.EVT_MENU, self._on_exit, id=wx.ID_EXIT)
        self.Bind(wx.EVT_MENU, self._on_about, id=wx.ID_ABOUT)

    def _on_new_project(self, _):
        dlg = NewProjectWizard(self, self.settings, self.store)
        dlg.ShowModal()
        dlg.Destroy()

    def _on_settings(self, _):
        dlg = SettingsDialog(self, self.settings, self.store)
        dlg.ShowModal()
        dlg.Destroy()
        apply_theme(self, self.settings.theme)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From gui/main_frame.py`
A comment indicating the source file: *From gui/main_frame.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
2. **Line 2:** `class MainFrame(wx.Frame):`
The main application window. Contains the menu bar, welcome panel, and status bar. Entry point for all user actions.
3. **Line 3:** `def __init__(self, parent, settings: Settings, store: ModelStore):`
Manages downloaded voice models on disk. Tracks which voices are installed, their file paths, and total disk usage.
4. **Line 4:** `super().__init__(parent, title=f"{APP_NAME} v{APP_VERSION}", size=(800, 600))`
Calls function `super()`.
5. **Line 5:** `self.settings = settings`
Stores `settings` in the instance attribute `settings`.
6. **Line 6:** `self.store = store`
Stores `store` in the instance attribute `store`.
7. **Line 7:** `self._build_menu()`
Calls `self._build_menu()` to perform an operation.
8. **Line 8:** `self._build_ui()`
Calls `self._build_ui()` to perform an operation.
9. **Line 9:** `self._bind_events()`
Calls `self._bind_events()` to perform an operation.
10. **Line 10:** `self.Centre()`
Calls `self.Centre()` to perform an operation.
11. **Line 11:** `self.SetStatusText("Ready")`
Calls `self.SetStatusText()` to perform an operation.
12. **Line 12:** `def _build_menu(self):`
Defines function `_build_menu`: Constructs the menu bar with File (New/Open/Recent/Exit), Edit (Resume/Restart/Remove), Tools (Record/Settings), and Help menus.
13. **Line 13:** `# File menu with New, Open, Recent, Exit`

A comment: *File menu with New, Open, Recent, Exit*. Comments are ignored by the Python interpreter and serve as documentation.

14. **Line 14:** `# Edit menu (future)`

A comment: *Edit menu (future)*. Comments are ignored by the Python interpreter and serve as documentation.

15. **Line 15:** `# Tools menu with Record`

A comment: *Tools menu with Record*. Comments are ignored by the Python interpreter and serve as documentation.

16. **Line 16:** `# Settings menu with Settings dialog`

A comment: *Settings menu with Settings dialog*. Comments are ignored by the Python interpreter and serve as documentation.

17. **Line 17:** `# Help menu with About`

A comment: *Help menu with About*. Comments are ignored by the Python interpreter and serve as documentation.

18. **Line 18:** `def _build_ui(self):`

Defines function `_build_ui` accepting parameters (self).

19. **Line 19:** `# Welcome panel with project info`

A comment: *Welcome panel with project info*. Comments are ignored by the Python interpreter and serve as documentation.

20. **Line 20:** `panel = wx.Panel(self)`

A container window that sits inside a Frame. Used to group child controls and manage tab traversal.

21. **Line 21:** `sizer = wx.BoxSizer(wx.VERTICAL)`

A layout manager that arranges child windows in a horizontal or vertical line. The primary layout mechanism in wxPython.

22. **Line 22:** `welcome = wx.StaticText(panel, label="Welcome to AI Voice Studio")`

A non-editable text label. Used for displaying information to the user.

23. **Line 23:** `sizer.Add(welcome, 0, wx.ALL | wx.ALIGN_CENTER, 20)`

`sizer.Add(welcome, 0, wx.ALL | wx.ALIGN_CENTER, 20)`

24. **Line 24:** `panel.SetSizer(sizer)`

`panel.SetSizer(sizer)`

25. **Line 25:** `def _bind_events(self):`

Defines function `_bind_events` accepting parameters (self).

26. **Line 26:** `# Menu event handlers`

A comment: *Menu event handlers*. Comments are ignored by the Python interpreter and serve as documentation.

27. **Line 27:** `self.Bind(wx.EVT_MENU, self._on_new_project, id=ID_NEW_PROJECT)`
Event type triggered when a menu item is clicked. Bind handlers with `frame.Bind(wx.EVT_MENU, handler, id=item_id)`.
28. **Line 28:** `self.Bind(wx.EVT_MENU, self._on_open_project, id=ID_OPEN_PROJECT)`
Event type triggered when a menu item is clicked. Bind handlers with `frame.Bind(wx.EVT_MENU, handler, id=item_id)`.
29. **Line 29:** `self.Bind(wx.EVT_MENU, self._on_settings, id=ID_SETTINGS)`
Event type triggered when a menu item is clicked. Bind handlers with `frame.Bind(wx.EVT_MENU, handler, id=item_id)`.
30. **Line 30:** `self.Bind(wx.EVT_MENU, self._on_record, id=ID_RECORD)`
Event type triggered when a menu item is clicked. Bind handlers with `frame.Bind(wx.EVT_MENU, handler, id=item_id)`.
31. **Line 31:** `self.Bind(wx.EVT_MENU, self._on_exit, id=wx.ID_EXIT)`
Event type triggered when a menu item is clicked. Bind handlers with `frame.Bind(wx.EVT_MENU, handler, id=item_id)`.
32. **Line 32:** `self.Bind(wx.EVT_MENU, self._on_about, id=wx.ID_ABOUT)`
Event type triggered when a menu item is clicked. Bind handlers with `frame.Bind(wx.EVT_MENU, handler, id=item_id)`.
33. **Line 33:** `def _on_new_project(self, _):`
Defines function `_on_new_project`: Opens the New Project wizard dialog. The wizard guides the user through document import and voice selection.
34. **Line 34:** `dlg = NewProjectWizard(self, self.settings, self.store)`
Two-page wizard for creating a new TTS project. Page 1: import document (PDF/DOCX/TXT). Page 2: select voice and split settings.
35. **Line 35:** `dlg.ShowModal()`
Displays a dialog modally (blocks input to the parent window until the dialog is closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.
36. **Line 36:** `dlg.Destroy()`
Destroys the dialog window and frees its resources. Must be called after `ShowModal()` returns.
37. **Line 37:** `def _on_settings(self, _):`
Defines function `_on_settings`: Opens the Settings dialog. Allows the user to download voices, change compute backend, and adjust preferences.
38. **Line 38:** `dlg = SettingsDialog(self, self.settings, self.store)`

Multi-tab dialog for application settings. Tabs: Voice Downloads, Compute Backend, Recording Defaults, Paths, Developer Mode.

39. **Line 39:** `dlg.ShowModal()`

Displays a dialog modally (blocks input to the parent window until the dialog is closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.

40. **Line 40:** `dlg.Destroy()`

Destroys the dialog window and frees its resources. Must be called after `ShowModal()` returns.

41. **Line 41:** `apply_theme(self, self.settings.theme)`

Applies the selected theme (light/dark/system) to the entire wxPython application by updating system colours and font settings.

51.2 Keyboard Shortcuts

Shortcut	Action
Ctrl+Shift+N	New Project
Ctrl+O	Open Project
Ctrl+Shift+R	Record
Ctrl+,	Settings
Alt+F4	Exit

51.3 Application Lifecycle

```
# From main.py
def main() -> int:
    settings = Settings()
    developer_mode = settings.get("developer_mode", False)
    _setup_logging(developer_mode)
    _init_python_runtime()
    _init_addons()

    app = wx.App(False)
    store = ModelStore()
    frame = MainFrame(None, settings, store)
    apply_theme(frame, settings.theme)
    frame.Show()
    result = app.MainLoop()
    return result or 0
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From main.py`
A comment indicating the source file: *From main.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
2. **Line 2:** `def main() -> int:`
Defines function `main`, returning `int`.
3. **Line 3:** `settings = Settings()`
Loads the persistent application settings from settings.json. Thread-safe; defaults are defined in `_DEFAULTS`. Provides get/set/reset methods.
4. **Line 4:** `developer_mode = settings.get("developer_mode", False)`
Assigns `settings.get("developer_mode", False)` to `developer_mode`.
5. **Line 5:** `_setup_logging(developer_mode)`
Calls function `_setup_logging()`.
6. **Line 6:** `_init_python_runtime()`
Calls function `_init_python_runtime()`.
7. **Line 7:** `_init_addons()`
Calls function `_init_addons()`.
8. **Line 8:** `app = wx.App(False)`
Assigns `wx.App(False)` to `app`.
9. **Line 9:** `store = ModelStore()`
Manages downloaded voice models on disk. Tracks which voices are installed, their file paths, and total disk usage.
10. **Line 10:** `frame = MainFrame(None, settings, store)`
The main application window. Contains the menu bar, welcome panel, and status bar. Entry point for all user actions.
11. **Line 11:** `apply_theme(frame, settings.theme)`
Applies the selected theme (light/dark/system) to the entire wxPython application by updating system colours and font settings.
12. **Line 12:** `frame.Show()`
`frame.Show()`
13. **Line 13:** `result = app.MainLoop()`

Assigns `app.MainLoop()` to `result` .

14. **Line 14:** `return result or 0`

Returns `result or 0` to the caller.

51.4 Key Takeaways

- **MainFrame** is the application hub — menus, status bar, event routing
- Five menus: **File, Edit, Tools, Settings, Help**
- **Keyboard shortcuts** follow Windows conventions (Ctrl+key)
- The application starts: **Settings → Logging → Runtime → Addons → wx.App → MainFrame**
- **apply_theme()** is called after settings changes to update the UI

Settings Dialog

i Info: Current implementation (2026.3.1). The dialog now has a **Voice Clone** page with one section per Voice Lab engine: install / remove the engine, its size and licence, the device selector (CPU / GPU / Auto), per-engine tuning defaults and a **Compute** combo next to each Preview button. Installs run in the engine's *own* Python environment, and the dialog reports the result of an import check when pip finishes. See [Chapter 77](#) and [Chapter 78](#). Package status labels are answered from a cached, non-blocking probe (`venv_packages`) so opening Settings stays instant under a screen reader.

Learning Objectives

- Understand the multi-tab Settings dialog architecture
- See how settings panels are loaded and managed
- Understand the NVDA SettingsPanel pattern

52.1 Settings Dialog Tabs

Tab	Panel Class	Purpose
General	GeneralPanel	Theme, developer mode
Download and remove	DownloadPanel	Download/remove voice models
Available TTS	AvailablePanel	List installed voices, preview
Voice Clone	ClonePanel	Create cloned voices (XTTS v2)
Compute	ComputePanel	GPU detection, OmniVoice install
OmniVoice Server	ServerPanel	HTTP server configuration
DAISY	DaisyPanel	DAISY book settings
Developer	DeveloperPanel	Pip install, log viewer

52.2 Panel Management Pattern

```
# From gui/settings_dialog.py
class SettingsDialog(wx.Dialog):
    def __init__(self, parent, settings, store):
        super().__init__(parent, title="Settings", size=(800, 600))

        # Create a notebook (tab control)
        self._notebook = wx.Notebook(self)

        # Create panels
        self._panels = []
        self._add_panel(GeneralPanel(self._notebook, settings))
        self._add_panel(DownloadPanel(self._notebook, store, downloader))
        self._add_panel(AvailablePanel(self._notebook, store))
        # ... more panels ...

    def _add_panel(self, panel):
        self._notebook.AddPage(panel, panel.title)
        self._panels.append(panel)

    def _on_ok(self, _):
        # Apply all panels
        for panel in self._panels:
            if hasattr(panel, "apply_to_settings"):
                panel.apply_to_settings()
        self.settings.save()
        self.EndModal(wx.ID_OK)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From gui/settings_dialog.py`
A comment indicating the source file: *From gui/settings_dialog.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
2. **Line 2:** `class SettingsDialog(wx.Dialog):`
Multi-tab dialog for application settings. Tabs: Voice Downloads, Compute Backend, Recording Defaults, Paths, Developer Mode.
3. **Line 3:** `def __init__(self, parent, settings, store):`
Defines function `__init__` accepting parameters (self, parent, settings, store).
4. **Line 4:** `super().__init__(parent, title="Settings", size=(800, 600))`
Calls function `super()`.
5. **Line 5:** `# Create a notebook (tab control)`
A comment: *Create a notebook (tab control)*. Comments are ignored by the Python interpreter and serve as documentation.
6. **Line 6:** `self._notebook = wx.Notebook(self)`
A tabbed container. Each tab is a separate panel. Used in the Settings dialog for tabbed pages.
7. **Line 7:** `# Create panels`
A comment: *Create panels*. Comments are ignored by the Python interpreter and serve as documentation.
8. **Line 8:** `self._panels = []`
Stores `[]` in the instance attribute `_panels`.
9. **Line 9:** `self._add_panel(GeneralPanel(self._notebook, settings))`
Calls `self._add_panel()` to perform an operation.
10. **Line 10:** `self._add_panel(DownloadPanel(self._notebook, store, downloader))`
Calls `self._add_panel()` to perform an operation.
11. **Line 11:** `self._add_panel(AvailablePanel(self._notebook, store))`
Calls `self._add_panel()` to perform an operation.
12. **Line 12:** `# ... more panels ...`
A comment: *... more panels ...*. Comments are ignored by the Python interpreter and serve as documentation.
13. **Line 13:** `def _add_panel(self, panel):`

Defines function `_add_panel` accepting parameters (self, panel).

14. **Line 14:** `self._notebook.AddPage(panel, panel.title)`

`self._notebook.AddPage(panel, panel.title)`

15. **Line 15:** `self._panels.append(panel)`

`self._panels.append(panel)`

16. **Line 16:** `def _on_ok(self, _):`

Defines function `_on_ok` accepting parameters (self, _).

17. **Line 17:** `# Apply all panels`

A comment: *Apply all panels*. Comments are ignored by the Python interpreter and serve as documentation.

18. **Line 18:** `for panel in self._panels:`

Iterates over `self._panels`, assigning each element to `panel`.

19. **Line 19:** `if hasattr(panel, "apply_to_settings"):`

Checks condition: `hasattr(panel, "apply_to_settings")`. The indented block below runs only when this is `True`.

20. **Line 20:** `panel.apply_to_settings()`

`panel.apply_to_settings()`

21. **Line 21:** `self.settings.save()`

`self.settings.save()`

22. **Line 22:** `self.EndModal(wx.ID_OK)`

Calls `self.EndModal()` to perform an operation.

52.3 Key Takeaways

- The Settings dialog uses a **wx.Notebook** with 8 tabs
 - Each tab is a **panel class** with its own UI and logic
 - Panels implement **apply_to_settings()** to save their state
 - The dialog follows the **NVDA SettingsPanel pattern** (title, description, apply)
 - Settings are saved **immediately on OK**
-

Model Management Panels

i Info: Current implementation (2026.3.1). Two additions to the panels described here: every panel with a Preview button now builds a **Compute** row from `gui/compute_choice.make_compute_row` (CPU, GPU when an NVIDIA GPU is detected, Auto when both exist), storing the choice under `preview_compute.<category>` and resolving it through `provider_for_preview`; and the available-voice list is enriched with each installed engine's **built-in** voices and any cloned voices, so they can be previewed and recorded everywhere. Package status is answered from the cached per-environment probes in `venv_packages`, never by blocking the UI thread. See [Chapter 78](#).

Learning Objectives

- Understand the Download and Available TTS panels
- See cascading combo-box selection (TTS → language → variant)
- Understand background download with progress

53.1 Cascading Selection Pattern

```
# Three cascading ComboBoxes narrow the selection:
# TTS Engine → Language → Variant → Voice

def populate_tts(combo, tts_list):
    combo.Clear()
    for tts in tts_list:
        combo.Append(tts["name"], tts["id"])

def populate_languages(combo, tts):
    combo.Clear()
    for lang in tts.get("languages", []):
        combo.Append(f"{lang['name']} ({lang['code']})", lang["code"])

def populate_variants(combo, tts, lang_code):
    combo.Clear()
    lang = catalog.find_language(tts, lang_code)
    if not lang: return
    for variant in lang.get("variants", []):
        combo.Append(variant["name"], variant["id"])

# Selection cascades: changing TTS refreshes Languages,
# changing Language refreshes variants
self.tts_combo.Bind(wx.EVT_COMBOBOX, self._on_tts)
self.lang_combo.Bind(wx.EVT_COMBOBOX, self._on_lang)
self.variant_combo.Bind(wx.EVT_COMBOBOX, self._on_variant)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Three cascading ComboBoxes narrow the selection:`

A comment: *Three cascading ComboBoxes narrow the selection:*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `# TTS Engine → Language → Variant → Voice`

A comment: *TTS Engine → Language → Variant → Voice*. Comments are ignored by the Python interpreter and serve as documentation.

3. **Line 3:** `def populate_tts(combo, tts_list):`

Defines function `populate_tts` accepting parameters (`combo`, `tts_list`).

4. **Line 4:** `combo.Clear()`

`combo.Clear()`

5. **Line 5:** `for tts in tts_list:`

Iterates over `tts_list`, assigning each element to `tts`.

6. **Line 6:** `combo.Append(tts["name"], tts["id"])`

`combo.Append(tts["name"], tts["id"])`

7. **Line 7:** `def populate_languages(combo, tts):`

Defines function `populate_languages` accepting parameters (`combo`, `tts`).

8. **Line 8:** `combo.Clear()`

`combo.Clear()`

9. **Line 9:** `for lang in tts.get("languages", []):`

Iterates over `tts.get("languages", [])`, assigning each element to `lang`.

10. **Line 10:** `combo.Append(f"{lang['name']} ({lang['code']})", lang["code"])`

`combo.Append(f"{lang['name']} ({lang['code']})", lang["code"])`

11. **Line 11:** `def populate_variants(combo, tts, lang_code):`

Defines function `populate_variants` accepting parameters (`combo`, `tts`, `lang_code`).

12. **Line 12:** `combo.Clear()`

`combo.Clear()`

13. **Line 13:** `lang = catalog.find_language(tts, lang_code)`

Assigns `catalog.find_language(tts, lang_code)` to `lang` .

14. **Line 14:** `if not lang: return`

Checks condition: `not lang` . The indented block below runs only when this is `True` .

15. **Line 15:** `for variant in lang.get("variants", []):`

Iterates over `lang.get("variants", [])` , assigning each element to `variant` .

16. **Line 16:** `combo.Append(variant["name"], variant["id"])`

`combo.Append(variant["name"], variant["id"])`

17. **Line 17:** `# Selection cascades: changing TTS refreshes languages,`

A comment: *Selection cascades: changing TTS refreshes languages,*. Comments are ignored by the Python interpreter and serve as documentation.

18. **Line 18:** `# changing language refreshes variants`

A comment: *changing language refreshes variants*. Comments are ignored by the Python interpreter and serve as documentation.

19. **Line 19:** `self.tts_combo.Bind(wx.EVT_COMBOBOX, self._on_tts)`

`self.tts_combo.Bind(wx.EVT_COMBOBOX, self._on_tts)`

20. **Line 20:** `self.lang_combo.Bind(wx.EVT_COMBOBOX, self._on_lang)`

`self.lang_combo.Bind(wx.EVT_COMBOBOX, self._on_lang)`

21. **Line 21:** `self.variant_combo.Bind(wx.EVT_COMBOBOX, self._on_variant)`

`self.variant_combo.Bind(wx.EVT_COMBOBOX, self._on_variant)`

53.2 Background Download with Progress

```
class DownloadPanel(_ManagerPanel):
    def _on_download(self, _):
        self._cancel_event = threading.Event()
        self._thread = threading.Thread(
            target=self._download_job,
            daemon=True,
        )
        self._thread.start()

    def _download_job(self):
        def progress(name, done, total):
            wx.PostEvent(self, DownloadProgressEvent(name, done, total))

        try:
            self.downloader.download_variant(tts_id, lang, variant, progress=progress)
            wx.PostEvent(self, DownloadFinishedEvent(True, "Done!"))
        except DownloadCancelled:
            wx.PostEvent(self, DownloadFinishedEvent(False, "Cancelled"))

    def _on_progress(self, evt: DownloadProgressEvent):
        pct = int(evt.done * 100 / evt.total) if evt.total else 0
        self.gauge.SetValue(pct)
        self.progress_label.SetLabel(
            f"Downloading: {evt.done / 1048576:.1f} MB"
        )
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `class DownloadPanel(_ManagerPanel):`
Defines class `DownloadPanel` inheriting from `_ManagerPanel`.
2. **Line 2:** `def _on_download(self, _):`
Defines function `_on_download` accepting parameters (self, _).
3. **Line 3:** `self._cancel_event = threading.Event()`
Stores `threading.Event()` in the instance attribute `_cancel_event`.
4. **Line 4:** `self._thread = threading.Thread(`
Creates a background thread. Used for TTS synthesis, downloads, and other long-running operations that shouldn't block the GUI.
5. **Line 5:** `target=self._download_job,`
Assigns `self._download_job` to `target`.
6. **Line 6:** `daemon=True,`
Assigns `True` to `daemon`.
7. **Line 7:** `)`
`)`
8. **Line 8:** `self._thread.start()`
`self._thread.start()`
9. **Line 9:** `def _download_job(self):`
Defines function `_download_job` accepting parameters (self).
10. **Line 10:** `def progress(name, done, total):`
Defines function `progress` accepting parameters (name, done, total).
11. **Line 11:** `wx.PostEvent(self, DownloadProgressEvent(name, done, total))`
Creates a wxPython `wx.PostEvent` widget.
12. **Line 12:** `try:`
Begins exception handling. Errors in this block will be caught below.
13. **Line 13:** `self.downloader.download_variant(tts_id, lang, variant, progress=progress)`

Downloads a voice model variant (e.g. 'en_US/medium') from the catalog URL to the local models directory. Shows progress in the GUI.

14. **Line 14:** `wx.PostEvent(self, DownloadFinishedEvent(True, "Done!"))`

Creates a wxPython `wx.PostEvent` widget.

15. **Line 15:** `except DownloadCancelled:`

Catches `DownloadCancelled` exceptions.

16. **Line 16:** `wx.PostEvent(self, DownloadFinishedEvent(False, "Cancelled"))`

Creates a wxPython `wx.PostEvent` widget.

17. **Line 17:** `def _on_progress(self, evt: DownloadProgressEvent):`

Defines function `_on_progress` accepting parameters (self, evt).

18. **Line 18:** `pct = int(evt.done * 100 / evt.total) if evt.total else 0`

Assigns `int(evt.done * 100 / evt.total) if evt.total else 0` to `pct`.

19. **Line 19:** `self.gauge.SetValue(pct)`

`self.gauge.SetValue(pct)`

20. **Line 20:** `self.progress_label.SetLabel(`

`self.progress_label.SetLabel(`

21. **Line 21:** `f"Downloading: {evt.done / 1048576:.1f} MB"`

`f"Downloading: {evt.done / 1048576:.1f} MB"`

22. **Line 22:** `)`

`)`

53.3 Key Takeaways

- **Cascading ComboBoxes** narrow selection: TTS → language → variant → voice
- **"Download all variants"** option fetches every quality level of a language
- **Background threading** keeps the GUI responsive during downloads
- **wx.PostEvent()** sends progress updates from worker to GUI
- **Cancellation** via threading.Event checked between download chunks

New Project Wizard

i Info: Current implementation (2026.3.1). The wizard keeps this shape; the additions are the DAISY project types (a DAISY 3 book with embedded text and images, and a DAISY 2.02 audio book) with their book-information fields, which are answered from the *DAISY settings* category as defaults, and the clipboard project type, which takes its text directly instead of a document file. Nothing about the finished project's recording changes: the Recording window opens on the new project exactly as described here.

Learning Objectives

- Understand the two-page wizard flow
 - See how project types affect document processing
 - Understand the clipboard mode
-

54.1 Wizard Pages

```
# Page 1: Project Details
class _DetailsPage(WizardPage):
    # Controls:
    # - Project name (TextCtrl)
    # - Project type (ComboBox): Audio Playlist, DAISY Audio, etc.
    # - Open document button (FileDialog)
    # - Punctuation mode (ComboBox)

# Page 2: Audio Mode
class _ModePage(WizardPage):
    # Controls:
    # - Radio buttons for splitting mode
    # - Description Label (updates when selection changes)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Page 1: Project Details`

A comment: *Page 1: Project Details*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `class _DetailsPage(WizardPage):`

Defines class `_DetailsPage` inheriting from `WizardPage`.

3. **Line 3:** `# Controls:`

A comment: *Controls:*. Comments are ignored by the Python interpreter and serve as documentation.

4. **Line 4:** `# - Project name (TextCtrl)`

A comment: *- Project name (TextCtrl)*. Comments are ignored by the Python interpreter and serve as documentation.

5. **Line 5:** `# - Project type (ComboBox): Audio Playlist, DAISY Audio, etc.`

A comment: *- Project type (ComboBox): Audio Playlist, DAISY Audio, etc..* Comments are ignored by the Python interpreter and serve as documentation.

6. **Line 6:** `# - Open document button (FileDialog)`

A comment: *- Open document button (FileDialog)*. Comments are ignored by the Python interpreter and serve as documentation.

7. **Line 7:** `# - Punctuation mode (ComboBox)`

A comment: *- Punctuation mode (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

8. **Line 8:** `# Page 2: Audio Mode`

A comment: *Page 2: Audio Mode*. Comments are ignored by the Python interpreter and serve as documentation.

9. **Line 9:** `class _ModePage(WizardPage):`

Defines class `_ModePage` inheriting from `WizardPage`.

10. **Line 10:** `# Controls:`

A comment: *Controls:*. Comments are ignored by the Python interpreter and serve as documentation.

11. **Line 11:** `# - Radio buttons for splitting mode`

A comment: *- Radio buttons for splitting mode*. Comments are ignored by the Python interpreter and serve as documentation.

12. **Line 12:** `# - Description label (updates when selection changes)`

A comment: *- Description label (updates when selection changes)*. Comments are ignored by the Python interpreter and serve as documentation.

Project Types

Type	Requires Document?	Output
Audio Playlist	Yes	Individual WAV/MP3 files per segment
DAISY Audio	Yes	DAISY 2.02 book (audio only)
DAISY Audio+Text	Yes	DAISY 2.02 book (audio + text sync)
Clipboard	No	Single segment from clipboard text

54.2 Finish Flow

```
def _on_finish(self, _):
    # 1. Get user selections
    name = self.page_details.name_ctrl.GetValue().strip()
    ptype = self.page_details.selected_project_type()
    mode = self.page_mode.selected()

    # 2. Clipboard mode – no document needed
    if ptype == PROJECT_TYPE_CLIPBOARD:
        # Create project with clipboard text
        project.create_project(pdir, name, "<clipboard>", ...)
        self.EndModal(wx.ID_OK)
        return

    # 3. Parse document
    wx.BeginBusyCursor()
    doc = parse_document(self.source_path)
    if ptype in (PROJECT_TYPE_DAISY_AUDIO, PROJECT_TYPE_DAISY_AUDIO_TEXT):
        segments = split_daisy_chapters(doc.blocks)
    else:
        segments = split_document(doc, mode)
    wx.EndBusyCursor()

    # 4. Create project and save
    project.create_project(pdir, name, self.source_path, mode, segments)
    self.settings.add_recent_project(name, pdir)
    self.EndModal(wx.ID_OK)

    # 5. Open recording window
    dlg = RecordingDialog(self.GetParent(), pdir, self.settings, self.store)
    dlg.ShowModal()
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def _on_finish(self, _):`
Defines function `_on_finish` accepting parameters (self, _).
2. **Line 2:** `# 1. Get user selections`
A comment: *1. Get user selections*. Comments are ignored by the Python interpreter and serve as documentation.
3. **Line 3:** `name = self.page_details.name_ctrl.GetValue().strip()`
Assigns `self.page_details.name_ctrl.GetValue().strip()` to `name`.
4. **Line 4:** `ptype = self.page_details.selected_project_type()`
Assigns `self.page_details.selected_project_type()` to `ptype`.
5. **Line 5:** `mode = self.page_mode.selected()`
Assigns `self.page_mode.selected()` to `mode`.
6. **Line 6:** `# 2. Clipboard mode – no document needed`
A comment: *2. Clipboard mode – no document needed*. Comments are ignored by the Python interpreter and serve as documentation.
7. **Line 7:** `if ptype == PROJECT_TYPE_CLIPBOARD:`
Checks condition: `ptype == PROJECT_TYPE_CLIPBOARD`. The indented block below runs only when this is `True`.
8. **Line 8:** `# Create project with clipboard text`
A comment: *Create project with clipboard text*. Comments are ignored by the Python interpreter and serve as documentation.
9. **Line 9:** `project.create_project(pdir, name, "<clipboard>", ...)`
`project.create_project(pdir, name, "<clipboard>", ...)`
10. **Line 10:** `self.EndModal(wx.ID_OK)`
Calls `self.EndModal()` to perform an operation.
11. **Line 11:** `return`
Returns `None` implicitly (exits the function).
12. **Line 12:** `# 3. Parse document`
A comment: *3. Parse document*. Comments are ignored by the Python interpreter and serve as documentation.
13. **Line 13:** `wx.BeginBusyCursor()`

Creates a wxPython `wx.BeginBusyCursor` widget.

14. **Line 14:** `doc = parse_document(self.source_path)`

Assigns `parse_document(self.source_path)` to `doc`.

15. **Line 15:** `if ptype in (PROJECT_TYPE_DAISY_AUDIO, PROJECT_TYPE_DAISY_AUDIO_TEXT):`

Checks condition: `ptype in (PROJECT_TYPE_DAISY_AUDIO, PROJECT_TYPE_DAISY_AUDIO_TEXT)`. The indented block below runs only when this is `True`.

16. **Line 16:** `segments = split_daisy_chapters(doc.blocks)`

Assigns `split_daisy_chapters(doc.blocks)` to `segments`.

17. **Line 17:** `else:`

Fallback: executes when none of the above conditions matched.

18. **Line 18:** `segments = split_document(doc, mode)`

Assigns `split_document(doc, mode)` to `segments`.

19. **Line 19:** `wx.EndBusyCursor()`

Creates a wxPython `wx.EndBusyCursor` widget.

20. **Line 20:** `# 4. Create project and save`

A comment: *4. Create project and save*. Comments are ignored by the Python interpreter and serve as documentation.

21. **Line 21:** `project.create_project(pdir, name, self.source_path, mode, segments)`

`project.create_project(pdir, name, self.source_path, mode, segments)`

22. **Line 22:** `self.settings.add_recent_project(name, pdir)`

`self.settings.add_recent_project(name, pdir)`

23. **Line 23:** `self.EndModal(wx.ID_OK)`

Calls `self.EndModal()` to perform an operation.

24. **Line 24:** `# 5. Open recording window`

A comment: *5. Open recording window*. Comments are ignored by the Python interpreter and serve as documentation.

25. **Line 25:** `dlg = RecordingDialog(self.GetParent(), pdir, self.settings, self.store)`

Dialog for recording audio for a project. Shows the text to read, records microphone input, and plays TTS reference audio.

26. **Line 26:** `dlg.ShowModal()`

Displays a dialog modally (blocks input to the parent window until the dialog is closed). Returns `wx.ID_OK` or `wx.ID_CANCEL`.

54.3 Key Takeaways

- **Two pages:** Details (name, type, document, punctuation) and Mode (splitting)
 - **4 project types** affect document processing and output format
 - **Clipboard mode** allows recording without a document
 - The wizard **automatically opens the Recording Dialog** after project creation
 - **Busy cursor** is shown during document parsing
-

Recording Dialog

i Info: Current implementation (2026.3.1). The window still opens on a project, but it now accepts a *start position* and a *scope* from the caller (`start_index` , and `single_segment` which becomes an exclusive `end_index` for the worker), which is how *Start Selected Recording...* records one chosen file, or every file from a chosen break to the end. A status line above the Start button reports what was chosen (“Ready: segment 12 of 40 — chapter 12...”)
before the first click. The compute selector is the same per-category combo described in [Chapter 78](#), and *Engine tuning...* edits this project's Voice Lab overrides (see [Chapter 80](#)).

Learning Objectives

- Understand the Recording Dialog's complete UI
- See how voice selection cascades with compute mode
- Understand the recording lifecycle (start → pause → stop → close)

55.1 Recording Dialog Components

```
# The Recording Dialog contains:  
# 1. Project title  
# 2. Text preview (read-only, multi-line)  
# 3. Compute back-end selector (ComboBox)  
# 4. TTS engine selector (ComboBox)  
# 5. Language selector (ComboBox)  
# 6. Variant selector (ComboBox)  
# 7. Voice selector (ComboBox)  
# 8. Rate, Pitch, Volume sliders  
# 9. Output format selector (ComboBox)  
# 10. Start, Pause, Resume, Stop buttons  
# 11. Progress gauge  
# 12. Status label  
# 13. Export DAISY button (conditional)  
# 14. Close button
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# The Recording Dialog contains:`

A comment: *The Recording Dialog contains:*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `# 1. Project title`

A comment: *1. Project title*. Comments are ignored by the Python interpreter and serve as documentation.

3. **Line 3:** `# 2. Text preview (read-only, multi-line)`

A comment: *2. Text preview (read-only, multi-line)*. Comments are ignored by the Python interpreter and serve as documentation.

4. **Line 4:** `# 3. Compute back-end selector (ComboBox)`

A comment: *3. Compute back-end selector (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

5. **Line 5:** `# 4. TTS engine selector (ComboBox)`

A comment: *4. TTS engine selector (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

6. **Line 6:** `# 5. Language selector (ComboBox)`

A comment: *5. Language selector (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

7. **Line 7:** `# 6. Variant selector (ComboBox)`

A comment: *6. Variant selector (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

8. **Line 8:** `# 7. Voice selector (ComboBox)`

A comment: *7. Voice selector (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

9. **Line 9:** `# 8. Rate, Pitch, Volume sliders`

A comment: *8. Rate, Pitch, Volume sliders*. Comments are ignored by the Python interpreter and serve as documentation.

10. **Line 10:** `# 9. Output format selector (ComboBox)`

A comment: *9. Output format selector (ComboBox)*. Comments are ignored by the Python interpreter and serve as documentation.

11. **Line 11:** `# 10. Start, Pause, Resume, Stop buttons`

A comment: *10. Start, Pause, Resume, Stop buttons*. Comments are ignored by the Python interpreter and serve as documentation.

12. **Line 12:** `# 11. Progress gauge`
A comment: *11. Progress gauge*. Comments are ignored by the Python interpreter and serve as documentation.
13. **Line 13:** `# 12. Status label`
A comment: *12. Status label*. Comments are ignored by the Python interpreter and serve as documentation.
14. **Line 14:** `# 13. Export DAISY button (conditional)`
A comment: *13. Export DAISY button (conditional)*. Comments are ignored by the Python interpreter and serve as documentation.
15. **Line 15:** `# 14. Close button`
A comment: *14. Close button*. Comments are ignored by the Python interpreter and serve as documentation.

55.2 Compute-Mode Voice Filtering

```
def _refresh_tts_for_compute(self):
    """Show only TTS engines compatible with selected compute."""
    is_omni = self._is_omnivoice_compute()
    self._voices = []
    for v in self._all_voices:
        engine = v.get("engine", "vits")
        if is_omni:
            if engine in _OMNIVOICE_ENGINES:
                self._voices.append(v)
        else:
            if engine in _ONNX_TTS_ENGINES:
                self._voices.append(v)
    # Refresh TTS combo with filtered voices
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `def _refresh_tts_for_compute(self):`
Defines function `_refresh_tts_for_compute` accepting parameters (self).
2. **Line 2:** `"""Show only TTS engines compatible with selected compute."""`
Docstring: *Show only TTS engines compatible with selected compute.*. Documents the purpose of the following code block.
3. **Line 3:** `is_omni = self._is_omnivoice_compute()`
Assigns `self._is_omnivoice_compute()` to `is_omni`.
4. **Line 4:** `self._voices = []`
Stores `[]` in the instance attribute `_voices`.
5. **Line 5:** `for v in self._all_voices:`
Iterates over `self._all_voices`, assigning each element to `v`.
6. **Line 6:** `engine = v.get("engine", "vits")`
Assigns `v.get("engine", "vits")` to `engine`.
7. **Line 7:** `if is_omni:`
Checks condition: `is_omni`. The indented block below runs only when this is `True`.
8. **Line 8:** `if engine in _OMNIVOICE_ENGINES:`
Checks condition: `engine in _OMNIVOICE_ENGINES`. The indented block below runs only when this is `True`.
9. **Line 9:** `self._voices.append(v)`
`self._voices.append(v)`
10. **Line 10:** `else:`
Fallback: executes when none of the above conditions matched.
11. **Line 11:** `if engine in _ONNX_TTS_ENGINES:`
Checks condition: `engine in _ONNX_TTS_ENGINES`. The indented block below runs only when this is `True`.
12. **Line 12:** `self._voices.append(v)`
`self._voices.append(v)`
13. **Line 13:** `# Refresh TTS combo with filtered voices`

A comment: *Refresh TTS combo with filtered voices*. Comments are ignored by the Python interpreter and serve as documentation.

55.3 Recording Lifecycle

```
# Start → creates SynthesisWorker thread
def _on_start(self, _):
    voice = self._selected_voice()
    params = self._current_params()
    start_index = project.first_pending_index(self.project_dir)

    self._worker = SynthesisWorker(
        segments=self._segments[start_index:],
        voice_entry=voice, params=params,
        output_dir=self.project_dir,
        start_index=start_index,
        on_segment_done=lambda i, t, p: self._segment_done(i, t, p),
        on_all_done=lambda: wx.PostEvent(self, SynthFinishedEvent()),
        on_error=lambda msg: wx.PostEvent(self, SynthErrorEvent(msg)),
        cancel_event=self._cancel_event,
        pause_event=self._pause_event,
        on_status=lambda msg: wx.PostEvent(self, SynthStatusEvent(msg)),
    )
    self._worker.start()

# Pause → sets pause event
def _on_pause(self, _):
    self._worker.pause()
    self._pause_event.set()

# Stop → sets cancel event
def _on_stop(self, _):
    self._worker.cancel()
    self._cancel_event.set()

# Close → saves TTS settings, ends modal
def _on_close(self, _):
    self._save_tts_to_project()
    self.EndModal(wx.ID_CLOSE)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Start → creates SynthesisWorker thread`

A comment: *Start → creates SynthesisWorker thread*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `def _on_start(self, _):`

Defines function `_on_start` accepting parameters (self, _).

3. **Line 3:** `voice = self._selected_voice()`

Assigns `self._selected_voice()` to `voice`.

4. **Line 4:** `params = self._current_params()`

Assigns `self._current_params()` to `params`.

5. **Line 5:** `start_index = project.first_pending_index(self.project_dir)`

Assigns `project.first_pending_index(self.project_dir)` to `start_index`.

6. **Line 6:** `self._worker = SynthesisWorker(`

Background thread that synthesizes text segments into audio files. Handles progress callbacks and cancellation.

7. **Line 7:** `segments=self._segments[start_index:],`

Assigns `self._segments[start_index:]` to `segments`.

8. **Line 8:** `voice_entry=voice, params=params,`

Assigns `voice, params=params` to `voice_entry`.

9. **Line 9:** `output_dir=self.project_dir,`

Assigns `self.project_dir` to `output_dir`.

10. **Line 10:** `start_index=start_index,`

Assigns `start_index` to `start_index`.

11. **Line 11:** `on_segment_done=lambda i, t, p: self._segment_done(i, t, p),`

Creates an anonymous function and assigns it to `on_segment_done`.

12. **Line 12:** `on_all_done=lambda: wx.PostEvent(self, SynthFinishedEvent()),`

Assigns `lambda: wx.PostEvent(self, SynthFinishedEvent())` to `on_all_done`.

13. **Line 13:** `on_error=lambda msg: wx.PostEvent(self, SynthErrorEvent(msg)),`

Creates an anonymous function and assigns it to `on_error` .

14. **Line 14:** `cancel_event=self._cancel_event,`
Assigns `self._cancel_event,` to `cancel_event` .

15. **Line 15:** `pause_event=self._pause_event,`
Assigns `self._pause_event,` to `pause_event` .

16. **Line 16:** `on_status=lambda msg: wx.PostEvent(self, SynthStatusEvent(msg)),`
Creates an anonymous function and assigns it to `on_status` .

17. **Line 17:** `)`
`)`

18. **Line 18:** `self._worker.start()`
`self._worker.start()`

19. **Line 19:** `# Pause → sets pause event`
A comment: *Pause → sets pause event*. Comments are ignored by the Python interpreter and serve as documentation.

20. **Line 20:** `def _on_pause(self, _):`
Defines function `_on_pause` accepting parameters (self, _).

21. **Line 21:** `self._worker.pause()`
`self._worker.pause()`

22. **Line 22:** `self._pause_event.set()`
`self._pause_event.set()`

23. **Line 23:** `# Stop → sets cancel event`
A comment: *Stop → sets cancel event*. Comments are ignored by the Python interpreter and serve as documentation.

24. **Line 24:** `def _on_stop(self, _):`
Defines function `_on_stop` accepting parameters (self, _).

25. **Line 25:** `self._worker.cancel()`
`self._worker.cancel()`

26. **Line 26:** `self._cancel_event.set()`
`self._cancel_event.set()`

27. **Line 27:** `# Close → saves TTS settings, ends modal`
A comment: *Close → saves TTS settings, ends modal*. Comments are ignored by the Python interpreter and serve as documentation.
28. **Line 28:** `def _on_close(self, _):`
Defines function `_on_close` accepting parameters (self, _).
29. **Line 29:** `self._save_tts_to_project()`
Calls `self._save_tts_to_project()` to perform an operation.
30. **Line 30:** `self.EndModal(wx.ID_CLOSE)`
Calls `self.EndModal()` to perform an operation.

55.4 What Changed Since the First Edition

The Recording window grew four features after the book's first edition. All of them are post-recording or pre-recording conveniences; the synthesis pipeline itself (Chapter 56) is unchanged.

1. Non-modal task progress dialog

Recording now shows a `TaskProgressDialog` (Chapter 75) instead of relying on the status line alone. The dialog pulses while segments synthesize, carries a real **Stop recording** button wired to the worker's cancel event, and closes itself when the worker finishes. Because it is non-modal, the Recording window stays usable.

2. OmniVoice options per project

When an OmniVoice engine is selected, an **OmniVoice voice options...** button appears. It stores clone/design options in the project's `tts.omni` dictionary; a summary line beside the button announces the current setting. Voice-library voices (created in Settings) carry their own identity, so for them the button hides and the summary describes the library voice instead.

3. Playlist generation on completion

For *Audio files with playlist* projects, the moment recording completes the dialog reloads `project.json` from disk (the worker thread has been updating segment status while the dialog sat idle) and calls `build_playlists()` — see Chapter 73 for the module itself and its line-by-line walkthrough. The completion message then names the three written files:

```
def _playlist_summary(self) -> str:
    ptype = self.data.get("project_type", "audio_playlist")
    if ptype != "audio_playlist" or not self._playlist_files:
        return ""
    names = ", ".join(os.path.basename(p) for p in self._playlist_files)
    return ("\n\nPlaylists were generated in the project folder "
            f"({names}) - open them in VLC, Windows Media Player, "
            "PotPlayer or any modern player to play the whole "
            "recording in order.")
```

Explain Line by Line Code Description (Block 5)

- Line 1:** `def _playlist_summary(self) -> str:`
Builds the extra paragraph appended to the "Recording complete" message when playlists were written. Returning an empty string for other project types keeps the message clean for DAISY projects.
- Line 2:** `ptype = self.data.get("project_type", "audio_playlist")`
Reads the project type with a default. The guard on the next line returns "" unless this really is an audio-playlist project *and* playlist files exist — `not self._playlist_files` covers the "nothing was recorded" case where `build_playlists` returned an empty list.
- Line 4:** `names = ", ".join(os.path.basename(p) for p in self._playlist_files)`
Compresses each written path to its bare file name with `os.path.basename` and joins them with commas, producing e.g. My Book.m3u8, My Book.pls, My Book.wpl — the user does not need the absolute paths, just the names to look for.
- Lines 5-9:** `return (...)`
Returns a two-sentence note: which files were generated and that they open in VLC, Windows Media Player, PotPlayer "or any modern player". The leading `\n\n` separates it from the DAISY summary that may precede it.

4. MP3/FLAC warning in DAISY projects

Choosing MP3 or FLAC while a DAISY project is open shows a warning every time (no "don't ask again"): DAISY players are much more reliable with WAV, and although the builders now re-encode low-rate MP3s (Chapter 59), WAV remains the recommended output. The Recording window's output-format combo stays enabled — the warning is informational, not a block.

55.5 Key Takeaways

- The Recording Dialog is the **most complex GUI component** in AIVS
- **Compute mode** filters which TTS engines are available
- Segments resume from **first_pending_index** — previously recorded segments are skipped

- **Pause/Resume/Stop** control the background synthesis thread
 - TTS settings are saved to **project.json** on every start and close
 - Post-recording steps read **fresh** project state from disk: DAISY books and playlists are built from what the worker actually saved
-

Background Synthesis Worker

i Info: Current implementation (2026.3.1). `SynthesisWorker` gained an optional exclusive `end_index` : with it set the worker stops cleanly after that one segment, which is what makes “only record the selected file” possible; with it `None` it records from the start position to the end of the project, the original behaviour. Either way playback of the run is unchanged — progress events, instant disk save per segment and the resume-from-first-unsaved rule all still apply, and after the last segment the playlists / DAISY files are written as before.

Learning Objectives

- Understand the complete `SynthesisWorker` implementation
 - See how audio is synthesized, processed, and saved
 - Understand the resume-after-crash pattern
-

56.1 SynthesisWorker Thread

```
class SynthesisWorker(threading.Thread):
    """Background thread that synthesizes all segments."""

    def __init__(self, segments, voice_entry, params, output_dir,
                 start_index, on_segment_done, on_all_done, on_error,
                 cancel_event, pause_event, on_status, ffmpeg_exe=None):
        super().__init__(daemon=True)
        self._segments = segments
        self._voice_entry = voice_entry
        self._params = params
        self._output_dir = output_dir
        self._start_index = start_index
        self._on_done = on_segment_done
        self._on_all_done = on_all_done
        self._on_error = on_error
        self._cancel = cancel_event
        self._pause = pause_event
        self._on_status = on_status
        self._ffmpeg_exe = ffmpeg_exe

    def run(self):
        try:
            engine = get_engine(self._voice_entry)

            for i, seg in enumerate(self._segments):
                # Check cancellation
                if self._cancel.is_set():
                    break

                # Check pause
                while self._pause.is_set():
                    if self._cancel.is_set():
                        break
                    time.sleep(0.1)

                # Synthesize
                self._on_status(f"Recording segment {seg.index}...")
                audio = engine.synthesize(
                    seg.text,
                    speed=self._params["rate"],
                    pitch=self._params["pitch"],
                    volume=self._params["volume"],
                )

                # Save audio
                filename = sanitize_filename(seg.title) + ".wav"
                path = os.path.join(self._output_dir, filename)
                write_wav(path, audio, engine.sample_rate)

                # Convert if needed
                if self._params["output_format"] != "wav" and self._ffmpeg_exe:
                    fmt = self._params["output_format"]
                    out_path = path.replace(".wav", f".{fmt}")
                    convert_wav(path, out_path, fmt, self._ffmpeg_exe)
                    path = out_path

                # Mark segment done (saves to project.json)
                self._on_done(seg.index, seg.title, path)
```

```
self._on_all_done()

except Exception as exc:
    self._on_error(str(exc))
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `class SynthesisWorker(threading.Thread):`
Background thread that synthesizes text segments into audio files. Handles progress callbacks and cancellation.
2. **Line 2:** `"""Background thread that synthesizes all segments."""`
Docstring: *Background thread that synthesizes all segments.*. Documents the purpose of the following code block.
3. **Line 3:** `def __init__(self, segments, voice_entry, params, output_dir,`
`def __init__(self, segments, voice_entry, params, output_dir,`
4. **Line 4:** `start_index, on_segment_done, on_all_done, on_error,`
`start_index, on_segment_done, on_all_done, on_error,`
5. **Line 5:** `cancel_event, pause_event, on_status, ffmpeg_exe=None):`
`cancel_event, pause_event, on_status, ffmpeg_exe=None):`
6. **Line 6:** `super().__init__(daemon=True)`
Calls function `super()` .
7. **Line 7:** `self._segments = segments`
Stores `segments` in the instance attribute `_segments` .
8. **Line 8:** `self._voice_entry = voice_entry`
Stores `voice_entry` in the instance attribute `_voice_entry` .
9. **Line 9:** `self._params = params`
Stores `params` in the instance attribute `_params` .
10. **Line 10:** `self._output_dir = output_dir`
Stores `output_dir` in the instance attribute `_output_dir` .
11. **Line 11:** `self._start_index = start_index`
Stores `start_index` in the instance attribute `_start_index` .
12. **Line 12:** `self._on_done = on_segment_done`
Stores `on_segment_done` in the instance attribute `_on_done` .
13. **Line 13:** `self._on_all_done = on_all_done`
Stores `on_all_done` in the instance attribute `_on_all_done` .

14. **Line 14:** `self._on_error = on_error`
Stores `on_error` in the instance attribute `_on_error`.
15. **Line 15:** `self._cancel = cancel_event`
Stores `cancel_event` in the instance attribute `_cancel`.
16. **Line 16:** `self._pause = pause_event`
Stores `pause_event` in the instance attribute `_pause`.
17. **Line 17:** `self._on_status = on_status`
Stores `on_status` in the instance attribute `_on_status`.
18. **Line 18:** `self._ffmpeg_exe = ffmpeg_exe`
Stores `ffmpeg_exe` in the instance attribute `_ffmpeg_exe`.
19. **Line 19:** `def run(self):`
Defines function `run`: Main synthesis loop. Iterates over segments, calls the TTS engine for each, and saves audio files.
20. **Line 20:** `try:`
Begins exception handling. Errors in this block will be caught below.
21. **Line 21:** `engine = get_engine(self._voice_entry)`
Assigns `get_engine(self._voice_entry)` to `engine`.
22. **Line 22:** `for i, seg in enumerate(self._segments):`
`for i, seg in enumerate(self._segments):`
23. **Line 23:** `# Check cancellation`
A comment: *Check cancellation*. Comments are ignored by the Python interpreter and serve as documentation.
24. **Line 24:** `if self._cancel.is_set():`
Checks condition: `self._cancel.is_set()`. The indented block below runs only when this is `True`.
25. **Line 25:** `break`
Exits the innermost loop immediately.
26. **Line 26:** `# Check pause`
A comment: *Check pause*. Comments are ignored by the Python interpreter and serve as documentation.
27. **Line 27:** `while self._pause.is_set():`
While loop: repeats as long as `self._pause.is_set()`.

28. **Line 28:** `if self._cancel.is_set():`
Checks condition: `self._cancel.is_set()` . The indented block below runs only when this is `True` .

29. **Line 29:** `break`
Exits the innermost loop immediately.

30. **Line 30:** `time.sleep(0.1)`
`time.sleep(0.1)`

31. **Line 31:** `# Synthesize`
A comment: *Synthesize*. Comments are ignored by the Python interpreter and serve as documentation.

32. **Line 32:** `self._on_status(f"Recording segment {seg.index}...")`
Calls `self._on_status()` to perform an operation.

33. **Line 33:** `audio = engine.synthesize(`
Assigns `engine.synthesize(` to `audio` .

34. **Line 34:** `seg.text,`
`seg.text,`

35. **Line 35:** `speed=self._params["rate"],`
Assigns `self._params["rate"],` to `speed` .

36. **Line 36:** `pitch=self._params["pitch"],`
Assigns `self._params["pitch"],` to `pitch` .

37. **Line 37:** `volume=self._params["volume"],`
Assigns `self._params["volume"],` to `volume` .

38. **Line 38:** `)`
`)`

39. **Line 39:** `# Save audio`
A comment: *Save audio*. Comments are ignored by the Python interpreter and serve as documentation.

40. **Line 40:** `filename = sanitize_filename(seg.title) + ".wav"`
Removes or replaces characters that are illegal in filenames (Windows: `<>:"/\\|?*).` Truncates to `max_length` to avoid filesystem limits.

41. **Line 41:** `path = os.path.join(self._output_dir, filename)`

Assigns `os.path.join(self._output_dir, filename)` to `path` .

42. **Line 42:** `write_wav(path, audio, engine.sample_rate)`

Calls function `write_wav()` .

43. **Line 43:** `# Convert if needed`

A comment: *Convert if needed*. Comments are ignored by the Python interpreter and serve as documentation.

44. **Line 44:** `if self._params["output_format"] != "wav" and self._ffmpeg_exe:`

Checks condition: `self._params["output_format"] != "wav" and self._ffmpeg_exe` . The indented block below runs only when this is `True` .

45. **Line 45:** `fmt = self._params["output_format"]`

Assigns `self._params["output_format"]` to `fmt` .

46. **Line 46:** `out_path = path.replace(".wav", f".{fmt}")`

Assigns `path.replace(".wav", f".{fmt}")` to `out_path` .

47. **Line 47:** `convert_wav(path, out_path, fmt, self._ffmpeg_exe)`

Calls function `convert_wav()` .

48. **Line 48:** `path = out_path`

Assigns `out_path` to `path` .

49. **Line 49:** `# Mark segment done (saves to project.json)`

A comment: *Mark segment done (saves to project.json)*. Comments are ignored by the Python interpreter and serve as documentation.

50. **Line 50:** `self._on_done(seg.index, seg.title, path)`

Calls `self._on_done()` to perform an operation.

51. **Line 51:** `self._on_all_done()`

Calls `self._on_all_done()` to perform an operation.

52. **Line 52:** `except Exception as exc:`

Catches `Exception` exceptions.

53. **Line 53:** `self._on_error(str(exc))`

Calls `self._on_error()` to perform an operation.

56.2 Resume Pattern

```
# The key insight: segments are saved IMMEDIATELY to disk
# When the app restarts, first_pending_index() finds where to resume

# In RecordingDialog._on_start:
start_index = project.first_pending_index(self.project_dir)
# If segments 1-5 are done, start_index = 5
# Worker processes segments[5:] (segments 6, 7, 8, ...)
```

Explain Line by Line Code Description (Block 2)

- Line 1:** `# The key insight: segments are saved IMMEDIATELY to disk`
A comment: *The key insight: segments are saved IMMEDIATELY to disk.* Comments are ignored by the Python interpreter and serve as documentation.
- Line 2:** `# When the app restarts, first_pending_index() finds where to resume`
A comment: *When the app restarts, first_pending_index() finds where to resume.* Comments are ignored by the Python interpreter and serve as documentation.
- Line 3:** `# In RecordingDialog._on_start:`
A comment: *In RecordingDialog._on_start:.* Comments are ignored by the Python interpreter and serve as documentation.
- Line 4:** `start_index = project.first_pending_index(self.project_dir)`
Assigns `project.first_pending_index(self.project_dir)` to `start_index`.
- Line 5:** `# If segments 1-5 are done, start_index = 5`
A comment: *If segments 1-5 are done, start_index = 5.* Comments are ignored by the Python interpreter and serve as documentation.
- Line 6:** `# Worker processes segments[5:] (segments 6, 7, 8, ...)`
A comment: *Worker processes segments[5:] (segments 6, 7, 8, ...).* Comments are ignored by the Python interpreter and serve as documentation.

56.3 Key Takeaways

- The worker is a **daemon thread** — dies with the main process
- **Immediate save** after each segment enables crash recovery
- **Cancel** exits after the current segment; **Pause** waits between segments
- Audio goes through: **synthesize** → **pitch/volume** → **save WAV** → **convert (optional)**

- The resume pattern: **first_pending_index()** → **skip done segments** → **continue**

[← Ch 55](#) | [Ch 57 — Voice Cloning](#) [→](#)

Voice Cloning: from XTTS v2 to the Voice Lab

⚠ **Warning: Superseded (2026.3.1).** XTTS v2 is no longer part of the application. Voice cloning is now provided by the **Voice Lab**: three pip-installed engines — **Pocket TTS** (Kyutai), **Bark** (Suno) and **F5-TTS** — each in its own Python environment, driven by a subprocess worker. This chapter still teaches the *ideas* that survived the change (a dedicated client, a JSON-lines protocol, one loaded model reused across segments), so read it for the architecture and then continue with:

- [Chapter 76 — the Voice Lab engine subsystem](#)
- [Chapter 77 — one environment per engine](#)
- [Chapter 78 — CPU and GPU compute](#)

A fourth engine (NeuTTS) was removed because it required a Hugging Face login; no part of this application depends on an account.

Learning Objectives

- Understand the voice cloning pipeline
- See the CloneClient subprocess architecture
- Understand the JSON-lines IPC protocol

57.1 Cloning Flow

```
# From clone/__init__.py
# 1. Download cloning engine (coqui-tts + PyTorch, ~2.5 GB)
# 2. User provides: name, language, 4-5 second WAV sample
# 3. Sample is copied into models directory
# 4. Voice appears in Available TTS and Recording windows
# 5. Synthesis uses XtTsCloneEngine → CloneClient → subprocess
```

Explain Line by Line Code Description (Block 1)

- Line 1:** `# From clone/__init__.py`
A comment indicating the source file: *From clone/__init__.py*. This helps the reader locate the code in the AIVoiceStudio codebase.
- Line 2:** `# 1. Download cloning engine (coqui-tts + PyTorch, ~2.5 GB)`
A comment: *1. Download cloning engine (coqui-tts + PyTorch, ~2.5 GB)*. Comments are ignored by the Python interpreter and serve as documentation.
- Line 3:** `# 2. User provides: name, language, 4-5 second WAV sample`
A comment: *2. User provides: name, language, 4-5 second WAV sample*. Comments are ignored by the Python interpreter and serve as documentation.
- Line 4:** `# 3. Sample is copied into models directory`
A comment: *3. Sample is copied into models directory*. Comments are ignored by the Python interpreter and serve as documentation.
- Line 5:** `# 4. Voice appears in Available TTS and Recording windows`
A comment: *4. Voice appears in Available TTS and Recording windows*. Comments are ignored by the Python interpreter and serve as documentation.
- Line 6:** `# 5. Synthesis uses XtTsCloneEngine → CloneClient → subprocess`
A comment: *5. Synthesis uses XtTsCloneEngine → CloneClient → subprocess*. Comments are ignored by the Python interpreter and serve as documentation.

57.2 CloneClient IPC Protocol

```
# JSON Lines protocol over stdin/stdout
# App → Worker:
{"cmd": "ping"}                                # Check engine availability
{"cmd": "synthesize", "text": "Hello",
 "sample_wav": "/path/to/sample.wav",
 "language": "en", "speed": 1.0}               # Synthesize with cloned voice
{"cmd": "quit"}                                # Shutdown

# Worker → App:
{"ok": true, "engine": true}                   # Ping response
{"ok": true, "wav": "<base64>"}                 # Audio response
{"ok": false, "error": "..."}                 # Error response
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# JSON lines protocol over stdin/stdout`
A comment: *JSON lines protocol over stdin/stdout*. Comments are ignored by the Python interpreter and serve as documentation.
2. **Line 2:** `# App → Worker:`
A comment: *App → Worker:*. Comments are ignored by the Python interpreter and serve as documentation.
3. **Line 3:** `{"cmd": "ping"} # Check engine availability`
`{"cmd": "ping"} # Check engine availability`
4. **Line 4:** `{"cmd": "synthesize", "text": "Hello",`
`{"cmd": "synthesize", "text": "Hello",`
5. **Line 5:** `"sample_wav": "/path/to/sample.wav",`
`"sample_wav": "/path/to/sample.wav",`
6. **Line 6:** `"language": "en", "speed": 1.0} # Synthesize with cloned voice`
`"language": "en", "speed": 1.0} # Synthesize with cloned voice`
7. **Line 7:** `{"cmd": "quit"} # Shutdown`
`{"cmd": "quit"} # Shutdown`
8. **Line 8:** `# Worker → App:`
A comment: *Worker → App:*. Comments are ignored by the Python interpreter and serve as documentation.
9. **Line 9:** `{"ok": true, "engine": true} # Ping response`
`{"ok": true, "engine": true} # Ping response`
10. **Line 10:** `{"ok": true, "wav": "<base64>"} # Audio response`
`{"ok": true, "wav": "<base64>"} # Audio response`
11. **Line 11:** `{"ok": false, "error": "..."} # Error response`
`{"ok": false, "error": "..."} # Error response`

57.3 Supported Languages

```
# XTTS v2 supports 17 Languages:
XTTS_LANGUAGES = [
    ("en", "English"), ("es", "Spanish"), ("fr", "French"),
    ("de", "German"), ("it", "Italian"), ("pt", "Portuguese"),
    ("pl", "Polish"), ("tr", "Turkish"), ("ru", "Russian"),
    ("nl", "Dutch"), ("cs", "Czech"), ("ar", "Arabic"),
    ("zh-cn", "Chinese"), ("ja", "Japanese"),
    ("hu", "Hungarian"), ("ko", "Korean"), ("hi", "Hindi"),
]
```

Explain Line by Line Code Description (Block 3)

- Line 1:** `# XTTS v2 supports 17 languages:`
A comment: *XTTS v2 supports 17 languages:*. Comments are ignored by the Python interpreter and serve as documentation.
- Line 2:** `XTTS_LANGUAGES = [`
Assigns `[` to `XTTS_LANGUAGES`.
- Line 3:** `("en", "English"), ("es", "Spanish"), ("fr", "French"),`
`("en", "English"), ("es", "Spanish"), ("fr", "French"),`
- Line 4:** `("de", "German"), ("it", "Italian"), ("pt", "Portuguese"),`
`("de", "German"), ("it", "Italian"), ("pt", "Portuguese"),`
- Line 5:** `("pl", "Polish"), ("tr", "Turkish"), ("ru", "Russian"),`
`("pl", "Polish"), ("tr", "Turkish"), ("ru", "Russian"),`
- Line 6:** `("nl", "Dutch"), ("cs", "Czech"), ("ar", "Arabic"),`
`("nl", "Dutch"), ("cs", "Czech"), ("ar", "Arabic"),`
- Line 7:** `("zh-cn", "Chinese"), ("ja", "Japanese"),`
`("zh-cn", "Chinese"), ("ja", "Japanese"),`
- Line 8:** `("hu", "Hungarian"), ("ko", "Korean"), ("hi", "Hindi"),`
`("hu", "Hungarian"), ("ko", "Korean"), ("hi", "Hindi"),`
- Line 9:** `]`
`]`

57.4 Key Takeaways

- **XTTS v2** is a zero-shot voice cloning model (4-5 second sample)
 - Runs in a **separate subprocess** to isolate PyTorch from sherpa-onnx
 - **JSON lines protocol** over stdin/stdout for IPC
 - Supports **17 languages** with multi-lingual cloning
 - The cloning engine is **optional** — downloaded on-demand from Settings
-

OmniVoice TTS Engine

i Info: Current implementation (2026.3.1). OmniVoice and OmniVoice Server both live in a *single shared* Python environment (`tts_envs/omnivoice`) — the one documented exception to “one environment per engine”, because both need the same base package and the same CUDA PyTorch (see [Chapter 77](#)). Install-time pins are applied as an install step so that `pip uninstall` keeps working, and the CUDA wheels come from the single index constant `PYTORCH_CUDA_INDEX` . An engine that was installed before per-engine environments existed is still honoured from the shared addon environment until it is reinstalled.

Learning Objectives

- Understand the OmniVoice TTS engine (600+ languages)
 - See the two inference modes (triton vs hybrid)
 - Understand the HTTP server variant
-

58.1 OmniVoice Features

- **600+ languages** — massively multilingual
 - **Voice cloning** — zero-shot from 3-15 second sample
 - **Voice design** — create voices from text descriptions
 - **Triton mode** — stable, ~1.5x faster than stock
 - **Hybrid mode** — ~3.4x faster (may leak VRAM)
-

58.2 OmniVoiceWorker (Subprocess)

```
class OmniVoiceWorker:
    """Manages the OmniVoice worker subprocess."""

    def _ensure_proc(self):
        if self._proc and self._proc.poll() is None:
            return
        # In frozen app, use managed venv Python
        if getattr(sys, "frozen", False):
            from ..python_runtime import get_runtime
            rt = get_runtime()
            worker_python = rt.python_exe
            # Write standalone worker script
            worker_script = os.path.join(rt.env_dir, "_omnivoice_worker.py")
            if not os.path.isfile(worker_script):
                with open(worker_script, "w") as f:
                    f.write(_read_worker_source())
            cmd = [worker_python, worker_script]
        else:
            cmd = [sys.executable, "worker.py"]

        self._proc = subprocess.Popen(
            cmd, stdin=subprocess.PIPE, stdout=subprocess.PIPE,
            stderr=subprocess.DEVNULL, text=True,
        )
        self._request({"cmd": "ping"})

    def synthesize(self, text, speed=1.0, ref_audio="",
                   ref_text="", instruct="", language="auto"):
        req = {"cmd": "synthesize", "text": text, "speed": speed}
        if ref_audio: req["ref_audio"] = ref_audio
        if instruct: req["instruct"] = instruct
        resp = self._request(req)
        raw = base64.b64decode(resp["wav"])
        return np.frombuffer(raw, dtype=np.int16).copy()
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `class OmniVoiceWorker:`
Background worker that runs the OmniVoice-Triton inference server. Manages GPU memory and request queue.
2. **Line 2:** `"""Manages the OmniVoice worker subprocess."""`
Docstring: *Manages the OmniVoice worker subprocess.*. Documents the purpose of the following code block.
3. **Line 3:** `def _ensure_proc(self):`
Defines function `_ensure_proc`: Checks if the clone subprocess is running. If not, starts it. If it crashed, restarts it automatically.
4. **Line 4:** `if self._proc and self._proc.poll() is None:`
Checks condition: `self._proc and self._proc.poll() is None`. The indented block below runs only when this is `True`.
5. **Line 5:** `return`
Returns `None` implicitly (exits the function).
6. **Line 6:** `# In frozen app, use managed venv Python`
A comment: *In frozen app, use managed venv Python*. Comments are ignored by the Python interpreter and serve as documentation.
7. **Line 7:** `if getattr(sys, "frozen", False):`
Checks condition: `getattr(sys, "frozen", False)`. The indented block below runs only when this is `True`.
8. **Line 8:** `from ..python_runtime import get_runtime`
Imports `get_runtime` from the `..python_runtime` module.
9. **Line 9:** `rt = get_runtime()`
Assigns `get_runtime()` to `rt`.
10. **Line 10:** `worker_python = rt.python_exe`
Assigns `rt.python_exe` to `worker_python`.
11. **Line 11:** `# Write standalone worker script`
A comment: *Write standalone worker script*. Comments are ignored by the Python interpreter and serve as documentation.
12. **Line 12:** `worker_script = os.path.join(rt.env_dir, "_omnivoice_worker.py")`
Assigns `os.path.join(rt.env_dir, "_omnivoice_worker.py")` to `worker_script`.

13. **Line 13:** `if not os.path.isfile(worker_script):`
Checks condition: `not os.path.isfile(worker_script)`. The indented block below runs only when this is `True`.
14. **Line 14:** `with open(worker_script, "w") as f:`
Context manager: `open(worker_script, "w") as f`. Automatically manages resource cleanup.
15. **Line 15:** `f.write(_read_worker_source())`
`f.write(_read_worker_source())`
16. **Line 16:** `cmd = [worker_python, worker_script]`
Assigns `[worker_python, worker_script]` to `cmd`.
17. **Line 17:** `else:`
Fallback: executes when none of the above conditions matched.
18. **Line 18:** `cmd = [sys.executable, "worker.py"]`
Assigns `[sys.executable, "worker.py"]` to `cmd`.
19. **Line 19:** `self._proc = subprocess.Popen(`
Starts a new process. Used by CloneClient and OmniVoiceWorker to run TTS engines in isolated processes.
20. **Line 20:** `cmd, stdin=subprocess.PIPE, stdout=subprocess.PIPE,`
`cmd, stdin=subprocess.PIPE, stdout=subprocess.PIPE,`
21. **Line 21:** `stderr=subprocess.DEVNULL, text=True,`
Assigns `subprocess.DEVNULL, text=True,` to `stderr`.
22. **Line 22:** `)`
`)`
23. **Line 23:** `self._request({"cmd": "ping"})`
Calls `self._request()` to perform an operation.
24. **Line 24:** `def synthesize(self, text, speed=1.0, ref_audio="",`
`def synthesize(self, text, speed=1.0, ref_audio="",`
25. **Line 25:** `ref_text="", instruct="", language="auto"):`
Assigns `"", instruct="", language="auto"):` to `ref_text`.
26. **Line 26:** `req = {"cmd": "synthesize", "text": text, "speed": speed}`

Assigns `{"cmd": "synthesize", "text": text, "speed": speed}` to `req`.

27. **Line 27:** `if ref_audio: req["ref_audio"] = ref_audio`

Checks condition: `ref_audio`. The indented block below runs only when this is `True`.

28. **Line 28:** `if instruct: req["instruct"] = instruct`

Checks condition: `instruct`. The indented block below runs only when this is `True`.

29. **Line 29:** `resp = self._request(req)`

Assigns `self._request(req)` to `resp`.

30. **Line 30:** `raw = base64.b64decode(resp["wav"])`

Assigns `base64.b64decode(resp["wav"])` to `raw`.

31. **Line 31:** `return np.frombuffer(raw, dtype=np.int16).copy()`

Returns `np.frombuffer(raw, dtype=np.int16).copy()` to the caller.

58.3 OmniVoice HTTP Server

```
class OmniVoiceServerManager:
    """Manages the omnivoice-server HTTP subprocess."""

    def start(self, timeout=120.0):
        # Start server subprocess
        cmd = [rt.python_exe, "-m", "omnivoice_server",
              "--host", self._host, "--port", str(self._port)]
        self._proc = subprocess.Popen(cmd, ...)
        self._wait_ready(timeout) # Poll /health endpoint

    def synthesize(self, text, voice="alloy", instructions="", speed=1.0):
        # POST to /v1/audio/speech
        url = f"{self.base_url}/v1/audio/speech"
        payload = {"model": "omnivoice", "input": text, ...}
        resp = urllib.request.urlopen(req, timeout=120)
        return self._wav_bytes_to_samples(resp.read())
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `class OmniVoiceServerManager:`
HTTP API server that exposes voice cloning over REST endpoints. Allows multiple clients to clone voices simultaneously.
2. **Line 2:** `"""Manages the omnivoice-server HTTP subprocess."""`
Docstring: *Manages the omnivoice-server HTTP subprocess.*. Documents the purpose of the following code block.
3. **Line 3:** `def start(self, timeout=120.0):`
Defines function `start` accepting parameters (self, timeout).
4. **Line 4:** `# Start server subprocess`
A comment: *Start server subprocess.* Comments are ignored by the Python interpreter and serve as documentation.
5. **Line 5:** `cmd = [rt.python_exe, "-m", "omnivoice_server",`
Assigns `[rt.python_exe, "-m", "omnivoice_server",` to `cmd`.
6. **Line 6:** `--host", self._host, "--port", str(self._port)]`
`--host", self._host, "--port", str(self._port)]`
7. **Line 7:** `self._proc = subprocess.Popen(cmd, ...)`
Starts a new process. Used by CloneClient and OmniVoiceWorker to run TTS engines in isolated processes.
8. **Line 8:** `self._wait_ready(timeout) # Poll /health endpoint`
Calls `self._wait_ready()` to perform an operation.
9. **Line 9:** `def synthesize(self, text, voice="alloy", instructions="", speed=1.0):`
Defines function `synthesize`: Core synthesis method. Takes text string, voice ID (sid), speed/pitch/volume floats. Returns a NumPy array of int16 audio samples at the engine's sample_rate.
10. **Line 10:** `# POST to /v1/audio/speech`
A comment: *POST to /v1/audio/speech.* Comments are ignored by the Python interpreter and serve as documentation.
11. **Line 11:** `url = f"{self.base_url}/v1/audio/speech"`
Assigns `f"{self.base_url}/v1/audio/speech"` to `url`.
12. **Line 12:** `payload = {"model": "omnivoice", "input": text, ...}`
Assigns `{"model": "omnivoice", "input": text, ...}` to `payload`.
13. **Line 13:** `resp = urllib.request.urlopen(req, timeout=120)`

Assigns `urllib.request.urlopen(req, timeout=120)` to `resp`.

14. **Line 14:** `return self._wav_bytes_to_samples(resp.read())`

Returns `self._wav_bytes_to_samples(resp.read())` to the caller.

58.4 Key Takeaways

- **OmniVoice** is the most powerful TTS engine — 600+ languages, cloning, design
- Two modes: **triton** (stable) and **hybrid** (fast, may leak VRAM)
- The **HTTP server variant** allows network access and OpenAI-compatible API
- Requires **CUDA GPU** — detected via `nvidia-smi`
- Like XTTS, runs in a **separate subprocess** for runtime isolation

DAISY Book Builder

i Info: Current implementation (2026.3.1). The DAISY 2.02 builder described here is joined by `documents/daisy3_builder.py` (Z39.86-2005 audio + text with images and paragraph-level synchronisation, [Chapter 74](#)) and by `documents/playlist_builder.py` (m3u8/pls/wpl for ordinary recorded projects, [Chapter 73](#)). Book metadata defaults come from the *DAISY settings* category, and the Recording window offers *Export DAISY as ZIP* once a DAISY project is complete.

Learning Objectives

- Understand the DAISY 2.02 standard
- See how AIVS generates NCC, OPF, and SMIL files
- Understand the DAISY ZIP export

59.1 What Is DAISY?

DAISY (Digital Accessible Information System) is a standard for accessible digital books. DAISY 2.02 books include synchronized audio and text, making them usable by blind and visually impaired readers with DAISY-capable players.

DAISY 2.02 File Structure

```
project/
├─ ncc.html      # Navigation Center Combined (master TOC)
├─ package.opf   # OPF content manifest
├─ audio/
│   └─ Chapter_1.wav # Audio files
│   └─ Chapter_2.wav
├─ smil/
│   └─ Chapter_1.smil # Audio-text sync files
│   └─ Chapter_2.smil
└─ Chapter_1.txt  # Text files (for audio+text mode)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `project/`
`project/`
2. **Line 2:** `|─ ncc.html # Navigation Center Combined (master TOC)`
`|─ ncc.html # Navigation Center Combined (master TOC)`
3. **Line 3:** `|─ package.opf # OPF content manifest`
`|─ package.opf # OPF content manifest`
4. **Line 4:** `|─ audio/`
`|─ audio/`
5. **Line 5:** `| |─ Chapter_1.wav # Audio files`
`| |─ Chapter_1.wav # Audio files`
6. **Line 6:** `| |─ Chapter_2.wav`
`| |─ Chapter_2.wav`
7. **Line 7:** `|─ smil/`
`|─ smil/`
8. **Line 8:** `| |─ Chapter_1.smil # Audio-text sync files`
`| |─ Chapter_1.smil # Audio-text sync files`
9. **Line 9:** `| |─ Chapter_2.smil`
`| |─ Chapter_2.smil`
10. **Line 10:** `|─ Chapter_1.txt # Text files (for audio+text mode)`
`|─ Chapter_1.txt # Text files (for audio+text mode)`

59.2 Building the DAISY Book

```
# From documents/daisy_builder.py
def build_daisy_book(output_dir, project_name, segments,
                    audio_format="mp3", include_text=False,
                    language="en", publisher=""):
    """Build the complete DAISY 2.02 structure."""
    ready = [s for s in segments if s.get("status") == "done"]
    book_uid = str(uuid.uuid4())
    now_iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")

    # 1. Generate NCC (Navigation Center Combined)
    _write_ncc(output_dir, project_name, ready, book_uid, now_iso,
              language, publisher, audio_format, include_text)

    # 2. Generate OPF (Open Packaging Format)
    _write_opf(output_dir, project_name, ready, book_uid, now_iso,
              language, publisher, audio_format, include_text)

    # 3. Generate SMIL files for each chapter
    _write_smil_files(output_dir, ready, audio_format, include_text)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# From documents/daisy_builder.py`

A comment indicating the source file: *From documents/daisy_builder.py*. This helps the reader locate the code in the AIVoiceStudio codebase.

2. **Line 2:** `def build_daisy_book(output_dir, project_name, segments,`

`def build_daisy_book(output_dir, project_name, segments,`

3. **Line 3:** `audio_format="mp3", include_text=False,`

Assigns `"mp3", include_text=False,` to `audio_format` .

4. **Line 4:** `language="en", publisher=""):`

Assigns `"en", publisher=""):` to `language` .

5. **Line 5:** `"""Build the complete DAISY 2.02 structure."""`

Docstring: *Build the complete DAISY 2.02 structure.*. Documents the purpose of the following code block.

6. **Line 6:** `ready = [s for s in segments if s.get("status") == "done"]`

Assigns `[s for s in segments if s.get("status") == "done"]` to `ready` .

7. **Line 7:** `book_uid = str(uuid.uuid4())`

Assigns `str(uuid.uuid4())` to `book_uid` .

8. **Line 8:** `now_iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")`

Assigns `datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")` to `now_iso` .

9. **Line 9:** `# 1. Generate NCC (Navigation Center Combined)`

A comment: *1. Generate NCC (Navigation Center Combined)*. Comments are ignored by the Python interpreter and serve as documentation.

10. **Line 10:** `_write_ncc(output_dir, project_name, ready, book_uid, now_iso,`

Calls function `_write_ncc()` .

11. **Line 11:** `language, publisher, audio_format, include_text)`

`language, publisher, audio_format, include_text)`

12. **Line 12:** `# 2. Generate OPF (Open Packaging Format)`

A comment: *2. Generate OPF (Open Packaging Format)*. Comments are ignored by the Python interpreter and serve as documentation.

13. **Line 13:** `_write_opf(output_dir, project_name, ready, book_uid, now_iso,`

Calls function `_write_opf()` .

14. **Line 14:** `language, publisher, audio_format, include_text)`

`language, publisher, audio_format, include_text)`

15. **Line 15:** `# 3. Generate SMIL files for each chapter`

A comment: *3. Generate SMIL files for each chapter*. Comments are ignored by the Python interpreter and serve as documentation.

16. **Line 16:** `_write_smil_files(output_dir, ready, audio_format, include_text)`

Calls function `_write_smil_files()` .

59.3 Export as ZIP

```
def export_daisy_zip(project_dir, zip_path):
    """Package the DAISY book into a distributable ZIP."""
    with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
        # Add NCC and OPF
        zf.write(os.path.join(project_dir, "ncc.html"), "ncc.html")
        zf.write(os.path.join(project_dir, "package.opf"), "package.opf")
        # Add audio and SMIL directories recursively
        for dirname in ["audio", "smil"]:
            dirpath = os.path.join(project_dir, dirname)
            for root, _, files in os.walk(dirpath):
                for file in files:
                    zf.write(os.path.join(root, file),
                            os.path.relpath(os.path.join(root, file), project_dir))
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `def export_daisy_zip(project_dir, zip_path):`
Defines function `export_daisy_zip` accepting parameters (`project_dir`, `zip_path`).
2. **Line 2:** `"""Package the DAISY book into a distributable ZIP."""`
Docstring: *Package the DAISY book into a distributable ZIP.*. Documents the purpose of the following code block.
3. **Line 3:** `with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:`
Context manager: `zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf`. Automatically manages resource cleanup.
4. **Line 4:** `# Add NCC and OPF`
A comment: *Add NCC and OPF*. Comments are ignored by the Python interpreter and serve as documentation.
5. **Line 5:** `zf.write(os.path.join(project_dir, "ncc.html"), "ncc.html")`
`zf.write(os.path.join(project_dir, "ncc.html"), "ncc.html")`
6. **Line 6:** `zf.write(os.path.join(project_dir, "package.opf"), "package.opf")`
`zf.write(os.path.join(project_dir, "package.opf"), "package.opf")`
7. **Line 7:** `# Add audio and SMIL directories recursively`
A comment: *Add audio and SMIL directories recursively*. Comments are ignored by the Python interpreter and serve as documentation.
8. **Line 8:** `for dirname in ["audio", "smil"]:`
Iterates over `["audio", "smil"]`, assigning each element to `dirname`.
9. **Line 9:** `dirpath = os.path.join(project_dir, dirname)`
Assigns `os.path.join(project_dir, dirname)` to `dirpath`.
10. **Line 10:** `for root, _, files in os.walk(dirpath):`
`for root, _, files in os.walk(dirpath):`
11. **Line 11:** `for file in files:`
Iterates over `files`, assigning each element to `file`.
12. **Line 12:** `zf.write(os.path.join(root, file),`
`zf.write(os.path.join(root, file),`
13. **Line 13:** `os.path.relpath(os.path.join(root, file), project_dir))`

```
os.path.relpath(os.path.join(root, file), project_dir))
```

59.4 What Changed Since the First Edition

The DAISY 2.02 builder received three significant upgrades. The most important one fixes the classic "audio cuts out" complaint.

1. MP3 normalization on staging

Two new functions in `audio/ffmpeg.py` give the builder control over problem MP3s. DAISY players reliably decode MPEG-1 Layer III; the TTS engines, however, emit MPEG-2 LSF MP3s at their native sample rate (22–24 kHz), which several players truncate or refuse while VLC plays the same file fine.

```
def needs_mp3_normalization(path: str) -> bool:
    exe = find_ffmpeg()
    if not exe:
        return False
    try:
        result = subprocess.run(
            [exe, "-i", path],
            capture_output=True,
            text=True,
            timeout=30,
            creationflags=getattr(subprocess, "CREATE_NO_WINDOW", 0),
        )
        import re
        stream = re.search(
            r"Stream #\d+:\d+.*?: Audio: mp3.*?, (\d+) Hz", result.stderr or ""
        )
        if not stream:
            return False
        return int(stream.group(1)) < 44100
    except Exception:
        return False
```


Explain Line by Line Code Description (Block 6)

1. **Line 1:** `def needs_mp3_normalization(path):`
Answers one question: is this MP3 file one that DAISY players will mishandle? The probe costs one FFmpeg invocation, so it is cheap enough to run per segment.
2. **Lines 2-3:** `exe = find_ffmpeg()` / `if not exe: return False`
Locates FFmpeg (bundled folder or PATH). When it is not installed the probe returns `False` — meaning "assume the file is fine" — because without FFmpeg there is also no way to re-encode, and skipping is the only graceful choice.
3. **Lines 4-11:** the `subprocess.run` probe
Runs `ffmpeg -i path` *without* an output file, which makes FFmpeg print stream information to `stderr` and exit. `capture_output=True, text=True` collects that text; `timeout=30` bounds a hung probe; `creationflags=CREATE_NO_WINDOW` (guarded with `getattr` so non-Windows platforms stay compatible) stops a console window from flashing on screen.
4. **Lines 12-14:** the regex
Searches FFmpeg's stream report for a line like `Stream #0:0: Audio: mp3, 24000 Hz, ...`. The pattern anchors on `Audio: mp3` and captures the sample-rate digits before `Hz`. When nothing matches (corrupt file, unexpected layout) the probe again answers `False`.
5. **Line 15:** `return int(stream.group(1)) < 44100`
The decision rule: anything below 44.1 kHz is MPEG-2 LSF territory and gets normalized. 44.1 kHz and above pass untouched.
6. **Lines 16-17:** the blanket `except`
Any failure (missing exe mid-run, timeout, decode error) degrades to `False`. A broken probe must never abort a book build — worst case the user gets an unnormalized MP3, which is exactly the pre-fix behaviour.

The companion `normalize_mp3_for_daisy(src, dst)` re-encodes such a file to **MPEG-1 Layer III, 44.1 kHz, mono** via `libmp3lame -q:a 2`, writing to a temp file and `os.replace`-ing it into place so a failed transcode never destroys the original. The DAISY 2.02 and DAISY 3 builders both call this pair while staging recorded audio into the book.

2. Book information from the wizard

The New Project wizard's DAISY page now collects title, creator, narrator, publisher, subject and date. These flow through the recording dialog into `build_daisy_book(..., meta=...)`; empty fields fall back to the global defaults in Settings or to sensible built-ins, and the running time is always measured from the actual audio — never trusted from metadata.

3. Cleaned metadata

Generator strings and build metadata were cleaned up so players show the book's own title rather than tooling artefacts, and the master SMIL/NCC cross-references were made strictly sibling-relative (same portability rule as the playlist builder in Chapter 73).

59.5 Key Takeaways

- **DAISY 2.02** is an accessible book format with audio-text synchronization
 - Three key files: **ncc.html** (TOC), **package.opf** (manifest), **SMIL** (sync)
 - AIVS supports both **audio-only** and **audio+text** DAISY modes (the legacy type stays loadable; new projects pick DAISY 3 for text+images — Chapter 74)
 - The builder is called **automatically after recording completes**
 - **ZIP export** packages everything into a single distributable file
 - Low-rate MP3s are **re-encoded to 44.1 kHz MPEG-1** during staging so every DAISY player decodes the full audio
-

Addon System

i Info: Current implementation (2026.3.1). Addons continue to run in the shared managed environment (`addon_env` , see [Chapter 77](#) for why it is now called *shared*: each pip-installed TTS engine has its own), and the Developer category that manages them is hidden until Developer Mode is on. Two consequences worth knowing when writing an addon: your modules are imported into the application process, so a heavy second runtime (another `onnxruntime` , PyTorch, CUDA) belongs in an engine environment reached through a worker subprocess; and addon-provided voices take part in the per-category **Compute** choice like any other voice ([Chapter 78](#)).

Learning Objectives

- Understand the addon architecture (inspired by NVDA)
- See the manifest.json format
- Understand addon discovery, loading, and registration

60.1 Addon Architecture

```
# Addons live in %APPDATA%/AIVoiceStudio/addons/  
# Each addon has:  
# - manifest.json (required)  
# - __init__.py (entry point, optional)  
# - engine.py (custom TTS engine, optional)  
# - voices/ (bundled voice data, optional)  
  
# manifest.json layout:  
{  
    "name": "My TTS Addon",  
    "version": "1.0.0",  
    "description": "Adds a new TTS engine",  
    "author": "Jane Doe",  
    "min_app_version": "0.1.0",  
    "engine_id": "my_tts",  
    "engine_name": "My TTS Engine",  
    "engine_entry_point": "engine:MyTtsEngine",  
    "dependencies": ["requests>=2.31"],  
    "provides": ["tts_engine", "voice"]  
}
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Addons live in %APPDATA%/AIVoiceStudio/addons/`

A comment: *Addons live in %APPDATA%/AIVoiceStudio/addons/*. Comments are ignored by the Python interpreter and serve as documentation.

2. **Line 2:** `# Each addon has:`

A comment: *Each addon has:*. Comments are ignored by the Python interpreter and serve as documentation.

3. **Line 3:** `# - manifest.json (required)`

A comment: *- manifest.json (required)*. Comments are ignored by the Python interpreter and serve as documentation.

4. **Line 4:** `# - __init__.py (entry point, optional)`

A comment: *- __init__.py (entry point, optional)*. Comments are ignored by the Python interpreter and serve as documentation.

5. **Line 5:** `# - engine.py (custom TTS engine, optional)`

A comment: *- engine.py (custom TTS engine, optional)*. Comments are ignored by the Python interpreter and serve as documentation.

6. **Line 6:** `# - voices/ (bundled voice data, optional)`

A comment: *- voices/ (bundled voice data, optional)*. Comments are ignored by the Python interpreter and serve as documentation.

7. **Line 7:** `# manifest.json layout:`

A comment: *manifest.json layout:*. Comments are ignored by the Python interpreter and serve as documentation.

8. **Line 8:** `{`

`{`

9. **Line 9:** `"name": "My TTS Addon",`

`"name": "My TTS Addon",`

10. **Line 10:** `"version": "1.0.0",`

`"version": "1.0.0",`

11. **Line 11:** `"description": "Adds a new TTS engine",`

`"description": "Adds a new TTS engine",`

12. **Line 12:** `"author": "Jane Doe",`

`"author": "Jane Doe",`

13. **Line 13:** `"min_app_version": "0.1.0",`

```
"min_app_version": "0.1.0",
```

14. **Line 14:** `"engine_id": "my_tts",`

```
"engine_id": "my_tts",
```

15. **Line 15:** `"engine_name": "My TTS Engine",`

```
"engine_name": "My TTS Engine",
```

16. **Line 16:** `"engine_entry_point": "engine:MyTtsEngine",`

Abstract base class that all TTS engines must implement. Defines the `synthesize()` contract: takes text + parameters, returns int16 audio samples.

17. **Line 17:** `"dependencies": ["requests>=2.31"],`

```
"dependencies": ["requests>=2.31"],
```

18. **Line 18:** `"provides": ["tts_engine", "voice"]`

```
"provides": ["tts_engine", "voice"]
```

19. **Line 19:** `}`

```
}
```

60.2 AddonManager

```
class AddonManager:
    """Discovers, loads, and manages addons."""

    def discover(self) -> list[AddonManifest]:
        """Scan addons directory and load manifests."""
        for name in os.listdir(self._dir):
            manifest_path = os.path.join(self._dir, name, "manifest.json")
            if os.path.isfile(manifest_path):
                with open(manifest_path) as f:
                    data = json.load(f)
                    manifest = AddonManifest(manifest_path, data)
                    if manifest.is_compatible:
                        self._addons[name] = manifest

    def load_addon(self, name: str) -> bool:
        """Load addon's Python module and register its engine."""
        manifest = self._addons[name]
        addon_dir = manifest.addon_dir

        # Dynamic import
        spec = importlib.util.spec_from_file_location(
            f"_aivs_addon_{name}",
            os.path.join(addon_dir, "__init__.py"),
        )
        module = importlib.util.module_from_spec(spec)
        sys.modules[f"_aivs_addon_{name}"] = module
        spec.loader.exec_module(module)

        # Register engine
        if manifest.engine_id:
            self._register_engine(manifest, addon_dir)

    def install_addon(self, addon_path: str) -> bool:
        """Install from ZIP or directory."""
        if addon_path.endswith(".zip"):
            # Extract ZIP to addons directory
            ...
        elif os.path.isdir(addon_path):
            # Copy directory
            shutil.copytree(addon_path, dest)
            self.discover()

# Singleton access
def get_addon_manager() -> AddonManager:
    global _default_manager
    if _default_manager is None:
        _default_manager = AddonManager()
    return _default_manager
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `class AddonManager:`
Defines class `AddonManager` .
2. **Line 2:** `"""Discovers, loads, and manages addons."""`
Docstring: *Discovers, loads, and manages addons.*. Documents the purpose of the following code block.
3. **Line 3:** `def discover(self) -> list[AddonManifest]:`
Defines function `discover` accepting parameters (self), returning `list[AddonManifest]` .
4. **Line 4:** `"""Scan addons directory and load manifests."""`
Docstring: *Scan addons directory and load manifests.*. Documents the purpose of the following code block.
5. **Line 5:** `for name in os.listdir(self._dir):`
Iterates over `os.listdir(self._dir)` , assigning each element to `name` .
6. **Line 6:** `manifest_path = os.path.join(self._dir, name, "manifest.json")`
Assigns `os.path.join(self._dir, name, "manifest.json")` to `manifest_path` .
7. **Line 7:** `if os.path.isfile(manifest_path):`
Checks condition: `os.path.isfile(manifest_path)` . The indented block below runs only when this is `True` .
8. **Line 8:** `with open(manifest_path) as f:`
Context manager: `open(manifest_path) as f` . Automatically manages resource cleanup.
9. **Line 9:** `data = json.load(f)`
Assigns `json.load(f)` to `data` .
10. **Line 10:** `manifest = AddonManifest(manifest_path, data)`
Assigns `AddonManifest(manifest_path, data)` to `manifest` .
11. **Line 11:** `if manifest.is_compatible:`
Checks condition: `manifest.is_compatible` . The indented block below runs only when this is `True` .
12. **Line 12:** `self._addons[name] = manifest`
`self._addons[name] = manifest`
13. **Line 13:** `def load_addon(self, name: str) -> bool:`
Defines function `load_addon` accepting parameters (self, name), returning `bool` .

14. **Line 14:** `"""Load addon's Python module and register its engine."""`
Docstring: *Load addon's Python module and register its engine..* Documents the purpose of the following code block.
15. **Line 15:** `manifest = self._addons[name]`
Assigns `self._addons[name]` to `manifest` .
16. **Line 16:** `addon_dir = manifest.addon_dir`
Assigns `manifest.addon_dir` to `addon_dir` .
17. **Line 17:** `# Dynamic import`
A comment: *Dynamic import.* Comments are ignored by the Python interpreter and serve as documentation.
18. **Line 18:** `spec = importlib.util.spec_from_file_location(`
Assigns `importlib.util.spec_from_file_location(` to `spec` .
19. **Line 19:** `f"_aivs_addon_{name}",`
`f"_aivs_addon_{name}",`
20. **Line 20:** `os.path.join(addon_dir, "__init__.py"),`
`os.path.join(addon_dir, "__init__.py"),`
21. **Line 21:** `)`
`)`
22. **Line 22:** `module = importlib.util.module_from_spec(spec)`
Assigns `importlib.util.module_from_spec(spec)` to `module` .
23. **Line 23:** `sys.modules[f"_aivs_addon_{name}"] = module`
`sys.modules[f"_aivs_addon_{name}"] = module`
24. **Line 24:** `spec.loader.exec_module(module)`
`spec.loader.exec_module(module)`
25. **Line 25:** `# Register engine`
A comment: *Register engine.* Comments are ignored by the Python interpreter and serve as documentation.
26. **Line 26:** `if manifest.engine_id:`
Checks condition: `manifest.engine_id` . The indented block below runs only when this is `True` .
27. **Line 27:** `self._register_engine(manifest, addon_dir)`

Calls `self._register_engine()` to perform an operation.

28. **Line 28:** `def install_addon(self, addon_path: str) -> bool:`

Defines function `install_addon` accepting parameters `(self, addon_path)`, returning `bool`.

29. **Line 29:** `"""Install from ZIP or directory."""`

Docstring: *Install from ZIP or directory..* Documents the purpose of the following code block.

30. **Line 30:** `if addon_path.endswith(".zip"):`

Checks condition: `addon_path.endswith(".zip")`. The indented block below runs only when this is `True`.

31. **Line 31:** `# Extract ZIP to addons directory`

A comment: *Extract ZIP to addons directory.* Comments are ignored by the Python interpreter and serve as documentation.

32. **Line 32:** `...`

Ellipsis placeholder: indicates an abstract method that subclasses must implement.

33. **Line 33:** `elif os.path.isdir(addon_path):`

Alternative condition: `os.path.isdir(addon_path)`. Checked only if all previous conditions were `False`.

34. **Line 34:** `# Copy directory`

A comment: *Copy directory.* Comments are ignored by the Python interpreter and serve as documentation.

35. **Line 35:** `shutil.copytree(addon_path, dest)`

`shutil.copytree(addon_path, dest)`

36. **Line 36:** `self.discover()`

Calls `self.discover()` to perform an operation.

37. **Line 37:** `# Singleton access`

A comment: *Singleton access.* Comments are ignored by the Python interpreter and serve as documentation.

38. **Line 38:** `def get_addon_manager() -> AddonManager:`

Defines function `get_addon_manager`, returning `AddonManager`.

39. **Line 39:** `global _default_manager`

`global _default_manager`

40. **Line 40:** `if _default_manager is None:`

Checks condition: `_default_manager is None`. The indented block below runs only when this is `True`.

```
41. Line 41: _default_manager = AddonManager()  
    Assigns AddonManager() to _default_manager .  
  
42. Line 42: return _default_manager  
    Returns _default_manager to the caller.
```

60.3 Key Takeaways

- Addons follow the **NVDA addon architecture** pattern
 - **manifest.json** declares name, version, engine, dependencies
 - **Dynamic imports** via importlib load addon code at runtime
 - Addons can provide **new TTS engines, voices, and punctuation modes**
 - The addon system is **thread-safe** with RLock protection
-

Testing, Packaging & Deployment

Unit Testing Strategies

Learning Objectives

- Understand the AIVS testing philosophy
 - Learn what makes a good test
 - See the complete test suite structure
-

61.1 AIVS Test Suite Overview

Test File	Tests	Strategy
test_catalog.py	14	Validate data structure integrity
test_compute.py	6	Mock GPU detection, verify logic
test_engine.py	8	Mock sherpa-onnx, test post-processing
test_parsers.py	12	Create temp files, parse and verify
test_project.py	10	Temp dirs, save/load cycle
test_splitter.py	15	Create Document objects, verify splits
test_settings.py	8	Temp file, dot-notation access
test_paths.py	7	Verify path resolution
test_gui_smoke.py	6	Create wx objects, verify no crash
test_runtime.py	5	Temp venv, run_in_env
test_clone.py	4	Mock subprocess, test protocol
test_daisy.py	4	Generate DAISY structure, verify files

61.2 Test Categories

Data Validation Tests

```
# Verify catalog structure integrity
def test_every_voice_has_id_and_name():
    for tts in catalog.get_tts_list():
        for lang in tts.get("languages", []):
            for variant in lang.get("variants", []):
                for voice in variant.get("voices", []):
                    assert voice["id"], "Voice must have an id"
                    assert voice.get("name"), "Voice must have a name"
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Verify catalog structure integrity`
A comment: *Verify catalog structure integrity.*
2. **Line 2:** `def test_every_voice_has_id_and_name():`
Test function. pytest discovers functions prefixed with 'test_'.
3. **Line 3:** `for tts in catalog.get_tts_list():`
For loop iteration.
4. **Line 4:** `for lang in tts.get("languages", []):`
For loop iteration.
5. **Line 5:** `for variant in lang.get("variants", []):`
For loop iteration.
6. **Line 6:** `for voice in variant.get("voices", []):`
For loop iteration.
7. **Line 7:** `assert voice["id"], "Voice must have an id"`
Assertion that verifies a condition is True.
8. **Line 8:** `assert voice.get("name"), "Voice must have a name"`
Assertion that verifies a condition is True.

Unit Tests with Mocking

```
# Test compute detection without real GPU
def test_cuda_offered_with_runtime_and_gpu():
    self._patch(runtime_installed=True, nvidia=True)
    det = compute.detect(force=True)
    assert det["cuda"] is True
    assert "cuda" in compute.available_compute()
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Test compute detection without real GPU`
A comment: *Test compute detection without real GPU.*
2. **Line 2:** `def test_cuda_offered_with_runtime_and_gpu():`
Test function. pytest discovers functions prefixed with 'test_'.
3. **Line 3:** `self._patch(runtime_installed=True, nvidia=True)`
Context manager that replaces a name with a Mock. Use 'as' to get the mock reference.
4. **Line 4:** `det = compute.detect(force=True)`
Assigns `compute.detect(force=True)` to `det`.
5. **Line 5:** `assert det["cuda"] is True`
Assertion that verifies a condition is True.
6. **Line 6:** `assert "cuda" in compute.available_compute()`
Assertion that verifies a condition is True.

File System Tests

```
# Use tempfile for isolated file system tests
class ResolveVoiceFilesTest(unittest.TestCase):
    def setUp(self):
        self.tmp = tempfile.TemporaryDirectory()
        # Mock paths to use temp directory
        self.patcher = mock.patch(
            "ai_voice_studio.paths.user_data_dir",
            return_value=self.tmp.name,
        )
        self.patcher.start()

    def test_matcha_finds_model(self):
        vdir = self._voice_dir("matcha_en/default")
        open(os.path.join(vdir, "model-steps-3.onnx"), "w").close()
        files = resolve_voice_files({"dir": vdir, "engine": "matcha"})
        assert files["model"].endswith("model-steps-3.onnx")
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Use tempfile for isolated file system tests`
A comment: *Use tempfile for isolated file system tests.*
2. **Line 2:** `class ResolveVoiceFilesTest(unittest.TestCase):`
Defines a new class.
3. **Line 3:** `def setUp(self):`
Runs before each test method. Used to create test fixtures.
4. **Line 4:** `self.tmp = tempfile.TemporaryDirectory()`
Returns the transposed array (swaps rows and columns for 2D arrays).
5. **Line 5:** `# Mock paths to use temp directory`
A comment: *Mock paths to use temp directory.*
6. **Line 6:** `self.patcher = mock.patch(`
Decorator/context manager that replaces a function/attribute with a Mock during testing.
7. **Line 7:** `"ai_voice_studio.paths.user_data_dir",`
`"ai_voice_studio.paths.user_data_dir",`
8. **Line 8:** `return_value=self.tmp.name,`
Assigns `self.tmp.name,` to `return_value`.
9. **Line 9:** `)`
`)`
10. **Line 10:** `self.patcher.start()`
`self.patcher.start()`
11. **Line 11:** `def test_matcha_finds_model(self):`
Test function. pytest discovers functions prefixed with 'test_'.
12. **Line 12:** `vdir = self._voice_dir("matcha_en/default")`
Assigns `self._voice_dir("matcha_en/default")` to `vdir`.
13. **Line 13:** `open(os.path.join(vdir, "model-steps-3.onnx"), "w").close()`
Opens a file and returns a file object.
14. **Line 14:** `files = resolve_voice_files({"dir": vdir, "engine": "matcha"})`

Assigns `resolve_voice_files({"dir": vdir, "engine": "matcha"})` to `files`.

15. **Line 15:** `assert files["model"].endswith("model-steps-3.onnx")`

Assertion that verifies a condition is True.

61.3 Running Tests

```
# Run all tests
python -m pytest tests/ -v

# Run with coverage
pip install pytest-cov
python -m pytest tests/ --cov=ai_voice_studio --cov-report=term-missing

# Run specific test class
python -m pytest tests/test_catalog.py::CatalogStructureTest -v

# Run specific test
python -m pytest tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique -v

# Run tests matching pattern
python -m pytest tests/ -k "piper" -v
```


Explain Line by Line Code Description (Block 4)

1. **Line 1:** `# Run all tests`
A comment: *Run all tests.*
2. **Line 2:** `python -m pytest tests/ -v`
The testing framework used by AIVS. Discovers test files named test_*.py.
3. **Line 3:** `# Run with coverage`
A comment: *Run with coverage.*
4. **Line 4:** `pip install pytest-cov`
The testing framework used by AIVS. Discovers test files named test_*.py.
5. **Line 5:** `python -m pytest tests/ --cov=ai_voice_studio --cov-report=term-missing`
The testing framework used by AIVS. Discovers test files named test_*.py.
6. **Line 6:** `# Run specific test class`
A comment: *Run specific test class.*
7. **Line 7:** `python -m pytest tests/test_catalog.py::CatalogStructureTest -v`
The testing framework used by AIVS. Discovers test files named test_*.py.
8. **Line 8:** `# Run specific test`
A comment: *Run specific test.*
9. **Line 9:** `python -m pytest tests/test_catalog.py::CatalogStructureTest::test_tts_ids_unique -v`
The testing framework used by AIVS. Discovers test files named test_*.py.
10. **Line 10:** `# Run tests matching pattern`
A comment: *Run tests matching pattern.*
11. **Line 11:** `python -m pytest tests/ -k "piper" -v`
The testing framework used by AIVS. Discovers test files named test_*.py.

61.4 Key Takeaways

- AIVS has **99+ tests** across 12 test files
- **Data validation tests** ensure catalog integrity

- **Mocking** isolates tests from external dependencies (GPU, network)
- **tempfile** provides isolated file system for tests
- The GUI smoke tests verify **wx objects can be created** without crashes

Mocking & Test Doubles

Learning Objectives

- Use unittest.mock for test isolation
 - Mock subprocess calls, file system, and network
 - See mocking patterns in the AIVS test suite
-

62.1 Mock Basics

```
from unittest import mock

# Mock a function return value
with mock.patch("ai_voice_studio.compute._has_nvidia_gpu", return_value=True):
    result = compute._has_nvidia_gpu()
    assert result is True

# Mock with side effects
mock_subprocess = mock.Mock()
mock_subprocess.run.return_value.returncode = 0
mock_subprocess.run.return_value.stdout = "NVIDIA RTX 3060\n"

# Patch multiple targets
with mock.patch("ai_voice_studio.compute.runtime.is_installed", return_value=True), \
    mock.patch.object(compute.subprocess, "run", return_value=FakeResult(0, "NVIDIA GPU\n")):
    det = compute.detect(force=True)
    assert det["cuda"] is True
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `from unittest import mock`
Imports a module.
2. **Line 2:** `# Mock a function return value`
A comment: *Mock a function return value.*
3. **Line 3:** `with mock.patch("ai_voice_studio.compute._has_nvidia_gpu", return_value=True):`
Decorator/context manager that replaces a function/attribute with a Mock during testing.
4. **Line 4:** `result = compute._has_nvidia_gpu()`
Assigns `compute._has_nvidia_gpu()` to `result`.
5. **Line 5:** `assert result is True`
Assertion that verifies a condition is True.
6. **Line 6:** `# Mock with side effects`
A comment: *Mock with side effects.*
7. **Line 7:** `mock_subprocess = mock.Mock()`
Creates a mock object. Any method call or attribute access returns a Mock.
8. **Line 8:** `mock_subprocess.run.return_value.returncode = 0`
Assigns `0` to `mock_subprocess.run.return_value.returncode`.
9. **Line 9:** `mock_subprocess.run.return_value.stdout = "NVIDIA RTX 3060\n"`
Assigns `"NVIDIA RTX 3060\n"` to `mock_subprocess.run.return_value.stdout`.
10. **Line 10:** `# Patch multiple targets`
A comment: *Patch multiple targets.*
11. **Line 11:** `with mock.patch("ai_voice_studio.compute.runtime.is_installed", return_value=True), \`
Decorator/context manager that replaces a function/attribute with a Mock during testing.
12. **Line 12:** `mock.patch.object(compute.subprocess, "run", return_value=FakeResult(0, "NVIDIA GPU\n")):`
`mock.patch.object(compute.subprocess, "run", return_value=FakeResult(0, "NVIDIA GPU\n")):`
13. **Line 13:** `det = compute.detect(force=True)`
Assigns `compute.detect(force=True)` to `det`.
14. **Line 14:** `assert det["cuda"] is True`

Assertion that verifies a condition is True.

62.2 AIVS Mocking Patterns

```
# Pattern 1: Mock paths to use temp directory
patcher = mock.patch(
    "ai_voice_studio.paths.user_data_dir",
    return_value=temp_dir,
)
patcher.start()
self.addCleanup(patcher.stop) # Auto-cleanup

# Pattern 2: Fake subprocess result
class _FakeResult:
    def __init__(self, returncode=0, stdout=""):
        self.returncode = returncode
        self.stdout = stdout

# Pattern 3: Mock GPU detection
def _patch(self, runtime_installed, nvidia):
    patches = [
        mock.patch("ai_voice_studio.compute.runtime.is_installed",
                    return_value=runtime_installed),
        mock.patch.object(compute.subprocess, "run",
                           return_value=_FakeResult(0, "NVIDIA GPU\n") if nvidia
                           else _FakeResult(1, "")),
    ]
    for p in patches:
        p.start()
        self.addCleanup(p.stop)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Pattern 1: Mock paths to use temp directory`
A comment: *Pattern 1: Mock paths to use temp directory.*
2. **Line 2:** `patcher = mock.patch(`
Decorator/context manager that replaces a function/attribute with a Mock during testing.
3. **Line 3:** `"ai_voice_studio.paths.user_data_dir",`
`"ai_voice_studio.paths.user_data_dir",`
4. **Line 4:** `return_value=temp_dir,`
Assigns `temp_dir` to `return_value` .
5. **Line 5:** `)`
`)`
6. **Line 6:** `patcher.start()`
`patcher.start()`
7. **Line 7:** `self.addCleanup(patcher.stop) # Auto-cleanup`
Calls `self.addCleanup()` .
8. **Line 8:** `# Pattern 2: Fake subprocess result`
A comment: *Pattern 2: Fake subprocess result.*
9. **Line 9:** `class _FakeResult:`
Defines a new class.
10. **Line 10:** `def __init__(self, returncode=0, stdout=""):`
Constructor method. Called when creating a new instance of a class.
11. **Line 11:** `self.returncode = returncode`
Stores `returncode` in instance attribute `returncode` .
12. **Line 12:** `self.stdout = stdout`
Stores `stdout` in instance attribute `stdout` .
13. **Line 13:** `# Pattern 3: Mock GPU detection`
A comment: *Pattern 3: Mock GPU detection.*

14. **Line 14:** `def _patch(self, runtime_installed, nvidia):`
Context manager that replaces a name with a Mock. Use 'as' to get the mock reference.
15. **Line 15:** `patches = [`
Assigns `[` to `patches` .
16. **Line 16:** `mock.patch("ai_voice_studio.compute.runtime.is_installed",`
Decorator/context manager that replaces a function/attribute with a Mock during testing.
17. **Line 17:** `return_value=runtime_installed),`
Assigns `runtime_installed),` to `return_value` .
18. **Line 18:** `mock.patch.object(compute.subprocess, "run",`
`mock.patch.object(compute.subprocess, "run",`
19. **Line 19:** `return_value=_FakeResult(0, "NVIDIA GPU\n") if nvidia`
Conditional check.
20. **Line 20:** `else _FakeResult(1, ""),`
`else _FakeResult(1, ""),`
21. **Line 21:** `]`
`]`
22. **Line 22:** `for p in patches:`
For loop iteration.
23. **Line 23:** `p.start()`
`p.start()`
24. **Line 24:** `self.addCleanup(p.stop)`
Calls `self.addCleanup()` .

62.3 Key Takeaways

- **mock.patch()** replaces functions/classes during tests
- Always use **addCleanup()** or **with** to restore mocks
- AIVS primarily mocks: **paths**, **subprocess.run**, **nvidia-smi**

- **tempfile.TemporaryDirectory** + path mocking isolates file system tests

[← Ch 61](#) | [Ch 63 — Integration](#) →

Integration Testing

Learning Objectives

- Understand integration vs. unit testing
 - See how AIVS tests end-to-end flows
 - Understand the GUI smoke test pattern
-

63.1 GUI Smoke Tests

```
# From test_gui_smoke.py – verify wx objects can be created
class GuiSmokeTest(unittest.TestCase):
    def test_main_frame_creation(self):
        """Verify MainFrame can be created without errors."""
        app = wx.App(False)
        settings = Settings()
        store = ModelStore()
        frame = MainFrame(None, settings, store)
        frame.Show()
        frame.Close()
        app.Destroy()

    def test_settings_dialog_creation(self):
        """Verify SettingsDialog can be created."""
        app = wx.App(False)
        dlg = SettingsDialog(None, Settings(), ModelStore())
        dlg.Destroy()
        app.Destroy()
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From test_gui_smoke.py – verify wx objects can be created`
A comment: *From test_gui_smoke.py – verify wx objects can be created.*
2. **Line 2:** `class GuiSmokeTest(unittest.TestCase):`
Defines a new class.
3. **Line 3:** `def test_main_frame_creation(self):`
Test function. pytest discovers functions prefixed with 'test_'.
4. **Line 4:** `"""Verify MainFrame can be created without errors."""`
Docstring: *Verify MainFrame can be created without errors..* Documents the following code.
5. **Line 5:** `app = wx.App(False)`
Assigns `wx.App(False)` to `app`.
6. **Line 6:** `settings = Settings()`
Assigns `Settings()` to `settings`.
7. **Line 7:** `store = ModelStore()`
Assigns `ModelStore()` to `store`.
8. **Line 8:** `frame = MainFrame(None, settings, store)`
Assigns `MainFrame(None, settings, store)` to `frame`.
9. **Line 9:** `frame.Show()`
`frame.Show()`
10. **Line 10:** `frame.Close()`
`frame.Close()`
11. **Line 11:** `app.Destroy()`
`app.Destroy()`
12. **Line 12:** `def test_settings_dialog_creation(self):`
Test function. pytest discovers functions prefixed with 'test_'.
13. **Line 13:** `"""Verify SettingsDialog can be created."""`
Docstring: *Verify SettingsDialog can be created..* Documents the following code.

14. **Line 14:** `app = wx.App(False)`
Assigns `wx.App(False)` to `app` .
15. **Line 15:** `dlg = SettingsDialog(None, Settings(), ModelStore())`
Assigns `SettingsDialog(None, Settings(), ModelStore())` to `dlg` .
16. **Line 16:** `dlg.Destroy()`
`dlg.Destroy()`
17. **Line 17:** `app.Destroy()`
`app.Destroy()`

63.2 End-to-End Test Patterns

```
# Test: parse document → split → create project → save/load
def test_full_project_lifecycle(self):
    # 1. Create a temp text file
    txt = os.path.join(self.tmp.name, "test.txt")
    with open(txt, "w") as f:
        f.write("Chapter 1\nHello world.\nChapter 2\nGoodbye.")

    # 2. Parse
    doc = parse_document(txt)
    assert len(doc.blocks) >= 2

    # 3. Split
    segments = split_document(doc, MODE_PAGE_WITH_H1)
    assert len(segments) >= 2

    # 4. Create project
    pdir = os.path.join(self.tmp.name, "project")
    project.create_project(pdir, "Test", txt, "page_with_h1",
        [{"index": s.index, "title": s.title, "text": s.text}
         for s in segments])

    # 5. Load and verify
    data = project.load_project(pdir)
    assert data["name"] == "Test"
    assert len(data["segments"]) >= 2

    # 6. Mark segment done
    project.mark_segment_done(pdir, 1, "test.wav")
    data = project.load_project(pdir)
    assert data["segments"][0]["status"] == "done"
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Test: parse document → split → create project → save/load`

A comment: *Test: parse document → split → create project → save/load.*

2. **Line 2:** `def test_full_project_lifecycle(self):`

Test function. pytest discovers functions prefixed with 'test_'.

3. **Line 3:** `# 1. Create a temp text file`

A comment: *1. Create a temp text file.*

4. **Line 4:** `txt = os.path.join(self.tmp.name, "test.txt")`

Assigns `os.path.join(self.tmp.name, "test.txt")` to `txt`.

5. **Line 5:** `with open(txt, "w") as f:`

Opens a file using a context manager. Auto-closes when the block exits.

6. **Line 6:** `f.write("Chapter 1\n\nHello world.\n\nChapter 2\n\nGoodbye.")`

`f.write("Chapter 1\n\nHello world.\n\nChapter 2\n\nGoodbye.")`

7. **Line 7:** `# 2. Parse`

A comment: *2. Parse.*

8. **Line 8:** `doc = parse_document(txt)`

Assigns `parse_document(txt)` to `doc`.

9. **Line 9:** `assert len(doc.blocks) >= 2`

Assertion that verifies a condition is True.

10. **Line 10:** `# 3. Split`

A comment: *3. Split.*

11. **Line 11:** `segments = split_document(doc, MODE_PAGE_WITH_H1)`

Assigns `split_document(doc, MODE_PAGE_WITH_H1)` to `segments`.

12. **Line 12:** `assert len(segments) >= 2`

Assertion that verifies a condition is True.

13. **Line 13:** `# 4. Create project`

A comment: *4. Create project.*

14. **Line 14:** `pdir = os.path.join(self.tmp.name, "project")`

- Assigns `os.path.join(self.tmp.name, "project")` to `pdir` .
15. **Line 15:** `project.create_project(pdir, "Test", txt, "page_with_h1",`
`project.create_project(pdir, "Test", txt, "page_with_h1",`
16. **Line 16:** `[{"index": s.index, "title": s.title, "text": s.text}`
The text content of a paragraph or page.
17. **Line 17:** `for s in segments])`
For loop iteration.
18. **Line 18:** `# 5. Load and verify`
A comment: *5. Load and verify.*
19. **Line 19:** `data = project.load_project(pdir)`
Assigns `project.load_project(pdir)` to `data` .
20. **Line 20:** `assert data["name"] == "Test"`
Assertion that verifies a condition is True.
21. **Line 21:** `assert len(data["segments"]) >= 2`
Assertion that verifies a condition is True.
22. **Line 22:** `# 6. Mark segment done`
A comment: *6. Mark segment done.*
23. **Line 23:** `project.mark_segment_done(pdir, 1, "test.wav")`
`project.mark_segment_done(pdir, 1, "test.wav")`
24. **Line 24:** `data = project.load_project(pdir)`
Assigns `project.load_project(pdir)` to `data` .
25. **Line 25:** `assert data["segments"][0]["status"] == "done"`
Assertion that verifies a condition is True.

63.3 Key Takeaways

- **Smoke tests** verify objects can be created without crashes
- **End-to-end tests** exercise the full pipeline (parse → split → save → load)

- AIVS tests avoid actual TTS synthesis (too slow, requires models)
 - The test suite runs in **under 30 seconds**
-

[← Ch 62](#) | [Ch 64 — PyInstaller](#) [→](#)

PyInstaller Packaging

i Info: Current implementation (2026.3.1). The spec now also bundles the whole `docs` folder — including `docs/book` — and lists the newer modules in `hiddenimports` : `ai_voice_studio.gui.accept_dialog` (the first-launch terms gate, which `main.py` imports before anything else), `gui.compute_choice` , `gui.clone_engines_panel` , `gui.voicelab_options_dialog` , `gui.omnivoice_options_dialog` , the `voicelab / omnivoice / omnivoice_server` packages, `documents.daisy3_builder` , `documents.playlist_builder` and `tts.windows_tts` . PyTorch and the Voice Lab engines are deliberately *not* frozen: they are installed at runtime into per-engine environments under `%APPDATA%\AIVoiceStudio\tts_envs` . A missing `hiddenimports` entry fails in the frozen build only, so smoke-run the frozen executable with an isolated `%APPDATA%` after every packaging change.

Learning Objectives

- Understand how PyInstaller bundles Python apps
- Configure the `.spec` file for AIVS
- Handle hidden imports and data files

64.1 What PyInstaller Does

PyInstaller analyzes Python imports and bundles everything — the interpreter, all dependencies, and data files — into a single executable or folder.

```
# Basic usage
pip install pyinstaller
pyinstaller --onefile main.py

# With spec file (what AIVS uses)
pyinstaller packaging/ai_voice_studio.spec --noconfirm
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Basic usage`
A comment: *Basic usage*.
2. **Line 2:** `pip install pyinstaller`
Command-line tool for building executables. Supports --onefile, --windowed, --add-data options.
3. **Line 3:** `pyinstaller --onefile main.py`
Command-line tool for building executables. Supports --onefile, --windowed, --add-data options.
4. **Line 4:** `# With spec file (what AIVS uses)`
A comment: *With spec file (what AIVS uses)*.
5. **Line 5:** `pyinstaller packaging/ai_voice_studio.spec --noconfirm`
Command-line tool for building executables. Supports --onefile, --windowed, --add-data options.

64.2 The AIVS Spec File

```
# packaging/ai_voice_studio.spec (simplified)
a = Analysis(
    ['main.py'],
    pathex=[],
    binaries=[],
    datas=[
        ('ai_voice_studio/tts/catalog.py', 'ai_voice_studio/tts'),
    ],
    hiddenimports=[
        'wx',
        'sherpa_onnx',
        'numpy',
        'pypdf',
        'docx',
    ],
    hookspath=[],
    runtime_hooks=[],
    excludes=[],
    noarchive=False,
)

pyz = PYZ(a.pure)

exe = EXE(
    pyz,
    a.scripts,
    a.binaries,
    a.datas,
    [],
    name='AI-Voice-Studio',
    debug=False,
    strip=False,
    upx=True,
    console=False, # No console window
    icon='packaging/icon.ico',
)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# packaging/ai_voice_studio.spec (simplified)`
A comment: *packaging/ai_voice_studio.spec (simplified)*.
2. **Line 2:** `a = Analysis(`
PyInstaller spec component that analyzes the script and collects dependencies.
3. **Line 3:** `['main.py'],`
`['main.py'],`
4. **Line 4:** `pathex=[],`
Assigns `[]`, to `pathex`.
5. **Line 5:** `binaries=[],`
Assigns `[]`, to `binaries`.
6. **Line 6:** `datas=[`
Assigns `[]` to `datas`.
7. **Line 7:** `('ai_voice_studio/tts/catalog.py', 'ai_voice_studio/tts'),`
`('ai_voice_studio/tts/catalog.py', 'ai_voice_studio/tts'),`
8. **Line 8:** `],`
`],`
9. **Line 9:** `hiddenimports=[`
PyInstaller parameter for modules that aren't automatically detected.
10. **Line 10:** `'wx',`
`'wx',`
11. **Line 11:** `'sherpa_onnx',`
`'sherpa_onnx',`
12. **Line 12:** `'numpy',`
`'numpy',`
13. **Line 13:** `'pypdf',`
`'pypdf',`

14. Line 14: `'docx',`
`'docx',`

15. Line 15: `],`
`],`

16. Line 16: `hookspath=[],`
Assigns `[]`, to `hookspath` .

17. Line 17: `runtime_hooks=[],`
Assigns `[]`, to `runtime_hooks` .

18. Line 18: `excludes=[],`
Assigns `[]`, to `excludes` .

19. Line 19: `noarchive=False,`
Assigns `False`, to `noarchive` .

20. Line 20: `)`
`)`

21. Line 21: `pyz = PYZ(a.pure)`
PyInstaller spec component that creates the compressed Python archive.

22. Line 22: `exe = EXE(`
PyInstaller spec component that creates the final executable.

23. Line 23: `pyz,`
`pyz,`

24. Line 24: `a.scripts,`
`a.scripts,`

25. Line 25: `a.binaries,`
`a.binaries,`

26. Line 26: `a.datas,`
`a.datas,`

27. Line 27: `[],`

```
[],  
  
28. Line 28: name='AI-Voice-Studio',  
Assigns 'AI-Voice-Studio', to name .  
  
29. Line 29: debug=False,  
Assigns False, to debug .  
  
30. Line 30: strip=False,  
Assigns False, to strip .  
  
31. Line 31: upx=True,  
Assigns True, to upx .  
  
32. Line 32: console=False, # No console window  
Assigns False, # No console window to console .  
  
33. Line 33: icon='packaging/icon.ico',  
Assigns 'packaging/icon.ico', to icon .  
  
34. Line 34: )  
)
```

64.3 Build Commands

```
# From packaging/build.ps1  
# Build x64 distribution  
.venv\Scripts\python -m PyInstaller packaging\ai_voice_studio.spec `  
    --noconfirm `  
    --distpath dist\AIVS-64 `  
    --workpath build\pyinstaller-x64  
  
# Output: dist/AIVS-64/AI-Voice-Studio/AI-Voice-Studio.exe
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# From packaging/build.ps1`
A comment: *From packaging/build.ps1.*
2. **Line 2:** `# Build x64 distribution`
A comment: *Build x64 distribution.*
3. **Line 3:** `.venv\Scripts\python -m PyInstaller packaging\ai_voice_studio.spec``
A tool that packages Python apps into standalone executables. Bundles Python interpreter and all dependencies.
4. **Line 4:** `--noconfirm``
PyInstaller flag: overwrites output without asking.
5. **Line 5:** `--distpath dist\AIVS-64``
`--distpath dist\AIVS-64``
6. **Line 6:** `--workpath build\pyinstaller-x64`
Command-line tool for building executables. Supports `--onefile`, `--windowed`, `--add-data` options.
7. **Line 7:** `# Output: dist/AIVS-64/AI-Voice-Studio/AI-Voice-Studio.exe`
A comment: *Output: dist/AIVS-64/AI-Voice-Studio/AI-Voice-Studio.exe.*

64.4 Key Takeaways

- PyInstaller bundles everything into a **single executable** or folder
 - The **.spec file** configures hidden imports, data files, and build options
 - AIVS output is ~**39 MB** (compressed)
 - **console=False** hides the console window for the GUI app
 - The build is automated via **packaging/build.ps1**
-

Inno Setup Installers

i Info: Current implementation (2026.3.1). The scripts are shared through `installer_common.iss`, which the two architecture files include. The version strings (`AppVersion`, `VersionInfoVersion`, `VersionInfoProductVersion`) live there and must match `constants.APP_VERSION`; the output file is `AI-Voice-Studio-v-<version>-Setup-x64.exe`. The `[Files]` section copies `LICENSE`, `README.md` and five documents into `{app}\docs`: `README.html`, `UserGuide.html`, `AddonDevelopmentGuide.html`, `AccessibilityGuide.html` and `THIRD-PARTY-LICENSES.html` (the last is also the installer's licence page, and it is what *Help → Third-Party Licences* opens). The book itself travels inside the PyInstaller output (`_internal\docs\book`) rather than being listed here.

Learning Objectives

- Understand the Inno Setup compilation process
- See the installer configuration
- Understand the full build pipeline

65.1 Build Pipeline

```
Development venv → PyInstaller → dist/AIVS-64/ → Inno Setup → Setup.exe
```

```
# Step 1: PyInstaller bundles the app
pyinstaller packaging/ai_voice_studio.spec
```

```
# Step 2: Inno Setup wraps it into an installer
"C:\Program Files\Inno Setup 7\ISCC.exe" packaging/installer_64.iss
```

```
# Output: dist/AI-Voice-Studio-v-2026-2-0-Setup-x64.exe (39 MB)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `Development venv → PyInstaller → dist/AIVS-64/ → Inno Setup → Setup.exe`
A tool that packages Python apps into standalone executables. Bundles Python interpreter and all dependencies.
2. **Line 2:** `# Step 1: PyInstaller bundles the app`
A comment: *Step 1: PyInstaller bundles the app.*
3. **Line 3:** `pyinstaller packaging/ai_voice_studio.spec`
Command-line tool for building executables. Supports `--onefile`, `--windowed`, `--add-data` options.
4. **Line 4:** `# Step 2: Inno Setup wraps it into an installer`
A comment: *Step 2: Inno Setup wraps it into an installer.*
5. **Line 5:** `"C:\Program Files\Inno Setup 7\ISCC.exe" packaging/installer_64.iss`
A Windows installer builder. Compiles `.iss` scripts into `setup.exe` files.
6. **Line 6:** `# Output: dist/AI-Voice-Studio-v-2026-2-0-Setup-x64.exe (39 MB)`
A comment: *Output: dist/AI-Voice-Studio-v-2026-2-0-Setup-x64.exe (39 MB).*

65.2 Installer Features

- **"Install for all users" / "for me only"** choice (no admin needed for per-user)
 - Install location picker
 - Start Menu folder choice
 - Optional **desktop icon**
 - "Open Read Me after install" checkbox
 - "Launch AI Voice Studio after setup" checkbox
 - **GPL-3.0 license** page
 - Uninstaller with clean removal
-

65.3 Inno Setup Script Structure

```
; packaging/installer_64.iss (simplified)
[Setup]
AppName=AI Voice Studio
AppVersion=2.0.0
DefaultDirName={autopf}\AI Voice Studio
DefaultGroupName=AI Voice Studio
LicenseFile=..\LICENSE
OutputDir=..\dist
OutputBaseFilename=AI-Voice-Studio-v-2026-2-0-Setup-x64
Compression=lzma2/ultra64
SolidCompression=yes
ArchitecturesAllowed=x64compatible
ArchitecturesInstallIn64BitMode=x64compatible

[Files]
Source: "..\dist\AIVS-64\AI-Voice-Studio\*"; DestDir: "{app}"; Flags: ignoreversion recursesubdirs

[Icons]
Name: "{group}\AI Voice Studio"; Filename: "{app}\AI-Voice-Studio.exe"
Name: "{autodesktop}\AI Voice Studio"; Filename: "{app}\AI-Voice-Studio.exe"; Tasks: desktopicon

[Run]
Filename: "{app}\AI-Voice-Studio.exe"; Description: "Launch AI Voice Studio"; Flags: nowait postinstall skipifsilent
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `; packaging/installer_64.iss (simplified)`
Inno Setup Script file. Defines the installer: files, icons, license, dialogs.
2. **Line 2:** `[Setup]`
Inno Setup section for installer metadata (name, version, output file).
3. **Line 3:** `AppName=AI Voice Studio`
Assigns `AI Voice Studio` to `AppName`.
4. **Line 4:** `AppVersion=2.0.0`
Inno Setup directive setting the application version number.
5. **Line 5:** `DefaultDirName={autopf}\AI Voice Studio`
Inno Setup directive setting the default installation directory.
6. **Line 6:** `DefaultGroupName=AI Voice Studio`
Assigns `AI Voice Studio` to `DefaultGroupName`.
7. **Line 7:** `LicenseFile=..\LICENSE`
Assigns `..\LICENSE` to `LicenseFile`.
8. **Line 8:** `OutputDir=..\dist`
Assigns `..\dist` to `OutputDir`.
9. **Line 9:** `OutputBaseFilename=AI-Voice-Studio-v-2026-2-0-Setup-x64`
Inno Setup directive setting the output installer filename.
10. **Line 10:** `Compression=lzma2/ultra64`
Assigns `lzma2/ultra64` to `Compression`.
11. **Line 11:** `SolidCompression=yes`
Assigns `yes` to `SolidCompression`.
12. **Line 12:** `ArchitecturesAllowed=x64compatible`
Assigns `x64compatible` to `ArchitecturesAllowed`.
13. **Line 13:** `ArchitecturesInstallIn64BitMode=x64compatible`
Assigns `x64compatible` to `ArchitecturesInstallIn64BitMode`.

14. **Line 14:** `[Files]`
Inno Setup section listing files to include in the installer.
15. **Line 15:**
`Source: "..\dist\AIVS-64\AI-Voice-Studio\*"; DestDir: "{app}"; Flags: ignoreversion recursesubdirs`
`Source: "..\dist\AIVS-64\AI-Voice-Studio\*"; DestDir: "{app}"; Flags: ignoreversion recursesubdirs`
16. **Line 16:** `[Icons]`
Inno Setup section for Start Menu and desktop shortcuts.
17. **Line 17:** `Name: "{group}\AI Voice Studio"; Filename: "{app}\AI-Voice-Studio.exe"`
`Name: "{group}\AI Voice Studio"; Filename: "{app}\AI-Voice-Studio.exe"`
18. **Line 18:**
`Name: "{autodesktop}\AI Voice Studio"; Filename: "{app}\AI-Voice-Studio.exe"; Tasks: desktopicon`
`Name: "{autodesktop}\AI Voice Studio"; Filename: "{app}\AI-Voice-Studio.exe"; Tasks: desktopicon`
19. **Line 19:** `[Run]`
Inno Setup section for programs to run after installation.
20. **Line 20:**
`Filename: "{app}\AI-Voice-Studio.exe"; Description: "Launch AI Voice Studio"; Flags: nowait postinst all skipifsilent`
`Filename: "{app}\AI-Voice-Studio.exe"; Description: "Launch AI Voice Studio"; Flags: nowait postinstall skipifsilent`

65.4 Key Takeaways

- **Inno Setup 7** creates professional Windows installers
 - The installer wraps the **PyInstaller output** into a setup.exe
 - **LZMA2 compression** reduces the 39 MB installer size
 - The build is automated via **build.ps1** (PowerShell)
 - Output: **AI-Voice-Studio-v-2026-2-0-Setup-x64.exe**
-

CI/CD & Release

Learning Objectives

- Understand continuous integration concepts
 - Set up GitHub Actions for automated testing
 - Plan the release workflow
-

66.1 GitHub Actions Workflow

```
# .github/workflows/test.yml
name: Tests
on: [push, pull_request]

jobs:
  test:
    runs-on: windows-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: '3.13'
      - run: pip install -r requirements.txt pytest
      - run: python -m pytest tests/ -v
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# .github/workflows/test.yml`

A comment: `.github/workflows/test.yml`.

2. **Line 2:** `name: Tests`

`name: Tests`

3. **Line 3:** `on: [push, pull_request]`

`on: [push, pull_request]`

4. **Line 4:** `jobs:`

`jobs:`

5. **Line 5:** `test:`

`test:`

6. **Line 6:** `runs-on: windows-latest`

`runs-on: windows-latest`

7. **Line 7:** `steps:`

`steps:`

8. **Line 8:** `- uses: actions/checkout@v4`

`- uses: actions/checkout@v4`

9. **Line 9:** `- uses: actions/setup-python@v5`

`- uses: actions/setup-python@v5`

10. **Line 10:** `with:`

`with:`

11. **Line 11:** `python-version: '3.13'`

`python-version: '3.13'`

12. **Line 12:** `- run: pip install -r requirements.txt pytest`

The testing framework used by AIVS. Discovers test files named `test_*.py`.

13. **Line 13:** `- run: python -m pytest tests/ -v`

The testing framework used by AIVS. Discovers test files named test_*.py.

66.2 Release Process

1. Update version in `constants.py`
2. Run full test suite
3. Run PyInstaller build
4. Compile Inno Setup installer
5. Test installer on clean Windows VM
6. Create GitHub release with installer

66.3 Key Takeaways

- **GitHub Actions** provides automated testing on every push
- The release pipeline: **Test** → **Build** → **Package** → **Test** → **Release**
- Always test the installer on a **clean machine** before releasing

Advanced Topics & Extensions

CHAPTER 67

Managed Python Runtime

i Info: Current implementation (2026.3.1). `python_runtime` no longer manages one environment. It now creates and owns **one virtualenv per pip-installed TTS engine** under `tts_envs\<engine>` :

- `environment_id(engine_id)` maps an engine id to its environment; `SHARED_ENVIRONMENTS` folds `omnivoice_server` onto `omnivoice` (the one deliberate exception), and everything else falls back to the shared `addon_env` .
- `engine_runtime(engine_id)` prefers the engine's own environment and falls back to `addon_env` for engines installed before this change.
- `venv_packages` caches its probes *per environment*, and `package_present()` answers from the environment's `site-packages` on disk for the decisions that must be right immediately — which interpreter starts an engine's worker.
- Installs pull CUDA PyTorch from `voicelab.engines.PYTORCH_CUDA_INDEX` when an NVIDIA GPU is present, and fall back to the plain PyPI packages when no CUDA wheel exists for the interpreter.

The full design, including the `ModuleNotFoundError: torch` bug that forced the site-packages check, is [Chapter 77](#).

Learning Objectives

- Understand why AIVS manages its own Python environment
- See how the PythonRuntime creates and manages venvs
- Understand the frozen app vs. development mode distinction

67.1 Why a Managed Runtime?

AIVS needs to install optional packages (OmniVoice, coqui-tts) into an isolated venv without touching the main application's packages. This is critical because:

- The main app uses **sherpa-onnx** (ONNX runtime)
 - Optional engines use **PyTorch/CUDA** (conflicts with ONNX)
 - Addons may need their own dependencies
 - Never modify the installer's packages
-

67.2 PythonRuntime Architecture

```
# %APPDATA%/AIVoiceStudio/addon_env/  
# └─ Scripts/python.exe      (Windows)  
# └─ Scripts/pip.exe  
# └─ Lib/site-packages/      (installed packages)  
  
class PythonRuntime:  
    def ensure_env(self):  
        """Create the venv if it doesn't exist."""  
        if not self.is_created:  
            subprocess.run([self.system_python, "-m", "venv", self._env_dir])  
  
    def pip_install(self, packages, progress=None):  
        """Install packages into the managed venv."""  
        pip = self.ensure_pip()  
        proc = subprocess.Popen([pip, "install"] + packages, ...)  
        for line in proc.stdout:  
            if progress: progress(line, 0, 0)  
  
    def run_in_env(self, script):  
        """Run a Python script inside the managed venv."""  
        return subprocess.run([self.python_exe, "-c", script], ...)  
  
    def is_package_installed(self, package_name):  
        """Check if a package is installed in the venv."""  
        result = self.run_in_env(  
            f"import importlib.metadata; print(importlib.metadata.version('{package_name}'))"  
        )  
        return result.returncode == 0
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# %APPDATA%/AIVoiceStudio/addon_env/`
A comment: `%APPDATA%/AIVoiceStudio/addon_env/`.
2. **Line 2:** `# |— Scripts/python.exe (Windows)`
A comment: `|— Scripts/python.exe (Windows)`.
3. **Line 3:** `# |— Scripts/pip.exe`
A comment: `|— Scripts/pip.exe`.
4. **Line 4:** `# |— Lib/site-packages/ (installed packages)`
A comment: `|— Lib/site-packages/ (installed packages)`.
5. **Line 5:** `class PythonRuntime:`
Manages the embedded Python interpreter for addons. Creates isolated venvs.
6. **Line 6:** `def ensure_env(self):`
Defines a function.
7. **Line 7:** `"""Create the venv if it doesn't exist."""`
Docstring: *Create the venv if it doesn't exist.*. Documents the following code.
8. **Line 8:** `if not self.is_created:`
Conditional check.
9. **Line 9:** `subprocess.run([self.system_python, "-m", "venv", self._env_dir])`
Runs a command, waits for completion. Returns CompletedProcess. Use `capture_output=True` to capture stdout/stderr.
10. **Line 10:** `def pip_install(self, packages, progress=None):`
Returns True if all elements in the iterable are truthy.
11. **Line 11:** `"""Install packages into the managed venv."""`
Docstring: *Install packages into the managed venv.*. Documents the following code.
12. **Line 12:** `pip = self.ensure_pip()`
Assigns `self.ensure_pip()` to `pip`.
13. **Line 13:** `proc = subprocess.Popen([pip, "install"] + packages, ...)`
Starts a process. More control than `run()` — can read stdout/stderr asynchronously.
14. **Line 14:** `for line in proc.stdout:`
For loop iteration.

15. **Line 15:** `if progress: progress(line, 0, 0)`
Conditional check.
16. **Line 16:** `def run_in_env(self, script):`
Defines a function.
17. **Line 17:** `"""Run a Python script inside the managed venv."""`
Docstring: *Run a Python script inside the managed venv..* Documents the following code.
18. **Line 18:** `return subprocess.run([self.python_exe, "-c", script], ...)`
Runs a command, waits for completion. Returns CompletedProcess. Use capture_output=True to capture stdout/stderr.
19. **Line 19:** `def is_package_installed(self, package_name):`
Defines a function.
20. **Line 20:** `"""Check if a package is installed in the venv."""`
Docstring: *Check if a package is installed in the venv..* Documents the following code.
21. **Line 21:** `result = self.run_in_env(`
Assigns `self.run_in_env(` to `result` .
22. **Line 22:** `f"import importlib.metadata; print(importlib.metadata.version('{package_name}'))"`
Imports a module.
23. **Line 23:** `)`
`)`
24. **Line 24:** `return result.returncode == 0`
Returns a value from a function.
-

67.3 Frozen App vs. Development

```
# In development: sys.executable is the venv Python
# In frozen app: sys.executable is the .exe (can't run pip)

def _find_system_python() -> str:
    if not getattr(sys, "frozen", False):
        return sys.executable # Dev: use current Python
    # Frozen: find system Python on PATH
    for name in ("python3", "python", "py"):
        path = shutil.which(name)
        if path: return path
    # Try common Windows install paths
    ...
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# In development: sys.executable is the venv Python`
A comment: *In development: sys.executable is the venv Python.*
2. **Line 2:** `# In frozen app: sys.executable is the .exe (can't run pip)`
A comment: *In frozen app: sys.executable is the .exe (can't run pip).*
3. **Line 3:** `def _find_system_python() -> str:`
Defines a function.
4. **Line 4:** `if not getattr(sys, "frozen", False):`
Gets an attribute value by name.
5. **Line 5:** `return sys.executable # Dev: use current Python`
Returns a value from a function.
6. **Line 6:** `# Frozen: find system Python on PATH`
A comment: *Frozen: find system Python on PATH.*
7. **Line 7:** `for name in ("python3", "python", "py"):`
For loop iteration.
8. **Line 8:** `path = shutil.which(name)`
Finds the full path of an executable in the system PATH. Returns None if not found.
9. **Line 9:** `if path: return path`
Returns a value from a function.
10. **Line 10:** `# Try common Windows install paths`
A comment: *Try common Windows install paths.*
11. **Line 11:** `...`
Ellipsis: abstract method placeholder.

67.4 Key Takeaways

- The **PythonRuntime** manages an isolated venv for optional packages
- **run_in_env()** executes scripts inside the managed venv

- The runtime must work in both **dev mode** and **frozen app** mode
 - Package installation is **cancellable** via `threading.Event`
-

[← Ch 66](#) | [Ch 68 — Subprocess](#) [→](#)

Subprocess Architecture

i Info: Current implementation (2026.3.1). The isolation table below is still the reason workers exist, but there are now more of them and each pip-installed engine's worker runs on *that engine's own* interpreter, from `%APPDATA%\AIVoiceStudio\tts_envs\<engine>` , instead of one shared environment ([Chapter 77](#)): the Voice Lab worker (`voicelab/worker.py` , JSON lines over stdout), the OmniVoice worker, the OmniVoice HTTP server, and the legacy fallback in the shared `addon_env` . Which interpreter is chosen is decided from the environment's `site-packages` listing, never from a cached probe.

Learning Objectives

- Understand why AIVS uses subprocess isolation
- See the JSON-lines IPC pattern
- Understand the frozen app worker embedding

68.1 Why Subprocess Isolation?

Component	Runtime	Isolation Reason
Main App	wxPython + sherpa-onnx (ONNX)	Core application
Voice Lab Worker	PyTorch + the engine's own packages (one environment per engine)	PyTorch conflicts with ONNX
OmniVoice Worker	PyTorch + CUDA	CUDA/PyTorch isolation
OmniVoice Server	HTTP server + PyTorch	Network service isolation

68.2 JSON-Lines IPC Pattern

```
# All workers follow the same pattern:
# 1. Worker reads JSON from stdin (one line per request)
# 2. Worker writes JSON to stdout (one line per response)
# 3. App manages the subprocess lifecycle

# Worker Loop
for line in sys.stdin:
    req = json.loads(line.strip())
    if req["cmd"] == "quit": break
    if req["cmd"] == "ping":
        respond({"ok": True, "engine": True})
    if req["cmd"] == "synthesize":
        audio = synthesize(req["text"])
        respond({"ok": True, "wav": base64.b64encode(audio)})

# App-side request
def _request(self, req):
    with self._lock:
        req["id"] = self._next_id
        self._proc.stdin.write(json.dumps(req) + "\n")
        line = self._proc.stdout.readline()
        return json.loads(line)
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# All workers follow the same pattern:`
A comment: *All workers follow the same pattern:.*
2. **Line 2:** `# 1. Worker reads JSON from stdin (one line per request)`
A comment: *1. Worker reads JSON from stdin (one line per request).*
3. **Line 3:** `# 2. Worker writes JSON to stdout (one line per response)`
A comment: *2. Worker writes JSON to stdout (one line per response).*
4. **Line 4:** `# 3. App manages the subprocess lifecycle`
A comment: *3. App manages the subprocess lifecycle.*
5. **Line 5:** `# Worker loop`
A comment: *Worker loop.*
6. **Line 6:** `for line in sys.stdin:`
For loop iteration.
7. **Line 7:** `req = json.loads(line.strip())`
Deserializes JSON from a file into a Python object.
8. **Line 8:** `if req["cmd"] == "quit": break`
Exits the innermost loop.
9. **Line 9:** `if req["cmd"] == "ping":`
Conditional check.
10. **Line 10:** `respond({"ok": True, "engine": True})`
Calls function `respond()` .
11. **Line 11:** `if req["cmd"] == "synthesize":`
Conditional check.
12. **Line 12:** `audio = synthesize(req["text"])`
Assigns `synthesize(req["text"])` to `audio` .
13. **Line 13:** `respond({"ok": True, "wav": base64.b64encode(audio)})`
Calls function `respond()` .
14. **Line 14:** `# App-side request`

A comment: *App-side request.*

15. **Line 15:** `def _request(self, req):`
Defines a function.
16. **Line 16:** `with self._lock:`
Context manager: `self._lock`. Auto-manages cleanup.
17. **Line 17:** `req["id"] = self._next_id`
`req["id"] = self._next_id`
18. **Line 18:** `self._proc.stdin.write(json.dumps(req) + "\n")`
Serializes a Python object to JSON and writes to a file.
19. **Line 19:** `line = self._proc.stdout.readline()`
Assigns `self._proc.stdout.readline()` to `line`.
20. **Line 20:** `return json.loads(line)`
Deserializes JSON from a file into a Python object.

68.3 Frozen App Worker Embedding

```
# In frozen apps, PyInstaller bundles .py files into a PYZ archive
# The managed venv's Python can't read .pyz files
# Solution: embed worker source as a string constant
```

```
_WORKER_SOURCE = r'''
import json, sys, numpy as np
def main():
    for line in sys.stdin:
        req = json.loads(line.strip())
        if req["cmd"] == "quit": break
        if req["cmd"] == "synthesize":
            # ... synthesize and respond ...
    ...

def _read_worker_source() -> str:
    if getattr(sys, "frozen", False):
        return _WORKER_SOURCE # Use embedded source
    # Dev mode: read from actual file
    with open("worker.py") as f:
        return f.read()
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# In frozen apps, PyInstaller bundles .py files into a PYZ archive`

A comment: *In frozen apps, PyInstaller bundles .py files into a PYZ archive.*

2. **Line 2:** `# The managed venv's Python can't read .pyz files`

A comment: *The managed venv's Python can't read .pyz files.*

3. **Line 3:** `# Solution: embed worker source as a string constant`

A comment: *Solution: embed worker source as a string constant.*

4. **Line 4:** `_WORKER_SOURCE = r'''`

Assigns `r'''` to `_WORKER_SOURCE`.

5. **Line 5:** `import json, sys, numpy as np`

Imports a module.

6. **Line 6:** `def main():`

Defines a function.

7. **Line 7:** `for line in sys.stdin:`

For loop iteration.

8. **Line 8:** `req = json.loads(line.strip())`

Deserializes JSON from a file into a Python object.

9. **Line 9:** `if req["cmd"] == "quit": break`

Exits the innermost loop.

10. **Line 10:** `if req["cmd"] == "synthesize":`

Conditional check.

11. **Line 11:** `# ... synthesize and respond ...`

A comment: *... synthesize and respond ...*

12. **Line 12:** `'''`

Opens a docstring block.

13. **Line 13:** `def _read_worker_source() -> str:`

Defines a function.

14. **Line 14:** `if getattr(sys, "frozen", False):`

Gets an attribute value by name.

15. **Line 15:** `return _WORKER_SOURCE # Use embedded source`
Returns a value from a function.
16. **Line 16:** `# Dev mode: read from actual file`
A comment: *Dev mode: read from actual file.*
17. **Line 17:** `with open("worker.py") as f:`
Opens a file using a context manager. Auto-closes when the block exits.
18. **Line 18:** `return f.read()`
Returns a value from a function.

68.4 Key Takeaways

- **Subprocess isolation** prevents runtime conflicts (ONNX vs PyTorch)
 - **JSON-lines protocol** is the universal IPC pattern
 - **Embedded worker source** solves the frozen app problem
 - All workers follow the same **lifecycle**: start → ping → process → quit
-

Logging & Diagnostics

i Info: Current implementation (2026.3.1). `heavy_logging.py` installs per-subsystem log files (`gui.log` , `tts.log` , `engine.log` , `recording.log` , `addons.log` , `download.log`) plus `crash.log` for unhandled exceptions, and `log_exception_with_context()` is the hook addon authors are told to use. Heavy logging is switched on by Developer Mode (`settings.developer_mode`), which is also what reveals the Developer category. The application always writes `logs\app.log` . A smoke run of the frozen build is checked by reading that file: a clean launch has the startup lines and no traceback, which is how the terms-dialog `NameError` and a missing `hiddenimports` entry were both caught.

Learning Objectives

- Understand the AIVS logging system (basic and heavy modes)
- See how developer mode enables detailed diagnostics
- Understand the crash log system

69.1 Two Logging Modes

```
# From main.py
def _setup_logging(developer_mode=False):
    if developer_mode:
        from ai_voice_studio.heavy_logging import setup_heavy_logging
        setup_heavy_logging(enabled=True)
    else:
        logging.basicConfig(
            filename=paths.log_file(),
            level=logging.INFO,
            format="%(asctime)s %(levelname)s %(name)s: %(message)s",
        )
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# From main.py`
A comment: *From main.py*.
2. **Line 2:** `def _setup_logging(developer_mode=False):`
Defines a function.
3. **Line 3:** `if developer_mode:`
Conditional check.
4. **Line 4:** `from ai_voice_studio.heavy_logging import setup_heavy_logging`
Imports a module.
5. **Line 5:** `setup_heavy_logging(enabled=True)`
Calls function `setup_heavy_logging()`.
6. **Line 6:** `else:`
Fallback branch.
7. **Line 7:** `logging.basicConfig(`
Configures the root logger. Sets format, level, and output destination (file or stream).
8. **Line 8:** `filename=paths.log_file(),`
Assigns `paths.log_file()`, to `filename`.
9. **Line 9:** `level=logging.INFO,`
Level 20. Default level. Shows normal operation messages.
10. **Line 10:** `format="%(asctime)s %(levelname)s %(name)s: %(message)s",`
Assigns `"%(asctime)s %(levelname)s %(name)s: %(message)s"`, to `format`.
11. **Line 11:** `)`
`)`

Basic Mode (Default)

- INFO level logging to `app.log`
- Single log file

Developer Mode (Heavy Logging)

- DEBUG level logging
 - Multiple subsystem log files
 - Full tracebacks for all caught exceptions
 - Thread crash logging
 - Crash log for unhandled exceptions
 - System info logging (OS, Python, GPU)
-

69.2 Logging Best Practices in AIVS

```
import logging
log = logging.getLogger(__name__)

# Info: normal operations
log.info("AI Voice Studio starting (Python {sys.version.split()[0]})")

# Warning: recoverable issues
log.warning("Model not found at {path}")

# Error: failures with context
log.error("Failed to load model: {exc}", exc_info=True)

# Debug: detailed diagnostic info
log.debug("Resolved compute mode: {mode}")
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `import logging`
Imports a module.
2. **Line 2:** `log = logging.getLogger(__name__)`
Creates or retrieves a logger by name. Hierarchical naming (a.b.c) enables per-module control.
3. **Line 3:** `# Info: normal operations`
A comment: *Info: normal operations*.
4. **Line 4:** `log.info("AI Voice Studio starting (Python {sys.version.split()[0]})")`
Logs an informational message. The default level. For normal operation events.
5. **Line 5:** `# Warning: recoverable issues`
A comment: *Warning: recoverable issues*.
6. **Line 6:** `log.warning("Model not found at {path}")`
Logs a warning. Something unexpected happened but the program continues.
7. **Line 7:** `# Error: failures with context`
A comment: *Error: failures with context*.
8. **Line 8:** `log.error("Failed to load model: {exc}", exc_info=True)`
Logs an error. Something failed but the program can still run.
9. **Line 9:** `# Debug: detailed diagnostic info`
A comment: *Debug: detailed diagnostic info*.
10. **Line 10:** `log.debug("Resolved compute mode: {mode}")`
Logs a debug message. Only visible when the level is set to DEBUG. For detailed diagnostic info.

69.3 Key Takeaways

- **Two modes:** basic (INFO to app.log) and heavy (DEBUG to multiple files)
 - **Developer mode** is toggled from Settings → General
 - **exc_info=True** captures full tracebacks in error logs
 - **Each module** gets its own logger: `logging.getLogger(__name__)`
-

Performance Optimization

i Info: Current implementation (2026.3.1). The single biggest performance decision in this release is the CPU thread budget: `compute.cpu_threads()` gives a run **80–95 %** of the machine's logical CPUs (one core kept free on small machines), and that value is passed to sherpa-onnx's `num_threads` and to `torch.set_num_threads` in the worker, so the same budget applies to every backend. On the GPU side the first CUDA device is used with the cuDNN autotuner, TF32 fast paths, high float32 matmul precision and the autograd engine disabled. Engine instances are cached per `(provider, model)`, so switching the Compute combo rebuilds an engine instead of leaking two. See [Chapter 78](#).

Learning Objectives

- Understand AIVS performance considerations
- See lazy loading, caching, and thread optimization
- Understand memory management for audio processing

70.1 Lazy Loading

```
# Heavy imports are deferred until needed
def heavy_function():
    import numpy # Only imported when function is called
    return np.array([1, 2, 3])

# TTS engines are loaded on first use
_engine_cache: dict[str, TtsEngine] = {}

def get_engine(voice_entry):
    key = voice_entry["voice"]
    if key not in _engine_cache:
        _engine_cache[key] = TtsEngine(voice_entry) # Expensive!
    return _engine_cache[key]
```


Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Heavy imports are deferred until needed`

A comment: *Heavy imports are deferred until needed.*

2. **Line 2:** `def heavy_function():`

Defines a function.

3. **Line 3:** `import numpy # Only imported when function is called`

Imports a module.

4. **Line 4:** `return np.array([1, 2, 3])`

Creates a NumPy array from a list or tuple. The dtype parameter specifies the data type (e.g., np.int16 for 16-bit audio samples).

5. **Line 5:** `# TTS engines are loaded on first use`

A comment: *TTS engines are loaded on first use.*

6. **Line 6:** `_engine_cache: dict[str, TtsEngine] = {}`

`_engine_cache: dict[str, TtsEngine] = {}`

7. **Line 7:** `def get_engine(voice_entry):`

Defines a function.

8. **Line 8:** `key = voice_entry["voice"]`

Assigns `voice_entry["voice"]` to `key`.

9. **Line 9:** `if key not in _engine_cache:`

Conditional check.

10. **Line 10:** `_engine_cache[key] = TtsEngine(voice_entry) # Expensive!`

`_engine_cache[key] = TtsEngine(voice_entry) # Expensive!`

11. **Line 11:** `return _engine_cache[key]`

Returns a value from a function.

70.2 Caching

```
# functools.lru_cache for expensive lookups
@functools.lru_cache(maxsize=128)
def find_tts_cached(tts_id: str):
    return catalog.find_tts(tts_id)

# Engine caching prevents reloading models
# (Models can be 50-500 MB, Loading takes seconds)
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# functools.lru_cache for expensive lookups`
A comment: *functools.lru_cache for expensive lookups.*
2. **Line 2:** `@functools.lru_cache(maxsize=128)`
Decorator that caches function results. Reduces repeated expensive computations. maxsize controls cache size.
3. **Line 3:** `def find_tts_cached(tts_id: str):`
Defines a function.
4. **Line 4:** `return catalog.find_tts(tts_id)`
Returns a value from a function.
5. **Line 5:** `# Engine caching prevents reloading models`
A comment: *Engine caching prevents reloading models.*
6. **Line 6:** `# (Models can be 50-500 MB, loading takes seconds)`
A comment: *(Models can be 50-500 MB, loading takes seconds).*

70.3 Audio Memory Management

```
# Audio arrays can be large (1 sec = 48KB at 24kHz/16-bit)
# A 10-minute recording = ~28 MB

# Use appropriate dtypes
samples = np.zeros(240000, dtype=np.int16) # 10 sec = 480 KB

# Release memory promptly
del large_array
gc.collect()

# Base64 encoding doubles size temporarily
wav_b64 = base64.b64encode(samples.tobytes()) # 2x memory
# Decode immediately after receiving
samples = np.frombuffer(base64.b64decode(wav_b64), dtype=np.int16)
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `# Audio arrays can be large (1 sec = 48KB at 24kHz/16-bit)`

A comment: *Audio arrays can be large (1 sec = 48KB at 24kHz/16-bit).*

2. **Line 2:** `# A 10-minute recording = ~28 MB`

A comment: *A 10-minute recording = ~28 MB.*

3. **Line 3:** `# Use appropriate dtypes`

A comment: *Use appropriate dtypes.*

4. **Line 4:** `samples = np.zeros(240000, dtype=np.int16) # 10 sec = 480 KB`

Creates an array filled with zeros. Shape is specified as (rows, cols) or (length,).

5. **Line 5:** `# Release memory promptly`

A comment: *Release memory promptly.*

6. **Line 6:** `del large_array`

`del large_array`

7. **Line 7:** `gc.collect()`

`gc.collect()`

8. **Line 8:** `# Base64 encoding doubles size temporarily`

A comment: *Base64 encoding doubles size temporarily.*

9. **Line 9:** `wav_b64 = base64.b64encode(samples.tobytes()) # 2x memory`

Assigns `base64.b64encode(samples.tobytes()) # 2x memory` to `wav_b64`.

10. **Line 10:** `# Decode immediately after receiving`

A comment: *Decode immediately after receiving.*

11. **Line 11:** `samples = np.frombuffer(base64.b64decode(wav_b64), dtype=np.int16)`

Creates an array from a binary buffer. Used for decoding raw audio data.

70.4 Thread Optimization

```
# Daemon threads die with the main process (no zombie processes)
t = threading.Thread(target=work, daemon=True)

# Lock granularity – don't hold locks longer than needed
with self._lock:
    # Minimal critical section
    self._proc.stdin.write(data)

# Use time.sleep(0.1) in poll loops to avoid CPU spinning
while self._pause_event.is_set():
    time.sleep(0.1) # Don't burn CPU while paused
```

Explain Line by Line Code Description (Block 4)

- Line 1:** `# Daemon threads die with the main process (no zombie processes)`
A comment: *Daemon threads die with the main process (no zombie processes).*
- Line 2:** `t = threading.Thread(target=work, daemon=True)`
Creates a background thread.
- Line 3:** `# Lock granularity – don't hold locks longer than needed`
A comment: *Lock granularity – don't hold locks longer than needed.*
- Line 4:** `with self._lock:`
Context manager: `self._lock`. Auto-manages cleanup.
- Line 5:** `# Minimal critical section`
A comment: *Minimal critical section.*
- Line 6:** `self._proc.stdin.write(data)`
`self._proc.stdin.write(data)`
- Line 7:** `# Use time.sleep(0.1) in poll loops to avoid CPU spinning`
A comment: *Use time.sleep(0.1) in poll loops to avoid CPU spinning.*
- Line 8:** `while self._pause_event.is_set():`
While loop.
- Line 9:** `time.sleep(0.1) # Don't burn CPU while paused`
While loop.

70.5 Key Takeaways

- **Lazy loading** defers expensive imports until needed
 - **Engine caching** prevents reloading 50-500 MB models
 - **int16 dtype** is the standard for audio (2 bytes per sample)
 - **Daemon threads** prevent zombie processes
 - **Lock granularity** — hold locks for minimal time
-

Security Considerations

Learning Objectives

- Understand security practices in AIVS
 - See how addons are sandboxed
 - Understand the HTTP server security model
-

71.1 Input Validation

```
# Filename sanitization prevents path traversal
def sanitize_filename(name: str, max_length: int = 80) -> str:
    name = re.sub(r'[<>:"/\|?*]', '_', name) # Remove dangerous chars
    name = re.sub(r'\s+', ' ', name).strip()
    if len(name) > max_length:
        name = name[:max_length].rstrip()
    return name

# JSON protocol validation
try:
    req = json.loads(line)
except json.JSONDecodeError:
    respond({"ok": False, "error": "Invalid request"})
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Filename sanitization prevents path traversal`
A comment: *Filename sanitization prevents path traversal.*
2. **Line 2:** `def sanitize_filename(name: str, max_length: int = 80) -> str:`
Defines a function.
3. **Line 3:** `name = re.sub(r'[<>:"/\|]?*', '_', name) # Remove dangerous chars`
Assigns `re.sub(r'[<>:"/\|]?*', '_', name)` to `name`.
4. **Line 4:** `name = re.sub(r'\s+', ' ', name).strip()`
Assigns `re.sub(r'\s+', ' ', name).strip()` to `name`.
5. **Line 5:** `if len(name) > max_length:`
Returns the number of items in a sequence (string, list, dict, tuple).
6. **Line 6:** `name = name[:max_length].rstrip()`
Assigns `name[:max_length].rstrip()` to `name`.
7. **Line 7:** `return name`
Returns a value from a function.
8. **Line 8:** `# JSON protocol validation`
A comment: *JSON protocol validation.*
9. **Line 9:** `try:`
Begins exception handling.
10. **Line 10:** `req = json.loads(line)`
Deserializes JSON from a file into a Python object.
11. **Line 11:** `except json.JSONDecodeError:`
Catches `json` exceptions.
12. **Line 12:** `respond({"ok": False, "error": "Invalid request"})`
Calls function `respond()`.

71.2 Addon Sandboxing

```
# Addons are isolated:
# 1. They live in their own directory
# 2. They install deps into the managed venv (not main venv)
# 3. Version compatibility is checked
if not manifest.is_compatible:
    log.warning("Addon requires app version {manifest.min_app_version}")
    continue # Skip incompatible addons
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** `# Addons are isolated:`
A comment: *Addons are isolated:*.
2. **Line 2:** `# 1. They live in their own directory`
A comment: *1. They live in their own directory.*
3. **Line 3:** `# 2. They install deps into the managed venv (not main venv)`
A comment: *2. They install deps into the managed venv (not main venv).*
4. **Line 4:** `# 3. Version compatibility is checked`
A comment: *3. Version compatibility is checked.*
5. **Line 5:** `if not manifest.is_compatible:`
Conditional check.
6. **Line 6:** `log.warning("Addon requires app version {manifest.min_app_version}")`
Logs a warning. Something unexpected happened but the program continues.
7. **Line 7:** `continue # Skip incompatible addons`
Skips to the next iteration.

71.3 HTTP Server Security

```
# OmniVoice Server supports:
# - Bearer token authentication
# - CORS origin restrictions
# - Localhost-only binding (default)

# Default: localhost only
DEFAULT_HOST = "127.0.0.1"

# API key authentication
if self._api_key:
    req.add_header("Authorization", f"Bearer {self._api_key}")
```

Explain Line by Line Code Description (Block 3)

- Line 1:** `# OmniVoice Server supports:`
A comment: *OmniVoice Server supports:*.
- Line 2:** `# - Bearer token authentication`
A comment: *- Bearer token authentication.*
- Line 3:** `# - CORS origin restrictions`
A comment: *- CORS origin restrictions.*
- Line 4:** `# - Localhost-only binding (default)`
A comment: *- Localhost-only binding (default).*
- Line 5:** `# Default: localhost only`
A comment: *Default: localhost only.*
- Line 6:** `DEFAULT_HOST = "127.0.0.1"`
Assigns `"127.0.0.1"` to `DEFAULT_HOST`.
- Line 7:** `# API key authentication`
A comment: *API key authentication.*
- Line 8:** `if self._api_key:`
Conditional check.
- Line 9:** `req.add_header("Authorization", f"Bearer {self._api_key}")`
`req.add_header("Authorization", f"Bearer {self._api_key}")`

71.4 Key Takeaways

- **Filename sanitization** prevents path traversal attacks
 - **Addons** are sandboxed in their own directories with version checks
 - The HTTP server defaults to **localhost-only** binding
 - **Bearer token auth** protects the server when exposed on the network
 - **JSON validation** prevents injection through the IPC protocol
-

[← Ch 70](#) | [Ch 72 — Roadmap](#) →

Future Directions & Roadmap

i Info: Status note (2026.3.1). Several items on the original roadmap have shipped and are now documented in [Part X](#): multiple voice-clone engines with per-engine Python environments, GPU support for every engine that supports it, a per-category compute choice (CPU 80–95 % / full GPU), FFmpeg-free audio decoding, DAISY 3 output, playlist generation and out-of-order recording for long books. Read the items below as the remaining ground rather than as an untouched list.

Learning Objectives

- Understand potential improvements to AIVS
- Explore advanced extension possibilities
- Plan your own contributions

72.1 Potential Improvements

Area	Improvement	Difficulty
GUI	Waveform visualization	Medium
TTS	Batch synthesis (multiple segments)	Easy
Audio	Background music mixing	Hard
DAISY	DAISY 3.0 support	Hard
Accessibility	Voice commands	Medium
Platform	macOS/Linux support	Hard
Testing	End-to-end GUI tests	Medium
Packaging	Flatpak/AppImage for Linux	Medium

72.2 Extension Ideas

- **Subtitle generator** — Generate SRT/VTT subtitles from audio
 - **Chapter marker editor** — Visual timeline for chapter editing
 - **Batch export** — Export all segments as individual files
 - **Cloud TTS** — Integrate Azure/Google/AWS TTS APIs
 - **Plugin marketplace** — Community-contributed addons
 - **Multi-language UI** — Internationalization (i18n)
 - **Audio editor** — Trim, merge, adjust volume per segment
 - **Metadata editor** — Edit DAISY book metadata visually
-

72.3 What You've Learned

If you've read this entire book, you now understand:

- **Python fundamentals** through advanced topics (72 chapters!)
 - **wxPython GUI development** with accessibility support
 - **Audio processing** with NumPy and sherpa-onnx
 - **Multi-engine TTS** with subprocess isolation
 - **Document parsing** (PDF, DOCX, HTML, Markdown)
 - **Project architecture** for a production desktop app
 - **Testing** with 99+ tests across 12 test files
 - **Packaging** with PyInstaller and Inno Setup
 - **Thread safety** and GUI event communication
 - **Addon systems** for extensibility
-

72.4 Final Advice

"The best way to learn is by building. Take what you've learned here and create something of your own. Start small, iterate often, and don't be afraid to break things — that's how you learn."

The AI Voice Studio project is open source. Clone it, modify it, extend it. Every line of code in this book comes from a real, working application. You now have the knowledge to understand every decision, every pattern, and every line.

Go build something amazing.

Playlist Builder

Learning Objectives

- Generate standard playlist files that modern players understand (M3U8, PLS, WPL)
- Keep playlists portable by referencing audio through sibling-relative file names only
- Escape XML attributes correctly with `xml.sax.saxutils.escape` so WPL files never break
- Wire playlist generation into the recording lifecycle without touching the audio pipeline

73.1 Why a Playlist Builder?

An *Audio files with playlist* project records each segment of a document into its own audio file. After a long recording session you may have fifty files named `01 page 1.wav`, `02 page 2.wav` and so on. Every modern player can open a folder of audio files, but none of them know the *order* in which the pages should be played. The `documents/playlist_builder.py` module solves exactly this problem: after recording completes it writes three standard playlist files next to the audio, and every player (VLC, PotPlayer, GOM, Windows Media Player, Winamp, mpv) understands at least one of them:

File	Format	Players
<code><project>.m3u8</code>	Extended M3U, UTF-8	VLC, PotPlayer, GOM, mpv
<code><project>.pls</code>	Winamp playlist (INI style)	VLC, GOM, PotPlayer, Winamp
<code><project>.wpl</code>	Windows Media Playlist (SMIL/XML)	Windows Media Player, PotPlayer, GOM

i Info: Portability rule: every entry references the audio by *file name only* — never an absolute path. The playlists keep working when the project folder is copied, moved or zipped, because the player resolves a bare name against the folder the playlist itself lives in.

73.2 Module Header and Imports

```
"""Playlist file generation for the ``audio_playlist`` project type.
```

```
After recording completes, three standard playlist files are written next to
the audio files in the project folder, so the narration opens in modern
media players (VLC, PotPlayer, GOM, Windows Media Player, Winamp, ...).
"""
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `"""Playlist file generation ..."""`

The module docstring. It documents which project type this module serves (`audio_playlist`) and names the three playlist formats it writes. Because this is a plain-text string at the top of the file, Python stores it in the module's `__doc__` attribute, and tools like `help()` display it automatically.

```
from __future__ import annotations

import os
from typing import Any, Dict, List
from xml.sax.saxutils import escape as _xml_escape
```


Explain Line by Line Code Description (Block 2)

1. **Line 1:** `from __future__ import annotations`

Postpones evaluation of type annotations (PEP 563). Annotations such as `List[Dict[str, Any]]` are then stored as plain strings instead of being evaluated at import time, which slightly speeds up module import and lets newer annotation syntax run on older interpreters.

2. **Line 3:** `import os`

Imports the operating-system module, used here for `os.path.basename` (strip the folder from a path, keep the file name) and `os.path.join` (build a portable path from a folder and a file name).

3. **Line 4:** `from typing import Any, Dict, List`

Imports the generic typing helpers used in the function signatures. `Any` means "any type at all", `Dict[str, Any]` describes the segment dictionaries loaded from `project.json`, and `List` wraps them into sequence types.

4. **Line 5:** `from xml.sax.saxutils import escape as _xml_escape`

Imports the standard-library XML escaper and renames it with a leading underscore (`_xml_escape`). The escape function turns the five XML-sensitive characters (`&` `<` `>` and quotes) into entities such as `&`. The alias keeps the rest of the file readable and marks the helper as module-private.

73.3 The XML Attribute Helper

```
def _quoted(value: str) -> str:
    """XML attribute value, always wrapped in double quotes."""
    return '%s' % _xml_escape(value).replace("'", '"')
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** `def _quoted(value: str) -> str:`

Defines a private helper that returns `value` as a safely quoted XML attribute value. The WPL writer uses it for every `src` and `title` attribute it emits.

2. **Line 2:** `"""XML attribute value, always wrapped in double quotes."""`

The docstring records the contract: the result is always wrapped in *double* quotes, which is what the Windows Media Player playlist parser expects.

3. **Line 3:** `return '%s' % _xml_escape(value).replace('"', '')`

Three steps in one expression. First `_xml_escape(value)` replaces `&`, `<` and `>` with entities. Then `.replace('"', '')` escapes double quotes — necessary because `escape()` deliberately leaves quotes alone (it escapes text nodes, not attributes). Finally the `%` operator wraps the result in literal double quotes. Without the quote escaping, a segment titled `He said "hello"` would produce a WPL file that Windows Media Player refuses to open.

 **Warning: Common pitfall:** `xml.sax.saxutils.escape()` only escapes the characters that are illegal in *text*. Attributes are delimited by quotes, so a quote inside an attribute value must also be escaped (`"`). This one-line helper is the difference between a WPL file that opens everywhere and one that only works for titles without quotes.

73.4 Selecting and Ordering Entries

```
def _ready_entries(segments: List[Dict[str, Any]]) -> List[Dict[str, Any]]:
    """Recorded segments (`status == "done"` with a saved file), in order."""
    entries = [
        s for s in segments
        if s.get("status") == "done" and s.get("saved")
    ]
    entries.sort(key=lambda s: s.get("index", 0))
    return entries
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `def _ready_entries(segments):`

Defines the filter that decides which segments appear in the playlists. The function takes the raw `segments` list loaded from `project.json` and returns only the ones that were actually recorded.

2. **Line 2:** `"""Recorded segments ... in order."""`

The docstring states both the filter condition and the guarantee: results come back sorted by segment index, i.e. in the same order the audio was recorded.

3. **Lines 3-5:** `entries = [s for s in segments if s.get("status") == "done" and s.get("saved")]`

A list comprehension that keeps a segment only when both conditions hold. `status == "done"` means the TTS engine finished that segment, and a truthy `s.get("saved")` means an audio file name was recorded for it. Segments that are still `pending` (or done but somehow without a saved file) are excluded, so a half-finished recording produces a playlist covering only the audio that really exists. `s.get(...)` is used instead of `s["status"]` so a malformed segment dict cannot raise `KeyError` — it simply fails both checks and is skipped.

4. **Line 6:** `entries.sort(key=lambda s: s.get("index", 0))`

Sorts the filtered list in place by the segment's `index` field. `sort(key=...)` calls the lambda once per element and orders by the returned number. The sort is stable, so segments with equal indices keep their original relative order. This matters because `project.json` may list segments in any order after edits and re-recording.

5. **Line 7:** `return entries`

Returns the ordered list to the caller.

```
def _titles(entries: List[Dict[str, Any]]) -> List[str]:
    return [str(s.get("title") or os.path.basename(s["saved"])) for s in entries]
```

```
def _files(entries: List[Dict[str, Any]]) -> List[str]:
    return [os.path.basename(str(s["saved"])) for s in entries]
```

Explain Line by Line Code Description (Block 5)

- Line 1:** `def _titles(entries):`
Defines the helper that extracts the display title for every playlist entry.
- Line 2:** `return [str(s.get("title") or os.path.basename(s["saved"])) for s in entries]`
A comprehension with a fallback chain. `s.get("title")` returns the segment title, or `None` if the project file has none. The `or` operator then falls back to `os.path.basename(s["saved"])` — the audio file's own name such as `01 page 1.wav`. `str(...)` guarantees the final value is a string even if the title field held a number. Players display these titles next to each track.
- Line 4:** `def _files(entries):`
Defines the helper that extracts the audio file names.
- Line 5:** `return [os.path.basename(str(s["saved"])) for s in entries]`
For every entry, takes the saved path and strips everything up to the last separator with `os.path.basename`. `str(...)` is defensive: even if a project file stored the path as something else, the playlist receives a plain string. The result is a bare file name like `02 page 2.wav`, which is exactly the portable reference the module promises.

73.5 The Three Writers

M3U8 (extended M3U, UTF-8)

```
def _write_m3u8(path: str, title: str, names: List[str], titles: List[str]) -> None:
    lines = ["#EXTM3U", f"#PLAYLIST:{title}"]
    for name, entry_title in zip(names, titles):
        lines.append(f"#EXTINF:-1,{entry_title}")
        lines.append(name)
    with open(path, "w", encoding="utf-8", newline="\n") as fh:
        fh.write("\n".join(lines) + "\n")
```

Explain Line by Line Code Description (Block 6)

1. **Line 1:** `def _write_m3u8(path, title, names, titles):`

Defines the writer for the extended M3U format. `path` is the output file, `title` the playlist name shown by players, `names` the audio file names and `titles` the matching display titles (same order).

2. **Line 2:** `lines = ["#EXTM3U", f"#PLAYLIST:{title}"]`

Starts the file with the two mandatory header lines. `#EXTM3U` tells the player "this is the extended M3U format, not a plain one". `#PLAYLIST:{title}` is an f-string that names the whole playlist; VLC shows this name in its title bar, PotPlayer in its playlist header.

3. **Line 3:** `for name, entry_title in zip(names, titles):`

Iterates over both lists in parallel: `zip(names, titles)` yields `(name, title)` pairs, one per track, stopping at the shorter list.

4. **Line 4:** `lines.append(f"#EXTINF:-1,{entry_title}")`

Appends the extended-info line for one track. The format is `#EXTINF:<duration in seconds>,<artist>-<title>`. The value `-1` means "duration unknown" — the player reads the real length from the audio file itself when it loads the track. Everything after the comma is the display title.

5. **Line 5:** `lines.append(name)`

Appends the track reference itself — a bare file name such as `01 page 1.wav`. Non-comment lines in M3U are always the media locations, resolved relative to the playlist file's folder.

6. **Line 6:** `with open(path, "w", encoding="utf-8", newline="\n") as fh:`

Opens the output file for writing. `encoding="utf-8"` is what makes this an *M3U8* file (the "8" stands for UTF-8) and lets non-ASCII segment titles survive. `newline="\n"` overrides the Windows default so the file ends every line with a bare LF — the line ending players expect — regardless of the host platform.

7. **Line 7:** `fh.write("\n".join(lines) + "\n")`

Joins all collected lines with newlines into one string and writes it, adding one final newline so the file is well-formed. A single `write` call is also faster than hundreds of small writes.

PLS (Winamp style)

```
def _write_pls(path: str, title: str, names: List[str], titles: List[str]) -> None:
    lines = ["[playlist]"]
    for i, (name, entry_title) in enumerate(zip(names, titles), start=1):
        lines.append(f"File{i}={name}")
        lines.append(f"Title{i}={entry_title}")
        lines.append(f"Length{i}=-1")
    lines.append(f"NumberOfEntries={len(names)}")
    lines.append("Version=2")
    with open(path, "w", encoding="utf-8", newline="\n") as fh:
        fh.write("\n".join(lines) + "\n")
```

Explain Line by Line Code Description (Block 7)

- Line 1:** `def _write_pls(path, title, names, titles):`
Defines the writer for the PLS (Winamp) playlist format, an INI-style key=value text format.
- Line 2:** `lines = ["[playlist]"]`
Starts the file with the mandatory `[playlist]` section header. Every PLS key must live inside this section.
- Line 3:** `for i, (name, entry_title) in enumerate(zip(names, titles), start=1):`
Loops over the parallel name/title pairs with a counter. `enumerate(..., start=1)` numbers the entries from 1, because the PLS format requires 1-based keys (`File1`, `File2`, ...) and a missing `File1` makes some parsers reject the whole file.
- Line 4:** `lines.append(f"File{i}={name}")`
Writes the track reference key. `File1=01 page 1.wav` means "the first track is this file". As in M3U, a bare name is resolved relative to the playlist's folder.
- Line 5:** `lines.append(f"Title{i}={entry_title}")`
Writes the display title for the same entry number. The number must match the `File` key exactly or players will pair the wrong title with the wrong file.
- Line 6:** `lines.append(f"Length{i}=-1")`
Writes the track length in seconds; `-1` again means "unknown, read it from the file".
- Lines 7-8:** `lines.append(f"NumberOfEntries={len(names)}")` and `lines.append("Version=2")`
Closes the section with the two mandatory footer keys. `NumberOfEntries` tells parsers how many tracks to expect (so a truncated file is detected), and `Version=2` identifies the PLS format revision every current player expects.
- Lines 9-10:** `with open(...) as fh: fh.write(...)`
Writes the joined lines with UTF-8 encoding and LF line endings — identical I/O pattern to the M3U8 writer, for the same reasons.

WPL (Windows Media Player, SMIL)

```
def _write_wpl(path: str, title: str, names: List[str]) -> None:
    body = "".join(
        f'        \n' for name in names
    )
    wpl = (
        '\n'
        '\n'
        ' \n'
        ' \n'
        f' \n'
        ' \n'
        ' \n'
        ' \n'
        ' \n'
        + body +
        ' \n'
        ' \n'
        '\n'
    )
    with open(path, "w", encoding="utf-8", newline="\n") as fh:
        fh.write(wpl)
```

Explain Line by Line Code Description (Block 8)

1. **Line 1:** `def _write_wpl(path, title, names):`

Defines the writer for the Windows Media Playlist format. Note it does not take the `titles` list: WPL has no per-track title attribute — Windows Media Player reads the media's own metadata instead.

2. **Lines 2-3:** `body = "".join(f' \n' for name in names)`

Builds the XML body with a generator expression joined into one string. For every audio file it emits one `<media src="..."/>` element — the SMIL way of saying "play this file in sequence". The `src` value passes through `_quoted()`, so quotes and XML-special characters inside a file name can never break the document. The six leading spaces indent each element to match the surrounding structure.

3. **Lines 4-16:** `wpl = ('' ... )`

Assembles the full SMIL document as one concatenated string. `<?wpl version="1.0"?>` is the processing instruction Windows Media Player uses to recognise the file type. The `<meta name="Generator">` element records which application produced the playlist. The `<seq>` (sequence) element means its children play one after another in document order — exactly the reading order of the recorded document.

4. **Lines 17-18:** `with open(...) as fh: fh.write(wpl)`

Writes the finished XML with UTF-8 encoding and LF line endings. The file is now a valid WPL playlist that Windows Media Player, PotPlayer and GOM all open.

73.6 The Public Entry Point

```
def build_playlists(
    project_dir: str,
    project_name: str,
    segments: List[Dict[str, Any]],
) -> List[str]:
    """Write the M3U8/PLS/WPL playlists into ``project_dir``.

    Returns the list of written file paths; empty when no segment has been
    recorded yet. Existing playlist files are overwritten so they stay in
    sync with the project's segments (e.g. after re-recording).
    """

    entries = _ready_entries(segments)
    if not entries:
        return []
    safe = "".join(
        c if c not in '\\/:*?"<>|' else "_" for c in project_name
    ).strip() or "playlist"
    names = _files(entries)
    titles = _titles(entries)

    written: List[str] = []
    m3u8_path = os.path.join(project_dir, safe + ".m3u8")
    _write_m3u8(m3u8_path, project_name, names, titles)
    written.append(m3u8_path)

    pls_path = os.path.join(project_dir, safe + ".pls")
    _write_pls(pls_path, project_name, names, titles)
    written.append(pls_path)

    wpl_path = os.path.join(project_dir, safe + ".wpl")
    _write_wpl(wpl_path, project_name, names)
    written.append(wpl_path)
    return written
```


Explain Line by Line Code Description (Block 9)

1. **Lines 1-4:** `def build_playlists(project_dir, project_name, segments):`

The one public function of the module, called by the Recording window after recording finishes. It takes the project folder (where the audio already lives), the human project name (which becomes the playlist file base name), and the full segment list from `project.json`. It returns the list of files it wrote — the caller turns this into the "playlists were generated" note in the completion dialog.

2. **Lines 5-6:** `entries = _ready_entries(segments) / if not entries: return []`

Filters and orders the recorded segments. If nothing was recorded the function returns an empty list immediately and no playlist files are created — there is nothing to list.

3. **Lines 7-9:**

`safe = "".join(c if c not in '\\/:*?"<>|' else "_" for c in project_name).strip() or "playlist"`

Turns the project name into a safe file-name base. The comprehension keeps every character except the nine characters Windows forbids in file names (`\ / : * ? " < > |`); each forbidden one becomes an underscore. `.strip()` removes leading/trailing spaces, and the `or "playlist"` fallback covers a name that was entirely punctuation — a project named `???` must not produce a file literally named `playlist.m3u8`'s base as an empty string.

4. **Lines 10-11:** `names = _files(entries) / titles = _titles(entries)`

Extracts the two parallel lists the writers need: bare audio file names and their display titles, both in segment order.

5. **Lines 13-21:** the three `os.path.join(project_dir, safe + ".ext")` blocks

Writes each playlist into the project folder next to the audio. Each writer call is followed by `written.append(...)`, so by the end the list holds the three absolute paths (`.m3u8`, `.pls`, `.wpl`) in that order. Opening a writer with `"w"` truncates any existing file, which is exactly the desired "regenerate on every recording" behaviour: after re-recording one segment, the playlists are rebuilt with the new content in the correct order.

73.7 How the Recording Window Uses It

The wiring lives in `gui/recording_dialog.py`. When the synthesis worker reports that all segments are done, the dialog runs:

```
def _build_playlists_if_needed(self):
    self._playlist_files = []
    ptype = self.data.get("project_type", "audio_playlist")
    if ptype != "audio_playlist":
        return
    try:
        fresh = project.load_project(self.project_dir)
        from ..documents.playlist_builder import build_playlists
        self._playlist_files = build_playlists(
            self.project_dir,
            fresh.get("name", "playlist"),
            fresh.get("segments", []),
        )
    except Exception as exc:
        log.warning("Playlist generation failed: %s", exc)
```

Explain Line by Line Code Description (Block 10)

1. **Line 2:** `self._playlist_files = []`
Resets the result list on every call, so a DAISY project (which returns early) never leaves a stale value behind.
2. **Line 3:** `ptype = self.data.get("project_type", "audio_playlist")`
Reads the project type with a default, then the guard on the next line makes playlist generation exclusive to *Audio files with playlist* projects — DAISY books get their own structure instead.
3. **Line 5:** `fresh = project.load_project(self.project_dir)`
Reloads `project.json` from disk instead of trusting the copy the dialog loaded when it opened. This is essential: the synthesis worker runs in a background thread and has been marking segments `done` in the file while the dialog sat idle — the in-memory copy still shows them as pending, which would produce an empty playlist.
4. **Line 6:** `from ..documents.playlist_builder import build_playlists`
A local (function-level) import. Importing lazily keeps the recording window's startup fast and avoids a hard module dependency at import time.
5. **Lines 7-10:** `self._playlist_files = build_playlists(...)`
Calls the builder with the freshly loaded name and segments and stores the three written paths for the completion summary.
6. **Lines 11-12:** `except Exception as exc: log.warning(...)`
Catches any failure (a read-only folder, a full disk) and logs a warning instead of crashing. Playlist generation is a convenience on top of a finished recording — the recording itself is already safely on disk, so the user must not lose the completion message because a playlist could not be written.

73.8 The Test Suite

`tests/test_playlist_builder.py` pins the format details that players rely on. Two representative assertions:

```
def test_wpl_escapes_xml(self):
    ...
    self.assertIn("'", text)

def test_returns_empty_when_nothing_recorded(self):
    with tempfile.TemporaryDirectory() as tmp:
        self.assertEqual(
            build_playlists(tmp, "Book", [{"index": 1, "title": "A",
                                           "status": "pending", "saved": None}]),
            [],
        )
        self.assertEqual(os.listdir(tmp), [])
```

Explain Line by Line Code Description (Block 11)

1. **Lines 1-4:** `def test_wpl_escapes_xml(self): ... assertIn("'", text)`

Guards the exact escaping contract found in section 73.3: a saved file name containing a double quote and an ampersand must come out as `"` and `&` inside the WPL `src` attribute. This test exists because the original implementation escaped `&` but not quotes — a regression this test now prevents forever.

2. **Lines 6-12:** `def test_returns_empty_when_nothing_recorded(self): ...`

Verifies the empty-recording contract from section 73.6: when every segment is still `pending`, `build_playlists` returns `[]` and the temporary project directory stays completely empty — no half-written playlist files, no stray `.m3u8` with zero entries.

73.9 Key Takeaways

- Three playlist formats cover all modern players: **M3U8** (VLC/PotPlayer/GOM), **PLS** (Winamp family), **WPL** (Windows Media Player)
- Entries are **file names only** — playlists survive moving, copying or zipping the project folder
- `duration = -1 / #EXTINF:-1` tells players to read the real length from each audio file
- XML attributes need `"` escaping too — `escape()` alone is not enough for WPL
- Generation runs **after** recording and reads **fresh** project state from disk, so it is a pure post-processing step that never touches the audio pipeline

DAISY 3 Builder (Z39.86-2005)

Learning Objectives

- Understand the DAISY 3 fileset: DTBook text, NCX navigation, SMIL synchronization
 - Generate a conforming DTBook 2005-3 document with frontmatter and images
 - Synchronize audio to text at phrase level using proportional time slicing
 - Embed source-document images as `imggroup` elements and OPF manifest entries
-

74.1 What DAISY 3 Adds Over DAISY 2.02

DAISY 2.02 stores its text in small HTML files; DAISY 3 (the Z39.86-2005 standard) replaces that with one **DTBook** XML document for the whole book, an **NCX** navigation file, and SMIL 2.0 documents that bind every paragraph to its place in the audio. The builder in `documents/daisy3_builder.py` (added after this book's first edition) produces a complete fileset in `<project>/DAISY3/`:

```
book.xml      -- DTBook 2005-3 text content (frontmatter, body, rearmatter)
ncx.xml       -- NCX navigation (navMap built from the heading hierarchy)
package.opf   -- OPF package manifest tying everything together
0001.smil ... -- one SMIL 2.0 document per recorded segment
aud0001.wav .. -- recorded audio renamed to digit-only names
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `book.xml -- DTBook 2005-3 text content`

The whole book's text in a single DTBook document: mandatory `frontmatter` (title, author), the `bodymatter` with all chapters, and a `rearmatter`. DAISY 3 players open this file's companion package and render the text while highlighting the spoken paragraph.

2. **Line 2:** `ncx.xml -- NCX navigation`

The navigation control file. Its `navMap` mirrors the document's heading hierarchy, so a reader can jump chapter by chapter (or heading by heading) exactly as in the printed book's table of contents.

3. **Line 3:** `package.opf -- OPF package manifest`

The entry point a DAISY 3 player opens first. It lists every file in the book with its MIME type and links the DTBook, NCX, SMILs and audio together.

4. **Line 4:** `0001.smil ... -- one SMIL per segment`

Each recorded segment gets one SMIL 2.0 document containing a `<par>` per paragraph: the text reference and the `<audio>` element with exact clip begin/end times.

5. **Line 5:** `aud0001.wav -- digit-only names`

Recorded audio is copied under zero-padded digit-only names. Segment titles can contain any characters; players only ever see `aud0001.wav`, `aud0002.mp3`, which keeps every URI in the book trivially safe.

74.2 The Entry Point: `build_daisy3_book()`

```
def build_daisy3_book(
    output_dir: str,
    project_name: str,
    segments: List[Dict[str, Any]],
    audio_format: str = "wav",
    language: str = "en",
    publisher: str = "",
    source_file: str = "",
    author: str = "",
    meta: Optional[Dict[str, str]] = None,
) -> str:
    ready = [s for s in segments if s.get("status") == "done" and s.get("saved")]
    if not ready:
        return ""

    daisy_dir = os.path.join(output_dir, DAISY3_OUTPUT_DIR_NAME)
    os.makedirs(daisy_dir, exist_ok=True)

    book_uid = "urn:uuid:" + str(uuid.uuid4())
    now_iso = datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")
```

Explain Line by Line Code Description (Block 2)

1. **Lines 1–10:** the signature

The parameters deliberately mirror `build_daisy_book()` from the DAISY 2.02 builder, so the recording dialog can call either one with the same argument list. `output_dir` is the project folder, `segments` the fresh segment list from `project.json`, and `meta` the book information entered in the wizard (title, creator, date, subject, narrator, producer). The return value is the path of the generated `package.opf` — the file players open — or an empty string when there was nothing to build.

2. **Line 11:** `ready = [s for s in segments if s.get("status") == "done" and s.get("saved")]`

The same filter the playlist builder uses: only segments that were fully recorded (status `done` with a saved file) become part of the book. A partially recorded project produces a book covering exactly the audio that exists.

3. **Line 12:** `if not ready: return ""`

No recorded audio means no book: the function returns an empty string and the caller skips its "book generated" message.

4. **Lines 14–15:** `daisy_dir = os.path.join(output_dir, DAISY3_OUTPUT_DIR_NAME) /
os.makedirs(daisy_dir, exist_ok=True)`

Creates the `DAISY3` folder inside the project folder. `exist_ok=True` makes the call idempotent — rebuilding the book after re-recording a segment simply reuses the existing folder.

5. **Line 17:** `book_uid = "urn:uuid:" + str(uuid.uuid4())`

Generates a fresh unique identifier for the book. DAISY metadata requires a `dc:Identifier` in URN form; a random UUID guarantees two exports of different books never collide in a player's library.

6. **Line 18:** `now_iso = datetime.now(timezone.utc).strftime(...)`

Timestamps the build in UTC with the ISO-8601 format (`%Y-%m-%dT%H:%M:%SZ`) that DAISY metadata fields demand. UTC avoids ambiguous local-time offsets in the book metadata.

i Info: MP3 normalization on staging: while copying each recorded segment into the book as `aud0001.wav` and friends, the builder calls the same `_stage_audio` helper as the DAISY 2.02 builder. An MP3 saved by a TTS engine is often MPEG-2 LSF at 22–24 kHz, which several DAISY players truncate ("audio cuts out"). If `ffmpeg.needs_mp3_normalization()` detects a rate below 44.1 kHz, the file is re-encoded to MPEG-1 Layer III, 44.1 kHz mono before entering the book. See Chapter 59.

74.3 Building the DTBook

The DTBook writer emits the official public identifier and required structure:

```
_DTBOOK_DOCTYPE = (
    ''
)
```

Explain Line by Line Code Description (Block 3)

1. Lines 1-3: `_DTBOOK_DOCTYPE = ('<!DOCTYPE dtbook PUBLIC ...')`

The official Document Type Declaration for DTBook 2005-3. Strict DAISY 3 players (FSReader, EasyReader, AMIS) validate the document against this DTD; writing the exact formal public identifier and system URL means the book conforms rather than merely parses. The string is split across two literals that Python concatenates, keeping the source line short.

Every segment's text becomes a chapter with one phrase per paragraph:

```
def _entry_blocks(entry: Dict[str, Any], id_counter: List[int]) -> List[Dict[str, Any]]:
    blocks: List[Dict[str, Any]] = []

    def add(dtbook_id: str, text: str) -> None:
        id_counter[0] += 1
        blocks.append({"id": dtbook_id, "text": text})

    for para in entry["paragraphs"]:
        add(f"dtb{para['index']}", para["text"])
    return blocks
```

Explain Line by Line Code Description (Block 4)

1. Line 1: `def _entry_blocks(entry, id_counter):`

Converts one recorded segment into the DTBook paragraph records. `entry` holds the segment's text broken into paragraphs, and `id_counter` is a one-element list used as a mutable counter (a closure cannot rebind a plain `int` of the enclosing scope, but it can mutate a list).

2. Lines 3-5: `def add(dtbook_id, text): ... blocks.append(...)`

A local helper closing over `blocks` and `id_counter`. Each call bumps the shared counter and appends one record with a DTBook element id (`dtb1`, `dtb2`, ...). These ids are what the SMIL files reference later, so they must be unique across the whole book.

3. Lines 7-8: `for para in entry["paragraphs"]: add(f"dtb{para['index']}", para["text"])`

One paragraph of the segment becomes one DTBook block, and one block later becomes one SMIL `<par>` — this 1:1 mapping is what gives the book phrase-level synchronization: while a paragraph is spoken, the player highlights exactly that paragraph's text.

74.4 Embedding Images

DAISY 3 is the first project type in AIVS that carries images. The builder extracts images from the source document and appends them as `imggroup` elements:

```
_IMAGE_EXT_MIME = {
    ".png": "image/png",
    ".jpg": "image/jpeg",
    ".jpeg": "image/jpeg",
    ".gif": "image/gif",
    ".svg": "image/svg+xml",
    ".webp": "image/webp",
    ".bmp": "image/bmp",
    ".tif": "image/tiff",
    ".tiff": "image/tiff",
}

_IMG_TAG_RE = re.compile(
    r"]*?\bsrc\s*=\s*['\"]([^\"]+)[\"]", re.IGNORECASE
)
```

Explain Line by Line Code Description (Block 5)

1. **Lines 1–11:** `_IMAGE_EXT_MIME = {...}`

Maps the image file extensions the builder understands to their official MIME types. The OPF manifest requires the correct MIME type for every resource, and the `imggroup` writer uses the same table when choosing the file extension for extracted media.

2. **Lines 13–14:** `_IMG_TAG_RE = re.compile(r"<img\b[^>]*?\bsrc\s*=\s*['\"]([^\"]+)[\"]", re.IGNORECASE)`

The regular expression that pulls image references out of HTML/EPUB content. Reading it piece by piece: `<img\b` matches the tag opening (`\b` prevents matching `<imginput` -style words); `[^>]*?` lazily skips any attributes before `src`; `\bsrc\s*=\s*` matches the `src` attribute with optional spaces around the equals sign; `['\"]([^\"]+)[\"]` captures the value in either single or double quotes; and `re.IGNORECASE` accepts `<IMG SRC=...>` as produced by some exporters. Everything the parser finds here is downloaded/extracted into the book and referenced from the DTBook.

74.5 NCX Navigation and SMIL Synchronization

The NCX nests navPoints by heading level, and each SMIL binds text to audio with `dtb:` attributes that players require:

```
_SMIL_DOCTYPE = (
    ,
)

_SMIL_DTB_NS = "http://www.daisy.org/z3986/2005/dtbook/"
```


Explain Line by Line Code Description (Block 6)

1. **Lines 1-3:** `_SMIL_DOCTYPE = ('<!DOCTYPE smil PUBLIC ...')`

The official SMIL 2.0 DTD declaration for DAISY 3 content documents. Like the DTBook doctype it makes the files conformant, not just well-formed.

2. **Line 5:** `_SMIL_DTB_NS = "http://www.daisy.org/z3986/2005/dtbook/"`

The namespace URI bound to the `dtb:` prefix. DAISY 3 SMIL files carry required custom attributes in this namespace — `dtb:requiredNamespace` on the root, `dtb:totalElapsed` / `dtb:timeInThisSmil` for timings, and `customTest` on each `<par>` — without which strict players refuse to play the book.

⚠ Warning: Conformance detail learned the hard way: the `<audio>` element must sit *directly inside* each `<par>` — wrapping it in a `<seq>` (which SMIL itself allows) makes several real DAISY 3 players skip the audio entirely. The builder therefore writes `<par><text .../><audio .../></par>` and nothing between them.

Audio timing is phrase-level and proportional. For a segment whose audio lasts 60 seconds and whose three paragraphs have 100, 200 and 300 characters, the SMIL assigns 10 s, 20 s and 30 s clip windows respectively. The helper converting stored values:

```
def _total_ms(total_ms_value: Any) -> int:
    try:
        return max(0, int(float(total_ms_value)))
    except (TypeError, ValueError):
        return 0
```

Explain Line by Line Code Description (Block 7)

1. **Line 1:** `def _total_ms(total_ms_value):`

Coerces a duration value of unknown type (JSON may hand back an `int`, `float`, `str` or `None`) into a non-negative millisecond integer for the SMIL clock calculations.

2. **Line 2:** `try:`

Begins the guarded conversion — `float("abc")` or `float(None)` would raise, and a bad stored value must never abort a whole book build.

3. **Line 3:** `return max(0, int(float(total_ms_value)))`

Two-step conversion: `float(...)` accepts strings like `"1234.5"`, then `int(...)` truncates to whole milliseconds; `max(0, ...)` clamps negative junk to zero so a clock value can never go backwards.

4. **Lines 4-5:** `except (TypeError, ValueError): return 0`

Both realistic failure modes are caught and mapped to 0 ms: `TypeError` for `None` /objects, `ValueError` for non-numeric strings. Zero keeps the SMIL timeline valid even when one paragraph's duration is missing.

74.6 The OPF Package and ZIP Export

`_write_opf()` emits the manifest that ties the book together, and `export_daisy3_zip()` packages the whole DAISY3 folder into a single ZIP for distribution (same UI as the DAISY 2.02 export button in the Recording window).

```
def export_daisy3_zip(project_dir: str, zip_path: str,
                      callback: Optional[Callable[[str], None]] = None) -> None:
    src = os.path.join(project_dir, DAISY3_OUTPUT_DIR_NAME)
    with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:
        for root, _dirs, files in os.walk(src):
            for name in files:
                full = os.path.join(root, name)
                rel = os.path.relpath(full, src)
                zf.write(full, rel)
                if callback:
                    callback(rel)
```

Explain Line by Line Code Description (Block 8)

1. **Lines 1-2:** `def export_daisy3_zip(project_dir, zip_path, callback=None):`
Packages the finished `DAISY3` folder into one ZIP file. The optional `callback` receives each archived file name, which lets the GUI show progress while a large book is being zipped.
2. **Line 3:** `src = os.path.join(project_dir, DAISY3_OUTPUT_DIR_NAME)`
Locates the book folder inside the project — the same constant (`DAISY3`) the builder used when creating it.
3. **Line 4:** `with zipfile.ZipFile(zip_path, "w", zipfile.ZIP_DEFLATED) as zf:`
Opens the target ZIP for writing with DEFLATE compression. The `with` block guarantees the archive's central directory is finalized (properly closed) even if an error occurs mid-zip.
4. **Lines 5-9:** the `os.walk` loop
Visits every file under the book folder. `os.path.relpath(full, src)` converts each absolute path into a path relative to the book root, so the ZIP unpacks as a clean DAISY 3 fileset with no drive letters or parent folders — exactly what a player expects to find. The callback fires after each file so a progress dialog can tick.

74.7 Choosing DAISY 3 in the Wizard

The New Project wizard offers **DAISY audio and text with images book (3)** as a project type. Choosing it adds a DAISY page where the book information (title, author, narrator, publisher...) and the chapter-splitting mode (heading style 1 only / break on every heading) are collected; the Recording window then builds the DAISY 3 fileset automatically when recording completes, and the **Export DAISY as ZIP** button appears.

74.8 Key Takeaways

- DAISY 3 = **DTBook** (one XML text document) + **NCX** (navigation) + **SMIL 2.0** (sync) + audio
- Official DTDs and `dtb:` attributes are mandatory for strict players
- `<audio>` goes **directly inside** `<par>` — no wrapping `<seq>`
- Phrase-level sync: paragraphs get proportional clip windows of the segment audio
- Images from DOCX/EPUB/HTML are embedded as `imggroup` and listed in the OPF manifest
- MP3s below 44.1 kHz are normalized on staging so DAISY players decode the full audio

Reusable Dialogs & Progress

i Info: Current implementation (2026.3.1). Three dialogs joined this group and are worth reading after the patterns below: `gui/accept_dialog.py` (the first-launch terms of use — all check boxes required, the bold final box ticking the six conditions, Escape swallowed at the key-hook level, closing counted as “no”, acceptance keyed to `TERMS_VERSION` ; [Chapter 81](#)), the *Start Selected Recording* picker in `gui/main_frame.py` (two radio groups where the first one relabels and refills a combo; [Chapter 80](#)), and the per-engine install/tuning dialogs in `gui/clone_engines_panel.py` / `gui/voicelab_options_dialog.py` . `gui/progress.py` is used for every long download and pip install, and pip failures are surfaced with the engine's own error text rather than an exit code.

Learning Objectives

- Build labelled-button dialogs that screen readers announce properly (vs. `wx.MessageBox`)
- Route engine errors through pattern matching to offer a server restart
- Write one progress dialog that serves both background threads and synchronous work
- Understand pulse animation with `wx.Timer` and forced repaints with `wx.Window.Update`

75.1 Why Not `wx.MessageBox` ?

The stock message box only offers standard buttons ("OK", "Yes", "No"). Two places in AIVS need *two meaningful buttons* instead:

- recording finished → **Open project folder** / **OK**
- an OmniVoice Server failure → **Restart OmniVoice server now** / **I will restart later**

The module `gui/dialogs.py` provides one small dialog class plus helpers, so both flows stay consistent — and every button is a real `wx.Button` with a real accessible name, which NVDA and JAWS announce reliably (unlike owner-drawn or generic message-box buttons).

75.2 The Action Dialog

```
def run_action_dialog(
    parent: wx.Window,
    title: str,
    message: str,
    actions: list[tuple[str, str]],
    default_key: str | None = None,
) -> str | None:
    dlg = _ActionDialog(parent, title, message, actions, default_key)
    try:
        if dlg.ShowModal() == wx.ID_OK:
            return dlg.selected_key
        return None
    finally:
        dlg.Destroy()
```

Explain Line by Line Code Description (Block 1)

1. **Lines 1-6:** the signature

The public helper wraps the private dialog class. `actions` is a list of `(key, label)` pairs — `key` is what the caller receives, `label` is what the user sees and the screen reader announces. `default_key` names the button that receives initial focus (Enter activates it), so the *safe* choice is always the Enter choice. The return value is the chosen key, or `None` when the dialog was dismissed with Escape or the close box.

2. **Line 7:** `dlg = _ActionDialog(parent, title, message, actions, default_key)`

Creates (but does not yet show) the modal dialog. Construction builds all controls and sizers; showing happens in `ShowModal()`.

3. **Lines 8-11:** `try: ... finally: dlg.Destroy()`

Shows the dialog modally and translates the result: `wx.ID_OK` means a button was pressed and the chosen key is returned; anything else (Escape, close box) maps to `None`. The `finally` block is essential wx hygiene — `Destroy()` releases the native window resources on *every* path, including exceptions, preventing window-handle leaks.

```

class _ActionDialog(wx.Dialog):
    def __init__(self, parent, title, message, actions, default_key=None):
        super().__init__(parent, title=title)
        self.selected_key: str | None = None
        self._keys: dict[int, str] = {}

        sizer = wx.BoxSizer(wx.VERTICAL)
        text = wx.StaticText(self, label=message)
        text.SetName(title)
        text.Wrap(520)
        sizer.Add(text, 0, wx.EXPAND | wx.ALL, 12)

        btn_row = wx.BoxSizer(wx.HORIZONTAL)
        btn_row.AddStretchSpacer(1)
        default_btn = None
        for key, label in actions:
            btn = wx.Button(self, label=label)
            btn.SetName(label)
            btn.SetToolTip(label)
            self._keys[btn.GetId()] = key
            if default_key is not None and key == default_key:
                default_btn = btn
            btn_row.Add(btn, 0, wx.ALL, 6)
            btn.Bind(wx.EVT_BUTTON, self._on_action)
        sizer.Add(btn_row, 0, wx.EXPAND | wx.LEFT | wx.RIGHT | wx.BOTTOM, 6)

        self.SetSizer(sizer)
        self.Fit()
        self.SetMinSize(self.GetSize())
        if parent is not None and parent.IsShown():
            self.CentreOnParent()
        else:
            self.Centre()
        target = default_btn or (btn_row.GetChildren()[0].GetWindow()
                                if btn_row.GetChildren() else None)
        if target is not None:
            wx.CallAfter(target.SetFocus)

```

Explain Line by Line Code Description (Block 2)

1. Lines 1-4: class header and initial state

The dialog keeps two pieces of state: `selected_key` (which button the user chose; `None` until one is pressed) and `_keys`, a dictionary mapping wx control ids to the caller's action keys. When a button event arrives, the id is looked up in this dictionary — this is how one event handler serves any number of buttons.

2. Lines 6-10: the message text

A vertical sizer holds a `StaticText` with the message. `text.SetName(title)` gives the label a real MSAA accessible name, and `text.Wrap(520)` breaks long messages at word boundaries around 520 pixels so the dialog never grows wider than the screen. `sizer.Add(text, 0, wx.EXPAND | wx.ALL, 12)` adds it with a 12-pixel border on all sides.

3. Lines 12-21: the button row

A horizontal sizer holds the action buttons, right-aligned after a stretch spacer. For every `(key, label)` pair a real `wx.Button` is created; `SetName(label)` makes its accessible name equal to its visible label (screen readers announce exactly the label), and `self._keys[btn.GetId()] = key` registers the mapping the event handler needs. When the pair matches `default_key`, that button is remembered as the default focus target.

4. Lines 23-28: layout and centering

`Fit()` sizes the dialog to its contents and `SetMinSize(self.GetSize())` stops the user shrinking it below that. Centering prefers the parent window (`CentreOnParent`) so the dialog appears near where the user is looking, falling back to screen center when the parent is hidden.

5. Lines 29-31: `target = default_btn or ... / wx.CallAfter(target.SetFocus)`

Chooses the button to focus: the default action if one was named, otherwise the first button. `wx.CallAfter` defers the focus call until after the dialog finishes its modal show cycle — setting focus during `__init__` would be overwritten by wx's own focus handling, so the call is queued for the next event-loop pass. Focus on the safe default means Enter triggers the conservative action ("I will restart later", "OK").

75.3 The "Open project folder" and Server-Restart Helpers

```
def show_recording_complete(parent, message, project_folder):
    key = run_action_dialog(
        parent, "Recording complete", message,
        [("open", "Open project folder"), ("ok", "OK")],
        default_key="ok",
    )
    if key == "open":
        open_folder(project_folder)

_SERVER_HINTS = (
    "omnivoice server",
    "server is not responding",
    "server exited prematurely",
    ...
)

def looks_like_server_error(message: str) -> bool:
    low = (message or "").lower()
    if any(hint in low for hint in _REQUEST_LEVEL_HINTS):
        return False
    return any(hint in low for hint in _SERVER_HINTS)
```

Explain Line by Line Code Description (Block 3)

1. **Lines 1-9:** `show_recording_complete(...)`

The recording-finished dialog. Two buttons are offered with `"ok"` as the default; when the user picks `"open"`, `open_folder(project_folder)` launches the OS file manager (via `os.startfile` on Windows, a `file://` URL as fallback). Any other outcome (OK, Escape) simply closes.

2. **Lines 11-17:** `_SERVER_HINTS = (...)`

A tuple of lowercase substrings that indicate the OmniVoice HTTP server itself failed (connection refused, exited prematurely, "try restarting it from settings", ...). Pattern matching on error text is deliberately simple: the engine layer raises many exception types, but the user-facing decision only needs "is this a server problem or not".

3. **Lines 19-23:** `def looks_like_server_error(message): ...`

The classifier. First the message is lowercased so matching is case-insensitive. Then the subtle part:

`_REQUEST_LEVEL_HINTS` (messages like "server returned HTTP 500: ...") are checked first and *veto* the classification — if the server answered with its own HTTP error, it is demonstrably alive and offering a restart would only confuse. Only when no veto matched do the `_SERVER_HINTS` decide. This two-tier check is why the restart dialog appears for dead-server errors but not for synthesis failures the server already reported.

75.4 One Progress Dialog, Two Usage Patterns

`gui/progress.py` serves two callers with opposite needs:

- **Background threads** (recording, server restart): the worker pushes updates through `wx.CallAfter` ; the normal event loop repaints.
- **Synchronous work** (document parsing in the wizard): the UI thread is busy inside a long call, so the dialog must repaint itself with `wx.Window.Update()` .

```
class TaskProgressDialog(wx.Dialog):
    def __init__(self, parent, title="Working", message="Working...",
                  cancel_label=None, on_cancel=None, style=None):
        if style is None:
            style = wx.CAPTION | wx.RESIZE_BORDER
        super().__init__(parent, title=title, style=style)
        self._on_cancel = on_cancel

        sizer = wx.BoxSizer(wx.VERTICAL)
        self._message = wx.StaticText(self, label=message)
        self._message.SetName("Progress message")
        self._message.Wrap(430)
        sizer.Add(self._message, 0, wx.ALL, 10)

        self.gauge = wx.Gauge(self, range=100, size=(420, 22))
        self.gauge.SetName("Progress")
        self.gauge.SetToolTip("Shows how much of the work has finished")
        sizer.Add(self.gauge, 0, wx.EXPAND | wx.LEFT | wx.RIGHT | wx.BOTTOM, 10)
```

Explain Line by Line Code Description (Block 4)

1. **Lines 1-5:** the signature and style fallback

The constructor accepts the dialog texts, an optional cancel button (label + callback) and an optional style. When no style is given it uses `wx.CAPTION | wx.RESIZE_BORDER` — deliberately *without* `wx.CLOSE_BOX` : the dialog's lifetime is controlled by the calling code (via `finish()`), not by the user clicking a close box mid-operation.

2. **Lines 6-10:** the message label

A `StaticText` with a fixed accessible name ("Progress message") that later gains the current text (see `set_message` below), and `Wrap(430)` so long status lines break cleanly. Screen readers announce the label whenever it changes because the name changes with it.

3. **Lines 12-15:** the gauge

A `wx.Gauge` with range 100 — values are always expressed as percentages. It gets its own accessible name and a tooltip. `range=100` also matches the pulse animation below, which walks the gauge between 0 and 95.

```

def start_pulse(self, interval_ms: int = 150) -> None:
    self._pulse_value = 0
    self._pulse_timer = wx.Timer(self)
    self.Bind(wx.EVT_TIMER, self._on_pulse_tick, self._pulse_timer)
    self._pulse_timer.Start(interval_ms)

def _on_pulse_tick(self, _):
    self._pulse_value = (self._pulse_value + 4) % 96
    self.show_progress(self._pulse_value)

def show_progress(self, value: int) -> None:
    self.gauge.SetValue(max(0, min(100, int(value))))
    self.gauge.Update()

def finish(self) -> None:
    self.stop_pulse()
    try:
        self.Destroy()
    except Exception:
        pass

```

Explain Line by Line Code Description (Block 5)

1. Lines 1-5: `start_pulse(interval_ms=150)`

Starts indeterminate ("busy") animation for work whose length is unknown — a server restart, a recording that depends on document size. A `wx.Timer` owned by the dialog fires every 150 ms on the UI thread (timers are the only safe way for GUI state to change periodically without a second thread touching widgets).

2. Lines 7-8: `_on_pulse_tick`

Each tick advances the stored pulse value by 4 and wraps with modulo 96, so the gauge sweeps 0→95→0 forever without ever reaching 100 (which would falsely signal completion). It then routes through `show_progress`, gaining the forced repaint for free.

3. Lines 10-12: `show_progress(value)`

Sets the gauge, clamped into 0–100 by `max(0, min(100, ...))` so a misbehaving worker cannot crash the dialog. The crucial detail is `self.gauge.Update()` — unlike `Refresh`, `Update()` repaints *immediately*, synchronously. This is what makes the bar move even while the UI thread is blocked inside a long parsing call and no event-loop paint pass will ever run.

4. Lines 14-18: `finish()`

The single, safe way to close the dialog: stop the pulse timer (a timer firing after destroy would raise), then `Destroy()` wrapped in a guard because two callers (worker thread via `CallAfter` and the UI handler) may both call `finish` — the second call must be a harmless no-op.

75.5 Where These Are Used

Call site	Helper	Buttons
Recording complete (Recording window)	<code>show_recording_complete</code>	Open project folder / OK
OmniVoice Server error (anywhere)	<code>notify_engine_error</code>	Restart server now / I will restart later
Recording progress (Recording window)	<code>TaskProgressDialog</code> + cancel → Stop	Stop recording
Server restart (Settings / error dialog)	<code>restart_omni_server</code> + <code>TaskProgressDialog</code>	(none — pulsing)
Document parsing (wizard)	<code>TaskProgressDialog.show_progress</code>	(none — synchronous)

75.6 Key Takeaways

- Real `wx.Button`s with `SetName(label)` are what screen readers announce — prefer them over `wx.MessageBox` when buttons carry meaning
- Map control ids → action keys in a dict so one handler serves any number of buttons
- Classify errors in two tiers: request-level messages veto, then server hints decide
- `wx.Window.Update()` repaints synchronously — the only way a progress bar can move while the UI thread is busy
- Guard `Destroy()` ; timers must stop before the window dies; `wx.CallAfter` sequences focus and cross-thread updates

Voice Lab: Building a Voice-Cloning Engine Subsystem

Learning Objectives

- Design a plugin registry that describes engines as data instead of code
- Turn a pip-installed model package into a catalog-shaped voice entry
- Drive a long-lived worker subprocess over a JSON-lines protocol
- Plug one engine wrapper into an existing engine cache without touching call sites
- Separate *pure* request-building code (easy to test) from the process code around it

76.1 Why a New Subsystem Was Needed

The first version of the Voice Clone page drove a single engine through one dedicated client class. Every new clone engine then promised the same four things — *install me*, *list my voices*, *load my model once*, *speak text* — but with completely different Python packages, different package names and different numbers of built-in voices. Writing one client per engine would have multiplied the same bugs by three.

The current design keeps the shape of the app (`tts.engine.get_engine` still returns something with a `synthesize()` method) and puts all the differences into **data**:

Concern	Where it lives now	Why
Which engines exist, what they install, which voices they ship	<code>voicelab/engines.py</code>	Pure data — importable without PyTorch, no GUI, testable
Per-engine tuning knobs (steps, temperature, seed...)	<code>voicelab/options.py</code>	One shared vocabulary, validated in one place
Actually running an engine	<code>voicelab/worker.py</code> (standalone script)	PyTorch must never be imported into the GUI process
Talking to the worker	<code>voicelab/__init__.py</code> (<code>CloneWorker</code>)	One IPC implementation for every engine
App-facing adapter	<code>VoicelabEngine</code>	Implements the existing <code>TtsEngine.synthesize</code> contract

i Info: The three engines in the current build are **Pocket TTS** (Kyutai, `pocket-tts`), **Bark** (Suno, via `transformers`) and **F5-TTS** (`f5-tts`). All three run on the CPU and can additionally run on an NVIDIA GPU — see [Chapter 78](#). An earlier fourth engine (NeuTTS) was removed from the project: it required a Hugging Face login, which no part of this application should depend on.

76.2 Engines as Data: the Registry

Every engine is one dictionary. The fields are deliberately small in number, because each one has exactly one consumer:

[illegible]

Explain Line by Line Code Description (Block 1)

- Lines 1-3:** `id`, `name`, `engine`
`id` is the key used everywhere else (settings keys, voice entries, worker requests). `name` is only ever displayed. `engine` is the string the voice entry carries so that `tts.engine.get_engine` can find the right wrapper; keeping it equal to `id` is a convention, not a requirement.
- Line 4:** `"package": "pocket-tts"`
The *single* pip distribution whose presence means “this engine is installed”. Chapter 77 shows how this one string decides both “Install engine...” availability and which virtualenv the worker is started with.
- Line 5:** `"packages": (...)`
Everything handed to `pip install`. Plain names only — deliberately no version specifiers, because the same tuple is used by “Remove engine” and `pip uninstall` cannot accept requirements like `name<1.0`.
- Line 6:** `"license"`
Shown in the install dialog and matched by the licence table in the packaged `THIRD-PARTY-LICENSES.html` (Chapter 82). F5-TTS is the important case: the *code* is MIT while the *weights* are CC-BY-NC-4.0, which forbids commercial use.
- Lines 7-8:** `homepage`, `size_hint`
Documentation the user can act on before downloading gigabytes. `size_hint` is a string on purpose: it is an estimate for humans, not a number any code branches on.
- Line 9:** `"devices": "cpu+gpu"`
Read by `gpu_supported()` with the substring test `"gpu" in devices`. Every Voice Lab engine is `cpu+gpu`: the CPU always works, the GPU is an *addition* — never a replacement.
- Line 10:** `"voice_cloning": True`
Lets the UI show the “clone your own voice” affordance only for engines that can actually do it. A flag in data beats an `isinstance` check in the GUI.
- Lines 11-12:** `note`
A paragraph from the registry straight into a label — the user reads the engine’s own description instead of a paraphrase maintained in the GUI.
- Lines 13-14:** `languages`
Built by the helper `_languages()` in the same catalog shape the JSON model catalog uses (`code` → `name` → `variants` → `voices`), so panels that already render catalog data render Voice Lab engines with no new code.

76.3 Voices That Come From the Package

Sherpa-onnx engines download *model artifacts* and then expose voices from the downloaded files. Voice Lab engines are different: their voices are supplied by the installed Python package, so they cannot live in the JSON catalog. Bark alone would need 130 rows (twelve languages × ten speakers plus the lightweight English checkpoint), and F5-TTS points at reference files *inside* its own wheel.

The solution is a factory that produces entries in exactly the shape `ModelStore.installed_voices` returns:

```
def _entry(engine_id, language, variant, voice, voice_name, **extra):
    entry = {
        "tts": engine_id,
        "tts_name": engine_name(engine_id),
        "language": language,
        "variant": variant,
        "voice": voice,
        "voice_name": voice_name,
        "sid": 0,
        "engine": engine_id,
        "dir": "",
        "builtin_voice": True,
        "requires_package": probe_package(engine_id),
    }
    entry.update(extra)
    return entry
```

Explain Line by Line Code Description (Block 2)

1. **Line 1:** the signature

Positional arguments describe the voice; `**extra` carries engine-specific additions (Bark's `repo`, F5-TTS's `ref_resource`). A variadic tail keeps the factory generic while still allowing one engine to add fields.

2. **Lines 2-4:** `tts`, `tts_name`, `engine`

Three keys that mean almost the same thing, each consumed by a different layer: `tts` is what the Download panel groups by, `tts_name` is the label, `engine` is what `get_engine` switches on. Reconciliation happens once, here, instead of in five panels.

3. **Lines 5-8:** `language`, `variant`, `voice`, `voice_name`

The same four-part voice identity used by the JSON catalog, so a voice entry is interchangeable no matter where it came from.

4. **Line 9:** `"sid": 0`

Speaker id for engines that address voices numerically. PyTorch engines address voices by *name* (Bark's `v2/en_speaker_3`), so the numeric id stays a harmless default rather than a special case in the renderer.

5. **Line 10:** `"dir": ""`

Empty on purpose: these voices have no model directory on disk. Panels that display `dir` get an empty string instead of a path that does not exist — the absence is *stated*, not implied.

6. **Line 11:** `"builtin_voice": True`

How the UI knows this voice cannot be downloaded or removed; it is a property of the engine's package.

7. **Line 12:** `"requires_package"`

Usually the same field name the catalog uses for “this engine needs a pip package in the shared environment”. Reusing it means the existing installed/not-installed logic applies unchanged — the panel that greys out unavailable voices needs no Voice Lab branch.

8. **Lines 13-14:** `entry.update(extra)`

Applied last so an engine may override a default if it must, and so callers that pass nothing get exactly the generic entry.

The Bark generator is worth reading because it shows how much UI code a data factory removes:


```

if engine_id == "bark":
    voices = []
    for code, label in _BARK_LANGUAGES:
        for speaker in range(10):
            voices.append(_entry(
                "bark", code, "bark_v2", f"v2/{code}_speaker_{speaker}",
                f"{label} speaker {speaker + 1} (of 10)",
            ))
    for speaker in range(10):
        voices.append(_entry(
            "bark", "en", "bark_small", f"v2/en_speaker_{speaker}",
            f"Bark small - English speaker {speaker + 1} (of 10)",
            repo="suno/bark-small",
        ))
    return voices

```

Explain Line by Line Code Description (Block 3)

1. **Lines 2-3:** the nested loops

Twelve languages × ten speakers = 120 preset voices, generated instead of typed. When Bark adds a language, one tuple entry adds ten voices.

2. **Line 5:** `f"v2/{code}_speaker_{speaker}"`

Bark's speaker id format. The `v2/` prefix is part of the identifier Bark itself expects, so it is built here rather than patched in later.

3. **Line 6:** the human label

"English speaker 3 (of 10)" tells the user how many options exist, which changes how they listen. Generating labels from data keeps them consistent for all 120 voices.

4. **Lines 8-13:** the small-checkpoint voices

`suno/bark-small` is English-only but much lighter on a CPU, so it is offered as a separate *variant* of English with `repo` set on the entry. The worker later prioritises `repo` over the default checkpoint, which is how the variant choice reaches the model loader.

 **Tip:** Voice lists like this are the reason `count_builtin_voices()` exists as a public helper: the GUI shows "26 voices", and a test can assert that number without re-implementing the loop.

76.4 Driving an Engine in a Subprocess

PyTorch and friends are heavy, slow to import and occasionally crash the process. None of that belongs in a GUI process that also holds the user's recordings. The application therefore runs `voicelab/worker.py` as a separate interpreter process and talks to it over pipes:

```
{ "cmd": "ping" }
{ "cmd": "synthesize", "engine": "bark", "device": "cuda", "text": "...",
  "threads": 15, "voice": "v2/en_speaker_3", "repo": "suno/bark",
  "options": { "temperature": 0.7 } }
```

One line of request in, one line of JSON out. The worker keeps the model loaded between requests, so recording 200 segments pays the load cost once. `CloneWorker._ensure_proc` starts the process and immediately sends `ping` — a worker that cannot even import its engine answers with an error *before* the first paragraph of a book is involved.

```
class CloneWorker:
    def __init__(self, engine_id, device=DEVICE_CPU, timeout=1800.0):
        self.engine_id = engine_id
        self.device = resolve_device(device)
        self._proc = None
        self._lock = threading.RLock()
        self._timeout = timeout
        self._next_id = 0
```

Explain Line by Line Code Description (Block 4)

1. Line 1: `class CloneWorker`

One instance per (engine, device) pair. The class owns exactly one process; the module-level `worker_for()` cache gives callers a shared instance so two panels do not start two copies of a multi-gigabyte model.

2. Lines 2-3: the constructor's first statements

`resolve_device` is called *once, here*: “auto” or “gpu” becomes a concrete `cpu` / `cuda` at construction time, so every later request describes what is actually happening rather than what the user asked for.

3. Line 4: `self._proc = None`

The process is created lazily on first use. Constructing a worker is free, which is what lets the GUI build voice lists — and tests create workers — without spawning interpreters.

4. Line 5: `self._lock = threading.RLock()`

A re-entrant lock, because the recording thread writes a request and then reads a response; `RLock` means a helper called from inside the locked section cannot deadlock itself. It is the only synchronisation the class needs, since a process pipe is otherwise usable from several threads.

5. Line 6: `self._timeout = 1800.0`

Thirty minutes. A CPU Bark run on a long paragraph is slow but healthy; a shorter timeout would kill work that is still progressing. A timeout exists at all so a wedged process cannot freeze the recording forever.

6. Line 7: `self._next_id = 0`

Request counter reserved for pairing requests with responses. JSON-lines is strictly sequential today; the id makes a future interleaved protocol possible without re-designing the class.

76.5 Building the Request (Pure Code, Easy Tests)

The hardest part of any IPC layer to test is the plumbing. The trick used here is to move every decision *out* of the plumbing: `build_synthesize_request()` takes plain values and returns a plain dictionary, so a unit test can assert the exact JSON the worker would receive with no process and no wx involved.

```
def build_synthesize_request(*, engine_id, device, text,
                             voice_entry=None, speed=1.0, tuning=None):
    entry = voice_entry or {}
    resolved = resolve_device(device)
    request = {
        "cmd": "synthesize",
        "engine": engine_id,
        "device": resolved,
        "text": text,
        "speed": float(speed),
        "threads": cpu_thread_count(),
    }
    for key in _VOICE_KEYS:
        value = entry.get(key)
        if value:
            request[key] = value
    if engine_id == "bark" and not request.get("repo"):
        request["repo"] = ("suno/bark-small"
                           if entry.get("variant") == "bark_small" else "suno/bark")
    overrides = _options.clean(engine_id, tuning)
    if not overrides:
        overrides = _options.clean(engine_id, entry.get("options"))
    if overrides:
        request["options"] = overrides
    return request
```

Explain Line by Line Code Description (Block 5)

1. **Lines 1-2:** keyword-only parameters (`*`)

The leading `*` forbids positional arguments. With six similar strings in play (“engine”, “device”, “text”), a positional call is a silent bug; keyword-only forces every call site to name what it passes.

2. **Line 3:** `entry = voice_entry or {}`

Still works when no voice entry is supplied (a quick preview may have none), and never mutates the caller’s dictionary.

3. **Line 4:** `resolved = resolve_device(device)`

“auto” is turned into a concrete device *before* the request is built, so the JSON on the wire is always truthful and the worker never needs to know what “auto” means.

4. **Lines 5-12:** the base request

Six keys every request carries. `threads` comes from `cpu_thread_count()`, which delegates to `compute.cpu_threads()` — the 80–95 % band of Chapter 78. The worker does not decide how much CPU to use; the application does, and the worker applies it.

5. **Lines 13-16:** copying the voice keys

Only non-empty values are copied, from an explicit allow-list (`_VOICE_KEYS`). An allow-list beats “send the whole entry” for two reasons: the request cannot leak UI-only fields into the engine, and the set of fields the worker must understand stays small enough to document.

6. **Lines 17-20:** the Bark checkpoint rule

Bark downloads one of two repositories. The *variant* (“Bark small”) is a voice-list concept the engine does not know, so it is translated here into the repository the worker passes to `transformers`. Note the `not request.get("repo")` guard: an entry that already set `repo` (as `_entry` does for bark-small voices) wins, so both paths agree.

7. **Lines 21-24:** tuning overrides

Two carriers are supported and cleaned by the same validator: the function argument (Settings defaults applied by the panel) and the voice entry (per-project overrides stored in the project). Empty results are omitted entirely, so an untouched project sends no `options` key at all and the engine uses its own defaults.

76.6 Plugging Into the Existing Engine Cache

The rest of the application knows only one thing about a speech engine: it can be asked to turn text into int16 samples. `VoicelabEngine` implements that contract and nothing more.

```

class VoicelabEngine:
    def __init__(self, voice_entry, device=DEVICE_CPU):
        engine_id = voice_entry.get("engine")
        if not engines.is_engine(engine_id):
            raise VoicelabError(f"'{engine_id}' is not a Voice Lab engine.")
        self.voice_entry = voice_entry
        self.engine_id = engine_id
        self.tuning = _options.clean(engine_id, voice_entry.get("options"))
        self.device = resolve_device(
            voice_entry.get("device") or device or DEVICE_CPU)
        self.sample_rate = _DEFAULT_SAMPLE_RATE
        self._worker = worker_for(engine_id, self.device)

    def synthesize(self, text, sid=0, speed=1.0, pitch=1.0, volume=1.0):
        if not text.strip():
            raise ValueError("Nothing to synthesize")
        request = build_synthesize_request(
            engine_id=self.engine_id, device=self.device, text=text,
            voice_entry=self.voice_entry, speed=speed, tuning=self.tuning)
        samples, sample_rate, report = self._worker.synthesize(request)
        self._report_unused_options(report)
        self._report_notes(report)
        if sample_rate:
            self.sample_rate = sample_rate
        if pitch != 1.0:
            samples = _shift_pitch(samples, pitch)
        if volume != 1.0:
            samples = _apply_volume(samples, volume)
        return samples

```

Explain Line by Line Code Description (Block 6)

1. **Lines 2-5:** the guard clause

A voice entry from the wrong source (say, a sherpa-onnx voice) fails immediately with a clear message instead of building a request no worker can serve. `VoicelabError` is a dedicated exception type so callers can catch engine problems without catching everything.

2. **Lines 8-9:** `self.tuning`

The voice entry's stored overrides are validated once at construction. Validation is not repeated per segment, and a malformed value can never reach the worker.

3. **Lines 10-12:** device resolution order

The voice entry wins, then the constructor argument, then CPU. This ordering is what makes “this project records on the GPU” survive from the Recording window all the way to the process.

4. **Line 13:** `worker_for(...)`

Returns the shared worker for that (engine, device). Two engines, or the same engine on two devices, still get separate processes — a CUDA process cannot change its device mid-run.

5. **Lines 16-17:** the empty-text guard

A blank segment (a page containing only a heading marker, say) would otherwise cost a model run that produces silence. Failing fast keeps the synthesizer's error handling in charge.

6. **Lines 18-20:** the request

All the intelligence lives in the pure builder — this method only decides *which* parameters come from the instance and which from the call.

7. **Lines 21-23:** the response triple

`samples`, the engine's real `sample_rate` and a `report` dictionary. The report is not an error channel: it carries “these options were skipped” and human notes such as “audio was decoded with soundfile because this machine has no FFmpeg”.

8. **Lines 24-27:** adapting the sample rate

Each engine speaks at its own rate (24 kHz here). Updating `self.sample_rate` from the response means the writer downstream creates WAV files with the correct header even if an engine changes its rate between versions.

9. **Lines 28-32:** pitch and volume

Pitch shifting and volume are applied by the *application*, on the returned samples, using the same helpers as the ONNX engines. One implementation means the Audio Effects slider behaves identically no matter which engine spoke, and an engine that cannot do pitch shifting does not need a new parameter.

⚠ Warning: Keep heavy imports out of the GUI. Nothing in `voicelab/__init__.py` imports `torch`. Voice lists, device detection and request building all work with plain Python + NumPy — which is also why the whole subsystem is unit-testable on a machine with no GPU and no models.

76.7 Mini Project: A Voice List From a Registry

Build a tiny script that turns the registry into a report — the same shape of work the Settings pages do:

```
from ai_voice_studio.voicelab import engines

for engine_id in engines.engine_ids():
    info = engines.engine(engine_id)
    print(f"{info['name']:<34} {engines.count_built_in_voices(engine_id):>4} voices")
    print(f"    installs : {' ', ' '.join(engines.packages(engine_id))}")
    print(f"    license   : {info['license']}")
    print(f"    gpu       : {engines.gpu_supported(engine_id)}")
```

Output on the current build (trimmed):

```
Pocket TTS (Kyutai)                26 voices
    installs : pocket-tts, soundfile, scipy
    license   : Apache-2.0 (code); per-voice licenses
    gpu       : True
Bark (Suno AI, transformer)        130 voices
    installs : transformers, torch, soundfile, scipy
    license   : MIT
    gpu       : True
F5-TTS (transformer)                2 voices
    installs : f5-tts, torch, torchaudio, soundfile
    license   : MIT (code); CC-BY-NC-4.0 (weights)
    gpu       : True
```

Exercise:

1. Add a fourth engine entry to `ENGINES` — a fake one is fine (`id="demo"` , `packages=("demo-tts",)`). Which other dictionaries in `engines.py` must gain an entry before the worker could load it, and why do the two lists exist separately?
2. Write a unit test that asserts every voice of every installed engine has a non-empty `voice` , `voice_name` and `language` , then run it. (Hint: no `wx`, no `torch` — iterate `all_built_in_voices()` .)
3. Change `build_synthesize_request` in a scratch copy so it always sends `options` , even when empty, and explain from the worker's side why “no key” is the better protocol.
4. Trace what happens to `threads` end to end: from `cpu_thread_count()` here to `_limit_cpu` in the worker. Which layer would you change to give a single engine only half the CPU?

76.8 Key Takeaways

- Describe engines as **data**; then the GUI, the installer and the worker all read one source of truth
- Generate catalog-shaped voice entries so existing panels need no special cases
- Never import a heavy ML framework into the GUI process — run it in a worker subprocess

- Keep request building *pure*: no processes, no wx, completely unit-testable
- An adapter that satisfies the existing interface is cheaper than teaching every call site about a new engine type
- Send “how much of the machine to use” from the application, and let the worker apply it

One Python Environment per Engine

Learning Objectives

- Create and manage a virtualenv per engine, from inside a frozen application
- Map an engine id to an environment folder deterministically
- Answer “is this package installed?” three ways — cached, asynchronous and straight from disk
- Plan a CUDA-aware pip install, and understand why the wheel index matters
- Verify an installation by importing the engine, and explain the failures that check catches

77.1 The Problem: One Shared Environment Cannot Hold Two Engines

TTS engines are ordinary Python packages that depend on machine-learning frameworks. Their requirements conflict constantly:

- two engines need two different `torch` releases;
- one engine pins `transformers<4.40` while another needs `4.46+`;
- one engine ships a `torchao`-style sub-dependency whose module was *renamed* in a later release, so a newer version breaks the engine at import time even though pip considers the install valid.

This is exactly the failure mode that made the project move from one shared environment to **one environment per engine**. The layout is:

```
%APPDATA%\AIVoiceStudio\
├─ addon_env\           shared environment (addons, Developer tab)
└─ tts_envs\
  ├─ pocket_tts\       Pocket TTS only
  ├─ bark\             Bark + transformers + torch
  ├─ f5tts\            F5-TTS + torch + torchaudio
  └─ omnivoice\        OmniVoice + OmniVoice Server (shared on purpose)
```

i Info: Engines that genuinely want the same packages may share an environment. In this project the two OmniVoice front-ends (the direct runner and the HTTP server) both need the same `omnivoice` base package and the same CUDA PyTorch, so a second environment would only duplicate gigabytes of wheels. That exception is written down as data (`SHARED_ENVIRONMENTS`) rather than hidden in a condition somewhere in the GUI.

77.2 Mapping an Engine to Its Environment

```
def _safe_env_name(engine_id: str | None) -> str:
    name = "".join(
        char if char.isalnum() or char in "_-" else "_"
        for char in (engine_id or "").strip().lower()
    )
    return name or "default"

def environment_id(engine_id: str | None) -> str:
    name = _safe_env_name(engine_id)
    return SHARED_ENVIRONMENTS.get(name, name)

def engine_env_dir(engine_id: str) -> str:
    return os.path.join(
        paths.user_data_dir(), _TTS_ENVS_DIR, environment_id(engine_id)
    )
```

Explain Line by Line Code Description (Block 1)

1. **Lines 1-6:** `_safe_env_name`

Sanitises an id into a safe folder name: every character that is not alphanumeric, `-` or `_` becomes `_`, and the result is lower-cased. This matters because the folder name is built from data that arrives from the registry, a settings file or a test — and a stray character would otherwise create a path that Windows refuses. The `or "default"` tail guarantees a non-empty name even for `None`.

2. **Lines 9-11:** `environment_id`

The single place where “which environment does engine X use?” is answered. Two OmniVoice ids collapse onto one folder here, which means *every* caller — installer, uninstaller, probe cache, worker starter — agrees automatically. Putting this mapping anywhere else would eventually produce an install in one folder and a probe in another.

3. **Lines 14-18:** `engine_env_dir`

`paths.user_data_dir()` resolves per-user, per-OS state (`%APPDATA%` on Windows, `~/.local/share` on Linux, `~/Library/Application Support` on macOS), so the application never writes inside its own installation folder — which matters because the installation folder is read-only for a normal user.

The runtime object itself is a thin wrapper over one environment directory:

```

class PythonRuntime:
    def __init__(self, env_dir: str | None = None):
        self._env_dir = env_dir or os.path.join(
            paths.user_data_dir(), _ADDON_ENV_DIR
        )
        self._system_python: str | None = None
        self._lock = threading.RLock()

    @property
    def python_exe(self) -> str:
        return _python_exe_for_env(self._env_dir)

    @property
    def is_created(self) -> bool:
        return os.path.isfile(self.python_exe)

    @property
    def site_packages_dirs(self) -> List[str]:
        found: List[str] = []
        for pattern in ("Lib/site-packages", "lib/python*/site-packages"):
            found.extend(sorted(glob.glob(os.path.join(self._env_dir, pattern))))
        return [path for path in found if os.path.isdir(path)]

    def ensure_env(self) -> str:
        if self.is_created:
            return self._env_dir
        os.makedirs(self._env_dir, exist_ok=True)
        subprocess.run(
            [self.system_python, "-m", "venv", self._env_dir, "--clear"],
            capture_output=True, text=True, check=True,
            cwd=tempfile.gettempdir(), creationflags=creationflags,
        )
        return self._env_dir

```

Explain Line by Line Code Description (Block 2)

1. Lines 2-5: the constructor

An explicit `env_dir` makes the class testable with a temporary folder; the default is the shared addon environment, so existing addon code is unchanged. `_system_python` is filled lazily (finding a base interpreter is a search) and a lock guards it, because several threads may build a runtime at once.

2. Lines 8-9: `python_exe`

A tiny platform-aware function chooses `Scripts/python.exe` on Windows versus `bin/python` elsewhere. Centralising it means no other module ever hard-codes a path separator.

3. Lines 12-13: `is_created`

The environment exists when its interpreter exists. That is the cheapest reliable test: a half-built folder without an interpreter is correctly treated as “not created”, and the next install repairs it.

4. Lines 16-21: `site_packages_dirs`

Returns the folder(s) that hold installed packages, looking for both the Windows layout and the POSIX layout with a wildcarded Python version. Two layouts, one list — and the empty list is a valid answer meaning “nothing installed yet”. This property is the foundation of the synchronous disk probes in 77.4.

5. Lines 23-31: `ensure_env`

Idempotent by design (`if self.is_created: return`), then creates the virtualenv with the *system* interpreter — a frozen PyInstaller application cannot use `sys.executable` for this, because that is the application itself. `check=True` turns a failed `venv` creation into an exception the caller can report, and `cwd=tempfile.gettempdir()` avoids inheriting a working directory that may no longer exist, which is a classic way for subprocess creation to fail on a background thread.

```
def get_runtime(engine_id: str | None = None) -> PythonRuntime:
    global _default_runtime
    if not engine_id:
        if _default_runtime is None:
            with _runtime_lock:
                if _default_runtime is None:
                    _default_runtime = PythonRuntime()
        return _default_runtime
    key = environment_id(engine_id)
    with _runtime_lock:
        runtime = _engine_runtimes.get(key)
        if runtime is None:
            runtime = PythonRuntime(engine_env_dir(engine_id))
            _engine_runtimes[key] = runtime
    return runtime
```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** the optional `engine_id`
Calling it with no argument returns the shared environment; calling it with an id returns that engine's own environment. One function serves both, so callers never pick a class.
2. **Lines 3-8:** the double-checked singleton
The lock is taken only on a miss, and inside it the check is repeated. Two threads racing to build the shared runtime end up with one object — important because two runtime objects would each cache their own “is it created?” answer and could disagree.
3. **Lines 9-14:** the per-engine dictionary
Keyed by `environment_id`, *not* by engine id, so the two OmniVoice engines deliberately share one runtime object.

77.3 Installing: Two pip Steps, Not One

On a machine with an NVIDIA GPU, the plain PyPI `torch` wheel for Windows is **CPU-only**. A GPU engine therefore needs wheels from PyTorch's own index. Those wheels are not mirrored on PyPI, and the index does not carry the engine's other packages — so the install is planned as an ordered list of steps:

```
PYTORCH_CUDA_INDEX = "https://download.pytorch.org/whl/cu128"

_CUDA_PACKAGES = {
    "pocket_tts": ("torch",),
    "bark": ("torch",),
    "f5tts": ("torch", "torchaudio"),
}

def install_plan(engine_id):
    packages = list(packages_for(engine_id))
    steps = []
    cuda = list(engines.cuda_packages(engine_id))
    index = engines.cuda_index_url(engine_id)
    if cuda and index and has_cuda() and not venv_packages.installed(
        "torch", engine=engine_id
    ):
        steps.append((cuda, index))
        packages = [p for p in packages if p not in cuda]
    if packages:
        steps.append((packages, None))
    return steps
```

Explain Line by Line Code Description (Block 4)

1. **Line 1:** `PYTORCH_CUDA_INDEX`

The wheel index for CUDA builds. The `cu128` tag is chosen to match the Python the application runs on: an older index (`cu121`) stops at Python 3.12 and has no wheels at all for 3.13, which produces the infuriating “*No matching distribution found for torch*” while installing an engine. Version tags like this belong in exactly one constant — and the OmniVoice installers read the same constant, so every CUDA wheel in the product comes from one index.

2. **Lines 3–7:** `_CUDA_PACKAGES`

Which packages must come from that index *per engine*. F5-TTS needs `torchaudio` as well, and taking the two from the same index is what stops a build mix (a CUDA `torch` with a CPU `torchaudio` is a runtime crash, not a warning).

3. **Line 10:** `packages = list(packages_for(engine_id))`

Start from the full list the registry declares, then remove whatever the CUDA step already installed. The registry stays the single source of truth about *what* is needed; the plan only decides *from where*.

4. **Line 13:** the condition

Three independent facts must hold: the engine has CUDA packages, the index is known, an NVIDIA GPU with a working driver is present, and PyTorch is not already installed in this environment. The last clause is what makes a re-run cheap and avoids re-downloading gigabytes when only a small package is missing.

5. **Lines 14–15:** step one

The CUDA packages plus the index URL. Returning `(packages, index)` pairs — rather than a single command line — lets the GUI show each step’s progress and keeps the pip invocation in one place.

6. **Lines 16–17:** step two

Everything else from the default index (`None` means “no `--index-url`”). Ordering matters: installing the CUDA build first prevents pip from resolving a CPU torch as a dependency of the engine and then having nothing to do in step one.

⚠ Warning: Never put version specifiers in the registry’s `packages` tuple. The same tuple feeds “Remove engine...”, and `pip uninstall "torchao<0.18"` fails — leaving the user with an engine that cannot be removed. Compatibility pins are applied as an install-time step, not as the package name.

77.4 Three Answers to “Is It Installed?”

The Settings dialog builds all of its pages up front. The original code asked the question by starting `python -c "import importlib.metadata..."` inside the environment — a second or more per package, a dozen of them, all while the window was being drawn. The current module answers the same question three ways:

API	Cost	Used by
<code>version(pkg, engine=...)</code>	memory; on a cold cache it starts one background probe <i>and</i> reads disk	labels that must fill in while the window stays responsive
<code>request(pkg, on_ready=..., engine=...)</code>	one worker thread	panels that want to refresh themselves when the answer arrives
<code>package_present(pkg, engine=...)</code>	a directory listing, synchronous	deciding <i>which interpreter the worker must run with</i>

```

def _disk_distributions(engine: Optional[str] = None) -> Dict[str, str]:
    try:
        from .python_runtime import get_runtime
        folders = get_runtime(engine).site_packages_dirs
    except Exception:
        return {}
    found: Dict[str, str] = {}
    for folder in folders:
        try:
            stamp = os.path.getmtime(folder)
        except OSError:
            continue
        with _lock:
            entry = _disk_cache.get(folder)
            if entry is not None and entry[0] == stamp:
                found.update(entry[1])
                continue
        listed: Dict[str, str] = {}
        try:
            for name in os.listdir(folder):
                if not name.endswith(".dist-info"):
                    continue
                distribution, _, version_text = name[:-len(".dist-info")].partition("-")
                listed[_normalize(distribution)] = version_text
        except OSError:
            continue
        with _lock:
            _disk_cache[folder] = (stamp, listed)
        found.update(listed)
    return found

```

Explain Line by Line Code Description (Block 5)

1. **Lines 2-5:** getting the folders

Delegates to the runtime object so this module never builds a path itself. Wrapped in `try/except` returning an empty dict: “I cannot tell” must not crash a GUI that is merely drawing a label.

2. **Lines 7-12:** the stamp check

The folder’s modification time is the cache key. Any pip operation changes it, so the cache is invalidated by the act of installing — without a single explicit “please clear the cache” call from the installer.

3. **Lines 13-20:** the listing

Every installed distribution leaves a `<name>-<version>.dist-info` folder. Splitting on the *first* dash (`partition`) and stripping the suffix yields name and version, and `_normalize()` applies PEP 503 normalisation so `f5_tts` , `f5-tts` and `F5.TTS` are one key. Reading files instead of importing them is why this is safe on the GUI thread.

4. **Lines 21-26:** caching and merging

The listing is stored under the stamp and merged into one dictionary across folders. A partially written directory (pip is mid-install) is handled by the outer `OSError` guard: the next call sees a new stamp and re-reads.

```
def package_present(package: str, engine: Optional[str] = None) -> bool:
    name = _normalize(package)
    if not name:
        return False
    if name in _disk_distributions(engine):
        return True
    try:
        folders = get_runtime(engine).site_packages_dirs
    except Exception:
        return False
    stem = name.replace("-", "_")
    for folder in folders:
        if os.path.isdir(os.path.join(folder, stem)):
            return True
        if os.path.isfile(os.path.join(folder, stem + ".py")):
            return True
    return False
```


Explain Line by Line Code Description (Block 6)

1. **Lines 2-5:** the normalised short-circuit

An empty name can never match, and the fast path asks the same disk listing used everywhere else.

2. **Lines 9-13:** the fallbacks

Some hand-patched or vendor-installed environments have the importable module or package folder but no `dist-info` metadata (this is common with wheels unpacked by other tools). Accepting a folder `stem/` or a module `stem.py` means such an environment is still detected as usable rather than looking empty.

3. **The point of the function**

It is *synchronous and cache-free*. It exists for exactly one decision, described next, where a slightly stale answer would send a worker into the wrong environment.

77.5 Choosing the Interpreter for the Worker

This is where the three probes earn their keep. A worker started in the wrong environment does not report the engine's real error; it reports `ModuleNotFoundError: No module named 'torch'` — a message that says nothing about which environment was used and sends the reader hunting in the wrong place.

```
def _runtime(self):
    required = tuple(engines.import_modules(self.engine_id)) or (
        engines.probe_package(self.engine_id),
    )
    required = tuple(name for name in required if name)
    engine_runtime = get_runtime(self.engine_id)
    if engine_runtime.is_created and (
        not required
        or all(venv_packages.package_present(name, engine=self.engine_id)
              for name in required)
    ):
        return engine_runtime
    runtime = get_runtime()
    if not runtime.is_created:
        runtime.ensure_env()
    if not required or all(venv_packages.package_present(name) for name in required):
        return runtime
    raise VoicelabError(
        f"{engines.engine_name(self.engine_id)} is not installed in its own "
        "Python environment any more. Install it again from Settings > Voice Clone."
    )
```

Explain Line by Line Code Description (Block 7)

1. **Lines 2-5:** what the engine needs

The worker's own module list (the same tuple the worker checks at start-up) with the single probe package as a fallback. The list is *shared data*: the worker and the application must not disagree about what “installed” means, and there is a test asserting the two copies have not drifted apart.

2. **Lines 6-12:** prefer the engine's own environment

Both conditions are required: the environment must exist *and* every required module must be present in it, checked with the synchronous `package_present`. Using the cached probe here would be the bug the docstring warns about — a probe that has not answered yet returns `None`, which reads as “not installed”, which silently downgrades to the shared environment.

3. **Lines 13-16:** the fallback

Engines installed before per-engine environments existed still live in the shared environment, so an installation is honoured there. The environment is created if missing, because the caller is about to start a process in it.

4. **Lines 17-21:** the honest failure

If neither environment can satisfy the engine, the error says so and tells the user exactly what to do. This is the difference between a support ticket that reads “it says No module named torch” and one that reads “it told me to reinstall the engine”.

77.6 Verify the Install, Do Not Assume It

pip succeeding means the dependency graph resolved — *not* that the engine can import. A resolved combination can still fail at import time when a sub-dependency renamed a module. So after installing, the application imports the engine once, in its own environment, and reports what happens:

```

def verify_engine_import(engine_id: str, timeout: float = 300.0) -> Optional[str]:
    modules = engines.import_modules(engine_id)
    if not modules:
        return None
    try:
        runtime = get_runtime(engine_id)
        if not runtime.is_created:
            return None
        python_exe = runtime.python_exe
    except Exception as exc:
        return f"Could not start the engine's Python environment: {exc}"
    if not os.path.isfile(python_exe):
        return None
    script = "import " + ", ".join(modules)
    try:
        result = subprocess.run(
            [python_exe, "-c", script],
            capture_output=True, text=True, timeout=timeout,
            creationflags=flags,
        )
    except subprocess.TimeoutExpired:
        return (f"Loading {engines.engine_name(engine_id)} took too long "
            f"({int(timeout)}s) and was stopped.")
    except OSError as exc:
        return f"Could not run the engine's Python environment: {exc}"
    if result.returncode == 0:
        return None
    detail = (result.stderr or result.stdout or "").strip().splitlines()
    last = detail[-1] if detail else f"exit code {result.returncode}"
    return f"{engines.engine_name(engine_id)} is installed but failed to load: {last}"

```

Explain Line by Line Code Description (Block 8)

1. **Lines 2-4:** the module list

Exactly the modules the worker will need. An empty list means the registry has nothing to verify and the check is skipped rather than guessed.

2. **Lines 5-11:** the interpreter

Each failure to even locate the environment is converted into a message, not an exception: this function's return value is the user-facing report, so it tells rather than raises.

3. **Line 13:** `script = "import " + ", ".join(modules)`

One interpreter run imports everything at once, so the empty case cannot happen and the cost is a single process start-up.

4. **Lines 15-21:** the subprocess call

A 300-second timeout is deliberately generous: the first import of a large model package may load native libraries from disk and take minutes on a cold cache, and killing it would report a false failure. `text=True` decodes the output so it can be shown verbatim.

5. **Lines 22-24:** the timeout branch

Telling the user “it took too long and was stopped” is honest and actionable; letting the GUI hang behind an import would look like a freeze.

6. **Lines 27-30:** the failure branch

Only the *last* line of stderr is surfaced. Python tracebacks end with the genuinely informative line, and the full traceback is far too long for a dialog — it belongs in the log, which the install dialog also writes.

7. **A note on the return convention**

“Problem, or `None`” keeps the caller trivial: `problem = verify_engine_import(id); if problem: show(problem)`. Compare that to a function that raises, where the caller must also remember to catch.

 **Tip:** This check is the single highest-value addition to the installer flow. A missing module discovered during installation costs a minute; discovered three chapters into recording, it costs the trust of the whole feature.

77.7 Mini Project: Inspect the Environments From Python

```
from ai_voice_studio import python_runtime, venv_packages

for engine_id in ("pocket_tts", "bark", "f5tts"):
    runtime = python_runtime.get_runtime(engine_id)
    print(f"{engine_id:12} env={runtime.env_dir}")
    print(f"{'':12} created={runtime.is_created}")
    for package in ("torch", "torchaudio", "soundfile"):
        present = venv_packages.package_present(package, engine=engine_id)
        print(f"{'':12} {package:10} {present}")
```

Run it once before an install and once after, and you are watching the same decision `CloneWorker._runtime` makes when it picks an interpreter.

 Exercise:

1. Why does `environment_id()` exist instead of every caller using `engine_id` directly? Give one concrete bug the current design prevents.
2. Add a fourth entry to `SHARED_ENVIRONMENTS` that makes `f5tts` use the `bark` environment. Predict which tests would fail, then run the suite and compare.
3. Write a small script that prints `install_plan("bark")` on this machine. Explain the output for a machine with an NVIDIA GPU and for one without.
4. Time `venv_packages.version()` on a cold cache versus a warm one. Then explain why the cold path still answers correctly instead of returning `None`.

77.8 Key Takeaways

- One virtualenv per engine is the only reliable answer to conflicting ML dependencies
 - Map ids to environments in *one* function so every subsystem agrees
 - CUDA wheels come from PyTorch's index, in a separate pip step, keyed to the Python version in use
 - Answer “is it installed?” from disk when correctness matters, and asynchronously when responsiveness matters
 - Never let a non-blocking cache decide which interpreter starts a worker
 - Verify by importing — a successful pip run is not a working engine
 - Never put version specifiers in the list used for uninstalling
-

Compute Choice: CPU, GPU and How Much of Each

Learning Objectives

- Split “how much of the machine may a run use?” from “which device does it run on?”
- Compute a CPU thread count that stays inside an 80–95 % band, on any core count
- Use the whole of whatever NVIDIA card is present, without model or VRAM checks
- Offer the choice per category in the UI, remembering each category’s answer
- Apply the decision inside a worker process with PyTorch’s own knobs

78.1 Two Questions Users Actually Ask

“Which processor should this use?” hides two requirements that look similar and must be implemented very differently:

Requirement	Question it answers	Where it lives
Device choice	GPU or CPU for this run?	<code>compute.resolve_compute</code> , <code>voicelab.resolve_device</code>
Budget	How much of that device may the run consume?	<code>compute.cpu_threads</code> , <code>worker _maximize_cuda</code> / <code>_limit_cpu</code>

The requirements as stated by the user were concrete:

- On **GPU**: use the maximum compute available on whatever NVIDIA card is installed — no model whitelist, no VRAM minimum.
- On **CPU**: use between 80 % and 95 % of the available CPU.

78.2 Detecting the GPU Once, Cheaply

```
_nvidia_cache: bool | None = None

def has_nvidia_gpu(force: bool = False) -> bool:
    global _nvidia_cache
    if _nvidia_cache is not None and not force:
        return _nvidia_cache
    try:
        flags = 0
        if sys.platform == "win32":
            flags = getattr(subprocess, "CREATE_NO_WINDOW", 0)
        result = subprocess.run(
            ["nvidia-smi", "--query-gpu=name", "--format=csv,noheader"],
            capture_output=True, text=True, timeout=10, creationflags=flags,
        )
        found = result.returncode == 0 and bool(result.stdout.strip())
    except Exception:
        found = False
    _nvidia_cache = found
    return found
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** a module-level cache

A tri-state (`None` = never asked). This matters: “not yet asked” and “asked, answer is no” are different, and collapsing them would re-run the driver query on every combo box the user opens.

2. **Lines 4-6:** the optional `force`

A “Re-detect” button and the tests need today’s hardware, not the answer cached when the process started. Caching plus an explicit escape hatch is the polite version of a global.

3. **Lines 8-11:** `CREATE_NO_WINDOW`

A Windows detail that matters in a GUI app: without this flag, running `nvidia-smi` flashes a console window over the user’s screen. `getattr` with a default keeps the same code working on Linux and macOS.

4. **Lines 12-16:** the query

Driver-only detection: `nvidia-smi` exists wherever an NVIDIA driver is installed, so this asks “is there a usable GPU?” without importing PyTorch or downloading a runtime. The query asks only for the *name*, because that is the smallest answer that can prove the driver responded.

5. **Line 17:** the `except Exception` branch

A missing executable, a non-zero exit, a timeout — every one of them means “no GPU”, never a crash. Detection is best-effort by definition.

The *option* “GPU” and the *ability to run* on the GPU are separate concerns, and the docstring in `compute.py` is explicit about it:

```
def _has_cuda(force: bool = False) -> bool:
    if not runtime.is_installed():
        return False
    return has_nvidia_gpu(force)

def detect(force: bool = False) -> Dict[str, bool]:
    cache = getattr(detect, "_cache", None)
    if cache is not None and not force:
        return cache
    result = {
        COMPUTE_CPU: True,
        COMPUTE_CUDA: _has_cuda(force),
        COMPUTE_DML: _has_npu(),
    }
    detect._cache = result
    return result
```

Explain Line by Line Code Description (Block 2)

1. **Lines 1-4:** `_has_cuda`

Two conditions, one per cause of failure. The ONNX engines need the optional GPU runtime *and* a driver; the runtime is an explicit download because it is roughly 1.8 GB and most users never need it. The pip-installed Voice Lab engines bypass this path entirely — they bring their own PyTorch, so for them a driver is enough (see `provider_for_preview` in 78.5).

2. **Line 8:** a cache stored on the function object

`detect._cache` is a small, readable alternative to decorating with `functools.lru_cache` — it can be cleared for tests, and it is obviously per-process state.

3. **Lines 10-15:** the result dictionary

CPU is hard-coded `True`: the bundled runtime always exists. DirectML stays `False` because no DirectML/NPU runtime exists for sherpa-onnx — the option is deliberately *not* offered rather than offered and broken.

4. **Why not import `onnxruntime` to ask?**

Because sherpa-onnx ships its own runtime and loading a second `onnxruntime` into one process corrupts the graph loader (two DLLs with the same module name). Detection therefore reads *files* and *drivers*, never imports.

78.3 The CPU Band: 80–95 % Done Right

“Use 80–95 % of the CPU” is easy to say and surprisingly fiddly to implement, because thread counts are integers and small machines break naive arithmetic. The whole rule is four useful lines:


```

CPU_THREAD_MIN_RATIO = 0.80
CPU_THREAD_MAX_RATIO = 0.95

def cpu_threads(total: Optional[int] = None) -> int:
    try:
        cpus = int(total if total is not None else (os.cpu_count() or 1))
    except (TypeError, ValueError):
        cpus = 1
    if cpus <= 1:
        return 1
    low = max(1, int(math.ceil(cpus * CPU_THREAD_MIN_RATIO)))
    high = max(1, int(math.floor(cpus * CPU_THREAD_MAX_RATIO)))
    if high < low:
        return max(1, cpus - 1)
    return max(1, min(high, cpus))

```

Explain Line by Line Code Description (Block 3)

1. **Lines 1-2:** the band as named constants

Two floats with names, not magic numbers inside the function. They are also importable, so documentation and tests quote the same rule the code implements.

2. **Lines 6-9:** reading the core count

`total` exists so tests can ask “what would you do on a 3-core machine?” without owning one. `os.cpu_count() or 1` handles the documented case where Python cannot report a count. The `int()` conversion inside a `try` guards against a caller passing a string.

3. **Lines 10-11:** the single-core case

A one-core machine gets its one core. No rule about leaving room can be satisfied when there is only one resource, so the honest answer is “all of it”.

4. **Lines 12-13:** the band, using `ceil` and `floor`

The lower bound rounds *up* (80 % of 10 cores is 8.0 → 8; 80 % of 6 is 4.8 → 5) and the upper bound rounds *down* (95 % of 10 is 9.5 → 9). Rounding *inwards* is what keeps the result inside the promised band instead of just outside it.

5. **Lines 14-15:** the small-machine escape

On two or three cores the band is empty (on 2 cores: low = 2, high = 1). Returning `cpus - 1` keeps the *intent* of the rule — leave a little headroom so the desktop, the Recording window and the audio writer stay responsive — instead of returning an impossible number or crashing.

6. **Line 16:** the chosen value

The top of the band, clamped to `cpus`. The requirement was “as much CPU as possible within the rule”, so the maximum of the band is deliberate: 15 of 16 threads, 30 of 32. The single reserved core is what keeps the UI responsive during a long recording.

Worked examples — the same function, six machines:

Logical CPUs	low (80 %)	high (95 %)	Threads used	Share
1	—	—	1	100 %
2	2	1	1	50 % (band empty → leave a core)
4	4	3	3	75 % (band empty → leave a core)
8	7	7	7	88 %
16	13	15	15	94 %
32	26	30	30	94 %

 **Note:** Watching that table is the fastest way to understand the `high < low` branch: on 4 cores, 80 % is 3.2 (→ 4) and 95 % is 3.8 (→ 3), so no integer satisfies the literal rule. The code chooses the closest honest behaviour — one core free — instead of pretending.

78.4 The GPU Rule: Any NVIDIA Card, All of It

```
def cuda_device_index() -> int:
    return 0

def gpu_summary() -> str:
    if not has_nvidia_gpu():
        return ""
    result = subprocess.run(
        ["nvidia-smi", "--query-gpu=name,memory.total", "--format=csv,noheader"],
        capture_output=True, text=True, timeout=10, creationflags=flags,
    )
    first = (result.stdout or "").strip().splitlines()
    if result.returncode == 0 and first:
        return first[0].strip()
    return ""
```

Explain Line by Line Code Description (Block 4)

1. **Lines 1-3:** `cuda_device_index`

Returns `0`, with the reasoning written in the docstring: an engine process drives one device, and the driver exposes the cards in a fixed order, so “the first” is the whole story. There is deliberately no VRAM threshold and no model list — any CUDA-capable NVIDIA card is accepted, which is exactly the requirement.

2. **Lines 6-7:** the “no GPU” answer

Empty string instead of an exception: this value is shown in a status label, and “no GPU” is a normal state, not an error.

3. **Lines 8-12:** the richer query

Name *and* total memory, purely for display. Users like knowing which card the application will use, and the memory number makes a slow run easier to explain than a bare model name.

4. **Lines 13-16:** the first line only

Multi-GPU machines list several cards. The application uses the first, so it displays the first — reporting a second card would describe a device the run does not touch.

```
def _maximize_cuda(torch):
    try:
        if not torch.cuda.is_available():
            return False
        torch.cuda.set_device(0)
    except Exception:
        return False
    try:
        torch.backends.cudnn.benchmark = True
        torch.backends.cuda.matmul.allow_tf32 = True
        torch.backends.cudnn.allow_tf32 = True
    except Exception:
        pass
    try:
        torch.set_float32_matmul_precision("high")
    except Exception:
        pass
    try:
        torch.set_grad_enabled(False)
    except Exception:
        pass
    return True
```

Explain Line by Line Code Description (Block 5)

1. **Lines 2-6:** the availability gate

`torch.cuda.is_available()` is the engine's own truth: a driver can exist while a CUDA build of PyTorch cannot use it. `set_device(0)` pins the process to the first card, matching `cuda_device_index()`.

2. **Lines 7-12:** the fast kernels

`cuda.benchmark` lets cuDNN pick the fastest convolution algorithm for the shapes it sees — ideal here, because recording a book reuses the same shapes thousands of times. `TensorFloat-32` enables the tensor-core paths on float32 maths, which is a large speed-up for inference on modern NVIDIA hardware.

3. **Lines 13-16:** precision policy

`"high"` float32 matmul precision means “use the fast tensor-core path where it is numerically safe”. It is the documented, portable way to ask for that, rather than relying on individual backend flags.

4. **Lines 17-20:** inference mode

These engines only ever run forward passes, so autograd's bookkeeping is pure overhead — memory and time spent recording gradient information nobody reads. Turning it off system-wide inside a dedicated worker process is safe precisely because that process does nothing else.

5. **Every block is guarded separately**

Each optimisation is wrapped in its own `try/except`. An older PyTorch may not have `set_float32_matmul_precision`, and one unavailable knob must not cost the user the other three.

```

def _limit_cpu(torch, threads):
    if not threads:
        return False
    try:
        value = max(1, int(threads))
    except (TypeError, ValueError):
        return False
    try:
        torch.set_num_threads(value)
    except Exception:
        pass
    try:
        torch.set_num_interop_threads(max(1, value // 2))
    except Exception:
        # Raised once parallel work has started; the per-op thread pool set
        # above is the one that decides how much CPU inference uses.
        pass
    return True

def _apply_compute_settings(device, threads=None):
    try:
        import torch
    except Exception:
        return
    if str(device).lower() == "cuda":
        _maximize_cuda(torch)
    else:
        _limit_cpu(torch, threads)

```

Explain Line by Line Code Description (Block 6)

1. **Lines 1-7:** validating the number

`threads` arrives over a JSON pipe from another process, so it is treated as untrusted input: a missing or unparsable value simply means “no limit specified”. `max(1, int(...))` prevents the write of a zero or negative thread count, which PyTorch rejects.

2. **Lines 8-11:** `set_num_threads`

The knob that actually decides how much CPU inference uses: the size of the intra-op thread pool. This is where the 80–95 % band becomes reality.

3. **Lines 12-17:** `set_num_interop_threads`

The pool used when *different* operations run in parallel — half the intra-op size, because inter-op parallelism is coarser and gains less from many threads. PyTorch raises if this is called after parallel work has begun; the comment records that the call above is the one that matters, so the exception is intentionally swallowed.

4. **Lines 19-24:** one entry point

The worker calls `_apply_compute_settings(device, threads)` once when it loads an engine, and the branch is deliberately a plain `if/else`: a GPU run maximises the card, everything else limits the CPU. A `try` around the import means a CPU-only environment can never fail because PyTorch is absent.

78.5 The Combo Box Next to Every Preview Button

The compute choice is offered where a voice is actually *heard*: next to each category's Preview button. The widget code is small because every decision was written as a pure function first.

```
SETTINGS_PREFIX = "preview_compute"

def choices() -> List[Tuple[str, str]]:
    options = [("cpu", "CPU")]
    if compute.has_nvidia_gpu():
        options.append(("cuda", "GPU (CUDA)"))
    if len(options) > 1:
        options.append(("auto", "Auto (GPU when possible)"))
    return options

def settings_key(category: str) -> str:
    return f"{SETTINGS_PREFIX}.{category}"

def saved_choice(settings, category: str) -> str:
    return _stored_choice(settings, category) or "cpu"

def provider_for_preview(choice: str, engine_id: Optional[str] = None) -> str:
    if engine_id:
        from ..voicelab import engines as voice_lab
        if voice_lab.is_engine(engine_id):
            from ..voicelab import resolve_device
            return resolve_device(choice)
    return compute.provider_for(compute.resolve_compute(choice))
```

Explain line by line code description (Block 7)

1. Line 1: `SETTINGS_PREFIX = "preview_compute"`

The key names and the constant have to agree forever, so the constant is the only place the string is written. The module docstring records why it is *not* `compute.preview` : `compute` is a plain string setting, so nesting under it makes `Settings.set` fail and silently drops every category's choice.

2. Lines 4-11: `choices()`

CPU is always offered; GPU appears only when a driver is detected; Auto appears only when there is something to choose between (with a single option, "Auto" is a synonym for "CPU" and would only confuse). `len(options) > 1` is checked *before* appending, so the logic reads top to bottom.

3. Lines 14-15: `settings_key`

One category, one key — "preview_compute.voicelab", "preview_compute.omnivoice". Per-category memory is what makes "GPU here, CPU there" possible.

4. Lines 18-19: `saved_choice`

Returns the stored value or the default `"cpu"`. Only values that are still valid on this machine are returned (see `_stored_choice`), so a settings file copied from a GPU machine to a CPU-only laptop does not select an option the combo cannot show.

5. Lines 22-28: `provider_for_preview`

The interesting branch. A Voice Lab engine resolves the device through its *own* PyTorch (`resolve_device` — no downloaded runtime needed), while every ONNX engine goes through `compute.resolve_compute` and needs the optional GPU runtime. That single difference is why the code asks "is this a Voice Lab engine?" rather than mapping the value directly.

```

def make_compute_row(parent, settings, category, default="cpu"):
    label = wx.StaticText(parent, label="Compute:")
    combo = wx.ComboBox(parent, style=wx.CB_READONLY, name="Compute")
    set_accessible_name(combo, "Compute")
    combo.SetToolTip(
        "Which back-end the Preview uses: CPU, GPU (CUDA, when an NVIDIA GPU "
        "with a driver is detected) or Auto."
    )
    options = choices()
    for value, text in options:
        combo.Append(text, value)
    available = {value for value, _text in options}
    stored = _stored_choice(settings, category)
    current = stored or (default if default in available else "cpu")
    if current not in available:
        current = "cpu"
    for index in range(combo.GetCount()):
        if combo.GetClientData(index) == current:
            combo.SetSelection(index)
            break
    else:
        combo.SetSelection(0)

def _persist(_evt):
    if settings is None:
        return
    try:
        settings.set(settings_key(category), _combo_value(combo))
    except Exception:
        pass

combo.Bind(wx.EVT_COMBOBOX, _persist)
return label, combo

```


Explain Line by Line Code Description (Block 8)

1. **Lines 2-4:** the label and combo

`CB_READONLY` prevents typing a value the code cannot resolve — a read-only combo is a promise that every reachable value is valid. Naming the control (“Compute”) is what makes NVDA announce it as something other than “combo box”.

2. **Lines 5-8:** the tooltip

A tooltip is not decoration here: it is the only place the user learns that ONNX engines need the GPU runtime downloaded from Settings, which is the difference between “GPU does not work” and “I need to install the runtime”.

3. **Lines 9-11:** appending values with client data

`combo.Append(text, value)` stores machine-readable data behind the human label, so no code ever compares translated display text. Renaming a label then cannot break the logic.

4. **Lines 12-14:** resolving the selection

Stored choice first, then the caller’s default if it is actually available, then CPU. The `if current not in available` re-check closes the last hole — an engine that *requires* the GPU passes `default="cuda"`, and on a machine without a GPU it degrades to CPU rather than showing an impossible selection.

5. **Lines 15-20:** selecting it

The loop searches by client data; the `else` clause of the `for` runs when nothing matched and selects the first item — a construct many Python programmers forget exists, and one that removes a “did I find it?” flag variable.

6. **Lines 22-30:** remembering the choice

Persisted on `EVT_COMBOBOX`, immediately. A preference that is forgotten until “OK” is a preference that is lost when the dialog is cancelled; and settings failures are swallowed because a preference must never raise inside an event handler.

 **Tip:** Return a `(label, combo)` tuple rather than a panel: the caller decides layout, and every settings category can place the row wherever its Preview button already is.

78.6 Mini Project: Watch the Band Decide

```
from ai_voice_studio import compute

print("this machine :", compute.cpu_threads(), "threads of",
      __import__("os").cpu_count())
for cores in (1, 2, 3, 4, 6, 8, 12, 16, 24, 32, 64):
    threads = compute.cpu_threads(cores)
    print(f"{cores:>3} cores -> {threads:>3} threads "
          f"({threads / cores:.0%})")

print("GPU      :", compute.has_nvidia_gpu())
print("card     :", compute.gpu_summary() or "(none detected)")
print("options  :", compute.available_compute())
print("resolved  :", compute.resolve_compute("auto"))
```

On a 16-thread CPU with no NVIDIA card the output ends with `options: ['cpu']` and `resolved: cpu` — and, importantly, the combo box in Settings then offers one entry instead of an unusable GPU choice.

 Exercise:

1. Change the band to 50–60 % in a scratch copy and predict the 4-core, 8-core and 16-core results *before* running. Then explain why rounding inwards still matters.
2. On a machine with an NVIDIA GPU, record a paragraph twice — once with the combo on CPU and once on GPU — and compare the wall-clock times. Which part of the difference comes from `_maximize_cuda` rather than raw hardware?
3. Write a test for `cpu_threads` that asserts the result is inside the band whenever the band is non-empty, and equals `cores - 1` when it is empty. Run it for every core count from 1 to 64.
4. Explain why `provider_for_preview` treats Voice Lab engines differently from ONNX engines, and what would break if it did not.

78.7 Key Takeaways

- Separate **device choice** from **resource budget** — they have different APIs and different failure modes
 - Detect hardware with the driver (`nvidia-smi`), never by importing a framework
 - Round *inwards* to stay inside a percentage band, and define behaviour for machines where the band is empty
 - Accept any CUDA-capable NVIDIA card: no model list, no VRAM minimum, use the whole card
 - Apply the CPU share in the worker with `set_num_threads` , and let the application decide the number
 - Offer the choice per category and store it under a namespaced key that cannot collide with an existing setting
 - Guard every optional backend flag separately, so one missing knob costs only itself
-

Audio I/O That Works on a Machine With No FFmpeg

Learning Objectives

- Understand why `torchaudio.load` stopped being self-contained
- Probe a third-party decoder on a real file instead of trusting documentation
- Replace one API with another transparently, and restore the original when it works
- Convert reference audio once, deterministically, instead of failing on a format
- Report a degraded-but-working path to the user instead of hiding it

79.1 The Failure That Started This

A user installed F5-TTS and pressed Preview. The engine loaded, the model downloaded, and then:

```
RuntimeError: Failed to create AudioDecoder for C:\Users\...\tmpvmlfm3u2.wav:
Could not load libtorchcodec. Likely causes:
  1. FFmpeg is not properly installed in your environment...
  2. The PyTorch version (2.11.0+cu128) is not compatible with this
     version of TorchCodec...
OSError: Could not load this library:
  ...\torchcodec\libtorchcodec_core9.dll
```

Three things were true at once, and it is worth separating them:

- From **torchaudio 2.9** onward, `load / save` decode through **TorchCodec**; torchaudio's own `backend` argument is accepted and *ignored*. Code that looks like it selects a decoder no longer selects anything.
- TorchCodec on Windows needs FFmpeg's **shared** libraries (`avcodec-*.dll` and friends), which belong to an FFmpeg build the application does not ship.
- The user's machine had no FFmpeg at all. Every version TorchCodec tried failed, which is why the message lists six versions — that wall of text is one failure repeated, not six problems.

i Info: The application already downloads FFmpeg *on request* for MP3/FLAC output. Requiring it for *reading* audio would turn an optional feature into a hard requirement, so the audio path was fixed in code instead.

79.2 Measure, Do Not Assume

TorchCodec is not always broken — it works perfectly on a machine with FFmpeg. Replacing it unconditionally would throw away a capable decoder. So the code measures it, on a real file:

```
def _write_probe_wav(path, sample_rate=16000, seconds=0.05):
    frames = max(1, int(sample_rate * seconds))
    with wave.open(path, "wb") as handle:
        handle.setnchannels(1)
        handle.setsampwidth(2)
        handle.setframerate(int(sample_rate))
        handle.writeframes(struct.pack(f"<{frames}h", *([0] * frames)))

def _torchcodec_works(torchaudio):
    path = os.path.join(tempfile.gettempdir(), "aivs_voicelab_probe.wav")
    try:
        _write_probe_wav(path)
        torchaudio.load(path)
        return True
    except Exception:
        return False
    finally:
        with contextlib.suppress(OSError):
            os.remove(path)
```

Explain Line by Line Code Description (Block 1)

1. **Lines 1-7:** `_write_probe_wav`

Builds the smallest possible WAV with the standard library: one channel, 16-bit samples, 0.05 seconds of silence. `wave` needs no dependency, so this works even on a machine where every audio library is broken — which is precisely the situation being tested. The `_write_probe_wav` name says what it writes; the `frames = max(1, ...)` guard prevents a zero-frame file, which some decoders reject for a different reason and would produce a misleading “broken” verdict.

2. **Line 4:** `struct.pack(f"<{frames}h", *([0] * frames))`

Little-endian sequence of signed 16-bit zeros — raw PCM in exactly the layout a WAV header promises. Generating samples in code beats bundling a fixture file, because the probe must not depend on anything the packaging step might forget to include.

3. **Lines 10-12:** the actual measurement

A real decode, not a version check. Feature detection by attempting the operation is the only method that survives version combinations nobody documented — and this bug is proof that documented compatibility tables are not enough.

4. **Lines 13-17:** cleanup on every path

`finally` removes the temp file even when the decode fails, so a broken decoder cannot leave litter in `%TEMP%` on every launch. `contextlib.suppress(OSError)` keeps a failed cleanup from masking the real result.

79.3 A Drop-In Replacement Built on soundfile

`soundfile` wraps `libsndfile`, and — crucially — **libsndfile ships inside the wheel**. Reading WAV, FLAC, OGG and MP3 therefore works with nothing installed on the machine.

```
def _soundfile_load(sf, torch, uri, frame_offset=0, num_frames=-1,
                    normalize=True, channels_first=True, format=None,
                    buffer_size=4096, backend=None):
    if hasattr(uri, "read"):
        data, sample_rate = sf.read(uri, dtype="float32", always_2d=True)
    else:
        try:
            start = max(0, int(frame_offset or 0))
        except (TypeError, ValueError):
            start = 0
        try:
            frames = int(num_frames)
        except (TypeError, ValueError):
            frames = -1
        data, sample_rate = sf.read(
            str(uri), start=start, frames=(frames if frames > 0 else -1),
            dtype="float32", always_2d=True,
        )
    samples = np.ascontiguousarray(data.T if channels_first else data)
    tensor = torch.from_numpy(samples.astype(np.float32, copy=False))
    return tensor, int(sample_rate)
```

Explain Line by Line Code Description (Block 2)

1. Lines 1–2: the signature

It mirrors `torchaudio.load` exactly, including the arguments the engines pass but that do not apply (`normalize` , `format` , `buffer_size` , `backend`). A shim that accepts a narrower signature is a shim that breaks the first time an engine passes a new keyword — the engines here are third-party code that changes between releases.

2. Line 3: file-like objects

Callers may pass an open file or a buffer rather than a path; `hasattr(uri, "read")` routes that case to soundfile's own handling, so the shim covers both call styles.

3. Lines 5–14: offsets and frame counts

Frame slicing is converted from torchaudio's conventions to soundfile's: a negative or unparsable `num_frames` becomes “the whole file”, and a negative offset clamps to zero. Defensive conversion here means a caller passing `None` gets audio instead of an exception about a type.

4. Line 15: the shape flip

soundfile returns `(frames, channels)` ; torchaudio returns `(channels, frames)` . `data.T` plus `np.ascontiguousarray` fixes both the axis order and the memory layout — the second part matters because a non-contiguous array passed to `torch.from_numpy` can produce wrong strides or copies deep inside an engine.

5. Line 16: the tensor

`astype(np.float32, copy=False)` avoids a copy when the dtype already matches, and `int(sample_rate)` normalises NumPy integers to plain Python ints, which is what the engines and WAV writers expect.

The saving side is the mirror image, with one extra concern — integer PCM must stay integer:

```
def _soundfile_save(sf, torch, uri, src, sample_rate, channels_first=True,
                   format=None, encoding=None, bits_per_sample=None,
                   buffer_size=4096, backend=None, compression=None):
    data = src
    if hasattr(data, "detach"):      # torch tensor
        data = data.detach()
    if hasattr(data, "cpu"):
        data = data.cpu()
    if hasattr(data, "numpy"):
        data = data.numpy()
    array = np.asarray(data)
    if channels_first and array.ndim > 1:
        array = array.T
    if array.dtype.kind in "iu":    # integer PCM stays integer
        sf.write(str(uri), array, int(sample_rate))
    else:
        sf.write(str(uri), np.clip(array.astype(np.float32), -1.0, 1.0),
                 int(sample_rate))
```

Explain Line by Line Code Description (Block 3)

1. **Lines 5–10:** duck-typed conversion

The argument may be a PyTorch tensor (possibly on the GPU), a NumPy array or a plain list. Each `hasattr` step unwraps one layer — `detach` before `cpu` before `numpy` is the order that works for a tensor with gradients on a CUDA device. Duck typing is the right tool here because importing torch inside these helpers would defeat the point of keeping them dependency-light.

2. **Lines 11–13:** channel order

The same flip as on load, guarded by `array.ndim > 1` so mono data is not accidentally squeezed into a 1-D array.

3. **Lines 14–15:** the integer branch

Sound is often handled as int16 PCM in this project. Writing int16 through `libsndfile` preserves it exactly; converting to float and back would add rounding for no reason. `dtype.kind in "iu"` is the compact way to say “signed or unsigned integer”.

4. **Lines 16–18:** the float branch

Float audio is clipped to $-1.0 \dots 1.0$ *before* writing, because `libsndfile`’s behaviour with out-of-range floats has changed between versions. An explicit clip makes the file’s contents predictable everywhere.

79.4 Installing the Shim Only When Needed

```
_AUDIO_SHIM_ENV = "AIVS_VOICELAB_NO_AUDIO_SHIM"

def _install_audio_io_shim(torchaudio=None, soundfile=None, torch=None):
    if str(os.environ.get(_AUDIO_SHIM_ENV, "")).strip().lower() in ("0", "false", "no"):
        return None
    if torchaudio is None:
        try:
            import torchaudio as _torchaudio
        except Exception:
            return None
        torchaudio = _torchaudio
    if getattr(torchaudio, "_aivs_soundfile_io", False):
        return None
    if _torchcodec_works(torchaudio):
        return None
    if soundfile is None:
        try:
            soundfile = _import_soundfile()
        except Exception:
            return (... a message explaining decoding is unavailable ...)
    if torch is None:
        try:
            import torch as _torch
        except Exception:
            return None
        torch = _torch

    def load(*args, **kwargs):
        return _soundfile_load(soundfile, torch, *args, **kwargs)

    def save(*args, **kwargs):
        return _soundfile_save(soundfile, torch, *args, **kwargs)

    torchaudio.load = load
    torchaudio.save = save
    torchaudio._aivs_soundfile_io = True
    return ("Audio files are read with the bundled soundfile library: this "
           "computer has no FFmpeg, which the TorchCodec decoder torchaudio "
           "normally uses needs. WAV, FLAC, OGG and MP3 work; formats only "
           "FFmpeg can decode (for example m4a/AAC) do not.")
```


Explain Line by Line Code Description (Block 4)

1. **Lines 1-4:** the opt-out switch

`AIVS_VOICELAB_NO_AUDIO_SHIM=0` disables the swap. An escape hatch for a machine where the measurement is wrong is what keeps a clever workaround from becoming an unfixable bug: a user (or the developer) can always get the original behaviour back without editing code.

2. **Lines 5-10:** getting hold of torchaudio

The module can be passed in (tests do that) or imported. If it cannot be imported at all there is nothing to shim — an engine that needs torchaudio will fail on its own import with a clearer message than anything this function could invent.

3. **Lines 11-12:** idempotence

The marker attribute on the module itself means a second call does nothing. Without it, the shim would wrap the shim:

`load` would call soundfile through a function that calls soundfile, and a future un-shim would be impossible.

4. **Lines 13-14:** the decision

The single most important line pair in the module: *if the real decoder works, do nothing*. This is what keeps the wider format support (and the platform's own codecs) for users who have FFmpeg.

5. **Lines 15-22:** the fallback library

If soundfile is missing too, the function returns a message rather than raising: the engine will then either fail with its own error or succeed some other way, and the message explains the situation in the log either way.

6. **Lines 24-33:** the swap

Two closures bound to the already-imported libraries, assigned onto the module object. Assigning module attributes (rather than wrapping the module in a proxy) means every engine that already did `import torchaudio` and calls `torchaudio.load(...)` picks up the replacement with no cooperation at all.

7. **The returned note**

The text is not a warning to be discarded — it travels back to the application with the synthesis response, where

`VoiceLabEngine._report_notes` logs it. A user who later wonders why an `.m4a` reference failed has the answer in the log next to the run that failed.

79.5 Reference Clips in Other Formats

Voice cloning feeds a *reference recording* to the engine. F5-TTS cleans its reference with `pydub`, which shells out to FFmpeg for anything that is not a WAV. An MP3 reference therefore died with `[WinError 2] The system cannot find the file specified` — even though the Voice Clone page advertises MP3, FLAC and OGG.

```

_WAV_SUFFIXES = (".wav", ".wave")

def _convert_to_wav(path, soundfile=None):
    try:
        sf = soundfile if soundfile is not None else _import_soundfile()
        size = os.path.getsize(path)
        stamp = int(os.path.getmtime(path))
        key = f"{path}|{size}|{stamp}".encode("utf-8", "replace")
        digest = hashlib.md5(key).hexdigest()[:12]
        destination = os.path.join(
            tempfile.gettempdir(), f"aivs_reference_{digest}.wav")
        if os.path.isfile(destination) and os.path.getsize(destination) > 0:
            return destination
        data, sample_rate = sf.read(str(path), dtype="float32", always_2d=True)
        sf.write(destination, data, int(sample_rate))
        return destination
    except Exception:
        return path

```

Explain Line by Line Code Description (Block 5)

1. Line 1: `_WAV_SUFFIXES`

The formats the engines' own pre-processing can already read. Anything else is converted before the engine sees it.

2. Lines 3-6: the cache key

Path + size + modification time, hashed to a short digest. Including size and mtime means the key changes when the file changes, so the converted copy can never be stale — the classic trap with a name-only cache. `errors="replace"` on `encode` keeps a path with unusual characters from raising.

3. Lines 7-11: the destination

A deterministic name in the temp folder, derived from the source. Deterministic means a second preview of the same voice reuses the converted file instead of converting again — which matters because converting is slower than reading.

4. Lines 12-13: the reuse check

`os.path.isfile(...)` and `os.path.getsize(...) > 0` rejects a zero-byte leftover from an interrupted write. Trusting existence alone would hand the engine an empty file and produce a mystifying error much later.

5. Lines 14-16: the conversion

Read as float32 and write as WAV with the same sample rate: no resampling, no normalisation, no quality loss beyond the original decode. The engine receives exactly the audio the user chose.

6. Lines 17-19: the honest fallback

On any failure the original path is returned. Leaving the file untouched lets the engine either handle it itself or report precisely what it cannot read — which is far more useful to the user than a silent swap to a file that may not be equivalent.

⚠ **Warning: Why not just install FFmpeg?** Because an application that requires a system-wide binary for its core feature is an application that fails on locked-down machines, and because the reference-clip path only needs a *decoder* — which the bundled `soundfile` wheel already is.

79.6 Mini Project: Prove the Fix on Your Machine

Run this in an engine's own environment (replace the path with your `tts_envs` folder) to see the difference the shim makes:

```
import os, tempfile, wave, struct

import torchaudio

from ai_voice_studio.voicelab.worker import _install_audio_io_shim

path = os.path.join(tempfile.gettempdir(), "probe.wav")
with wave.open(path, "wb") as handle:
    handle.setnchannels(1)
    handle.setsampwidth(2)
    handle.setframerate(16000)
    handle.writeframes(struct.pack("<1600h", *([0] * 1600)))

try:
    audio, rate = torchaudio.load(path)
    print("before shim: OK", audio.shape, rate)
except Exception as exc:
    print("before shim: FAILED", type(exc).__name__)

note = _install_audio_io_shim()
print("note:", note)

audio, rate = torchaudio.load(path)
print("after shim : OK", audio.shape, rate)
```

On a machine with FFmpeg the note is `None` and both loads succeed; on a machine without it the first load raises and the second returns a tensor.

Exercise:

1. Why does the shim live in the *worker* rather than in the application process? Answer in terms of process boundaries and dependencies.
2. Add (in a scratch copy) a `_soundfile_load` that ignores `channels_first`. Which engine would break first, and how would the symptom look — an exception, silence, or garbage audio?
3. Time a preview with an MP3 reference twice. Explain what the second run saves and why the cache key must include the file's mtime.
4. Convert an `.m4a` file with the current code and explain the result from the note's text rather than from an error message.

79.7 Key Takeaways

- Library behaviour changes across versions: `torchaudio` now delegates decoding to TorchCodec
 - Detect capability by *trying the operation* on a real input, then fall back only if it truly fails
 - Write shims with the full signature of the API they replace, and assign them onto the module object
 - Make workarounds idempotent and switchable with an environment variable
 - Convert user media once, with a cache key derived from path, size and mtime
 - Never silently change what the user gave you — if you cannot process it, return it untouched
 - Send notes back to the application and log them: a degraded path that works should still be explainable
-

Start Selected Recording: Out-of-Order Recording for Long Books

Learning Objectives

- Turn a “resume where you stopped” feature into a general “record these segments” range
- Model a worker’s position with a start index *and* an exclusive end index
- Build a dialog whose combo box changes content based on a sidebar radio group
- Keep the keyboard-and-screen-reader path usable when controls are rebuilt
- Remove the guard rails that fight the user’s actual workflow

80.1 The Workflow It Has to Support

A 100-chapter book. The narrator records chapter 75 first because that is the chapter they have time for today, then jumps back to chapter 12, then finally records the introduction. The old behaviour — “start where you left off and continue to the end” — cannot express any of that.

The feature is called **Start Selected Recording...** and it answers two questions separately:

1. **Where does the list of choices come from?** Either the audio files already recorded, or every file break in the project (recorded or not).
2. **How far should recording go?** Only the chosen file, or the chosen file and everything after it.

Those are two radio groups, and the first one also decides what the combo box contains. That single coupling is what makes the dialog feel obvious: one sidebar choice changes the list *and* its label.

Sidebar choice	Combo label	Combo contents	Typical intent
Select an audio file	Choose which recorded file to record again:	only files already recorded	“that take was wrong, redo it”
Select by file break	Choose which file to record by file break:	all 12 (or 100) breaks, recorded or not	“start on chapter 75 today”

80.2 The Range Contract in the Worker

Everything starts with the synthesis worker, because a dialog that cannot express “just this one” would have to lie. The worker already took a `start_index`; the feature adds an exclusive `end_index`:

```
def __init__(self, ..., start_index: int = 0, end_index: Optional[int] = None):
    ...
    self.start_index = max(0, start_index)
    # end_index = start_index + 1 records a single segment; None means
    # "to the end of the project" (the plain Start recording behaviour).
    self.end_index = end_index

def _segment_positions(self, total: int):
    if self.end_index is not None:
        stop = max(self.start_index, min(self.end_index, total))
    else:
        stop = total
    for idx in range(self.start_index, stop):
        yield idx
```

Explain Line by Line Code Description (Block 1)

1. **Lines 1-3:** the two parameters

Both are optional, so every existing call site keeps working *unchanged*: no argument means “start at the first pending segment and go to the end”, which is exactly the old behaviour. Backwards compatibility by default value is the cheapest kind of refactor.

2. **Line 5:** `self.start_index = max(0, start_index)`

A negative index would silently mean “from the end” in Python slicing rules, which is not what “start at segment -1” should ever do. Clamping to 0 makes the invalid input harmless.

3. **Line 8:** the comment that defines the vocabulary

One sentence that makes `end_index` self-documenting: it is *exclusive*, and `start_index + 1` is the idiom for a single segment. Without that line, the next reader would have to guess whether the end is inclusive — and would guess wrong half the time.

4. **Line 14:** `stop = max(self.start_index, min(self.end_index, total))`

Two clamps in one expression: never go past the end of the project (`min`), and never produce an empty range when the caller passes an end before the start (`max`). A manual `if` for each would be four lines doing exactly this.

5. **Lines 17-18:** a generator

The worker iterates positions through `range(...)`, which keeps the loop body free of index arithmetic and index-boundary bugs. “What percent am I done?” is derived from the range, not from counters that can drift.

💡 **Tip:** Exclusive ends are a convention worth adopting deliberately: `range(a, b)`, `list[a:b]` and this worker all agree, so no reader has to remember a special case.

80.3 One Entry per Segment, Built Purely

The dialog needs, for each segment, three things: its 0-based position, its display name, and the audio file it already has (if any). Building that mapping is pure data work — no wx, no disk scanning beyond the files the caller passes in — so it lives in its own module-level function where it can be tested directly.

```
def _picker_entries(data: dict, saved: list[str]) -> list[dict]:
    from ..util import sanitize_filename

    segments = data.get("segments", []) or []
    entries = []
    by_stem: dict[str, int] = {}
    by_name: dict[str, int] = {}
    for position, seg in enumerate(segments):
        title = str(seg.get("title") or f"segment {position + 1}")
        entries.append({"position": position, "title": title, "file": None})
        by_stem.setdefault(sanitize_filename(title, 80), position)
        stored = os.path.basename(str(seg.get("saved") or ""))
        if stored:
            by_name.setdefault(stored, position)
    for name in saved:
        position = by_name.get(name)
        if position is None:
            position = by_stem.get(os.path.splitext(name)[0])
        if position is None:
            continue # A stray file that no file break refers to.
        if entries[position]["file"] is None:
            entries[position]["file"] = name
    return entries
```

Explain Line by Line Code Description (Block 2)

1. **Lines 4-6:** the starting structures

One entry per segment, in document order, with `file` unknown for now; plus two lookup dictionaries that will be filled in the same pass. Doing it in one pass means the two mapping strategies cannot disagree about positions.

2. **Line 7:** reading the project data

`data.get("segments", [])` or `[]` tolerates both a missing key and an explicit `None`. Project files are user-editable JSON, so “absent” and “null” must both be survivable.

3. **Lines 8-9:** the title fallback

A segment without a title gets “segment 7”. A visible, correct placeholder beats an empty combo row that the screen reader announces as blank.

4. **Line 10:** `sanitize_filename(title, 80)`

The same function the recorder used to name audio files, with the same length limit. Reusing it is what makes `by_stem` a *valid* fallback: the audio file’s name minus its extension equals the sanitised title only if both sides agree on sanitation — and they do, because it is one function.

5. **Lines 11-13:** the authoritative mapping

A segment’s `saved` field records the file it produced, so that path is the primary link. `os.path.basename` normalises a full path to a name, and `setdefault` keeps the first mapping if a project contains duplicates.

6. **Lines 14-16:** matching the files on disk

Primary key is the recorded name; the secondary key is the sanitised title. Two strategies, because a metadata field can be out of date while the file names remain correct — the fallback rescues exactly that case.

7. **Lines 17-18:** the deliberate `continue`

An audio file that no break refers to is skipped rather than listed. The mode’s promise is “files that belong to this project”, and a stray file has no position to record from — offering it would create a choice the worker cannot execute.

8. **Line 19:** first-wins assignment

`if entries[position]["file"] is None` keeps the earliest name when two files match one segment (for example an old take and a new one). Predictable, order-dependent, and easy to test.

80.4 The Dialog: Two Radio Groups, One Combo

```
class _StartRecordingDialog(wx.Dialog):
    def __init__(self, parent, entries):
        super().__init__(parent, title="Start selected recording", size=(600, 320))
        self._entries = [dict(entry) for entry in entries]

        outer = wx.BoxSizer(wx.HORIZONTAL)
        side_panel = wx.Panel(self)
        sidebar = wx.BoxSizer(wx.VERTICAL)
        self.file_radio = wx.RadioButton(
            side_panel, label="Select an audio file", style=wx.RB_GROUP)
        self.break_radio = wx.RadioButton(side_panel, label="Select by file break")
        for radio in (self.file_radio, self.break_radio):
            radio.SetName(radio.GetLabel())
            sidebar.Add(radio, 0, wx.ALL, 6)
        side_panel.SetSizerAndFit(sidebar)
        side_panel.SetMinSize((240, -1))
        side_panel.SetName("Recording selection mode")
        if not any(entry.get("file") for entry in self._entries):
            self.break_radio.SetValue(True)
        outer.Add(side_panel, 0, wx.EXPAND | wx.ALL, 8)
```

Explain Line by Line Code Description (Block 3)

1. **Line 3:** the constructor

The dialog receives *already computed* entries. It knows nothing about project files, the disk or the worker — which is why every UI state in this chapter can be tested without a project fixture.

2. **Line 4:** copying the entries

`dict(entry)` per item means the dialog cannot modify the caller's list. Defensive copying at a boundary like this costs microseconds and prevents a class of bug that is painful to trace.

3. **Lines 6-9:** the sidebar

A horizontal sizer with a narrow left panel: the layout the requirements described. The first radio carries `wx.RB_GROUP`, which is what makes wxPython treat the two as *one* mutually exclusive group rather than two independent toggles.

4. **Lines 10-12:** accessible names

`SetName(radio.GetLabel())` is the difference between NVDA announcing “Select an audio file, radio button, checked” and “radio button, checked”. Naming every control from its own label keeps the label and the accessible name from drifting apart.

5. **Line 13:** `SetSizerAndFit`

The panel sizes itself to its content, *then* the minimum width is raised to 240 px. Fixing a sidebar width avoids the layout reflowing when the combobox content below changes.

6. **Lines 15-17:** the sensible default

If nothing has been recorded yet, the file mode has an empty list and would show the user a dead combo, so “Select by file break” is pre-selected. This is the dialog doing the reasoning the user would otherwise have to do — and it is also why the workflow works on a brand-new project.

7. **Line 18:** `outer.Add(side_panel, 0, wx.EXPAND | wx.ALL, 8)`

Proportion `0`: the sidebar keeps its width while the combo column takes the remaining space. Getting proportions right here is what keeps the dialog from looking stretched when a project has 700 segments.

```

def _fill_combo(self):
    self.combo.Clear()
    if self.file_radio.GetValue():
        self.choice_label.SetLabel("Choose which recorded file to record again:")
        update_accessible_name(self.combo, "Recorded audio files")
        self.combo.SetToolTip(
            "Audio files that are already recorded in this project")
        items = [(entry["file"], entry["position"])
                  for entry in self._entries if entry.get("file")]
    else:
        self.choice_label.SetLabel("Choose which file to record by file break:")
        update_accessible_name(self.combo, "Project file breaks")
        self.combo.SetToolTip(
            "Every file break of the project, recorded or not")
        items = [(entry["title"], entry["position"])
                  for entry in self._entries]
    for label, position in items:
        self.combo.Append(str(label), position)
    if self.combo.GetCount():
        self.combo.SetSelection(0)
    self.combo.Enable(self.combo.GetCount() > 0)
    ok_btn = self.FindWindowById(wx.ID_OK)
    if ok_btn is not None:
        ok_btn.Enable(self.combo.GetCount() > 0)
    self.Layout()

```

Explain Line by Line Code Description (Block 4)

1. **Line 2:** `self.combo.Clear()`

Refilling always starts by emptying. Forgetting this is the classic radio-button bug: the two lists concatenate and the combo shows recorded files followed by every break.

2. **Lines 3-9:** the recorded-files branch

Label, accessible name and tooltip are set together with the items, so the three can never describe different things. The list comprehension filters on `entry.get("file")` — the same truth the dialog used to pick its default mode.

3. **Lines 10-16:** the file-break branch

Every break, unconditionally. This is the branch that makes chapter 75 of a 100-chapter book recordable today: the list is the *project's plan*, not its progress.

4. **Lines 17-18:** labels and client data

The displayed string is what the user asked to see; the appended `position` is what the worker needs. Keeping both in the combo means the dialog never has to look anything up again after the user chooses.

5. **Lines 19-22:** selection and enablement

Selecting the first item, then disabling the combo and **the OK button** when the list is empty. Disabling OK is the important half: a dialog that can be confirmed with no choice is a dialog that fails later with a confusing message.

6. **Line 23:** `self.Layout()`

A label that grows from “Choose which recorded file...” to “Choose which file to record by file break...” changes the required width. Laying out after the change is what stops the text being clipped — a small detail that is very visible to a user with a screen reader reading a truncated label.

80.5 Wiring It Together, and Deleting the Guard

```
def _restart_selected(self):
    recent = self._require_selected()
    if not recent:
        return
    try:
        data = project.load_project(recent["path"])
    except (OSError, ValueError, KeyError) as exc:
        wx.MessageBox(f"Could not open the project: {exc}",
                      "Start selected recording", style=wx.OK | wx.ICON_ERROR)
        return
    saved = project.recorded_files(recent["path"])
    # Deliberately no "has anything been recorded?" guard: a long book may be
    # recorded out of order (chapter 75 of 100 today, the rest later), so the
    # picker always opens.
    if not data.get("segments"):
        wx.MessageBox(
            "This project has no file breaks yet, so there is nothing to record.",
            "Start selected recording", style=wx.OK | wx.ICON_INFORMATION)
        return
    dlg = _StartRecordingDialog(self, _picker_entries(data, saved))
    try:
        if dlg.ShowModal() != wx.ID_OK:
            return
        start_index = dlg.start_index()
        only_selected = dlg.only_selected()
        chosen = dlg.chosen_file()
    finally:
        dlg.Destroy()
    if start_index is None:
        return
    if chosen:
        project.remove_recording(recent["path"], chosen)
    self._open_recording(recent["path"], start_index=start_index,
                        single_segment=only_selected)
```

Explain Line by Line Code Description (Block 5)

1. **Lines 2-3:** the selection check

This remains, because the command operates on *a* project: with nothing selected there is no context to ask about. It reports why instead of failing silently.

2. **Lines 4-9:** loading the project

A narrow tuple of exception types (`OSError` , `ValueError` , `KeyError`) rather than a bare `except` : a corrupt project is reported, while a genuine programming error (`AttributeError` , `TypeError`) still surfaces as a crash worth fixing.

3. **Line 10:** the files on disk

`project.recorded_files()` reads what is actually in the project folder, not just the metadata. This is why “Select an audio file” shows everything recorded, including files whose metadata was lost.

4. **Lines 11-16:** the guard that was removed, and the one that stayed

The comment records the product decision: the old “this project has no recorded files yet” message made out-of-order recording impossible and was deleted. What remains is a real precondition — a project whose document was never split has no segments to choose from, so there is genuinely nothing to do. Distinguishing “the user has not finished everything” from “the user cannot start” is the whole point.

5. **Lines 17-25:** the modal call

Three values are read out of the dialog *before* `Destroy()` , in a `try/finally` so a failed read still destroys the window. Reading widget state after destroying the window is a use-after-free waiting to happen, and wxPython reports it in unhelpful ways.

6. **Lines 26-31:** delete, then record

The chosen recording is removed first so the segment behaves exactly as if it had never been recorded: the worker sees a pending segment. The call then passes the two things the picker produced, and nothing else — the recording window remains the single place where engine and voice are chosen.

 **Note: Why delete instead of overwrite?** Because the project, the `processed_text.json` log and the DAISY builders all key off “is this segment recorded?”. Removing the file marks the segment pending in one step, so every downstream consumer agrees without any of them needing to know about this dialog.

80.6 Handing the Choice to the Recording Window

```
if self._start_index is None:
    start_index = project.first_pending_index(self.project_dir)
else:
    start_index = min(self._start_index, len(self._segments))

end_index = start_index + 1 if self._single_segment else None

worker = SynthesisWorker(
    ...,
    start_index=start_index,
    end_index=end_index,
)
```

Explain Line by Line Code Description (Block 6)

1. **Lines 1-4:** the index

`None` means the user did not choose: keep the historical behaviour and resume at the first pending segment. Otherwise use the choice, clamped to the segment count so a project edited between opening the dialog and pressing Start cannot produce an out-of-range index.

2. **Line 6:** one line that encodes the scope

`start_index + 1 if self._single_segment else None` — “just this one” becomes an exclusive end one past the start; “from here” becomes `None`, meaning to the end. The dialog’s radio buttons therefore need no further glue: they map onto the worker’s contract directly.

3. **Lines 8-12:** construction

Both values travel as constructor arguments. Because the worker validates and clamps them (80.2), the dialog can hand over what the user chose without a defensive re-check — validation lives with the code that owns the range.

⚠ Warning: Single-segment recording has one consequence worth knowing: when the run stops after its one segment, nothing is queued behind it, so “Stop” simply means “this take is done”. Users who expect the old continue-to-the-end behaviour must pick *Record all files from here*.

80.7 Mini Project: Drive the Range Without a GUI

The worker range is pure logic, so it can be exercised in five lines:

```

from ai_voice_studio.jobs.synthesizer import SynthesisWorker

def positions(start, end, total):
    worker = SynthesisWorker.__new__(SynthesisWorker)  # no thread, no wx
    worker.start_index = start
    worker.end_index = end
    return list(worker._segment_positions(total))

print(positions(0, None, 5))  # everything
print(positions(3, 4, 5))    # only segment 4
print(positions(3, None, 5))  # segment 4 to the end
print(positions(4, 3, 5))    # empty: end before start
print(positions(2, 99, 5))   # clamped to the project's end

```

Exercise:

1. Why is `end_index` exclusive rather than inclusive? Give both a readability argument and a bug the inclusive convention invites.
2. Add a third sidebar mode, “Select by page range”, to a scratch copy. What must change in `_picker_entries`, the dialog and the worker?
3. Write a test that builds a 4-segment project with no recordings and asserts the dialog opens in file-break mode and that `wx.MessageBox` is never called. (This is the regression test for the guard that was removed.)
4. Explain why the dialog reads its three values before `Destroy()`, and what symptom you would expect if it did not.

80.8 Key Takeaways

- Give the worker an explicit range (start + exclusive end) instead of a “resume” flag; the dialog then becomes a thin front end
- Keep list-building in a pure function so every UI state is testable without a project fixture
- One radio group can legitimately change another control’s contents — set the label, accessible name and items together
- Choose the default that keeps a brand-new project usable, and disable OK when a list is genuinely empty
- Delete guards that encode a workflow assumption the user did not ask for; keep the ones that reflect a real precondition
- Mark a segment pending by removing its file, so every downstream consumer agrees in one step

A First-Launch Terms Dialog That Cannot Be Bypassed

Learning Objectives

- Gate application start-up behind a modal dialog in a way that survives every exit path
 - Remember acceptance per application version, so updates re-prompt and daily use does not
 - Gate buttons on a group of checkboxes, and let one master checkbox drive the rest
 - Handle the wx traps: Escape, the close box, **Alt+F4**, and modal vs non-modal code paths
 - Test a modal dialog without a user, and why “shown non-modally” matters
-

81.1 The Requirement, Stated Precisely

The application synthesises speech from arbitrary documents and clones voices. The requirement was a first-launch dialog with six acknowledgements that:

1. must be **all ticked** to proceed;
2. enables its **I Agree** and **Disagree (Close Program)** buttons only once every box is ticked;
3. must **not** let the user through by refusing, by Escape, or by forcibly closing the dialog;
4. must never appear again after agreement — **until the next update or a fresh install**.

Requirement 4 is the subtle one: “once per version, forever” is a completely different store from “once per installation”.

81.2 Versioned Acceptance in Settings

```
APP_VERSION = "2026.3.1"

# Version of the first-launch terms/acceptance dialog. Bump this on every
# release so the dialog shows again after an update.
TERMS_VERSION = "2026.3.1"

def terms_current(settings) -> bool:
    return settings.get("terms_version", "") == TERMS_VERSION

def record_acceptance(settings) -> None:
    settings.set("terms_version", TERMS_VERSION)
    settings.save()
```

Explain Line by Line Code Description (Block 1)

1. **Lines 1-5:** two version constants, on purpose

The application version and the terms version can move independently: a bug-fix release may ship without changing the legal text, a podgy release may change only the terms. Keeping them separate means changing the wording is *one line*, and no one has to reason about which releases “count”.

2. **Lines 8-10:** `terms_current`

A pure comparison against a settings value, with an empty-default for the first run. It returns `False` for anything that is not an exact match — including a settings file from an older version and a corrupt value. That default is what makes an update prompt again without any migration code.

3. **Lines 13-15:** `record_acceptance`

Writes *and saves immediately*. Deferring the save to a later “save settings on exit” would lose the acceptance in exactly the case that matters — the user accepts and then the application crashes for an unrelated reason.

⚠ Warning: Uninstalling does not delete user data. Settings live in `%APPDATA%\AIVoiceStudio`, which the uninstaller deliberately leaves alone (it contains the user’s projects). So a reinstall that ships the *same* terms version will *not* re-prompt. That is why the version must be bumped — it is the only mechanism that re-prompts an existing user.

81.3 The Dialog, Control by Control

```
class AcceptanceDialog(wx.Dialog):
    def __init__(self, parent=None):
        super().__init__(
            parent,
            title="Welcome to AI Voice Studio - Terms of Use",
            style=wx.DEFAULT_DIALOG_STYLE & ~wx.CLOSE_BOX,
        )
        self._accepted = False

        panel = wx.Panel(self)
        sizer = wx.BoxSizer(wx.VERTICAL)

        intro = wx.StaticText(panel, label=(
            "Welcome to AI Voice Studio.\n\n"
            "Please read each point below carefully. You must accept "
            "all of them to use this application.))
        intro.Wrap(560)
        sizer.Add(intro, 0, wx.ALL, 10)
```

Explain Line by Line Code Description (Block 2)

1. **Lines 3-8:** the style expression

`wx.DEFAULT_DIALOG_STYLE & ~wx.CLOSE_BOX` clears the close-box bit from a set of style flags. This removes the title-bar × button on platforms where the close box is part of that style — removing the obvious way to dismiss the dialog without deciding. It is a hint, not a guarantee, which is why the close *event* is also handled (81.5).

2. **Line 9:** `self._accepted = False`

State is initialised to the safe value. If any code path forgets to set it, the result is “not accepted”, and the application does not start — the failure mode leans in the safe direction.

3. **Lines 11-12:** panel and vertical sizer

A `wx.Panel` gives the contents a proper background and correct tab order; the vertical sizer stacks the terms and lets `Fit` size the dialog around them.

4. **Lines 14-19:** the introduction

Short, and it says what is required (“You must accept all of them”). A user who reads one line before ticking boxes should learn from it what the buttons will do.

5. **Line 20:** `intro.Wrap(560)`

A fixed wrap width keeps the dialog’s shape stable regardless of the platform’s font and DPI. Without it, the dialog is as wide as the longest sentence — which on a 100 % or 150 % DPI machine is off-screen.

```

self._checkboxes = []
for index, text in enumerate(_TERMS, start=1):
    wrapped = "\n".join(textwrap.wrap(text, width=90))
    box = wx.CheckBox(panel, label=f"{index}. {wrapped}")
    box.Bind(wx.EVT_CHECKBOX, self._on_toggle)
    self._checkboxes.append(box)
    sizer.Add(box, 0, wx.LEFT | wx.RIGHT | wx.BOTTOM, 10)

self._agree_all = wx.CheckBox(panel, label=_AGREE_ALL_LABEL)
self._agree_all.SetFont(self._agree_all.GetFont().Bold())
self._agree_all.Bind(wx.EVT_CHECKBOX, self._on_agree_all)
self._checkboxes.append(self._agree_all)
sizer.Add(self._agree_all, 0, wx.LEFT | wx.RIGHT | wx.TOP, 10)

```

Explain Line by Line Code Description (Block 3)

1. **Line 1:** one list holds every checkbox

The term checkboxes *and* the agree-all one live in `self._checkboxes`. That single collection is what makes “is everything ticked?” a one-line expression — and it is also the reason the property below has to exist.

2. **Line 3:** `textwrap.wrap(text, width=90)`

The terms are long sentences. wxPython’s `wx.CheckBox` has no `Wrap()` method (unlike `StaticText`), so a long label is simply cut off or produces an absurdly wide dialog. Wrapping the string *before* creating the control — and joining the lines with `\n` — is the portable way to get a multi-line checkbox. The newline also preserves the full text for a screen reader.

3. **Line 4:** the numbered label

“1.” through “6.” lets a screen-reader user say “I disagree with point 3” instead of reading the sentence aloud. The two spaces after the number keep the text visually aligned.

4. **Line 5:** binding the handler

Every box gets `_on_toggle`, including the master one bound separately below, so any change updates the button state immediately. Re-using one handler is what keeps the gating logic in a single place.

5. **Lines 9-12:** the master checkbox

Bold, because it means “all of the above”, and appended into the same list so that “all ticked” includes it. The requirement was explicit that the agree-all box itself is required — it is a shortcut for the other six, not a replacement for them.

```

btn_row = wx.BoxSizer(wx.HORIZONTAL)
self._agree_btn = wx.Button(panel, wx.ID_OK, _AGREE_LABEL)
self._disagree_btn = wx.Button(panel, wx.ID_CANCEL, _DISAGREE_LABEL)
self._agree_btn.Bind(wx.EVT_BUTTON, self._on_agree)
self._disagree_btn.Bind(wx.EVT_BUTTON, self._on_disagree)
btn_row.Add(self._disagree_btn, 0, wx.RIGHT, 8)
btn_row.Add(self._agree_btn, 0, wx.LEFT, 8)
sizer.Add(btn_row, 0, wx.ALIGN_CENTRE | wx.ALL, 12)

panel.SetSizer(sizer)
sizer.Fit(self)
self.CentreOnScreen()

self.Bind(wx.EVT_CHAR_HOOK, self._on_char_hook)
self.Bind(wx.EVT_CLOSE, self._on_close)
self._on_toggle(None)

```

Explain Line by Line Code Description (Block 4)

1. **Lines 2-3:** explicit ids, explicit labels

`wx.ID_OK` and `wx.ID_CANCEL` are used for the standard “modal result” plumbing, while the visible text says exactly what each button does. *Disagree (Close Program)* tells the user the consequence before they press it — the honest alternative to “Cancel”.

2. **Line 8:** `wx.ALIGN_CENTRE`

(Native wx spelling.) Centring the two buttons gives the dialog a stable, symmetric shape that does not shift when one of them is disabled.

3. **Line 11:** `sizer.Fit(self)`

Sizes the dialog around its contents once, rather than hard-coding a pixel size. Text metrics differ between platforms and DPI settings; fitting is the only thing that behaves the same everywhere.

4. **Line 12:** `self.CentreOnScreen()`

On a multi-monitor setup a centred-on-parent dialog may open on a screen the user is not looking at. “On screen” (with no parent, since this dialog has none) is the right choice for a dialog that appears before the main window exists.

5. **Lines 14-15:** the two escape hatches

`EVT_CHAR_HOOK` sees keystrokes *before* any child control, which is what makes an Escape handler reliable even when focus is inside a checkbox; `EVT_CLOSE` catches the window-close path. Both are handled in 81.5.

6. **Line 16:** the initial gating

Calling the handler once at the end applies the rule to the freshly built state. Without it the buttons would be enabled until something is clicked — an inconsistency the user can see.

81.4 Gating, and a Master Checkbox

```
@property
def _term_checkboxes(self):
    """Every checkbox except the final "agree with all" one."""
    return self._checkboxes[:-1]

def _on_agree_all(self, event) -> None:
    value = self._agree_all.GetValue()
    for box in self._term_checkboxes:
        box.SetValue(value)
    self._on_toggle(None)

def _all_checked(self) -> bool:
    return all(box.GetValue() for box in self._checkboxes)

def _on_toggle(self, event) -> None:
    ready = self._all_checked()
    self._agree_btn.Enable(ready)
    self._disagree_btn.Enable(ready)
```

Explain Line by Line Code Description (Block 5)

1. **Lines 2-4:** the slicing property

A name for “everything except the last one”, instead of `[:-1]` scattered through the class. If a future release adds a seventh term, only this property changes — the master checkbox still drives exactly the term list.

2. **Lines 6-9:** the master handler

Reads the master box’s own value and copies it to every term box. Copying (rather than toggling) makes the operation idempotent: clicking the master twice leaves all boxes consistent, which is what a user expects and what makes the behaviour easy to test.

3. **Line 10:** re-evaluating afterwards

`_on_toggle(None)` is called directly. Programmatic changes (`SetValue`) do *not* fire `EVT_CHECKBOX`, so without this call the buttons would stay disabled after the user ticked the master box — a real bug caught while testing this class.

4. **Lines 12-13:** the single source of truth

`all(...)` over the whole list, including the master box. One expression answers the only question that matters, so “all of them, plus the agree-all one” cannot drift between code paths.

5. **Lines 15-18:** the gating itself

Both buttons are enabled together. That is deliberate: the user should be able to make a decision either way once they have read the terms, and disabling only one of them would make “Disagree” unreachable and look like a trap.

6. **The parameter is `event`, and it is unused**

The handler is called both by events and directly with `None`, so the signature must accept the event object. A comment or a default value would do; passing `None` is the simplest convention and is used consistently here.

81.5 Every Exit Path Leads to “Not Accepted”

```
def _on_agree(self, event) -> None:
    if not self._all_checked():
        return
    self._accepted = True
    self._finish(wx.ID_OK)

def _on_disagree(self, event) -> None:
    self._accepted = False
    self._finish(wx.ID_NO)

def _finish(self, code: int) -> None:
    if self.IsModal():
        self.EndModal(code)
    else:
        self.Close()

def _on_char_hook(self, event) -> None:
    if event.GetKeyCode() == wx.WXK_ESCAPE:
        return
    event.Skip()

def _on_close(self, event) -> None:
    if not self._accepted:
        self._accepted = False
        if self.IsModal():
            self.EndModal(wx.ID_NO)
        else:
            event.Skip()
    return
    event.Skip()
```

Explain Line by Line Code Description (Block 6)

1. **Lines 2-3:** the re-check

The button is disabled when the conditions are not met, but the handler checks anyway. UI state and logic state are different things — a disabled button can still be activated by an accelerator, a test, or a stray programmatic

`ProcessEvent`. The check costs one line and removes that class of accident.

2. **Lines 7-9:** disagreement

`wx.ID_NO` (not `ID_CANCEL`) is returned, so the caller can distinguish “the user said no” from “something else closed this”. Callers that only care whether it is OK are unaffected.

3. **Lines 11-15:** `_finish`, the modal/non-modal split

The important detail: `EndModal` on a dialog that was shown with `Show()` raises an assertion, and tests show dialogs non-modally. Asking `IsModal()` makes one code path serve the application (modal) and the tests (non-modal) without a test-only hook.

4. **Lines 17-20:** the Escape hook

Escape is swallowed — the handler returns without calling `event.Skip()`, so no further handler ever sees the key. Swallowing is the correct response here: refusing the terms is a decision, and it should require pressing the button that says what it does. Every other key is forwarded with `event.Skip()` so normal navigation, Tab and Space keep working.

5. **Lines 22-29:** the close handler

Reaching this method without acceptance means the user closed the window (title-bar ×, Alt+F4, a window manager command). The dialog then ends with `ID_NO` — closing is treated as disagreement, which is exactly the requirement that force-closing must not launch the application.

6. **Line 30:** the accepted path

If the user already accepted, the close event is passed along normally (`event.Skip()`) so the window closes without a second result being reported.

81.6 Wiring It Into Start-Up

```
def _require_terms_acceptance(settings: Settings) -> bool:
    if terms_current(settings):
        return True
    log = logging.getLogger("main")
    log.info("Showing first-launch terms acceptance dialog")
    app = wx.App(False)
    dialog = AcceptanceDialog(None)
    try:
        accepted = dialog.ShowModal() == wx.ID_OK and dialog.accepted
    finally:
        dialog.Destroy()
    if accepted:
        record_acceptance(settings)
        log.info("Terms accepted (version %s)", settings.get("terms_version"))
    else:
        log.info("Terms not accepted - application will not start")
    return accepted

def main() -> int:
    settings = Settings()
    developer_mode = settings.get("developer_mode", False)
    _setup_logging(developer_mode)

    log = logging.getLogger("main")
    log.info("AI Voice Studio starting (Python %s)", sys.version.split()[0])

    if not _require_terms_acceptance(settings):
        return 0

    _init_python_runtime()
    _init_addons()
    _warm_package_cache()

    app = wx.App(False)
    store = ModelStore()
    frame = MainFrame(None, settings, store)
    apply_theme(frame, settings.theme)
    frame.Show()
    return app.MainLoop() or 0
```

Explain Line by Line Code Description (Block 7)

1. **Lines 2-3:** the fast path

An accepted user pays one settings read. Nothing is imported and no window is created, which is what keeps the dialog from slowing down every launch for the rest of the version's life.

2. **Lines 5-7:** a `wx.App` for the dialog

`wxPython` needs an application object with a message loop before any window can be created. Creating it here — and letting the local reference go when the function returns — means the main window's own `wx.App` in `main()` can be created afterwards without conflict (verified: a second `wx.App(False)` succeeds once the first is released).

3. **Line 8:** reading both the result and the flag

`ShowModal() == wx.ID_OK and dialog.accepted`. Two independent conditions, which is the belt-and-braces version of a security-relevant check: a future refactor that returns `ID_OK` from the wrong place still cannot let the application start.

4. **Lines 9-10:** `try/finally` with `Destroy()`

Standard `wx` hygiene: the native window is released on every path, including an exception raised while showing the dialog.

5. **Lines 11-15:** recording the outcome either way

Both branches log. The acceptance line includes the version that was accepted, which is the single most useful line when a user later asks “why did the dialog come back?”. The decline line makes a support question (“it closes immediately”) answerable from the log.

6. **Lines 22-23:** before any subsystem

This position is deliberate. The check runs *after* logging is configured (so the dialog can be logged) and *before* the runtime, addons and package probes (so nothing expensive or network-touching happens for a user who has not accepted).

7. **Line 25:** the early return

Returning `0` from `main()` means a clean exit code for “the user chose not to continue” — not an error. An installer or launcher that checks the exit code then does not report a failure.

⚠ Warning: A real bug this wiring produced once: an early version logged from a helper that assumed the module-level `log` object, which did not exist at that point in start-up — the frozen build died instantly with `NameError`. The fix was to obtain the logger locally (`logging.getLogger("main")`) in the function, and the lesson is general: code that runs *before* initialisation completes may only rely on what it obtained itself. A frozen smoke test that launches the packaged executable with an isolated data folder is what catches this class of bug.

81.7 Testing a Modal Dialog Without a User

Behaviour to prove	How the test drives it
Buttons disabled until every box is ticked	build the dialog, assert <code>IsEnabled()</code> is <code>False</code> , tick all boxes, call the toggle handler, assert <code>True</code>
Agree records the version	tick everything, call <code>_on_agree</code> , assert <code>settings.get("terms_version")</code>
Disagree records nothing	tick everything, call <code>_on_disagree</code> , assert the setting is still empty
Master box drives the terms	<code>SetValue(True)</code> on the master, call its handler, assert all six term boxes are ticked; then repeat with <code>False</code>
Escape cannot dismiss	send <code>EVT_CHAR_HOOK</code> with <code>WXK_ESCAPE</code> and assert no result was produced
A new version re-prompts	<code>terms_current()</code> against a settings object holding an older version string

 **Tip:** Note how the tests call the handlers directly rather than simulating clicks. Because the handlers take an `event` they never use, `handler(None)` works — a small design decision that makes a whole class of dialog testable without a robot.

81.8 Mini Project: A Settings-Backed One-Time Prompt

The same pattern, minus wx, for anything you build yourself:

```

import json, os

PROMPT_VERSION = "3"
PATH = os.path.join(os.path.expanduser("~"), ".my_tool_state.json")

def load_state():
    try:
        with open(PATH, encoding="utf-8") as handle:
            return json.load(handle)
    except (OSError, ValueError):
        return {}

def save_state(state):
    with open(PATH, "w", encoding="utf-8") as handle:
        json.dump(state, handle, indent=2)

def needs_prompt():
    return load_state().get("accepted_version") != PROMPT_VERSION

def mark_accepted():
    state = load_state()
    state["accepted_version"] = PROMPT_VERSION
    save_state(state)

```

Bump `PROMPT_VERSION` when the text changes; a new installation (no state file) prompts automatically because the missing key never equals the version.

Exercise:

1. Why is the close event treated as disagreement rather than being ignored? What would “ignore” mean for the user?
2. Add a seventh term in a scratch copy. Which lines of the dialog change, and which do not? Explain using `_term_checkboxes`.
3. Delete the `_on_toggle(None)` call at the end of `__init__` and describe the visible symptom on a real launch.
4. Write a test that proves a settings file carrying the previous terms version re-prompts, and one that proves the current version does not. Run both.

81.9 Key Takeaways

- Gate on a **versioned** acceptance value, not on a boolean, if future updates must re-prompt
- Set the safe default first and check twice at the boundary: style tricks and disabled buttons are hints, not guarantees
- Handle Escape with `EVT_CHAR_HOOK` and the close box with `EVT_CLOSE`; both must lead to “not accepted”
- Programmatic `SetValue` fires no events — call the state handler yourself

- Branch on `IsModal()` so one implementation serves the application and the tests
- Run the prompt before any subsystem initialisation, and log both outcomes with the version
- Uninstallers leave user data behind: the version bump is the only reliable re-prompt

Licensing & Compliance for a Multi-Engine Audio Application

Learning Objectives

- Separate the licence of an application, of its libraries, of model code and of model weights
- Record licence names as data, next to the thing they apply to
- Ship the licence texts where users and auditors will actually find them
- Build a commercial-use matrix a non-lawyer can act on
- Audit your own dependency set before a release, not after a complaint

82.1 Four Layers of Licensing in One Application

A TTS studio is not “licensed” in the singular. Four layers stack, and each can carry different terms:

Layer	Example in this project	Typical licence
Application	AI Voice Studio itself	GPL-3.0
Python libraries	wxPython, NumPy, sherpa-onnx, ONNX Runtime, requests, pypdf, python-docx	MIT / BSD-3 / Apache-2.0 / wxWindows
Model code	f5-tts, pocket-tts, transformers, Bark’s repo	MIT / Apache-2.0
Model weights & voices	F5-TTS checkpoints, MMS, AISHELL3 data, Pocket TTS voices	CC-BY-NC-4.0, per-voice terms, dataset licences

The fourth layer is where the surprises live. F5-TTS is the canonical example in this project: the *code* is MIT, but the released **weights are CC-BY-NC-4.0**, which forbids commercial use. Anyone who reads only the repository’s main licence file and ships a product will be wrong.

 **Note:** Nothing in this chapter is legal advice, and none of it replaces reading a licence before shipping. The engineering point is narrower and fully actionable: *record which licence applies to which component, and put that information in front of the user*. Guessing is how projects end up unable to answer the simplest question a distributor asks.

82.2 Licences as Data, Next to the Component

The engine registry carries a `license` field per engine, and the JSON model catalog carries one per TTS family. This is the single most useful decision in the whole chapter: the licence lives *with the component it describes*, so it cannot silently become stale in a separate document.

```
{
  "id": "f5tts",
  "name": "F5-TTS (transformer)",
  "package": "f5-tts",
  "packages": ("f5-tts", "torch", "torchaudio", "soundfile"),
  "license": "MIT (code); CC-BY-NC-4.0 (weights)",
  "homepage": "https://github.com/SWivid/F5-TTS",
}
```

Explain Line by Line Code Description (Block 1)

1. **Line 5:** the two-part licence string

“MIT (code); CC-BY-NC-4.0 (weights)” names both layers and marks which is which. A single word here would be either misleading or useless: MIT alone hides the non-commercial weights, and CC-BY-NC-4.0 alone would deny the permissive code.

2. **Line 6:** `homepage`

The URL makes every claim checkable in one click, for a user and for an auditor. A licence statement without a source is a rumour.

3. **Why in the registry, not in a document?**

Because the registry is what the install dialog reads. When the user presses “Install engine...”, the sentence they see comes from this field — the disclosure happens at the moment of choice rather than in a file nobody opened.

The same pattern in the model catalog, for the sherpa-onnx families:

```
{
  "id": "piper",      "license": "MIT"},
  "id": "kokoro",    "license": "Apache-2.0"},
  "id": "kitten",    "license": "Apache-2.0"},
  "id": "vits_zh",   "license": "Apache-2.0"},
  "id": "matcha",    "license": "CC-BY-4.0 (icefall conversion); LJSpeech/other datasets may have their own terms"},
  "id": "omnivoice",  "license": "Apache-2.0"},
  "id": "omnivoice_server", "license": "Apache-2.0 / MIT"},
  "id": "sapi5",      "license": "Windows (built in)"},
  "id": "windows_core", "license": "Windows (built in)"}
```

Explain Line by Line Code Description (Block 2)

1. **Lines 1-4:** the permissive families

Short, unambiguous strings. Uniform values are still recorded per entry, because the moment one of them changes (a new checkpoint under a different licence) the data is already the right shape.

2. **Line 5:** Matcha-TTS

The longest string, and the most honest: the *conversion* is CC-BY-4.0 while the training data behind the checkpoints may impose its own conditions. Saying “may have their own terms” is not vagueness — it is the accurate statement, and it warns an integrator to check.

3. **Lines 6-7:** the OmniVoice pair

Two front-ends over one project, with slightly different terms — which is exactly why the licence is per entry and not per “voice engine”.

4. **Lines 8-9:** Windows voices

“Windows (built in)” is the truthful answer for SAPI5 and Windows Core voices: they are licensed by the operating system or the voice vendor, not by this application. This is the case the first-launch terms call out explicitly — the user must own the right to use those voices.

82.3 Where the Licences Have to Appear

Recording the data is half the job; the other half is putting it where people are. The project ships it in four places, each with a different audience:

Artifact	Contents	Audience
<code>LICENSE</code>	the application’s GPL-3.0 text plus a third-party summary	anyone reading the repository
<code>docs/THIRD-PARTY-LICENSES.html</code>	summary table of every bundled component, full texts for Apache-2.0 / MIT / BSD-3, links and accurate summaries for the rest	users and auditors, opened from the <i>Help</i> menu
Installer licence page	<code>LICENSE</code> , shown before installation proceeds	everyone who installs the application
Install dialog per engine	the registry’s <code>license</code> string for the engine being installed	the user at the moment they choose an engine

```
[Setup]
LicenseFile=..\LICENSE
```

```
[Files]
Source: "..\docs\THIRD-PARTY-LICENSES.html"; DestDir: "{app}\docs"; Flags: ignoreversion
```


Explain Line by Line Code Description (Block 3)

1. **Lines 1-2:**

`LicenseFile`

Inno Setup renders the given file on a page the user must accept to continue. Pointing it at the repository's `LICENSE` means there is one file to maintain, and the installer cannot drift from it.

2. **Lines 4-5:** the HTML page

Copied into the application folder so the Help menu can open it offline, long after installation. A licence page that only exists on the internet is unavailable in the one place it is most needed — a machine with no network.

3. **Why HTML rather than a text file?**

The page contains tables, links and several full-licence sections. HTML keeps it navigable by keyboard and readable by a screen reader, matching the rest of this application's documentation.

82.4 Building an Auditable Inventory

Because the metadata is data, an inventory is a loop — not a manual table that goes stale the day someone adds an engine:

```
import json

from ai_voice_studio.voicelab import engines

print("== Voice Lab clone engines ==")
for info in engines.ENGINES:
    print(f"{info['name']:<34} {info['license']}")
    print(f"{'':34} {info['homepage']}")

print()
print("== Catalog TTS families ==")
with open("ai_voice_studio/tts/models_catalog.json", encoding="utf-8") as handle:
    catalog = json.load(handle)
    for entry in catalog["tts"]:
        print(f"{entry['name']:<34} {entry.get('license', '(not recorded)')}")
```

Output (trimmed) on the current project:

```

== Voice Lab clone engines ==
Pocket TTS (Kyutai)           Apache-2.0 (code); per-voice licenses
Bark (Suno AI, transformer)    MIT
F5-TTS (transformer)          MIT (code); CC-BY-NC-4.0 (weights)

== Catalog TTS families ==
Piper                          MIT
Kokoro-82M                     Apache-2.0
Kitten TTS                     Apache-2.0
VITS zh-AISHELL3               Apache-2.0
Matcha-TTS                     CC-BY-4.0 (icefall conversion); ...
OmniVoice                      Apache-2.0
OmniVoice Server               Apache-2.0 / MIT
SAPI5 (Windows voices)         Windows (built in)
Windows Core voices            Windows (built in)

```

💡 **Tip:** Run this script (or a test that does the same) before every release. A component with an empty licence field is the only thing you need to fix, and it takes seconds to find.

82.5 The Commercial-Use Matrix

The question users actually ask is not “what licence is Kokoro?” but “may I sell the audiobook I made?”.

Converting a licence name into an answer requires knowing whether it touches the *output* or only the *software*:

Component	Licence	Commercial use of the software	Commercial use of the audio
Piper	MIT	yes	yes (check the individual voice)
Kokoro-82M	Apache-2.0	yes	yes
Kitten TTS	Apache-2.0	yes	yes
VITS zh-AISHELL3	Apache-2.0 code / AISHELL3 data	yes	check: the dataset is non-commercial
Matcha-TTS	CC-BY-4.0 (conversion)	yes	check the checkpoint's training data
OmniVoice / Server	Apache-2.0	yes	yes
Pocket TTS	Apache-2.0 code / per-voice	yes	check the individual voice
Bark	MIT	yes	yes
F5-TTS	MIT code / CC-BY-NC-4.0 weights	yes (the code)	no with the released weights
SAPI5 / Windows Core voices	Windows and vendor EULAs	depends on your licence for those voices	depends on the voice

⚠ Warning: Two entries deserve to be highlighted in any user-facing documentation: **F5-TTS weights are non-commercial**, and **MMS / AISHELL3-family data is non-commercial**. Both are fine for personal and educational use, which is why they remain in the catalog — with the terms stated where the engine or voice is chosen, the user can make an informed decision instead of discovering the limitation after publishing.

82.6 Obligations That Come With the Permissive Licences

“Permissive” does not mean “no obligations”. Three attach to almost everything in this dependency set:

1. **Attribution.** MIT, BSD and Apache-2.0 all require the copyright notice and licence text to be preserved in distributions. That is the entire reason `THIRD-PARTY-LICENSES.html` exists and is installed alongside the app.
2. **NOTICE file (Apache-2.0).** If a component ships a `NOTICE` file, its contents must be reproduced. Check the wheels you bundle.
3. **State changes (Apache-2.0).** If you modify Apache-2.0 code, say so. This is why a build that patches a library needs a note.

4. **Copyleft.** wxPython's *wxWindows Library Licence* is LGPL-compatible; dynamic linking is the usual reason such a GUI toolkit can be bundled in a non-LGPL application. Its licence text belongs in the same document as the rest.
5. **Optional binaries.** FFmpeg is downloaded on demand and is distributed under LGPL/GPL depending on the build; the page states that this is the user's download, which keeps the question of redistribution with the user.

82.7 Mini Project: A Release Licence Check

Turn the inventory into a check that fails loudly when something is missing. A test is a better place than a checklist, because it runs every time:

```
import json
from pathlib import Path

from ai_voice_studio.voicelab import engines

ROOT = Path(__file__).resolve().parents[1]
PAGE = ROOT / "docs" / "THIRD-PARTY-LICENSES.html"

def test_every_engine_states_its_licence():
    for info in engines.ENGINES:
        assert info.get("license"), f"{info['id']} has no licence recorded"
        assert info.get("homepage"), f"{info['id']} has no source link"

def test_catalog_families_state_their_licence():
    catalog = json.loads(
        (ROOT / "ai_voice_studio" / "tts" / "models_catalog.json").read_text("utf-8")
    )
    for entry in catalog["tts"]:
        assert entry.get("license"), f"{entry['id']} has no licence recorded"

def test_licence_page_exists_for_the_help_menu():
    assert PAGE.is_file()
    text = PAGE.read_text("utf-8").lower()
    for needle in ("apache", "mit", "bsd", "cc-by-nc"):
        assert needle in text, f"{needle} is not covered in the licence page"
```

Explain Line by Line Code Description (Block 4)

1. **Lines 6–7:** resolving the repository root

Tests must not depend on the working directory: deriving paths from `__file__` means the same check runs from an IDE, from `pytest` and from CI.

2. **Lines 10–13:** the engine check

Two assertions per engine: a licence *and* a source link. Recording one without the other is the half-finished state this test exists to prevent — an unexplained licence string is as hard to verify as a missing one.

3. **Lines 16–22:** the catalog check

Reads the JSON data file rather than importing the catalog module, so the test keeps working even when the import path changes — it asserts the data, which is what ships.

4. **Lines 25–29:** the page check

Asserts the file exists *and* mentions the four licence families that always apply. That single test would have caught the release where the page was missing entirely, or where the Apache text was dropped during an edit.

82.8 Key Takeaways

- Track four layers — application, libraries, model code, model weights — because they can differ
- Store the licence next to the component as data, so UI and documentation read one source
- Disclose at the moment of choice (the install dialog) and permanently (the installed licence page)
- Give users a commercial-use matrix, not a list of licence names
- Permissive licences still require attribution and, for Apache-2.0, NOTICE preservation
- Make the inventory executable: a test that fails on a missing licence beats a checklist
- Flag non-commercial components (F5-TTS weights, AISHELL3/MMS data) in bold, where the user chooses them

Python Quick Reference

A.1 Data Types

Type	Example	Mutable?
<code>int</code>	<code>42</code>	No
<code>float</code>	<code>3.14</code>	No
<code>str</code>	<code>"hello"</code>	No
<code>bool</code>	<code>True</code> , <code>False</code>	No
<code>list</code>	<code>[1, 2, 3]</code>	Yes
<code>tuple</code>	<code>(1, 2, 3)</code>	No
<code>dict</code>	<code>{"a": 1}</code>	Yes
<code>set</code>	<code>{1, 2, 3}</code>	Yes
<code>None</code>	<code>None</code>	No

A.2 String Methods

Method	Description
<code>s.upper()</code>	Convert to uppercase
<code>s.lower()</code>	Convert to lowercase
<code>s.strip()</code>	Remove leading/trailing whitespace
<code>s.split(sep)</code>	Split into list
<code>s.join(list)</code>	Join list with separator
<code>s.replace(old, new)</code>	Replace substring
<code>s.find(sub)</code>	Find index (or -1)
<code>s.startswith(prefix)</code>	Check prefix
<code>s.endswith(suffix)</code>	Check suffix
<code>s.format(**kwargs)</code>	Format string

A.3 List Methods

Method	Description
<code>lst.append(x)</code>	Add to end
<code>lst.extend(iterable)</code>	Extend with iterable
<code>lst.insert(i, x)</code>	Insert at index
<code>lst.remove(x)</code>	Remove first occurrence
<code>lst.pop(i)</code>	Remove and return at index
<code>lst.sort()</code>	Sort in place
<code>lst.reverse()</code>	Reverse in place
<code>lst.copy()</code>	Shallow copy

A.4 Dict Methods

Method	Description
<code>d.get(key, default)</code>	Get with default
<code>d.keys()</code>	All keys
<code>d.values()</code>	All values
<code>d.items()</code>	All key-value pairs
<code>d.update(other)</code>	Merge dicts
<code>d.pop(key)</code>	Remove and return
<code>d.setdefault(key, val)</code>	Get or set default

A.5 Built-in Functions

Function	Description
<code>len(x)</code>	Length
<code>type(x)</code>	Type
<code>isinstance(x, T)</code>	Type check
<code>range(n)</code>	Number sequence
<code>enumerate(lst)</code>	Index + value pairs
<code>zip(a, b)</code>	Parallel iteration
<code>map(fn, iterable)</code>	Apply function
<code>filter(fn, iterable)</code>	Filter elements
<code>sorted(iterable)</code>	Return sorted list
<code>any(iterable)</code>	True if any element is truthy
<code>all(iterable)</code>	True if all elements are truthy
<code>sum(iterable)</code>	Sum of elements
<code>min/max(iterable)</code>	Minimum/maximum
<code>abs(x)</code>	Absolute value
<code>round(x, n)</code>	Round to n decimals

A.6 f-string Format Specifiers

Spec	Description	Example
<code>:.2f</code>	2 decimal places	<code>f"{3.14:.2f}"</code> → <code>3.14</code>
<code>:d</code>	Decimal integer	<code>f"{42:d}"</code> → <code>42</code>
<code>:05d</code>	Zero-padded	<code>f"{42:05d}"</code> → <code>00042</code>
<code>:,</code>	Thousands separator	<code>f"{1000000:,}"</code> → <code>1,000,000</code>
<code>:.1%</code>	Percentage	<code>f"{0.857:.1%}"</code> → <code>85.7%</code>
<code>:>10</code>	Right-align	<code>f"'hi':>10"</code> → <code>hi</code>
<code>:^10</code>	Center	<code>f"'hi':^10"</code> → <code>hi</code>
<code>:=</code>	Debug expression	<code>f"x = "</code> → <code>x = 42</code>

A.7 Common Patterns

```
# Conditional assignment
x = value if condition else default

# Dictionary comprehension
{key: value for item in iterable}

# List comprehension with filter
[x for x in iterable if predicate(x)]

# Generator expression
sum(x for x in iterable)

# Ternary with collections
first = next((x for x in lst if pred(x)), None)

# Swap variables
a, b = b, a

# Unpack with ignore
first, _, last = [1, 2, 3, 4, 5]
```

Explain Line by Line Code Description (Block 1)

1. **Line 1:** `# Conditional assignment`

A comment explaining: *Conditional assignment*. Comments are ignored by the Python interpreter and are used for documentation.

2. **Line 2:** `x = value if condition else default`

Assigns the expression `value if condition else default` to the variable `x`.

3. **Line 3:** `# Dictionary comprehension`

A comment explaining: *Dictionary comprehension*. Comments are ignored by the Python interpreter and are used for documentation.

4. **Line 4:** `{key: value for item in iterable}`

`{key: value for item in iterable}`

5. **Line 5:** `# List comprehension with filter`

A comment explaining: *List comprehension with filter*. Comments are ignored by the Python interpreter and are used for documentation.

6. **Line 6:** `[x for x in iterable if predicate(x)]`

`[x for x in iterable if predicate(x)]`

7. **Line 7:** `# Generator expression`

A comment explaining: *Generator expression*. Comments are ignored by the Python interpreter and are used for documentation.

8. **Line 8:** `sum(x for x in iterable)`

`sum(x for x in iterable)`

9. **Line 9:** `# Ternary with collections`

A comment explaining: *Ternary with collections*. Comments are ignored by the Python interpreter and are used for documentation.

10. **Line 10:** `first = next((x for x in lst if pred(x)), None)`

Assigns `None` to `first`, indicating no value or an uninitialized state.

11. **Line 11:** `# Swap variables`

A comment explaining: *Swap variables*. Comments are ignored by the Python interpreter and are used for documentation.

12. **Line 12:** `a, b = b, a`

`a, b = b, a`

13. **Line 13:** `# Unpack with ignore`

A comment explaining: *Unpack with ignore*. Comments are ignored by the Python interpreter and are used for documentation.

14. **Line 14:** `first, _, last = [1, 2, 3, 4, 5]`

`first, _, last = [1, 2, 3, 4, 5]`

AIVS API Reference

B.1 Core Modules

paths.py

Function	Returns	Description
<code>user_data_dir()</code>	str	%APPDATA%/AIVoiceStudio
<code>models_dir()</code>	str	Models directory
<code>project_dir(name)</code>	str	Project directory by name
<code>settings_file()</code>	str	Path to settings.json
<code>log_file()</code>	str	Path to app.log

settings.py — Settings

Method	Description
<code>get(key, default)</code>	Get setting value
<code>set(key, value)</code>	Set and save
<code>save()</code>	Save to disk
<code>add_recent_project(name, path)</code>	Add to recent list

project.py

Function	Description
<code>create_project(dir, name, source, mode, segments)</code>	Create new project
<code>load_project(dir)</code>	Load project.json
<code>save_project(dir, data)</code>	Atomic JSON save
<code>mark_segment_done(dir, index, path)</code>	Mark segment recorded
<code>first_pending_index(dir)</code>	First unrecorded segment

B.2 TTS Modules

tts/catalog.py

Function	Description
<code>get_tts_list()</code>	All TTS engines
<code>find_tts(tts_id)</code>	Find engine by ID
<code>find_language(tts, code)</code>	Find language in engine
<code>find_variant(tts, lang, var)</code>	Find variant
<code>find_voice(tts, lang, var, voice)</code>	Find specific voice

tts/engine.py

Function/Class	Description
<code>get_engine(voice_entry)</code>	Factory: get/create TTS engine
<code>TtsEngine.synthesize(text, sid, speed, pitch, volume)</code>	Synthesize speech
<code>_shift_pitch(samples, pitch)</code>	Resample for pitch shift
<code>_apply_volume(samples, volume)</code>	Scale amplitude

tts/models.py — ModelStore

Method	Description
<code>variant_dir(tts, lang, var)</code>	Disk path for variant
<code>is_variant_installed(tts, lang, var)</code>	Check if on disk
<code>mark_installed(tts, lang, var, path)</code>	Register as installed
<code>installed_voices()</code>	List all installed voices
<code>custom_voices()</code>	List cloned voices

B.3 Document Modules

documents/parsers.py

Function	Description
<code>parse_document(path)</code>	Parse any supported file
<code>ParseError</code>	Exception for parse failures

documents/splitter.py

Function/Class	Description
<code>Segment(index, title, text)</code>	Data class for a text segment
<code>split_document(doc, mode)</code>	Split document into segments
<code>split_daisy_chapters(blocks)</code>	Split for DAISY output

B.4 GUI Modules

gui/events.py

Event	Data
<code>SynthStatusEvent</code>	message: str
<code>SynthSegmentDoneEvent</code>	index, title, path
<code>SynthFinishedEvent</code>	(none)
<code>SynthErrorEvent</code>	message: str
<code>DownloadProgressEvent</code>	name, done, total
<code>DownloadFinishedEvent</code>	success, message

gui/a11y.py

Function	Description
<code>add_labeled(parent, sizer, text, ctrl)</code>	Label+control with a11y name
<code>add_check(parent, sizer, text)</code>	Accessible checkbox
<code>set_accessible_name(ctrl, name)</code>	Set screen reader name

B.5 Voice Lab, Compute and Environment Modules

The modules added after the first edition (see [Part X](#)). Everything here is importable without PyTorch and without wx, which is what makes the subsystem unit-testable on any machine.

compute.py

Function	Returns	Description
<code>has_nvidia_gpu(force=False)</code>	bool	<code>nvidia-smi</code> driver probe, cached for the session
<code>detect(force=False)</code>	dict	<code>{'cpu': True, 'cuda': ..., 'dml': False}</code>
<code>cpu_threads(total=None)</code>	int	Threads a CPU run may use: the top of the 80–95 % band
<code>cuda_device_index()</code>	int	Always 0 : the first NVIDIA card the driver exposes
<code>gpu_summary()</code>	str	Card name and memory, for display
<code>available_compute()</code>	list	Options the UI may offer, with <code>auto</code> when there is a choice
<code>resolve_compute(choice)</code>	str	Turn <code>auto cpu cuda dml</code> into a concrete back-end
<code>provider_for(compute)</code>	str	Back-end → the string sherpa-onnx expects
<code>runtime_hint(compute)</code>	str	Human note about which runtime a back-end needs

gui/compute_choice.py

Function	Description
<code>choices()</code>	<code>(value, label)</code> pairs for the per-category combo
<code>settings_key(category)</code>	<code>preview_compute.<category></code>
<code>saved_choice(settings, category)</code>	Stored choice, or <code>cpu</code>
<code>provider_for_preview(choice, engine_id=None)</code>	Voice Lab → its own device; others → ONNX compute detection
<code>make_compute_row(parent, settings, category, default="cpu")</code>	Builds <code>(label, combo)</code> and persists the choice on change

voicelab/engines.py

Function	Description
<code>ENGINES</code>	The registry: one dict per clone engine (packages, licence, homepage, languages)
<code>engine_ids()</code> / <code>is_engine(id)</code> / <code>engine(id)</code>	Lookups used by the GUI, the worker and the settings pages
<code>packages(id)</code>	pip packages for install <i>and</i> uninstall (plain names only)
<code>import_modules(id)</code>	Modules the worker checks before the engine can speak
<code>cuda_packages(id)</code> / <code>cuda_index_url(id)</code>	What to take from the PyTorch CUDA wheel index, and that index
<code>probe_package(id)</code>	The single distribution whose presence means “installed”
<code>builtin_voices(id)</code> / <code>all_builtin_voices()</code>	Package-provided voices in catalog shape
<code>count_builtin_voices(id)</code>	How many voices the engine ships
<code>PYTORCH_CUDA_INDEX</code>	https://download.pytorch.org/whl/cu128 — one index for all CUDA wheels

voicelab/_init_.py

Function / class	Description
<code>has_cuda(force=False)</code>	NVIDIA driver probe (same source of truth as <code>compute</code>)
<code>device_options()</code>	CPU always; GPU and Auto when a driver is present
<code>resolve_device(choice)</code>	<code>cpu cuda auto</code> → concrete device, falling back to CPU
<code>is_installed(id) / installed_version(id)</code>	Per-engine environment first, shared environment as fallback
<code>install_plan(id)</code>	<code>[(packages, index_url_or_None), ...]</code> — CUDA step first when there is a GPU
<code>packages_for(id)</code>	The registry's package list (never version-pinned)
<code>verify_engine_import(id, timeout=300)</code>	Imports the engine once in its own env; returns a problem or <code>None</code>
<code>cpu_thread_count()</code>	Delegates to <code>compute.cpu_threads()</code>
<code>build_synthesize_request(**)</code>	Pure builder for the worker's JSON request
<code>CloneWorker(engine_id, device, timeout)</code>	One persistent worker process; <code>synthesize()</code> , <code>close()</code>
<code>worker_for(id, device) / close_workers()</code>	Shared worker cache and shutdown
<code>VoicelabEngine(voice_entry, device)</code>	Adapter implementing <code>TtsEngine.synthesize</code>
<code>VoicelabError</code>	Raised when an engine is missing or fails to synthesize

voicelab/options.py

Function	Description
<code>specs(id) / spec(id, key)</code>	The tuning vocabulary: type, range, default, label
<code>defaults(id)</code>	Every key at its engine default
<code>clean(id, values)</code>	Coerce, clamp and drop defaults — what the worker receives
<code>problems(id, values)</code>	Human-readable validation messages for the dialogs
<code>describe_overrides(id, values)</code>	One-line summary of what the user changed
<code>settings_key(id)</code>	Where the per-engine defaults live in Settings
<code>max_seconds_note(id)</code>	Note about engines that cap segment length

python_runtime/_init_.py

Function	Description
<code>environment_id(engine_id)</code>	Folder id of the env that holds an engine (OmniVoice pair shares one)
<code>engine_env_dir(engine_id)</code>	<code>%APPDATA%/AIVoiceStudio/tts_envs/<env></code>
<code>get_runtime(engine_id=None)</code>	Lazy singleton <code>PythonRuntime</code> per environment
<code>engine_runtime(engine_id)</code>	The env that actually holds an engine's packages, with shared fallback
<code>engine_env_exists(engine_id)</code>	True when the engine has its own (created) env
<code>PythonRuntime.ensure_env()</code> / <code>ensure_pip()</code>	Create the virtualenv and its pip
<code>PythonRuntime.pip_install(...)</code> / <code>pip_uninstall(...)</code>	Install/remove packages (optional <code>--index-url</code>)
<code>PythonRuntime.pip_list()</code>	Installed packages as dicts
<code>PythonRuntime.run_in_env(script)</code>	Run a snippet in the env and capture its output
<code>PythonRuntime.site_packages_dirs</code>	The env's site-packages folder(s) — used for on-disk probes

venv_packages.py

Function	Description
<code>version(package, engine=None)</code>	Cached version; on a cold cache reads site-packages directly. Never blocks
<code>installed(package, engine=None)</code>	<code>bool(version(...))</code>
<code>package_present(package, engine=None)</code>	Synchronous disk check — decides the worker's interpreter
<code>request(package, on_ready, engine=None)</code>	Start one background probe; <code>on_ready(version_or_None)</code> runs on the worker thread
<code>is_known(package, engine=None)</code>	True when a fresh cached answer exists
<code>invalidate(*packages, engine=None)</code>	Forget cached results after install/uninstall
<code>warm(packages=None)</code>	Background warm-up of the shared environment's probes

voicelab/worker.py (runs in the engine's own interpreter)

Function	Description
<code>_maximize_cuda(torch)</code>	Whole-card inference: cuDNN autotune, TF32, "high" precision, no autograd
<code>_limit_cpu(torch, threads)</code>	Applies the 80–95 % thread count the application computed
<code>_apply_compute_settings(device, threads)</code>	One entry point for the device choice
<code>_torchcodec_works(torchaudio)</code>	Measures the real decoder on a tiny probe WAV
<code>_install_audio_io_shim(...)</code>	Backs <code>torchaudio.load/save</code> with soundfile when TorchCodec is broken
<code>_convert_to_wav(path)</code>	Converts a reference clip once (cached by path, size, mtime)
<code>_resolve_resource(spec)</code>	Resolves "package:relative/path" to a real file inside a package
<code>_supported_kwargs(callable, kwargs)</code>	Feature-detects which tuning options an engine version accepts
<code>Backend</code>	Base class: one loaded engine (<code>PocketBackend</code> , <code>BarkBackend</code> , <code>F5Backend</code>)

gui/accept_dialog.py

Name	Description
<code>AcceptanceDialog</code>	First-launch terms dialog; every exit path except “I Agree” returns <code>ID_NO</code>
<code>terms_current(settings)</code>	True when <code>terms_version</code> equals <code>TERMS_VERSION</code>
<code>record_acceptance(settings)</code>	Stores and saves the accepted version
<code>_TERMS</code>	The six acknowledgement strings, in display order

jobs/synthesizer.py — SynthesisWorker (range arguments)

Argument	Description
<code>start_index</code>	0-based segment to start from (default <code>0</code>)
<code>end_index</code>	Exclusive end; <code>start_index + 1</code> records one segment, <code>None</code> means “to the end”

Appendix C: Glossary of Terms

A comprehensive glossary of technical terms used throughout this book, covering Python, wxPython, AI/TTS, audio processing, and packaging concepts.

A

Accessibility (a11y)

The design practice of making software usable by people with disabilities. In wxPython, this includes screen reader support via accessible names, roles, and descriptions. The `a11y` abbreviation comes from counting the letters between 'a' and 'y' in "accessibility."

ACB (Audio Callback)

In sherpa-onnx, an audio callback receives synthesized audio samples from the TTS engine. The callback is invoked from a background thread whenever a chunk of audio is ready for output.

Aggregator

In the AIVoiceStudio catalog system, an aggregator would fetch and cache model catalogs from multiple upstream sources (Piper, Kokoro, sherpa-onnx releases, ...) and provide a unified interface for querying available models. In the current implementation the catalog is a single reviewed JSON file (`tts/models_catalog.json`) plus the Voice Lab entries generated from `voicelab/engines.py` , so no aggregator runs at runtime.

Async/Await

Python's built-in asynchronous programming syntax. `async def` defines a coroutine function, and `await` suspends execution until an awaitable (coroutine, Future, Task) completes. Used with `asyncio` for non-blocking I/O operations.

Attribute

A variable associated with an object or class. In Python, attributes are accessed with dot notation (e.g., `obj.attr`). Class attributes are shared; instance attributes are per-object.

Augmented Assignment

Compound assignment operators that combine an operation with assignment: `+=` , `-=` , `*=` , `/=` , `%=` , `**=` , `//=` , etc.

B

Backend

In AIVoiceStudio, the compute backend refers to the hardware accelerator used for TTS inference: CPU, CUDA (NVIDIA GPU), DirectML (Windows ML), or CoreML (Apple). Also refers to sherpa-onnx's execution provider configuration.

Background Thread

A thread that runs concurrently with the main (GUI) thread, used for long-running operations like TTS synthesis, file downloads, or model loading. In wxPython, background threads must use `wx.CallAfter` to update the GUI safely.

Base64

A binary-to-text encoding scheme that represents binary data using 64 ASCII characters. Used in web APIs and configuration to embed binary data (like audio samples) in text format.

Benchmark

A standardized test that measures the performance of hardware or software. In AIVoiceStudio, compute benchmarks measure how fast TTS inference runs on different backends (CPU vs. GPU).

Binder

In wxPython, a binder associates an event type with a handler function. `self.Bind(wx.EVT_BUTTON, self.on_click, self.my_button)` tells wxPython to call `on_click` when the button is clicked.

BOM (Byte Order Mark)

A special Unicode character (U+FEFF) placed at the beginning of a text file to indicate its encoding and byte order. UTF-8 BOM is common in Windows-generated text files.

Bytecode

Intermediate compiled representation of Python source code. Python compiles `.py` files to `.pyc` bytecode files in `__pycache__`, which the interpreter executes faster than raw source.

C

Callback

A function passed as an argument to another function, to be called when an event occurs or an operation completes. In sherpa-onnx TTS, audio callbacks receive synthesized audio chunks.

Casting

Converting a value from one type to another: `int("42")`, `float(5)`, `str(3.14)`. Also called type conversion.

Catalog

In AIVoiceStudio, the TTS catalog is a registry of all available voice models, organized by engine (Piper, Kokoro-82M, Kitten, Matcha, VITS, OmniVoice, SAPI5, Windows Core) plus the Voice Lab engines, then by language and quality. Stored as a JSON file with metadata for each model, with the Voice Lab entries appended from `voicelab/engines.py`.

CLI (Command-Line Interface)

A text-based interface for interacting with software. Python's `argparse` module provides CLI argument parsing. AIVoiceStudio uses CLI for its build scripts.

Comprehension

A concise syntax for creating lists, dicts, or sets from iterables: `[x**2 for x in range(10)]` (list), `{k: v for k, v in items}` (dict), `{x for x in items}` (set). Also generator expressions with parentheses.

Context Manager

An object that implements `__enter__` and `__exit__` methods for use with the `with` statement. Ensures proper resource acquisition and cleanup. Examples: `open()` , `threading.Lock()` , wxPython's `BusyCursor` .

Compute

In AIVoiceStudio, refers to the hardware and software stack used for AI inference. Detected via `compute.py` which probes for CUDA, DirectML, CoreML, and CPU capabilities using `py-cpuinfo` and `onnxruntime`.

CUDA

NVIDIA's parallel computing platform for GPU-accelerated computation. When CUDA is available, sherpa-onnx can use it for much faster TTS synthesis. Detected by checking for CUDA runtime libraries.

Coroutine

A function defined with `async def` that can be paused and resumed. Coroutines are scheduled on the event loop and cooperatively yield control at `await` points.

CPython

The reference implementation of Python, written in C. Most Python installations use CPython. Identified by `sys.implementation.name == "cpython"` .

Custom Event

In wxPython, a user-defined event type created with `wx.NewEventType()` and `wx.PyEventBinder()` . Used for thread-safe communication between background workers and the GUI.

D

DAISY

Digital Accessible Information System — a technical standard for accessible digital publications.

AIVoiceStudio's DAISY builder creates multi-level audio publications with navigation from document chapters.

Dataclass

A Python decorator (`@dataclass`) that automatically generates `__init__` , `__repr__` , `__eq__` , and other methods from class attribute annotations. Used extensively in AIVoiceStudio for configuration and data models.

Decorator

A function that takes a callable and returns a modified callable. Used with `@decorator` syntax above function or class definitions. Examples: `@property` , `@staticmethod` , `@dataclass` , `@functools.lru_cache` .

Desktop Application

A graphical application that runs locally on a user's computer, as opposed to a web application.

AIVoiceStudio is a desktop app built with wxPython and packaged with PyInstaller.

DirectML

Microsoft's Direct Machine Learning API for hardware-accelerated AI inference on Windows. Works with any DirectX 12-compatible GPU, not just NVIDIA. Available via `onnxruntime-directml` .

Docstring

A string literal placed as the first statement in a module, class, or function to document its purpose. Triple-quoted: `"""Docstring."""`. Accessible via `__doc__` and the `help()` function.

DPI (Dots Per Inch)

A measure of display resolution. High-DPI displays (Retina, 4K) require special handling in desktop apps. wxPython supports DPI awareness through system settings and scaling.

Dunder Method

Special methods with double underscores: `__init__`, `__str__`, `__repr__`, `__len__`, `__getitem__`, etc. Also called magic methods. They define how objects behave with built-in operations.

E

Enumerate

A built-in function that returns pairs of (index, value) from an iterable: `for i, val in enumerate(items)`. Avoids manual counter management.

Event

In GUI programming, an event represents a user action (click, keypress) or system occurrence (timer, resize). wxPython events carry data and are dispatched to bound handler functions.

Event Loop

The core mechanism of a GUI framework that continuously polls for events and dispatches them to handlers. wxPython's `MainLoop()` runs the event loop. The application blocks at this call until the window is closed.

Event Binder

In wxPython, `wx.PyEventBinder` creates bindable event types. Used to associate custom events with handler methods: `self.Bind(MY_EVENT, self.handler)`.

Exception

An error that occurs during program execution. Python uses exceptions for error handling with `try / except / finally`. Exceptions are objects of classes inheriting from `BaseException`.

Execution Provider

In ONNX Runtime, the execution provider specifies the hardware backend: `CPUExecutionProvider`, `CUDAExecutionProvider`, `DmlExecutionProvider`, `CoreMLExecutionProvider`.

Expression

A piece of code that evaluates to a value. Examples: `2 + 3`, `"hello".upper()`, `[x for x in range(5)]`. Distinguished from statements, which don't return values.

Extension

A file suffix indicating file type: `.py` for Python, `.wav` for audio, `.json` for data. Python maps extensions to interpreters; in wxPython, extensions are used for file dialog filters.

F

F-String

Formatted string literals (Python 3.6+): `f"Hello, {name}!"` . Support expressions inside curly braces: `f"{price:.2f}"` . Faster and more readable than `.format()` or `%` formatting.

Factory Function

A function that creates and returns objects, serving as an alternative to constructors. In wxPython, many widget creation functions act as factories.

Fallback

A default value or behavior used when the preferred option is unavailable. In AIVoiceStudio, compute detection falls back through CUDA → DirectML → CPU if higher-performance backends aren't available.

FFmpeg

A powerful open-source multimedia framework for audio/video processing. AIVoiceStudio uses FFmpeg for audio format conversion (MP3, OGG, FLAC, WAV) and audio normalization.

Frame

In wxPython, a Frame is a top-level window with a title bar, menu bar, and close button. `wx.Frame` is the base class. AIVoiceStudio's main window is a Frame subclass.

Function

A reusable block of code defined with `def` . Functions can accept parameters, return values, and be passed as arguments. Python functions are first-class objects.

Functor

An object that can be "called" like a function by implementing `__call__` . Used in AIVoiceStudio for callback objects that need to maintain state between invocations.

G

Garbage Collection

Python's automatic memory management that reclaims memory from objects no longer in use. Uses reference counting plus a cyclic garbage collector for objects with circular references.

Generator

A function using `yield` that produces values lazily, one at a time. Generators are memory-efficient for large datasets and support the iterator protocol.

GPU (Graphics Processing Unit)

A specialized processor for parallel computation. NVIDIA GPUs support CUDA for AI inference, providing 10-50x speedup over CPU for TTS synthesis.

GridBagSizer

A wxPython layout manager that positions widgets in a grid with flexible row/column spanning. Used in AIVoiceStudio's settings dialog for complex form layouts.

Guard Clause

An early return statement that handles error conditions at the top of a function, reducing nesting. A common Python pattern: `if not valid: return` instead of wrapping the rest in `if valid: .`

H

Hash

A fixed-size value computed from data using a hash function. Python's built-in `hash()` is used for dictionary keys and set membership. `__hash__` defines custom hashing.

Heap

Memory area where Python allocates objects. Also refers to the `heapq` module for priority queues. In AIVoiceStudio, heavy logging uses heap memory tracking.

HFS+ / APFS

Apple's file system formats (Hierarchical File System Plus / Apple File System). Relevant for macOS path handling and case-sensitivity differences.

Hungarian Notation

A naming convention that prefixes variable names with their type: `nCount` (integer), `strName` (string), `btnOK` (button). Common in wxPython codebases.

I

Idle Event

In wxPython, `wx.EVT_IDLE` fires when the event loop has no pending events. Used for deferred operations and periodic updates without explicit timers.

Import

The `import` statement loads modules and their contents into the current namespace. Forms: `import os` , `from os import path` , `from os import path as p` .

INI File

A simple text file format for configuration with sections and key-value pairs. Python's `configparser` module reads and writes INI files. AIVoiceStudio uses them for user settings.

Instance

A specific object created from a class definition. Each instance has its own attributes but shares the class methods. Created by calling the class: `obj = MyClass()` .

Inheritance

A mechanism where a class (child/subclass) inherits attributes and methods from another class (parent/superclass). Supports code reuse and polymorphism. Python supports multiple inheritance.

Installer

A program that installs software on a user's computer. AIVoiceStudio uses Inno Setup to create a Windows installer that copies files, creates shortcuts, and registers the application.

Iterable

Any object that can be iterated over, implementing `__iter__` or `__getitem__` . Includes lists, tuples, strings, dicts, sets, generators, and files.

Iterator

An object implementing `__iter__` (returning itself) and `__next__` (returning the next value). Used by `for` loops internally. Created by `iter()` function.

ISCC

Inno Setup Compiler Command-line. Used to compile `.iss` scripts into setup executables: `"C:\Program Files\Inno Setup 7\ISCC.exe" script.iss` .

J

JSON (JavaScript Object Notation)

A lightweight data interchange format. Python's `json` module provides `dumps()` / `loads()` for serialization/deserialization. AIVoiceStudio uses JSON for settings, catalogs, and project files.

Job Queue

A data structure for managing tasks to be processed sequentially or in parallel. In AIVoiceStudio, synthesis jobs are queued and processed by background workers with progress callbacks.

Join

String method that concatenates iterable elements: `", ".join(words)` . Also refers to `thread.join()` which blocks until a thread completes, and SQL joins for database queries.

K

Keyword Argument

A function argument passed with a name: `func(name="Alice", age=30)` . Allows explicit argument naming, default values, and arbitrary order. Syntax: `**kwargs` captures keyword arguments as a dict.

Keystroke

A key press event in wxPython. `wx.KeyEvent` carries the key code, modifiers (Ctrl, Shift, Alt), and Unicode character. AIVoiceStudio binds keyboard shortcuts for common operations.

L

Lambda

An anonymous function: `lambda x: x * 2` . Limited to a single expression. Commonly used for short callbacks, `sorted()` keys, and `map()` / `filter()` operations.

Lifecycle

The stages an object goes through from creation to destruction. In wxPython widgets: construction → adding to sizer → show → hide → destroy. In AIVoiceStudio projects: new → open → edit → save → close.

LSP (Language Server Protocol)

A protocol for communication between code editors and language analysis tools. Provides features like autocompletion, error checking, and go-to-definition.

LUT (Lookup Table)

A data structure that maps keys to values for fast retrieval. Python dicts are hash-based LUTs with $O(1)$ average lookup. In audio processing, LUTs map sample values for effects processing.

M

MainLoop

wxPython's event loop that runs until the application exits. Called as `app.MainLoop()`. Blocks the calling thread while dispatching events to handlers.

Memoryview

A built-in type that provides a typed, buffer-protocol view of binary data without copying. Useful for zero-copy slicing of large byte buffers in audio processing.

Metaclass

A class whose instances are classes. `type` is the default metaclass. Custom metaclasses can modify class creation. Used in wxPython for event system internals and in ORMs for model definitions.

Method Resolution Order (MRO)

The order in which Python looks up methods in a class hierarchy with multiple inheritance. Determined by the C3 linearization algorithm. Accessible via `ClassName.__mro__`.

MIME

Multipurpose Internet Mail Extensions. A standard for classifying file types. wxPython uses MIME types for drag-and-drop operations and clipboard data.

Module

A `.py` file containing Python definitions and statements. Modules provide namespaces for organizing code. The `__name__` attribute identifies the module; `"__main__"` indicates direct execution.

Multiprocessing

Running code in separate processes, each with its own Python interpreter and memory space. Bypasses the GIL for CPU-bound parallelism. Python's `multiprocessing` module provides the API.

Mutability

Whether an object can be modified after creation. Lists, dicts, and sets are mutable; strings, tuples, and ints are immutable. Mutable default arguments in functions are a common pitfall.

N

Namespace

A mapping from names to objects. Each module, class, and function has its own namespace. The LEGB rule (Local, Enclosing, Global, Built-in) determines name resolution.

NIL / None

In Python, `None` represents the absence of a value. It's a singleton — only one instance exists. Commonly used as a default parameter and sentinel value.

Numpy

A numerical computing library providing N-dimensional arrays, mathematical functions, and linear algebra operations. AIVoiceStudio uses numpy for audio sample array manipulation.

Nuitka

A Python-to-C compiler that translates Python code to optimized C/C++ for distribution. An alternative to PyInstaller for creating standalone executables.

O

O(n) / Big-O Notation

Mathematical notation describing algorithm complexity. $O(1)$ = constant, $O(n)$ = linear, $O(n^2)$ = quadratic, $O(\log n)$ = logarithmic. Dict lookup is $O(1)$; list search is $O(n)$.

Object

The fundamental unit of Python — everything is an object, including functions, classes, and modules.

Objects have identity (`id()`), type (`type()`), and attributes.

OmniVoice

An advanced TTS engine in AIVoiceStudio supporting voice cloning. Uses a server-client architecture where the OmniVoice server runs inference and the worker communicates via HTTP.

ONNX (Open Neural Network Exchange)

An open format for representing machine learning models. sherpa-onnx uses ONNX models for TTS inference, enabling cross-platform deployment with multiple execution providers.

OpenCV

An open-source computer vision library. Not directly used in AIVoiceStudio, but similar to how sherpa-onnx uses pre-compiled native libraries for performance-critical operations.

Operator Overloading

Defining how custom objects respond to built-in operators via dunder methods: `__add__` for `+`, `__eq__` for `==`, `__lt__` for `<`. Enables intuitive syntax for custom types.

Output Buffer

A memory region where audio samples are accumulated before being written to a file or played. In AIVoiceStudio's TTS engine, audio samples from the callback are appended to a buffer list, then concatenated into a numpy array.

P

Package

A Python directory containing an `__init__.py` file, making it importable as a module. Packages can be nested (sub-packages). AIVoiceStudio is a package with sub-packages: `tts` , `gui` , `audio` , etc.

Parameter vs. Argument

A *parameter* is a variable in a function definition; an *argument* is the value passed when calling the function. "The function accepts a `path` parameter" vs. "pass `'/tmp'` as an argument."

Parsing

Converting data from one format to a structured representation. AIVoiceStudio parses PDF (pypdf), DOCX (python-docx), TXT, EPUB, and DAISY documents into plain text for TTS processing.

Pathlib

Python's modern path handling module (3.4+). `Path` objects provide intuitive operators: `parent` , `/` for joining, `.stem` , `.suffix` , `.exists()` . Preferred over `os.path` for readability.

Piper

A fast, local neural TTS engine. AIVoiceStudio integrates Piper via sherpa-onnx for high-quality text-to-speech synthesis with multiple voices and languages.

Priority Queue

A data structure where elements are dequeued by priority rather than insertion order. Python's `heapq` module implements a min-heap. Used for task scheduling and job processing.

Property

A Python descriptor that manages attribute access with getter/setter/deleter methods. Defined with `@property` decorator. Provides computed attributes and encapsulation.

PyInstaller

A tool that bundles Python applications into standalone executables. Packages the Python interpreter, all dependencies, and native libraries into a distributable folder or single file.

PyPDF (pypdf)

A Python library for reading, splitting, and manipulating PDF files. AIVoiceStudio uses it to extract text from PDF documents for TTS processing.

Python-docx

A Python library for creating and reading Microsoft Word (.docx) files. AIVoiceStudio uses it to extract text from Word documents for TTS processing.

PythonExec

In AIVoiceStudio, the Python runtime subsystem for executing user-provided Python scripts within the application's environment, with sandboxed access to project files and settings.

Q

Queue

A FIFO (First In, First Out) data structure. `queue.Queue` is thread-safe for inter-thread communication. `asyncio.Queue` is for coroutines. Used for job processing in AIVoiceStudio.

QuickStart

A feature in AIVoiceStudio that provides one-click setup for downloading a default TTS model and creating a test project, allowing users to try the application immediately.

R

RAII (Resource Acquisition Is Initialization)

A C++ concept adopted in Python via context managers. Resources are acquired in `__enter__` and released in `__exit__`, guaranteeing cleanup even if exceptions occur.

Raw String

A string prefixed with `r` that doesn't process escape sequences: `r"C:\Users\test"`. Essential for regex patterns and Windows paths.

Refactoring

Restructuring code to improve readability, performance, or maintainability without changing external behavior. Common refactorings: extract method, rename, move, inline.

Reference Counting

Python's primary memory management mechanism. Each object tracks how many references point to it. When the count reaches zero, the object is immediately deallocated. Supplemental cyclic GC handles circular references.

Requests

A popular Python HTTP library. AIVoiceStudio uses it for downloading TTS models, checking API endpoints, and communicating with the OmniVoice server.

ReStructuredText (RST)

A markup language for documentation, used by Sphinx. Python's docstring conventions follow RST format. Not used in AIVoiceStudio's runtime, but common in Python ecosystem.

Runtime

The period during which a program is executing. In AIVoiceStudio, the runtime subsystem manages audio output devices, playback state, and session-scoped settings.

S

Sash Window

A wxPython window that can be resized by dragging a divider (sash). Used in AIVoiceStudio's main frame to create a resizable split between the project tree and content area.

Semaphore

A synchronization primitive that limits concurrent access to a resource. `threading.Semaphore(n)` allows up to `n` threads to proceed simultaneously. `asyncio.Semaphore` for async code.

sherpa-onnx

A C++/Python toolkit for running speech processing models (TTS, ASR, speaker identification) using ONNX Runtime. AIVoiceStudio's primary TTS engine for local, offline synthesis.

Sizer

wxPython's layout management system. Sizers automatically arrange child widgets within a container.

Types: `BoxSizer` (linear), `GridSizer` (grid), `FlexGridSizer`, `GridBagSizer`.

Splitter Window

A wxPython widget that divides its area into two resizable panes separated by a movable sash.

`wx.SplitterWindow` is used for master-detail layouts.

Static Method

A method defined with `@staticmethod` that doesn't receive `self` or `cls`. It's a regular function that lives inside a class's namespace for organizational purposes.

Streaming

Processing data incrementally as it arrives, rather than loading everything first. In TTS, streaming synthesis outputs audio chunks as they're generated, reducing latency. AIVoiceStudio's audio callback pattern implements streaming.

String Interpolation

Embedding expressions inside string literals. Python supports f-strings (`f"{x}"`), `.format()`, and `%` formatting. F-strings are preferred for readability and performance.

Subclass

A class that inherits from another class. Also called a child class or derived class. Can override parent methods and add new functionality.

Subprocess

Running external programs from Python via `subprocess.run()`, `subprocess.Popen()`. AIVoiceStudio uses subprocess to run FFmpeg for audio conversion.

Superclass

A class that is inherited from. Also called a parent class or base class. `super()` provides access to superclass methods for cooperative multiple inheritance.

Synthesis

Generating audio from text (TTS synthesis). In AIVoiceStudio, text is split into chunks, each chunk is sent to the TTS engine, and the resulting audio samples are concatenated into a complete audio file.

SysTray

System tray — the area near the clock in the taskbar. wxPython's `TaskBarIcon` allows applications to display an icon in the system tray with a context menu.

T

Tab Completion

IDE/REPL feature that suggests completions as you type. Powered by introspection (`dir()`, `__doc__`) and static analysis. IPython and modern IDEs provide rich tab completion.

Template Method Pattern

A design pattern where a base class defines the algorithm skeleton and subclasses override specific steps. In wxPython, `EvtHandler.ProcessEvent` is a template method.

Text Chunking

Breaking long text into smaller pieces for TTS processing. AIVoiceStudio's `TextSplitter` splits by sentence boundaries, respecting maximum character limits per chunk for the TTS engine.

Thread

A lightweight execution unit within a process. Python threads share memory but are limited by the GIL for CPU-bound work. Useful for I/O-bound tasks like downloads and TTS synthesis. `threading.Thread` is the API.

Thread Safety

Code that functions correctly when accessed by multiple threads simultaneously. In wxPython, GUI updates must use `wx.CallAfter()` from background threads. Locks and queues provide thread safety.

Throttle

Limiting the rate of operations. In AIVoiceStudio, UI updates during synthesis are throttled to avoid overwhelming the GUI event loop with too many status messages.

Timer

A wxPython widget that generates events at regular intervals: `wx.Timer`. Used for periodic checks, animations, and updating UI elements. Started with `timer.Start(milliseconds)`.

Tkinter

Python's standard GUI toolkit, included with Python. Simpler than wxPython but less feature-rich. wxPython is preferred for professional cross-platform applications.

TTS (Text-to-Speech)

Technology that converts written text into spoken audio. AIVoiceStudio uses local TTS engines (Piper via sherpa-onnx) for offline, private speech synthesis.

Type Hint

Optional annotations in Python function signatures and variables: `def greet(name: str) -> None:`. Used by type checkers (mypy, pyright) and IDEs for static analysis, not enforced at runtime.

Type Checking

Verifying that code uses types correctly. *Static* type checking (mypy, pyright) analyzes code without running it. *Dynamic* type checking validates types at runtime.

U

Unpacking

Extracting values from iterables: `a, b = [1, 2]` (tuple unpacking), `*rest = [1, 2, 3, 4]` (star unpacking), `func(**dict)` (keyword unpacking).

UTC

Coordinated Universal Time — the primary time standard. Python's `datetime.timezone.utc` and `datetime.utcnow()` work with UTC. Essential for logging and timestamp accuracy.

UTF-8

A variable-width character encoding for Unicode. The dominant encoding for web and text files. Python 3 source files default to UTF-8. Handles all world scripts including CJK, Arabic, and emoji.

UUID (Universally Unique Identifier)

A 128-bit identifier designed to be unique across all computers and time. Python's `uuid` module generates v4 (random) UUIDs: `uuid.uuid4()`. Used for session IDs and unique project identifiers.

V

Virtual Environment

An isolated Python installation with its own packages, independent of the system Python. Created with `python -m venv .venv`. Prevents dependency conflicts between projects.

Void Return

Functions that don't explicitly return a value implicitly return `None`. Type-hinted as `-> None`. Common for side-effect functions (printing, modifying state, event handlers).

Viewport

The visible area of a scrollable window. In wxPython, `GetClientSize()` returns the viewport dimensions. AIVoiceStudio accounts for viewport size in layout calculations.

Voice Clone

AI technology that creates a synthetic voice matching a specific person's speech patterns from audio samples. AIVoiceStudio's clone feature uses user-recorded audio to generate a voice profile for TTS.

Voice Model

A trained neural network file that produces speech for a specific voice. Each model has parameters: language, speaker ID, quality level, and file size. Stored in ONNX format for cross-platform use.

W

WAV (Waveform Audio File Format)

An uncompressed audio format. The standard intermediate format for TTS output in AIVoiceStudio. Converted to compressed formats (MP3, OGG, FLAC) via FFmpeg for distribution.

Widget

A GUI element in wxPython: buttons, text fields, labels, panels, etc. All widgets inherit from `wx.Window`. Widgets are added to sizers for layout management.

Wildcard

In file dialogs, a pattern for filtering files by extension: `"PDF files (*.pdf)|*.pdf|All files (*.*)|*.*"`. In Python, `*` in function parameters captures positional args as a tuple.

wxPython

A Python wrapper around the wxWidgets C++ GUI library. Provides native-look widgets on Windows, macOS, and Linux. AIVoiceStudio's GUI framework, chosen for its cross-platform capabilities and

professional appearance.

`wx.CallAfter()`

A thread-safe method to schedule a function call on the main GUI thread. Essential for updating the UI from background threads in wxPython. Events are queued and processed in the next event loop iteration.

`wx.Panel`

A generic container widget that can hold other widgets. Panels provide a background for drawing and serve as parents for child controls. Used extensively in AIVoiceStudio's dialog layouts.

X

XML (Extensible Markup Language)

A markup language for structured data. Used in DAISY documents, DOCX files (OOXML), and many configuration formats. Python's `xml.etree.ElementTree` provides parsing.

XOR (Exclusive OR)

A bitwise operation returning true when inputs differ. In Python: `^` operator. Used for simple encryption, flag toggling, and hash computations.

Y

Yield

The keyword that makes a function a generator. `yield value` pauses the function and produces a value. The function resumes from where it left off when the next value is requested.

YAML (YAML Ain't Markup Language)

A human-readable data serialization format. Alternative to JSON with support for comments and complex types. Not used in AIVoiceStudio (which prefers JSON), but common in Python projects.

Z

Zip File

A compressed archive format. Python's `zipfile` module provides read/write access. AIVoiceStudio can import/export project data as zip archives.

Zero-Copy

Operations that don't create new data copies, improving performance. Python's `memoryview` and buffer protocol enable zero-copy slicing. NumPy array views are zero-copy.

Zone

In audio processing, a zone defines a region of audio playback, editing, or effects. In AIVoiceStudio, synthesis zones map text segments to audio output regions.

Glossary additions — Part X (the current system)

Terms introduced by the Voice Lab subsystem, the per-engine environments, the compute choice and the licensing model. Each one links to the chapter that explains it.

Bark

Suno AI's transformer audio model, used in the Voice Lab as a voice-cloning engine. MIT licensed, over one hundred speaker presets across twelve languages. Installed as the `transformers` package plus PyTorch. See [Chapter 76](#).

Compute choice

The per-category CPU / GPU / Auto combo box shown next to every Preview button, remembered per category in Settings under `preview_compute.<category>`. See [Chapter 78](#).

CPU thread band (80–95 %)

The share of logical CPUs a CPU run may use, computed by `compute.cpu_threads()` and applied in the worker with `torch.set_num_threads`. The top of the band is used; machines too small for a non-empty band leave one core free.

CUDA wheel index

`https://download.pytorch.org/whl/cu128` — the source of the PyTorch CUDA build. Used because the plain PyPI `torch` wheel is CPU-only on Windows. The `cu128` tag must match the Python version in use.

dist-info folder

A `<name>-<version>.dist-info` directory pip writes for every installed distribution. Listing these folders is how the application answers “is this package installed?” without starting an interpreter.

Engine environment (`tts_envs`)

One Python virtualenv per TTS engine under `%APPDATA%\AIVoiceStudio\tts_envs\<engine>`, so engines with conflicting dependencies never share site-packages. See [Chapter 77](#).

F5-TTS

A flow-matching transformer TTS with zero-shot voice cloning. MIT licensed *code*, but its released **weights are CC-BY-NC-4.0** (non-commercial). See [Chapter 82](#).

GPU (CUDA) path

A run on any CUDA-capable NVIDIA card — no model list and no VRAM minimum. The worker maximises the card with cuDNN autotuning, TF32 and `"high"` float32 matmul precision.

JSON-lines worker protocol

The Voice Lab IPC format: one JSON request per line on stdin, one JSON response per line on stdout. Keeps the model loaded between segments.

libsndfile

The native library bundled inside the `soundfile` wheel. It decodes WAV, FLAC, OGG and MP3 with nothing installed on the machine — the foundation of the FFmpeg-free audio path.

Pocket TTS

Kyutai's 100M-parameter, CPU-first voice-cloning TTS (`pocket-tts`), 26 built-in voices in six languages. Apache-2.0 code, per-voice licences.

Preview compute

See *Compute choice*. The value stored is one of `cpu` , `cuda` or `auto` .

soundfile audio shim

The runtime replacement of `torchaudio.load / save` with `soundfile`, installed only after a real decode proves TorchCodec cannot load (a machine without FFmpeg). Opt out with

`AIVS_VOICELAB_NO_AUDIO_SHIM=0` . See [Chapter 79](#).

Start Selected Recording

The Edit-menu command that opens the recording picker: choose a mode (recorded audio file, or any file break), a target and a scope (this file only, or this file and everything after it). See [Chapter 80](#).

Terms version (`TERMS_VERSION`)

The value compared against the stored acceptance. Bumping it in `constants.py` is what makes the first-launch dialog appear again after an update. See [Chapter 81](#).

TorchCodec

The decoding library torchaudio uses from version 2.9 onward. On Windows it requires FFmpeg's shared DLLs, and its `backend` argument no longer selects a decoder.

TF32 (TensorFloat-32)

A tensor-core fast path for float32 maths, enabled for GPU inference. Practically indistinguishable in TTS output, markedly faster.

Voice Lab

The subsystem that installs and drives the pip-based voice-clone engines (Pocket TTS, Bark, F5-TTS) inside their own Python environments, through a worker subprocess. See [Chapter 76](#).

Voice registry

`voicelab/engines.py` : the data description of each engine — packages, licence, homepage, device support and built-in voices — read by the GUI, the installer and the worker alike.